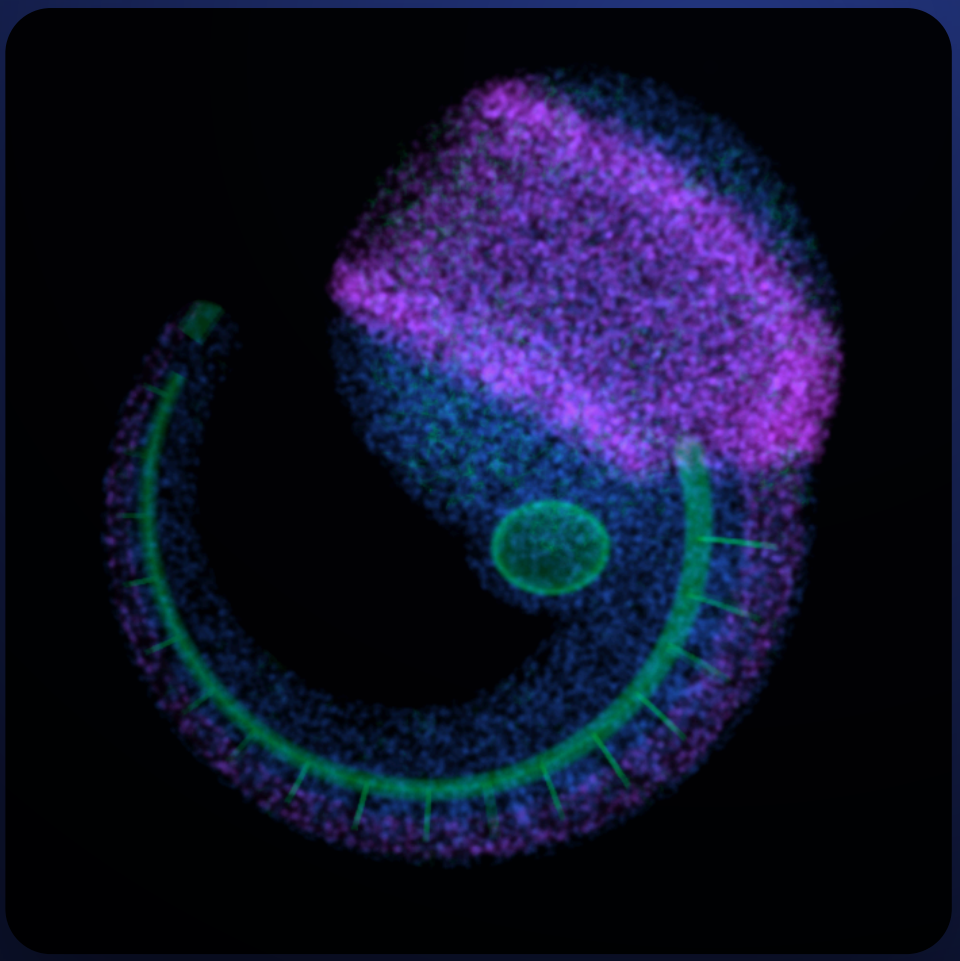


IRIBHM · ULB — LUMEN3D

Lumen3D, de A à Z

Documentation complète de la plateforme

Comment un fichier Imaris de plusieurs gigaoctets devient un embryon que l'on fait tourner dans un navigateur — et tout ce qui entoure ce voyage : les langages et les pages, le nettoyage de l'image, la compression, les briques, le rendu 3D, le streaming, les histogrammes, chaque outil et son fonctionnement interne, les plugins, la personnalisation et les langues, les formats de données, l'import, l'hébergement, les mises à jour, la sécurité. 21 chapitres écrits pour des biologistes, illustrés à chaque page.

Plateforme Web 1.65.0 · Pipeline 0.22.0 · Format de données 5

Octobre 2026

Sommaire

Comment lire ce document

- Quatre façons de le lire
- Les encadrés
- Les conventions

1. La plateforme en un coup d'œil

- 1.1 Qu'est-ce que Lumen3D, et pour qui ?
- 1.2 Le problème : un fichier trop gros pour un navigateur
- 1.3 Les trois types de jeux de données
- 1.4 Le voyage d'une acquisition
- 1.5 Chiffres clés
- 1.6 Carte de la documentation

2. Les technologies : qui fait quoi

- 2.1 Une page web, c'est un corps humain
- 2.2 Le client : ce qui tourne dans votre navigateur
- 2.3 GLSL : le programme minuscule exécuté des millions de fois
- 2.4 Le serveur : un guichet, pas un cerveau
- 2.5 Python : l'établi du laboratoire
- 2.6 Les formats de fichiers : des contenants
- 2.7 Pourquoi tout est « auto-hébergé »
- 2.8 Ce qu'il faut côté navigateur
- 2.9 Le dossier du projet

3. Le tour des pages

- 3.1 La carte des pages
- 3.2 L'Accueil
- 3.3 L'Explorateur
- 3.4 Le viewer 3D
- 3.5 Le viewer Live : la ligne de temps
- 3.6 La page 2D
- 3.7 Comparer
- 3.8 À propos et Mentions légales
- 3.9 L'administration : un aperçu
- 3.10 Les adresses à connaître

4. Du microscope au fichier Imaris

- 4.1 Du pixel au voxel
- 4.2 Une pile de coupes, plusieurs canaux
- 4.3 Combien de niveaux de gris ? 8 ou 16 bits
- 4.4 Un .ims est un « disque dur dans un fichier »
- 4.5 La pyramide d'Imaris : ignorée
- 4.6 L'étape 1 : lire les métadonnées
- 4.7 La calibration : la taille du voxel
- 4.8 L'horloge d'une séquence temporelle

5. Nettoyer l'image : retirer le bruit de fond

- 5.1 Pourquoi nettoyer ?
- 5.2 L'idée en une page
- 5.3 Un exemple complet, voxel par voxel
- 5.4 Étape A : le plancher (`bg_floor`)
- 5.5 Étape B : le plafond (`sig_max`)
- 5.6 Étape C : le masque, ou « où est l'embryon ? »
- 5.7 Étape D : la médiane, hors du masque seulement
- 5.8 Étape E : la fenêtre, de 16 à 8 bits
- 5.9 Sur de vraies images
- 5.10 Taille minimale d'un objet
- 5.11 Avant et après
- 5.12 Ce qui a été essayé, puis abandonné
- 5.13 Séries temporelles : une seule fenêtre
- 5.14 Gros volumes : par tuiles, sans changer le résultat
- 5.15 Ce que cela change pour vous

6. Réduire la taille sans rien perdre d'important

- 6.1 Le budget de taille en chiffres
- 6.2 De 16 bits à 8 bits : diviser par deux
- 6.3 La pyramide de résolutions
- 6.4 Compresser sans rien perdre
- 6.5 Ne pas stocker le vide
- 6.6 Taille finale : du .ims au dossier publié

7. Le découpage en briques

- 7.1 Pourquoi découper un volume ?
- 7.2 Une brique : 64 voxels utiles et une bordure
- 7.3 D'un cube à une image : la mosaïque
- 7.4 La grille de briques de chaque niveau
- 7.5 Les paquets : des boîtes de 64 briques
- 7.6 L'index et le manifeste
- 7.7 L'arbre d'un jeu de données publié
- 7.8 `planes/` : une coupe XY sans lire 64 plans
- 7.9 `mips/` : le maximum d'une couche de 64 plans
- 7.10 Les formats de données 1 à 5
- 7.11 Le format 5 : une matrice résolution × compression

8. La fin du pipeline : métadonnées, suivi cellulaire, publication

8.1 La vignette
8.2 Les histogrammes
8.3 `metadata.json` : la carte d'identité du jeu de données
8.4 Le suivi cellulaire (séries temporelles)
8.5 Publier « tout ou rien »
8.6 Ce que le pipeline conserve quand on retraite un jeu de données
8.7 Le dossier `download/`
8.8 Les photographies 2D, en une demi-page
8.9 Lancer tout cela : `RUN.bat`

9. Le viewer 3D : Three.js et le lancer de rayons

9.1 Deux cerveaux : le processeur et la carte graphique
9.2 WebGL2 et Three.js : qui fait quoi
9.3 Le lancer de rayons, pas à pas
9.4 Le grain aléatoire contre les bandes
9.5 Sauter le vide
9.6 Les trois modes de rendu
9.7 Lisser entre les voxels
9.8 Fluide quand vous bougez, net quand vous vous arrêtez
9.9 Le garde-fou contre les plantages de la carte graphique
9.10 Les vraies proportions : calibration et étirement en Z

10. Afficher des gigaoctets : le streaming

10.1 La séquence de chargement
10.2 Le menu de qualité : 512, 1024, Natif
10.3 L'atlas GPU et la table des pages
10.4 Le budget de mémoire vidéo
10.5 Télécharger et décoder sans bloquer la page
10.6 Les paquets : un téléchargement ordonné
10.7 Les séries temporelles
10.8 Le mode « Détail au zoom »
10.9 Quand la carte graphique plante

11. Couleurs, contrastes et histogrammes

11.1 La chaîne de traitement d'un canal
11.2 L'étape cachée : le plancher du fond
11.3 La fenêtre min / max et le gamma
11.4 Le panneau d'un canal
11.5 Les histogrammes
11.6 Les boutons de réglage rapide
11.7 Couleurs, opacité, exposition, solo
11.8 Le flou gaussien
11.9 Garde-fou : ce que les niveaux de gris veulent dire

12. Les outils, un par un

12.1 La barre d'outils d'un coup d'œil
12.2 Mesurer une distance
12.3 La coupe oblique
12.4 L'explorateur Z-Stack
12.5 Les axes d'orientation, la grille et les affichages
12.6 Capture, export, présentation, téléchargement, canaux, chunks
12.7 Le Studio : annoter et exporter une figure
12.8 La page Comparer
12.9 La frise temporelle et le suivi cellulaire
12.10 La page 2D : les photographies
12.11 Raccourcis clavier et partage d'une vue

13. Les outils sous le capot

13.1 Le suivi cellulaire dans le viewer
13.2 La stabilisation : on déplace le regard
13.3 Le Studio en profondeur
13.4 Comparer sous le capot
13.5 La page 2D en profondeur
13.6 La frise temporelle et les séries live
13.7 Ce que la plateforme retient : lien, espace de travail, navigateur
13.8 Algorithmes de mesure et de pose
13.9 L'orientation en pratique, et le sens de l'échantillon
13.10 Galerie, jeux liés, vignette

14. Le panneau d'administration en bref

14.1 À quoi sert le panneau, et comment y accéder
14.2 Les 15 onglets, en 4 groupes
14.3 Mettre un dataset en ligne
14.4 Éditer un dataset
14.5 Mises à jour des données
14.6 Plugins et catalogue signé
14.7 Mettre à jour la plateforme
14.8 Personnaliser le site (marque blanche)

15. Les plugins, la confiance et le catalogue signé

15.1 Qu'est-ce qu'un plugin ?
15.2 La carte d'identité : `plugin.json` champ par champ
15.3 Comment la plateforme trouve les plugins
15.4 Le chargement : sept barrières
15.5 Qui se charge où ? `dataTypes` et `contexts`
15.6 Le cycle de vie d'un plugin
15.7 Ce que reçoit un plugin : l'objet `ctx`
15.8 La barre d'outils est fabriquée, pas écrite
15.9 Plugins et espace de travail, exports, pages embarquées
15.10 Les 28 plugins du catalogue
15.11 Écrire un plugin
15.12 Pourquoi se méfier d'un plugin
15.13 L'empreinte : une signature du contenu exact
15.14 L'approbation de l'opérateur, et le bac à sable
15.15 La compatibilité : `platformCompat`
15.16 Mettre à jour la plateforme : l'effacement réversible
15.17 Le catalogue signé : la boutique officielle
15.18 Installer, mettre à jour, désinstaller
15.19 Publier un plugin (côté éditeur)
15.20 Les modules du noyau qui font fonctionner tout cela
15.21 Un plugin ne s'affiche pas : que vérifier ?

16. Personnaliser et traduire

16.1 Une plateforme, plusieurs maisons
16.2 `instance.json` , champ par champ
16.3 Comment une valeur arrive à l'écran
16.4 Les écrans de personnalisation
16.5 Le thème : couleurs, police, arrondis
16.6 Le constructeur de pages
16.7 Les mentions légales
16.8 D'où vient la couleur de départ d'un canal ?
16.9 Traduire : comment le site parle quatre langues
16.10 Accessibilité, thème et mobile
16.11 Les statistiques d'usage
16.12 `PerfTelemetry` : le carnet de bord du viewer
16.13 Récapitulatif

17. Les données en profondeur : fichiers, formats, migrations, import

17.1 Un jeu de données, un dossier : les trois arbres
17.2 Qui écrit quoi : trois auteurs
17.3 `metadata.json` , champ par champ
17.4 Les manifestes et les fichiers de description
17.5 Le catalogue et le vocabulaire des types
17.6 `download/` et `gallery/`
17.7 Les formats 1 à 5
17.8 Quatre migrations, un registre
17.9 Les unités de travail et le journal
17.10 Deux exécutants
17.11 `finalize` : l'échange et le changement de version
17.12 Le régulateur réseau et les hébergeurs lents
17.13 Le parcours d'un import
17.14 La liste blanche
17.15 Les paliers : pourquoi un jeu est éditable avant la fin
17.16 Les blocs et le journal d'import
17.17 Les états d'un import et la purge
17.18 Valider, publier, remplacer

18. Héberger, mettre à jour, publier une version

18.1 Trois façons de servir Lumen3D
18.2 Le trajet d'une requête
18.3 Le cache HTTP : ne rien retélécharger pour rien
18.4 Compression et plages d'octets
18.5 Les dossiers fermés et les fichiers-garde
18.6 Derrière un proxy, limites de débit et verrou de session
18.7 Exploiter un serveur au quotidien
18.8 Mettre à jour la plateforme : le principe
18.9 La signature : prouver qu'un colis est authentique
18.10 La mise à jour sur un serveur Python : huit étapes
18.11 Le basculement et le retour arrière
18.12 La mise à jour sur un hébergement PHP
18.13 Ce qu'une mise à jour ne touche jamais
18.14 Première installation : `install.php`
18.15 Publier une version (pour l'éditeur)
18.16 La suite de tests : ce qui garde la porte
18.17 Travailler sans Internet
18.18 Que faire si… ?

19. Sécurité et fiabilité

19.1 Le mot de passe administrateur
19.2 La politique de sécurité du contenu (CSP)
19.3 Les fichiers importés ne sont jamais servis
19.4 Les plugins : confiance et cage
19.5 Versions et catalogue signés
19.6 Vos données restent chez vous
19.7 Fiabilité
19.8 Que se passe-t-il si… ?
19.9 À l'écran : briques, workers et métadonnées
19.10 La carte graphique : mémoire pleine et contexte perdu
19.11 Imports, conversions et métadonnées
19.12 Serveur : mises à jour, accès et limites d'hébergement
19.13 Ce qu'il faut retenir

20. Annexes

20.A Carte des modules
20.B Mémoires du navigateur et paramètres d'adresse
20.C Ce que Lumen3D ne fait pas
20.D Chiffres clés de la plateforme
20.E Versions et historique

21. Glossaire

Comment lire ce document

Ce document explique **tout** ce que fait Lumen3D, sans supposer que vous savez programmer. Chaque page a au moins une illustration ; le texte reste court.

Quatre façons de le lire

🚀 En 20 minutes
Lisez seulement les encadrés verts **« En 30 secondes »** de chaque chapitre et regardez les figures.

🧬 « Mon dataset »
Chapitres **4 à 8** : ce qui arrive à votre fichier Imaris, étape par étape, avec de vrais chiffres.

🖥️ « L'écran »
Chapitres **9 à 12** : comment le viewer dessine le volume et comment fonctionnent les outils.

⚙️ « Les coulisses »
Chapitres **13 à 20** : le fonctionnement interne des outils, l'administration, les plugins, la personnalisation, les formats, l'hébergement, la sécurité.

Les encadrés

⏱ EN 30 SECONDES
Le résumé du chapitre ou de la section, en trois points.

💡 UNE IMAGE POUR COMPRENDRE
Une comparaison avec la vie de tous les jours.

🔧 POUR LES CURIEUX
Le détail technique (formules, noms de fichiers). Vous pouvez le sauter.

🔢 MICRO-EXEMPLE
Un calcul fait à la main avec de vrais nombres.

⚠️ ATTENTION
Une limite à connaître avant d'interpréter vos images.

Les conventions

VOUS VOYEZ…	CELA SIGNIFIE…
`Enregistrer`	un bouton ou un libellé tel qu'il apparaît à l'écran

VOUS VOYEZ…	CELA SIGNIFIE…
3	le repère numéroté n° 3 sur la capture d'écran juste au-dessus
`metadata.json`	un nom de fichier ou une valeur technique
→ **chapitre 7**	la notion est expliquée en détail ailleurs

ℹ️ **À SAVOIR**

Les images de volumes proviennent d'**embryons de démonstration synthétiques**, traités par le vrai pipeline ; ce ne sont pas des données du laboratoire. Les chiffres sont lus dans le code (plateforme 1.65.0, pipeline 0.22.0). Version courte : **« L'essentiel »** (environ 25 pages), dans l'onglet *Documentation* du panneau d'administration.

1. La plateforme en un coup d'œil

- **Lumen3D** est un site web qui permet d'explorer, dans un navigateur ordinaire, des images de microscopie confocale pesant plusieurs gigaoctets : rien à installer.
- Une pile de ~10 Go ne tient ni dans la mémoire d'un navigateur ni dans celle d'une carte graphique : toute la plateforme est conçue pour ne charger que ce que l'écran montre.
- Il existe **trois types** de jeux de données (3D, Live, 2D) et un chemin unique du microscope à l'écran, que ce chapitre vous fait parcourir.

1.1 Qu'est-ce que Lumen3D, et pour qui ?

Lumen3D (*Light-based Unified Microscopy Exploration in 3D*) est né à l'IRIBHM (ULB) pour regarder des embryons de souris. Il est ensuite devenu une plateforme « marque blanche » : nom, couleurs, pages et vocabulaire se changent depuis le panneau d'administration, sans écrire une ligne de code.

Quatre personnes l'utilisent, chacune avec une part différente de ce document :

Le biologiste
Ouvre un embryon, règle les couleurs, mesure, coupe, compare, fabrique une figure.
chapitres 3, 11, 12

Le technicien
Lance le **pipeline** Python qui transforme le fichier Imaris en dossier web.
chapitres 4 à 8

L'administrateur
Importe, publie, personnalise le site, met à jour la plateforme.
chapitres 14 à 19

Le visiteur public
Parcourt le catalogue publié et regarde, sans compte ni mot de passe.
chapitre 3

ℹ️ **À SAVOIR**

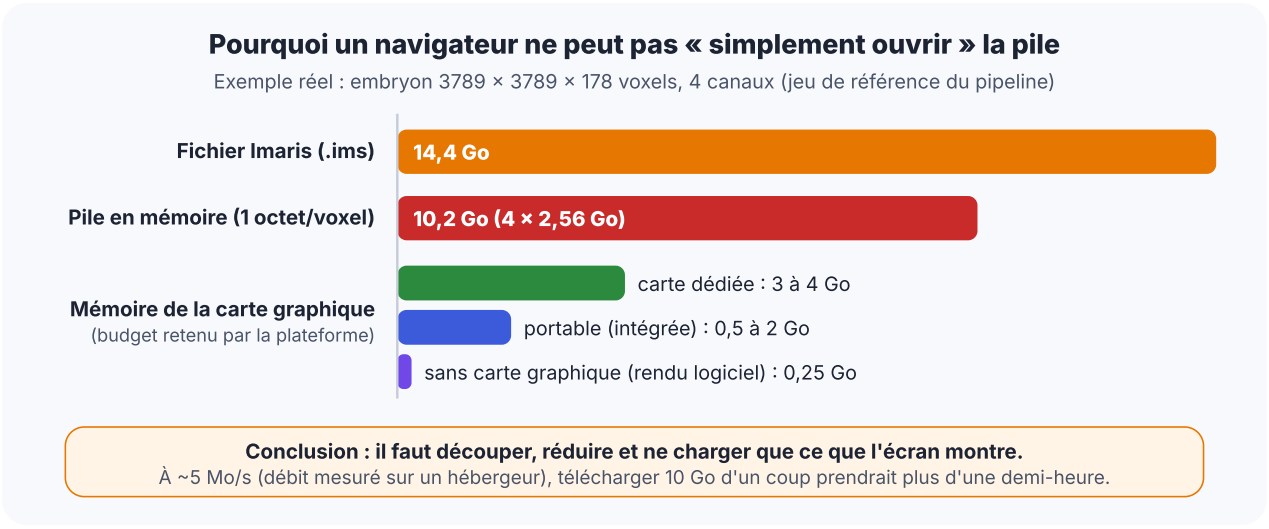
Il n'y a **aucun compte utilisateur**. Seule l'administration est protégée par un mot de passe ; tout ce qui est publié est visible par quiconque a l'adresse du site.

1.2 Le problème : un fichier trop gros pour un navigateur

Prenons le plus gros jeu de référence du pipeline : un embryon de **3789 × 3789 × 178 voxels** sur **4 canaux**. Le fichier Imaris d'origine pèse **14,4 Go**.

🔢 **MICRO-EXEMPLE**

Le calcul. 3789 × 3789 × 178 = 2 555 460 738 voxels par canal. À 1 octet par voxel : **≈ 2,56 Go par canal**, donc **≈ 10,2 Go** pour 4 canaux. La carte graphique, elle, n'offre à la plateforme que 0,25 à 4 Go selon sa classe.



▲ Même en le réduisant à 1 octet par voxel, la pile dépasse la mémoire de la plupart des cartes graphiques.

Trois obstacles se cumulent :

- **le réseau** : à environ 5 Mo/s, 10 Go demandent plus d'une demi-heure ;
- **la mémoire du navigateur** : il n'est pas fait pour garder des gigaoctets d'image ;
- **la mémoire de la carte graphique** (VRAM) : c'est elle qui dessine, et elle est petite.

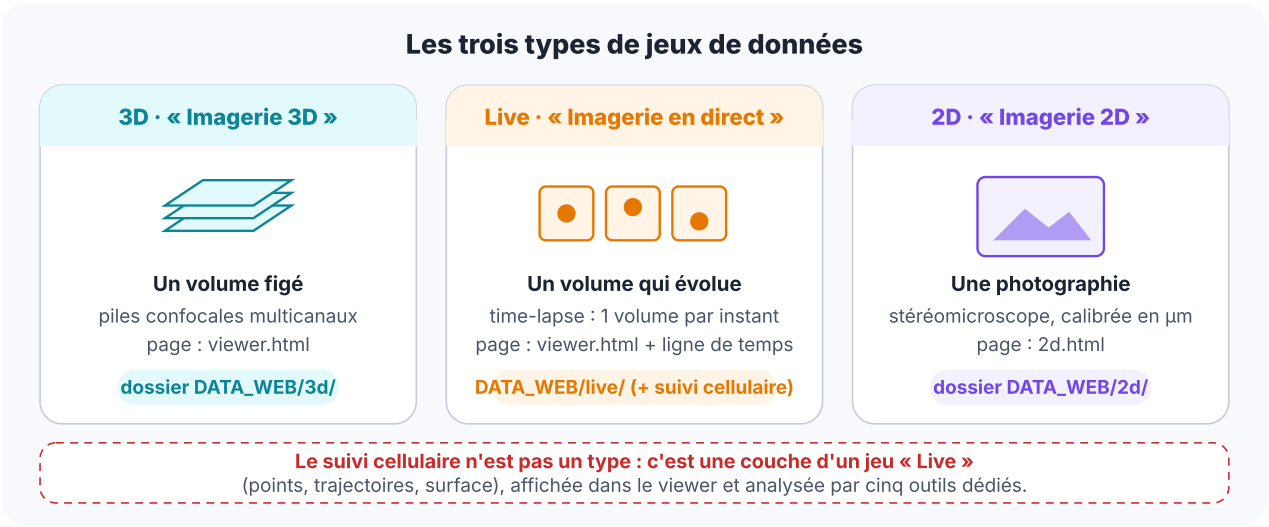
> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Une encyclopédie de 50 tomes.** Personne ne pose les 50 tomes ouverts sur son bureau pour lire un paragraphe. On prend l'index, on ouvre le bon tome à la bonne page. Lumen3D fait pareil avec l'image : un index (le *manifeste*) et des petits morceaux (les *briques*) que l'on va chercher à la demande.

La réponse tient en trois idées, détaillées aux chapitres 5 à 10 :

1. **Réduire** : retirer le bruit de fond, passer en 8 bits, fabriquer des versions plus petites de l'image (les niveaux de détail).
2. **Découper** : cuber l'image en briques de 64 voxels de côté et les ranger dans quelques gros fichiers.
3. **Streamer** : le navigateur télécharge d'abord la version grossière (image instantanée), puis affine seulement la zone regardée.

1.3 Les trois types de jeux de données

Tout jeu de données publié est d'un de ces trois types. Le même mot sert de nom de dossier, de début d'identifiant (par exemple `3d/Embryo-E95-Em2-Pecam1-Sox2`) et de filtre dans l'Explorateur.



▲ Trois types, trois pages d'affichage. Les noms entre guillemets sont ceux que le public lit par défaut.

ℹ️ À SAVOIR

Les noms affichés (« 3D », « Live », « 2D ») appartiennent à l'administrateur : il peut les renommer dans l'onglet **Types de données** sans toucher aux dossiers. Dans ce document, on utilise les noms par défaut.

TYPE	CE QUE CONTIENT LE JEU	EXEMPLE DU JEU DE DÉMONSTRATION
3D	un volume, 1 à 4 canaux	`Embryo-E95-Em2-Pecam1-Sox2` (768 × 576 × 112, 3 canaux)
Live	un volume par instant, avec ligne de temps	`Demo-Lumen3D-E85-Em1-30min-2ch-4tp` (4 instants)
2D	une photographie calibrée en µm	`DLL4xCD1-E95-x3.2-240913-1` (coloration X-gal)

❓ POURQUOI CE CHOIX ?

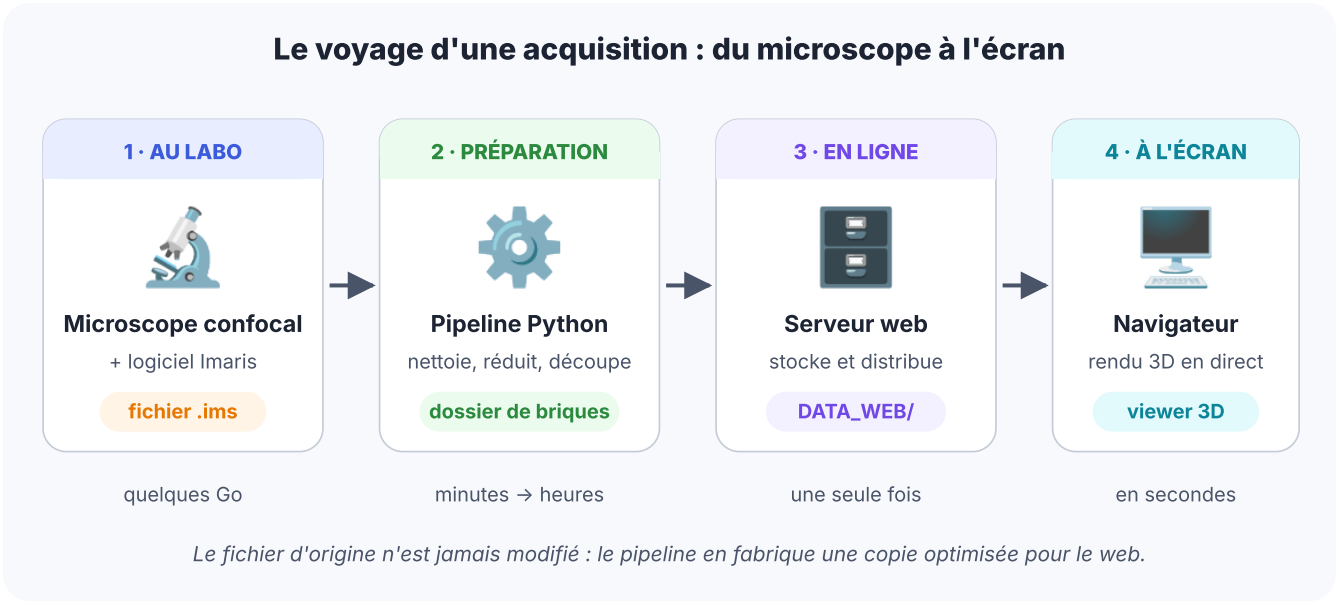
Pourquoi le suivi cellulaire n'est-il pas un type ? Parce qu'il décrit un jeu Live, il ne le remplace pas. Les positions des cellules, leurs trajectoires et la surface sont des **couches** dessinées par-dessus le volume, et cinq outils dédiés les analysent (chapitre 12). Un jeu Live sans suivi reste un jeu Live.

⚠️ ATTENTION

Les jeux de ce document sont des **jeux de démonstration synthétiques**, fabriqués pour l'occasion : ce ne sont pas des données réelles du laboratoire.

1.4 Le voyage d'une acquisition

De l'échantillon à l'écran, les données traversent quatre lieux.



▲ Du microscope à l'écran, en quatre étapes.

ÉTAPE	QUI	DURÉE TYPIQUE	CHAPITRE
1. Acquisition	microscope confocal + logiciel Imaris	selon l'échantillon	4
2. Préparation	technicien, pipeline Python sur son ordinateur	minutes à heures, une seule fois	5 à 8
3. Mise en ligne	administrateur : import par glisser-déposer ou SFTP, puis publication	selon le débit	14 et 17
4. Affichage	n'importe qui, avec un navigateur	quelques secondes pour la première image	9 à 12

📌 **À RETENIR**

Le travail lourd (nettoyage, découpage) se fait **une seule fois, hors ligne**. Ensuite, chaque visiteur ne télécharge que des morceaux : c'est ce qui rend l'exploration fluide.

1.5 Chiffres clés

3
types de données : 3D, Live, 2D

64^3
voxels par brique (66^3 avec bord, formats 4 et 5)

4
canaux affichés au maximum

4
langues : anglais, français, espagnol, néerlandais

28	≈ 126 000	512 · 1024 · natif	201
plugins au catalogue signé	lignes de code, hors tests	les trois qualités de rendu	fichiers de tests automatiques

Quelques repères de version, valables à la date de ce document :

- plateforme web **1.65.0** ; pipeline de préparation **0.22.0** ;
- format de données courant : `format 5` (les formats 1 à 4 sont convertis en place, voir chapitre 17) ;
- chaque version est vérifiée par les tests automatiques avant d'être publiée.

1.6 Carte de la documentation

Ce document compte **21 chapitres**, organisés comme le trajet de la donnée : du microscope à l'écran, puis l'administration, la fiabilité et les annexes. Retrouvez votre question ci-dessous.

Carte de la documentation : quelle question, quel chapitre ?

Découvrir chapitres 1 à 3	1 À quoi sert Lumen3D ? Quels types de données ? 2 Quelles technologies, et à quoi servent-elles ? 3 Où cliquer ? À quoi ressemble chaque page ?
Préparer Python, chapitres 4 à 8	4 Que contient un fichier Imaris ? 5 Comment le bruit de fond est-il retiré ? 6 Comment passer de 14 Go à quelques centaines de Mo ? 7 Que sont les briques, les packs, les niveaux de détail ? 8 Métadonnées, suivi cellulaire, publication sans risque
Afficher le navigateur, chapitres 9 à 13	9 Comment l'image 3D est-elle calculée ? 10 Comment afficher des gigaoctets sans attendre ? 11 Couleurs, contrastes, histogrammes : que signifient-ils ? 12 Que fait chaque outil de la barre d'outils ? 13 Comment les outils fonctionnent-ils sous le capot ?
Administrer et étendre, chapitres 14 à 18	14 Que peut faire l'administrateur ? (aperçu) 15 Plugins : installation, confiance, catalogue signé 16 Changer le nom, les couleurs, les langues, les pages 17 Fichiers, formats, conversions et import en détail 18 Héberger, mettre à jour, publier une version
Se fier et retrouver, chapitres 19 à 21	19 Mes données sont-elles protégées ? Que se passe-t-il en cas de panne ? 20 Où est quoi dans le code ? Que ne fait-il pas ? Quels chiffres ? 21 Que veut dire ce mot ? (glossaire)

▲ Quelle question, quel chapitre ?

CHAPITRE	TITRE	POUR QUI
1 – 3	La plateforme en un coup d'œil ; les technologies ; le tour des pages	tout le monde
4 – 8	De l'Imaris au dossier web : fond, réduction, briques, métadonnées et suivi	technicien

CHAPITRE	TITRE	POUR QUI
9 – 11	Le rendu 3D, le streaming, les canaux et les histogrammes	biologiste, curieux
12 – 13	Les outils un par un, puis sous le capot	biologiste
14	Le panneau d'administration en bref	administrateur
15	Les plugins, la confiance et le catalogue signé	administrateur
16	Personnaliser et traduire	administrateur
17	Les données en profondeur : fichiers, formats, migrations, import	administrateur, technicien
18	Héberger, mettre à jour, publier une version	administrateur système
19	Sécurité et fiabilité, avec le tableau « Que se passe-t-il si… ? »	tout le monde
20	Annexes : carte des modules, mémoires du navigateur, limites, chiffres, versions	référence
21	Glossaire	tout le monde

👉 **ASTUCE**

Pressé ? Lisez les encadrés **En 30 secondes** au début de chaque chapitre : ils suffisent pour savoir si la suite vous concerne.

2. Les technologies : qui fait quoi

- Lumen3D mélange cinq langages et quelques formats de fichiers ; chacun a **un seul métier**, et ce chapitre vous dit lequel.
- Tout ce que le navigateur exécute est **fourni par le site lui-même** (aucun service extérieur) : la plateforme fonctionne hors ligne et reste sous contrôle.
- Le serveur ne calcule jamais l'image 3D : il **distribue des fichiers**. Le dessin se fait chez vous, sur votre carte graphique.

2.1 Une page web, c'est un corps humain

Avant les détails, une image pour tout retenir :

HTML : le squelette
Dit ce qu'il y a sur la page : un titre, un bouton, une zone de dessin. Un fichier par page, à la racine du projet.

CSS : les vêtements
Dit à quoi ça ressemble : couleurs, tailles, thème clair ou sombre. 15 feuilles de style (plus le thème généré par l'administrateur).

JavaScript : les muscles
Fait bouger les choses : clic sur un bouton, téléchargement des briques, calcul des mesures.

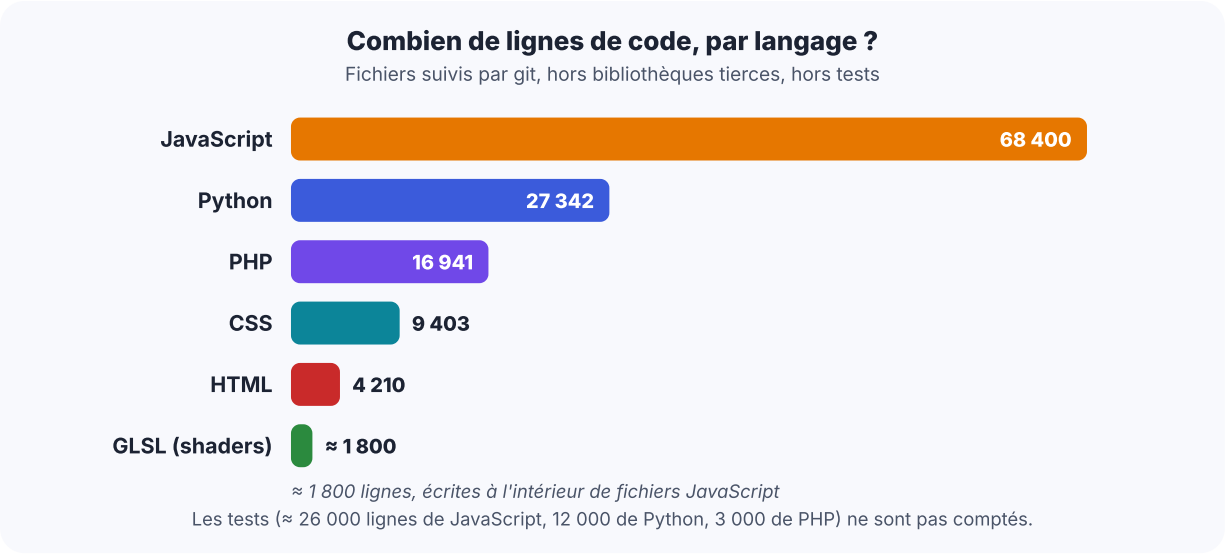
💡 UNE IMAGE POUR COMPRENDRE

Une page web = un corps. Sans squelette (HTML) rien ne tient ; sans vêtements (CSS) c'est austère ; sans muscles (JavaScript) rien ne bouge. Lumen3D ajoute un quatrième organe : un **œil** très rapide, le shader (section 2.3).

2.2 Le client : ce qui tourne dans votre navigateur

Le « client », c'est votre navigateur. Il reçoit les fichiers HTML, CSS et JavaScript, puis fait tout le travail visible.

- **JavaScript « vanilla »** : du JavaScript pur, organisé en petits modules, sans framework. Environ **68 400 lignes**.
- **Web Workers** : des « employés de l'ombre » du navigateur. Ils décodent les images WebP et font les calculs lourds pour que l'écran ne se fige jamais.
- **Bibliothèques** : Three.js (scène 3D), Lucide (icônes), Plotly (graphiques de suivi). Elles sont copiées dans le dossier `js/vendor/`.



▲ Les langages, par volume de code (la barre « shaders » est comprise dans le JavaScript).

❓ POURQUOI CE CHOIX ?

Pourquoi du JavaScript sans framework ? Pas d'étape de fabrication : on modifie un fichier, on recharge la page, c'est tout. Le code reste lisible par quelqu'un qui n'a pas appris un outil de plus, et il n'y a aucune dépendance qui puisse disparaître ou changer d'un mois à l'autre. Seul le panneau d'administration utilise les modules modernes (`import` / `export`).

2.3 GLSL : le programme minuscule exécuté des millions de fois

L'image 3D n'est pas une photo : elle est **calculée à chaque image affichée**. Pour chaque pixel de l'écran, on fait traverser le volume à un rayon imaginaire et on cumule la lumière rencontrée (chapitre 9). Cela représente des millions de petits calculs identiques.

La carte graphique est faite pour cela : elle les exécute en parallèle. Le petit programme qu'elle exécute est écrit en **GLSL**, le langage des **shaders**.

💡 UNE IMAGE POUR COMPRENDRE

Un shader, c'est une recette donnée à un million de cuisiniers. Chaque cuisinier prépare un seul pixel, tous en même temps, avec la même recette. Un processeur ordinaire serait un seul cuisinier très rapide : il perdrait la course.

- Environ **1 800 lignes** de GLSL, écrites dans des fichiers JavaScript (le rendu du volume, la coupe oblique, le recolorage des coupes).
- Le navigateur les confie à **WebGL2**, l'interface standard pour parler à la carte graphique.

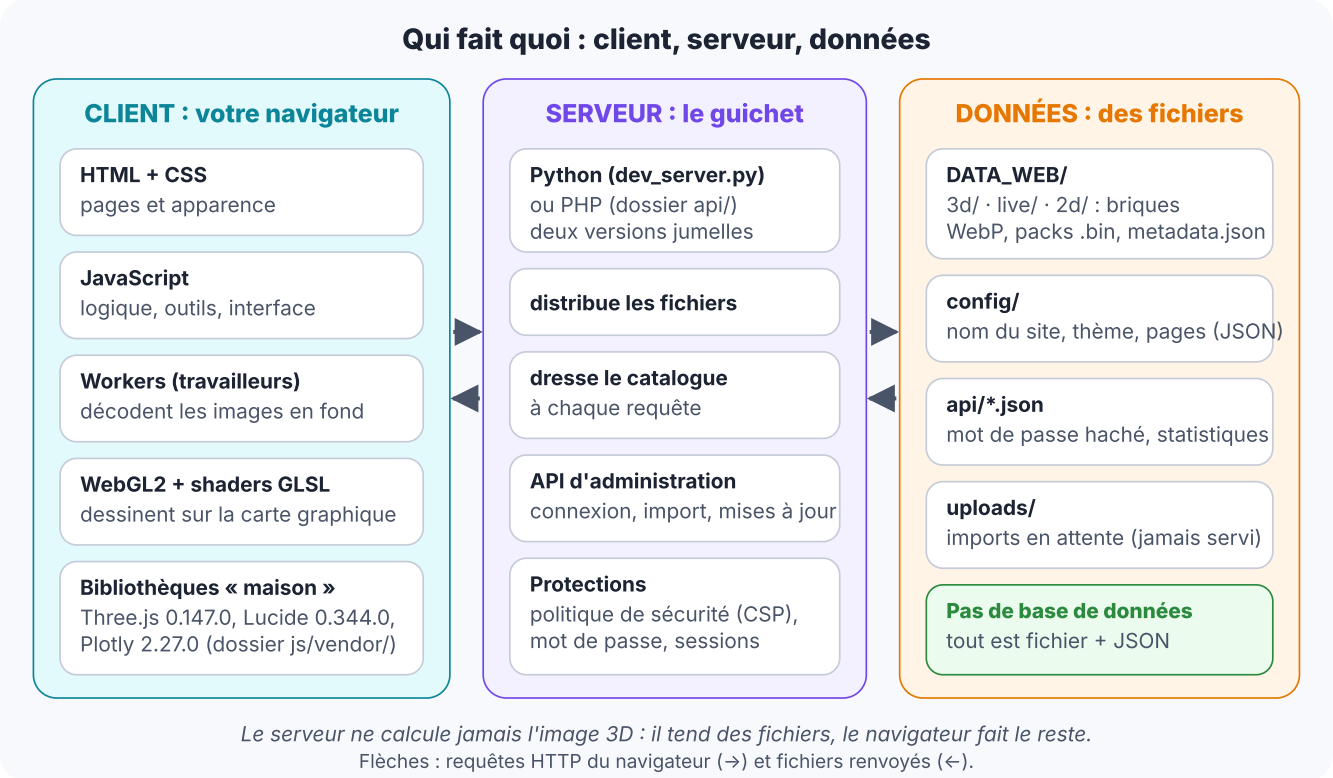
🔧 POUR LES CURIEUX

Le volume est stocké dans la carte graphique sous forme de **textures 3D**, découpées en emplacements de 64^3 voxels (66^3 en formats 4 et 5). Le shader lit ces emplacements ; ceux qui sont vides sont sautés. Les détails sont aux chapitres 9 et 10.

2.4 Le serveur : un guichet, pas un cerveau

Le serveur est un programme qui répond aux demandes du navigateur. Ici, il fait **cinq métiers** :

MÉTIER	EN CLAIR
Distribuer des fichiers	pages, scripts, briques ; avec compression et mise en cache
Dresser le catalogue	la liste des jeux est **recalculée à chaque requête** en lisant les `metadata.json`
API d'administration	connexion, édition des fiches, statistiques
Importer	recevoir un dossier par morceaux, le vérifier, le publier
Se mettre à jour	télécharger une version signée, la contrôler, basculer, annuler si besoin



Le serveur ne calcule jamais l'image 3D : il tend des fichiers, le navigateur fait le reste.
Flèches : requêtes HTTP du navigateur (→) et fichiers renvoyés (←).

▲ Les trois acteurs : le navigateur dessine, le serveur tend des fichiers, les données sont de simples fichiers.

🔢 **MICRO-EXEMPLE**

Le catalogue n'est pas un fichier. Vous copiez un dossier de jeu dans `DATA_WEB/3d/` : il apparaît tout de suite dans l'Explorateur. Rien à régénérer, parce que le serveur relit les fiches à chaque demande (il mémorise le résultat tant qu'aucun fichier n'a changé).

Deux serveurs jumeaux

Le même serveur existe **en deux langages**, avec la même interface vers le navigateur :

Python : `dev_server.py`

- recommandé, port 8080 ;
- bibliothèque standard seulement : rien à installer ;
- il se met à jour en un bloc : échange vérifié par une sonde de santé, avec **retour arrière automatique** si la nouvelle version ne démarre pas.

PHP : dossier `api/`

- pour les hébergements classiques d'université ou d'institut, qui offrent PHP mais pas Python ;
- mêmes fichiers, mêmes formats, même mot de passe : un import commencé sous l'un reprend sous l'autre ;
- PHP 8.1 ou plus ;
- il sait aussi se mettre à jour (en une seule requête, avec vérification SHA-256 et signature, sans redémarrage), mais **sans retour arrière automatique** : une mise à jour ratée se relance simplement (chapitre 18).

> 🔧 **POUR LES CURIEUX**
>
> Un petit serveur `fast_server.py` ne fait que distribuer des fichiers (sans administration) : il sert aux mesures de performance. Les deux serveurs complets sont maintenus identiques par des tests automatiques qui comparent leurs résultats.

2.5 Python : l'établi du laboratoire

Le **pipeline de préparation** est écrit en Python, parce que c'est la boîte à outils de la science : lire un fichier HDF5 (`h5py`), calculer sur de gros tableaux (`numpy`), filtrer (`scipy`), écrire des images (`Pillow`).

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **L'établi.** Le pipeline est l'atelier où l'on prépare la pièce, hors de la vitrine. Le visiteur ne le voit jamais : il ne voit que la pièce finie, sur l'étagère (`DATA_WEB/`).

- Il tourne sur l'ordinateur du technicien, **une seule fois par jeu de données**.
- Dépendances figées : `h5py` 3.16.0, `numpy` 2.5.1, `scipy` 1.16.0, `Pillow` 11.1.0, `tqdm` 4.67.1.
- Python 3.10 ou plus. Détail des cinq étapes : chapitres 4 à 8.

2.6 Les formats de fichiers : des contenants

Un format est un **contenant** ; ce qui compte, c'est ce qu'on met dedans.

FORMAT	IMAGE MENTALE	RÔLE DANS LUMEN3D
JSON	des fiches cartonnées étiquetées	`metadata.json` (fiche d'un jeu), `manifest.json` (l'index des briques), configuration, traductions
WebP sans perte	une boîte hermétique pour une image	chaque brique ; les pixels sont restitués exactement

FORMAT	IMAGE MENTALE	RÔLE DANS LUMEN3D
PNG	idem, format plus ancien	les plans entiers du format 2 (coupes rapides)
`.bin` **(packs)**	un classeur à anneaux	des centaines de briques mises bout à bout, pour moins de fichiers
HDF5 (`.ims` **)**	un **système de fichiers dans un fichier**	le fichier Imaris d'origine : dossiers, tableaux et métadonnées dans un seul bloc
glTF / GLB	une maquette 3D en carton	`model.glb`, la surface exportée avec le suivi cellulaire
TIFF	le négatif de laboratoire	uniquement les téléchargements facultatifs et l'entrée des photographies

ℹ️ **À SAVOIR**

La photographie d'un jeu 2D est enregistrée en WebP **avec** une légère perte (qualité 90) par défaut. Les briques des volumes, elles, sont toujours sans perte.

2.7 Pourquoi tout est « auto-hébergé »

Three.js, Lucide et Plotly ne sont pas chargés depuis Internet : leurs fichiers sont **dans le dossier du site**. Voici pourquoi.

1. **Hors ligne** : sur un réseau d'institut fermé, rien ne se casse.
2. **Sécurité** : la politique de sécurité du navigateur (CSP) n'accepte que du code venant du site lui-même. Un script étranger serait **bloqué**.
3. **Intégrité** : chaque bibliothèque est vérifiée par une empreinte (SRI) ; si un octet change, elle est refusée.
4. **Confidentialité** : aucun tiers ne sait qui regarde quel jeu de données.

BIBLIOTHÈQUE	VERSION	RÔLE
Three.js	0.147.0	scène 3D, caméra, cube de départ des rayons
Lucide	0.344.0	icônes des boutons
Plotly	2.27.0	graphiques de suivi (chargée seulement à l'ouverture d'un graphique)

❓ POURQUOI CE CHOIX ?

Pourquoi une version de Three.js aussi ancienne ? La 0.147.0 est figée à dessein : un seul fichier stable, qui ne change pas sous les pieds de la plateforme. Le calcul du volume, lui, est écrit sur mesure (le « lanceur de rayons » du chapitre 9) ; Three.js ne fournit que le cadre.

ℹ️ À SAVOIR

Seule exception : les polices de caractères (Google Fonts), chargées en arrière-plan. Elles ne servent qu'à l'apparence du texte.

2.8 Ce qu'il faut côté navigateur

- un navigateur récent avec **WebGL2** et les **textures 3D** ; sans cela, la plateforme refuse d'ouvrir un volume et affiche un message plutôt que de planter ;
- une carte graphique : plus elle a de mémoire, plus la qualité atteignable est élevée ;
- le thème clair/sombre, la langue et un filtre pour daltoniens sont dans l'en-tête de chaque page.

La plateforme **mesure** la mémoire disponible et s'y adapte :

CLASSE DE CARTE GRAPHIQUE	BUDGET MÉMOIRE RETENU
sans carte (rendu logiciel)	256 Mio
intégrée (portable)	0,5 à 2 Gio
dédiée	3 à 4 Gio
inconnue	1 à 2 Gio

📌 À RETENIR

Si la qualité demandée dépasse le budget, la plateforme choisit le niveau plus petit et vous le dit : elle ne plante pas (chapitre 10).

2.9 Le dossier du projet

À quoi servent les dossiers que vous verrez si vous ouvrez le projet ? Un coup d'œil sur les deux premiers niveaux :

Le dossier du projet, vu de haut (deux niveaux)

Ce que voit le visiteur

`*.html`	une page = un fichier (accueil, viewer…)
`css/`	16 feuilles de style (l'habillage)
`js/`	tout le code du navigateur : `core/ pages/ viewers/ components/ workers/ modules/ (outils) vendor/`
`lang/`	traductions : en, fr, es, nl
`config/`	nom du site, thème, pages
`assets/ logos/`	images fixes
`DATA_WEB/`	les jeux de données publiés

Ce qui sert les fichiers

`dev_server.py`	serveur Python complet
`api/`	jumeau PHP du serveur
`install.php`	installeur en un fichier
`.htaccess, router.php`	règles Apache / PHP
`upload_staging.py`	moteur d'import
`dataset_migrations.py`	mises à jour des données
`fast_server.py`	serveur de test simple

Ce qui prépare les données

`preprocess/`	le pipeline Python (étapes 1 à 5)
`SCRIPTS/`	analyses du labo (suivi cellulaire)

Le pipeline tourne sur l'ordinateur du technicien, pas sur le serveur web.

Ce qui garantit la qualité

`tests/`	201 fichiers de tests automatiques
`changelog/`	une fiche par version (147)
`tools/`	fabrication des versions signées
`marketplace/`	catalogue de plugins signé

Créés sur un serveur en fonctionnement (jamais publics)

`uploads/`	imports en cours	`backups/`	sauvegardes avant mise à jour
`logs/`	journaux du serveur	`secrets/`	clés de signature (poste local)

Le serveur refuse de les servir : une adresse web ne peut jamais les atteindre.

▲ Le projet vu de haut : visiteur, serveur, préparation, qualité.

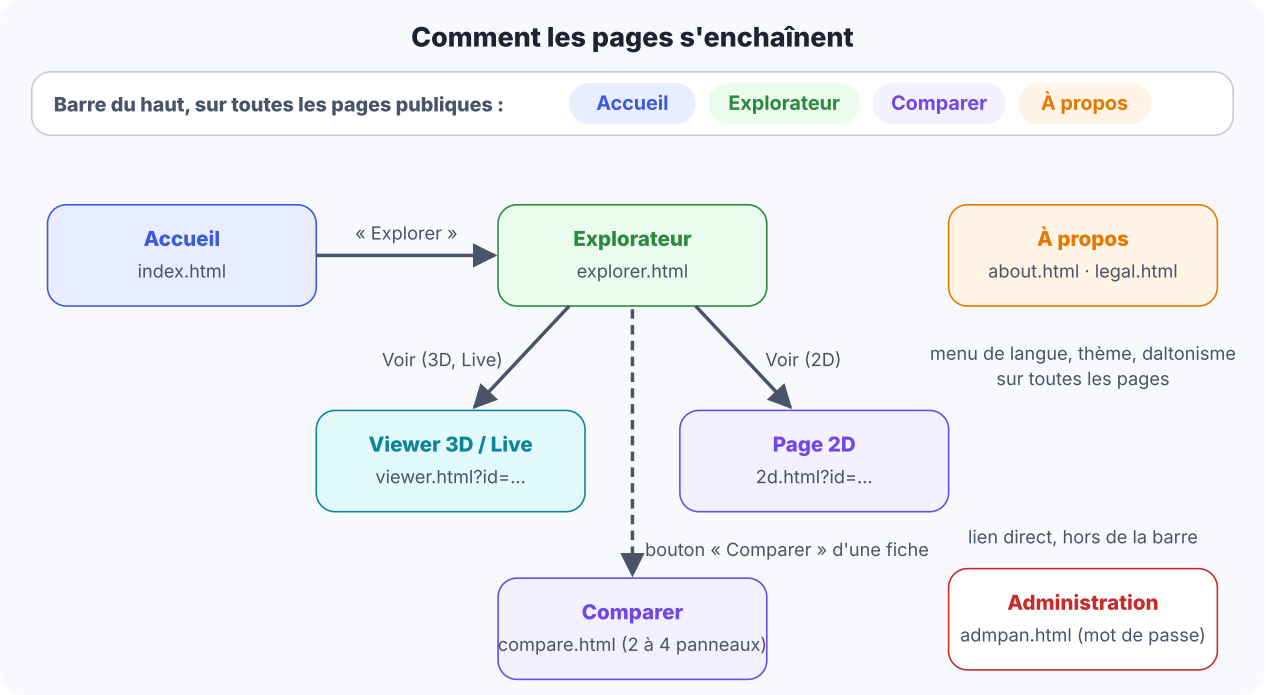
📖 **POUR ALLER PLUS LOIN**

Les dossiers `DATA_WEB/` (données publiées) et `uploads/` (imports en attente) sont décrits au chapitre 7 et au chapitre 17. Les règles qui empêchent de les atteindre depuis une adresse web sont au chapitre 19.

3. Le tour des pages

- Le site public compte quatre grandes pages : **Accueil**, **Explorateur**, **Viewer** (3D et Live) et **Comparer**, plus la page **2D** et les pages **À propos** / **Mentions légales**.
- On passe de l'une à l'autre par la barre du haut ou par les boutons des fiches ; **l'adresse (URL)** d'une page retient quel jeu de données est ouvert, et même votre vue.
- Chaque section ci-dessous montre une vraie capture d'écran (jeu de démonstration) avec des repères numérotés.

3.1 La carte des pages



▲ Les pages publiques et leurs liens. Les flèches pleines sont des clics ; les pointillés, des liens directs.

Toutes les pages publiques partagent la même barre du haut : le logo, les quatre liens, puis trois boutons à droite : **langue**, **filtre pour daltoniens** et **thème clair/sombre**.

> ℹ️ **À SAVOIR**
>
> Les captures de ce document sont en français et en thème sombre. Le nom du site (« IRIBHM — ULB »), les couleurs et même les textes de l'accueil sont ceux de l'instance photographiée : une autre institution peut les avoir changés (chapitre 16).

3.2 L'Accueil

C'est la vitrine : un message d'accueil, quelques chiffres et trois cartes, une par type de données.

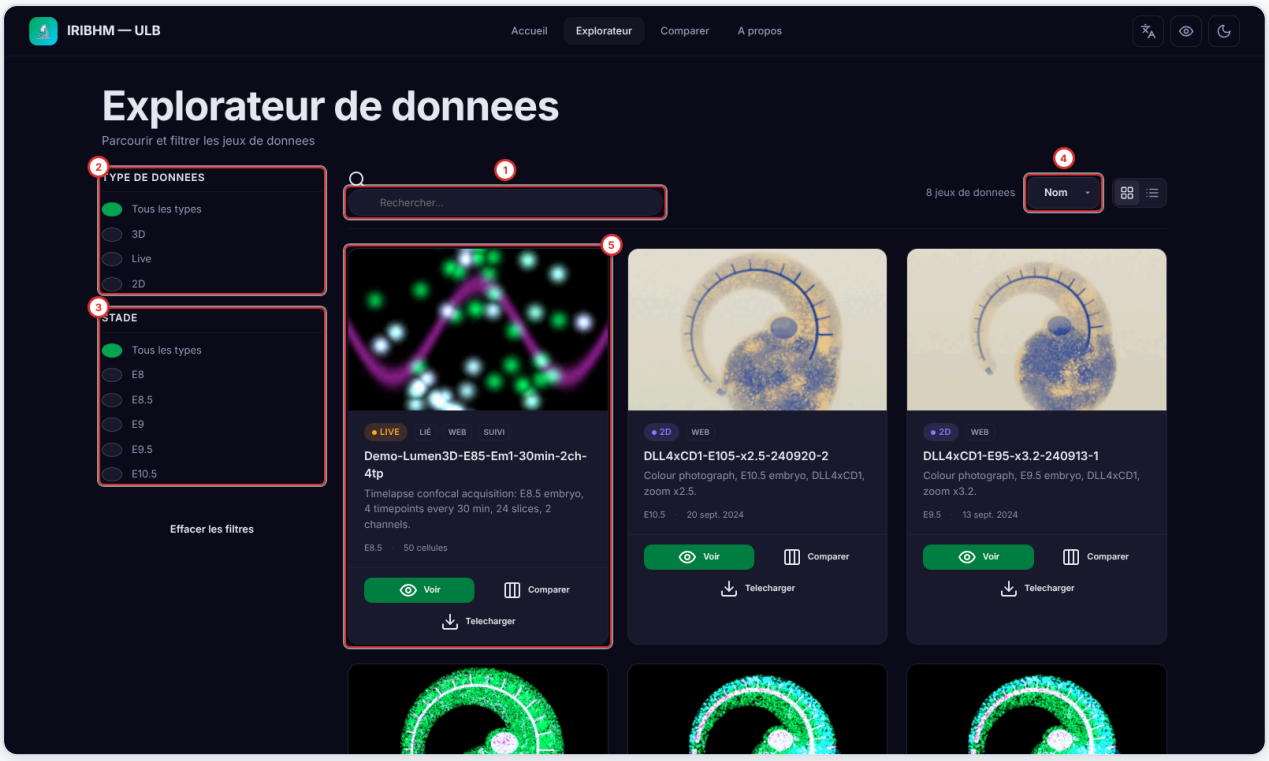
▲ Page d'accueil (index.html).

N	CE QUE C'EST
1	La barre de navigation : Accueil, Explorateur, Comparer, À propos.
2	Le titre : « Explorez les embryons ». Le mot « embryons » vient d'un réglage de l'instance.
3	**Explorer les données** ouvre l'Explorateur ; **En savoir plus** ouvre À propos.
4	Les compteurs : jeux de données, embryons, cellules suivies, régions. Ils se calculent sur le catalogue publié.
5	Les cartes de types : un clic ouvre l'Explorateur déjà filtré (`explorer.html?type=3d`).

En dessous, la section **Jeux de données en vedette** met en avant trois jeux choisis automatiquement (le suivi le plus riche, le volume le plus profond…). L'administrateur peut remplacer toute l'accueil par une page de son cru.

3.3 L'Explorateur

L'Explorateur liste les jeux publiés. Les jeux **masqués** n'y figurent pas.



▲ Explorateur de données (explorer.html).

N	CE QUE C'EST
1	La recherche : elle cherche dans le nom, la description, les noms de canaux, le stade et l'embryon.
2	Le filtre **Type de données** : 3D, Live ou 2D.
3	Le filtre **Stade** : une case par stade présent (E8, E8.5, E9…). **Effacer les filtres** remet tout à zéro.
4	Le tri (**Nom (A-Z)**, **Nom (Z-A)**, **Plus récents d'abord**, **Stade (le plus précoce)**) et le choix grille / liste ; à côté, le nombre de résultats.
5	Une fiche : miniature, type, pastilles, nom, description, stade et date.

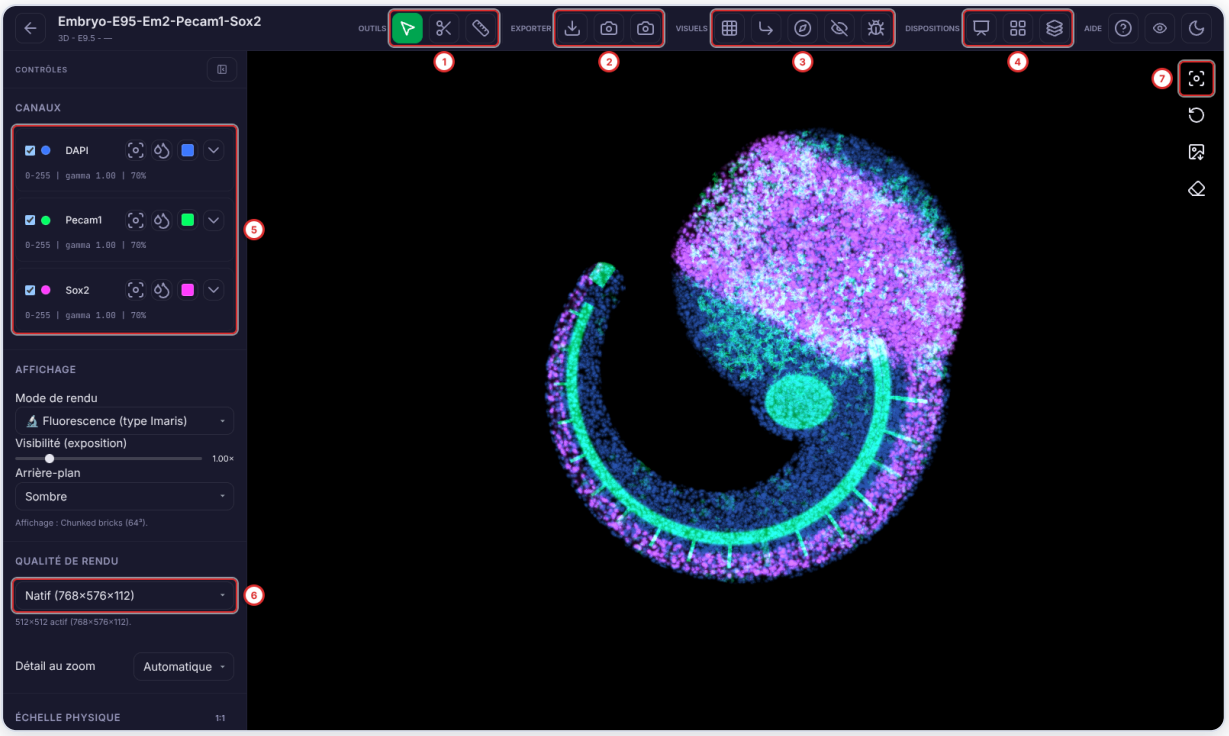
Sur chaque fiche, trois actions :

- **Voir** ouvre le viewer (ou la page 2D pour une photographie) ;
- **Comparer** ajoute le jeu à la page Comparer ;
- **Télécharger** ouvre le centre de téléchargement du jeu (originaux et exports, quand l'administrateur les a joints).

> 👉 **ASTUCE**
>
> Les pastilles de la fiche en disent long : **Web** (affichable), **Brut** (fichier d'origine à télécharger), **Suivi** (cellules suivies), **Lié** (d'autres jeux sont rattachés, par exemple une photographie du même embryon).

3.4 Le viewer 3D

C'est le cœur de la plateforme. Ouvrez un jeu 3D : le volume apparaît d'abord flou puis s'affine.



▲ Le viewer sur un jeu 3D (viewer.html?id=3d/...).

N	CE QUE C'EST
1	Groupe **Outils** : **Naviguer** (par défaut), **Couper à travers le volume**, **Mesurer la distance**.
2	Groupe **Exporter** : **Centre de téléchargement**, **Capture d'écran**.
3	Groupe **Visuels** : grille, axes, axes d'orientation, masquer le volume, débogage des briques.
4	Groupe **Dispositions** : **Mode présentation**, **Décomposer par canal**, **Explorateur Z-Stack**.
5	Panneau **Canaux** : une ligne par canal (case, couleur, histogramme réglable).
6	**Qualité de Rendu** : 512, 1024 ou natif ; sur un jeu au format 5, un tableau « résolution × compression » (chapitre 10). Le texte dessous donne la qualité réellement active.
7	Quatre boutons sur le volume : centrer l'échantillon, réinitialiser la vue, exporter la vue en PNG, réinitialiser l'espace de travail.

Bouger : glisser = tourner ; Maj + glisser (ou bouton droit) = déplacer ; molette = zoomer ; deux doigts = déplacer et pincer.

Sous les canaux, le panneau **Affichage** règle le **Mode de rendu** (par défaut « Fluorescence »), la **Visibilité (exposition)** et l'**Arrière-plan**. Plus bas, **Détail au zoom** charge des briques plus fines quand vous zoomez, et **Échelle Physique** étire l'affichage en Z.

ℹ️ **À SAVOIR**

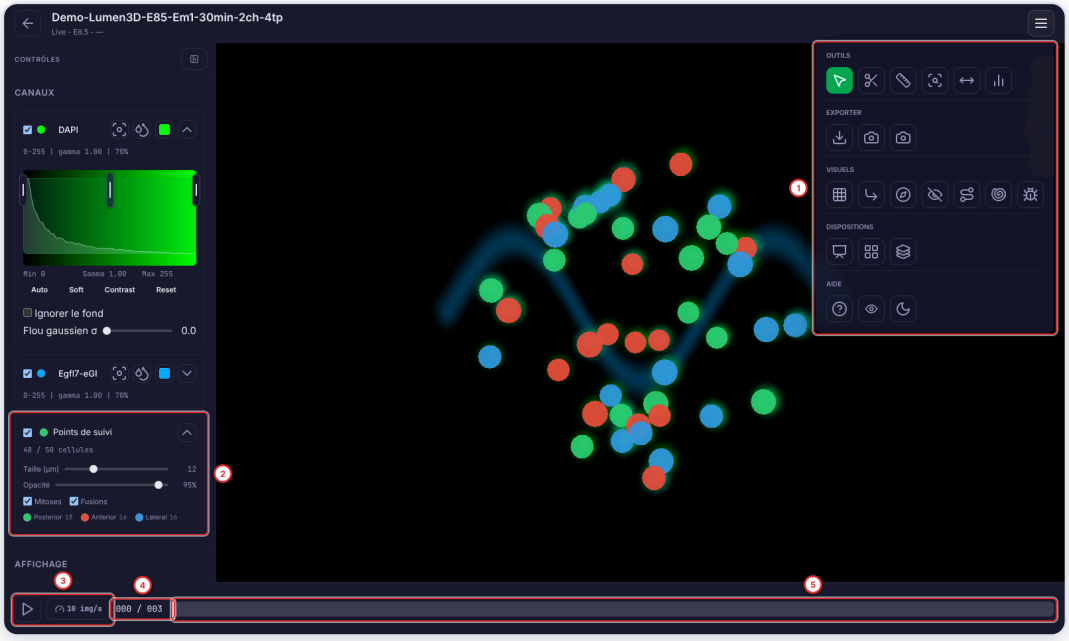
Les boutons dépendent des **plugins installés** sur votre instance. Chapitre 12 : chaque outil ; chapitre 11 : couleurs et histogrammes.

👉 **ASTUCE**

Partager votre vue. L'adresse du navigateur se termine bientôt par `#state=…` : elle contient caméra, canaux, outil actif et mesures. Copiez-la, la personne qui l'ouvre peut **Ouvrir la vue enregistrée**.

3.5 Le viewer Live : la ligne de temps

Un jeu Live s'ouvre dans la même page. Deux choses s'ajoutent : la **ligne de temps** en bas, et, s'il a été suivi, la couche **Points de suivi** dans le panneau des canaux. Sur un vrai time-lapse, la lecture **attend** que l'instant demandé soit chargé : vous voyez des images terminées, à la vitesse où elles arrivent (chapitre 10).

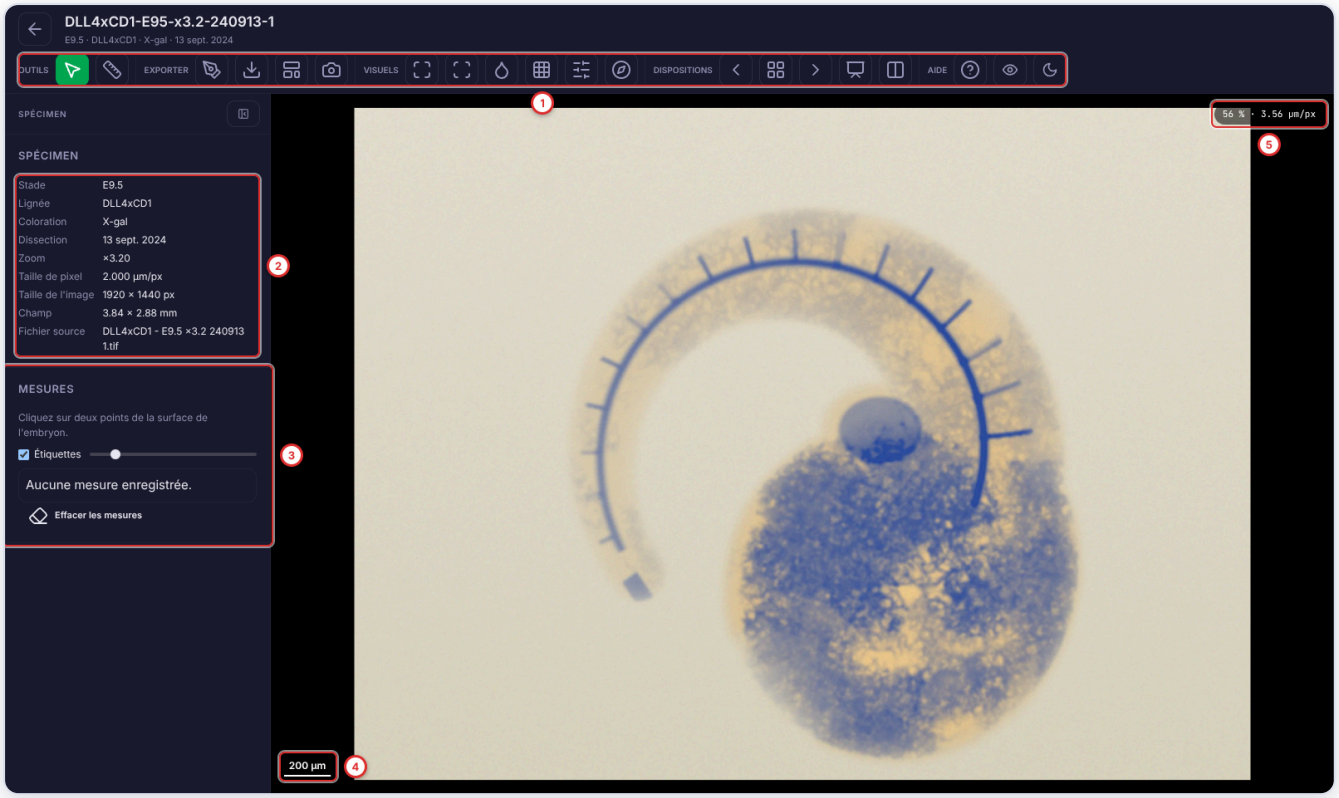


▲ Le viewer sur un jeu Live (jeu de démonstration à 4 instants, 50 cellules suivies). Le menu des outils est ouvert.

N	CE QUE C'EST
1	Le menu des outils. Quand le titre est long ou la fenêtre étroite, les groupes se replient derrière le bouton ☰. Y figurent les outils de suivi (inspecter une cellule, distance entre cellules, graphiques, trajectoires, surface).
2	La couche **Points de suivi** : taille et opacité des sphères, mitoses, fusions, couleur par région.
3	Lecture / pause et vitesse (en images par seconde ; chaque clic change de vitesse).
4	Le compteur : instant actuel / dernier instant (numérotés à partir de 0).
5	Le curseur de temps ; la bande claire indique les instants déjà en mémoire.

3.6 La page 2D

Une photographie de stéréomicroscope, calibrée en micromètres. Pas de canaux ni de 3D : on déplace, on zoome, on mesure.



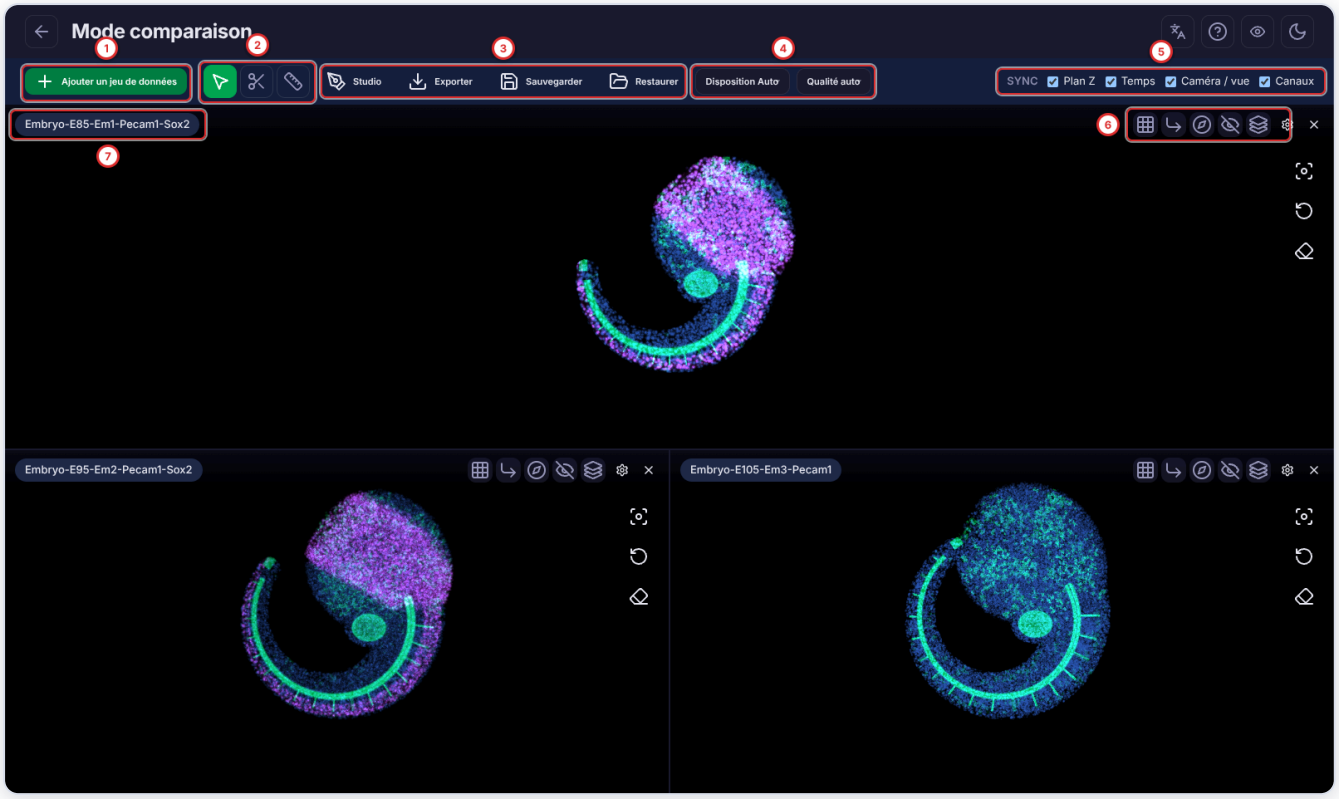
▲ La page 2D (2d.html?id=2d/...).

N	CE QUE C'EST
1	La barre d'outils : Naviguer, Mesurer ; Studio, téléchargement, planche, capture ; ajuster, résolution native 1:1, isoler le marquage ; navigation dans la collection.
2	La fiche du **Spécimen** : stade, lignée, coloration, dissection, zoom, taille de pixel, champ.
3	La section **Mesures** : cliquer deux points donne une distance en µm.
4	La barre d'échelle (ici 200 µm), qui suit le zoom.
5	Le zoom affiché, et la taille d'un pixel d'écran en µm.

- Molette = zoom, glisser = déplacer, double-clic = ajuster à l'écran.
- **Parcourir la collection** ouvre une planche-contact de toutes les photographies ; ← et → passent à la suivante.
- La page affiche d'abord un aperçu léger, puis la résolution native dès qu'elle est prête.

3.7 Comparer

Jusqu'à **quatre panneaux** côte à côte. Chaque panneau est un **vrai viewer** (ou une vraie page 2D) intégré : tout ce que vous savez faire dans le viewer, vous le faites ici, et on peut mélanger volumes et photographies.



▲ Comparer trois embryons (E8.5, E9.5, E10.5 : jeu de démonstration).

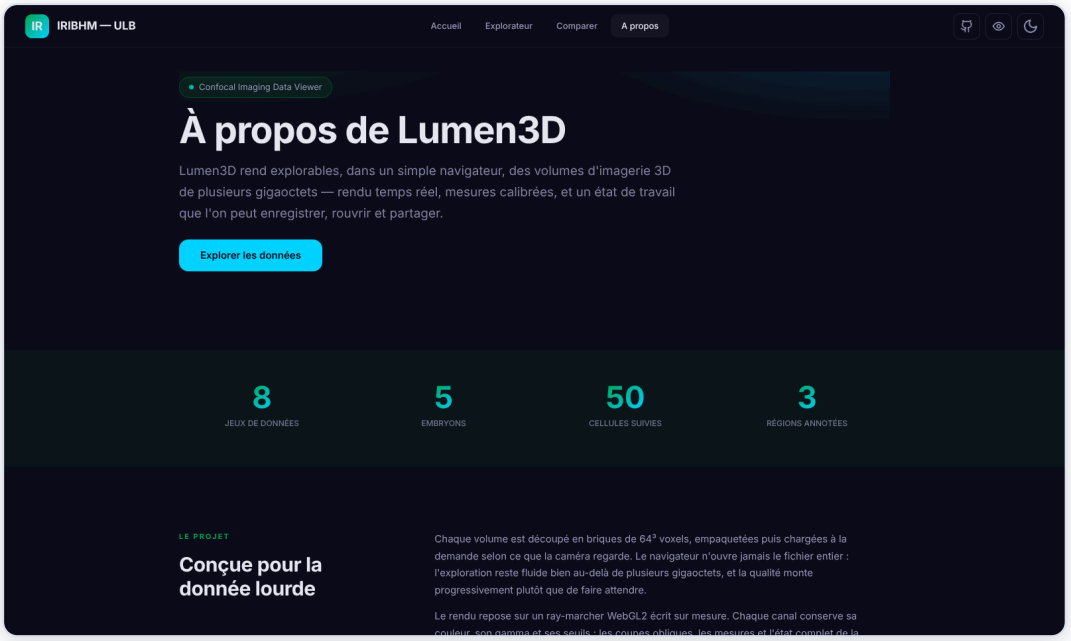
N	CE QUE C'EST
1	**Ajouter un jeu de données** : ouvre une fenêtre de choix avec recherche et filtres par type.
2	Les outils communs (Naviguer, Couper, Mesurer…) : un clic s'applique à tous les panneaux.
3	**Studio** (une figure composée de tous les panneaux), **Exporter**, **Sauvegarder** et **Restaurer** l'espace de travail.
4	**Disposition Auto** (colonnes, lignes, grille) et **Qualité auto**.
5	**SYNC** : ce qui est synchronisé entre panneaux, **Plan Z**, **Temps**, **Caméra / vue**, **Canaux**. Tout est coché par défaut.
6	Les boutons propres à un panneau (grille, axes, orientation, volume, Z-stack), son réglage et sa fermeture.
7	Le nom du jeu du panneau.

- **Caméra / vue** coché : tourner un embryon fait tourner les autres, même calibrés différemment. Pour des time-lapses de longueurs différentes, on synchronise la **fraction** du temps écoulé.
- **Qualité auto** : les panneaux s'ouvrent à 512, puis montent un par un (jusqu'à 1024 pour deux volumes, 512 au-delà) pour tenir dans une mémoire graphique commune.

- Un seul volume à plusieurs canaux ? Le bouton **Décomposer** le clone en un panneau par canal.
- L'adresse de la page (`#state=…`) retient les panneaux et leur disposition : copiez-la pour partager la comparaison.

3.8 À propos et Mentions légales

À propos présente le projet. Son contenu par défaut est une page modifiable par l'administrateur (éditeur de pages, chapitre 16). **Mentions légales** affiche le texte juridique saisi dans l'onglet **Mentions légales** ; le lien du pied de page n'apparaît que s'il est activé.



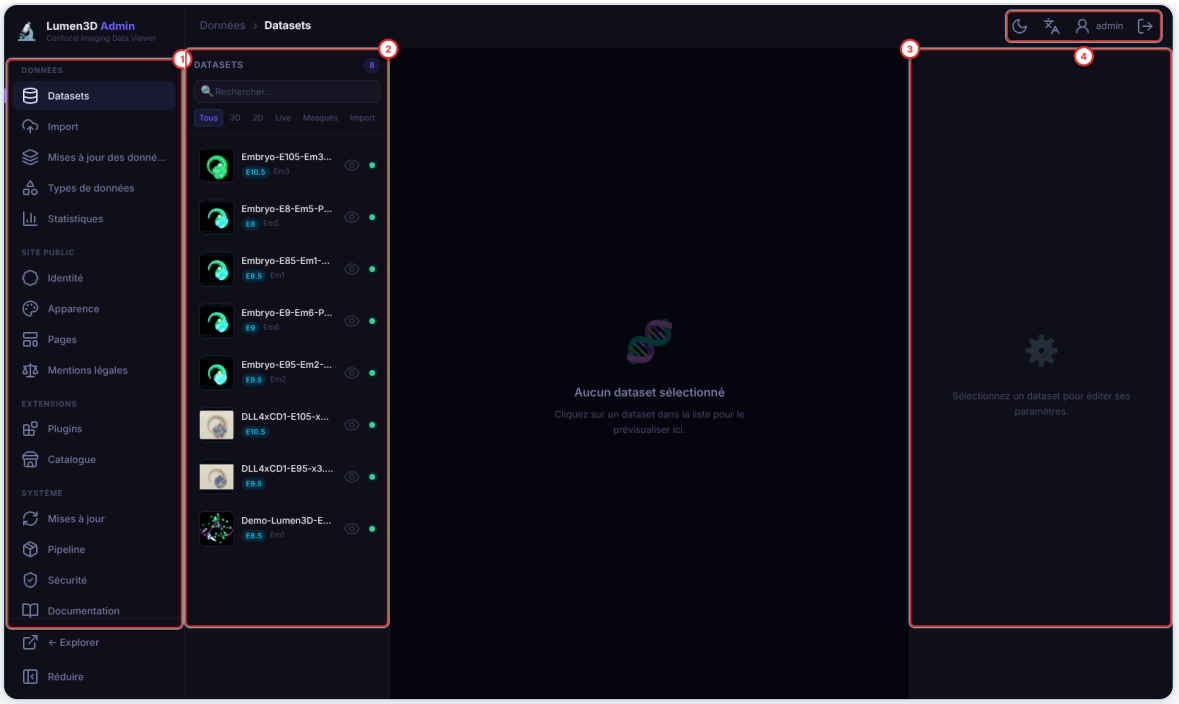
▲ La page À propos.

> ℹ️ **À SAVOIR**
>
> Sur une installation neuve, les mentions légales sont un modèle à compléter (nom de l'organisation, adresse, hébergeur) : ce sont des champs entre crochets que l'administrateur remplit.

D'autres pages peuvent exister : l'administrateur peut créer des **pages personnalisées**, ouvertes par `page.html?slug=<nom>` et ajoutées à la barre du haut.

3.9 L'administration : un aperçu

L'administration vit dans `admpan.html` et demande un mot de passe. Elle n'est pas dans la barre publique. Voici seulement son allure ; le détail est au chapitre 14.



▲ Le panneau d'administration, onglet Datasets.

N	CE QUE C'EST
1	Le menu : quatre familles, **Données**, **Site public**, **Extensions**, **Système**.
2	La liste des jeux (publiés, masqués, en cours d'import) avec recherche et filtres.
3	La zone d'aperçu du jeu choisi : le viewer y est affiché tel que le verront les visiteurs.
4	Le thème, la langue et la déconnexion.

3.10 Les adresses à connaître

Une adresse (URL) peut tout dire d'une page. Les plus utiles :

ADRESSE	EFFET
`viewer.html?id=3d/<dossier>`	ouvre un volume 3D (même forme pour `live/…`)
`2d.html?id=2d/<dossier>`	ouvre une photographie
`explorer.html?type=3d`	Explorateur déjà filtré (`3d`, `live` ou `2d`)
`compare.html?add=<id>&add=<id>`	Comparer avec ces jeux déjà posés (4 au plus)
`…#state=…`	la vue exacte (caméra, canaux, outils, mesures) : à copier pour partager
`page.html?slug=<nom>`	une page personnalisée

⚠️ **ATTENTION**

Les paramètres `hideHeader` et `panelIndex` servent à la page Comparer pour intégrer des pages sans leur en-tête ; ne les utilisez pas pour un usage normal.

👉 **ASTUCE**

Raccourcis clavier (viewer) : V ou Échap Naviguer ; C couper ; M mesurer ; I inspecter une cellule ; D distance entre cellules. Page 2D : F ajuster, B planche, ← → photographie précédente / suivante.

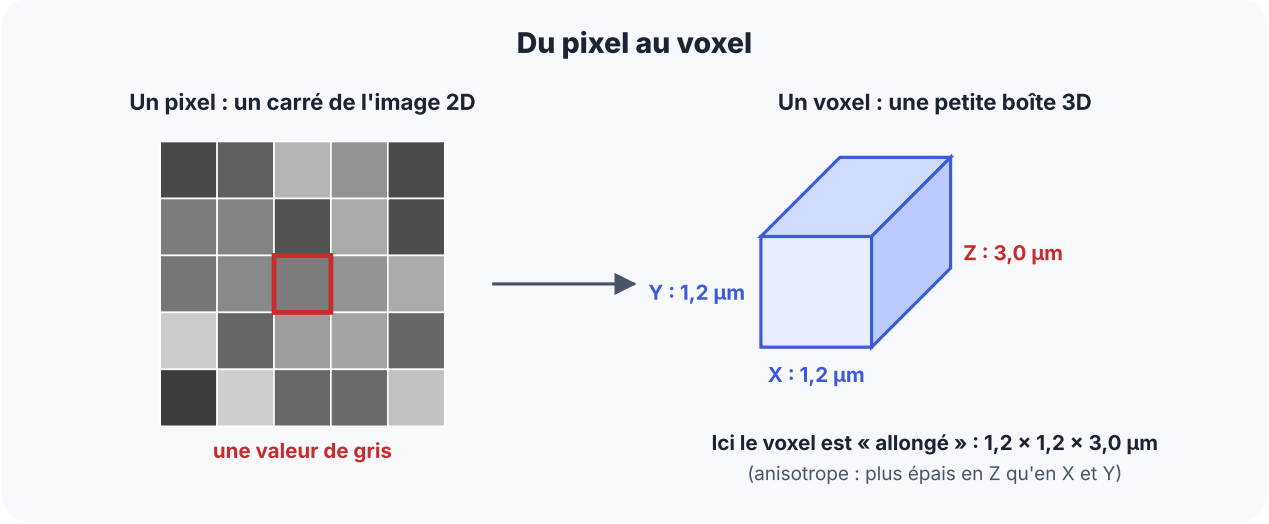
4. Du microscope au fichier Imaris

- Une image 3D est une **pile de coupes** ; chaque case 3D s'appelle un **voxel**, et sa taille réelle en micromètres compte.
- Le fichier **.ims** d'Imaris est un « système de fichiers dans un fichier » (HDF5) : le pipeline y lit seulement la taille de l'image, la calibration, les noms de canaux, l'horloge et les voxels en pleine résolution.
- **L'étape 1** du pipeline écrit un petit fichier `meta.json` : elle ne calcule rien d'autre que la taille du voxel, et n'invente jamais une calibration absente.

4.1 Du pixel au voxel

Une photo numérique est une grille de **pixels** : chaque petit carré porte une valeur de gris (ou de couleur). Un microscope confocal ne fait pas une photo, mais une série de photos prises à des profondeurs différentes.

Un point de cette série s'appelle un **voxel** : un pixel qui a aussi une épaisseur. C'est une petite boîte 3D, et c'est l'unité de base de tout ce document.



▲ Du pixel au voxel. Ici le voxel mesure 1,2 × 1,2 µm dans le plan et 3,0 µm en profondeur : c'est la calibration du jeu de démonstration (embryon synthétique).

💡 **UNE IMAGE POUR COMPRENDRE**

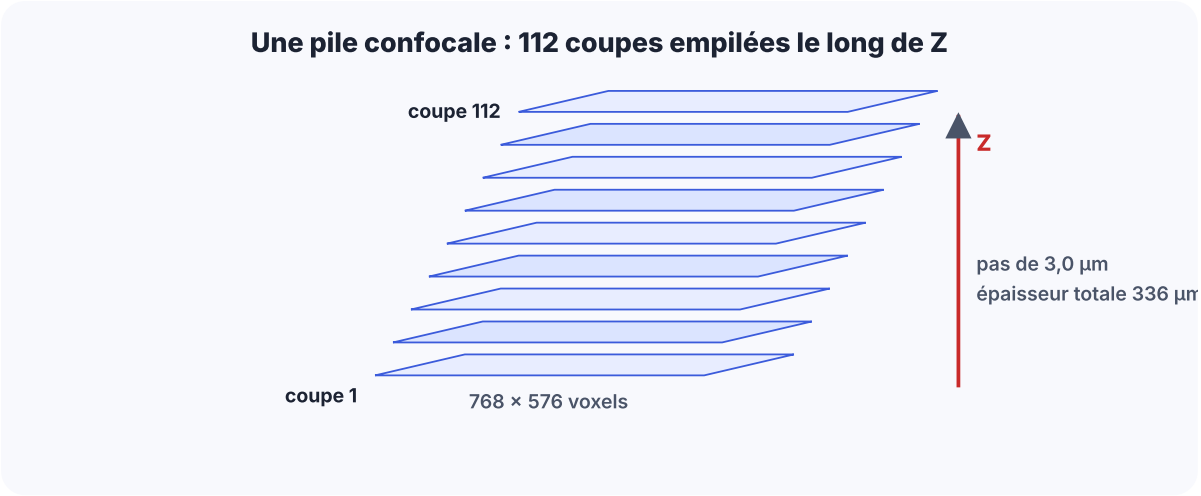
Pensez à une **boîte de Lego** : une plaque plate de plots est une image 2D (pixels). Empilez 112 plaques et chaque plot devient une brique 3D : un voxel. Si vos plaques sont plus épaisses que la largeur d'un plot, vos briques sont « allongées » en hauteur.

ℹ️ **À SAVOIR**

Un voxel **anisotrope** (plus épais en Z qu'en X et Y) est la règle en confocal : changer de profondeur coûte du temps de balayage. Le pipeline en tient compte à chaque étape (voir chapitre 6 pour la pyramide, chapitre 9 pour l'affichage).

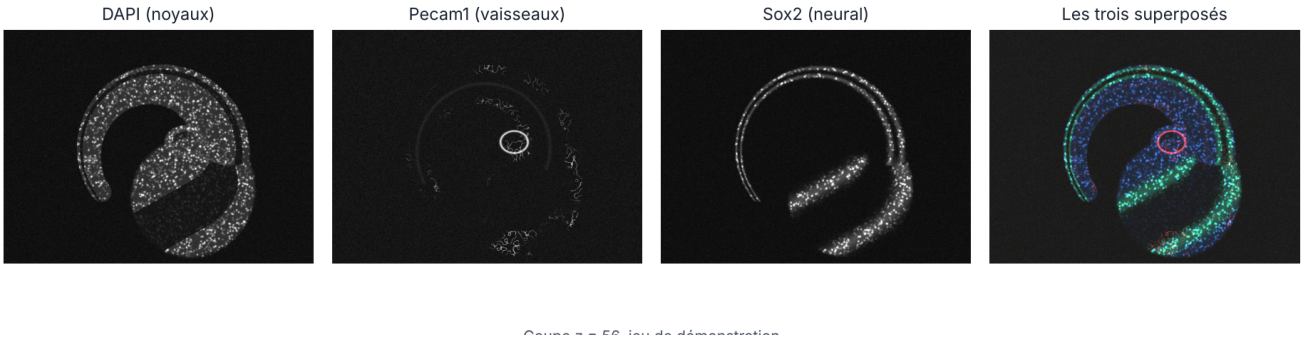
4.2 Une pile de coupes, plusieurs canaux

Le microscope balaye l'échantillon coupe par coupe, le long de l'axe **Z**. Le jeu de démonstration compte **112 coupes** de 768 × 576 voxels, espacées de 3,0 µm : l'embryon est donc épais de 112 × 3,0 = **336 µm**.



▲ Une pile confocale : 112 coupes empilées le long de Z (schéma, chiffres du jeu de démonstration).

Un **canal** est une pile complète enregistrée pour un seul fluorophore, c'est-à-dire une seule couleur d'émission. Le jeu de démonstration a **trois canaux** : DAPI (les noyaux), Pecam1 (les vaisseaux) et Sox2 (le tissu neural).



▲ La même coupe (z = 56) vue dans chacun des trois canaux, puis superposée. Les images viennent du jeu de démonstration (embryon synthétique) ; chaque canal est étiré entre 0 et son 99,9e percentile pour l'affichage.

⏱ EN 30 SECONDES

- 1 canal = 1 pile de coupes = 1 fluorophore.
- Les canaux ne sont **pas mélangés** dans le fichier : chacun a son propre tableau 3D.
- Le mélange de couleurs se fait plus tard, dans le navigateur (chapitre 11).

4.3 Combien de niveaux de gris ? 8 ou 16 bits

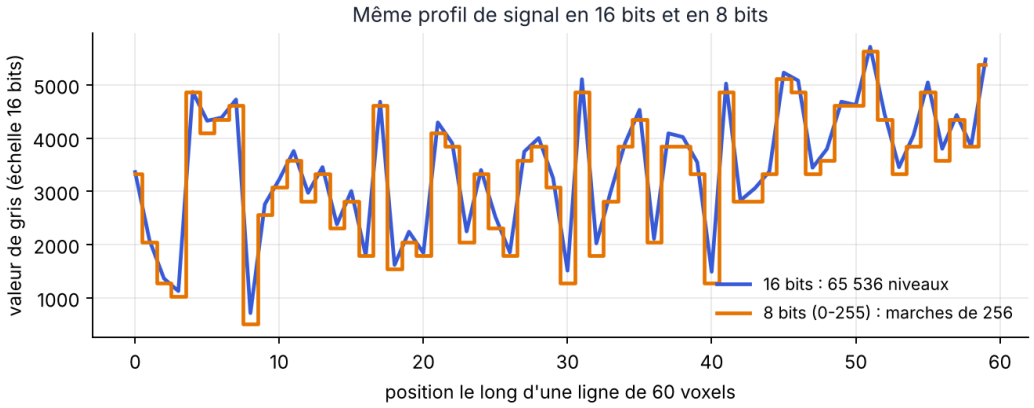
Chaque voxel porte un **entier** : l'intensité de lumière mesurée. Le nombre de bits dit combien de valeurs sont possibles.

PROFONDEUR	VALEURS POSSIBLES	PLAGE
8 bits	2^8 = **256**	0 à 255
16 bits	2^{16} = **65 536**	0 à 65 535

Les fichiers Imaris du laboratoire sont en 8 ou 16 bits selon l'acquisition ; le pipeline lit les deux. Le jeu de démonstration est en **16 bits** (type `uint16`).

🔢 MICRO-EXEMPLE

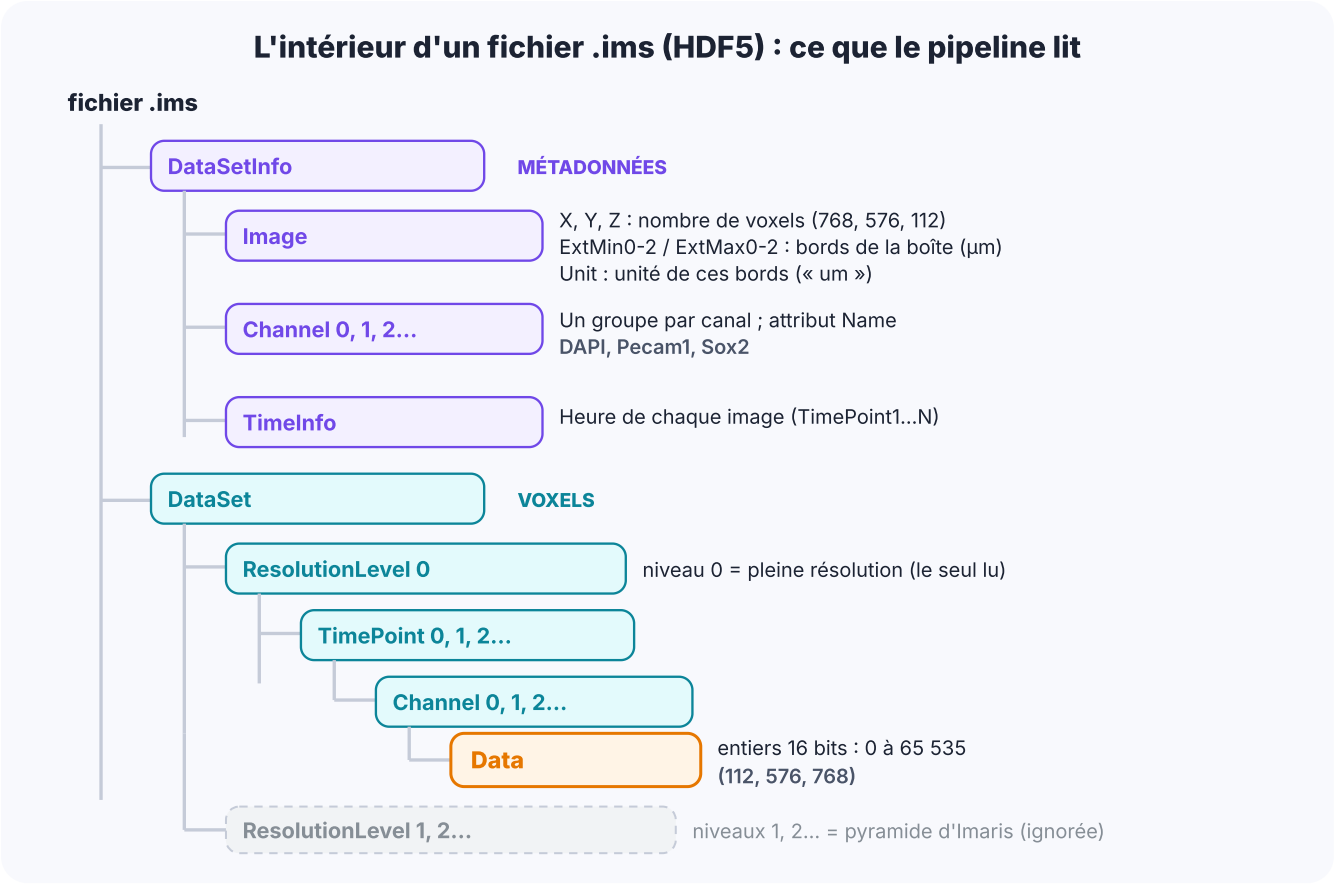
Deux taches de fluorescence valent **1 000** et **1 200** sur une échelle 16 bits : 200 niveaux d'écart, très mesurables. Si l'on divisait simplement par 256 pour passer à 8 bits, elles deviendraient **3** et **4** : un seul cran d'écart. C'est pourquoi le pipeline ne divise pas bêtement : il règle d'abord une « fenêtre » adaptée (chapitre 5).



▲ Un vrai profil de 60 voxels (canal Pecam1, jeu de démonstration) : la courbe bleue garde toutes ses nuances en 16 bits ; en orange, les mêmes valeurs rangées dans des marches de 256. Ici le signal est fort et les marches restent discrètes ; sur un signal faible, elles écrasent les détails.

4.4 Un .ims est un « disque dur dans un fichier »

Un fichier **.ims** est un conteneur au format **HDF5**. Imaginez un dossier de dossiers : on y trouve des groupes (les dossiers), des attributs (de petites étiquettes de texte) et des tableaux (les voxels).



▲ Les chemins réellement lus par le pipeline dans le .ims du jeu de démonstration. En violet : les métadonnées (étape 1). En vert : les voxels (étape 2).

🔢 **MICRO-EXEMPLE**

Dans le fichier de démonstration, `DataSetInfo/Image` contient les attributs `X = 768`, `Y = 576`, `Z = 112`, `ExtMin0 = 0.000`, `ExtMax0 = 921.600` et `Unit = um`. Tout est écrit sous forme de **texte** : le pipeline le convertit en nombres.

🔧 **POUR LES CURIEUX**

Les attributs HDF5 sont parfois stockés comme tableaux d'octets ASCII ; `attr_str` (étape 1) les recolle en texte. Les tableaux de voxels sont compressés (gzip) par blocs de 16 × 64 × 64 voxels : le pipeline lit toujours des sous-boîtes, jamais plus que la taille déclarée.

4.5 La pyramide d'Imaris : ignorée

Imaris range souvent, à côté du niveau 0 (pleine résolution), des versions réduites : `ResolutionLevel 1`, `2`… C'est sa propre pyramide, pensée pour sa propre fenêtre d'affichage. Le jeu de démonstration n'en contient pas : seul le niveau 0 existe. Un fichier réel du laboratoire en a plusieurs.

Le pipeline **n'utilise que le niveau 0**, et jamais les autres, pour le volume.

❓ POURQUOI CE CHOIX ?

La pyramide d'Imaris est calculée sur les données **brutes**, avec ses propres tailles de blocs : elle ne correspond ni au découpage en briques de Lumen3D, ni au nettoyage du chapitre 5. Mieux vaut nettoyer le niveau 0 puis fabriquer **sa propre pyramide** à partir du résultat (chapitre 6) : chaque niveau est alors cohérent avec les autres.

4.6 L'étape 1 : lire les métadonnées

L'étape 1 (`1-ims_metadata.py`) ouvre le .ims, lit quelques attributs et écrit `meta.json`. Elle ne touche aucun voxel.

CHAMP DE `META.JSON`	CE QUE C'EST	VALEUR (DÉMO)
`width` , `height` , `depth`	Nombre de voxels en X, Y, Z	768, 576, 112
`n_channels`	Nombre de canaux	3
`n_timepoints`	Nombre d'images dans le temps	1
`voxel_size`	Taille d'un voxel en µm (x, y, z)	1,2 ; 1,2 ; 3,0
`extent`	Boîte de l'image en µm (min, max)	0 → 921,6 / 691,2 / 336,0
`timestamps`	Heure de chaque image	2026-03-11 10:00:00
`time_interval_minutes`	Intervalle médian entre images	vide (une seule image)
`channel_names`	Noms des canaux	DAPI, Pecam1, Sox2

ℹ️ À SAVOIR

Le nombre d'images détermine le **type** de dataset : une seule image donne un dataset `3d` ; plusieurs donnent un `live` (séquence temporelle). C'est l'étape 1 qui tranche.

Les noms de canaux sont nettoyés : tout ce qui suit un caractère nul est supprimé, et un nom vide ou du type « ch1 » devient « Channel 1 », « Channel 2 »... Les couleurs ne viennent pas du .ims : elles sont attribuées plus tard, avec une palette fixe (chapitre 8).

4.7 La calibration : la taille du voxel

Le fichier ne donne pas la taille du voxel directement. Il donne la **boîte** (ExtMin / ExtMax) et le nombre de voxels. Le pipeline divise.

▲ Taille du voxel = (ExtMax − ExtMin) ÷ N, axe par axe. Valeurs du fichier de démonstration (embryon synthétique).

🔢 **MICRO-EXEMPLE**

Sur X : (921,6 − 0) ÷ 768 = **1,2 µm**. Sur Z : (336,0 − 0) ÷ 112 = **3,0 µm**. On divise par N (et non par N − 1), choix voulu par le code.

Si l'unité du fichier n'est pas des µm, elle est d'abord convertie :

UNITÉ DANS LE FICHIER	FACTEUR VERS MM
`um`, `µm`, `micron`…	× 1
`nm`	× 0,001
`mm`	× 1 000
`m`	× 1 000 000

⚠️ **ATTENTION**

La calibration n'est jamais inventée. Si `ExtMin/ExtMax` manquent ou si l'unité est inconnue, les trois tailles de voxel valent 0 et `extent` est vide ; la plateforme enregistre `calibrationStatus = metadata-missing` : aucune barre d'échelle ne sera inventée.

4.8 L'horloge d'une séquence temporelle

Pour un film (plusieurs images), Imaris note l'heure de chaque image : `TimePoint1`, `TimePoint2`… (la numérotation commence à 1 ici, alors que les tableaux de voxels commencent à 0).

L'étape 1 convertit ces textes en dates, puis calcule l'**intervalle médian** entre images successives. La **médiane** (valeur du milieu) résiste à un retard ponctuel : une image décalée de quelques minutes ne fausse pas l'intervalle annoncé.

▲ Exemple sur le jeu de démonstration en séquence (4 images, une toutes les 30 minutes). L'intervalle médian, 30 minutes, devient `time_interval_minutes`.

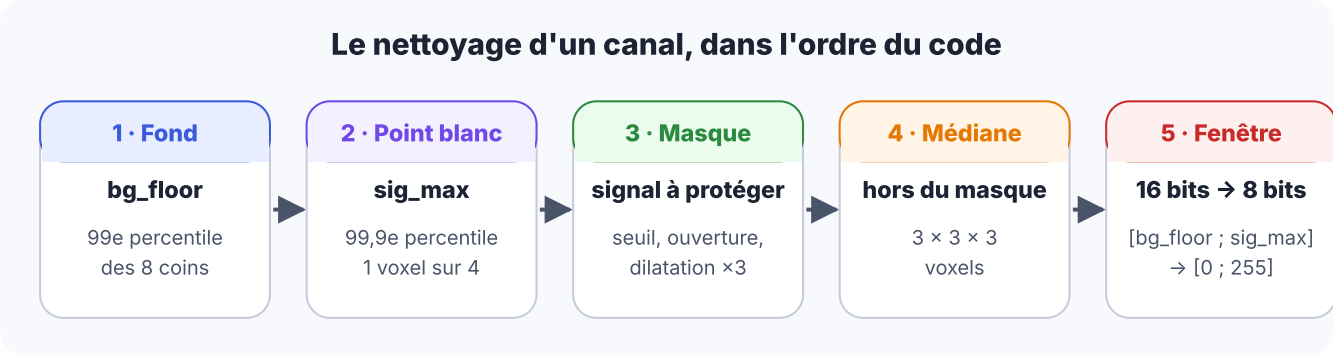
📌 **À RETENIR**

- Le voxel = une case 3D ; sa taille vient de **(ExtMax − ExtMin) ÷ N**.
- Seul le niveau 0 du .ims est lu ; la pyramide d'Imaris est ignorée.
- Les noms de canaux, l'horloge et la calibration sont lus ; **rien n'est deviné**.

5. Nettoyer l'image : retirer le bruit de fond

- Le pipeline traite **chaque canal séparément** en cinq temps : mesurer le fond, mesurer le blanc, protéger le signal, lisser le fond, puis convertir de 16 à 8 bits.
- Tout est calculé **à partir de voxels de référence** (les 8 coins, un voxel sur 4) : aucune valeur n'est choisie à la main, et rien n'est « embelli » ni inventé.
- Résultat : un fond **exactement noir (0)**, un signal intact, et des valeurs 8 bits **relatives** à chaque canal. Le .ims original reste la seule source quantitative.

Après une vue d'ensemble et un exemple complet (§ 5.2 et 5.3), ce chapitre suit **l'ordre exact du code** (`2-image_processor.py`), étape A à E. Toutes les images viennent du jeu de démonstration (embryon synthétique), canal DAPI, coupe z = 56 ; elles sont calculées avec les fonctions du pipeline lui-même.



▲ Les cinq temps du nettoyage d'un canal.

5.1 Pourquoi nettoyer ?

Une caméra de microscope n'est jamais parfaitement noire : même sans échantillon, chaque voxel affiche un petit nombre aléatoire (le **bruit de caméra**). Dans le jeu de démonstration, il oscille autour de 2 600 sur une échelle de 0 à 65 535.

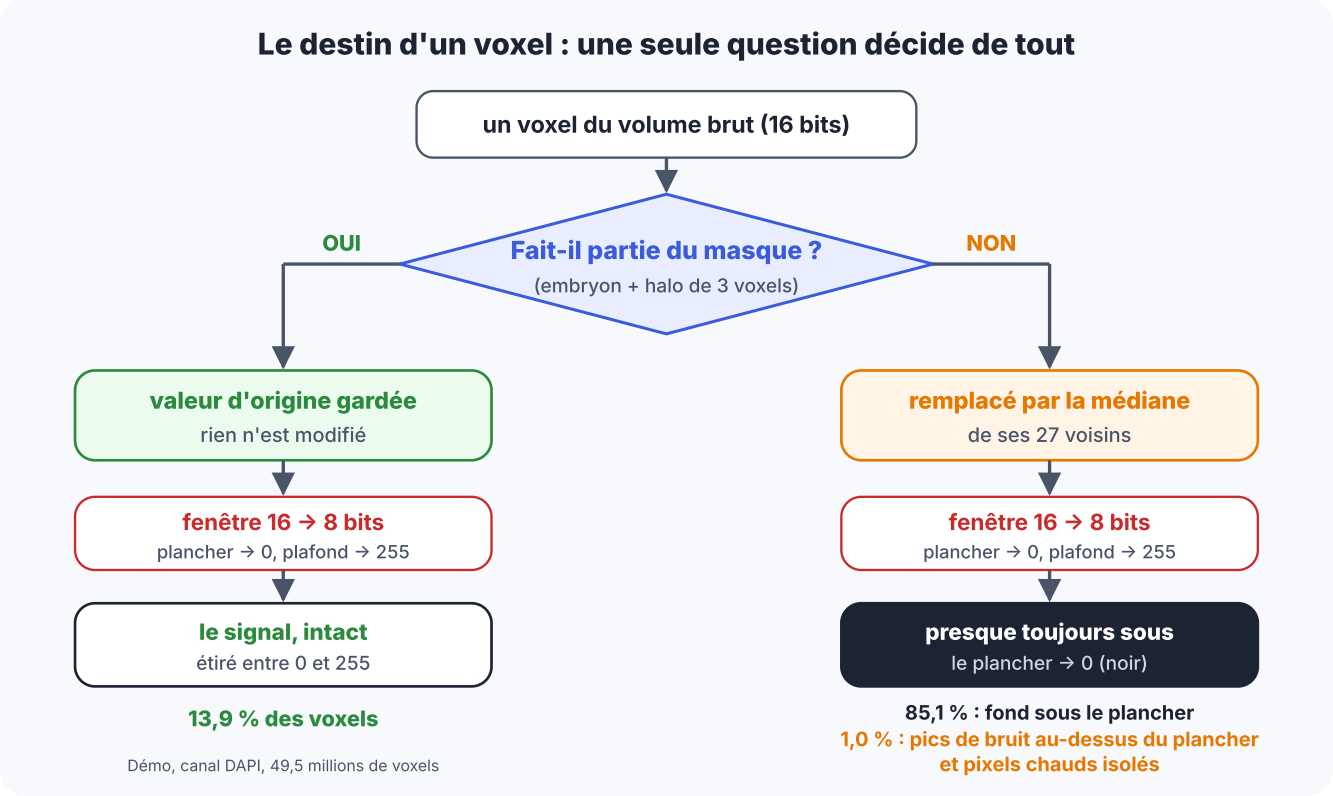
Dans le navigateur, le rendu **cumule** la lumière le long de chaque rayon. Un fond de 2 600 répété sur 112 coupes finit en brouillard gris qui masque l'embryon.

💡 **UNE IMAGE POUR COMPRENDRE**

C'est une **salle de cinéma avec un écran jamais tout à fait noir** : les lumières de secours éclairent la salle. Le nettoyage règle le « noir » de l'écran sur le niveau de la salle, pour que seul le film reste visible.

5.2 L'idée en une page

Tout le nettoyage se résume à **une seule question**, posée à chaque voxel : *fait-il partie de l'embryon ?*



▲ Le destin d'un voxel. Les pourcentages sont mesurés sur tout le volume du jeu de démonstration (canal DAPI).

OUI (il est dans le « masque ») :

- on **ne touche à rien** ;
- sa valeur d'origine est simplement convertie en 8 bits.

NON (il est hors du masque) :

- on le **remplace par la médiane** de ses 27 voisins ;
- cette valeur est presque toujours sous le plancher, donc il devient **0**.

85,1 % fond déjà sous le plancher

1,0 % pics de bruit au-dessus du plancher

13,9 % dans le masque : gardés tels quels

Pour répondre à la question, le pipeline a besoin de **deux repères** et d'**une carte** :

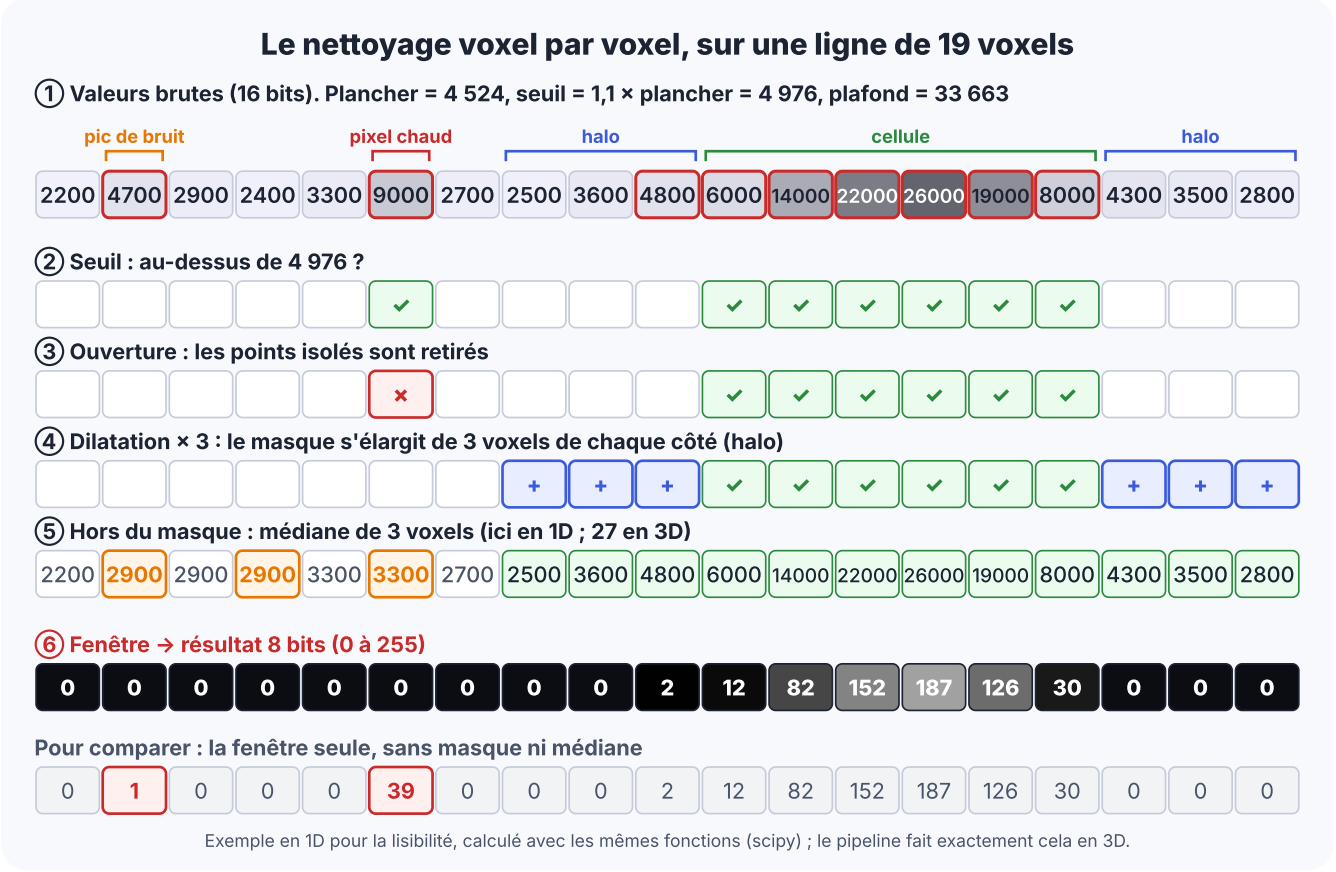
QUESTION	RÉPONSE DU PIPELINE	NOM DANS LE CODE
À quel niveau commence le vrai signal ?	99 % du bruit des 8 coins est en dessous	**plancher** (`bg_floor`)
À quel niveau mettre le blanc ?	99,9 % de tous les voxels sont en dessous	**plafond** (`sig_max`)
Où est l'embryon ?	les voxels nettement au-dessus du plancher, sans les points isolés, avec un halo	**masque**
Que faire du reste ?	remplacer chaque voxel par la médiane de ses voisins	**médiane**
Comment passer en 8 bits ?	plancher → 0, plafond → 255, une droite entre les deux	**fenêtre**

💡 **UNE IMAGE POUR COMPRENDRE**

Pensez à un **correcteur de copies** qui doit effacer les taches d'encre d'une feuille. Il ne touche pas aux lignes écrites (le masque). Une tache isolée (un point) est effacée. Pour savoir ce qu'est une tache, il regarde autour : si tout autour est blanc, c'est une tache ; si c'est au milieu d'un mot, c'est de l'écriture.

5.3 Un exemple complet, voxel par voxel

Prenons **19 voxels alignés**, avec les vrais repères du jeu de démonstration (plancher 4 524, plafond 33 663). On y a placé un **pic de bruit** (4 700), un **pixel chaud** (9 000) et une **cellule** entourée de son halo.



▲ Les six étapes, ligne par ligne. Exemple en une dimension pour qu'on puisse lire chaque nombre ; le pipeline applique exactement les mêmes règles en 3D.

Lisez la figure de haut en bas :

1. **Valeurs brutes.** Les cases encadrées de rouge dépassent le plancher. Il y en a 8 : le pic de bruit, le pixel chaud, la cellule et le bord du halo (4 800).

2. **Seuil.** On ne garde comme « signal certain » que ce qui dépasse **1,1 × plancher = 4 976**. Le pic de bruit (4 700) et le bord du halo (4 800) ne passent pas ; le pixel chaud (9 000) passe.

3. **Ouverture.** Le pixel chaud est **seul** : il est retiré (×). La cellule, assez large, reste.

4. **Dilatation × 3.** Le masque s'élargit de **3 voxels de chaque côté** de la cellule (+). Le halo, y compris le 4 800, est maintenant protégé.

5 **Médiane.** Hors du masque, chaque voxel prend la valeur du milieu de ses voisins. Le pic de bruit 4 700 devient **2 900** ; le pixel chaud 9 000 devient **3 300**. Dans le masque (vert), rien ne change.

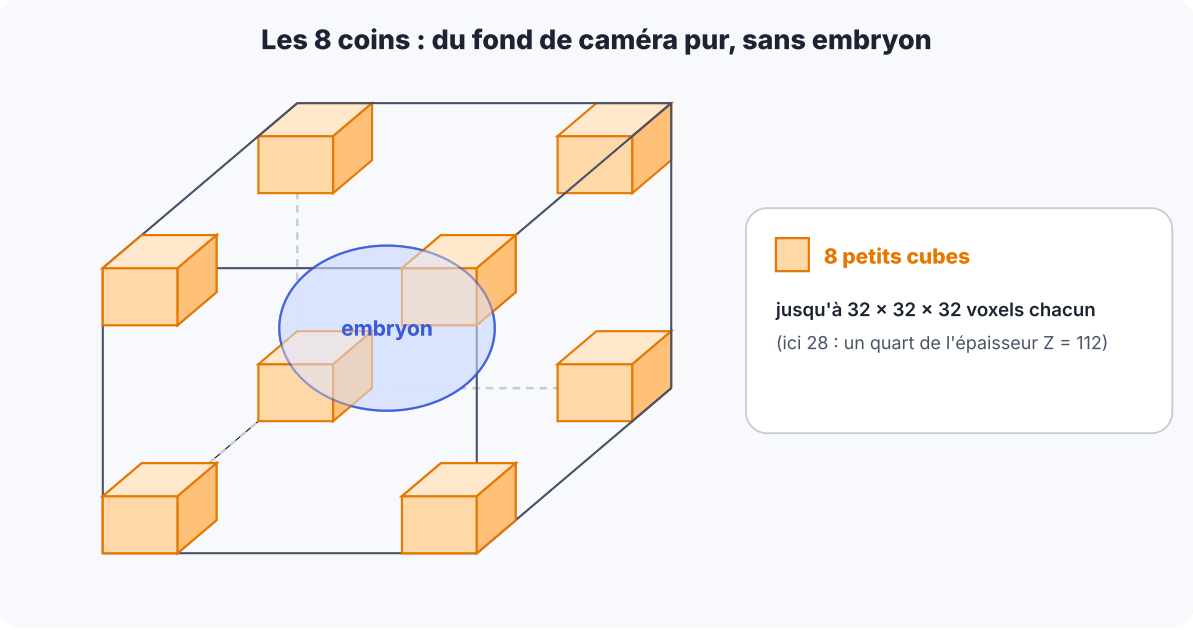
6 **Fenêtre.** Tout ce qui est sous 4 524 devient **0**. La cellule va de 12 à 187 ; le halo à 4 800 donne **2** (il est gardé, très sombre).

🔢 MICRO-EXEMPLE

Et sans masque ni médiane ? (dernière ligne de la figure) La fenêtre seule laisserait passer le pic de bruit (**1**) et le pixel chaud (**39**) : deux points gris parasites sur fond noir. C'est tout l'intérêt du masque et de la médiane.

5.4 Étape A : le plancher (`bg_floor`)

Où trouver du fond pur, sans embryon ? **Dans les coins** du volume : l'embryon est au centre, rien n'y touche. Le pipeline prélève **8 cubes**, un par coin, de **32 voxels de côté au maximum**.



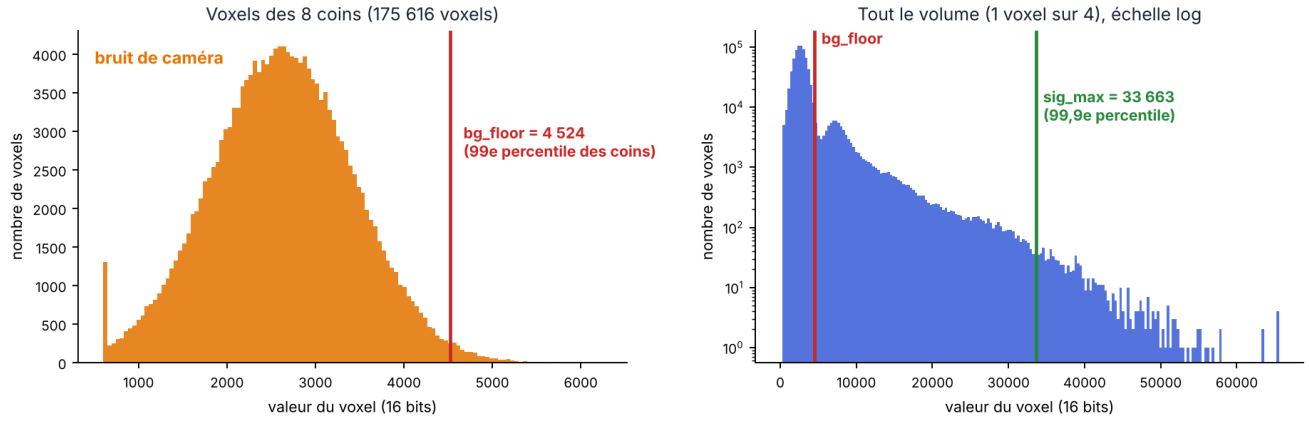
▲ Les 8 cubes de coin (schéma). Dans le jeu de démonstration, la pile ne compte que 112 coupes : le côté vaut min(32, X÷4, Y÷4, Z÷4) = 28 voxels, soit $8 \times 28^3 = 175\,616$ voxels de référence.

Puis : **plancher = 99e percentile de ces voxels**, c'est-à-dire la valeur sous laquelle se trouvent 99 % d'entre eux.

🔢 MICRO-EXEMPLE

On range les 175 616 voxels de coin du plus petit au plus grand. Le voxel qui se trouve aux 99 % de la file vaut **4 524** : c'est le plancher.

- la moyenne du bruit est d'environ 2 600 ;
- 99 % du bruit est **sous 4 524** ;
- 1 % du bruit le **dépasse** (le plus fort voxel de coin vaut 6 238).



▲ À gauche : les voxels des 8 coins forment une « cloche » de bruit ; la ligne rouge est leur 99e percentile. À droite : tout le volume (1 voxel sur 4, échelle logarithmique) ; le gros pic à gauche est le fond, la longue traîne est le signal (jeu de démonstration, canal DAPI).

❓ **POURQUOI CE CHOIX ?**

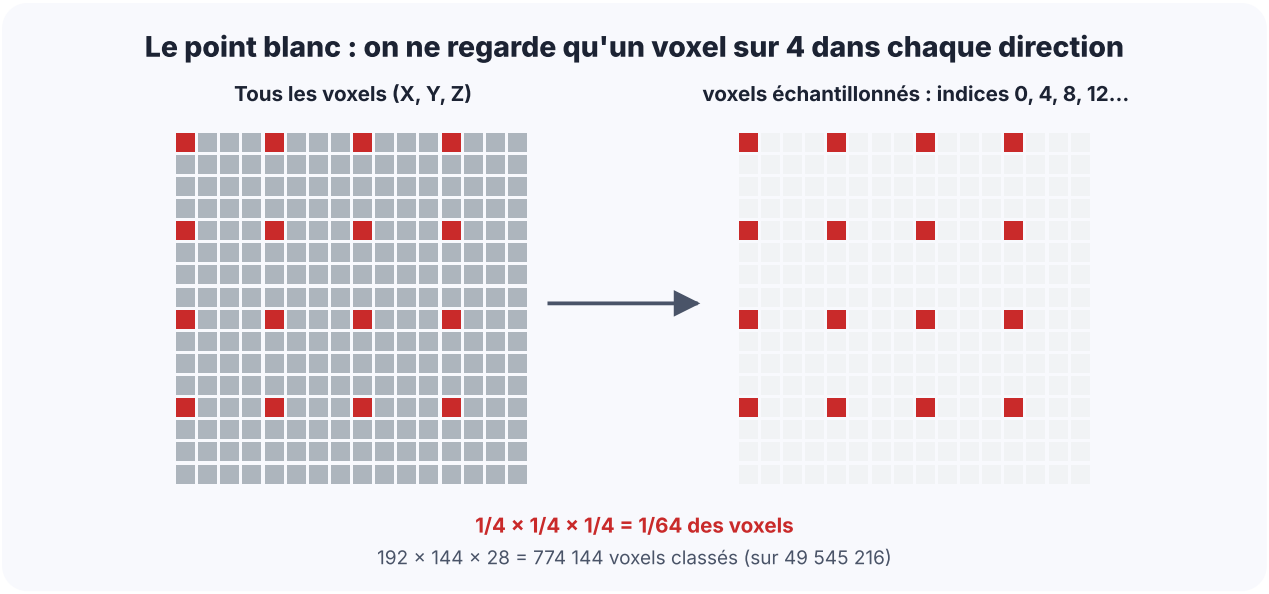
Pourquoi le 99e et pas le maximum ? Le maximum dépend d'un seul voxel, le plus extrême : il varie beaucoup d'une acquisition à l'autre et couperait du signal faible. Le 99e percentile est stable ; le 1 % de bruit qui le dépasse est traité ensuite par le masque et la médiane. Le code a testé d'autres règles (5e puis 20e percentile, 10e percentile des coins) avant de retenir celle-ci.

⚠️ **ATTENTION**

Si un coin touche l'échantillon (coupe serrée, mosaïque de tuiles), le plancher peut être trop haut et couper du signal faible. Le pipeline l'**écrit dans son journal** (« corner noise well above the volume median ») mais ne change rien : il ne corrige jamais en silence.

5.5 Étape B : le plafond (`sig_max`)

Pour le haut de l'échelle, on regarde **tout le volume**, mais en ne lisant **qu'un voxel sur 4** dans chaque direction (indices 0, 4, 8…). On lit ainsi 1 voxel sur 64, sans changer la statistique.



▲ Échantillonnage du point blanc : 1/64 des voxels suffisent.

Plafond = 99,9e percentile de ces voxels : les 0,1 % les plus brillants seront affichés en blanc (« saturés »). Dans le jeu de démonstration : **33 663**.

⏱ EN 30 SECONDES

- Plancher = 4 524 : ce qui est en dessous deviendra 0.
- Plafond = 33 663 : ce qui est au-dessus deviendra 255.
- La plage utile est donc **[4 524 ; 33 663]**, et non [0 ; 65 535].

5.6 Étape C : le masque, ou « où est l'embryon ? »

Le masque est une **carte oui / non** de la taille du volume : *oui* = ce voxel appartient à l'embryon (ou à son halo), on n'y touchera pas. Il se construit en trois gestes.

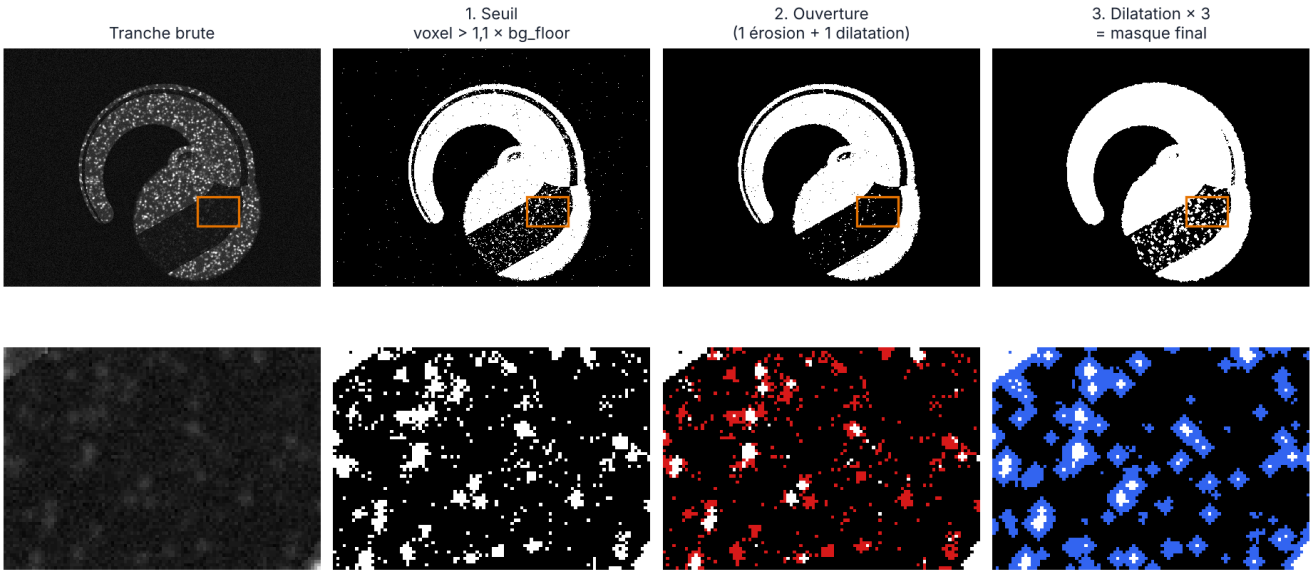
1. Seuil : plus de 1,1 × plancher. Ici 1,1 × 4 523,85 = **4 976**.

❓ POURQUOI CE CHOIX ?

Pourquoi 10 % au-dessus du plancher ? Par définition, 1 % du bruit dépasse le plancher. Avec 10 % de marge, il n'en reste que **0,2 %** (mesuré dans les coins du jeu de démonstration) : cinq fois moins de bruit pris pour du signal.

2. Ouverture : retirer les points isolés. C'est une **érosion** (on « ronge » 1 voxel sur tout le pourtour) suivie d'une **dilatation** (on « rend » 1 voxel). Un objet assez épais retrouve sa forme ; un point isolé, rongé entièrement à la première étape, ne revient pas.

3. Dilatation × 3 : ajouter un halo. Le masque est élargi de **3 voxels** tout autour. La fluorescence ne s'arrête pas net au bord d'une cellule : elle décroît. Le halo protège cette décroissance (et du signal faible tout proche) de la médiane.



▲ Les trois gestes du masque sur la coupe z = 56. Ligne du bas : zoom. En rouge, les voxels retirés par l'ouverture ; en bleu, ceux ajoutés par la dilatation.

11,77 % voxels au-dessus du seuil

11,32 % après l'ouverture

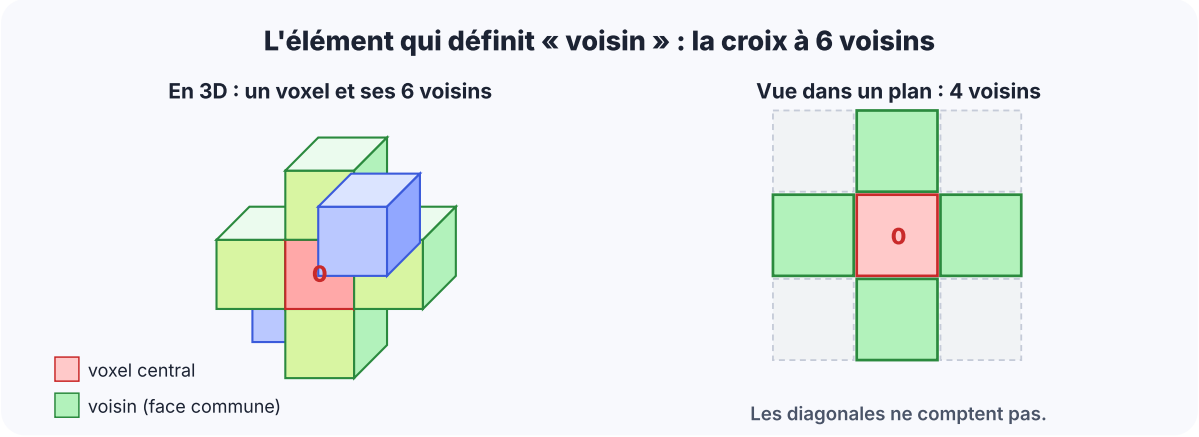
13,92 % masque final (avec halo)

🔢 **MICRO-EXEMPLE**

Le halo en micromètres. Avec les voxels du jeu de démonstration (1,2 × 1,2 × 3,0 µm), 3 voxels représentent **3,6 µm** en X et en Y, et **9 µm** en Z. Comme la dilatation se fait par faces, le halo est un peu plus étroit en diagonale (forme de losange).

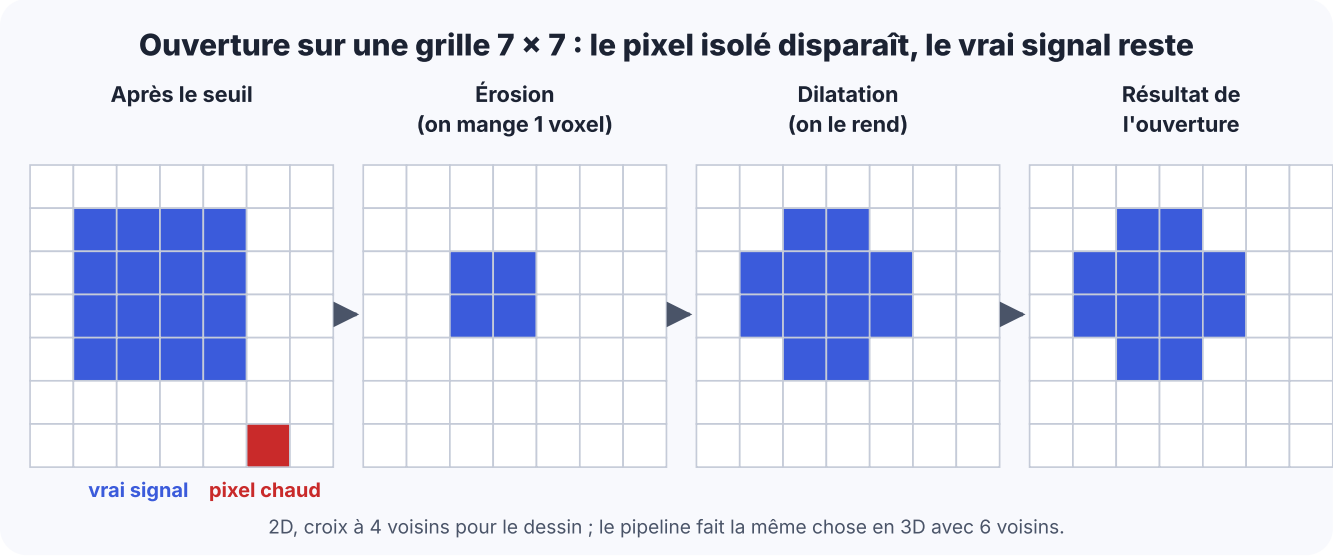
Qu'est-ce qu'un « voisin » ?

L'érosion et la dilatation regardent les voisins d'un voxel. Le pipeline utilise la **croix à 6 voisins** : les deux voisins en X, en Y et en Z, qui partagent une face. Les diagonales ne comptent pas.



▲ L'élément structurant : la croix 3D à 6 voisins (réglage par défaut de scipy, non modifié par le pipeline).

L'ouverture en miniature



▲ Mini-grille 7 × 7. Le bloc de 4 × 4 pixels survit (ses coins sont un peu arrondis), le pixel chaud isolé disparaît. Calculé avec scipy.

🔢 **MICRO-EXEMPLE**

Sur la grille : 17 pixels allumés au départ (16 de signal + 1 chaud) ; après l'ouverture il en reste **12** : le pixel chaud et les 4 coins du bloc sont partis. Les coins du bloc reviennent ensuite grâce à la dilatation × 3.

❓ **POURQUOI CE CHOIX ?**

L'ouverture est faite **avant** la dilatation. Dans l'ordre inverse, un pixel chaud serait « protégé » et même **agrandi** par la dilatation. Ici, il reste hors du masque, donc il sera écrasé par la médiane.

5.7 Étape D : la médiane, hors du masque seulement

Pour chaque voxel **hors du masque**, le pipeline prend les **27 voxels** du petit cube 3 × 3 × 3 qui l'entoure, les range dans l'ordre, et garde **celui du milieu** (le 14e). Dans le masque, la valeur d'origine est gardée.

▲ Médiane contre moyenne sur 9 valeurs (exemple 2D) : un pixel chaud à 52 000 est écrasé par la médiane (2 990), mais fait grimper la moyenne à 8 404.

💡 **UNE IMAGE POUR COMPRENDRE**

La **médiane**, c'est le salaire « du milieu » d'une rue : un milliardaire qui emménage ne la change pas. La **moyenne**, elle, explose. C'est pourquoi un filtre médian écrase un pixel chaud sans inventer de valeur nouvelle.

À quoi sert-elle vraiment ? À traiter le **1 % de bruit qui dépasse le plancher**. Sans elle, ces voxels deviendraient de petits points gris dans le fond noir.

341 674 points de bruit visibles avec la fenêtre seule

10 069 avec masque + médiane

– 97 % de points parasites

ℹ️ **À SAVOIR**

Les 10 069 voxels qui restent (0,02 % du volume) ont une médiane au-dessus du plancher : ce sont des **zones faibles mais étendues**, où plus de la moitié des 27 voisins dépassent le plancher. Un point isolé, lui, ne survit jamais : ses voisins sont du fond.

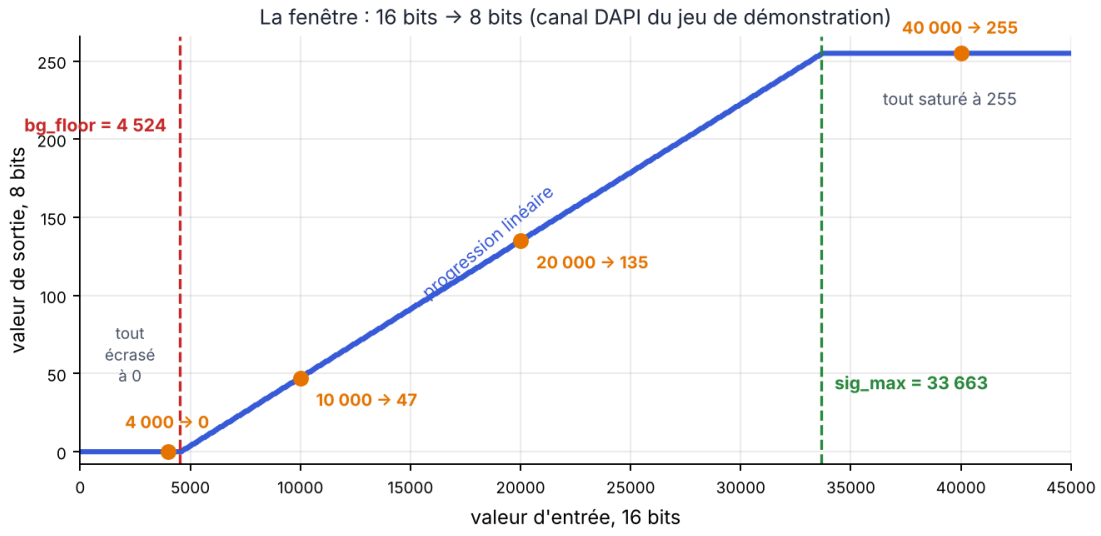
5.8 Étape E : la fenêtre, de 16 à 8 bits

Dernière opération, voxel par voxel, la même pour tout le volume. La formule exacte du code :

🔢 **MICRO-EXEMPLE**

u8 = tronqué(255 × (clip(v, plancher, plafond) – plancher) ÷ (plafond – plancher))

v est la valeur du voxel (d'origine dans le masque, médiane en dehors) ; *clip* ramène *v* entre le plancher et le plafond ; « tronqué » veut dire que l'on jette les décimales (pas d'arrondi).



▲ La fenêtre : plat à 0 sous le plancher, plat à 255 au-dessus du plafond, linéaire entre les deux. Les points orange sont les micro-exemples ci-dessous.

🔢 **MICRO-EXEMPLE**

Avec plancher = 4 523,85 et plafond = 33 662,57 (écart : 29 138,72) :

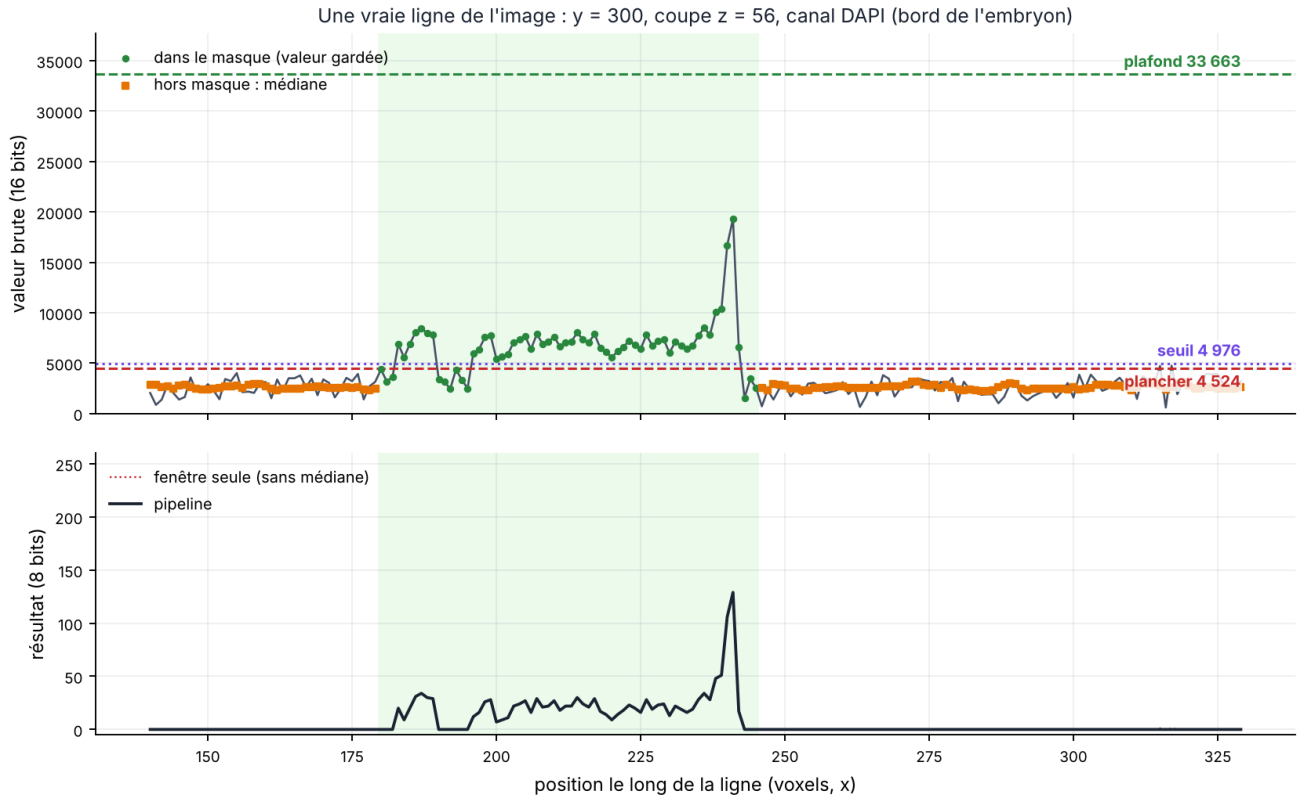
- v = 4 000 → sous le plancher → **0** ;
- v = 10 000 → 255 × 5 476,15 ÷ 29 138,72 = 47,9 → **47** ;
- v = 20 000 → 255 × 15 476,15 ÷ 29 138,72 = 135,4 → **135** ;
- v = 40 000 → au-dessus du plafond → **255**.

ℹ️ **À SAVOIR**

La conversion est **linéaire**, sans courbe gamma. Tout voxel ≤ plancher devient **exactement 0** : le fond est un noir parfait, ce qui permet de ne stocker aucune brique vide (chapitre 6). Dans le jeu de démonstration, **88,2 %** des voxels du canal DAPI valent 0 après nettoyage.

5.9 Sur de vraies images

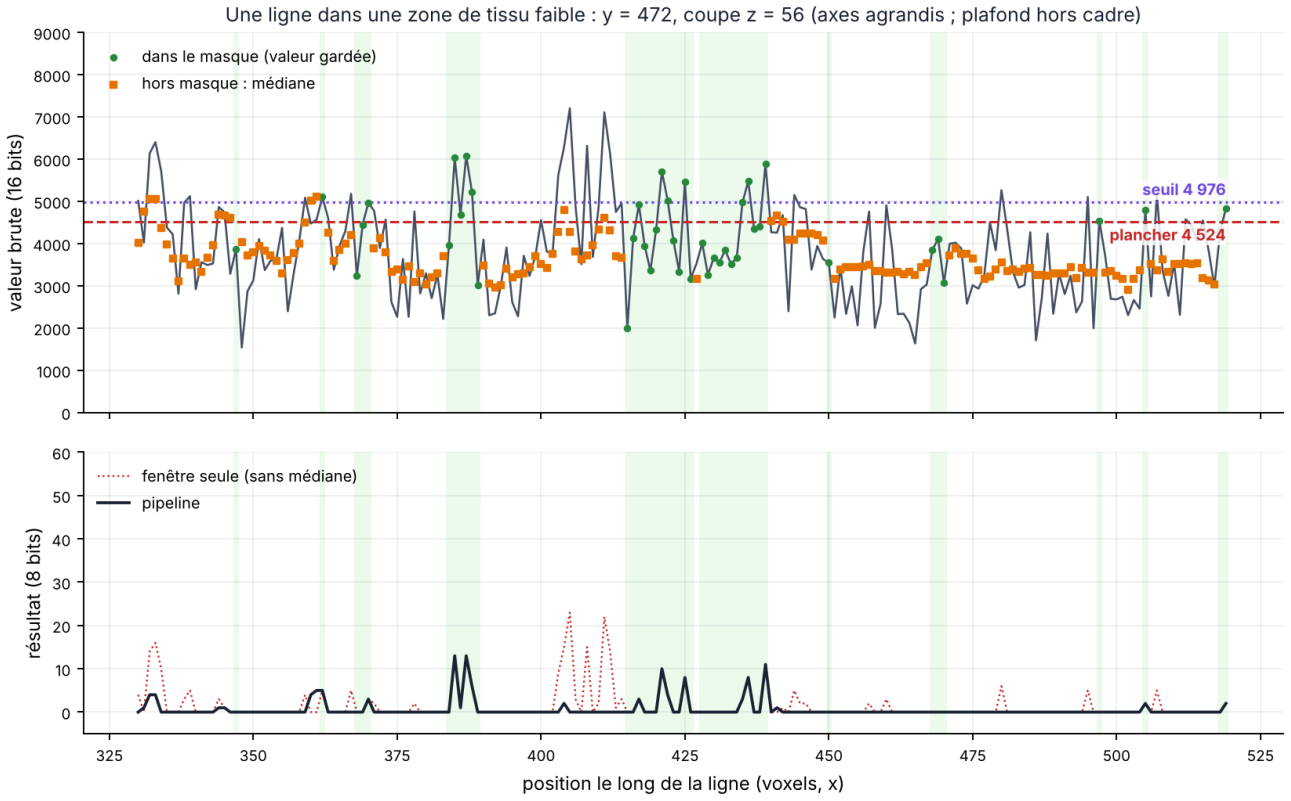
Une ligne au bord de l'embryon



▲ En haut, les valeurs brutes le long d'une ligne : vert = dans le masque (valeur gardée), orange = hors masque (la médiane, qui remplace la valeur). Bande verte = masque. En bas, le résultat 8 bits.

- **À gauche et à droite** (fond) : les valeurs brutes oscillent entre 1 000 et 4 000 ; la médiane (orange) les lisse autour de 2 600 ; résultat **0**.
- **Au centre** (tissu) : la bande verte couvre le tissu **et ses 3 voxels de halo** ; les valeurs sont gardées, et le creux sombre du milieu reste un creux.

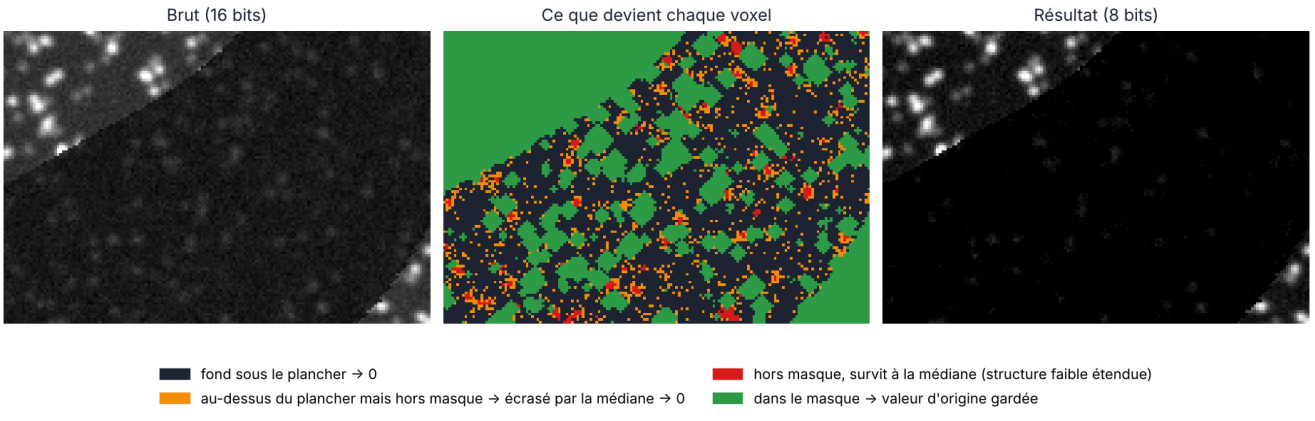
Une ligne dans une zone faible



▲ Même lecture, dans une zone où le tissu est à peine au-dessus du bruit (axes agrandis). Pointillé rouge : ce que donnerait la fenêtre seule.

- Seuls les pics qui forment un **petit amas** (au moins quelques voxels dans les 3 directions) entrent dans le masque et sont gardés.
- Les pics étroits (vers x = 405–412), même à 7 000, sont **retirés** : le pointillé rouge montre ce qu'on aurait vu sans médiane.

La carte des destins



▲ À gauche le brut, au milieu ce que devient chaque voxel, à droite le résultat. Dans une zone faible, le masque (vert) ne garde que les amas ; l'orange est écrasé par la médiane ; le rouge, rare, survit à la médiane.

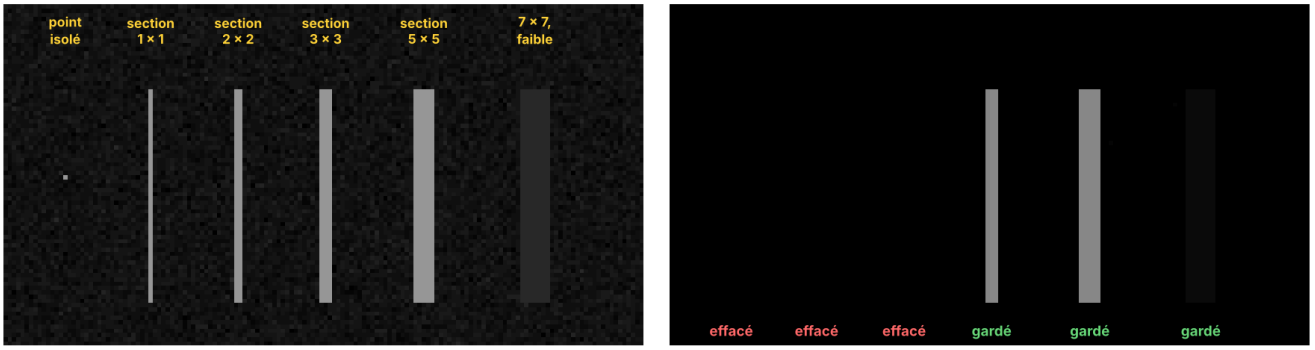
5.10 Taille minimale d’un objet

Conséquence directe de l’ouverture et de la médiane : **un objet trop fin est effacé, même s’il est très brillant.**

Taille minimale : environ 3 voxels d'épaisseur dans chaque direction

Avant : 5 objets brillants (20 000) et 1 faible (6 000), sur du bruit de caméra

Après le nettoyage (résultat 8 bits)



Volume synthétique de test, coupe du milieu : chaque barre a une section carrée (même épaisseur en X et en Z). Même plancher et même plafond que le canal DAPI de démonstration.

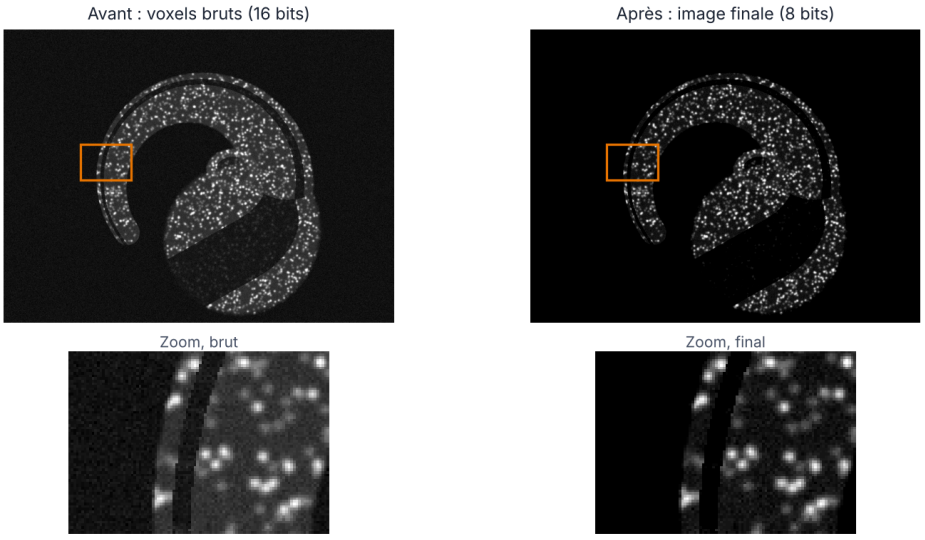
▲ Barres de section carrée, à 20 000 (brillant) ou 6 000 (faible). Les sections 1 × 1 et 2 × 2 disparaissent ; à partir de 3 × 3, l’objet est gardé intact, même faible.

FORME DE L’OBJET (EN VOXELS)	RÉSULTAT	POURQUOI
point isolé, fil de 1 × 1, feuille de 1 d’épaisseur	**effacé**	l’ouverture le retire ; la médiane de ses 27 voisins est du fond
fil de 2 × 2, petit cube 2 × 2 × 2	**effacé**	même raison : moins de 14 voisins sur 27 sont brillants
feuille de 2 d’épaisseur	gardé	l’ouverture le retire, mais la médiane reste brillante (18 voisins sur 27)
au moins 3 voxels dans chaque direction	**gardé intact**	il contient une croix complète : il entre dans le masque

⚠️ **ATTENTION**

Ordre de grandeur : environ 3 voxels dans chaque direction. Avec les voxels du jeu de démonstration (1,2 × 1,2 × 3,0 µm), cela fait **3,6 µm en X et en Y et 9 µm en Z**. Un objet présent sur une ou deux coupes seulement risque donc d’être effacé, quelle que soit sa brillance. Faites le calcul avec **vos** tailles de voxel ; en cas de doute, comparez avec le `.ims` d’origine. En pratique, le flou optique du microscope étale la plupart des objets sur plusieurs voxels, et un objet fin **collé** à un objet épais (à moins de 3 voxels) est protégé par le halo.

5.11 Avant et après



Même coupe z = 56, canal DAPI (jeu de démonstration). Le brut est affiché de 0 à sig_max pour une comparaison loyale.

▲ Même coupe avant (voxels bruts affichés de 0 au plafond) et après nettoyage. En bas : zoom sur le bord de l'embryon. Le grain gris du fond a disparu ; les noyaux sont intacts (jeu de démonstration).

Regardez le zoom : à gauche du bord, le fond gris granuleux est devenu noir pur ; à droite, les noyaux brillants ont la même forme et le même contraste.

5.12 Ce qui a été essayé, puis abandonné

Le pipeline a connu des méthodes plus sophistiquées, retirées à la version 0.12.0 :

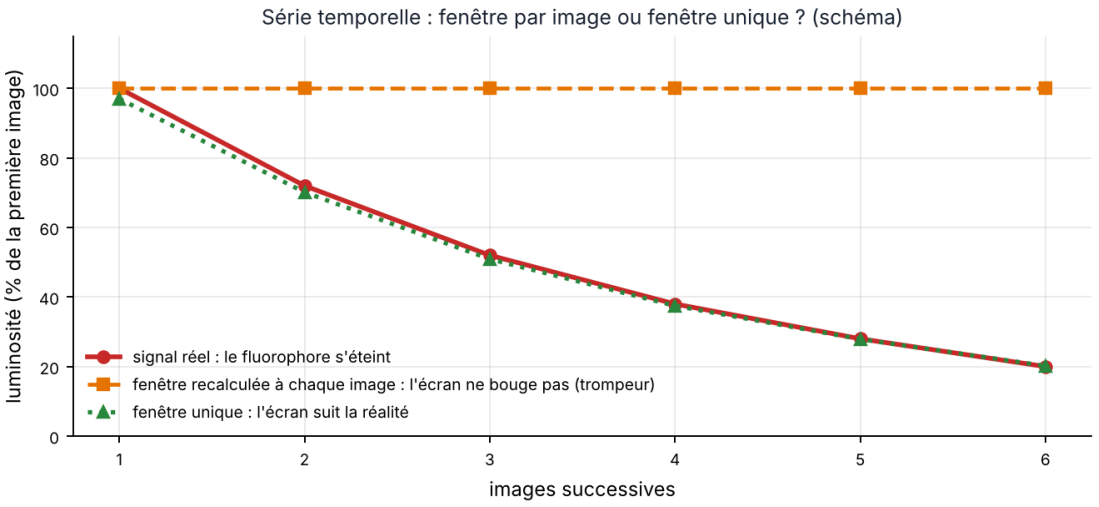
MÉTHODE	POURQUOI ABANDONNÉE
Seuil d'Otsu	Soustraction du masque : créait de **gros blobs colorés** artificiels
Noise2Void (réseau de neurones)	Même résultat artefactuel ; l'embryon perdait sa texture naturelle

❓ **POURQUOI CE CHOIX ?**

Choix actuel : le pipeline **ne retire que le fond global de la caméra** et garde les intensités natives du microscope. Pas de réseau de neurones, pas de correction de champ plat. Un test sur un jeu de référence a retrouvé les 97 empreintes SHA-256 des paquets de production, octet pour octet.

5.13 Séries temporelles : une seule fenêtre

Pour un film (plusieurs images), faut-il une fenêtre par image ? **Non : une seule pour toute la série**, pour chaque canal. Sinon l'image « clignote ».



▲ Schéma (chiffres illustratifs). Un fluorophore s'éteint (photoblanchiment). Si chaque image est étirée sur sa propre fenêtre, l'écran reste constant alors que le signal s'effondre : c'est faux. Avec une fenêtre unique, l'écran baisse comme le signal réel.

Comment la fenêtre unique est-elle mesurée ?

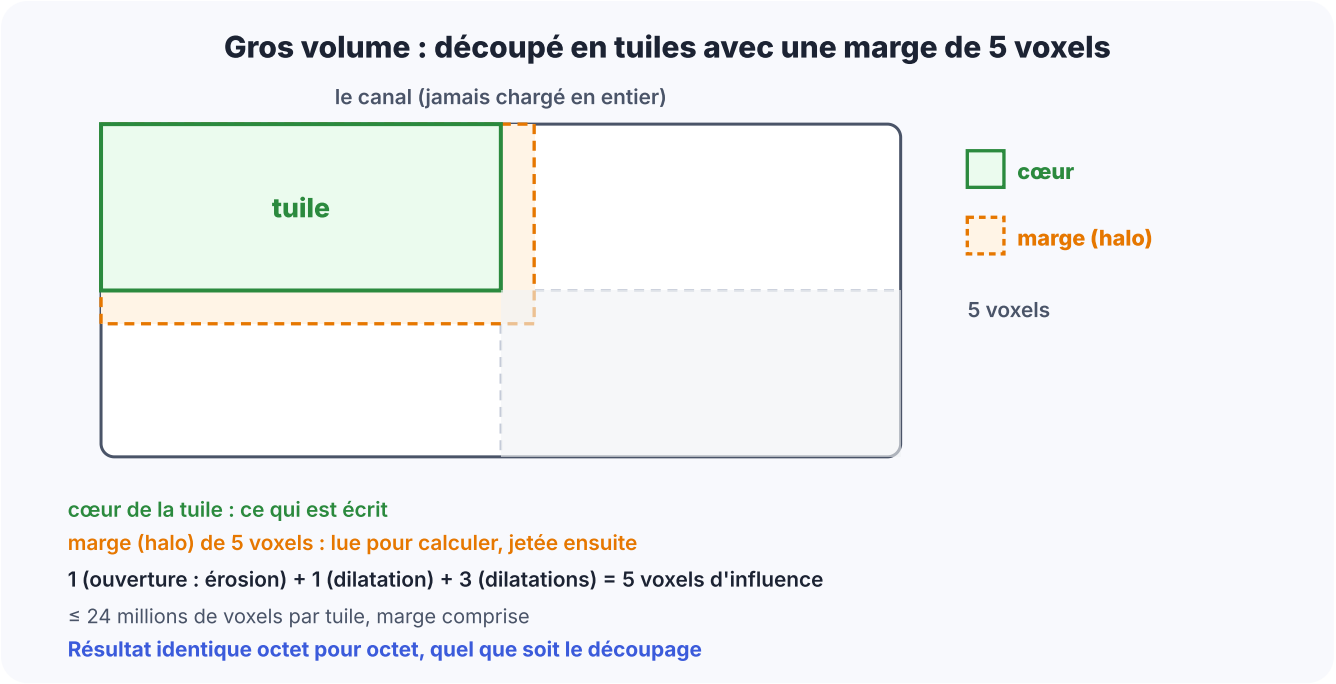
- jusqu'à **8 images** échantillonnées (régulièrement espacées, toujours la première et la dernière) ;
- `bg_floor` = 99e percentile des coins de toutes ces images réunies ;
- `sig_max` = 99,9e percentile **des seuls voxels au-dessus de** `bg_floor`, s'il y en a au moins **1 000**.

> **❓ POURQUOI CE CHOIX ?**
>
> Dans une série éparse, le signal ne couvre que ~0,4 % des voxels : le 99,9e percentile du volume entier tomberait **dans le fond** et ferait saturer 15 % du vrai signal. En ne classant que les voxels au-dessus du fond, la part saturée tombe à environ 0,06 % (mesures du journal de version 0.15.0).

Le photoblanchiment est **mesuré, jamais effacé** : le pipeline enregistre un niveau de signal par image et par canal (99,9e percentile du brut) dans `metadata.json`. Aucun réglage du viewer n'utilise encore ces niveaux : une série qui s'éteint s'affiche donc réellement plus sombre, et rien ne la « rattrape » à l'écran.

5.14 Gros volumes : par tuiles, sans changer le résultat

Un canal de 3789 × 3789 × 178 voxels représente 2,56 milliards de voxels, soit 10,2 Go en flottants 32 bits : il ne tient pas en mémoire. Le pipeline le lit donc **par tuiles** d'au plus **24 millions de voxels** (marge comprise).



▲ Découpage en tuiles. La marge de 5 voxels fournit aux calculs de bord (ouverture, dilatations) les voisins qui leur manquent ; seul le cœur est écrit.

Pourquoi 5 voxels ? L'érosion (1) + la dilatation de l'ouverture (1) + les 3 dilatations (3) propagent l'influence d'un voxel sur 5 voxels de distance. La médiane en demande 1 (couvert par la même marge).

🔢 **MICRO-EXEMPLE**

Vérifié sur le jeu de démonstration (3 canaux) : découpé en 4 tuiles (réglage par défaut), puis en **256 tuiles** (0,5 M de voxels), le volume 8 bits est **identique octet pour octet** au calcul d'un seul bloc, et à celui du dataset publié par le pipeline (coupes z = 0, 30, 56 et 100).

5.15 Ce que cela change pour vous

⚠️ **ATTENTION**

Les valeurs 8 bits sont **relatives**, pas absolues. Chaque canal a sa propre fenêtre [`bg_floor` ; `sig_max`] ; 128 sur DAPI et 128 sur Pecam1 ne représentent pas la même quantité de lumière. Ne comparez pas d'intensités entre canaux ni entre datasets.

- Dans une **série temporelle**, les valeurs sont comparables entre images **d'un même canal** (fenêtre unique).
- Le fichier **.ims original** n'est jamais modifié ; c'est la source pour toute mesure d'intensité. Il peut être téléchargé depuis le dossier `download/` si le dataset a été publié avec ses fichiers d'origine.
- Peuvent être effacés : un signal **très faible** (sous 1,1 × `bg_floor`) qui ne forme pas une zone étendue, et tout objet **plus fin qu'environ 3 voxels** dans une direction, même brillant (§ 5.10). C'est le prix d'un fond parfaitement noir. Dans le masque, en revanche, les valeurs d'origine sont gardées.

📌 **À RETENIR**

1. `bg_floor` = 99e percentile de 8 cubes de coin ; `sig_max` = 99,9e percentile d'un voxel sur 4.
2. Masque = seuil 1,1 × `bg_floor` → ouverture (retire les points isolés) → dilatation × 3 (halo).
3. Médiane 3 × 3 × 3 **hors masque** ; le signal reste intact. Un objet de moins de ~3 voxels d'épaisseur est effacé.
4. 8 bits = fenêtre linéaire [`bg_floor` ; `sig_max`] → [0 ; 255], tronquée ; fond = 0 exact.
5. Une seule fenêtre pour tout un film ; les tuiles ne changent pas le résultat.

6. Réduire la taille sans rien perdre d'important

- Un embryon de plusieurs gigaoctets ne passe pas sur Internet tel quel : le pipeline le **réduit** en quatre mouvements — 16 → 8 bits, pyramide de résolutions, suppression des briques vides, compression **sans perte**.
- Le **format 5** ajoute, pour l'affichage seulement, quatre compressions plus légères (dont deux avec perte) ; ce qui mesure lit toujours la version sans perte (section 7.11).
- Aucun de ces mouvements ne modifie une intensité du niveau le plus fin : ce que vous voyez à pleine résolution est **exactement** le résultat du nettoyage du chapitre 5.
- Sur le jeu de démonstration : **297 Mo de voxels bruts → 9,3 Mo de briques**, soit environ 32 fois moins.

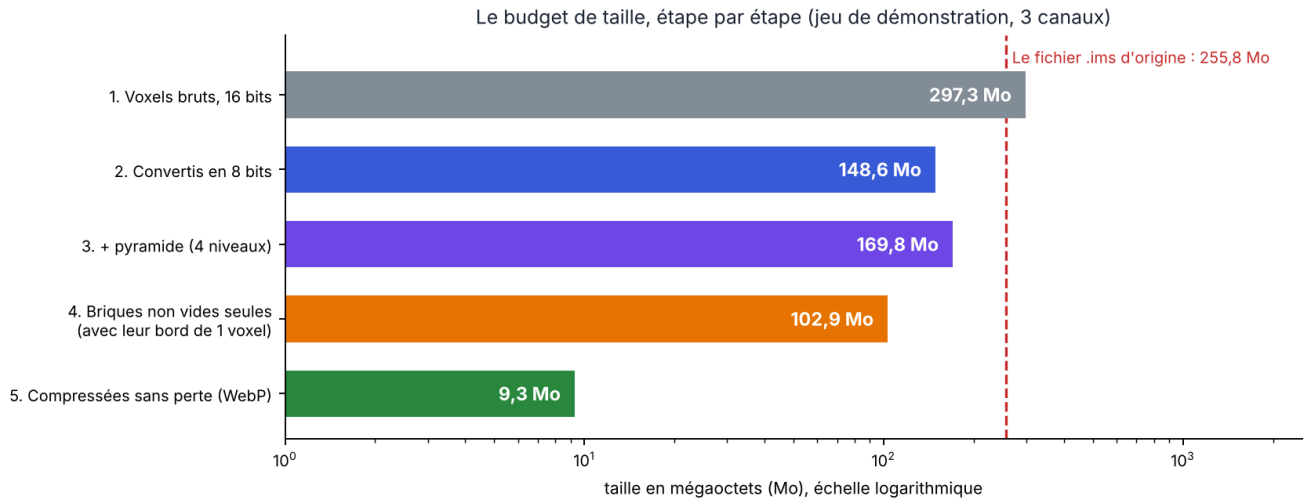
Tous les chiffres « démo » de ce chapitre sont **mesurés** sur l'embryon synthétique `Embryo-E95-Em2-Pecam1-Sox2` (768 × 576 × 112 voxels, 3 canaux), publié par le vrai pipeline (format 4).

6.1 Le budget de taille en chiffres

Prenons d'abord un **gros** embryon du laboratoire : 3789 × 3789 × 178 voxels, 4 canaux. Cela fait 10,2 milliards de voxels.

ÉTAPE	CALCUL	TAILLE
Voxels bruts, 16 bits	10,22 G voxels × 2 octets	**20,4 Go**
Convertis en 8 bits	10,22 G voxels × 1 octet	**10,2 Go**
+ pyramide de résolutions	× 1,286 pour ce découpage	**13,1 Go**
Fichier .ims réel de ce type	compressé par Imaris	14,4 Go

Les deux dernières étapes (briques vides retirées, compression) dépendent du contenu de l'image. Mesurons-les sur le jeu de démonstration, étape par étape :



▲ Budget de taille du jeu de démonstration. L'axe est logarithmique : chaque graduation vaut dix fois la précédente.

297 Mo 16 bits bruts

149 Mo en 8 bits

103 Mo briques non vides

9,3 Mo sur le disque

ℹ️ **À SAVOIR**

La pyramide **ajoute** des octets (+14 % ici) : elle est le prix de l'affichage rapide. Le gain vient des deux étapes suivantes : ne pas stocker le vide, et compresser.

6.2 De 16 bits à 8 bits : diviser par deux

C'est la fenêtre du chapitre 5 : un voxel de 2 octets devient un voxel d'1 octet. On perd la finesse des 65 536 niveaux, mais le navigateur n'en affiche que 256 : l'écran ne sait pas montrer plus.

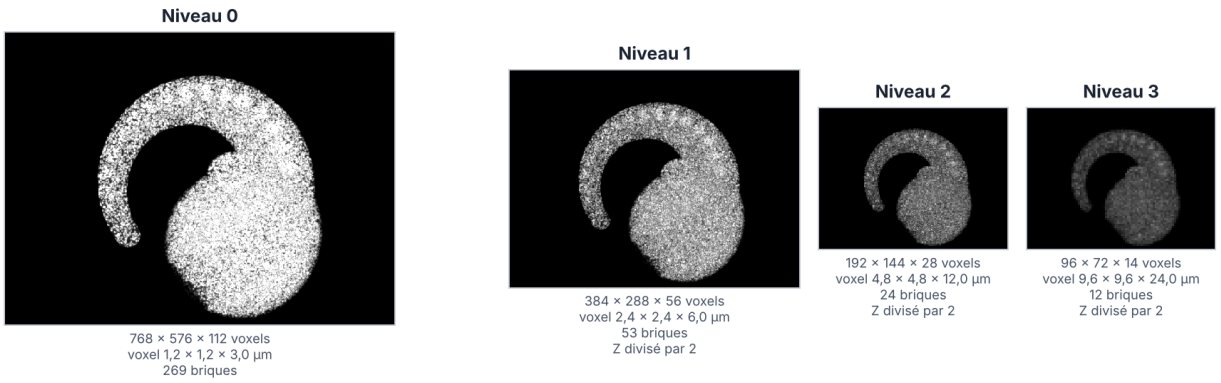
⚠️ **ATTENTION**

C'est **la seule étape qui perd de l'information** (les nuances au-dessus de 256 niveaux), et elle a lieu au chapitre 5, pas ici. Tout ce qui suit dans ce chapitre est **sans perte**.

6.3 La pyramide de résolutions

Quand vous regardez l'embryon entier, vous n'avez pas besoin de tous les voxels : l'écran ne compte que quelques millions de pixels. Le pipeline prépare donc **plusieurs versions** de l'image, de plus en plus petites : les **niveaux**.

La pyramide de résolutions : même coupe projetée (MIP), niveaux 0 à 3



▲ Les quatre niveaux du jeu de démonstration, dessinés à l'échelle (projection de l'intensité maximale du canal DAPI). Même forme, de moins en moins de voxels.

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> C'est une **carte routière** : la carte du pays pour voyager, la carte de la ville pour se repérer, le plan de la rue pour trouver la porte. On ne charge que la carte utile.

La règle du format 4

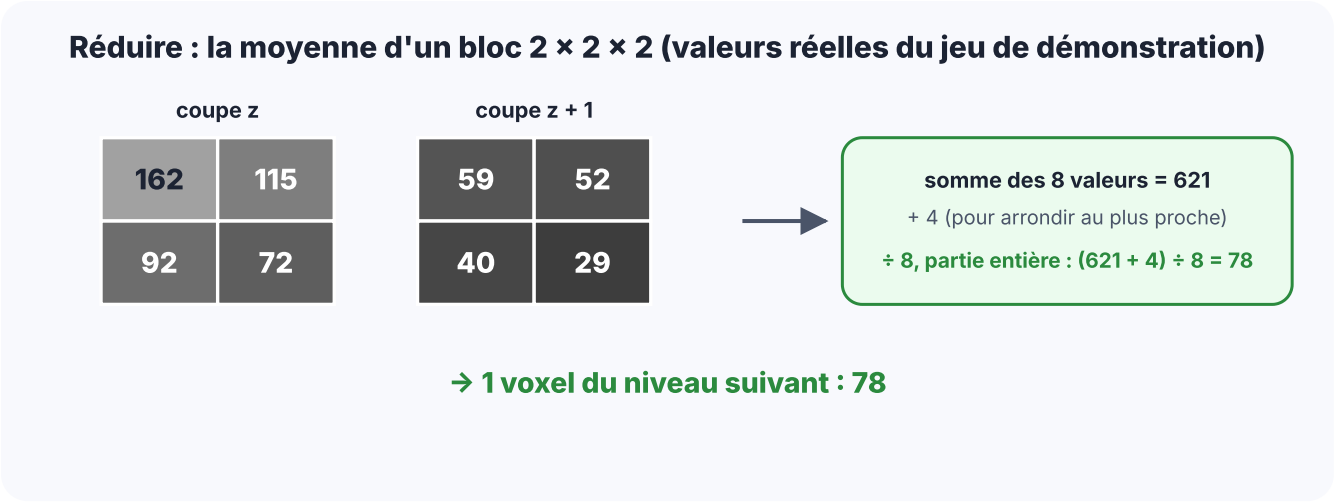
- **Niveau 0** : l'image nettoyée, **telle quelle** (jamais rééchantillonnée).
- **Niveau suivant** : X et Y sont **divisés par 2** (arrondi vers le haut) et leur voxel double.
- **Z est divisé par 2 seulement si** la profondeur du voxel reste inférieure ou égale à **1,5 fois** le nouveau voxel en XY. Sinon Z est gardé.
- On s'arrête quand le plus grand côté en XY est **inférieur ou égal à 128** voxels.

> ❓ **POURQUOI CE CHOIX ?**
>
> Un voxel trop « plat » ou trop « allongé » déforme l'image. Diviser Z seulement quand il est encore comparable à XY ramène peu à peu le voxel vers un **cube** (c'est la convention des formats d'images biomédicales modernes, comme OME-Zarr).

Le calcul d'un voxel du niveau suivant

Chaque voxel du niveau suivant est la **moyenne** d'un bloc de 2 × 2 (× 2) voxels du niveau précédent. Le calcul se fait en **nombres entiers**, **arrondi à la valeur la plus proche, la moitié vers le haut** : (somme + n ÷ 2) ÷ n, partie entière, n étant le nombre de voxels du bloc.

> 🔢 **MICRO-EXEMPLE**
>
> Bloc réel du jeu de démonstration : 162, 115, 92, 72, 59, 52, 40, 29. Somme = **621**. On ajoute 4 (la moitié de 8), puis on divise par 8 : (621 + 4) ÷ 8 = 78,1 → **78**. Sans l'ajout de 4, on aurait obtenu 77 : le biais aurait assombri chaque niveau.



▲ Le calcul d'un voxel du niveau suivant, sur un bloc réel du jeu de démonstration.

Au bord du volume, un bloc peut n'avoir que 1 voxel de large : on ne moyenne alors **que les voxels qui existent**.

L'échelle du jeu de démonstration

Voxel de départ : 1,2 × 1,2 × 3,0 µm. Pour chaque niveau, la règle de Z est testée avec le **nouveau** voxel XY.

NIVEAU	VOXELS (X × Y × Z)	VOXEL (MM)	Z DIVISÉ ?
0	768 × 576 × 112	1,2 × 1,2 × 3,0	(base)
1	384 × 288 × 56	2,4 × 2,4 × 6,0	oui : 3,0 ≤ 1,5 × 2,4 = 3,6
2	192 × 144 × 28	4,8 × 4,8 × 12,0	oui : 6,0 ≤ 1,5 × 4,8 = 7,2
3	96 × 72 × 14	9,6 × 9,6 × 24,0	oui : 12,0 ≤ 1,5 × 9,6 = 14,4

On s'arrête au niveau 3 : 96 ≤ 128.

🔧 **POUR LES CURIEUX**

Quand Z n'est pas divisé. Pour une pile 3789 × 3789 × 178 avec un voxel (0,33 ; 0,33 ; 1,876 µm) (tailles de voxel *illustratives*), le niveau 1 garde Z : 1,876 > 1,5 × 0,66 = 0,99. Le niveau 2 le divise : 1,876 ≤ 1,5 × 1,32 = 1,98. Le résultat est 3789 × 3789 × 178 → 1895 × 1895 × 178 → 948 × 948 × 89 → 474 × 474 × 45 → 237 × 237 × 23 → 119 × 119 × 12 (6 niveaux).

ℹ️ **À SAVOIR**

J'ai reconstruit cette pyramide avec les fonctions du pipeline à partir du résultat du chapitre 5, puis comparé aux briques publiées : **358 briques sur 358 identiques**, voxel pour voxel.

Et en format 5 ?

Le format 5 (section 7.11) change la règle de la pyramide :

	FORMAT 4	FORMAT 5
tailles des niveaux	divisées par 2 à chaque fois	grand côté **exactement** 4096, 2048, 1024, 512, 256
tranches (Z)	divisées tant que le voxel reste ≈ cubique	**toutes gardées** à chaque niveau
calcul	moyenne de 2 × 2 (× 2) voxels du niveau précédent	moyenne **exacte de surface** des voxels natifs, depuis le niveau 0
jeu de démonstration	768, 384, 192, 96 (et 112, 56, 28, 14 tranches)	768 (natif), 512, 256, toujours 112 tranches

Le niveau 0 reste, dans les deux cas, l'image nettoyée telle quelle.

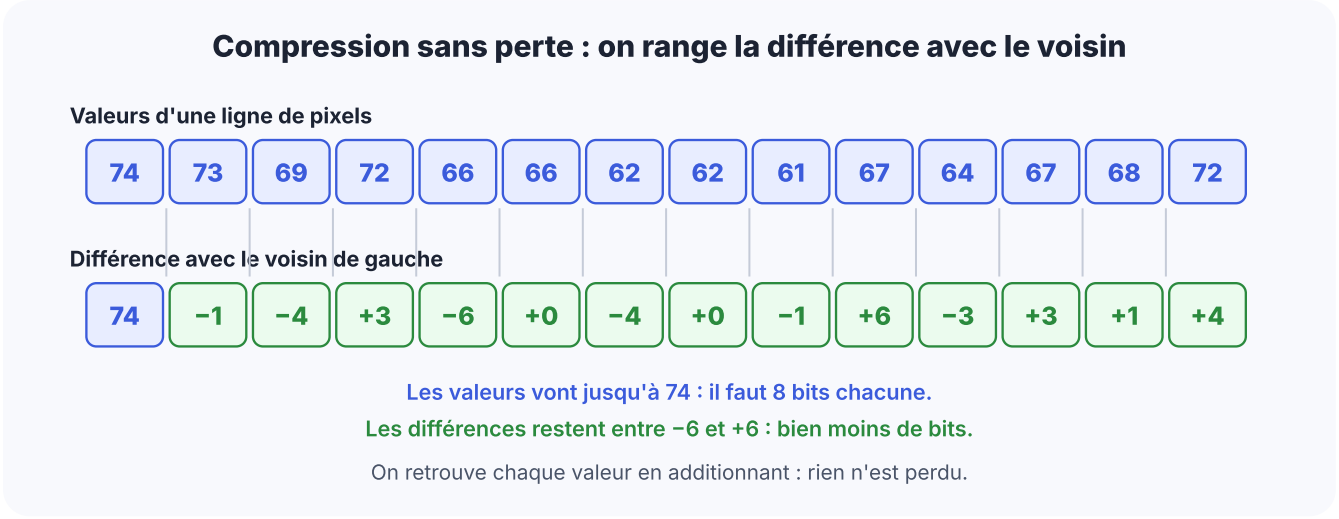
6.4 Compresser sans rien perdre

Pourquoi « sans perte » ?

Une compression **avec perte** (JPEG) change légèrement les valeurs pour gagner de la place : acceptable pour une photo de vacances, pas pour une **mesure**. Une compression **sans perte** permet de retrouver chaque voxel à l'identique.

Comment ça marche, en clair

L'idée : un voxel ressemble à son voisin. Plutôt que de ranger chaque valeur, on range **la différence avec le voisin**, qui est petite et se compresse bien.



▲ Une vraie ligne de 14 voxels du canal Sox2 (jeu de démonstration). Les différences sont petites ; en les additionnant dans l'ordre, on retrouve les valeurs d'origine.

🔧 POUR LES CURIEUX

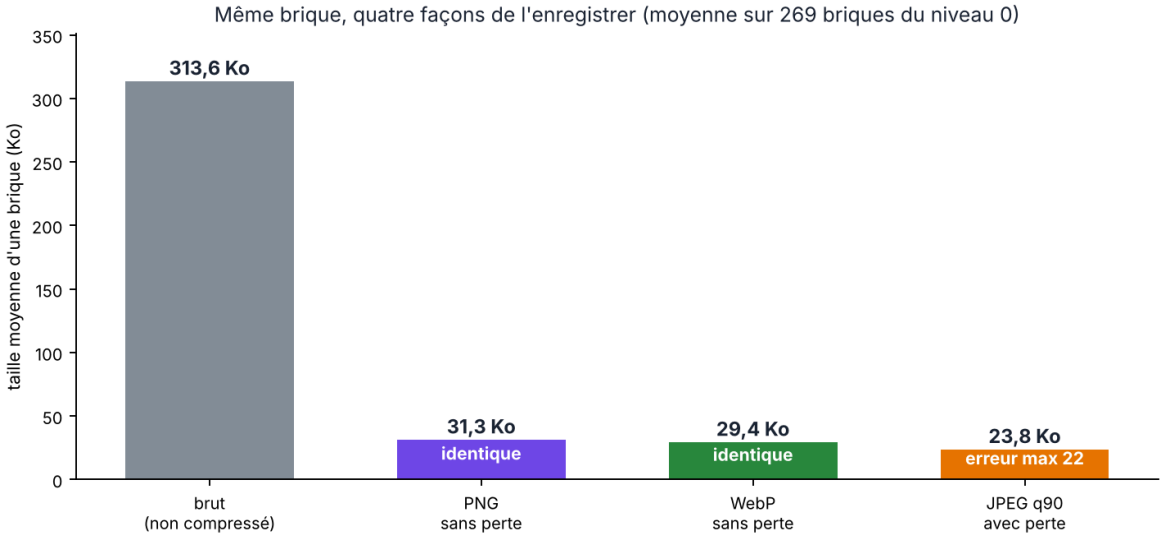
Les compresseurs réels sont plus fins : ils prédisent un voxel à partir de plusieurs voisins (gauche, haut…), puis codent économiquement les écarts, les plus fréquents en premier. Le principe reste celui-ci, et il est exactement réversible.

Les briques sont des images WebP sans perte

Une brique 3D n'est pas une image 2D. Le pipeline range donc ses 66 coupes côte à côte dans une **mosaïque 594 × 528 pixels** (9 colonnes × 8 lignes ; chapitre 7), et enregistre la mosaïque en **WebP sans perte**.

La mesure : quatre façons d'enregistrer la même brique

J'ai pris les **269 briques non vides du niveau 0** (3 canaux) du jeu de démonstration et enregistré chaque mosaïque de 594 × 528 pixels de quatre façons.



▲ Taille moyenne d'une brique. Brut : 313,6 Ko. PNG (meilleure compression) : 31,3 Ko. WebP sans perte (réglage du pipeline) : 29,4 Ko. JPEG qualité 90 : 23,8 Ko, mais avec une erreur allant jusqu'à 22 niveaux de gris.

FORMAT	TAILLE MOYENNE	EXACT ?
Brut (non compressé)	313,6 Ko	oui
PNG, niveau 9 optimisé	31,3 Ko	oui
WebP sans perte (choix du pipeline)	**29,4 Ko**	oui
JPEG, qualité 90	23,8 Ko	**non**

⚠️ **ATTENTION**

JPEG est le plus petit, mais il est faux. Sur ces briques : erreur maximale de 22 niveaux (sur 255), **11,7 %** des voxels modifiés, et **1 644 202 voxels qui valaient exactement 0 sont devenus non nuls** (2,2 % du fond). Or le « 0 exact » est ce qui permet de reconnaître une brique vide (6.5) : JPEG le détruirait.

▲ Zoom sur une coupe de brique (canal Pecam1) : après JPEG, de faibles halos gris apparaissent autour des structures fines, là où l'original est parfaitement noir. La différence est amplifiée 8 fois à droite.

❓ **POURQUOI CE CHOIX ?**

Pourquoi WebP plutôt que PNG ? Sur ces briques, WebP sans perte est seulement **6 % plus petit** que le meilleur PNG (29,4 contre 31,3 Ko). Le code donne une autre raison principale : le navigateur **décode WebP nativement** et vite (`createImageBitmap`), alors qu'il n'existe aucun format d'image 3D. Les tranches 2D de `planes/` , elles, sont en PNG : le viewer les lit avec un petit décodeur PNG maison, jamais via le canevas du navigateur, dont la lecture n'est pas garantie exacte.

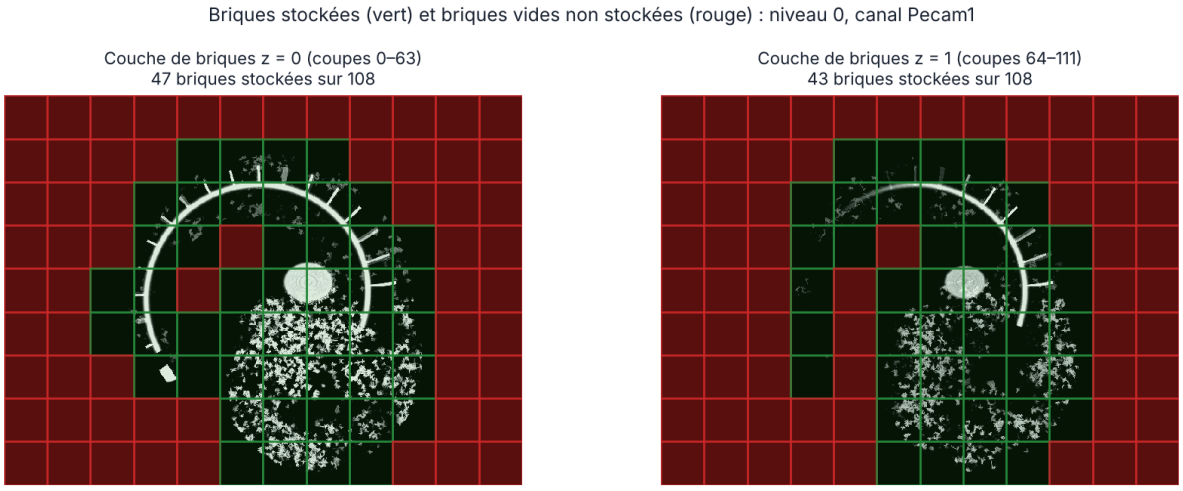
ℹ️ **À SAVOIR**

Les octets du pipeline sont reproductibles : en recodant les mosaïques avec les mêmes réglages (Pillow, WebP sans perte, méthode 4), j'ai retrouvé **exactement les mêmes octets** que dans les fichiers publiés. Seule la version de la bibliothèque WebP compte : c'est pourquoi Pillow est épinglé.

6.5 Ne pas stocker le vide

Un embryon de souris ne remplit pas son cadre. Beaucoup de briques ne contiennent que du noir : à quoi bon les enregistrer ?

La règle (format 4) : une brique est stockée **si et seulement si** son intérieur de 64 × 64 × 64 voxels contient **au moins un voxel ≥ 1**. Pas de tolérance : un seul voxel faible suffit à la garder.



▲ Niveau 0, canal Pecam1, deux couches de briques. Vert : brique stockée. Rouge : brique vide, non stockée. Les briques sont tracées sur une projection de l'intensité maximale de la couche.

Le comptage exact, lu dans `index.bin` du jeu de démonstration (3 canaux) :

NIVEAU	BRIQUES POSSIBLES	BRIQUES STOCKÉES
0	648 (216 par canal)	**269** (DAPI 100, Pecam1 90, Sox2 79)
1	90	53
2	27	24
3	12	12
Total	**777**	**358** (46 %)

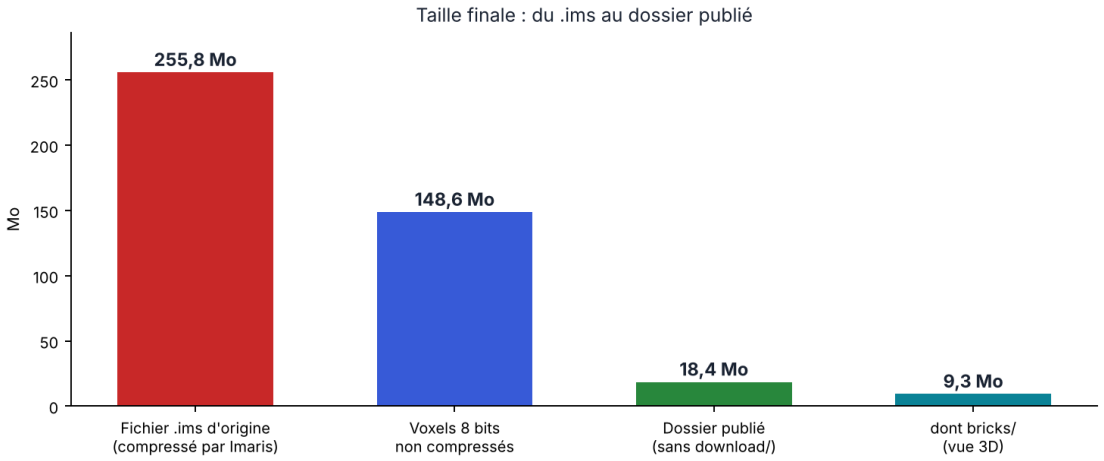
🔢 **MICRO-EXEMPLE**

Le canal Pecam1 n'a que **0,75 %** de voxels non nuls, mais **90 briques sur 216** du niveau 0 sont stockées : quelques vaisseaux fins traversent beaucoup de briques. Une brique stockée n'est pas pleine ; elle contient au moins un voxel.

ℹ️ **À SAVOIR**

Une brique **absente** vaut zéro partout, y compris sur son bord de 1 voxel. Le viewer ne la télécharge pas : l'espace vide ne coûte ni octets, ni temps de transfert (chapitre 10).

6.6 Taille finale : du .ims au dossier publié



▲ Du fichier d'origine au dossier publié, pour le jeu de démonstration. Mesures en octets sur le disque (somme des fichiers).

CONTENU	TAILLE
Fichier `.ims` d'origine (compressé par Imaris)	255,8 Mo
`bricks/` (affichage 3D)	9,28 Mo
`planes/` (coupes natives, PNG)	8,64 Mo
`mips/` (projections par couche)	0,47 Mo
`thumbnail.webp` + `metadata.json`	0,04 Mo
Dossier publié (hors `download/`)	**18,4 Mo**

14× plus petit que le .ims

28× pour la seule vue 3D

46 % briques stockées

⚠️ **ATTENTION**

Le dossier `download/` (fichier d'origine, TIFF, projections…) n'est pas compté : il ne sert pas à l'affichage. Il pèse 506 Mo ici, car il contient le .ims.

📌 **À RETENIR**

- 16 → 8 bits est la **seule** perte, décidée au chapitre 5 ; tout le reste est exact.
- La pyramide : X, Y ÷ 2 à chaque niveau, Z ÷ 2 seulement tant que son voxel ≤ 1,5 × le voxel XY ; moyenne entière arrondie vers le haut ; arrêt à 128 voxels.
- Compression **WebP sans perte** : JPEG est plus petit mais abîme les valeurs et le « 0 exact ».
- Une brique est stockée seulement si son intérieur contient un voxel ≥ 1 : 358 briques stockées sur 777 possibles.

7. Le découpage en briques

- Un volume de plusieurs gigaoctets ne peut pas être chargé d'un bloc : on le découpe en **briques** (des petits cubes de 64 × 64 × 64 voxels) que le navigateur va chercher une par une.
- Chaque brique devient **une seule image WebP** (sans perte) ; les briques sont rangées dans des **paquets**, retrouvées grâce à un **index** de quelques kilo-octets.
- Deux structures annexes, `planes/` et `mips/`, rendent les coupes et les projections rapides ; le tout forme le **format de données 4**.
- Le **format 5** (section 7.11) range le même volume en une **matrice** : plusieurs résolutions × cinq compressions, plus trois copies pour couper vite.

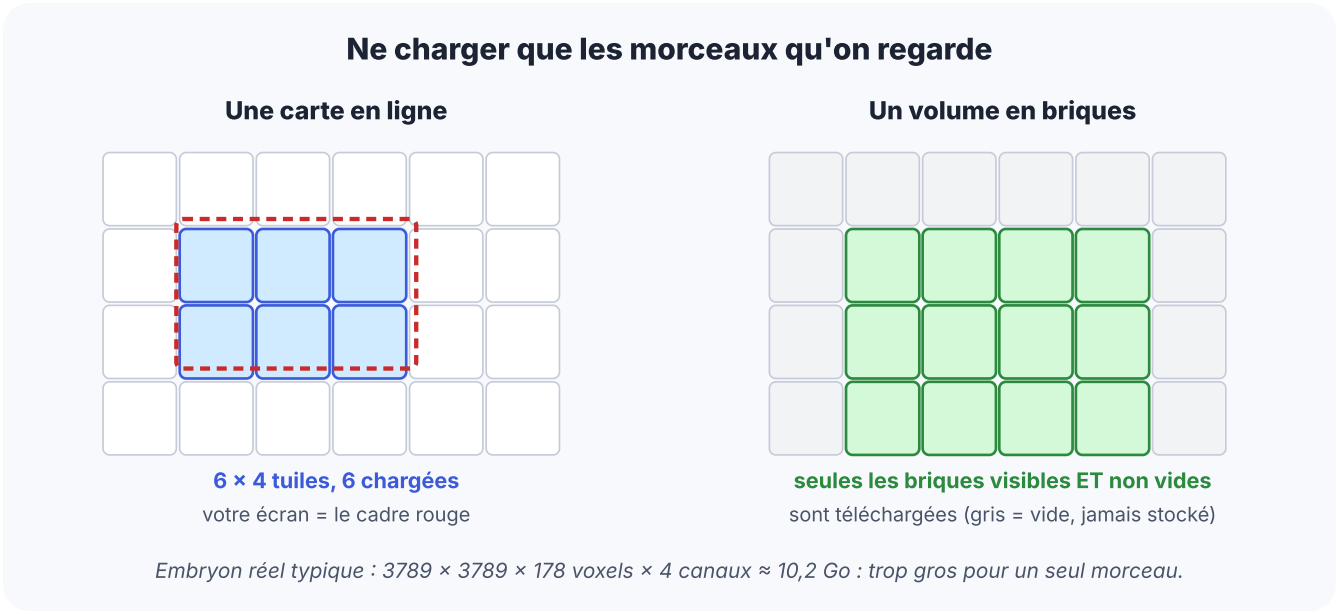
Dans ce chapitre, les images de données (mosaïque, couture, paquets…) viennent toutes du **jeu de démonstration** : l'embryon synthétique `Embryo-E95-Em2-Pecam1-Sox2` (768 × 576 × 112 voxels, 3 canaux), publié par le vrai pipeline. Ce n'est pas une donnée du laboratoire.

7.1 Pourquoi découper un volume ?

Un embryon typique du laboratoire fait 3789 × 3789 × 178 voxels sur 4 canaux : environ **10,2 Go** une fois réduit à 8 bits. Aucun navigateur n'avale cela d'un bloc, et vous n'en regardez à chaque instant qu'une petite partie.

💡 **UNE IMAGE POUR COMPRENDRE**

Une carte en ligne. Quand vous zoomez sur une ville, le site ne télécharge pas la planète : il charge seulement les **tuiles** qui sont dans votre fenêtre, à la bonne finesse. Les briques sont les tuiles de votre embryon, avec une dimension de plus.



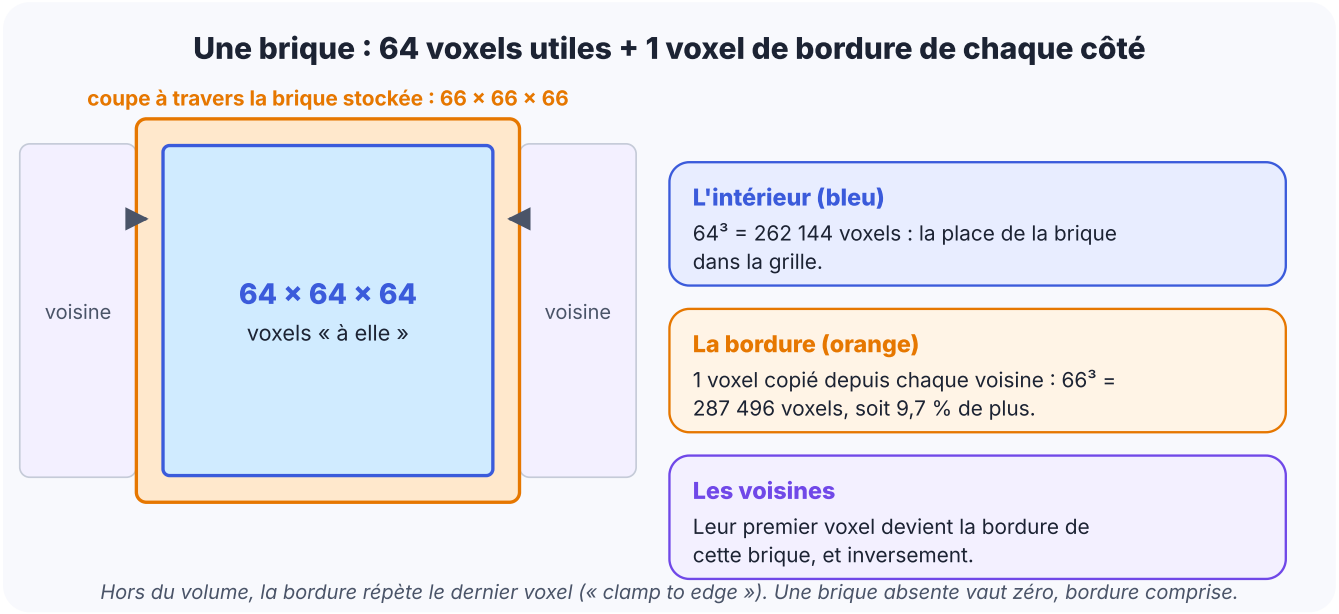
▲ Comme les tuiles d'une carte, seules les briques visibles et non vides sont téléchargées.

Trois avantages, tous visibles à l'écran :

- **le début est rapide** : les premières briques arrivent en une fraction de seconde ;
- **la mémoire est bornée** : la carte graphique ne reçoit que ce qu'elle peut loger (chapitre 10) ;
- **le vide ne coûte rien** : une brique entièrement noire n'est jamais écrite sur le disque.

7.2 Une brique : 64 voxels utiles et une bordure

Une brique couvre **64 voxels** dans chaque direction. Mais elle est stockée avec **un voxel de plus de chaque côté** : 64 + 1 + 1 = **66**. Ce voxel de bordure est une copie du voxel voisin, pris dans la brique d'à côté.

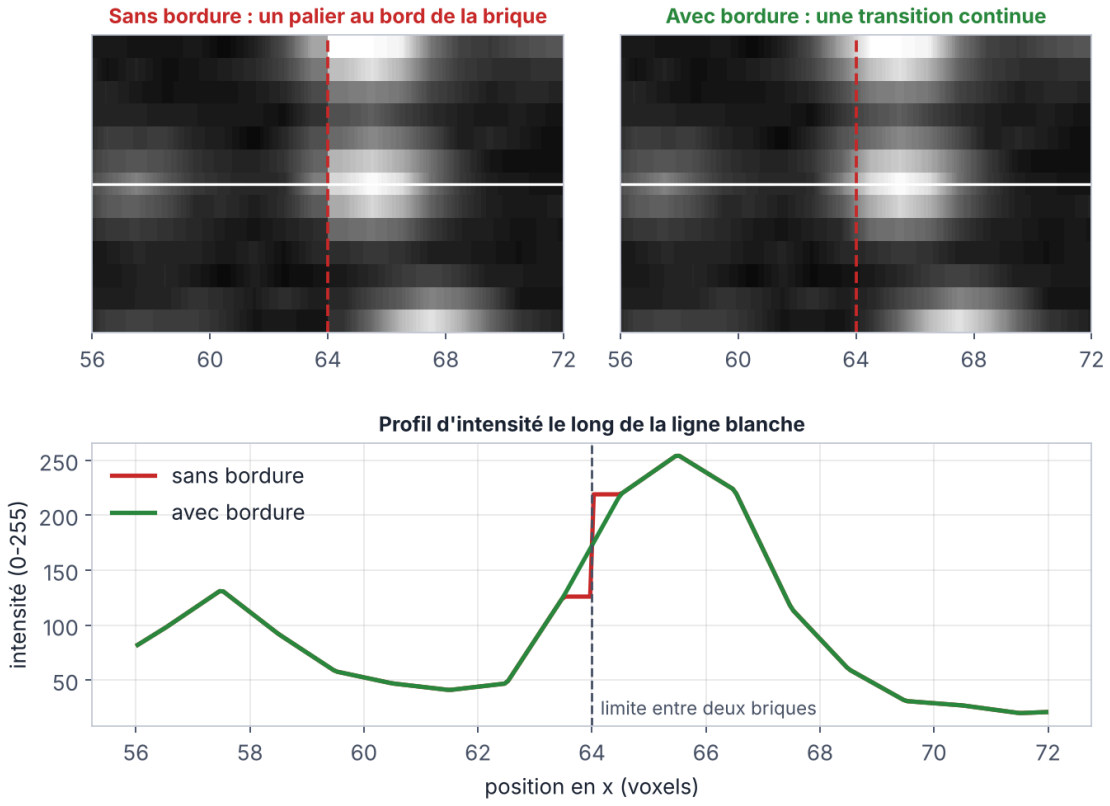


▲ Le cœur de la brique (bleu) est entouré d'une bordure d'un voxel (orange) copiée sur les briques voisines (violet).

❓ **POURQUOI CE CHOIX ?**

À quoi sert la bordure ? Pour dessiner une image lisse, la carte graphique *interpole* : la valeur à un point est un mélange pondéré des voxels qui l'entourent. Au bord d'une brique, il manque le voisin de l'autre côté. Sans bordure, la carte graphique devrait répéter le dernier voxel : on verrait un quadrillage régulier à la limite des briques. Avec la bordure, le voisin est là, et l'image est continue.

La figure suivante le montre sur de vraies données : le même profil d'intensité, à cheval sur la limite entre deux briques (en voxels 56 à 72).



▲ Sans bordure (rouge) la valeur fait un palier puis saute ; avec bordure (vert) elle varie sans coupure. Jeu de démonstration, canal DAPI.

🔢 **MICRO-EXEMPLE**

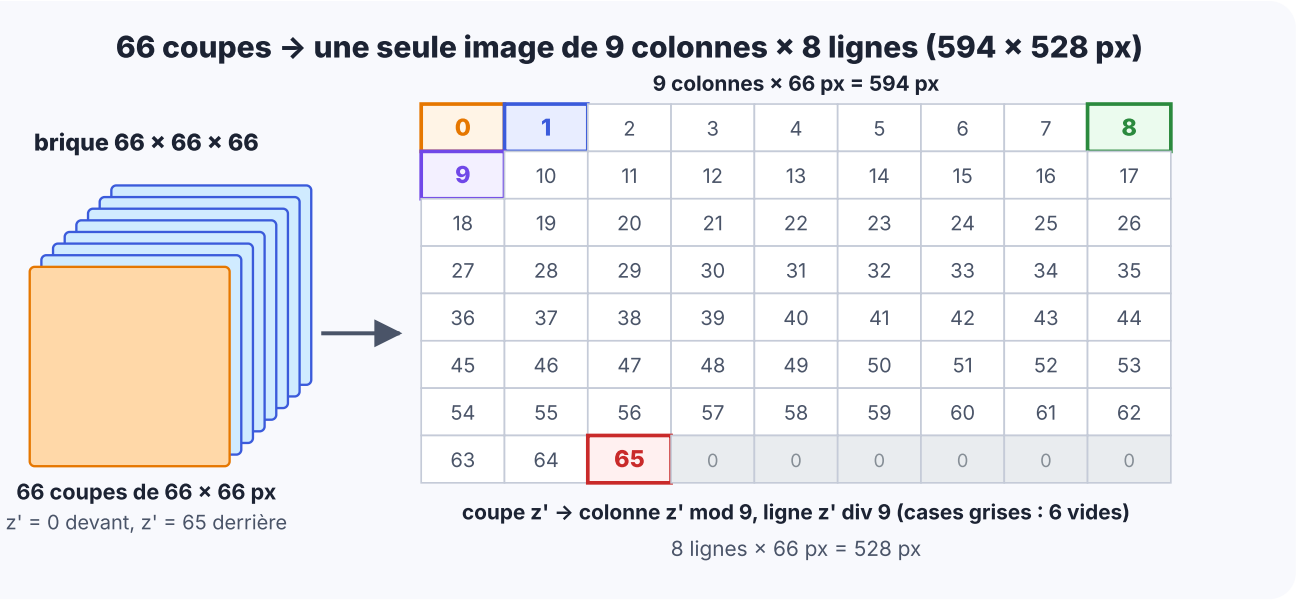
Le surcoût de la bordure. 66^3 = 287 496 voxels contre 64^3 = 262 144 : **9,7 % de plus** stockés, en échange d'images sans couture à tous les niveaux de qualité.

Deux règles pour les cas limites :

- **hors du volume**, la bordure répète le dernier voxel (« clamp to edge ») ;
- une **brique absente** (vide) vaut zéro partout, bordure comprise : on ne reconstruit jamais une bordure à partir des voisines.

7.3 D'un cube à une image : la mosaïque

Le navigateur sait décoder une image 2D très vite, mais n'a pas de format d'image 3D. L'astuce : la brique de 66 coupes est **posée à plat** en une seule image, comme une planche-contact de 66 photos.



0	1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44
45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62
63	64	65	0	0	0	0	0	0

▲ Les 66 coupes d'une brique rangées en 9 colonnes × 8 lignes ; les 6 dernières cases restent vides.

Chaque coupe fait 66 × 66 pixels ; la grille fait donc **9 × 66 = 594** pixels de large et **8 × 66 = 528** de haut, en niveaux de gris 8 bits. La règle est simple :

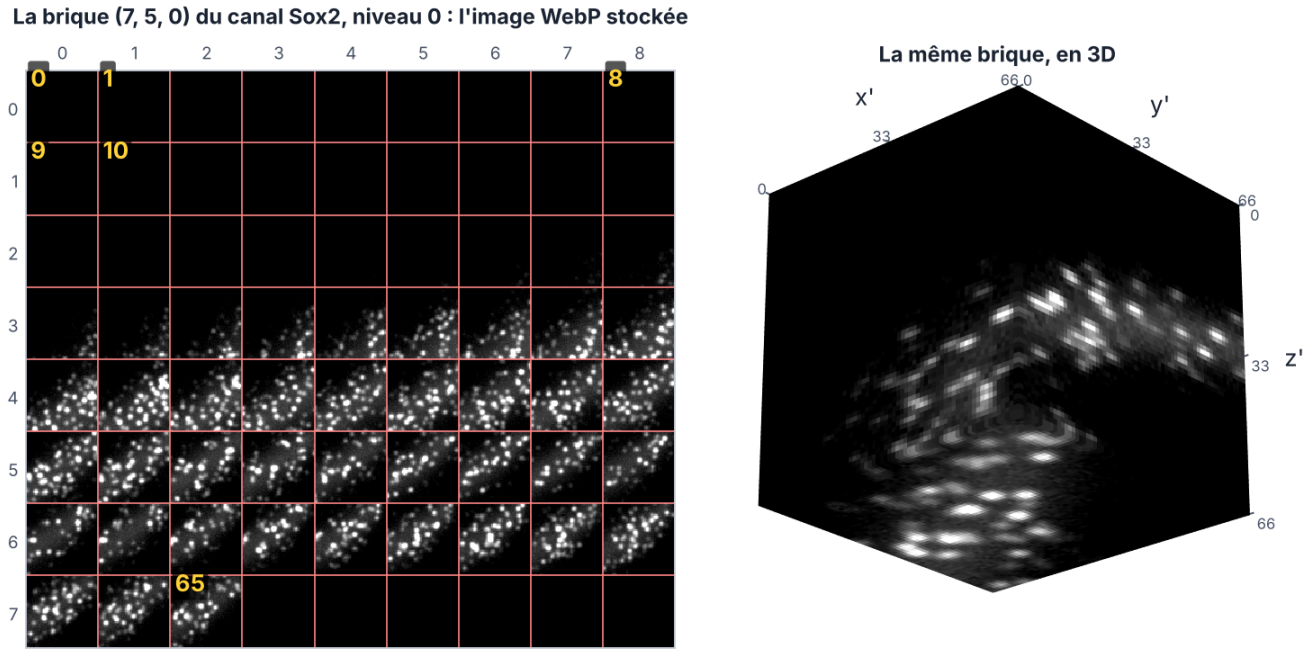
🔢 **MICRO-EXEMPLE**

Où est la coupe z' ? Colonne = z′ modulo 9, ligne = z′ divisé par 9 (division entière).

COUPE Z′	COLONNE	LIGNE	COIN HAUT-GAUCHE (PX)
0	0	0	(0 ; 0)
1	1	0	(66 ; 0)
8	8	0	(528 ; 0)
9	0	1	(0 ; 66)
65	2	7	(132 ; 462)

Pour un voxel (x′, y′, z′) de la brique de 66^3, son pixel dans l'image est : **colonne = (z′ mod 9) × 66 + x′**, **ligne = (z′ div 9) × 66 + y′**. Le voxel intérieur numéro *i* est à l'indice *i* + 1, à cause de la bordure.

Voici une vraie brique du jeu de démonstration, **telle qu'elle est stockée**, à côté de son cube en 3D.



▲ La brique (7, 5, 0) du canal Sox2 : l'image WebP décodée, avec les numéros de quelques coupes, et le cube reconstitué.

> ℹ️ **À SAVOIR**
>
> Les premières coupes sont noires parce que l'embryon de démonstration n'occupe que le bas de cette brique. Les cases 66 à 71 (les 6 dernières, en bas à droite) sont toujours vides : 72 places pour 66 coupes.

Une image par canal

Chaque canal a ses **propres** briques : la brique (7, 5, 0) existe en trois exemplaires indépendants (DAPI, Pecam1, Sox2). Le navigateur les assemble en couleur au dernier moment. Un canal peu dense, comme Pecam1 ici, donne une image beaucoup plus petite.

▲ Les trois canaux de la même brique : même géométrie, contenus et poids différents.

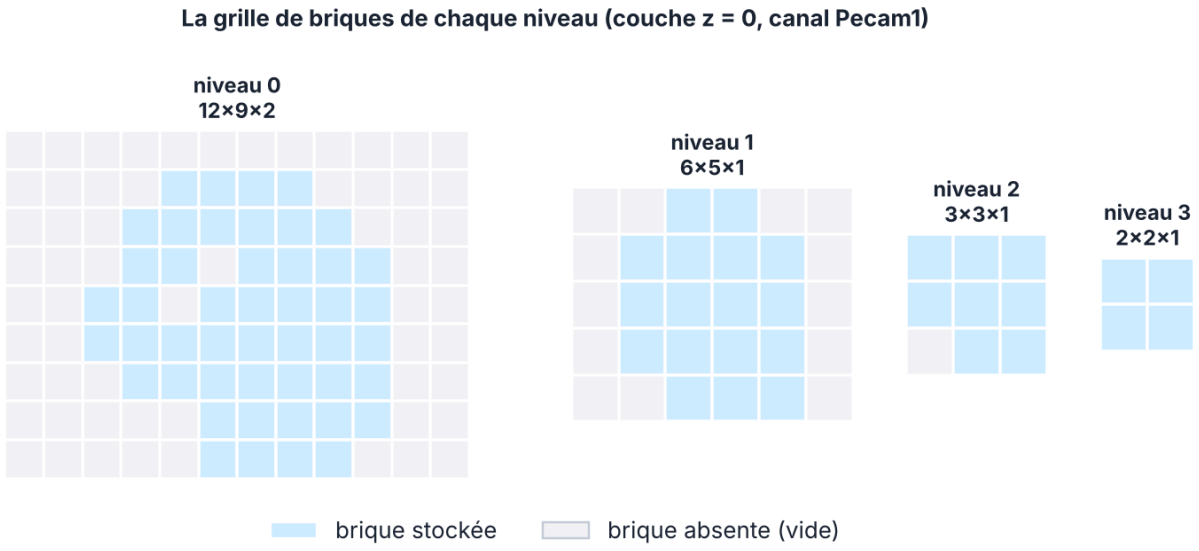
> 🔧 **POUR LES CURIEUX**
>
> **Encodage.** Chaque mosaïque est enregistrée en WebP *sans perte* (bibliothèque Pillow, `lossless=True`, qualité 75, méthode 4) : le voxel relu est identique au voxel écrit. Une brique n'est conservée que si son intérieur 64^3 contient **au moins un voxel ≥ 1** ; sinon elle est « absente » (longueur 0 dans l'index). Cette règle est calculée canal par canal.

7.4 La grille de briques de chaque niveau

Le chapitre 6 a construit la pyramide de résolutions. À chaque niveau, le volume est découpé en une **grille** de briques : on arrondit au supérieur le nombre de voxels divisé par 64.

NIVEAU	VOXELS (X, Y, Z)	GRILLE DE BRIQUES	BRIQUES STOCKÉES (3 CANAUX)
0	768 × 576 × 112	12 × 9 × 2	269
1	384 × 288 × 56	6 × 5 × 1	53
2	192 × 144 × 28	3 × 3 × 1	24
3	96 × 72 × 14	2 × 2 × 1	12

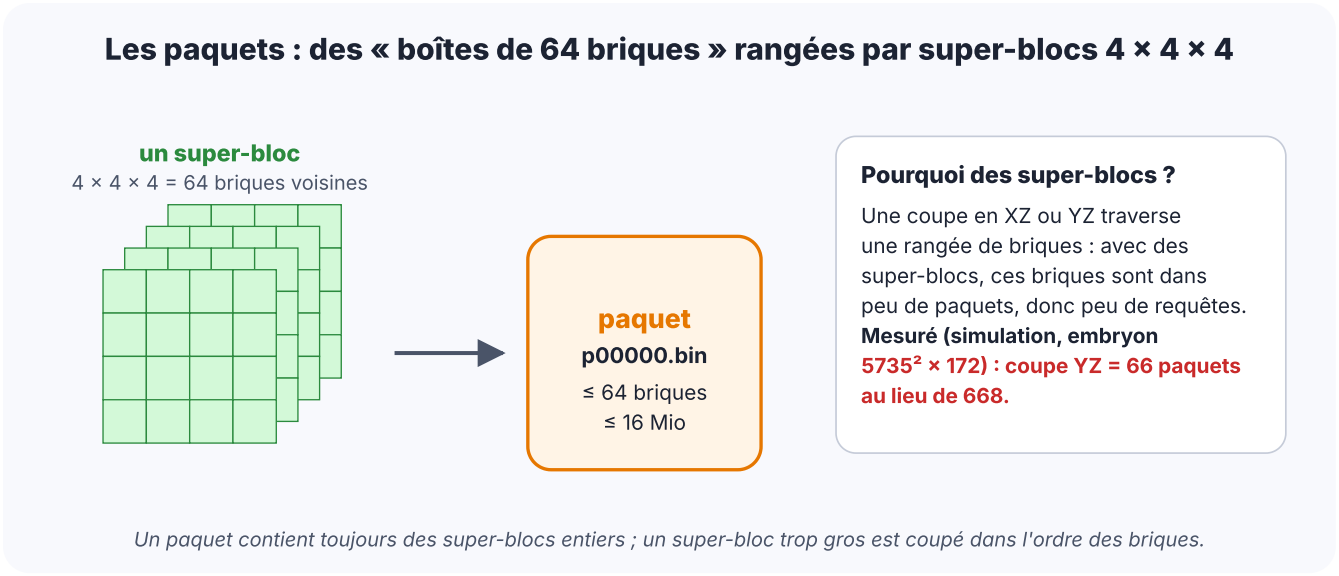
La grille du niveau 0 compte 12 × 9 × 2 = 216 cases par canal, mais seules 100 (DAPI), 90 (Pecam1) et 79 (Sox2) contiennent du signal : les autres coins sont vides.



▲ La grille de briques de chaque niveau pour une couche et un canal : en gris, les briques jamais écrites.

7.5 Les paquets : des boîtes de 64 briques

Des milliers de petits fichiers seraient lents à servir. Les briques d'un même niveau et d'un même canal sont donc regroupées dans des **paquets** : `bricks/l0/c1/p00000.bin` est le premier paquet du niveau 0, canal 1.



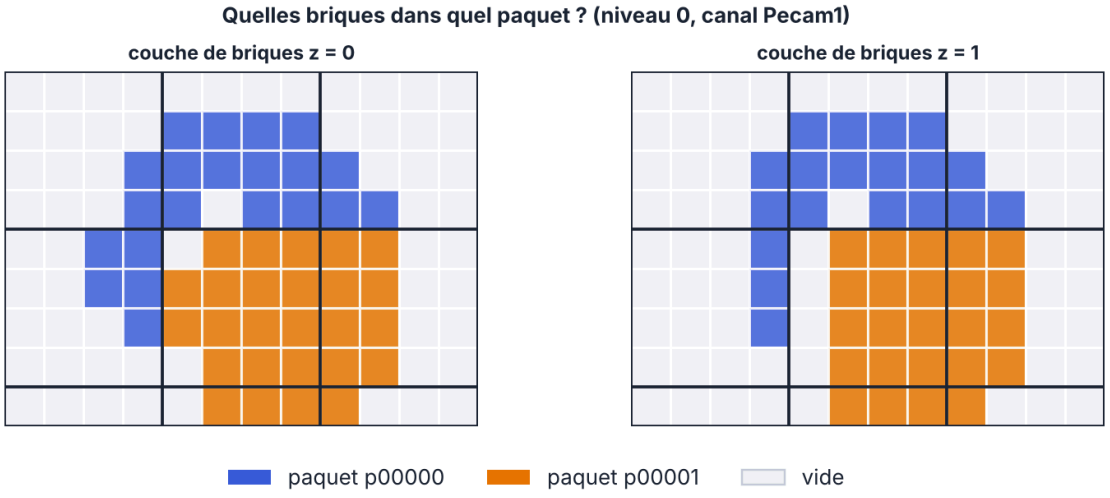
▲ Un paquet contient des super-blocs entiers de 4 × 4 × 4 briques : une coupe croise peu de paquets.

Les règles de remplissage :

- les briques sont classées par **super-blocs de 4 × 4 × 4** briques voisines ;
- un paquet contient des **super-blocs entiers**, au plus **64 briques** ou **16 Mio** ;
- si un seul super-bloc dépasse la limite, il est coupé dans l'ordre des briques.

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un déménagement.** On range les cartons pièce par pièce (les super-blocs), pas au hasard dans les camions. Quand vous cherchez « tout ce qui est dans la cuisine », il suffit d'ouvrir un camion.

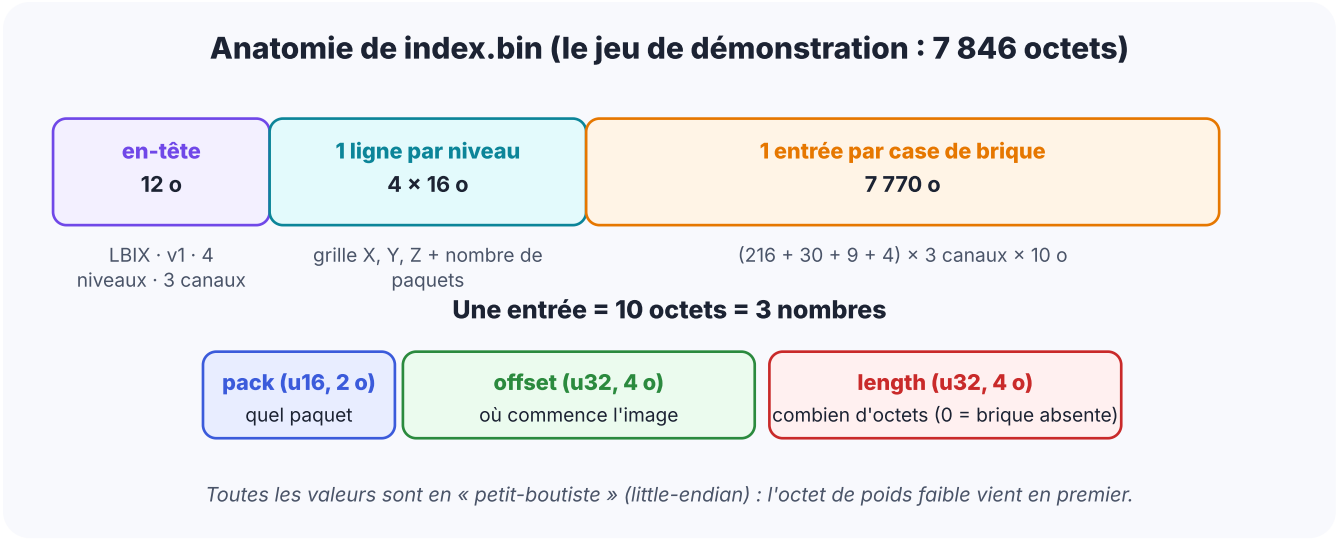
Dans le jeu de démonstration, le niveau 0 du canal Pecam1 tient dans **deux paquets** (en bleu et orange ci-dessous) : les traits noirs sont les limites des super-blocs.



▲ Quelles briques dans quel paquet, pour les deux couches du niveau 0 (canal Pecam1).

7.6 L'index et le manifeste

Pour récupérer une brique, le navigateur doit savoir **dans quel paquet** elle est, **où elle commence** et **combien elle pèse**. Ces trois nombres sont dans `bricks/index.bin`, un petit fichier binaire : **10 octets par case de brique**.



▲ Un fichier index.bin : un en-tête, une ligne par niveau, puis une entrée de 10 octets par case de brique.

Voici les octets réels du jeu de démonstration, décodés à la main : l'en-tête, puis la brique (7, 5, 0) de chaque canal.

CE QUE L'ON LIT	OCTETS (HEXADÉCIMAL)	VALEUR DÉCODÉE
début du fichier	`4c 42 49 58`	« LBIX » (la signature)
niveaux · canaux	`04 00` · `03 00`	4 niveaux · 3 canaux
niveau 0	`0c 00 00 00 09 00 00 00 02 00 00 00 06 00 00 00`	grille 12 × 9 × 2, 6 paquets
brique (7, 5, 0), DAPI	`01 00 ce d3 07 00 c0 94 02 00`	paquet 1, début 512 974, longueur 169 152
brique (7, 5, 0), Pecam1	`01 00 88 1c 01 00 a2 30 00 00`	paquet 1, début 72 840, longueur 12 450
brique (7, 5, 0), Sox2	`00 00 a0 e2 03 00 74 7a 01 00`	paquet 0, début 254 624, longueur 96 884

🔧 **POUR LES CURIEUX**

Lecture d'un nombre. Les octets sont en « petit-boutiste » : `c0 94 02 00` se lit à l'envers, `0x000294c0` = 169 152. Une longueur de **0** signifie « brique absente » (le paquet et le début valent alors aussi 0). Ce fichier fait 12 + 4 × 16 + 7 770 = **7 846 octets** ; sur un grand embryon, l'ancien manifeste JSON pesait 7,6 Mo contre 233 ko pour l'index binaire.

À côté de l'index, `bricks/manifest.json` décrit le jeu de briques en clair. Extrait réel (abrégé) :

```
{"schema":"iribhm-bricks-v3", "formatVersion":4, "channels":3,
 "brickSize":64, "apron":1,
 "brickPacking":{"mode":"grid","cols":9,"rows":8,"slice":66},
 "encoding":"webp-lossless",
 "levels":[{"level":0, "dimensions":{"x":768,"y":576,"z":112},
            "voxelSize":{"x":1.2,"y":1.2,"z":3.0},
            "gridSize":{"x":12,"y":9,"z":2}, "brickCount":269}, ...],
 "index":{"url":"index.bin", "bytes":7846, "sha256":"aea09808…"},
 "histograms":[...]}
```

La signature `sha256` de l'index est vérifiée avant usage : un index altéré est refusé. Les `histograms` servent aux curseurs de contraste (chapitres 8 et 11).

7.7 L'arbre d'un jeu de données publié

Voici le contenu réel du dossier du jeu de démonstration (abrégé ; les tailles sont celles du disque).

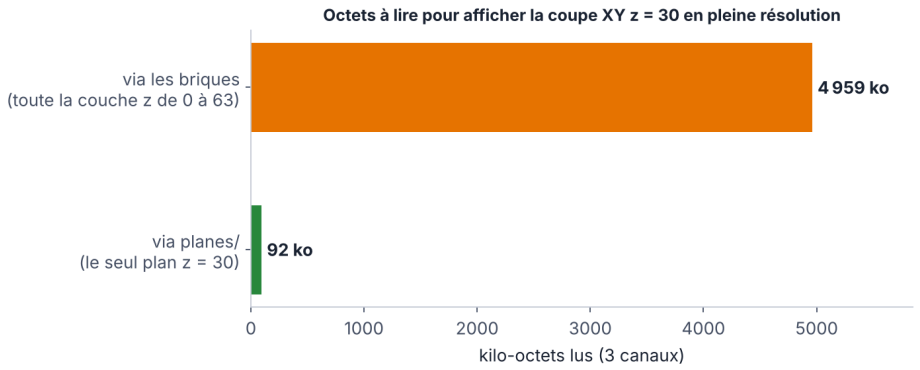
```
3d/Embryo-E95-Em2-Pecam1-Sox2/
├── metadata.json            1,8 ko   les faits du jeu de données
├── thumbnail.webp          33,6 ko   la vignette
├── bricks/                  9,3 Mo   les briques (format 4)
│   ├── manifest.json        3,7 ko
│   ├── index.bin            7,8 ko
│   ├── l0/ c0/ p00000.bin  1,4 Mo    niveau 0, canal 0
│   │       │   p00001.bin  3,9 Mo
│   │       c1/ …  c2/ …
│   └── l1/ l2/ l3/ …                 niveaux plus grossiers
├── planes/                  8,6 Mo   un fichier par plan Z
│   ├── manifest.json
│   └── z00000.bin … z00111.bin       (112 fichiers)
├── mips/                  473 ko     un fichier par couche de 64 plans
│   ├── manifest.json
│   └── l00000.bin  l00001.bin
└── download/                         (voir chapitre 8)
```

> ℹ️ **À SAVOIR**
>
> Les niveaux se répartissent ainsi : niveau 0 = 7,9 Mo, niveau 1 = 1,1 Mo, niveau 2 = 0,18 Mo, niveau 3 = 0,03 Mo. Les niveaux grossiers sont minuscules : c'est pourquoi une première image apparaît presque instantanément.

7.8 `planes/` : une coupe XY sans lire 64 plans

Les briques sont des cubes. Pour afficher **une** coupe XY à pleine résolution (par exemple pour une figure du Studio), il faudrait lire toutes les briques de la couche, soit **64 plans de données** pour n'en utiliser qu'un.

Le dossier `planes/` contient donc le niveau 0 **redécoupé plan par plan** : un fichier `zNNNNN.bin` par plan Z (`z00000.bin`, `z00001.bin` …). Chaque plan est coupé en tuiles de **512 × 512** pixels, enregistrées en PNG sans perte ; une tuile entièrement noire n'occupe aucun octet.



▲ Octets lus pour la coupe z = 30 des trois canaux : 5 078 ko par les briques contre 94 ko par planes/ (54 fois moins).

🔢 **MICRO-EXEMPLE**

Le chiffre du jeu de démonstration. La coupe XY z = 30 des trois canaux demande 5 078 326 octets via les briques de la couche 0, contre **94 053 octets** via `planes/z00030.bin`. Sur un grand embryon du laboratoire, une coupe de ce genre représentait environ 190 Mo, soit 40 secondes à 5 Mo/s.

Le résultat est **pixel pour pixel identique** à celui des briques, qui restent la solution de repli. `planes/` ne sert qu'aux coupes XY ; la vue 3D, les coupes XZ, YZ et obliques utilisent toujours les briques. Prix : environ 1,3 fois la taille du niveau natif sur le disque.

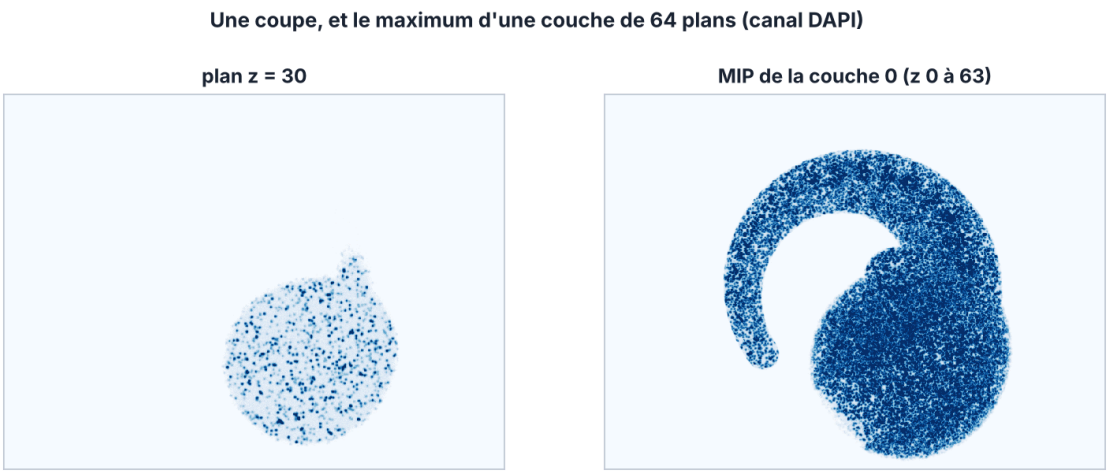
🔧 **POUR LES CURIEUX**

Un fichier plan commence par un en-tête `LPLN` (16 octets), suivi d'une table (décalage 8 octets + longueur 4 octets par tuile et par canal), puis des PNG en niveaux de gris, filtre « None », compression zlib niveau 6. Le navigateur les décode avec son propre décodeur PNG (jamais par lecture d'un canevas, qui n'est pas garantie exacte).

7.9 `mips/` : le maximum d'une couche de 64 plans

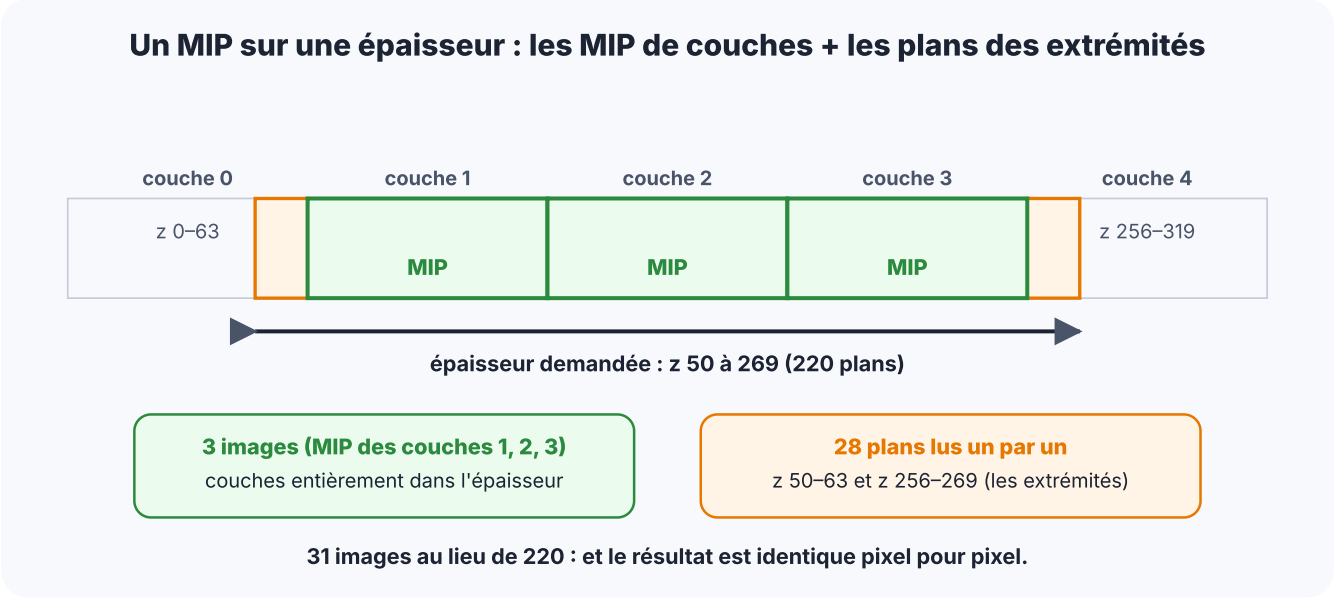
Une **projection d'intensité maximale** (MIP) garde, pour chaque pixel, la valeur la plus brillante rencontrée le long de Z. Pour un empilement épais, la calculer à la demande lirait tous les plans.

Le dossier `mips/` en garde donc une copie toute faite **pour chaque couche de 64 plans** (`l00000.bin` = plans 0 à 63, `l00001.bin` = plans 64 à 111…), à la pleine résolution XY.



▲ Une coupe (à gauche) et le MIP de la couche de 64 plans (à droite). Jeu de démonstration, canal DAPI.

Pour une épaisseur quelconque, le Studio combine : les **MIP des couches entièrement contenues** dans l'épaisseur, et les **plans individuels** des deux extrémités.



▲ Épaisseur de 220 plans : 3 MIP de couches + 28 plans d'extrémité, soit 31 images au lieu de 220, avec un résultat identique.

7.10 Les formats de données 1 à 5

Au fil des versions, l'organisation des fichiers a évolué. Chaque jeu de données porte un numéro, `formatVersion`, dans son `metadata.json`. Le format **5** est le format actuel (plateforme 1.65).

FORMAT	CE QU'IL AJOUTE	MIGRATION
1	briques seules (v2 : 64^3 sans bordure, mosaïque 8 × 8, un manifeste JSON)	—
2	`planes/` : coupes XY rapides	`m002-planes`
3	`mips/` : MIP par couche de 64 plans	`m003-layer-mips`

FORMAT	CE QU'IL AJOUTE	MIGRATION
4	briques v3 : bordure 66^3, mosaïque 9 × 8, paquets en super-blocs, `index.bin`	`m004-bricks-v3`
5	la **matrice** : des niveaux à $256 \cdot 2^k$ qui gardent toutes les tranches, **cinq compressions** par niveau, trois copies pour couper vite (section 7.11)	`m005-matrix`

Le pipeline (version 0.22.0) écrit **le format 4 par défaut**, et le format 5 sur demande (`--format 5`). Un jeu publié en format 4, ou plus ancien, n'est pas perdu : la plateforme sait le mettre à niveau, jusqu'au 5.

« Mises à jour des données »

Comme une mise à jour de logiciel, l'onglet **Mises à jour des données** du panneau d'administration (chapitre 17) détecte les jeux de données en retard et les convertit **sur place**, un par un. Le niveau natif est conservé voxel pour voxel : rien n'est recalculé à partir de l'original.

Dans ce navigateur
Votre navigateur fait le travail (décodage et encodage WebP) et envoie les fichiers au serveur. Sur un hébergement PHP, c'est **la seule** façon de passer au format 5.

Sur le serveur
Le serveur convertit lui-même, si son hébergement sait le faire (serveur Python pour le format 5).

Pause et reprise
Le travail est découpé en petites unités notées dans un journal : on peut mettre en pause, fermer, reprendre, même en changeant d'exécutant.

7.11 Le format 5 : une matrice résolution × compression

⏱ EN 30 SECONDES

- Le même volume est rangé **plusieurs fois** : à plusieurs **résolutions** (les lignes) et dans **cinq compressions** (les colonnes). Le visiteur choisit sa case.
- Les niveaux réduits **gardent toutes les tranches** ; leur grand côté vaut exactement 4096, 2048, 1024, 512 ou 256.
- Trois **copies recoupées** (XY, XZ / YZ, oblique) rendent les coupes du Studio rapides, et tout ce qui **mesure** lit toujours les voxels **sans perte**.

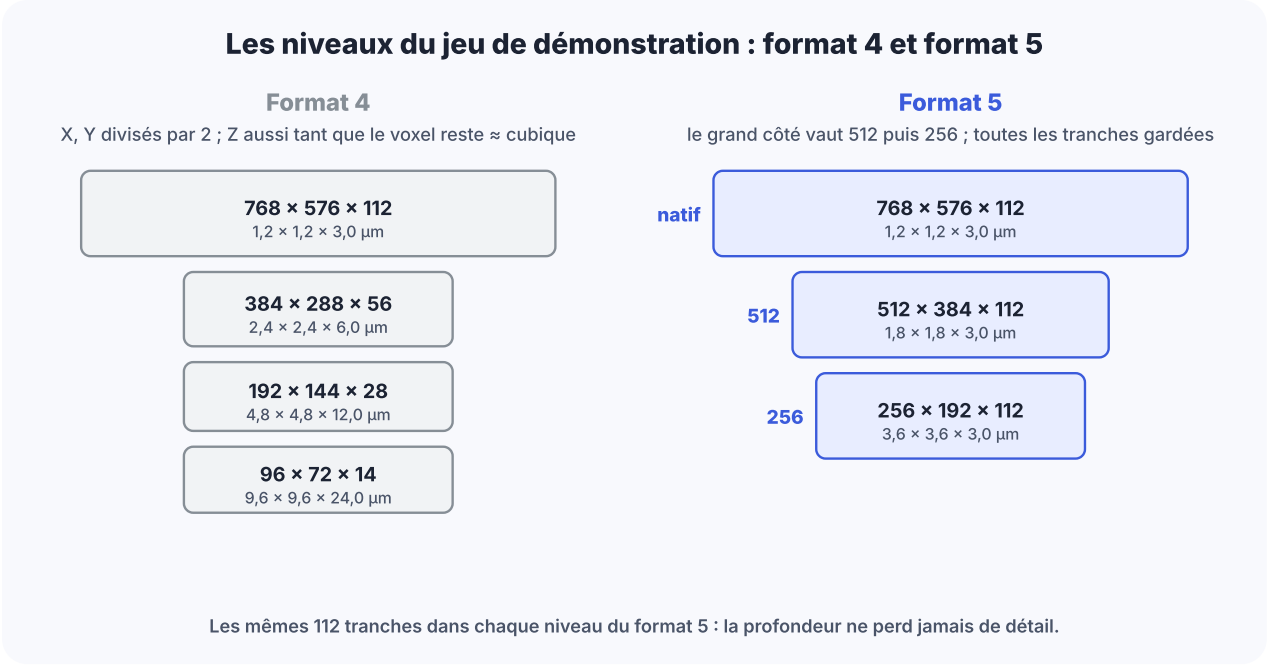
💡 UNE IMAGE POUR COMPRENDRE

Une **photothèque** garde chaque photo en plusieurs tailles et plusieurs qualités : la vignette pour feuilleter, l'original pour imprimer. Le format 5 fait de même avec un volume : on **regarde** dans une version légère, on **mesure** et on **exporte** dans l'original.

Le principe : **le disque est dépensé, le réseau et la carte graphique ne le sont pas.** Ce qui s'affiche peut être compressé avec perte, tant que l'image n'est pas visiblement altérée ; ce qui mesure lit

l'original.

Les lignes : des niveaux qui gardent toutes les tranches



▲ Les niveaux du même jeu de démonstration en format 4 et en format 5 (dimensions réelles).

- Le **niveau 0** est le volume natif, jamais rééchantillonné.
- Les autres niveaux ont pour grand côté XY **exactement** 256, 512, 1024, 2048 ou 4096 voxels : toutes ces puissances de deux plus petites que le natif sont faites. Un jeu de 5192 × 5192 × 312 voxels a donc six lignes : natif, 4096, 2048, 1024, 512, 256.
- **Z n'est jamais réduit** : chaque niveau garde les 312 tranches (ici 112). En format 4, les niveaux grossiers perdaient des tranches.
- Chaque voxel d'un niveau est la **moyenne exacte de la surface** des voxels natifs qu'il recouvre, en nombres entiers, arrondie une seule fois. Chaque niveau est calculé **directement depuis le natif**, jamais depuis un autre niveau.

🔢 **MICRO-EXEMPLE**

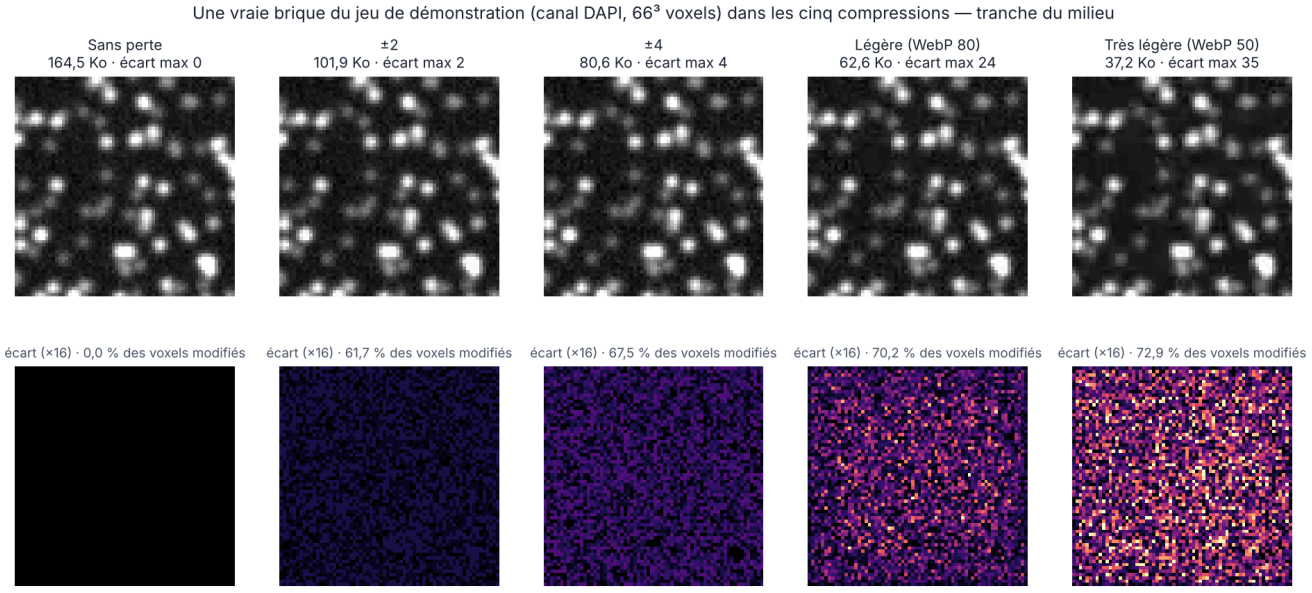
Pour passer de 768 à 512 voxels, un voxel réduit recouvre 1,5 voxel natif. Le pipeline découpe la ligne en morceaux entiers (768 × 512 unités), compte combien chaque voxel natif recouvre de chaque voxel réduit, et fait la moyenne pondérée : le voxel réduit 0 prend tout le voxel natif 0 et la moitié du voxel 1. Aucun voxel n'est oublié ni compté deux fois.

Les colonnes : cinq compressions

COLONNE	À L'ÉCRAN	CE QUI EST GARANTI	TAILLE (PAR RAPPORT À « SANS PERTE »)
Sans perte (`ll`)	Sans perte	chaque voxel exact (comme le format 4)	1
±2 (`b2`)	±2	aucun voxel ne bouge de plus de **2** sur 255	0,62 à 0,82

COLONNE	À L'ÉCRAN	CE QUI EST GARANTI	TAILLE (PAR RAPPORT À « SANS PERTE »)
±4 (`b4`)	**±4**	aucun voxel ne bouge de plus de **4**	0,50 à 0,69
Légère (`q80`)	**Légère**	WebP avec perte, qualité 80 (pas de borne ; ≈ 36 dB)	0,35 à 0,61
Très légère (`q50`)	**Très légère**	WebP avec perte, qualité 50 (≈ 31–33 dB)	0,22 à 0,40

Tailles mesurées sur 1 673 briques natives de trois embryons du laboratoire (spécification, § 14.11).

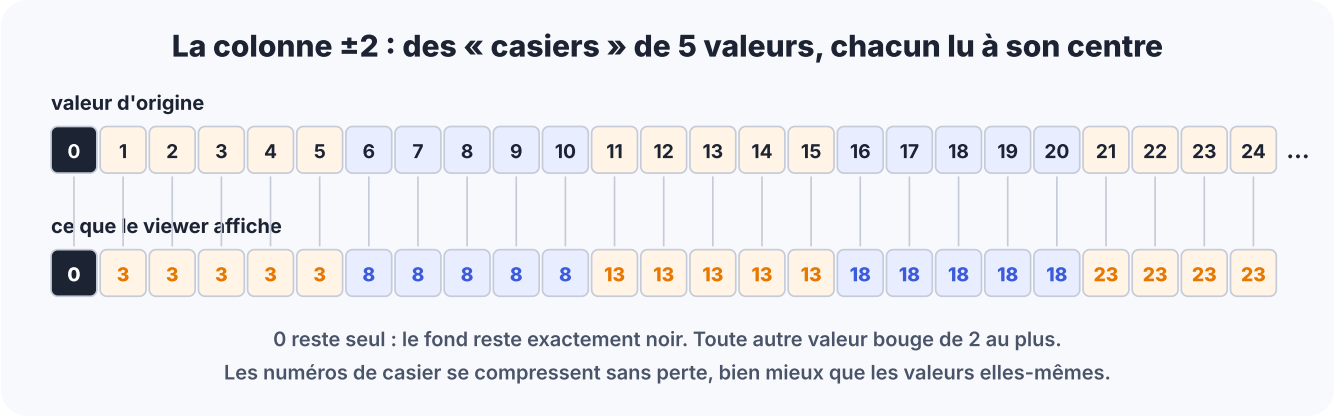


▲ La même vraie brique dans les cinq compressions : en haut la tranche du milieu, en bas l'écart avec l'original, multiplié par 16 pour être visible.

🔢 **MICRO-EXEMPLE**

Lisez la figure. Sur cette brique dense (164 Ko sans perte), ±2 pèse 102 Ko et ne change aucun voxel de plus de 2 ; « Très légère » pèse 37 Ko (4,4 fois moins) mais un voxel peut s'écarter de 35. À l'œil, les cinq tranches du haut sont presque identiques : c'est la promesse des colonnes légères. Pour mesurer, la plateforme ne les utilise **jamais** (section « Ce qui mesure lit l'original »).

±2 et ±4 : des « casiers » de valeurs

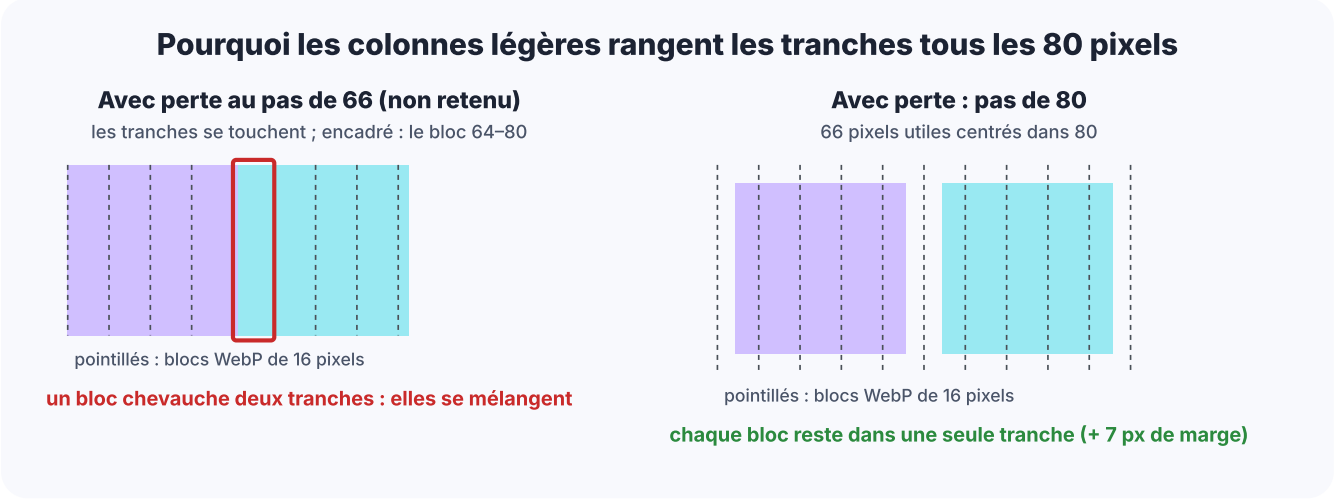


▲ La colonne ±2 : les valeurs sont rangées par casiers de 5 ; le viewer affiche le centre du casier.

- La valeur **0 reste seule** dans son casier : le fond reste exactement noir, les briques vides restent vides.
- Les autres valeurs sont groupées par 5 (±2) ou par 9 (±4) ; on stocke **le numéro du casier**, sans perte. Moins de valeurs différentes → l'image se compresse mieux.
- Le manifeste porte la table « casier → valeur » ; le viewer l'applique **en même temps** que son réglage d'affichage, sans calcul en plus.

Légère et Très légère : du WebP avec perte, mais surveillé

1. **Une zone morte.** Une compression avec perte « bave » : quelques voxels du fond noir prennent de petites valeurs. L'encodeur, qui connaît l'original, choisit un seuil τ (de 0 à 12) sous lequel tout redevient 0, et l'écrit dans **un octet** à la fin de la brique.
2. **Jamais plus lourde qu'une colonne fidèle.** Sur une brique presque vide, l'image avec perte peut peser plus que l'image sans perte. L'encodeur garde alors la plus petite des trois : l'image avec perte, l'image ±4 ou l'image sans perte. L'octet final dit laquelle (0 à 12 = τ ; 254 = ±4 ; 255 = sans perte).
3. **Une mosaïque au pas de 80.** Le WebP avec perte travaille par blocs de 16 pixels. Au pas de 66, un bloc chevaucherait deux tranches voisines et les mélangerait. Au pas de 80, chaque tranche de 66 pixels est centrée avec 7 pixels de marge : aucun bloc ne touche deux tranches.



▲ Pourquoi les colonnes avec perte rangent les tranches tous les 80 pixels.

🔢 MICRO-EXEMPLE

Mesuré sur le canal DAPI du jeu de démonstration (100 briques natives, colonne Très légère) : sans zone morte, 3,9 % des voxels noirs deviennent non nuls ; avec elle, 1,8 %. Et 12 briques sur 100 sont rangées en ±4 plutôt qu'en WebP avec perte, parce que c'était plus petit.

❓ POURQUOI CE CHOIX ?

Pourquoi pas AVIF ou JPEG XL ? AVIF donne le même compromis taille / qualité que WebP à quelques pour cent près, encode deux fois plus lentement et n'a pas d'encodeur dans les navigateurs, qui ne pourraient donc pas faire la conversion. JPEG XL n'est pas décodé par Chrome ni Firefox par défaut (octobre 2026). Mesures : spécification § 14.3.

Trois copies pour couper vite

Une coupe tirée des briques de 64^3 télécharge bien plus qu'elle n'affiche : une coupe XY d'un voxel d'épaisseur lit 64 tranches. Le format 5 ajoute, **à chaque niveau**, trois copies recoupées, toutes **sans perte** :

▲ Les trois copies de coupe, et ce qu'elles économisent.

COUPE	LUE DANS	CE QUI EST LU
XY	`planes/` (natif) ou `xy/` (niveaux réduits)	**un** plan
XZ ou YZ	`xz/` ou `yz/` : des paquets de 64 lignes recoupées	une ligne : trois petites lectures par canal

COUPE	LUE DANS	CE QUI EST LU
oblique	`cuts/` : des petites briques de 32 × 32 × *bz* voxels, sans bordure	seulement celles que le plan traverse

bz est choisi pour que la petite brique soit à peu près cubique **en micromètres** : 32 × 32 × 16 sur le jeu de démonstration (voxel 1,2 × 1,2 × 3 µm), 32 × 32 × 4 sur un embryon à 0,344 × 0,344 × 3 µm.

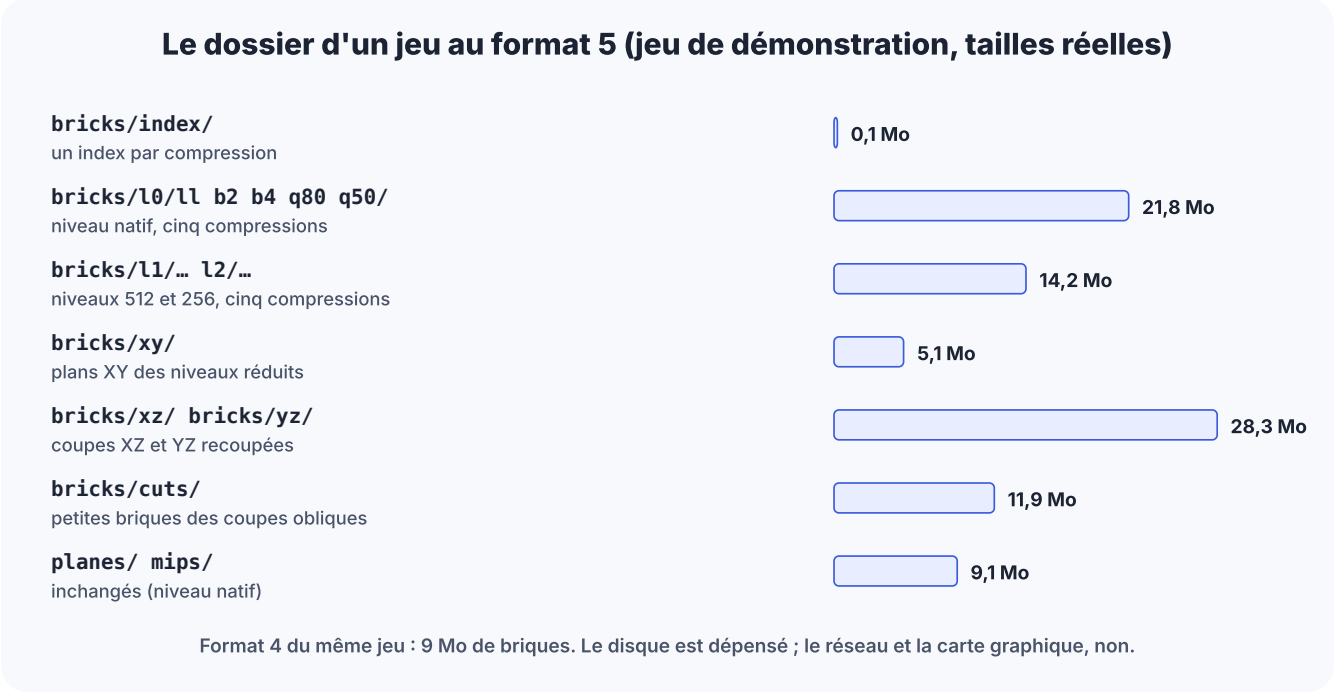
🔢 **MICRO-EXEMPLE**

Mesuré sur un embryon de 1946² × 72 voxels (plateforme 1.64) : une coupe oblique native a téléchargé **23 Mo en 15 requêtes** au lieu de 60 Mo en 30 par les briques, et chaque image est identique voxel pour voxel.

Ce qui mesure lit l'original

Toute mesure, tout plugin qui rend un nombre, et tout export « natif » lit la colonne **sans perte du niveau natif**, quelle que soit la case affichée (chapitre 12 et 15). Un réglage d'affichage léger ne fausse donc jamais un chiffre.

L'arbre d'un jeu au format 5



▲ Le dossier `bricks/` du jeu de démonstration après conversion, avec les tailles réelles.

- Tout ce que le format 5 ajoute vit **dans** `bricks/` : un seul manifeste (`iribhm-bricks-v5`) le décrit, et un seul renommage l'installe.
- `bricks/index/ll.bin`, `b2.bin`… : un index par colonne (même structure que l'`index.bin` du format 4). Le viewer lit l'index « sans perte » à l'ouverture, les autres seulement quand on choisit leur colonne.
- Les paquets de la colonne sans perte du niveau 0 **sont** ceux du format 4 : la migration les relie (lien physique) au lieu de les recoder.
- `planes/` et `mips/` ne changent pas : ils contiennent déjà le niveau natif.

⚠️ **ATTENTION**

La place disque augmente beaucoup : sur le jeu de démonstration, 9 Mo de briques en format 4 deviennent environ 80 Mo en format 5 (dont 28 Mo de copies XZ / YZ). C'est voulu : ce qui coûte, c'est ce que chaque visiteur télécharge et ce que sa carte graphique garde, et cela, le format 5 le réduit. Vérifiez l'espace libre de l'hébergement avant de convertir de gros jeux.

📌 **À RETENIR**

- Une brique = 64^3 voxels utiles + 1 voxel de bordure = **66^3**, stockée en **une image WebP sans perte** de 594 × 528 pixels (9 × 8 coupes).
- Les briques sont dans des **paquets** (super-blocs de 4 × 4 × 4) ; `index.bin` donne paquet, début et longueur de chacune (10 octets par brique).
- `planes/` accélère les coupes XY, `mips/` les projections.
- Format actuel = **5** : des niveaux à $256 \cdot 2^k$ qui gardent toutes les tranches, cinq compressions par niveau (sans perte, ±2, ±4, légère, très légère), trois copies recoupées pour les coupes ; tout ce qui mesure lit la colonne sans perte du natif.

8. La fin du pipeline : métadonnées, suivi cellulaire, publication

- Après les briques, le pipeline fabrique la **vignette**, les **histogrammes** et le fichier `metadata.json` (stade, embryon, calibration, canaux), et, pour une série temporelle, le **suivi cellulaire**.
- Le résultat est publié **« tout ou rien »** : le jeu de données déjà en ligne n'est jamais laissé à moitié remplacé, et ce que vous avez saisi à la main est conservé.
- Vous lancez tout par `RUN.bat`, puis vous copiez le dossier produit sur le serveur (FTP) ou vous le glissez dans l'onglet **Import**.

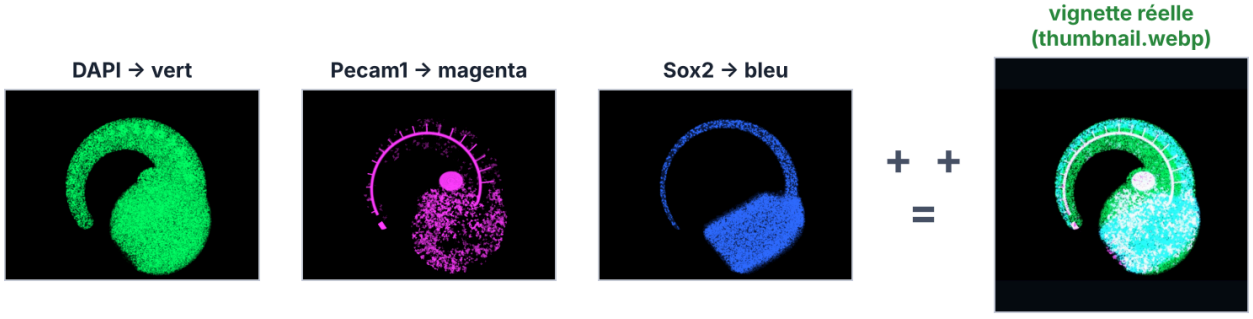
Les images de données de ce chapitre viennent du **jeu de démonstration** (embryons synthétiques générés pour cette documentation, traités par le vrai pipeline) : ce ne sont pas des données du laboratoire.

8.1 La vignette

La vignette est la petite image de la carte du jeu de données dans l'explorateur. Elle est faite à partir de la **première image** de l'acquisition, en prenant le niveau de résolution le plus fin dont le grand côté ne dépasse pas 1024 pixels.

Pour chaque canal, le pipeline calcule un **MIP** (le voxel le plus brillant sur toute l'épaisseur), le colorie, puis **additionne** les canaux : où deux canaux se chevauchent, les couleurs s'ajoutent.

La vignette : le MIP de chaque canal, colorié puis additionné



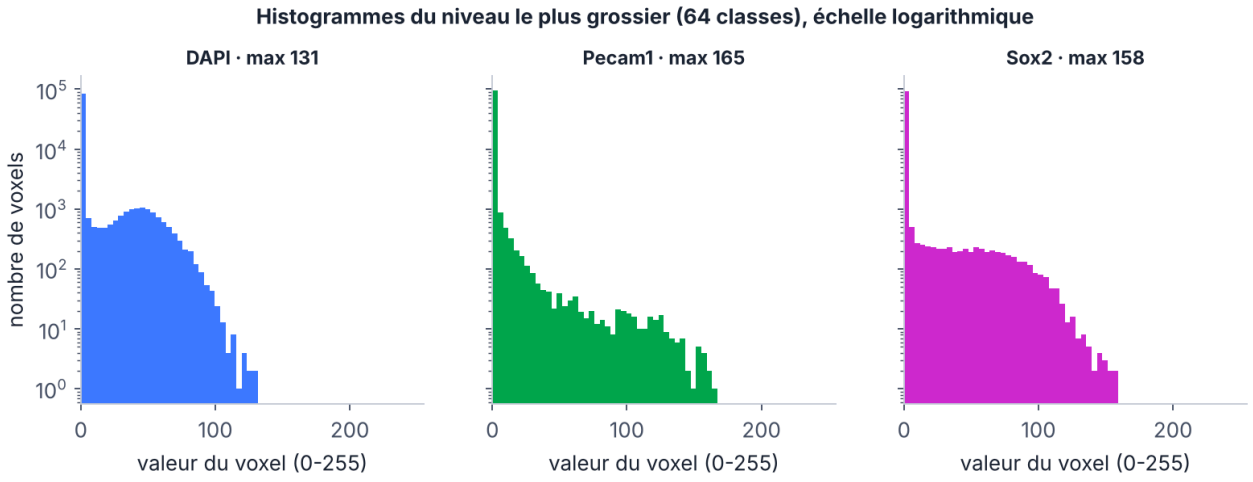
▲ Trois MIP coloriés, additionnés, donnent la vignette réelle du jeu de démonstration.

🔧 **POUR LES CURIEUX**

Couleurs de la vignette, dans l'ordre des canaux : vert, magenta, bleu, rouge, jaune, violet, cyan (rappelées en boucle). Elles n'ont pas de lien avec les couleurs choisies dans le viewer. L'image est réduite à 512 pixels pour son grand côté (interpolation de Lanczos), centrée sur un carré de fond `#080a12`, puis enregistrée en **WebP avec perte**, qualité 88 (33,6 ko ici).

8.2 Les histogrammes

Pour chaque canal, le pipeline compte combien de voxels ont chaque valeur de gris, sur le **niveau le plus grossier** (le plus rapide à lire). Les valeurs 0 à 255 sont réparties en **64 classes** de largeur 3,98.



▲ Les trois histogrammes stockés dans le manifeste du jeu de démonstration (échelle logarithmique : le fond à 0 écrase tout le reste en échelle linéaire).

Ils sont rangés dans `bricks/manifest.json` (`counts` , `edges` , `total` , `max` , `mean` , `std`) ; le viewer les affiche dans le panneau des canaux (chapitre 11). Pour une série temporelle, il y en a **un jeu par image**, car le photoblanchiment changerait l'échelle d'un jeu commun.

8.3 `metadata.json` : la carte d'identité du jeu de données

C'est le seul endroit où sont écrits les faits sur un jeu de données ; le catalogue du site est calculé à partir de lui à chaque requête. Extrait **réel** (abrégé) du jeu de démonstration :

```
{
  "id": "3d/Embryo-E95-Em2-Pecam1-Sox2",
  "formatVersion": 4,
  "type": "3d",
  "stage": "E9.5", "stageNumeric": 9.5, "embryo": "Em2",
  "dimensions": { "x": 768, "y": 576, "z": 112, "c": 3, "t": 1 },
  "voxel_size": { "x": 1.2, "y": 1.2, "z": 3.0 },
  "calibrationStatus": "exact",
  "channels": [ { "name": "DAPI", "color": "#3D7BFF",
                  "min": 0.0, "max": 1.0, "gamma": 1.0 }, ... ],
  "description": "Confocal imaging stack: E9.5 fixed embryo, ...",
  "volumeSources": [ { "kind": "bricks", "multiscale": true, ... } ],
  "hidden": false
}
```

CHAMP	CE QUE C'EST
`id` , `type`	le type (`3d` ou `live`) et le dossier : `3d/<nom>`
`formatVersion`	le format de données (chapitre 7) ; écrit par le pipeline, jamais édité à la main
`stage` , `embryo`	lus dans le nom du fichier (section suivante)

CHAMP	CE QUE C'EST
`dimensions` , `voxel_size`	taille en voxels et en micromètres, lue dans le fichier Imaris
`calibrationStatus`	`exact` si les trois tailles de voxel sont connues, sinon `metadata-missing` : aucune échelle n'est inventée
`channels`	nom, couleur, fenêtre (`min` , `max`) et gamma de départ de chaque canal
`volumeSources`	où le viewer trouve les briques

ℹ️ **À SAVOIR**

Pour une série temporelle (`live`), le fichier contient aussi `timeline` (nombre d'images, intervalle, horodatages) et `intensityNormalization` (la fenêtre de normalisation commune à toute la série, et un niveau de signal par image pour mesurer le photoblanchiment ; le viewer ne l'exploite pas encore, voir chapitre 5).

ℹ️ **À SAVOIR**

Le pipeline écrit **tous** les canaux du fichier d'origine ; le viewer, lui, n'en **affiche que les quatre premiers** (limite de la texture graphique RGBA, chapitre 11).

Stade et embryon, lus dans le nom du fichier

Le nom du fichier `.ims` est **interprété**. Le stade est un jeton `E…` encadré de tirets, de soulignés ou d'espaces ; l'embryon est un jeton `Em<n>`, ou, à défaut, le nombre qui suit directement un stade entier.

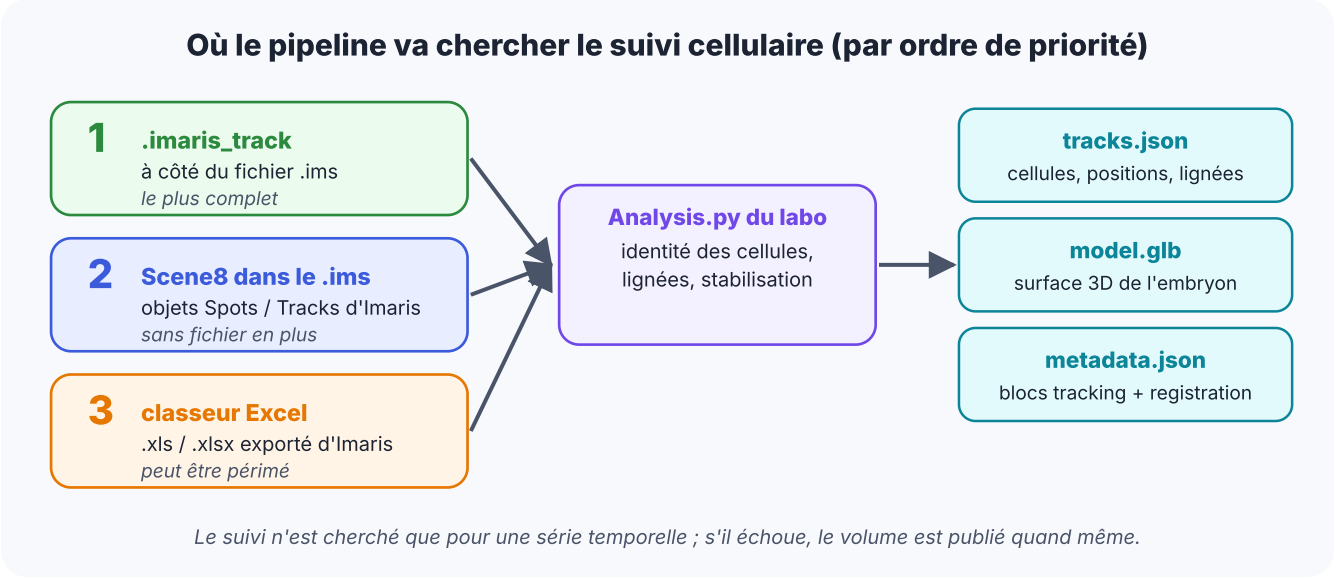
NOM (EXTRAIT)	STADE	EMBRYON	À REMARQUER
`…-E8.5-…` ou `E8,5`	E8.5	—	point ou virgule
`…-E8-5-…`	E8.5	—	tiret : confirmé par le laboratoire
`…-E85-…`	E8.5	—	forme compacte
`…-E105-…`	E10.5	—	deux chiffres de jour si ça commence par 1
`…-E95-Em2-…`	E9.5	Em2	embryon explicite
`…-E8-1-DAPI-…`	E8	Em1	`-1` n'est **pas** une fraction : après un tiret, seules .25, .5 et .75 en sont
`…-E95-1-…`	E9.5	Em1	le nombre après le stade
`Embryo-Pecam1`	inconnu	—	aucun jeton `E…`

⚠️ **ATTENTION**

Le jeton doit être encadré de tirets, de soulignés ou d'espaces : `-E85-Em1-` est lu correctement, `E85Em1` non. Si le stade est faux, corrigez-le dans l'onglet **Datasets** : le pipeline le conservera aux passages suivants (section 8.7).

8.4 Le suivi cellulaire (séries temporelles)

Pour une série temporelle seulement, le pipeline cherche une **analyse de suivi** (cellules, trajectoires, divisions) et l'attache au jeu de données. Il la cherche à trois endroits, du meilleur au moins bon.



▲ Les trois sources possibles, du fichier le plus complet au classeur le plus fragile.

❓ **POURQUOI CE CHOIX ?**

Pourquoi Scene8 passe avant Excel ? Un classeur exporté peut être périmé : sur un jeu de données réel, il contenait 155 cellules alors que le volume en comptait 172. Les objets enregistrés dans le `.ims` sont toujours à jour. Le suivi est cherché automatiquement (option `--tracking auto`) ; si quelque chose échoue, le volume est publié quand même.

Le suivi apporte trois fichiers ou blocs :

- `tracks.json` (et sa copie compressée `tracks.json.gz`) : pour chaque cellule, son identifiant, sa région, ses positions à chaque image en **µm** (stabilisées et brutes), ses marqueurs (rouge = division, noir = fusion), sa cellule-mère et ses filles ;
- `model.glb` : la **surface 3D** de l'embryon à chaque image, reconstruite à partir des cellules (au moins 4 par image) ;
- dans `metadata.json`, les blocs `tracking` (compte de cellules, régions, origine des données) et `registration` (la transformation de stabilisation, ci-dessous).

🔢 MICRO-EXEMPLE

Le jeu de démonstration en temps. 50 cellules, 4 images, 3 régions (Posterior 18, Anterior 16, Lateral 16), 1 division et 0 fusion ; `model.glb` pèse 412 ko et `tracks.json` 32,7 ko. Les positions sont numérotées à partir de 1 (comme Imaris) et recalées sur les images du volume, numérotées à partir de 0 (décalage −1).

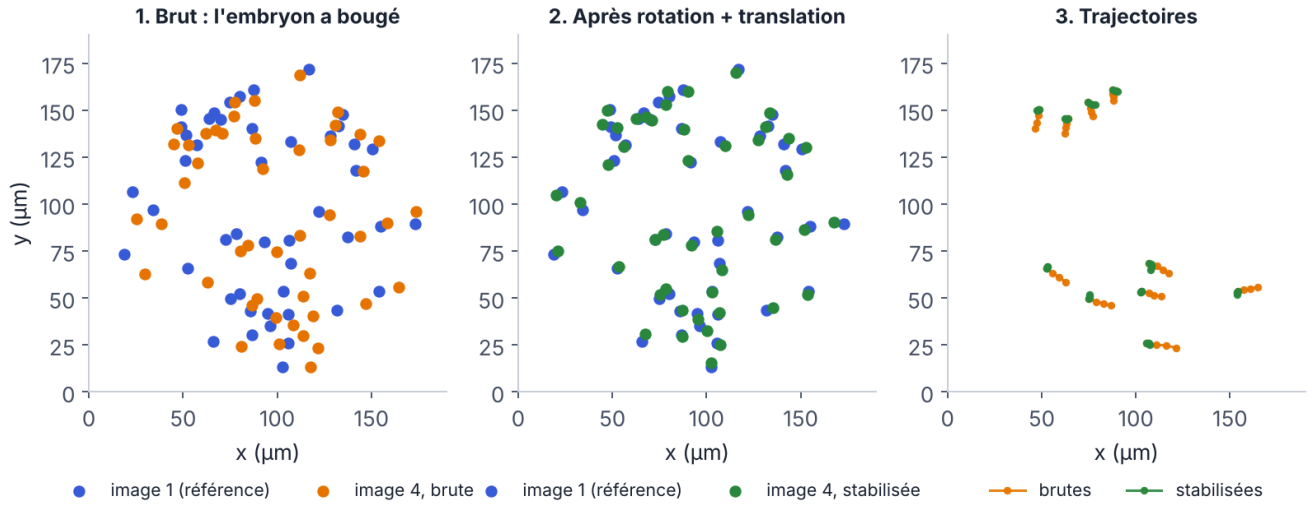
Stabiliser : annuler les mouvements de l'embryon

Pendant des heures, l'embryon dérive et tourne un peu dans le champ. Le pipeline **le remet dans la même position** à chaque image, afin que le mouvement visible soit celui des cellules et non celui de l'échantillon.

💡 UNE IMAGE POUR COMPRENDRE

Un calque transparent. Dessinez les cellules de l'image 1 sur un calque, puis celles de l'image 4 sur un autre. Pour les superposer, vous faites glisser le calque et vous le faites pivoter, mais vous ne l'étirez jamais : c'est exactement un mouvement « rigide ».

Sur le jeu de démonstration, voici la même idée avec les vraies cellules (47 cellules présentes aux images 1 et 4, projection sur le plan x-y).



▲ À gauche, les cellules de l'image 4 ont dérivé ; au centre, après rotation de 7,03 ° et translation, elles se superposent à l'image 1 ; à droite, les trajectoires brutes dérivent, les stabilisées restent en place.

L'algorithme utilisé s'appelle **Kabsch** (décomposition en valeurs singulières). Il choisit des **cellules de référence** (celles qui ne se divisent pas, présentes à au moins deux images) et cherche, image après image, la rotation et la translation qui les superposent au mieux ; il en faut au moins 3 en commun.

🔧 POUR LES CURIEUX

Les maths. Soient P les positions stabilisées de l'image n−1 et Q les positions brutes de l'image n (cellules de référence), centrées sur leurs centres de gravité c_P et c_Q. On forme $H = Q_c^T P_c$, on la décompose en $H = U S V^T$, puis $\mathbf{R = V \cdot diag(1, 1, signe(det(V U^T))) \cdot U^T}$ (une vraie rotation, jamais un miroir) et $\mathbf{t = c_P - R\, c_Q}$. La transformation (R, t) est appliquée à **toutes** les cellules de l'image n. La première image est la référence.

Le bloc `registration` : appliquer la même chose aux images

Le suivi est stabilisé, mais **pas les images**. Comme le mouvement est rigide, la même transformation permet au viewer de superposer les trajectoires aux images en déplaçant simplement les coordonnées de lecture, **sans rééchantillonner** un seul voxel. Pour chaque image, le fichier garde une matrice 4 × 4, l'angle et la translation :

IMAGE	ROTATION	TRANSLATION (MM)	RÉSIDU (MM)
1	0°	(0 ; 0 ; 0)	4×10^{-14}
2	2,33°	(−5,65 ; 5,01 ; −0,37)	6×10^{-14}
3	4,66°	(−11,09 ; 10,60 ; −1,01)	4×10^{-14}
4	7,03°	(−16,14 ; 16,39 ; −1,79)	8×10^{-14}

📌 **À RETENIR**

Test de rigidité. Le pipeline vérifie que la transformation est bien rigide : le plus grand résidu doit rester **≤ 0,05 µm**. Ici il vaut $7{,}6 \times 10^{-14}$ µm (zéro, à l'erreur d'arrondi près), donc la stabilisation est appliquée aux images. Si le test échouait, un avertissement l'indiquerait et elle ne serait pas appliquée.

8.5 Publier « tout ou rien »

Un traitement dure parfois des heures. Pendant ce temps, **le jeu de données déjà en ligne continue de fonctionner** : tout est construit dans un dossier de travail privé, au même endroit du disque que `DATA_WEB`, et n'est installé qu'à la fin.

💡 **UNE IMAGE POUR COMPRENDRE**

La vitrine d'un magasin. On prépare la nouvelle vitrine dans l'arrière-boutique ; on n'enlève l'ancienne qu'une fois la nouvelle entièrement montée. Si un problème survient, les clients voient toujours l'ancienne.

▲ Les cinq temps de la publication ; `metadata.json` est écrit en dernier, c'est le point de non-retour.

Un **nouveau** jeu de données arrive en un seul renommage de dossier. Pour un jeu **existant**, seuls les éléments du pipeline sont remplacés (`bricks/`, `planes/`, `mips/`, la vignette, et `tracks.json`, `tracks.json.gz`, `model.glb` si un nouveau suivi existe). Tout le reste du dossier, comme `download/` ou `gallery/`, n'est jamais touché.

8.6 Ce que le pipeline conserve quand on retraite un jeu de données

Vous avez corrigé un nom, orienté l'embryon, ajouté des légendes dans l'administration ? Un nouveau passage du pipeline **ne l'écrase pas** : il fusionne, juste avant l'installation, les champs saisis à la main dans le nouveau `metadata.json`.

- **Identité** : `name`, `description`, `stage`, `stageNumeric`, `embryo`, `line`, `staining`, `reporter`, `tags`, `notes`, `created` ;
- **Affichage** : `hidden`, `orientation`, `orientationAxes`, `upsideDown`, `defaultView`, `exposure` ;
- **Liens et images** : `gallery`, `linkedTrackingId`, `relatedIds`, et tout champ que le nouveau passage ne produit pas (par exemple `tracking`).

Les couleurs et réglages des canaux sont aussi conservés, tant que le **nombre de canaux** n'a pas changé. À l'inverse, `formatVersion` n'est **jamais** repris : il reflète toujours la structure réellement écrite sur le disque.

⚠️ **ATTENTION**

Si un plantage survient pendant l'installation (coupure de courant), le marqueur `.swap-in-progress` permet au passage suivant de **finir ou d'annuler** proprement le changement. Ne supprimez pas ce fichier à la main.

8.7 Le dossier `download/`

Avec l'option `--with-downloads`, le pipeline prépare, **après** la publication (un échec ici ne défait donc jamais le jeu de données), un dossier de fichiers à télécharger. Contenu réel du jeu de démonstration :

```
download/
├── Embryo-E95-Em2-Pecam1-Sox2.ims              255,8 Mo  l'original
├── Embryo-E95-Em2-Pecam1-Sox2.tif              254,6 Mo  composite ImageJ
├── Embryo-E95-Em2-Pecam1-Sox2_C1_DAPI_MIP.png    368 ko  un MIP par canal
├── …_C2_Pecam1_MIP.png  …_C3_Sox2_MIP.png
├── Embryo-E95-Em2-Pecam1-Sox2_web.zip           18,4 Mo  le jeu de données web
└── README.txt                                            provenance
```

- **L'original** `.ims` est un **lien physique** (« hard link ») : il apparaît dans `download/` sans occuper de place en plus ; il n'est jamais modifié.
- **Le** `.tif` est un composite pour ImageJ/Fiji au **format natif** (8 ou 16 bits, valeurs brutes), calibré en µm, avec les couleurs et la plage d'affichage de chaque canal. Il est tiré du niveau de résolution Imaris le plus proche de 2048 pixels.
- **Les MIP en PNG** donnent un aperçu par canal (normalisation entre les 1er et 99,9^{e} centiles).
- **Le** `_web.zip` archive le dossier servi, sans recompression (WebP et PNG le sont déjà).
- **Le** `README.txt` rappelle dimensions, tailles de voxel, canaux et comment citer le jeu de données.

Pour une série temporelle, le `.tif` et les MIP ne montrent qu'**une image** (la première) ; le `.ims` les contient toutes.

8.8 Les photographies 2D, en une demi-page

Les photographies (stéréomicroscope, TIFF exporté depuis un fichier Leica) ont leur **propre outil**, `2d_importer.py` : un fichier TIFF donne un dossier `2d/<nom>/`.

FICHIER	RÔLE	JEU DE DÉMONSTRATION
`image.webp`	l'image en taille native	1920 × 1440 px, 191 ko
`preview.webp`	aperçu de 640 px, affiché d'abord	10,6 ko
`thumbnail.webp`	vignette 512 px	12,8 ko
`metadata.json`	stade, ligne, date, taille de pixel (2,0 µm ici)	écrit en dernier

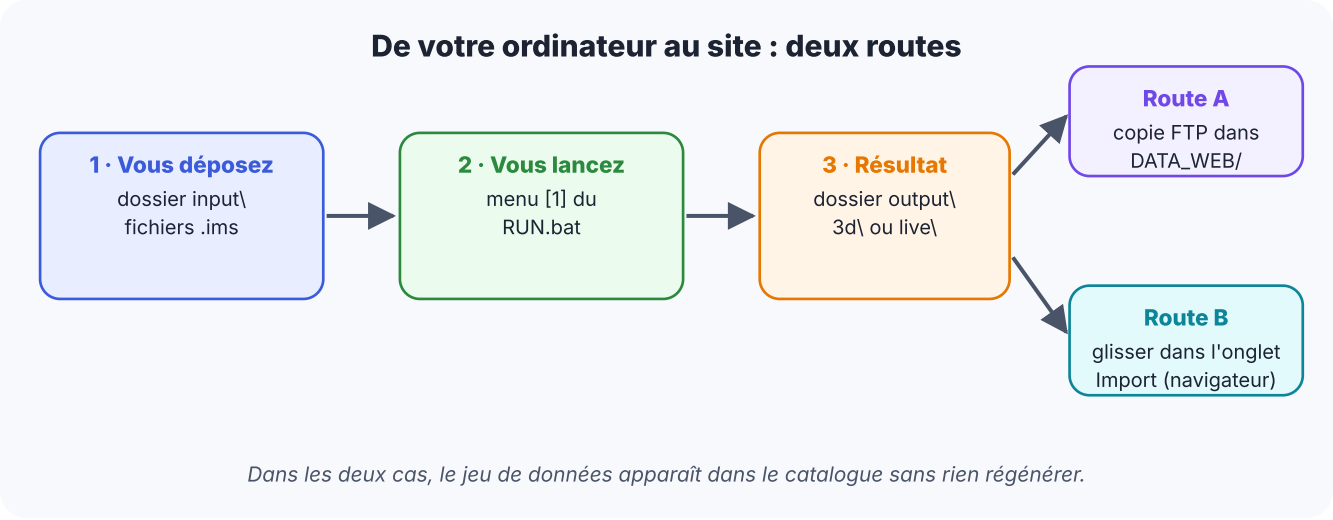
❓ **POURQUOI CE CHOIX ?**

Pourquoi WebP « avec perte » par défaut ? Il divise la taille par environ 40 par rapport au TIFF d'origine (8,3 Mo contre 191 ko ici), avec une qualité de 90. Mais le WebP avec perte **sous-échantillonne la couleur** (la chrominance est stockée avec moitié moins de pixels). Or l'isolement de la coloration X-gal du viewer lit le **rapport bleu/rouge** pixel par pixel : sur les contours nets, ce rapport est légèrement perturbé. L'option `--lossless` conserve chaque pixel exact, au prix de fichiers plus gros.

La taille de pixel n'est lue que si le TIFF déclare une unité en microns ; sinon `pixelSizeUm` reste vide et le viewer n'affiche **ni échelle ni mesure**. Le stade, le zoom et la date de dissection sont lus dans le nom du fichier, avec le même lecteur de stade que les volumes.

8.9 Lancer tout cela : `RUN.bat`

Le **pack Pipeline** (un dossier à décompresser sur un PC Windows) contient tout. Double-cliquez sur `RUN.bat` : il vérifie l'intégrité du pack (empreintes SHA-256), prépare Python et les bibliothèques, puis affiche un menu.



▲ De votre ordinateur au site : on dépose les fichiers, on lance, puis on copie ou on glisse le résultat.

CHOIX	CE QUE FAIT LE MENU
`[1]`	Preprocessing de volumes Imaris (.ims → `output\`, suivi inclus)
`[2]`	Analyse de tracking Imaris (Excel → `tracking\OUTPUT\`)
`[3]`	Import de photographies 2D (.tif → `output\2d\`)
`[4]`	Attacher un tracking à un dataset déjà traité
`[5]`	Vérifier l'environnement seulement
`[0]`	Quitter

Le menu pose quelques questions (dossier des `.ims`, dossier de sortie, archives de téléchargement ou non) puis lance `run_preprocess.py`. Vous pouvez aussi l'appeler vous-même :

```
python run_preprocess.py --input input --output DATA_WEB --only "*E95*" --with-downloads
```

OPTION	EFFET
`--input`	dossier des `.ims` (non récursif, un fichier à la fois)
`--output`	la racine `DATA_WEB` : le résultat va dans `3d/<nom>/` ou `live/<nom>/`
`--only`	filtre sur le nom des fichiers (ex. `"*E8*"`)
`--with-downloads`	prépare aussi le dossier `download/` (plus long)

OPTION	EFFET
`--tracking`	`auto` (défaut), `off`, ou le chemin d'un fichier de suivi

Le type est décidé tout seul : plus d'une image dans le fichier, c'est `live`, sinon `3d`. `Ctrl+C` demande confirmation avant d'arrêter, et nettoie les fichiers temporaires.

Comment le résultat arrive sur le serveur

- **Route A, par FTP** : copiez le dossier produit (`3d\<nom>` ou `live\<nom>`) dans le `DATA_WEB` du serveur. Il apparaît aussitôt dans le catalogue, il n'y a rien à régénérer.
- **Route B, sans FTP** : dans le panneau d'administration, ouvrez l'onglet **Import** et glissez le dossier dans la zone **Glissez un dossier ici**. Le transfert reprend là où il s'est arrêté si vous le relancez ; ensuite, cliquez sur **Publier**.

> ⚠️ **ATTENTION**
>
> Un jeu de données publié par l'onglet **Import** est **masqué par défaut** : allez dans l'onglet **Datasets** et activez sa visibilité pour qu'il apparaisse dans l'explorateur public (chapitre 14).

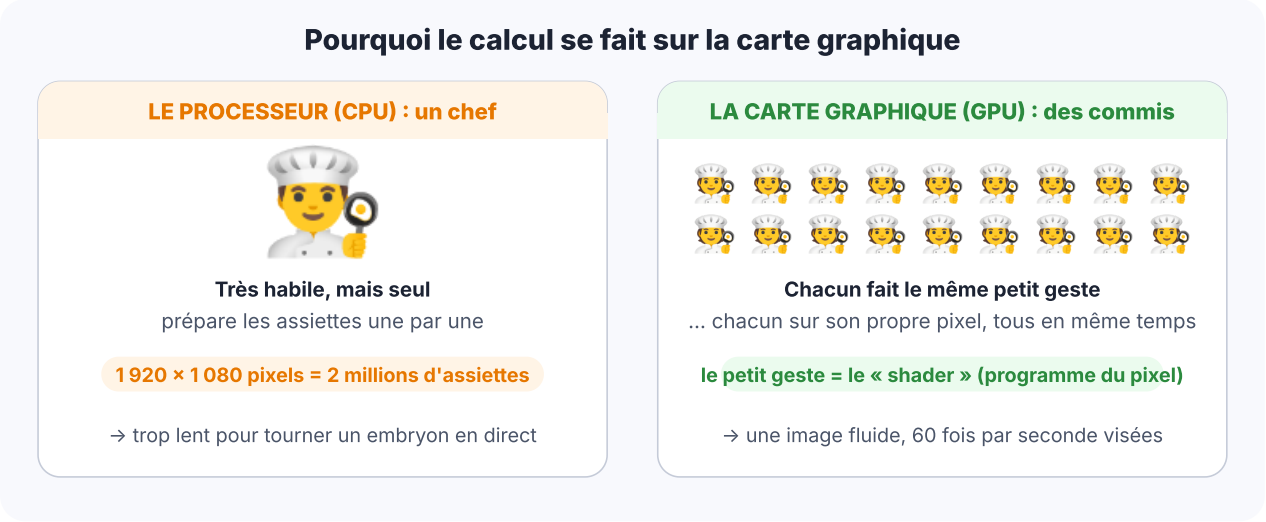
> 📌 **À RETENIR**
>
> - La vignette, les histogrammes et `metadata.json` décrivent le jeu de données ; le suivi cellulaire est une **couche** d'une série temporelle, stabilisée par un mouvement rigide.
> - La publication est **tout ou rien**, et ce que vous avez corrigé à la main est conservé aux retraitements.
> - `RUN.bat` → choix `[1]` → dossier `output\` → FTP ou onglet **Import**.

9. Le viewer 3D : Three.js et le lancer de rayons

- Chaque pixel de l'écran est calculé par **un rayon** qui traverse le volume : c'est le **lancer de rayons**, exécuté par la carte graphique.
- **Three.js** fournit la scène, la caméra et les textures ; le calcul qui compte pour la science (les trois modes de rendu) est **écrit sur mesure** et reste lisible.
- Pendant que vous tournez l'embryon, la netteté baisse un instant pour garder la fluidité ; au repos, tout revient à pleine qualité.

9.1 Deux cerveaux : le processeur et la carte graphique

Votre ordinateur possède deux types de « cerveaux ». Le **processeur** (CPU) est généraliste. La **carte graphique** (GPU) est spécialisée dans une seule chose : faire le même petit calcul sur des milliers de points à la fois.



▲ Un grand chef contre des milliers de commis : la carte graphique gagne dès qu'il faut répéter le même geste des millions de fois.

💡 **UNE IMAGE POUR COMPRENDRE**

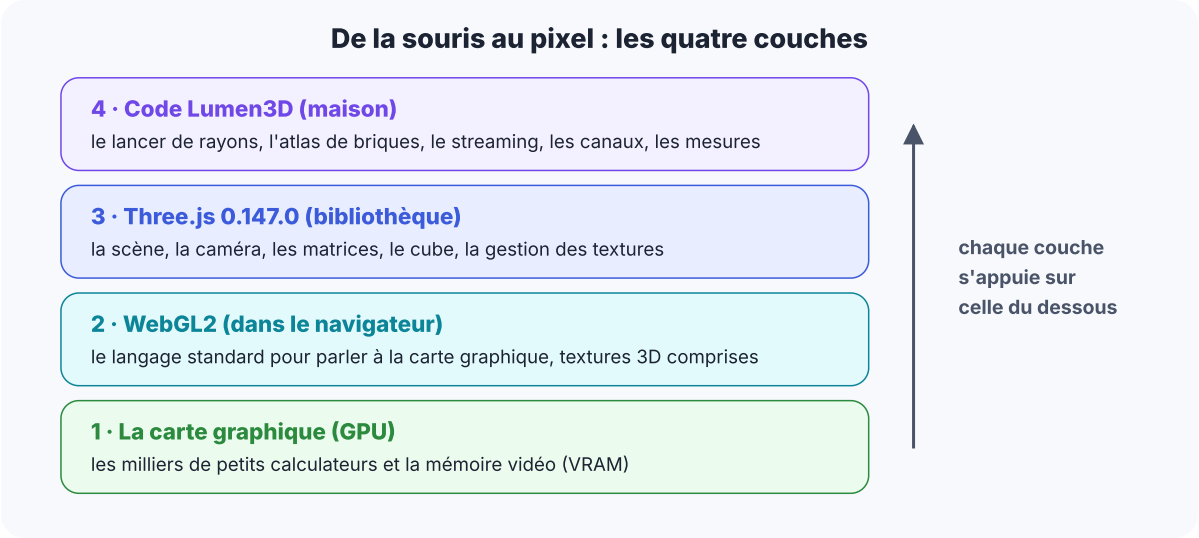
Un grand chef étoilé prépare une assiette parfaite en deux minutes. Pour servir 2 millions de convives, il lui faudrait des années. Mille commis, chacun avec **la même recette simple**, servent la salle en un instant. Un écran de 1 920 × 1 080 pixels, c'est exactement cette salle.

Dans Lumen3D, la « recette » s'appelle un **shader** : un petit programme écrit une fois, exécuté simultanément pour chaque pixel. Tout le calcul du volume se passe là.

9.2 WebGL2 et Three.js : qui fait quoi

WebGL2 est le langage standard que les navigateurs comprennent pour parler à la carte graphique. Il sait notamment manipuler des **textures 3D** : des blocs de voxels stockés directement dans la mémoire vidéo. C'est indispensable pour un volume.

Three.js (version **0.147.0**) est une bibliothèque qui se place au-dessus de WebGL2 et s'occupe de la « plomberie ». Elle est **hébergée sur le serveur de la plateforme** (dossier `js/vendor/`), pas téléchargée depuis Internet.



▲ Les quatre couches entre votre souris et le pixel affiché.

FOURNIS PAR THREE.JS	ÉCRIT SUR MESURE POUR LUMEN3D
Le moteur de dessin (le « renderer »), la scène	Les **shaders** du lancer de rayons (3 modes de rendu)
La caméra en perspective (angle de vue 45°)	L'**atlas** de briques et sa table des pages
Le cube (une boîte à 6 faces qui sert de point de départ aux rayons)	Le **streaming** des briques (chapitre 10)
Les matrices, quaternions, vecteurs	La qualité adaptative et le budget de mémoire
L'enveloppe des textures 3D	La lecture de profondeur sous la souris (mesures)
Le chargeur de modèles 3D (surface du suivi cellulaire)	Les canaux, histogrammes, filtre gaussien

❓ **POURQUOI CE CHOIX ?**

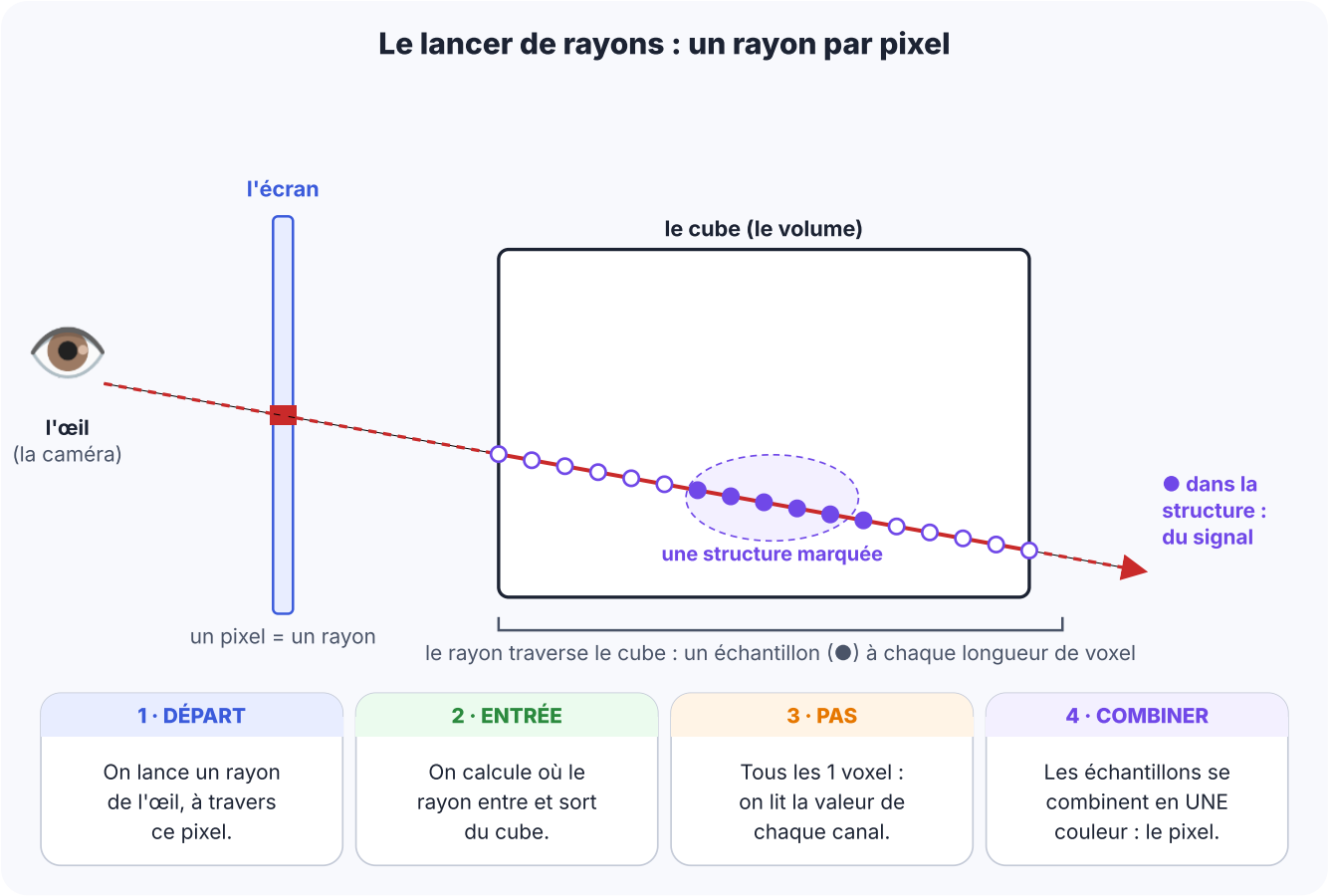
- **Mûr et léger** : Three.js est une bibliothèque très répandue, qui ne demande aucun outil d'assemblage (pas de « build ») : un fichier, un navigateur.
- **Hors ligne** : copiée sur le serveur, elle fonctionne sans Internet.
- **Auditable** : la partie qui détermine ce que vous voyez (les formules de rendu) est du code du laboratoire, commenté ligne à ligne. Un moteur de jeu complet n'était pas nécessaire : il aurait apporté des milliers de fonctions inutiles à la microscopie.

ℹ️ **À SAVOIR**

Le viewer exige **WebGL2** (il n'existe pas de version dégradée en WebGL1). Si la carte graphique ne sait pas créer de textures 3D, le chargement est refusé avec un message clair, jamais par un plantage de l'onglet.

9.3 Le lancer de rayons, pas à pas

Le volume est un énorme tableau de valeurs. Pour en faire une image, on procède **pixel par pixel**. Chaque pixel de l'écran lance un **rayon** depuis l'œil, qui traverse le cube. On lit le volume le long de ce rayon, puis on résume ce qu'on a rencontré en une seule couleur.



▲ Un rayon par pixel : entrée dans le cube, un échantillon par voxel, puis combinaison.

1. **Départ.** Le pixel désigne une direction : celle qui part de l'œil et passe par ce pixel.
2. **Entrée et sortie.** Le shader calcule où le rayon entre dans le cube et où il en sort (c'est « l'intersection rayon / boîte »). Hors du cube, le pixel reste transparent.
3. **Pas régulier.** On avance d'**un voxel** à chaque fois et on lit la valeur de chaque canal.
4. **Combinaison.** Les lectures sont résumées en une couleur, selon le mode de rendu (section 9.6).

Un échantillon par voxel

Au repos, le viewer prend **exactement un échantillon par longueur de voxel**, quelle que soit la direction du rayon. C'est ce qui garantit qu'**une structure d'un seul voxel ne passe jamais à travers**

les mailles du filet.

> 🔢 **MICRO-EXEMPLE**
>
> Le jeu de démonstration E9.5 fait 768 × 576 × 112 voxels. Un rayon qui traverse tout le cube dans la longueur (axe X) fait donc **768 échantillons**. Un rayon en diagonale, plus long ($\sqrt{768^2 + 576^2 + 112^2} \approx 966$ voxels), en fait **environ 966**.

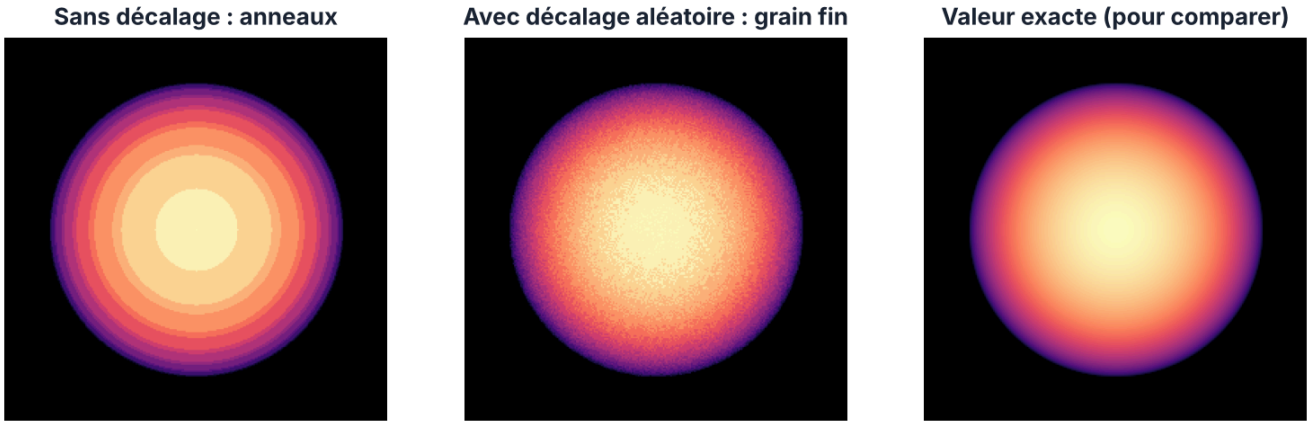
> 🔧 **POUR LES CURIEUX**
>
> Le pas (en unités du cube) est `delta = 1 / (taux × nu)`, où `nu = ‖direction × (nombre de voxels du niveau)‖` mesure combien de voxels le rayon traverse par unité de longueur et `taux = 1,0` au repos. Le pas est donc uniforme en voxels, pas en micromètres. Si le rayon demandait plus de 4 096 pas, ils sont répartis plus largement (voir 9.9).

9.4 Le grain aléatoire contre les bandes

Avec un pas fixe, tous les rayons voisins échantillonnent à la **même profondeur**. Résultat : des **anneaux** (« banding »), comme les courbes de niveau d'une carte. Ils n'existent pas dans l'échantillon : ils sont créés par le calcul.

La parade est simple : le **premier échantillon de chaque rayon est décalé** d'une fraction aléatoire du pas (un nombre pseudo-aléatoire calculé à partir de la position du pixel). Les anneaux se transforment en un **grain fin**, que l'œil moyenne et ignore.

Une sphère uniforme, un échantillon tous les 14 pixels (simulation)

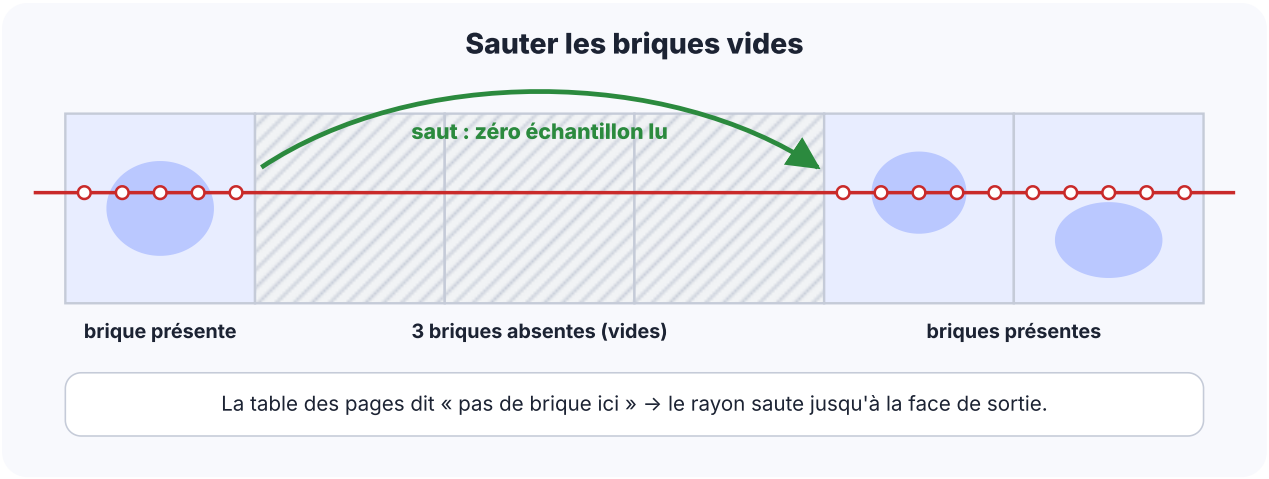


▲ Simulation : sphère uniforme, un échantillon tous les 14 pixels. À gauche, les anneaux ; au milieu, le grain après décalage aléatoire.

Le même décalage est utilisé à l'export d'une image : il se poursuit d'une tuile à l'autre, sans couture.

9.5 Sauter le vide

Un embryon n'occupe qu'une partie de sa boîte. Les briques sans signal ne sont même pas stockées (chapitre 7). Quand un rayon arrive dans une brique **absente**, il n'y lit rien : il **saute directement** à la face de sortie de cette brique.



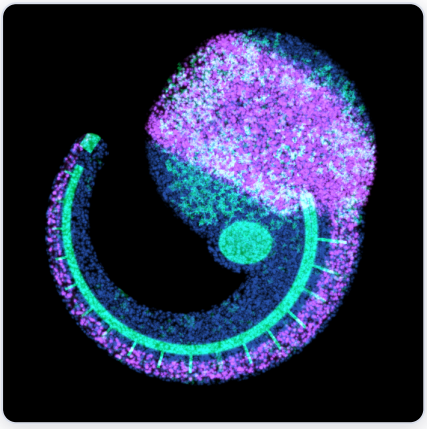
▲ Le rayon franchit trois briques vides sans lire un seul voxel.

Le saut respecte le même quadrillage aléatoire que le reste du rayon : l'image ne change pas, elle est seulement calculée plus vite.

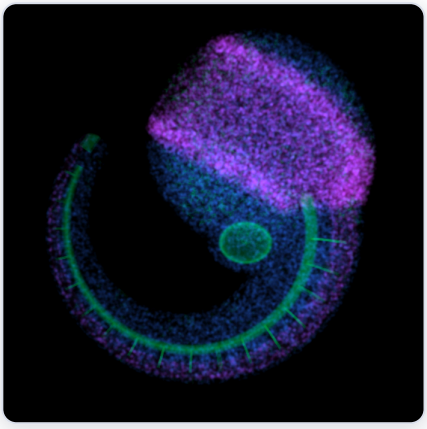
> ℹ️ **À SAVOIR**
>
> Une brique absente se lit comme des **zéros** : du noir qui ne coûte presque rien, pas un trou dans l'espace.

9.6 Les trois modes de rendu

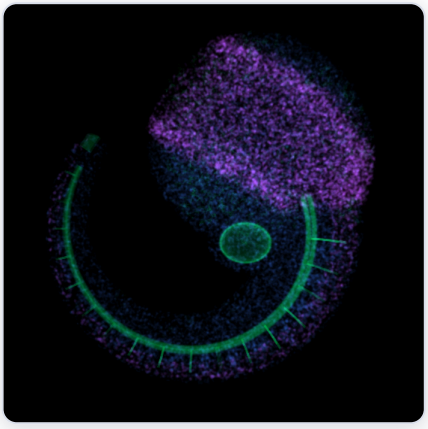
Le menu **Mode de rendu** (panneau de gauche) propose trois façons de résumer les valeurs d'un rayon. Voici **le même point de vue** du jeu de démonstration E9.5 (DAPI, Pecam1, Sox2), rendu trois fois.



▲ **Fluorescence (type Imaris)**



▲ **Fluorescence naturelle (profondeur)**



▲ **Structure (DVR)**

> ℹ️ **À SAVOIR**
>
> Les captures du milieu et de droite sont éclairées avec un curseur **Visibilité (exposition)** plus haut (voir la légende de la figure suivante) : ces deux modes produisent naturellement une image plus sombre à exposition égale.

MODE	CE QU'IL FAIT LE LONG DU RAYON	PROFONDEUR ?
Fluorescence (mode par défaut)	garde la valeur **la plus forte** de chaque canal	non, tout se superpose
Fluorescence naturelle	chaque point **brille** ; ce qui est devant **assombrit** ce qui est derrière	oui
Structure (DVR)	empile des couches **semi-transparentes** de l'avant vers l'arrière	oui

Mode 1 : Fluorescence (le maximum)

C'est le rendu « classique » des logiciels de microscopie. Pour **chaque canal séparément**, le viewer retient la **valeur maximale** rencontrée sur le rayon. Les couleurs des canaux sont ensuite **additionnées**.

🔢 **MICRO-EXEMPLE**

Un rayon croise du vert à 0,85 puis du magenta à 0,75. Le pixel reçoit 0,85 de vert **et** 0,75 de magenta : le magenta n'est pas caché par le vert qui est devant. Vert + magenta additionnés donnent du blanc là où les deux sont forts.

🔧 **POUR LES CURIEUX**

`mip = max(mip, v)` pour chaque canal ; couleur finale = Σ `mip_i × couleur_i` × exposition. Le rayon s'arrête tôt quand **chaque** canal actif a atteint son plafond (opacité × 0,999). Un pixel dont le maximum est inférieur à 0,004 reste transparent.

Mode 2 : Fluorescence naturelle (émission et absorption)

Image mentale : chaque fluorophore est une **petite lampe** qui brille dans sa couleur, et la matière dense devant elle **filtre** la lumière. On obtient du relief et de la profondeur, tout en gardant des couleurs fidèles.

🔧 **POUR LES CURIEUX**

Pour chaque échantillon de densité `d` = valeur du canal le plus fort (si `d > 0,0025`) :

- opacité du segment (loi de Beer-Lambert) : `a = 1 − exp(−absorption × delta × d)` avec `absorption = 1,8` ;
- lumière émise : `T × emissionGain × exposition × delta × Σ v_i × couleur_i` avec `emissionGain = 2,2` , où `T` est la **transmittance** (la part de lumière qui n'a pas encore été absorbée, de 1 à 0) ;
- puis `T ← T × (1 − a)` ; arrêt quand `T < 0,004` .

À la fin, la **luminance** (formule Rec. 709) est compressée par une courbe de Reinhard étendue (point blanc 2,0) et les trois composantes sont multipliées par **le même rapport** (« verrouillage de la chromaticité ») : un fluorophore très brillant reste vert, il ne vire jamais au blanc. Seul le recouvrement de 3 ou 4 canaux est éclairci (au plus 50 %). Une saturation de 1,18 est appliquée à luminance constante.

⚠️ **ATTENTION**

L'absorption n'est **pas une mesure** : c'est une constante d'affichage (1,8), indépendante de l'échantillon. Elle sert à donner du relief, pas à quantifier l'opacité réelle du tissu.

Mode 3 : Structure (DVR)

Le DVR (« Direct Volume Rendering ») empile des couches **semi-transparentes** comme des vitres teintées, en partant de la plus proche. Une fois la pile presque opaque (**97 %**), le rayon s'arrête : ce qui est derrière n'est plus visible.

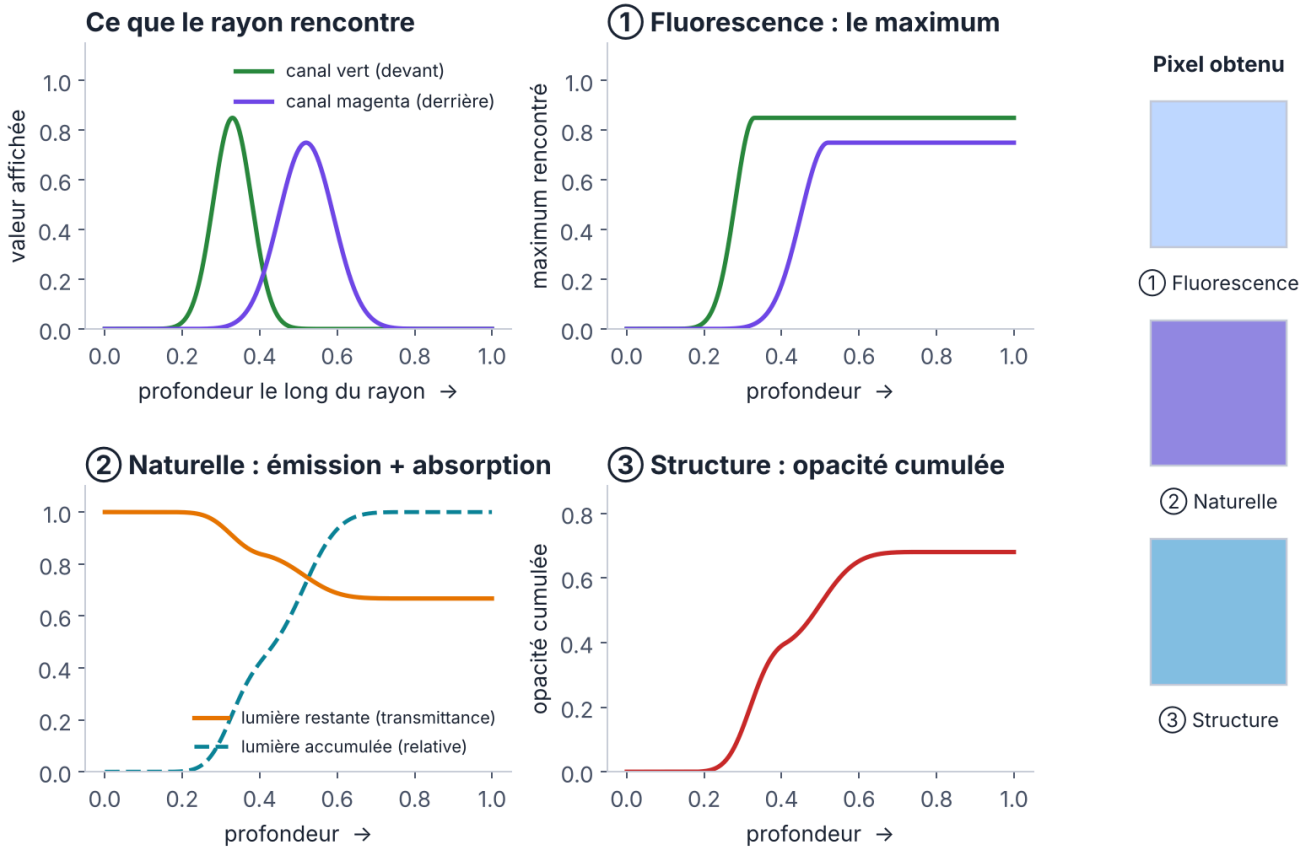
🔧 **POUR LES CURIEUX**

Pour chaque échantillon d'opacité locale `a = max des canaux` (si `a > 0,01`) : `pas = 1 − (1 − 0,05 × a)^(delta / 0,01)` puis `couleur += (1 − alpha) × pas × couleur_locale` et `alpha += (1 − alpha) × pas`.

L'exposant fait qu'une colonne de matière opacifie **selon sa longueur**, pas selon le nombre d'échantillons : l'image ne change pas quand le viewer allège le pas pendant la rotation.

Un rayon, trois réponses

Imaginons un rayon qui rencontre d'abord un nuage vert, puis, un peu derrière, un nuage magenta qui le recouvre en partie.



▲ Le même rayon, résumé par les trois modes (calcul avec les formules du viewer ; curseur d'exposition 1 pour le mode 1, 4 pour le mode 2 et 3 pour le mode 3, comme sur les captures).

- **Fluorescence** : on garde le plus fort de chaque canal, **sans tenir compte de l'ordre**.

- **Naturelle** : la lumière du nuage magenta est un peu **atténuée** par le nuage vert qui est devant (la transmittance descend à environ 0,67).
- **Structure** : le nuage vert **cache** partiellement le magenta ; l'opacité cumulée finit autour de 0,68.

9.7 Lisser entre les voxels

Entre deux voxels, le viewer ne montre pas des marches d'escalier : il **interpole**. Sur le **format 4** (celui de tous les jeux de démonstration), la carte graphique mélange les huit voxels voisins : c'est l'**interpolation trilinéaire**.

Pour que ce mélange ne déborde jamais d'une brique sur sa voisine, chaque brique est stockée avec **1 voxel de bordure** (66^3 au lieu de 64^3). Aux frontières entre briques, l'image est donc continue, sans couture.

📌 **À RETENIR**

- **Format 4** et texture 3D complète : interpolation trilinéaire (lisse).
- **Ancien format 2**, quand l'atlas est découpé en briques : le voxel le plus proche est affiché tel quel (blocs visibles en zoom).

9.8 Fluide quand vous bougez, net quand vous vous arrêtez

Faire tourner un volume à 60 images par seconde demande de calculer chaque image en moins de **16,7 ms**. Le viewer mesure le temps de chaque image pendant que vous glissez et **ajuste deux réglages** pour tenir ce budget.

▲ Pendant le glissement, la définition de l'image et le nombre d'échantillons sont réglés en continu ; dès que vous vous arrêtez, tout revient à la qualité maximale.

RÉGLAGE	PENDANT LE GLISSEMENT	AU REPOS
Définition de l'image	de ×0,25 à ×1 (départ ×0,75)	1 pixel d'écran (jusqu'à ×2)

RÉGLAGE	PENDANT LE GLISSEMENT	AU REPOS
Échantillons par voxel	de 0,1 à 0,75 (départ 0,35)	**1,0**

- L'interaction est considérée comme terminée **250 ms** après le dernier mouvement.
- Si une image dépasse 1,3 × 16,7 ms, on baisse d'abord la définition, puis les échantillons ; si elle reste sous 1,1 ×, on remonte doucement.
- Le temps est mesuré par le **chronomètre de la carte graphique** quand le navigateur le propose, sinon par l'intervalle entre deux images.

> ℹ️ **À SAVOIR**
>
> Rien n'est dessiné en continu : le viewer ne calcule une image que si quelque chose change (caméra, réglage, arrivée de briques). Pendant le chargement, les redessins dus aux nouvelles briques sont groupés **toutes les 250 ms** au plus.

9.9 Le garde-fou contre les plantages de la carte graphique

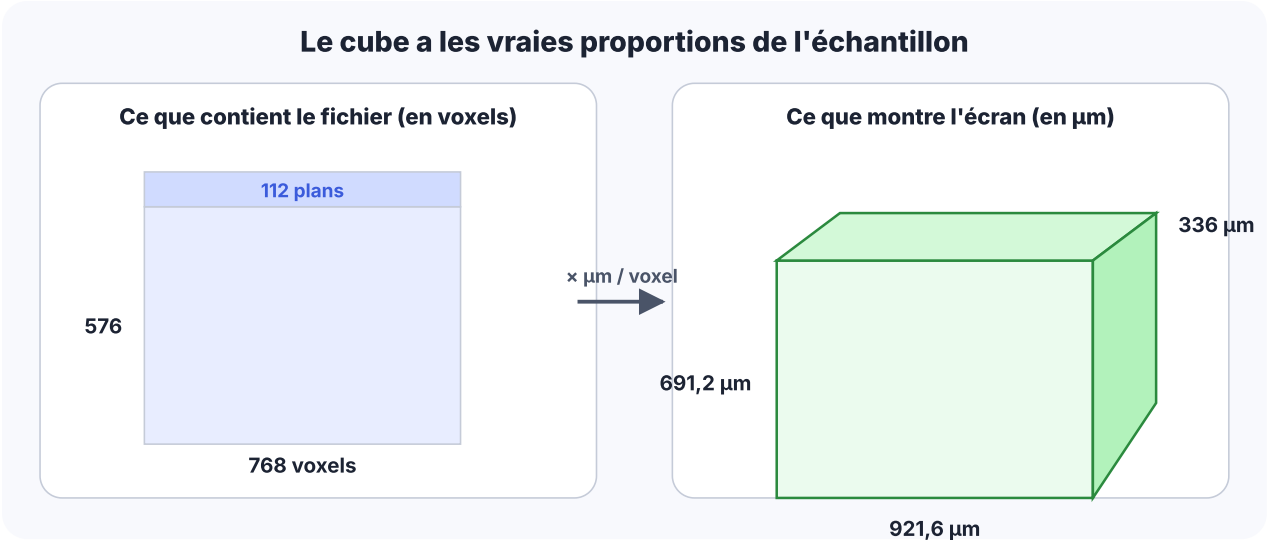
Sous Windows, si un seul calcul de la carte graphique dure plus de **2 secondes environ**, le système la réinitialise : c'est un « TDR » et l'écran clignote. Le viewer s'en protège.

> 🔧 **POUR LES CURIEUX**
>
> Pour une image **au repos**, le nombre maximal d'échantillons par rayon est `clamp(⌊6 × 10⁹ / nombre de pixels⌋, 256, 4096)` . Exemple : un écran de 1 920 × 1 080 donne ⌊6 × 10⁹ / 2 073 600⌋ = **2 893** échantillons au plus par rayon ; le même écran en pixels doublés (Retina, 8,3 millions de pixels) en donne 723.
>
> Ce plafond est ajusté en cours d'usage : multiplié par `max(0,5 ; 150 / durée)` si une image au repos dépasse 200 ms, relevé de 10 % si elle reste sous 60 ms, et divisé par deux après une perte de contexte graphique (chapitre 10). Pendant un glissement, il vaut un dixième de cette valeur.

Pour vous, cela signifie qu'un rayon exceptionnellement long peut, sur un très grand écran, être échantillonné un peu plus largement qu'un voxel. Dans l'usage normal (volumes de quelques centaines de voxels), le plafond n'est jamais atteint.

9.10 Les vraies proportions : calibration et étirement en Z

Le cube n'est pas dessiné d'après le **nombre de voxels** mais d'après les **dimensions réelles en micromètres**. Les voxels sont souvent plus épais en Z qu'en X et Y ; sans correction, l'embryon paraîtrait écrasé.



▲ Le fichier compte des voxels ; l'écran montre des micromètres. Jeu de démonstration E9.5 : voxels de 1,2 × 1,2 × 3,0 µm.

🔢 **MICRO-EXEMPLE**

Jeu E9.5 : 768 × 576 × 112 voxels de 1,2 × 1,2 × 3,0 µm.

- X : 768 × 1,2 = **921,6 µm** ; Y : 576 × 1,2 = **691,2 µm**.
- Z : (112 − 1) × 3,0 + épaisseur de coupe optique 3,0 = **336 µm**.
- Le plus grand côté (X) vaut 1 ; le cube mesure donc **1 × 0,75 × 0,365** (336 / 921,6).

Le viewer classe la calibration en trois états : **exacte** (tailles de voxel et épaisseur de coupe connues), **estimée**, ou **absente**. Sans calibration, la **barre d'échelle** 3D est cachée plutôt que fausse. Ce statut est **recalculé par le viewer** à partir des métadonnées reçues (« exacte » seulement si une épaisseur de coupe est connue) ; le pipeline, lui, n'écrit que `exact` ou `metadata-missing` (chapitre 8).

Dans le panneau **Échelle Physique**, le curseur **Surcharge Z** (de ×0,25 à ×2,0) étire seulement l'**affichage** en Z pour mieux voir les couches fines ; le bouton **1:1** le remet à zéro. Les mesures en µm ignorent cet étirement.

⚠️ **ATTENTION**

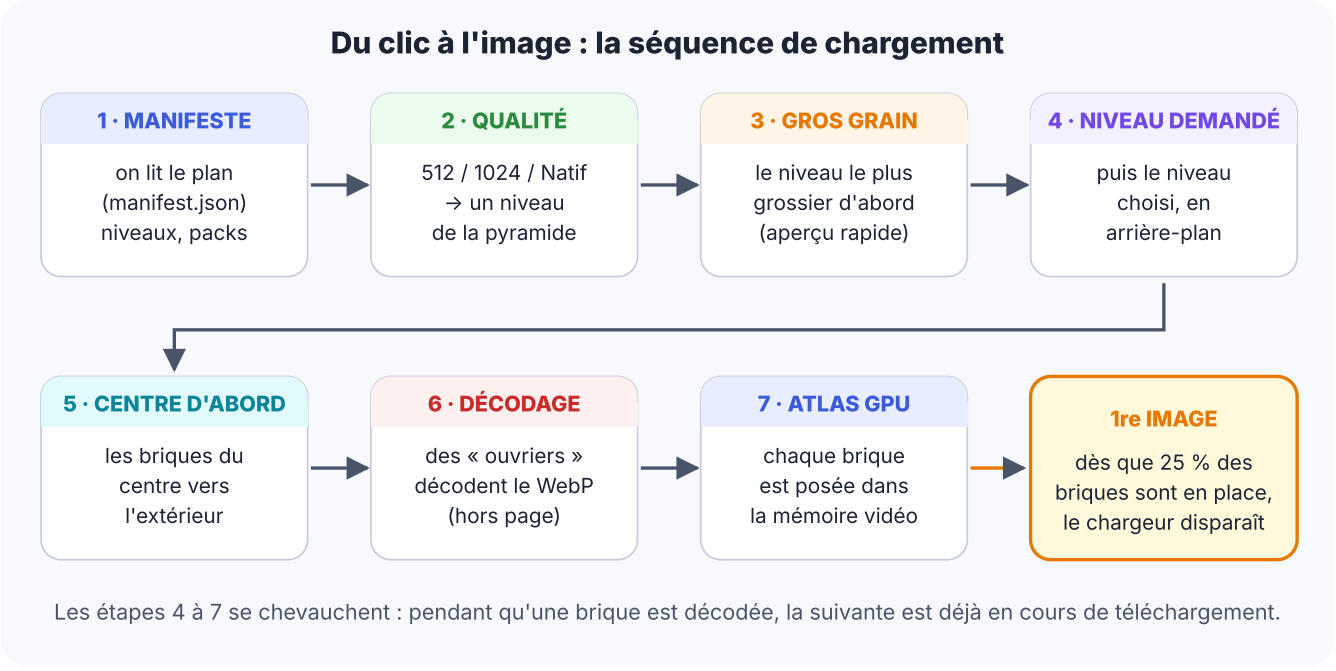
La barre d'échelle 3D est exacte pour ce qui se trouve **à la profondeur du centre de l'échantillon**. La vue est en perspective : ce qui est plus près paraît plus grand, ce qui est plus loin paraît plus petit.

10. Afficher des gigaoctets : le streaming

- Le navigateur ne charge **jamais** tout le volume : il télécharge des **briques** (petits cubes de 64^3 voxels) du centre vers l'extérieur et les range dans la mémoire de la carte graphique.
- Une première image apparaît dès que **25 %** des briques sont en place ; la qualité monte ensuite sans bloquer l'écran.
- Si la mémoire graphique est insuffisante, le viewer choisit tout seul un **niveau plus grossier** et vous le dit.

10.1 La séquence de chargement

Quand vous ouvrez un jeu de données, sept étapes s'enchaînent. Vous ne les voyez pas toutes, mais elles expliquent pourquoi l'image devient nette **progressivement**.



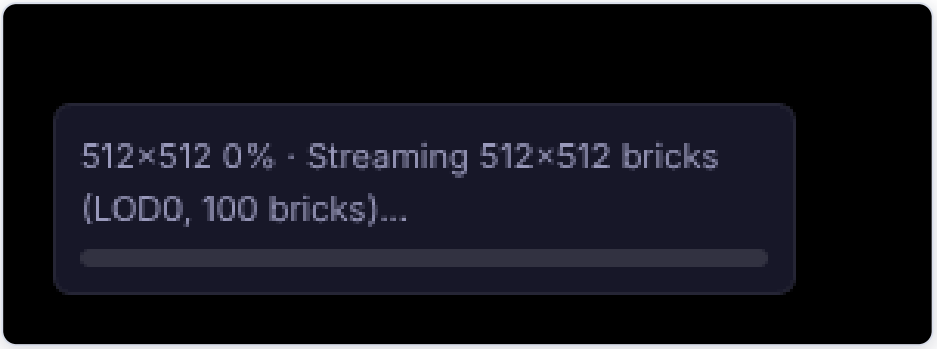
▲ Du clic à l'image : sept étapes qui se chevauchent.

1. **Manifeste.** Le viewer lit le « plan » du jeu de données (`bricks/manifest.json`) : niveaux de la pyramide, taille des briques, fichiers de paquets.
2. **Qualité.** Le menu **Qualité de Rendu** se traduit en un niveau de la pyramide (section 10.2).
3. **Gros grain d'abord.** À la première ouverture, le niveau **le plus grossier** est chargé en premier : quelques briques, une image en un clin d'œil.
4. **Niveau demandé.** Le niveau choisi se charge ensuite en arrière-plan.

5. **Du centre vers l'extérieur.** Les briques sont classées par distance au centre, **mesurée en micromètres** (pour que la forte épaisseur des voxels en Z ne fausse pas le « centre »).
6. **Décodage.** Des « ouvriers » (workers) décodent chaque brique hors de la page (section 10.5).
7. **Atlas.** La brique décodée est copiée à sa place dans la mémoire de la carte graphique (section 10.3).

> 📌 **À RETENIR**
>
> La **première image** s'affiche quand **25 %** des briques du niveau en cours sont sur la carte graphique ; la carte « chargement » disparaît alors. Les redessins dus aux briques qui arrivent sont groupés (au plus un toutes les 250 ms).

Où voir la progression

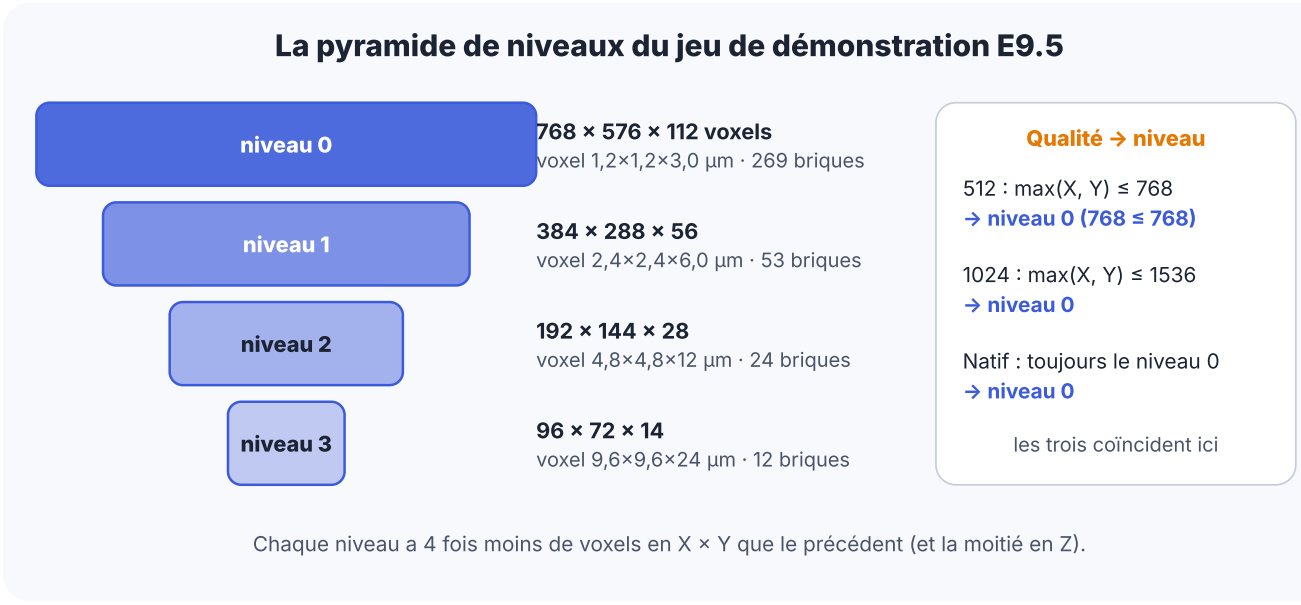


▲ La ligne de progression en bas à gauche du viewer : qualité, pourcentage de briques traitées et étape en cours (le texte suit la langue de l'interface, par exemple « Diffusion des briques 1024×1024 (LOD..., N briques)... »).

Le pourcentage est la part des briques **déjà traitées** (déposées ou abandonnées), pas des octets téléchargés. La ligne disparaît quand le niveau est complet.

10.2 Le menu de qualité : 512, 1024, Natif

La pyramide de niveaux a été fabriquée par le pipeline (chapitre 6) : chaque niveau a **4 fois moins de voxels en X × Y** que le précédent. Le menu choisit **le plus fin des niveaux qui reste raisonnable** pour le réglage demandé.



▲ La pyramide réelle du jeu de démonstration E9.5 et la règle des trois qualités.

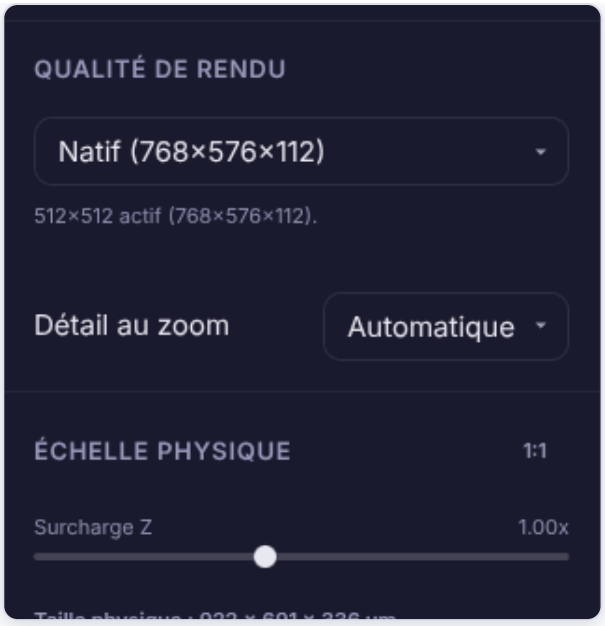
RÉGLAGE	NIVEAU RETENU (FORMAT 4)
512 (défaut)	le niveau le plus fin dont le plus grand côté XY est **≤ 768 voxels**
1024	le niveau le plus fin dont le plus grand côté XY est **≤ 1 536 voxels**
Natif	toujours le **niveau 0** (pleine résolution)

🔢 **MICRO-EXEMPLE**

Jeu **fictif** de 3 072 × 2 304 voxels en XY, niveaux à 3 072, 1 536, 768 et 384 :

- **512** → niveau à 768 (≤ 768) ; **1024** → niveau à 1 536 (≤ 1 536) ; **Natif** → 3 072.

Sur le jeu de démonstration E9.5, le plus grand côté est 768 : les trois réglages tombent sur le niveau 0. Le menu **n'affiche alors qu'une seule option**, avec les vraies dimensions : **Natif (768×576×112)**.



▲ Le panneau **Qualité de Rendu** : une seule ligne par niveau réellement distinct (ici une seule, avec ses dimensions), la ligne d'état (« 512×512 actif ») et, dessous, le sélecteur **Détail au zoom** (section 10.8).

> ℹ️ **À SAVOIR**
>
> Le viewer s'ouvre en qualité **512**. Un changement de qualité qui échoue laisse le volume précédent à l'écran et remet le menu en place.

Sur un jeu au format 5 : la matrice

Sur un jeu au format 5 (section 7.11), le menu laisse place à un **tableau** : une ligne par niveau, une colonne par compression. Chaque case est une façon d'afficher le même volume ; **un clic la charge**.

▲ Le panneau **Qualité de Rendu** d'un jeu au format 5 (jeu de démonstration ; barre latérale élargie pour montrer les cinq colonnes).

N	CE QUE C'EST
1	Les **lignes** : un niveau, avec ses vraies dimensions (ici natif 768 × 576 × 112, puis 512 et 256).
2	Les **colonnes** : `Sans perte` , `±2` , `±4` , `Légère` , `Très légère` . Une info-bulle sur l'en-tête explique chaque compression.
3	La **mémoire graphique** du niveau (« GPU 21,0 MB »), avec un trait de couleur.
4	La **case active**, encadrée : ici **512 · ±2**, le réglage d'ouverture. Chaque case indique le **poids à télécharger** et, une fois le débit mesuré, le **temps estimé**.

COULEUR	SIGNIFICATION
vert	tient dans les trois quarts de la mémoire graphique estimée
orange	juste : au-delà des trois quarts, ou seulement grâce à la compression graphique (section 10.3)
rouge	ne tient pas : l'ensemble reste au niveau le plus fin qui tient, et le **détail au zoom** (section 10.8) va chercher la case choisie autour de ce que vous regardez

🔢 **MICRO-EXEMPLE**

Jeu de démonstration (lu sur la capture). Le natif sans perte pèse **7,5 Mo** ; la case d'ouverture, 512 · ±2, **2,3 Mo**, trois fois moins ; 256 · Très légère, **0,4 Mo**. La mémoire graphique dépend seulement de la ligne : 189 Mo au natif, 84 Mo à 512, 21 Mo à 256.

- La case choisie est gardée dans l'espace de travail, l'adresse de la page (`512x512@b2` , `native@ll` …) et ce navigateur.
- Le **détail au zoom** reste dans la **même colonne** que la case choisie.
- La page **Comparer** répartit toujours la mémoire par lignes ; chaque panneau garde sa colonne.
- La barre latérale fait 320 pixels de large : les dernières colonnes se voient en **faisant défiler le tableau vers la droite**.

⚠️ **ATTENTION**

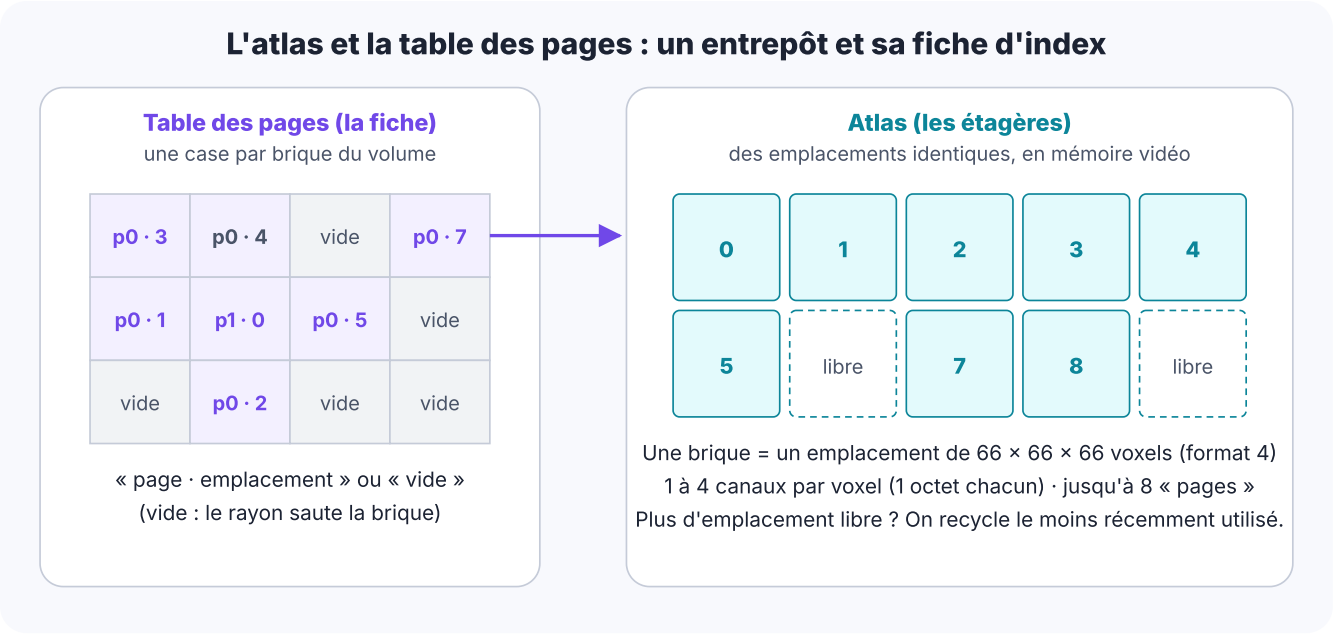
Une colonne avec perte **change les voxels affichés**, pas les données : les mesures, les plugins de calcul et les exports « sans perte » lisent toujours la colonne sans perte du niveau natif (chapitres 12 et 15).

10.3 L'atlas GPU et la table des pages

Une brique de 64^3 voxels est trop petite pour mériter sa propre texture ; en créer des centaines serait ingérable. Le viewer réserve donc en mémoire vidéo **de grands blocs d'emplacements identiques** : l'**atlas**.

💡 UNE IMAGE POUR COMPRENDRE

L'atlas est un **entrepôt** dont toutes les étagères ont la même taille. La **table des pages** est la **fiche d'index** à l'entrée : « la brique n° 17 est sur l'étagère 4 ». Quand le rayon a besoin d'un voxel, il consulte la fiche, puis va directement à la bonne étagère. Si la fiche dit « vide », il ne se déplace même pas.



▲ L'atlas et sa table des pages.

- **Une case de la table par brique** du volume : numéro de page + emplacement, ou « vide ».
- Jusqu'à **8 pages** d'atlas, de 4, 8 ou 16 emplacements de côté.
- **1 à 4 canaux** par voxel, un octet chacun : 1 canal = 1 octet par voxel, 2 canaux = 2, 3 ou 4 canaux = 4.
- Quand tous les emplacements sont occupés, on **recycle le moins récemment utilisé**.
- L'atlas est **dimensionné avant d'être alloué** : jamais de texture créée au hasard par brique.

🔧 POUR LES CURIEUX

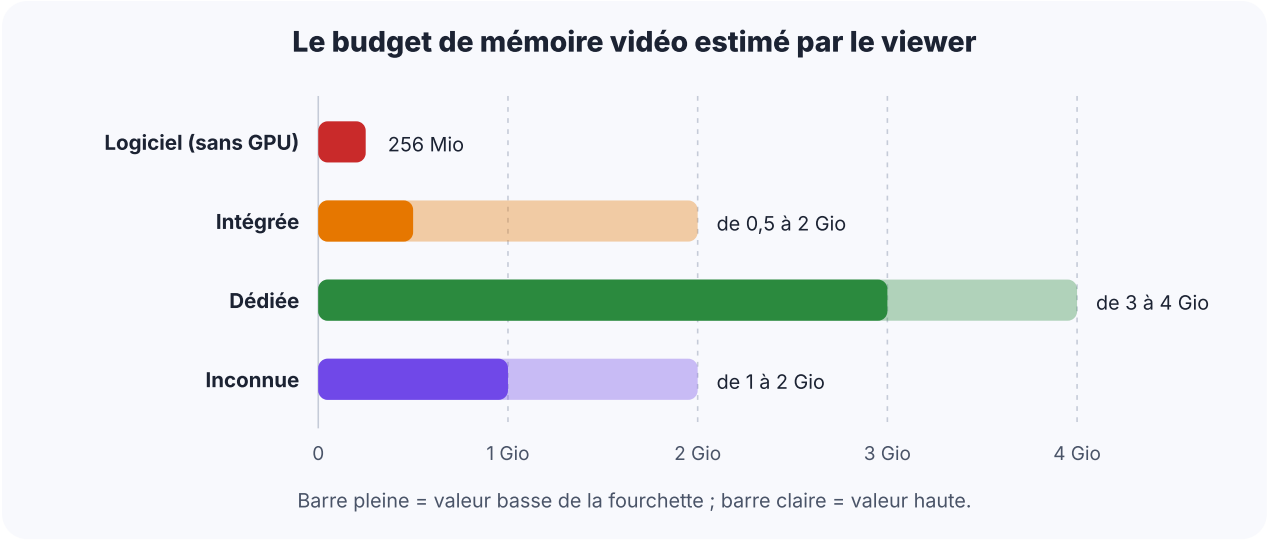
Un petit volume reste d'un seul tenant : si le niveau tient dans moins de **512 Mio** (en 4 octets par voxel) et dans les limites de la carte, il est stocké dans **une seule texture 3D** dense. Le jeu E9.5 au niveau 0 en est un exemple : 768 × 576 × 112 × 4 octets ≈ **189 Mio**. L'atlas à pages ne sert que pour les gros niveaux.

La compression graphique (timelapses)

Depuis la plateforme 1.60, les images d'un **timelapse** peuvent être gardées dans la carte graphique sous forme **compressée** (blocs BC4 / BC5) : **0,5 octet** par voxel et par canal au lieu de 1, donc environ **deux fois plus d'instants** en mémoire. C'est de l'affichage seulement (les valeurs affichées s'écartent d'environ 1 à 2 niveaux de gris) ; le Studio et le détail au zoom restent exacts. Réglage **Compression GPU** du panneau : **Timelapses** (par défaut), **Activé** ou **Désactivé**. Sur un rendu logiciel, l'option est indisponible et le panneau le dit.

10.4 Le budget de mémoire vidéo

Aucun navigateur ne dit combien de mémoire vidéo il reste. Le viewer **l'estime** d'après le type de carte graphique et, si disponible, la mémoire de l'appareil.



▲ Le budget estimé selon le type de carte graphique (1 Gio = 1 024 Mio).

TYPE DE GPU	BUDGET ESTIMÉ
Logiciel (pas de vraie carte graphique)	**256 Mio**
Intégrée (Intel, Apple…)	de **0,5 à 2 Gio** selon la mémoire de l'appareil
Dédiée (NVIDIA, Radeon…)	**4 Gio** (3 Gio si l'appareil déclare moins de 8 Go)
Inconnue	1 ou 2 Gio

Chaque **perte de contexte** graphique de la dernière semaine **divise le budget par deux** (plancher : 256 Mio). L'administrateur peut le fixer à la main si besoin.

Quand ça ne rentre pas

Avant d'allouer quoi que ce soit, le viewer calcule la taille du niveau demandé. Si elle dépasse le budget, il essaie le niveau **immédiatement plus grossier**, et ainsi de suite.

🔢 **MICRO-EXEMPLE**

Exemple **fictif** : vous demandez **Natif** et le niveau 0 réclame 1,9 Gio pour un budget de 1 Gio : le viewer passe au niveau 1 (environ 4 fois moins), qui rentre. Un avis s'affiche :

« La résolution {demandée} dépasse la mémoire GPU disponible — affichage en {obtenue}. ({nécessaire} Mo nécessaires, {disponible} Mo disponibles pour le volume.) »

Si la carte **refuse** une allocation malgré le calcul, le viewer retombe aussi sur un niveau plus grossier ; l'avis le précise : « (Le GPU a refusé l'allocation.) ». Le volume reste **toujours visible**, jamais un onglet planté.

10.5 Télécharger et décoder sans bloquer la page

Décoder une brique (une image WebP à défaire en voxels) est un calcul lourd. Il est confié à un **groupe d'ouvriers** (workers) qui travaillent **en parallèle de la page** : l'interface reste fluide.

- **Nombre d'ouvriers** : `min(8, nombre de cœurs − 1)`, au moins 1.
- **Décodage fidèle** : aucune gestion des couleurs du navigateur n'est appliquée ; les octets sont des **intensités mesurées**, pas des couleurs.
- **Mosaïque** : une brique de 66^3 voxels est une image WebP de 594 × 528 pixels (9 × 8 tuiles de 66 × 66) ; une image d'une autre taille est refusée comme corrompue.
- **Pas de copie inutile** : les octets décodés sont transférés tels quels à la page, puis envoyés à la carte graphique.
- Un ouvrier qui plante ou se fige (30 s) est **remplacé** ; la brique est retentée jusqu'à 3 fois, puis comptée comme manquante.

> ⚠️ **ATTENTION**
>
> Une brique corrompue n'est **jamais** affichée : elle reste vide et le viewer le signale (par exemple : « N briques n'ont pas pu être chargées »). Un flux incomplet n'est pas gardé en mémoire : rechargez pour réessayer.

10.6 Les paquets : un téléchargement ordonné

Les briques sont rangées dans des **paquets** (fichiers `.bin`). Télécharger un paquet entier est bien plus efficace que des milliers de petites requêtes.

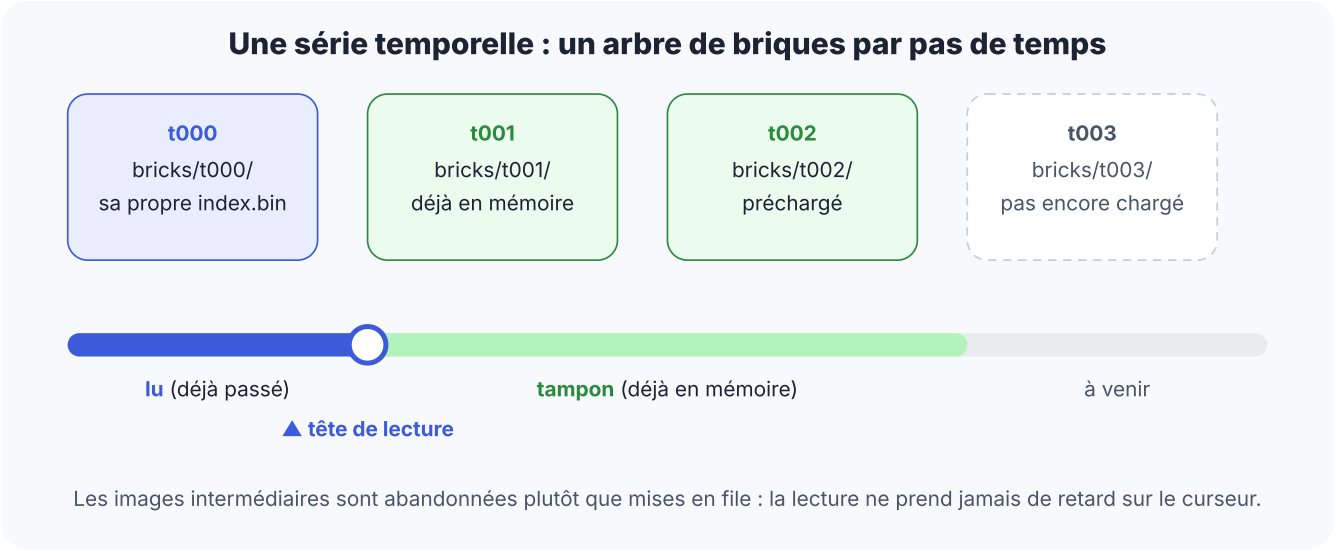
- **Ordre « paquet par paquet »** : le viewer part de la brique centrale et progresse paquet après paquet ; chaque paquet est téléchargé **une seule fois** et libéré dès que ses briques sont découpées.
- **4 téléchargements de paquets au plus en même temps**, car un navigateur ouvre 6 connexions par serveur : les appels de la page ne doivent pas attendre derrière les gros fichiers.
- **Coupe précise** : pour une coupe ou une image du Studio, on demande seulement les **plages d'octets** utiles.

CACHE (OCTETS COMPRESSÉS)	TAILLE
Paquets entiers, tant qu'un chargement en a besoin	192 Mio
Plages d'octets (coupes)	64 Mio
Préchargement (images voisines d'une série)	32 Mio

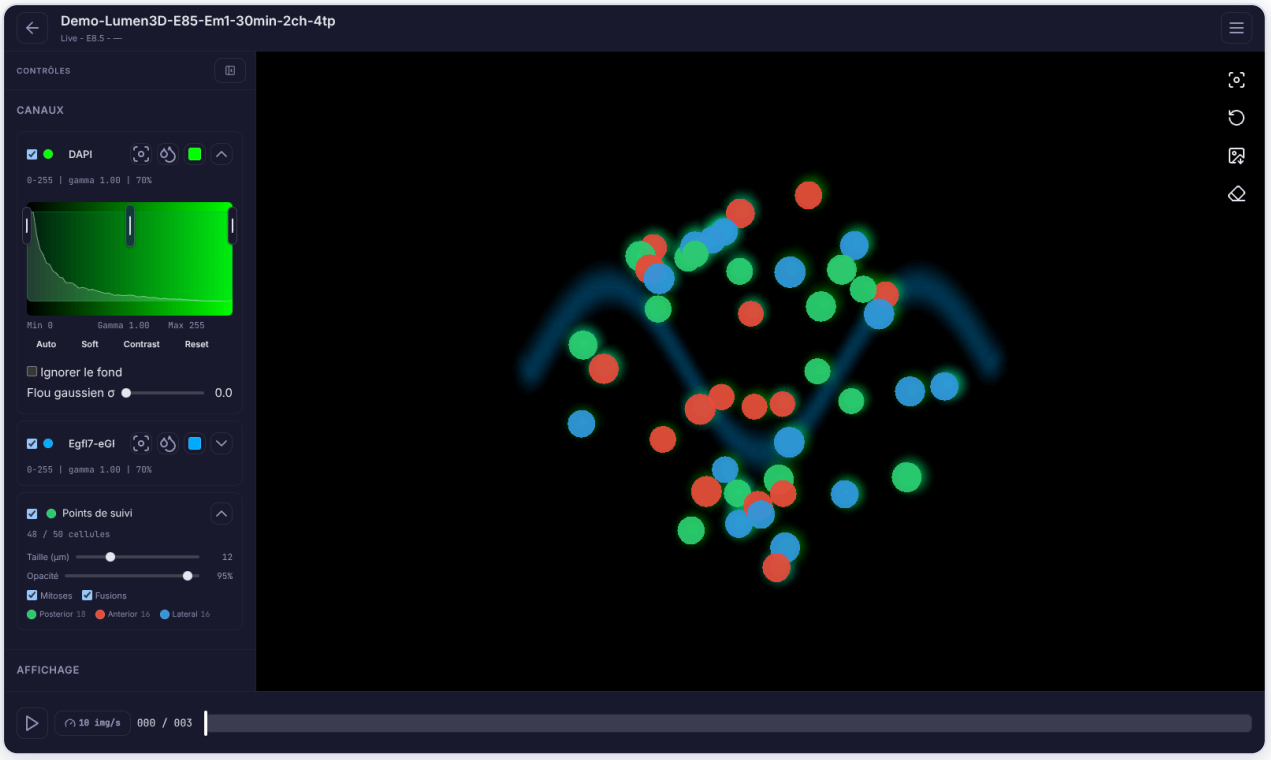
> 🔧 **POUR LES CURIEUX**
>
> Les briques **décodées** ne sont jamais gardées côté page : l'atlas GPU est leur seul domicile. Chaque adresse de paquet porte un cachet `?v=` (une empreinte des paquets du manifeste) ; le serveur répond alors « à garder un an » : si le jeu de données change, l'empreinte change et le navigateur retélécharge.

10.7 Les séries temporelles

Un jeu **live** (série temporelle) contient **un arbre de briques par pas de temps**. Chaque pas de temps est chargé comme un petit volume à part entière.



▲ Chaque pas de temps a son arbre ; le tampon montre les images déjà en mémoire.



▲ La frise du jeu de démonstration Demo-Lumen3D-E85-Em1-30min-2ch-4tp : lecture, vitesse, curseur de temps et barre de tampon.

- **Lecture** : vitesses **0,5 – 1 – 2 – 5 – 10 – 20 images par seconde** (10 par défaut, mémorisée dans votre navigateur).
- **Une demande à la fois** : si vous déplacez le curseur vite, seule la **dernière** demande est servie.
- **La lecture attend l'image** : l'horloge est suspendue pendant qu'une image charge ; la lecture va à la vitesse réelle d'arrivée.
- **Tampon** : le viewer préserve en mémoire les images voisines (jusqu'à `min(768 Mio, max(128 Mio, 40 % du budget))`). La barre de la frise montre celles **réellement présentes** à la qualité courante.

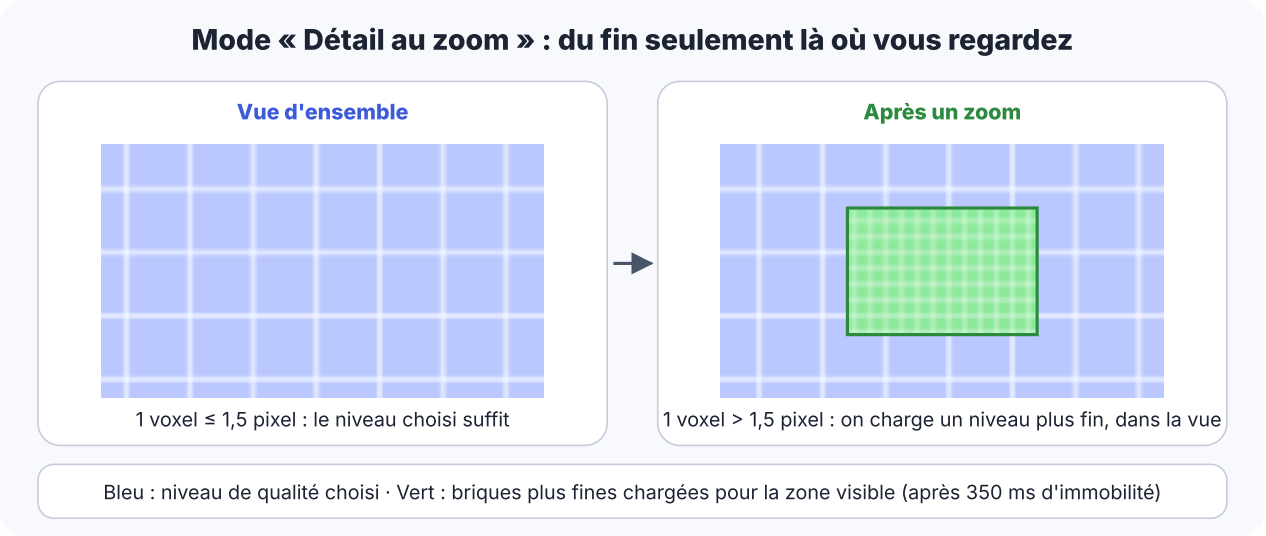
Un texte précise si la série ne tient pas entièrement : « Tampon n/total · qualité — la série entière ne tient pas en mémoire (… images max) ».

- **Préchargement** : pendant la lecture, les paquets des images suivantes (t+1, t+2…) sont préchargés en temps libre.

> ℹ️ **À SAVOIR**
>
> Le préchargement complet n'est possible que pour les niveaux stockés en **texture dense** : un niveau en atlas à pages n'est pas préchargé.

10.8 Le mode « Détail au zoom »

Vous zoomez fort sur une zone : les voxels du niveau choisi deviennent plus gros que l'écran n'en a besoin, l'image paraît floue. Le mode `Détail au zoom` charge alors **des briques plus fines, seulement dans la zone visible**.



▲ Le niveau choisi partout (bleu), et des briques plus fines uniquement dans la vue (vert).

OPTION	EFFET
`Automatique`	actif si le budget mémoire dépasse **1 Gio**
`Activé`	actif même sous 1 Gio
`Désactivé`	jamais

- **Déclencheur** : un voxel du niveau courant couvre **plus de 1,5 pixel d'écran**.
- Le niveau retenu est **le plus grossier** des niveaux plus fins qui descend sous 1,5 pixel par voxel (le moins d'octets possible).
- Le chargement démarre **350 ms** après que la caméra s'est arrêtée, 8 briques à la fois, **les plus grosses et les plus centrales d'abord**. Il utilise au plus 768 Mio et 75 % de la mémoire restante.
- À la frontière de la zone fine, l'image passe d'un niveau à l'autre **à une face de brique**, sans fondu.

⚠️ **ATTENTION**

Le mode n'existe que pour les **formats 4 et 5** (briques à bordure) ; en format 5, il reste dans la colonne de compression choisie. Il est suspendu pendant qu'un volume se charge et dès que vous appliquez un **flou gaussien** à un canal (chapitre 11), car des briques brutes sur un canal flou donneraient une autre image.

Le texte d'état affiche, selon le cas : « Détail : N briques du niveau L visibles », « Détail : chargement de x sur y briques du niveau L », ou « Détail indisponible : mémoire GPU insuffisante ».

10.9 Quand la carte graphique plante

Un navigateur peut retirer sa carte graphique à une page (pilote en difficulté, trop de mémoire demandée). Le viewer sait **repartir seul**.

▲ La reprise après une perte de contexte graphique.

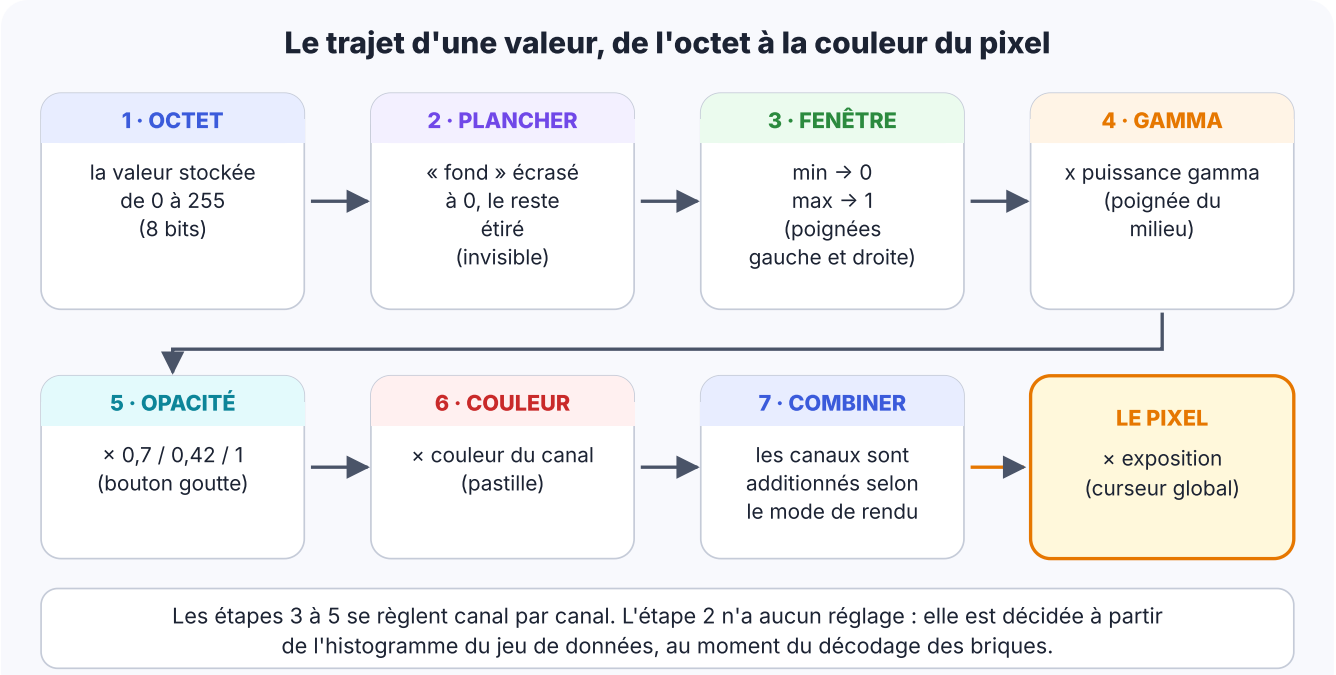
Vous voyez d'abord « Contexte GPU perdu : le rendu est en pause jusqu'à ce que le navigateur le rende. », puis le volume se recharge en plus léger. Après **trois pertes en moins de 120 secondes**, il s'arrête et vous invite à choisir une qualité plus basse ou à recharger la page.

11. Couleurs, contrastes et histogrammes

- Chaque canal suit la même chaîne de réglages : **plancher invisible → fenêtre min / max → gamma → opacité → couleur**, puis les canaux sont combinés.
- L'**histogramme** montre comment les 256 niveaux de gris sont répartis ; ses trois poignées règlent la fenêtre et le gamma.
- Les niveaux de gris sont **relatifs** à un jeu de données : ce ne sont pas des intensités de fluorescence calibrées.

11.1 La chaîne de traitement d'un canal

Chaque voxel est stocké sous la forme d'un **octet** (un nombre de 0 à 255). Avant de devenir un pixel coloré, il traverse sept étapes. Cinq d'entre elles sont à votre portée ; une seule est cachée.



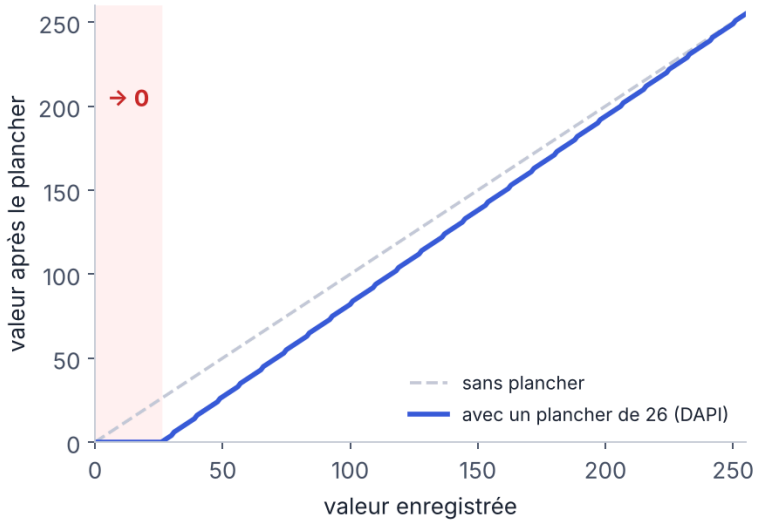
▲ Le trajet d'une valeur : de l'octet stocké à la couleur du pixel.

ÉTAPE	QUI DÉCIDE ?	OÙ LA RÉGLER
Plancher	automatique (histogramme)	nulle part
Fenêtre min / max	vous	poignées gauche et droite
Gamma	vous	poignée du milieu
Opacité	vous	bouton goutte (3 valeurs)
Couleur	vous	pastille de couleur

ÉTAPE	QUI DÉCIDE ?	OÙ LA RÉGLER
Exposition	vous	curseur **Visibilité (exposition)**, global

11.2 L'étape cachée : le plancher du fond

Avant même que vous touchiez à un réglage, le viewer **écrase le fond** : toutes les valeurs inférieures ou égales à un seuil, le **plancher**, deviennent **0**, et les valeurs restantes sont **étirées** pour occuper toute l'échelle 0–255.



▲ L'effet d'un plancher de 26 (valeur réelle estimée pour le canal DAPI du jeu E9.5).

🔧 **POUR LES CURIEUX**

Le plancher `f` est estimé à partir de l'histogramme du jeu de données : on cherche le premier niveau où le cumul atteint **la case 0 + 20 % des voxels non nuls**, on prend le bord supérieur de cette case, on ajoute 2 et on borne le résultat entre **6 et 48**. Ensuite : `valeur' = 0` si `valeur ≤ f`, sinon `arrondi((valeur − f) × 255 / (255 − f))`.

🔢 **MICRO-EXEMPLE**

Plancher de 26 (DAPI) : un voxel stocké à **120** devient (120 − 26) × 255 / 229 ≈ **105** ; un voxel à 20 devient **0**. Pour le jeu E9.5 les planchers estimés sont 26 (DAPI), 10 (Pecam1) et 22 (Sox2).

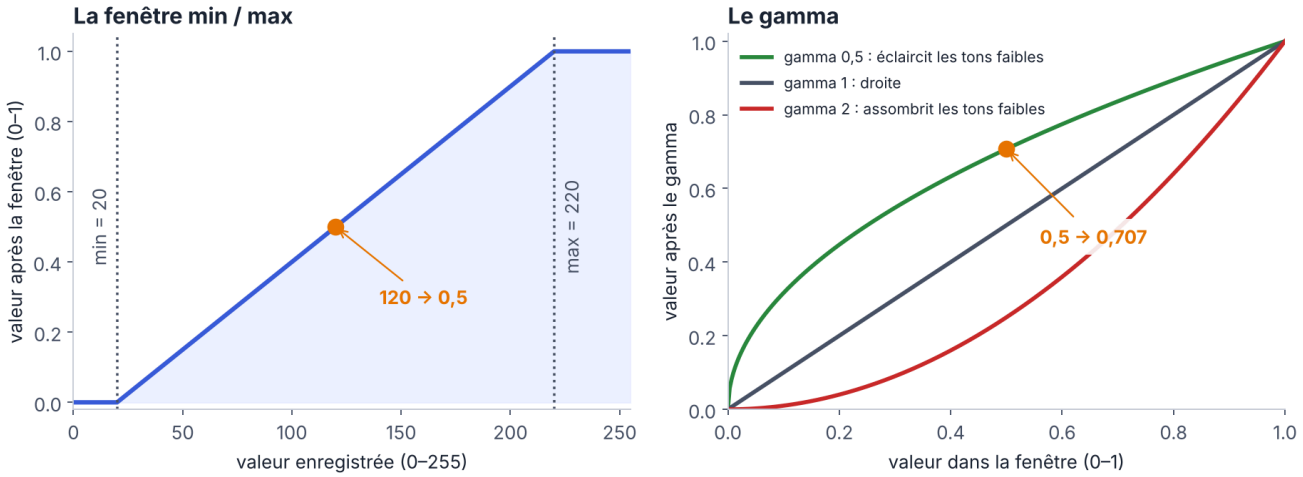
ℹ️ **À SAVOIR**

Les valeurs que vous lisez sur les poignées (Min 0, Max 255…) sont déjà **après** ce plancher. Vous ne le voyez jamais, mais il explique pourquoi le fond est si propre.

11.3 La fenêtre min / max et le gamma

La **fenêtre** choisit quelle portion de l'échelle devient « du noir au plein éclat ». Les valeurs sous **Min** deviennent noires, celles au-dessus de **Max** saturent, entre les deux le passage est **linéaire**.

Le **gamma** courbe ensuite cette échelle : un gamma inférieur à 1 **éclaircit** les tons faibles (utile pour des signaux ténus), un gamma supérieur à 1 les **assombrit**.



▲ À gauche, la fenêtre min 20 / max 220 ; à droite, l'effet du gamma 0,5 / 1 / 2.

🔢 **MICRO-EXEMPLE**

Un voxel de valeur **120**, avec Min = 20, Max = 220, gamma = 0,5 et opacité = 70 % :

1. Fenêtre : (120 − 20) / (220 − 20) = **0,5**
2. Gamma : 0,5 puissance 0,5 = **0,707**
3. Opacité : 0,707 × 0,7 = **0,495**
4. Couleur : sur un canal vert (0 ; 1 ; 0), le voxel contribue (0 ; **0,495** ; 0), avant l'exposition.

La poignée du milieu

La poignée du milieu ne règle pas le gamma directement : elle désigne le niveau (**dans la fenêtre**) qui doit s'afficher à **50 %**. Le viewer en déduit le gamma, borné entre **0,18 et 5,5** :

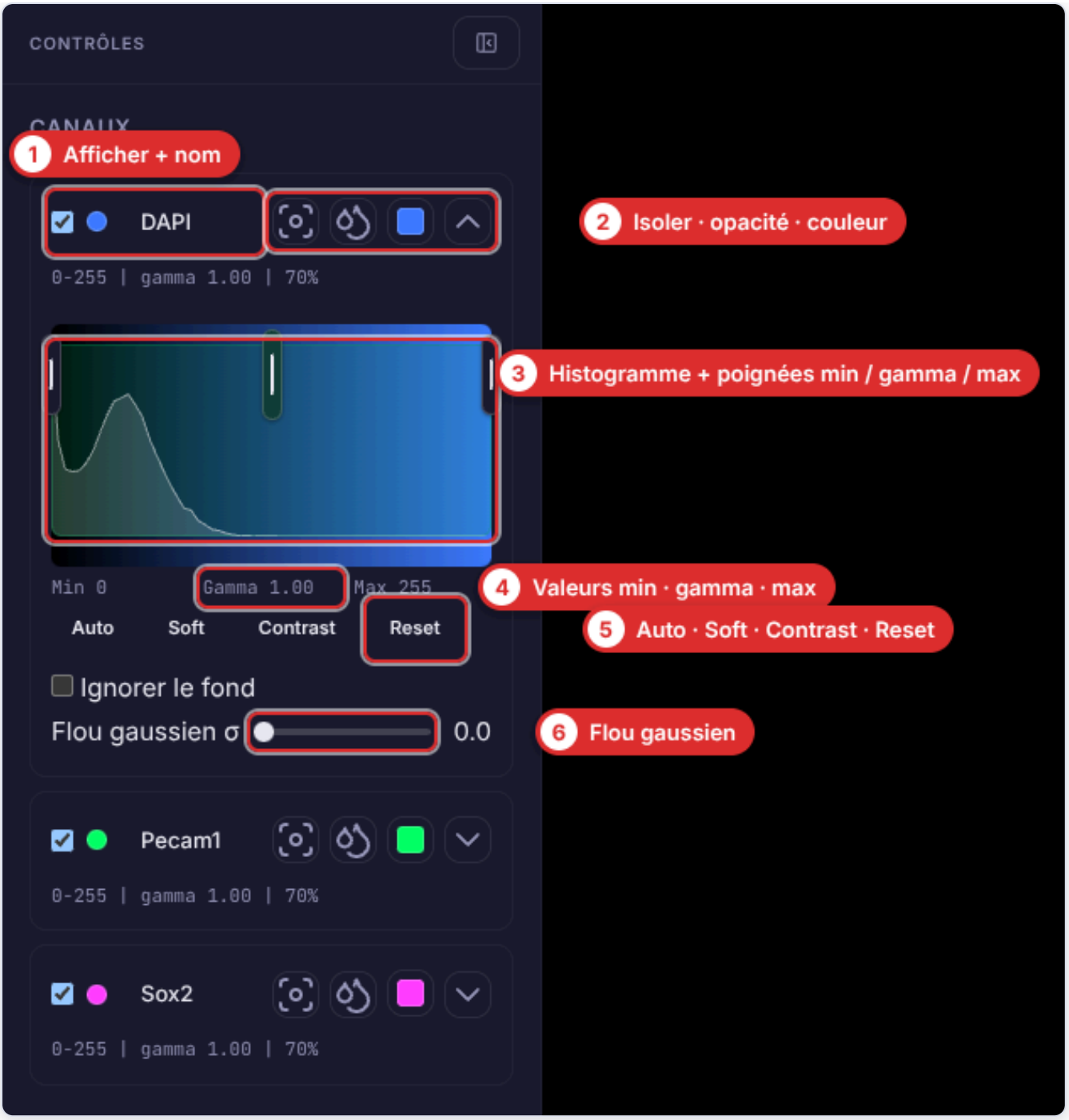
`gamma = ln 0,5 / ln m`, où `m` est la position relative de la poignée entre Min et Max.

🔢 **MICRO-EXEMPLE**

Poignée au quart de la fenêtre (m = 0,25) : gamma = ln 0,5 / ln 0,25 = **0,5**. Avec Min = 20 et Max = 220, ce quart correspond à la valeur 20 + 0,25 × 200 = **70** : une valeur de 70 s'affiche à 50 %. Poignée au milieu (m = 0,5) : gamma = 1.

11.4 Le panneau d'un canal

Chaque canal a sa carte dans le panneau **Canaux**. La ligne de résumé (par exemple `0-255 | gamma 1.00 | 70%`) rappelle la fenêtre, le gamma et l'opacité même quand la carte est repliée.



▲ La carte d'un canal dépliée (DAPI, jeu de démonstration E9.5).

(numéros : voir la légende ci-dessous.)

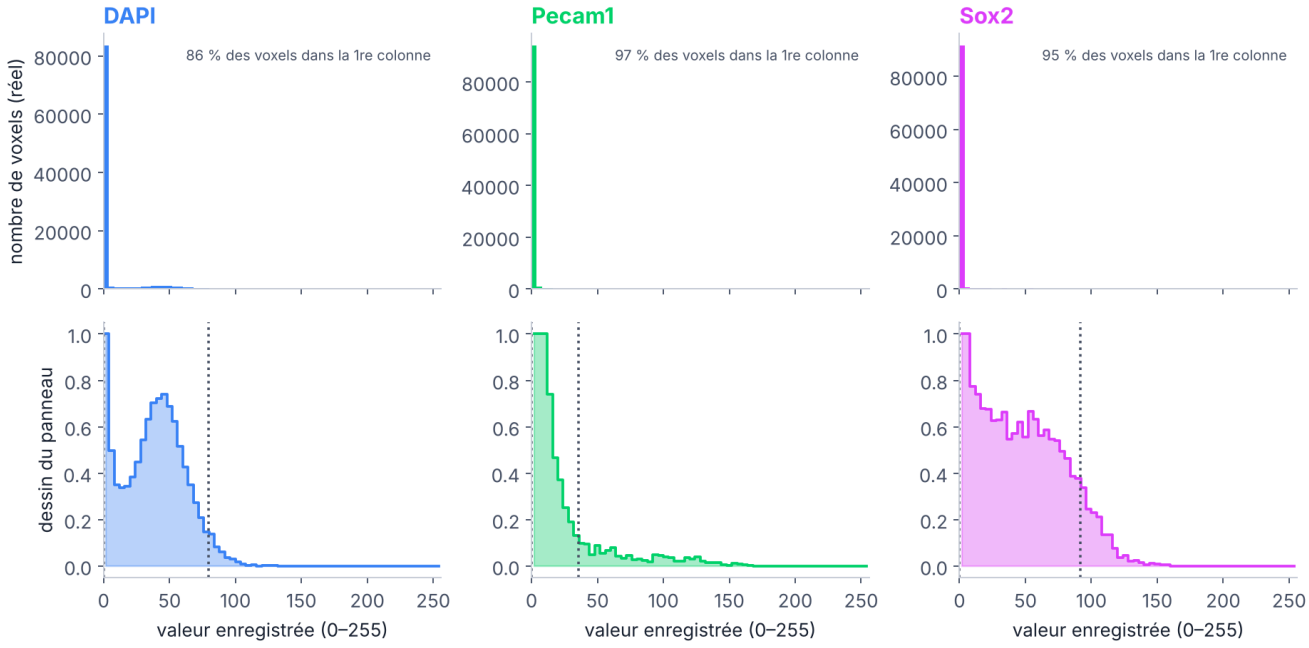
N	CE QUE C'EST
1	case à cocher (afficher / masquer le canal) et nom du canal, modifiable
2	boutons : **Isoler le canal** (Solo), opacité (goutte), couleur (pastille)
3	histogramme, avec les trois poignées : Min (gauche), gamma (milieu), Max (droite)
4	valeurs lues : Min, Gamma, Max

N	CE QUE C'EST
5	boutons de réglage rapide : Auto, Doux, Contraste, Réinit. (voir ci-dessous)
6	curseur **Flou gaussien σ** (section 11.7)

11.5 Les histogrammes

Un **histogramme** compte combien de voxels ont chaque valeur. Celui du panneau a **64 colonnes** (4 niveaux de gris par colonne).

- Il est calculé **par le pipeline**, sur le **niveau le plus grossier** de la pyramide, et enregistré dans le manifeste. Le navigateur ne le recalcule pas.
- Il porte sur **tous** les voxels, fond compris : c'est pourquoi la première colonne est énorme (86 à 97 % des voxels du jeu E9.5).
- Dans une **série temporelle**, chaque pas de temps a son propre histogramme ; le panneau le suit quand vous changez d'image.
- Il représente les **octets stockés** (avant le plancher et la fenêtre).



▲ Les histogrammes réels des trois canaux du jeu de démonstration E9.5. En haut : les comptes bruts. En bas : ce que dessine le panneau. Les traits pointillés marquent la plage choisie par **Auto**.

Comment le panneau dessine l'histogramme

Une colonne géante au niveau 0 écraserait tout le reste. Le panneau applique donc trois règles :

- échelle **linéaire**, normalisée par la colonne la plus haute ;
- les **4 % de colonnes les plus basses** (les 3 premières sur 64) sont **plafonnées à 1,35 fois** la plus haute colonne qui les suit ;
- les « trous en peigne » (une colonne sous 20 % de ses deux voisines) sont comblés par la moyenne de ses voisines, pour masquer un artefact de décodage du WebP.

ℹ️ **À SAVOIR**

La case **Ignorer le fond** ne change que le **dessin** : elle multiplie les colonnes affichées par $(i / 63)^2$ pour aplatir la bosse du fond. Elle **ne modifie pas l'image**.

11.6 Les boutons de réglage rapide

Les boutons sous l'histogramme posent d'un coup Min, Max et le milieu. Ils s'appellent **Auto**, **Doux**, **Contraste** et **Réinit.** dans le panneau (comme tout le panneau des canaux, ils suivent la langue de l'interface).

BOUTON	MIN	MAX	GAMMA OBTENU
Auto	percentile 0,5 %	percentile 99,5 %	1 (milieu de la fenêtre)
Doux	5	240	0,83
Contraste	20	209	1,22
Réinit.	0	255	1

🔧 **POUR LES CURIEUX**

Auto parcourt l'histogramme (64 colonnes) : Min = dernière colonne dont le cumul reste ≤ 0,5 % des voxels, Max = fin de la dernière colonne dont le cumul reste ≤ 99,5 %. Sans histogramme : Min 0,01 et Max 0,98 (sur 1). Les presets sont exprimés sur l'échelle 0–1 : Doux = 0,02 / 0,94 avec milieu à 0,42 ; Contraste = 0,08 / 0,82 avec milieu à 0,5 ; Réinit. = 0 / 1 / 0,5.

🔢 **MICRO-EXEMPLE**

Auto sur le jeu E9.5 donne Max = 80 pour DAPI, 36 pour Pecam1 et 92 pour Sox2, et Min = 0 : comme 86 à 97 % des voxels sont du fond, le 0,5 % inférieur est entièrement dans la première colonne. **Réinit.** remet 0 / 255.

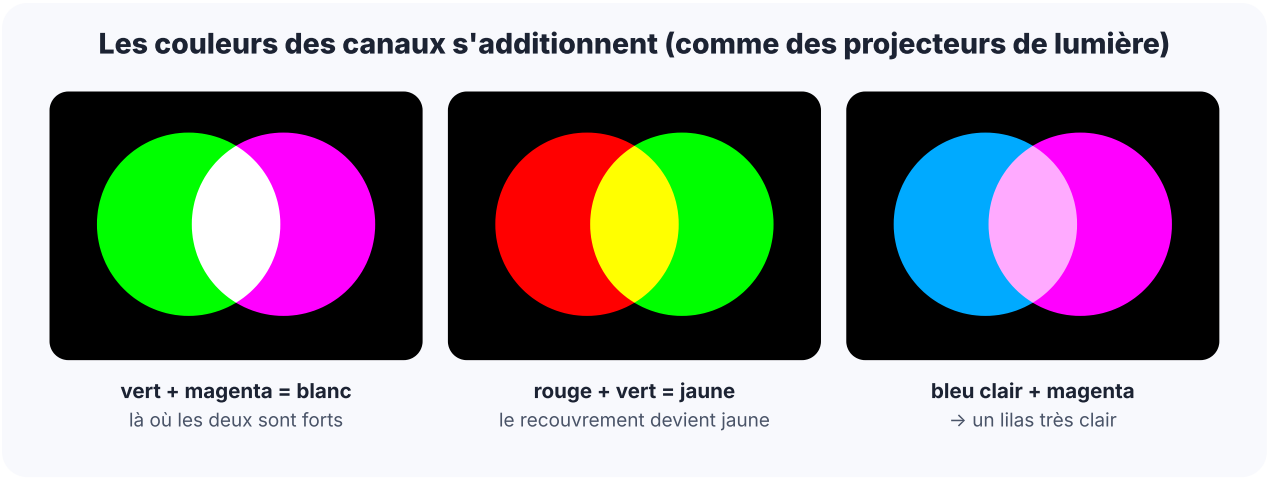
⚠️ **ATTENTION**

Auto reflète la **distribution de tous les voxels**, fond compris. Sur un signal rare, il peut placer Max très bas : l'image sature alors volontairement. Gardez un œil sur l'histogramme et ajustez les poignées à la main si besoin.

11.7 Couleurs, opacité, exposition, solo

Couleur : une addition de lumières

La couleur d'un canal est un simple **multiplicateur RVB**. Les canaux sont **additionnés**, comme des projecteurs : là où deux canaux sont forts, les couleurs se mélangent, puis tout dépassement de 1 est coupé à l'écran.



▲ Mélanges additifs : vert + magenta donne du blanc.

- Le sélecteur propose **27 couleurs** : trois lignes (vives, claires, foncées) de neuf teintes.
- Par défaut (sans couleur dans les métadonnées) : vert, bleu clair, magenta, rouge.
- Le menu des **simulations de déficiences de la vision des couleurs** de l'en-tête applique un filtre à l'écran entier : il sert à vérifier qu'une figure reste lisible, il ne change pas les données.

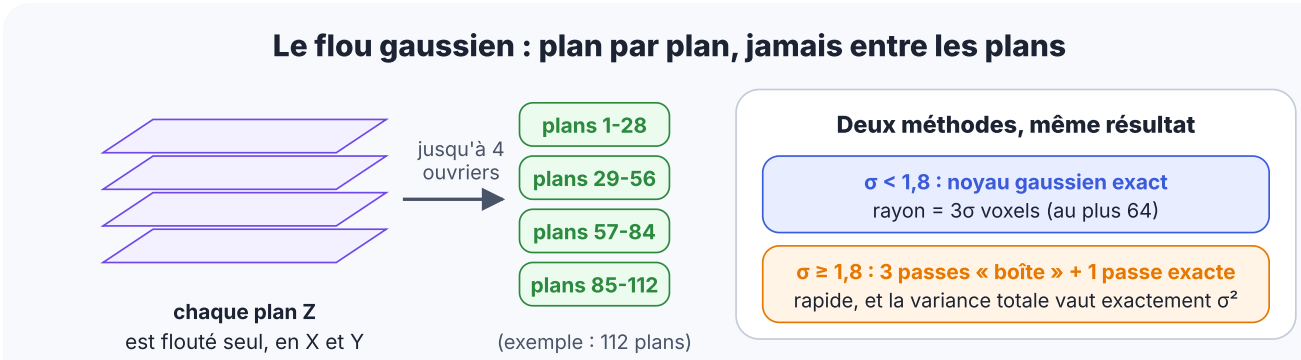
> 👉 **ASTUCE**
>
> Évitez le rouge et le vert côte à côte pour des figures destinées à un public large : choisissez plutôt vert et magenta, qui restent distincts pour la plupart des déficiences.

Opacité, exposition, solo

- **Opacité** : le bouton goutte fait défiler **70 %, 42 %, 100 %**. C'est un multiplicateur d'intensité (0,7 ; 0,42 ; 1), pas une absorption physique.
- **Exposition** : le curseur **Visibilité (exposition)** (de 20 à 500 %, soit ×0,2 à ×5) multiplie la couleur finale ; dans le mode naturel, il multiplie l'émission.
- **Solo** : **Isoler le canal** n'affiche que ce canal ; un deuxième clic (**Afficher tous les canaux**) restaure l'état précédent.
- Un jeu à **plus de 4 canaux** n'en montre que **4** (limite de la texture graphique).

11.8 Le flou gaussien

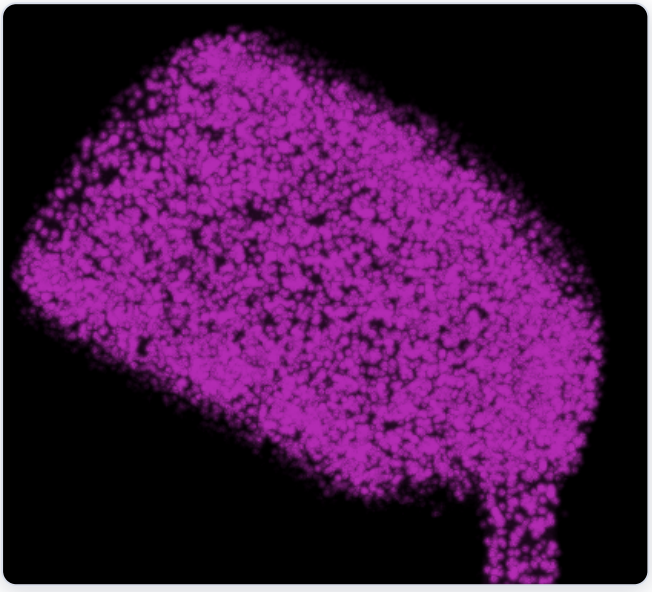
Le curseur **Flou gaussien σ** (de 0 à 5, par pas de 0,1) lisse le bruit **d'un canal**. L'effet est appliqué **au relâchement** du curseur ; ramener σ à 0 restaure le canal d'origine. Le réglage ne modifie que l'affichage.



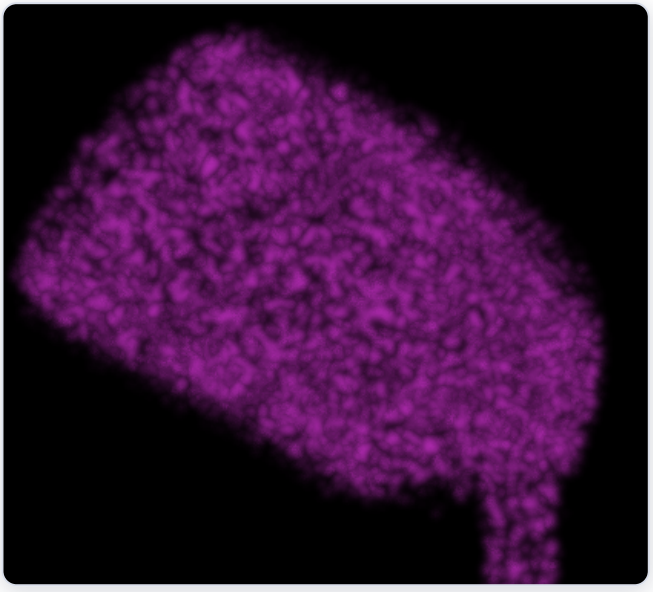
▲ Plan par plan, avec plusieurs ouvriers ; deux méthodes de calcul selon σ.

📌 **À RETENIR**

- **2D dans le plan** : chaque plan Z est flouté séparément, jamais d'un plan à l'autre.
- σ est en **voxels du niveau affiché**.
- Calcul par un groupe de **jusqu'à 4 ouvriers**, hors de la page ; une bulle indique que le calcul est en cours.



▲ Canal Sox2 seul, σ = 0.



▲ Même zone, σ = 2,0.

⚠️ **ATTENTION**

Le flou n'est disponible que si le niveau affiché tient dans **une seule texture dense**. Pour un gros niveau stocké en atlas, le curseur est grisé avec le message : « Indisponible à cette qualité (atlas de briques creux) : choisissez une qualité inférieure pour utiliser le flou. » Utiliser le flou désactive aussi le mode **Détail au zoom** (chapitre 10).

11.9 Garde-fou : ce que les niveaux de gris veulent dire

⚠️ **ATTENTION**

Les niveaux de gris affichés sont relatifs, pas calibrés. Le pipeline a ramené les intensités d'origine (12 ou 16 bits) sur 0–255, entre un fond estimé (99e percentile des coins du volume) et un signal maximal (99,9e percentile) ; le viewer y ajoute son plancher. Une valeur de 120 ne correspond donc à **aucun nombre de photons**.

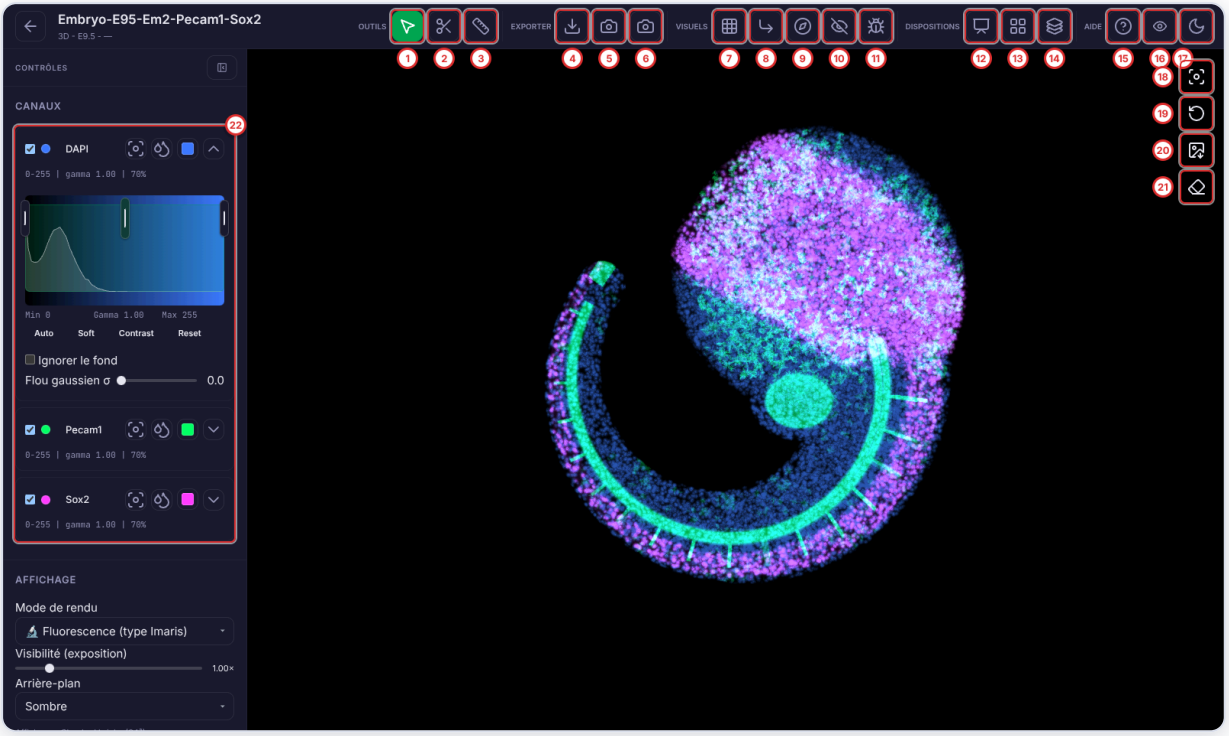
- Comparer **deux canaux** d'un même jeu est délicat : chacun est étiré sur sa propre échelle.
- Comparer **deux jeux de données** par leurs niveaux de gris n'est pas valable, même avec les mêmes réglages.
- Dans une **série temporelle**, la même fenêtre est partagée par tous les pas de temps d'un canal : les variations d'intensité dans le temps restent comparables entre elles.
- Pour une **mesure quantitative**, retournez aux données d'origine (fichier Imaris).

12. Les outils, un par un

- Ce chapitre est le **mode d'emploi** de chaque bouton du viewer, de la page Comparer, de la frise temporelle et de la page 2D.
- Chaque outil suit le même plan : **à quoi il sert**, **comment l'utiliser**, **ce qu'il fait vraiment** (en mots simples) et **ses limites**.
- Toutes les captures montrent le **jeu de démonstration** (embryons synthétiques générés pour cette documentation, pas des données réelles du laboratoire).

12.1 La barre d'outils d'un coup d'œil

Survolez un bouton : une info-bulle donne son nom. La barre est rangée en **cinq groupes** (Outils, Exporter, Visuels, Dispositions, Aide). Quand la fenêtre est étroite ou le titre très long, elle se replie derrière le bouton ☰.



▲ La barre d'outils du viewer, numérotée (jeu de démonstration).

N	CE QUE C'EST	N	CE QUE C'EST
1	**Naviguer** (V ou Échap)	12	**Mode présentation**
2	**Couper à travers le volume** (C)	13	**Décomposer par canal**
3	**Mesurer la distance** (M)	14	**Explorateur Z-Stack**

N	CE QUE C'EST
4	**Centre de téléchargement**
5	Capture en « bac à sable » (plugin de démonstration)
6	**Capture d'écran**
7	**Basculer la Grille (Aucune / Normale / Fine)**
8	**Basculer les Axes**
9	**Basculer les axes d'orientation**
10	**Masquer / Afficher le Volume 3D (conserver les projections)**
11	**Débogage des chunks (arêtes des bricks)**

N	CE QUE C'EST
15	**Aide et méthodes** : un lien vers la page À propos (présentation et méthodes)
16	**Filtres daltonisme**
17	**Changer le thème** (clair / sombre)
18	**Centrer l'échantillon**
19	**Réinitialiser la vue**
20	**Exporter la vue 3D en PNG**
21	**Réinitialiser l'espace de travail (vue, outils, mesures)**
22	Le panneau **Canaux** (voir le chapitre 11)

ℹ️ **À SAVOIR**

Votre barre peut être différente. Les outils sont des *plugins* installés par l'administrateur depuis le catalogue (chapitre 15). Le débogage des chunks, la capture « bac à sable » et les outils 2D sont optionnels. Les outils de suivi cellulaire n'apparaissent que sur une série temporelle suivie.

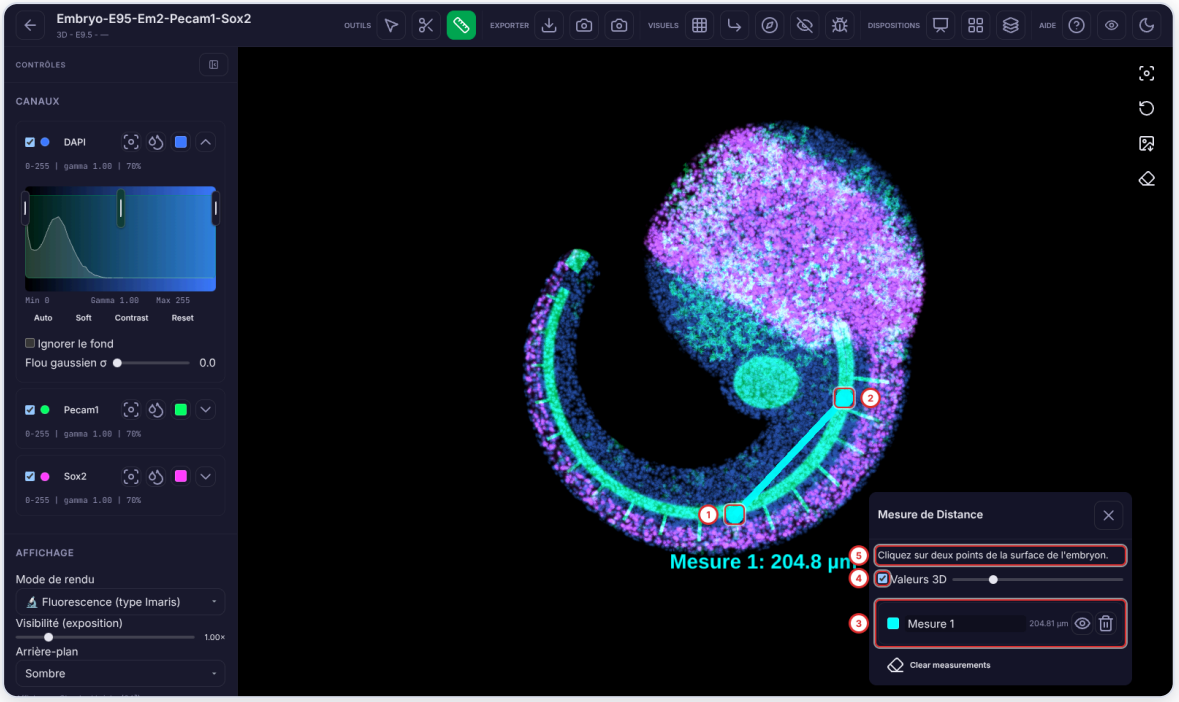
Je veux… → j'utilise…

JE VEUX…	J'UTILISE…	RACCOURCI
Mesurer une distance en µm	**Mesurer la distance**	M
Voir une coupe, même oblique	**Couper à travers le volume**	C
Feuilleter les plans un par un	**Explorateur Z-Stack**	—
Savoir où est l'avant de l'embryon	**Basculer les axes d'orientation**	—
Une figure annotée pour un article	Le Studio (§ 12.7)	—
Une image 3D très grande ou transparente	**Exporter la vue 3D en PNG**	—
Une capture rapide de l'écran	**Capture d'écran**	—
Un panneau par canal	**Décomposer par canal**	—
Récupérer le fichier d'origine	**Centre de téléchargement**	—
Comparer deux embryons	La page Comparer (§ 12.8)	—
Suivre des cellules dans le temps	Les cinq outils de suivi (§ 12.9)	I, D
Partager ce que je vois	Copier l'adresse de la page (§ 12.11)	—

12.2 Mesurer une distance

⏱ EN 30 SECONDES

- **À quoi ça sert** : connaître en micromètres la distance entre deux points du volume.
- Le point ne se pose pas « au hasard » : il se place sur **la première structure vive** sous votre curseur.
- Un clic dans le vide est **refusé**.



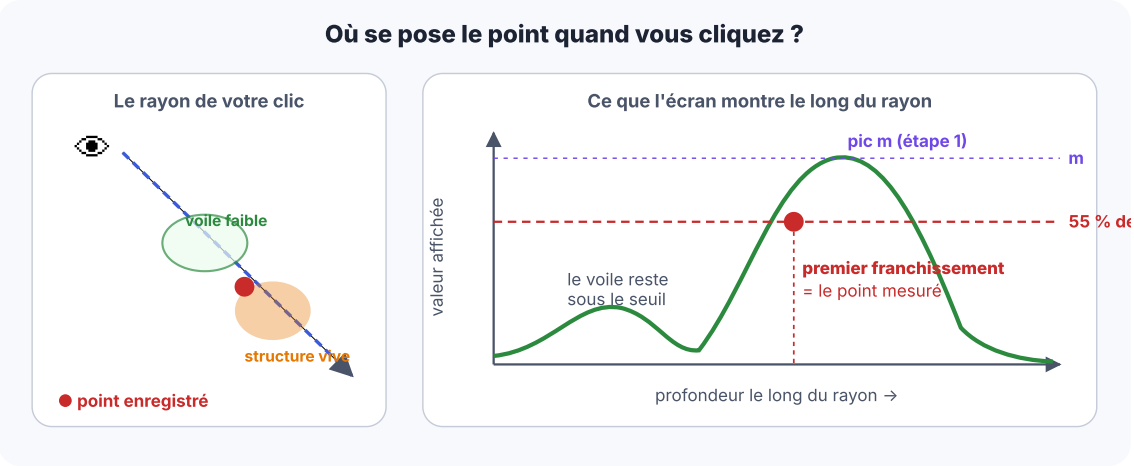
▲ Deux points (1, 2), le segment et la liste des mesures. Jeu de démonstration, 204,8 µm.

N	CE QUE C'EST
1, 2	Les points A et B posés par vos deux clics
3	La liste : couleur, nom modifiable, valeur, œil (masquer), poubelle
4	Case **Valeurs 3D** : affiche ou non l'étiquette dans la vue, avec son curseur de taille
5	La consigne du panneau **Mesure de Distance**

1. Appuyez sur M (ou cliquez la règle). Le panneau **Mesure de Distance** s'ouvre.
2. Cliquez le point A sur une partie **visible** de l'échantillon.
3. Cliquez le point B : la distance et l'écart en Z s'affichent, un segment coloré apparaît.
4. Un troisième clic commence une nouvelle mesure. Glissez pour tourner l'embryon sans poser de point.

Comment le point est trouvé

Le viewer ne garde **aucune copie** du volume en mémoire centrale : il interroge la carte graphique. Pour votre clic, il refait un mini-rendu d'**un seul pixel** avec exactement les mêmes réglages que l'écran (fenêtre, gamma, canaux allumés, rognage).



▲ Le point mesuré est la première structure qui atteint 55 % du maximum affiché.

🔧 **POUR LES CURIEUX**

Deux passes le long du rayon. Passe 1 : le maximum affiché `m`. Passe 2 : le premier point où la valeur affichée atteint `max(0,02 ; 0,55 × m)`, interpolé entre les deux échantillons qui encadrent le seuil. Le plancher 0,02 empêche de « mesurer » du bruit quand tout est presque noir. Conséquence : **éteindre un canal ou changer sa fenêtre déplace le point** (vous mesurez ce que vous voyez).

⚠️ **ATTENTION**

Un clic sans structure dessous (fond noir) est **refusé** : **Aucune structure sous le curseur : cliquez sur une partie visible de l'échantillon.** Sans calibration physique dans les métadonnées, la mesure est aussi refusée. Sur une **paroi de la grille** (§ 12.5), le point est posé sur la paroi, pas sur l'échantillon.

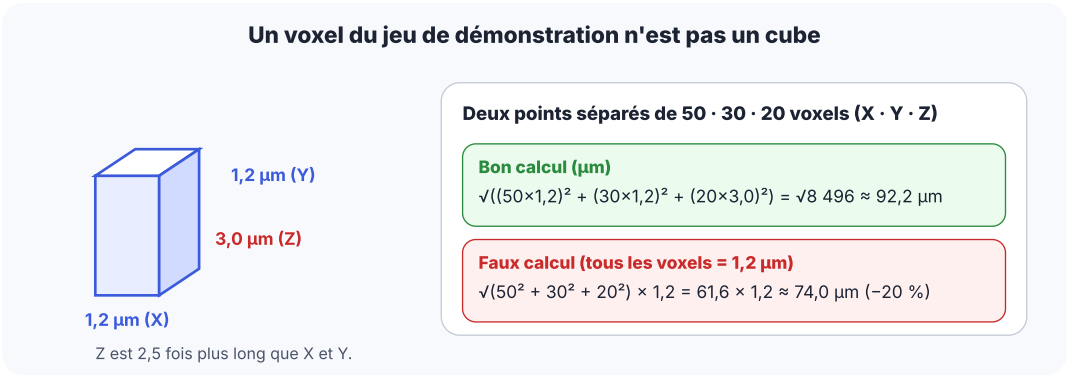
Le calcul, avec de vrais nombres

Les coordonnées normalisées (0 à 1) sont converties en micromètres avec la taille physique du volume (nombre de voxels × taille de voxel). Puis, sur les coordonnées **physiques** :

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}$$

🔢 **MICRO-EXEMPLE**

Les points de la capture : A = (498,5 ; 163,4 ; 211,1) µm et B = (639,3 ; 311,9 ; 203,2) µm. Les écarts valent Δx = 140,8, Δy = 148,5, Δz = 8,0 µm. Donc d = √(19 838 + 22 043 + 63) = √41 944 ≈ **204,8 µm**, la valeur affichée.



▲ Pourquoi on ne peut pas « compter les voxels » : Z est 2,5 fois plus long.

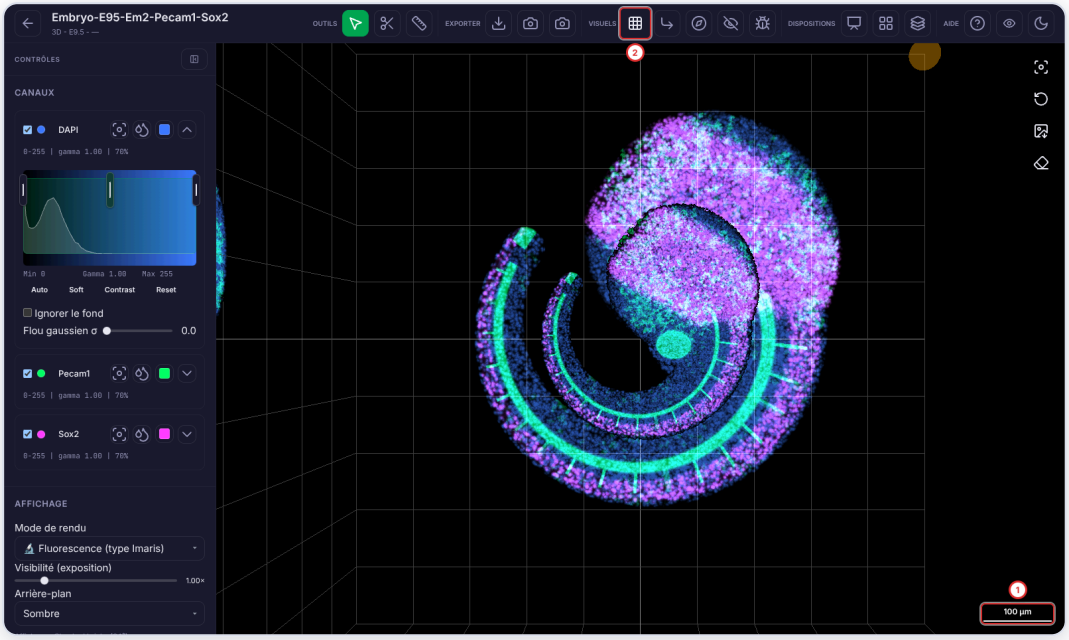
❓ POURQUOI CE CHOIX ?

Le jeu de démonstration a des voxels de 1,2 × 1,2 × 3,0 µm. Compter les voxels comme s'ils étaient cubiques sous-estime ici de 20 % une distance en biais. Le viewer fait toujours le calcul en µm, jamais en voxels.

Où sont gardées les mesures ? Uniquement **en mémoire de la page** : fermer l'onglet les efface. Pour les conserver, copiez l'adresse de la page (elles voyagent dans le lien, § 12.11) ou téléchargez **Mesures CSV** dans le Centre de téléchargement.

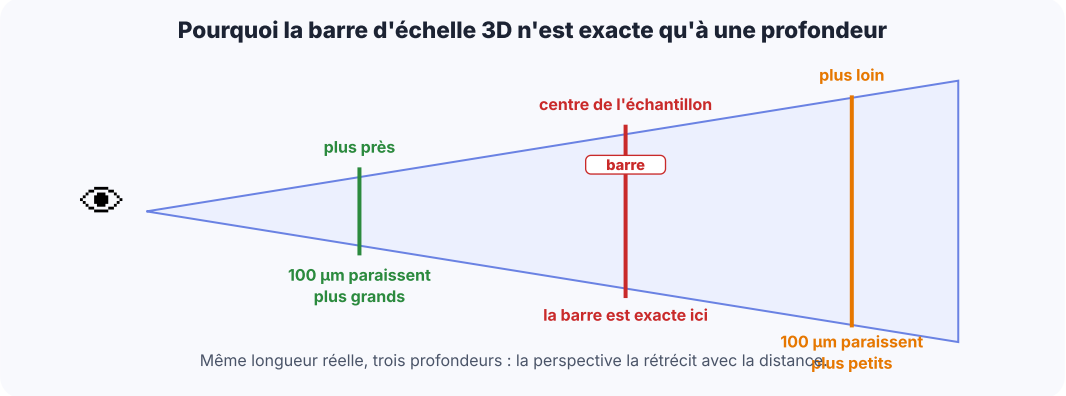
La barre d'échelle de la vue 3D

Allumez la grille (bouton 7) : une barre d'échelle apparaît en bas à droite, avec une longueur « ronde » (règle **1-2-5** : 1, 2, 5, 10, 20, 50, 100 µm…). Elle vise environ 20 % de la largeur de la vue, entre 60 et 200 pixels.



▲ La grille et la barre d'échelle (1) : 100 µm. Le bouton de la grille est le 2.

À cause de la **perspective**, un objet lointain paraît plus petit : la barre n'est exacte qu'à la profondeur du **centre de l'échantillon**.



▲ Même longueur, trois profondeurs : la barre vaut pour le plan central.

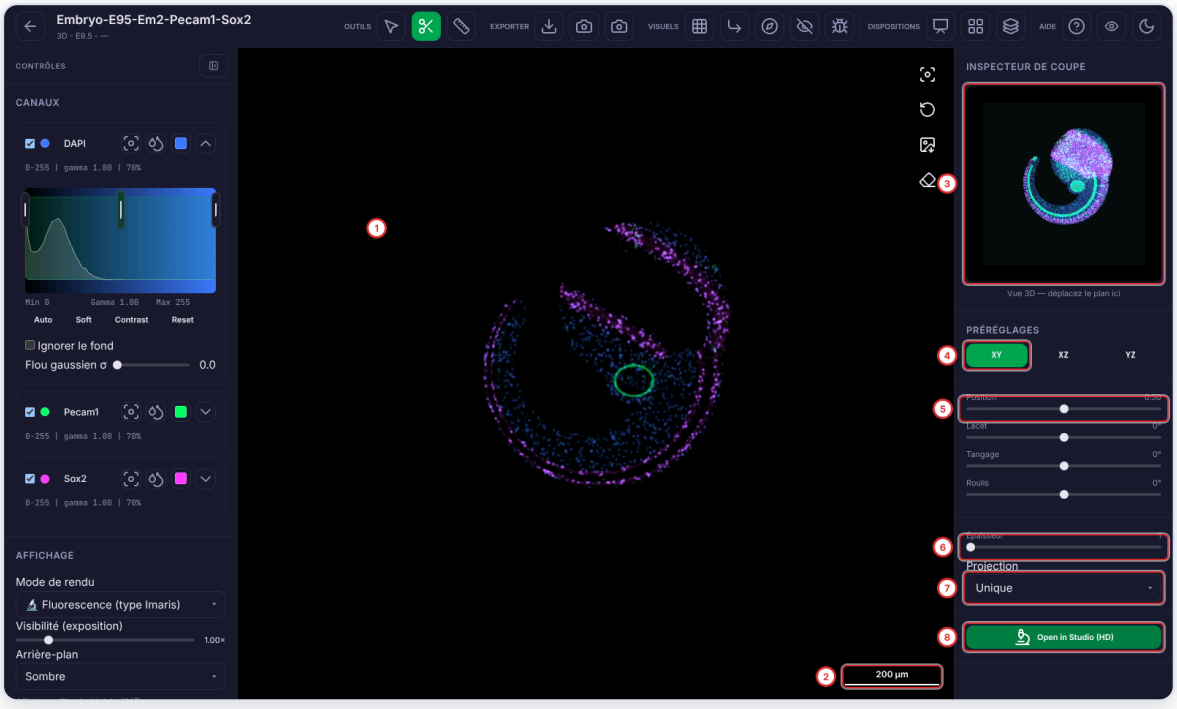
🔧 **POUR LES CURIEUX**

µm par pixel = 2·tan(champ/2) · profondeur · (taille physique X / échelle du cube) / hauteur de la vue en pixels. La barre est masquée sans calibration ou si la grille est éteinte.

12.3 La coupe oblique

⏱ **EN 30 SECONDES**

- **À quoi ça sert** : voir une tranche plane du volume, droite ou inclinée, au lieu de la vue d'ensemble.
- Pendant l'outil, **les deux vues échangent leur place** : la coupe prend tout l'écran, le 3D devient une petite fenêtre.
- On ne peut pas l'utiliser en même temps que l'explorateur Z-Stack.



▲ Coupe XY au milieu du volume (jeu de démonstration).

N	CE QUE C'EST
1	La coupe, calculée par la carte graphique, avec les couleurs de vos canaux
2	Barre d'échelle exacte de la coupe (200 µm ici)
3	Le volume en miniature : on y déplace le plan
4	Préréglages **XY**, **XZ**, **YZ**

N	CE QUE C'EST
5	**Position** du plan, puis **Lacet**, **Tangage**, **Roulis**
6	**Épaisseur** : nombre d'échantillons dans la tranche
7	**Projection** : **Unique**, **MIP** ou **Moyenne**
8	Bouton vers le Studio (§ 12.7)

1. Appuyez sur C : un plan translucide apparaît, la coupe remplit l'écran.
2. Choisissez un préréglage, ou déplacez le plan dans la miniature (molette sur le plan : 2 % du volume par cran).
3. Les curseurs **Lacet**, **Tangage**, **Roulis** rendent le plan **oblique**.
4. Épaississez la tranche et choisissez comment l'écraser : une seule coupe, le plus clair (MIP) ou la moyenne.

🔧 **POUR LES CURIEUX**

La coupe lit le **même atlas** que la vue 3D, donc mêmes fenêtres, gamma et couleurs. Le plan est la rotation Ry(−lacet)·Rx(−tangage)·Rz(roulis) dans le repère aux proportions physiques ; comme les voxels sont plus longs en Z, la normale du plan est corrigée ($n_{tex} \propto S^{-1} \cdot n$). Pendant que vous déplacez le plan, l'image fait 1024 px ; une fois immobile (160 ms), elle est affinée jusqu'à 2048 px. Une épaisseur compte en échantillons (au plus 1024), pas en µm.

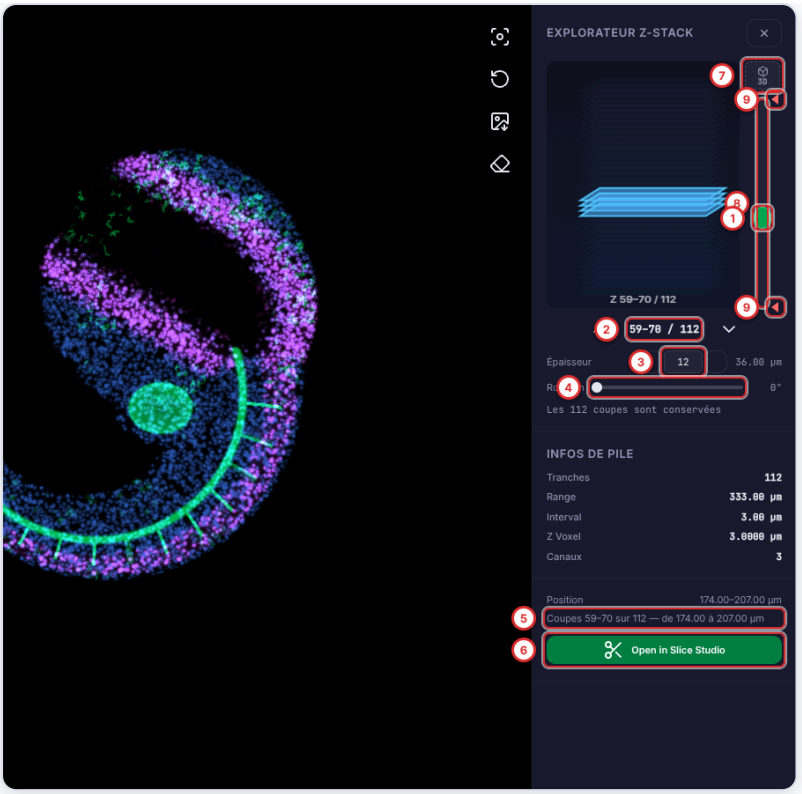
⚠️ **ATTENTION**

Les préréglages XY/XZ/YZ suivent les axes **du fichier**, pas ceux de l'embryon, sauf si le jeu de données a été calibré (§ 12.5).

12.4 L'explorateur Z-Stack

⏱ **EN 30 SECONDES**

- **À quoi ça sert** : feuilleter les plans de la pile comme dans un logiciel de microscopie.
- À l'ouverture, la pile **se couche à plat** en 1,5 s, face supérieure vers vous.
- Le curseur peut montrer une coupe, plusieurs coupes (une « épaisseur ») ou toute la pile en 3D.



▲ Z-stack en mode coupe : 12 coupes (59 à 70), soit 36 µm.

N	CE QUE C'EST
1	Le **curseur** : sa hauteur = nombre de coupes montrées
2	**59–70 / 112** : coupes affichées / total
3	**Épaisseur** : 12 coupes = 36,00 µm
4	**Rotation** 0–360°
5	Position : 174 à 207 µm de profondeur
6	**Ouvrir dans le Studio de coupe** : ouvrir la tranche dans le Studio
7	Le **cran 3D** : curseur ici = toute la pile en 3D
8	La **piste** : une position par coupe
9	**Triangles de rognage** (dessus / dessous)

1. Cliquez **Explorateur Z-Stack** : la pile se couche à plat, en mode 3D (cran 7).
2. Tirez le curseur **vers le bas** dans la piste (8) : la vue se verrouille de dessus sur la coupe choisie.
3. Tirez un **bord** du curseur, ou le bouton **Épaisseur**, pour montrer plusieurs coupes.
4. Les triangles (9) masquent des coupes en haut ou en bas (**Les 112 coupes sont conservées** devient **Conservées…**). Remontez le curseur dans le cran 7 pour revenir au 3D.

🔧 **POUR LES CURIEUX**

Une seule opération rogne le volume en Z : `setClipRange_z` avec l'intervalle `[début / z ; (fin+1) / z]`. Le calcul de rendu se confine à cette boîte et corrige l'émission (« gain de tranche mince ») pour qu'une coupe unique ne soit pas noire. Profondeur = indice × taille de voxel Z (3 µm ici). Au clavier, curseur sélectionné : ↑ ↓ une coupe, PageHaut/PageBas 10 % de la pile, Début = cran 3D, Fin = dernière coupe, + / − épaisseur.

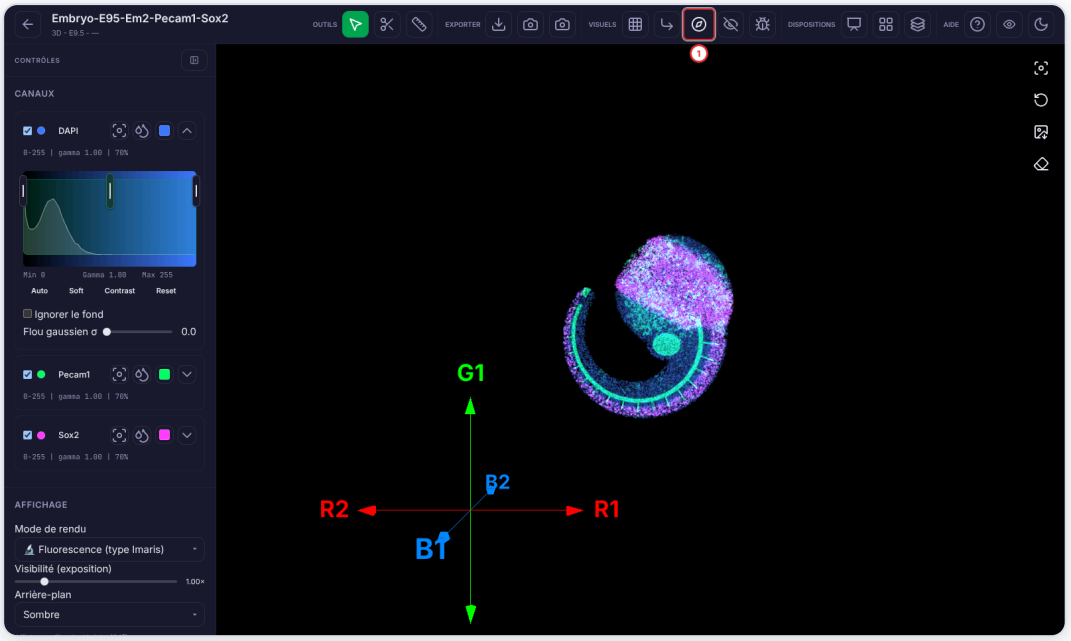
Échantillon « à l'envers ». Certains fichiers Imaris montrent l'échantillon vu de dessous. L'administrateur l'indique dans les métadonnées (`upsideDown`) : la « face supérieure » est alors la face −Z, et le Z-stack, ainsi que les figures du Studio, la présentent du bon côté. La calibration d'orientation n'en dépend pas.

12.5 Les axes d'orientation, la grille et les affichages

⏱ EN 30 SECONDES

- **Axes d'orientation** : une boussole de l'embryon (R1/R2, G1/G2, B1/B2).
- **Grille** : trois murs gradués avec projections ; **Axes** : un repère XYZ déplaçable.
- **Volume** : masque le volume 3D en gardant les projections.

Les axes d'orientation



▲ La boussole : rouge R1/R2 (±X), vert G1/G2 (±Y), bleu B1/B2 (±Z).

Chaque bras a un **nom court** (R1, R2, G1, G2, B1, B2). L'administrateur peut les renommer (« Antérieur », « Dorsal »…) ou en masquer. Glissez la petite sphère centrale pour déplacer la boussole ; sa position est conservée dans l'espace de travail.

💡 UNE IMAGE POUR COMPRENDRE

Une calibration d'orientation, c'est un geste de la main. Le microscope enregistre l'embryon dans l'orientation où il était posé, pas « tête en haut ». La calibration dit : « pour remettre l'embryon à l'endroit, faites tourner le fichier d'**un** mouvement, autour de **cet** axe, de **tant** de degrés ». L'ordinateur range ce geste sous forme de quatre nombres (un *quaternion*). Vous n'avez rien à régler : l'administrateur la fait une fois, dans le panneau d'administration.

La vue par défaut est une pose de l'embryon (par exemple « face ventrale, antérieur en haut ») enregistrée avec le jeu de données : elle est appliquée **avant** le chargement de la première brique, et le bouton **Réinitialiser la vue** y revient. Les préréglages proposés sont ventral, dorsal, gauche, droite, antérieur, postérieur.

🔧 **POUR LES CURIEUX**

Le fichier a sa rotation `Q_base` (axes du fichier → axes anatomiques). La boussole est dessinée avec `Q_anat = Q_cube · Q_base⁻¹`, et une vue par défaut est stockée sous la forme `Q_anat`, appliquée en `Q_cube = Q_anat · Q_base`. La calibration n'est modifiable que depuis l'administration (aperçu ↔ viewer par messages) ; un visiteur ne peut ni la changer, ni enregistrer une vue par défaut.

La grille, les axes, le volume

- **Grille** (7) : un clic fait défiler *Aucune → Normale (10 divisions) → Fine (40) → Aucune*. Les trois murs montrent une **projection** du volume avec le même mode de rendu. Une poignée orange redimensionne un mur (double-clic : retour).
- **Axes** (8) : un repère XYZ coloré, à ne pas confondre avec la boussole. Glissez sa sphère pour le déplacer.
- **Volume** (10) : cache le volume ray-marché ; la grille, les axes, les mesures et le suivi restent.

12.6 Capture, export, présentation, téléchargement, canaux, chunks

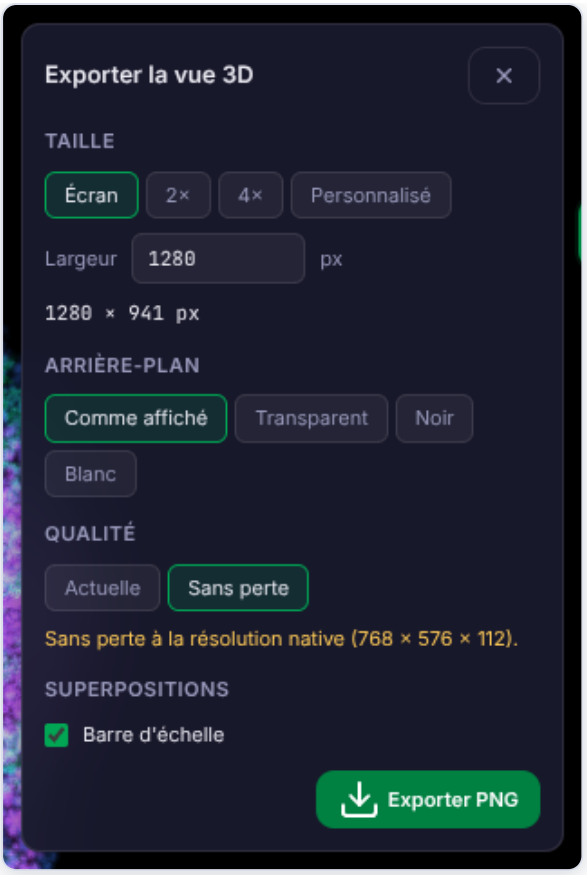
⏱ **EN 30 SECONDES**

- Trois façons d'obtenir une image : **Capture d'écran** (rapide), **Exporter la vue 3D en PNG** (grande taille), le Studio (figure annotée).
- Le **Centre de téléchargement** ne donne pas des captures : il donne les **fichiers** du jeu de données.

Capture d'écran et export de la vue 3D

La **capture** (bouton 6) enregistre en un clic ce qui est à l'écran, au format PNG (`<jeu>_screenshot.png`), à la résolution de l'écran.

L'**export de la vue 3D** (bouton 20) refait le rendu **à n'importe quelle taille**.



▲ La fenêtre d'export de la vue 3D (bouton 20 de la barre), avec le choix de qualité.

- **Taille** : **Écran**, 2×, 4×, **Personnalisé** (largeur de 16 à 16 384 px).
- **Arrière-plan** : comme affiché, **Transparent**, noir, blanc.
- **Barre d'échelle** : désactivée sans calibration (elle compterait des voxels, pas des µm).
- **Qualité** : **Actuelle** (la case affichée) ou **Sans perte**. *Sans perte* charge la colonne sans perte du niveau natif, ou du niveau le plus fin que la carte graphique peut contenir (la fenêtre dit lequel), rend l'image, puis remet la qualité précédente.
- Fichier : `<jeu>_3d_<L>x<H>.png`.

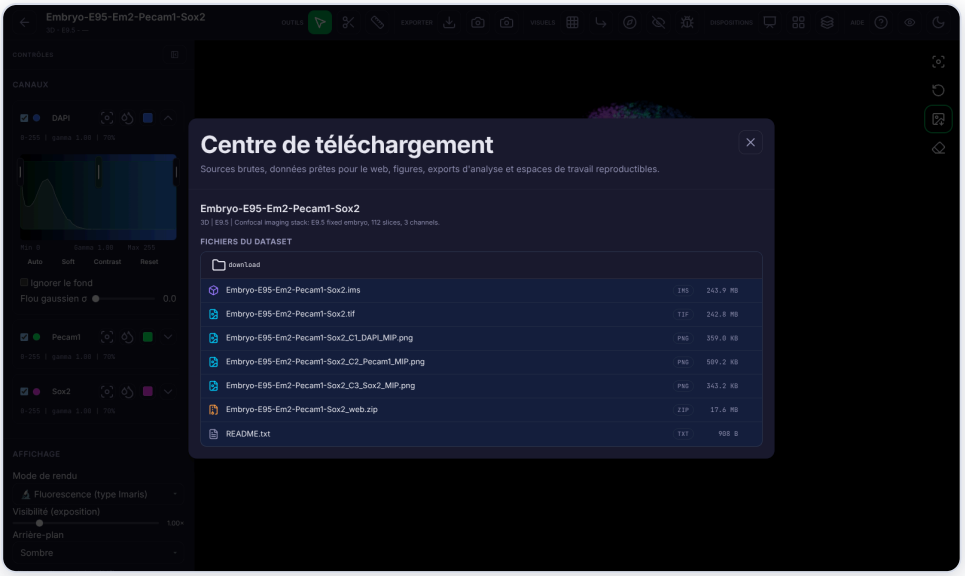
🔧 **POUR LES CURIEUX**

Une carte graphique ne sait pas dessiner d'un coup une image géante. Le viewer attend la fin du chargement, puis rend la vue **par tuiles** d'au plus 2048 px (chacune est une fenêtre du champ de vision, une tuile par tâche pour ne pas figer la page) avec le nombre de pas d'un rendu au repos. Le fond transparent écrit une opacité par pixel `a = max(opacité, max(canaux))` ; sur noir, il redonne l'écran. Si l'affichage change pendant le rendu, il recommence (3 essais). Plafond : 16 384 px de côté, 268 mégapixels.

Mode présentation

Le bouton 12 masque l'interface pour que le volume remplisse l'écran : utile pour projeter ou filmer. Même bouton pour revenir.

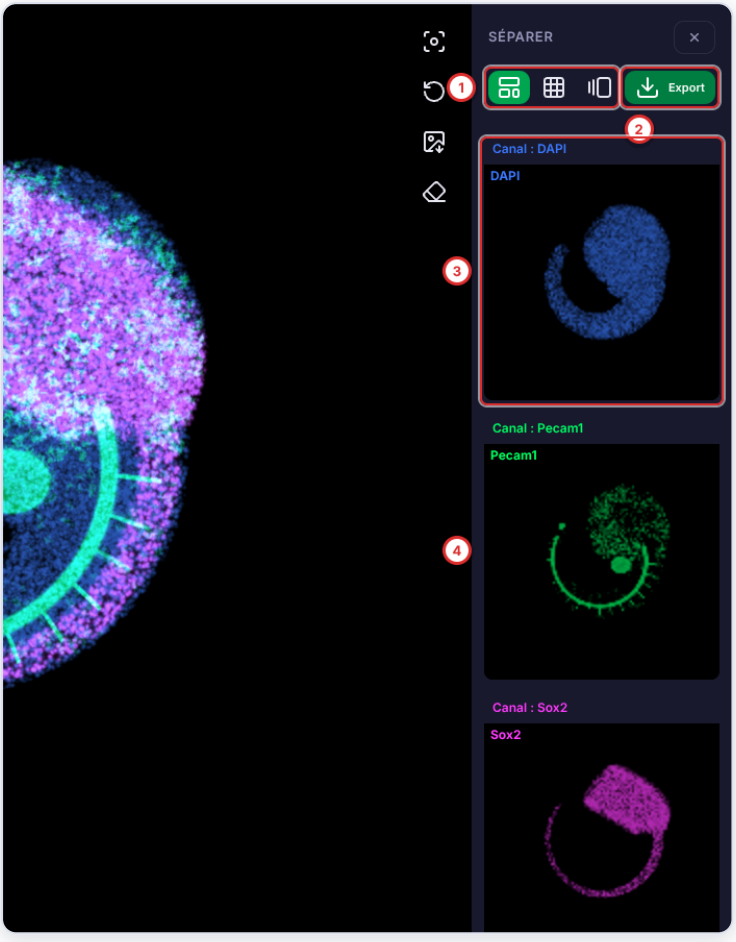
Centre de téléchargement



▲ Les fichiers du dossier `download/` d'un jeu de données (jeu de démonstration).

Dans le viewer, il affiche une **liste de fichiers** : le fichier Imaris d'origine, le TIFF calibré, les projections par canal (PNG), l'archive web, un fichier README. Ils n'existent que si le pipeline a été lancé avec les téléchargements (chapitre 8). Ces fichiers sont toujours servis en **pièce jointe**. S'il y a des mesures, un bouton **Mesures CSV** apparaît (colonnes : étiquette, type, distance, unité, point A, point B…).

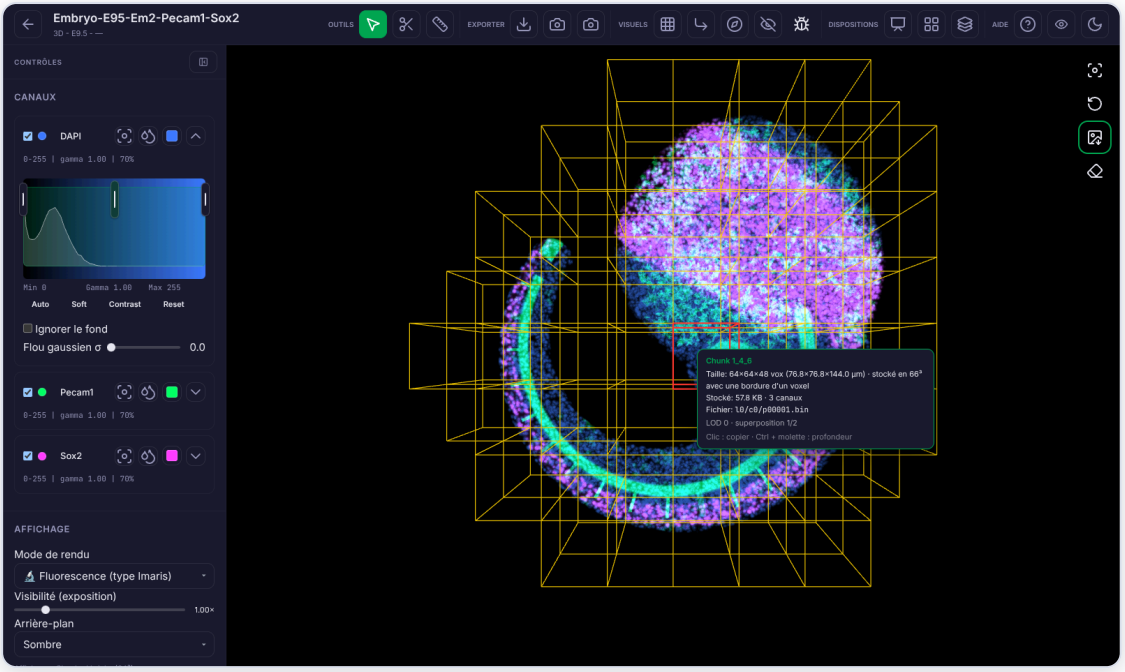
Décomposer par canal



Chaque canal allumé reçoit sa **vignette** (le même volume, avec ce canal seul). Cliquer une vignette permet de régler ce canal dans le panneau de gauche (**Terminé** pour sortir, **Revenir à l'original**). Les trois boutons (1) choisissent la disposition ; **Exporter** (2) écrit un PNG `decomposition_<disposition>_<date>.png`. Chaque vignette est rendue à la taille du volume (512 à 4096 px) et le viewer refuse une image trop grande pour la carte graphique (« Trop de vues pour une seule image »).

▲ Une vignette par canal : DAPI, Pecam1, Sox2.

Débogage des chunks



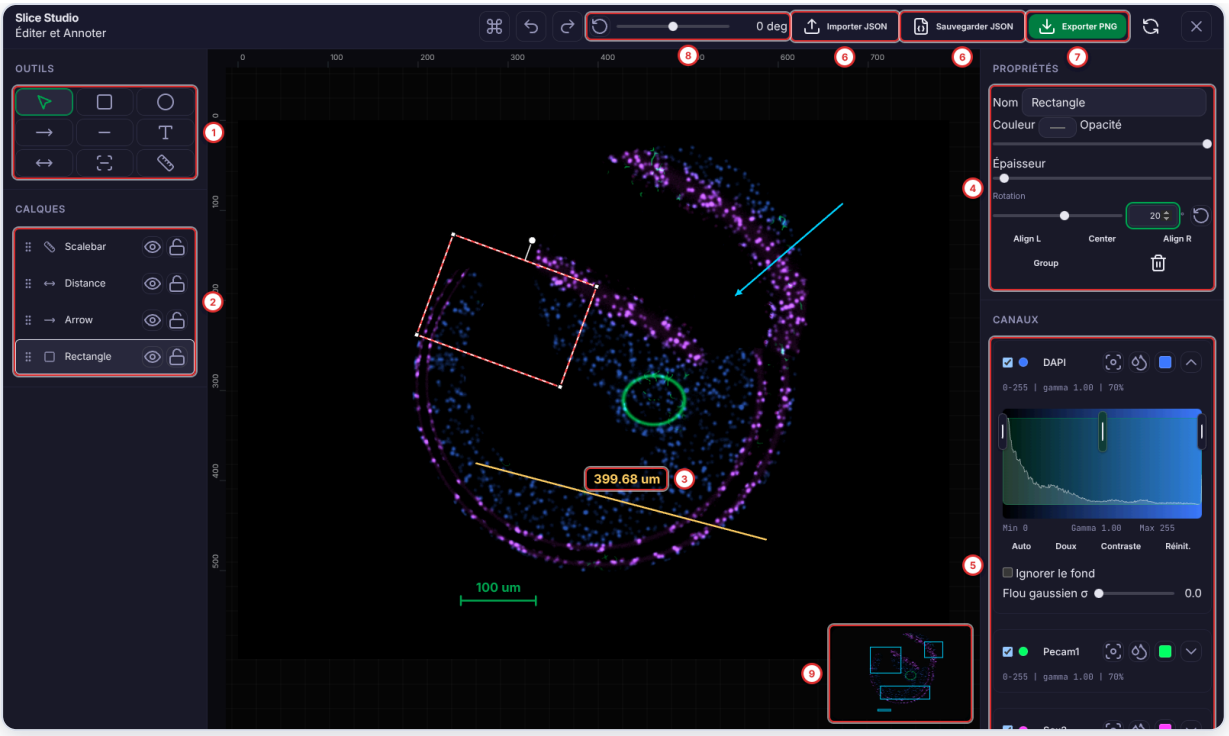
▲ Les briques de 64^3 voxels dessinées en jaune ; survol de l'une d'elles.

L'outil le plus pédagogique du chapitre 7 : il **dessine les arêtes de chaque brique non vide** du niveau affiché. Survolez une brique : son identifiant, sa taille (ici 64×64×48 voxels, 76,8 × 76,8 × 144 µm), sa taille stockée (66^3 avec une bordure d'un voxel), son fichier de pack et son niveau de détail. Clic : copie ces informations. Ctrl + molette : parcourt les briques superposées sous le curseur.

12.7 Le Studio : annoter et exporter une figure

⏱ EN 30 SECONDES

- **À quoi ça sert** : fabriquer une figure de publication à partir d'une coupe (barre d'échelle, flèches, mesures, texte).
- Il s'ouvre **tout de suite** sur un aperçu, puis recharge la coupe à la **résolution native**.
- Les canaux se **recolorient** dans le Studio sans refaire le rendu du volume.



▲ Le Slice Studio avec un rectangle tourné de 20°, une flèche, une distance, une barre d'échelle.

N	CE QUE C'EST
1	Outils : sélection, rectangle, ellipse, flèche, ligne, texte, distance, angle, barre d'échelle
2	Calques (réordonner, masquer, verrouiller)
3	Une mesure de distance : 399,68 µm (étiquette calculée en µm)
4	**Propriétés** du calque choisi : nom, couleur, opacité, épaisseur, **Rotation** (−180 à 180°)
5	**Canaux** : couleur, min/max, gamma, on/off, avec leurs histogrammes

N	CE QUE C'EST
6	**Importer JSON** / **Sauvegarder JSON**
7	**Exporter PNG**
8	Rotation de la **vue** (affichage seulement)
9	Mini-carte

1. Dans la coupe oblique ou le Z-stack, cliquez le bouton d'ouverture du Studio.
2. Annotez : chaque outil a une lettre (tableau § 12.11). Une barre d'échelle est posée à l'ouverture si le volume est calibré.

3. Recolorez les canaux si besoin (colonne de droite).
4. **Exporter PNG** pour l'image, **Sauvegarder JSON** pour pouvoir reprendre plus tard.

Les calques

- **Distance** : l'étiquette vaut `√((Δx·px_x)² + (Δy·px_y)²)` µm, avec la taille de pixel de la figure. **Angle** : trois clics, calculé en µm. **Barre d'échelle** : sa longueur (µm, mm, cm ou px) est convertie en pixels.
- **Rotation d'un calque** : un rectangle, une ellipse ou un texte tourne autour de son centre ; une ligne, une flèche ou une distance voient leurs points tournés (la distance garde sa valeur en µm). Curseur, champ, poignée au-dessus de la sélection (Maj = pas de 15°), ou `[` / `]`.
- L'historique garde 80 étapes (Ctrl+Z).

La passe « native »

L'aperçu (au plus 2048 px) vient de la carte graphique. La passe native va chercher les **voxels de la résolution maximale** pour ce plan précis, et remplace l'image pendant qu'elle arrive. Une barre indique octets, chunks et temps restant, avec **S'arrêter ici**.

- Une coupe **XY** d'un jeu de format 2 ou plus lit le dossier `planes/` : uniquement les voxels du plan.
- Un plan **oblique** ou un volume ancien passe par les briques (lectures d'octets ciblées, chapitre 10).
- Une brique absente parce que vide est comptée comme **zéro**, jamais comme « manquante ».
- Si une figure demande plus de 256 Mo, une boîte propose natif, résolution réduite ou aperçu.
- Sur un jeu au **format 5**, la passe native lit les **copies de coupe** (section 7.11) : une coupe XZ ou YZ = une ligne du paquet `xz/` ou `yz/`, trois petites lectures par canal ; une coupe oblique = seulement les petites briques de 32 × 32 × *bz* que le plan traverse. Si une copie manque ou échoue, la passe retombe sur les briques. Les images obtenues sont identiques, voxel pour voxel.

> ℹ️ **À SAVOIR**
>
> Sur le jeu de démonstration (768 × 576 px), le natif est aussi petit que l'aperçu : la barre de progression n'a presque rien à charger. Elle prend tout son sens sur un volume de plusieurs gigaoctets.

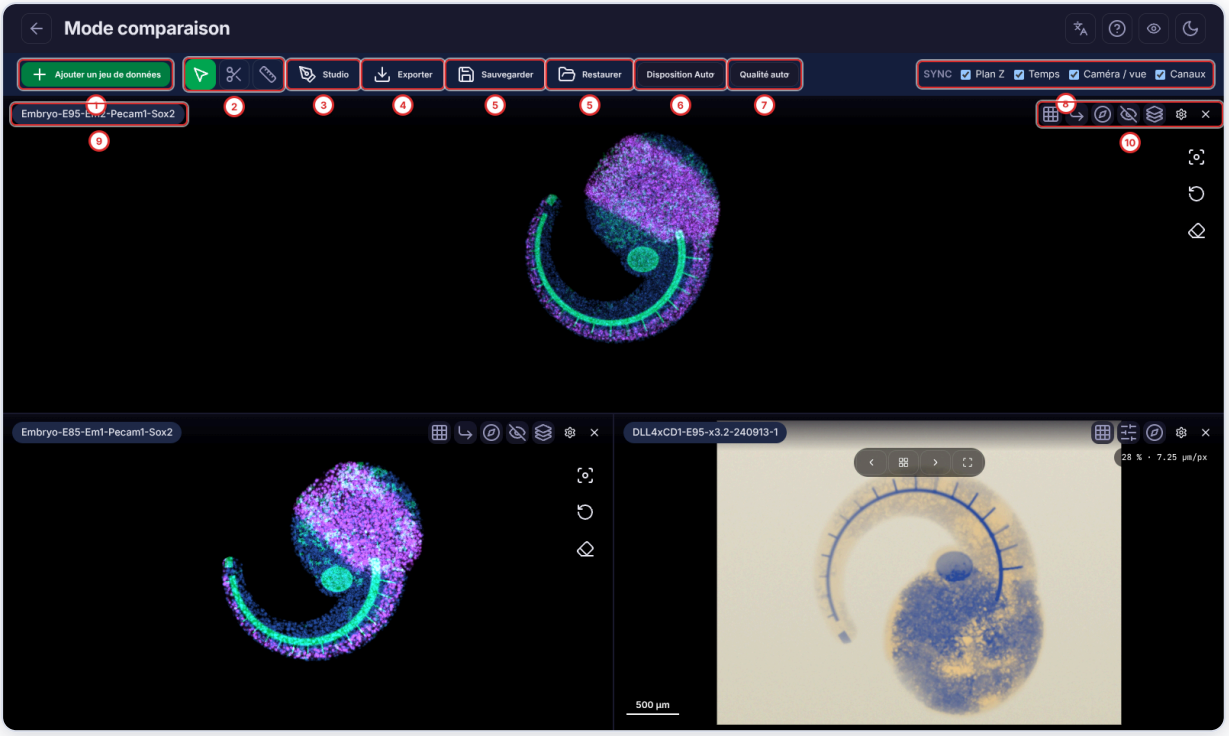
Exports et limites

- **PNG** : taille native, fond noir, avec une **ligne de légende toujours incrustée** en haut à gauche (jeu | plan | taille | pixel). Refusé au-delà de 16 384 px de côté ou 2^{28} pixels.
- Avant l'export PNG, une question : **Native sans perte** ou **Cette image**. *Native sans perte* attend la passe native (ou la relance si vous l'aviez arrêtée) ; les calques sont remis à l'échelle de la nouvelle image et **gardent leur place**. *Décomposer par canal* pose la même question.
- **JSON** (`<jeu>_studio.json`) : calques, repères, canaux, plan, calibration. **Jamais de pixels.** À l'import, un fichier de plus de **5 Mo**, 2 000 calques ou 10 000 points est refusé ; un fichier d'une autre figure n'applique que les calques.
- Au-delà de 4 canaux, seuls les 4 premiers sont montrés. Sans calibration : pas de barre d'échelle, mesures en pixels.

12.8 La page Comparer

⏱ EN 30 SECONDES

- **À quoi ça sert** : afficher jusqu'à **quatre** jeux de données côte à côte, en gardant les vues synchronisées.
- Chaque panneau est la **vraie page** du viewer (ou de la page 2D) : mêmes outils, mêmes raccourcis.
- Une mémoire graphique unique est **partagée** entre les panneaux (qualité automatique).



▲ Trois panneaux : deux volumes et une photographie. (1) ajouter, (2) outils communs, (8) synchronisations.

N	CE QUE C'EST
1	**Ajouter un jeu de données** (recherche, filtre par type)
2	Outils communs : ce qu'offrent les pages ouvertes
3	**Studio** : une figure avec tous les panneaux
4	**Exporter** : figure PNG / WebP de la grille
5	**Sauvegarder** / **Restaurer** l'espace de travail

N	CE QUE C'EST
6	Disposition : Auto, colonnes, lignes, grille (gouttières réglables)
7	Qualité : auto, 512, 1024, natif
8	SYNC : **Plan Z**, **Temps**, **Caméra / vue**, **Canaux**
9	Le nom du jeu de données du panneau
10	Les boutons à bascule **décrits par la page** de ce panneau

Ce qui est synchronisé

OPTION	CE QUI VOYAGE ENTRE PANNEAUX
Plan Z	coupe du Z-stack, plan de la coupe oblique
Temps	position dans la série, **en fraction** (frame = fraction × (N − 1)) : 100 images et 10 images restent alignées
Caméra / vue	angle de vue 3D (corrigé de la calibration de chacun) ; pour une photographie, l'échelle physique
Canaux	état et exposition, **par nom de canal**

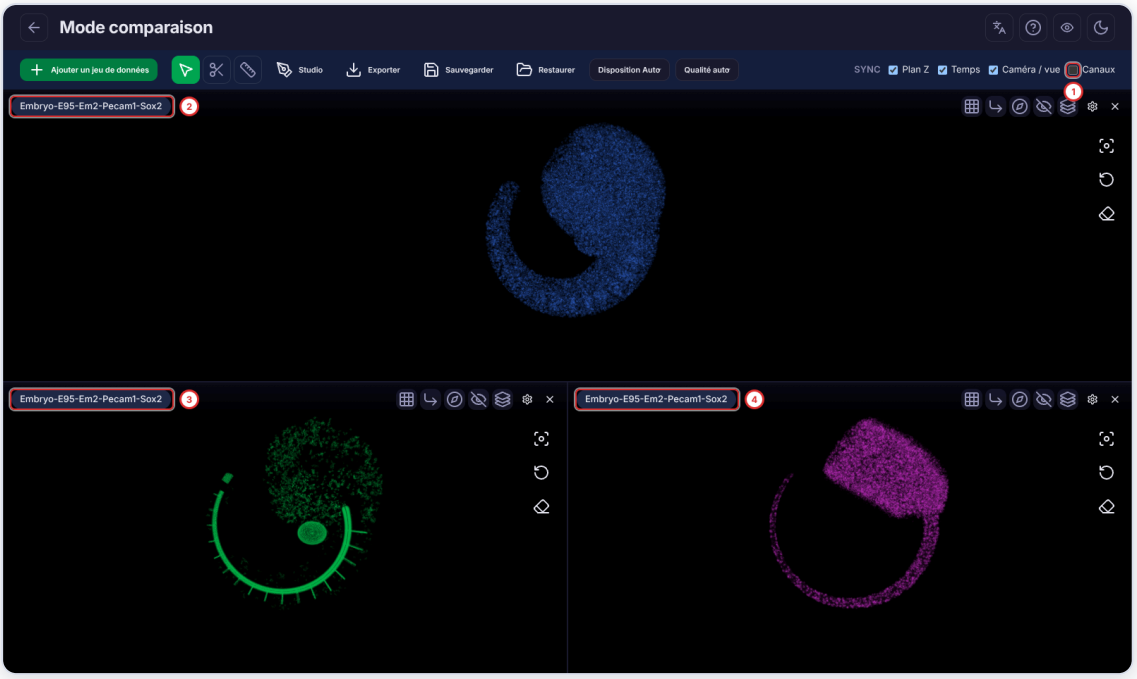
Un panneau ajouté tard est « rattrapé » : il reçoit la dernière caméra, le dernier temps, etc.

La mémoire partagée

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un buffet à budget fixe.** Comparer retient un budget commun **fixe** de 1,5 Gio pour l'ensemble des panneaux (ce n'est pas une mesure de votre carte graphique). Chaque volume dit ce que coûte chaque qualité. Le viewer rabaisse le plus gourmand tant que le total dépasse, puis remonte le plus modeste tant que ça rentre, sans dépasser 1024 en automatique. Ce réglage ne concerne que les **volumes** : une photographie 2D n'a pas de qualité.

Les panneaux démarrent en 512 ; la montée de qualité se fait **un panneau à la fois** ; un panneau qui ne se stabilise pas en 180 s est abandonné (message dans la console) et le suivant est traité. Choisir 512/1024/Natif à la main force le niveau.

Décomposer, figure, espace de travail



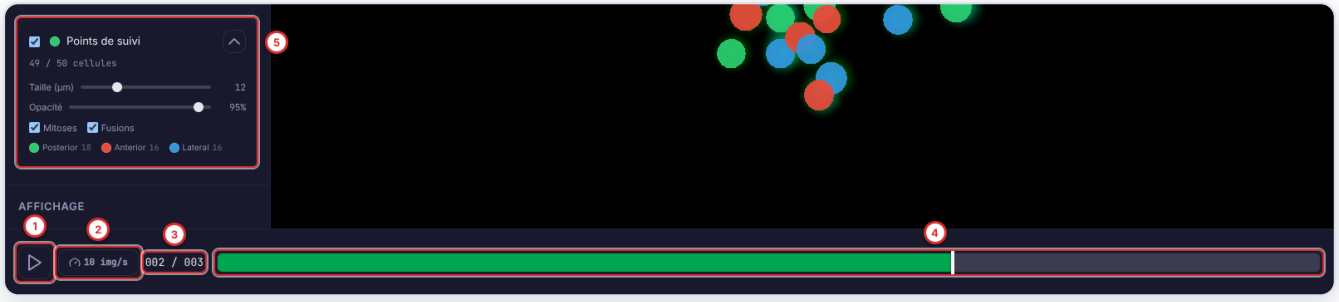
▲ Un volume à trois canaux, **Décomposer** : trois panneaux, un canal chacun. La synchro **Canaux** (1) a été décochée automatiquement.

- **Décomposer** n'apparaît que s'il y a **un seul** panneau volumique de plusieurs canaux : il le clone en panneaux (4 au plus), un canal chacun, en gardant temps et caméra synchrones.
- **Studio** : une figure avec tous les panneaux, en **Taille Visuelle** ou en **Échelle Physique** (une seule échelle en µm/px pour tous, jamais sur-échantillonnée).
- **Exporter** compose la grille visible (4096 px de côté au plus).
- **Sauvegarder** garde l'espace de travail dans le navigateur ; **Restaurer** le recharge. L'adresse de la page contient aussi l'état (`#state=…`).

12.9 La frise temporelle et le suivi cellulaire

⏱ EN 30 SECONDES

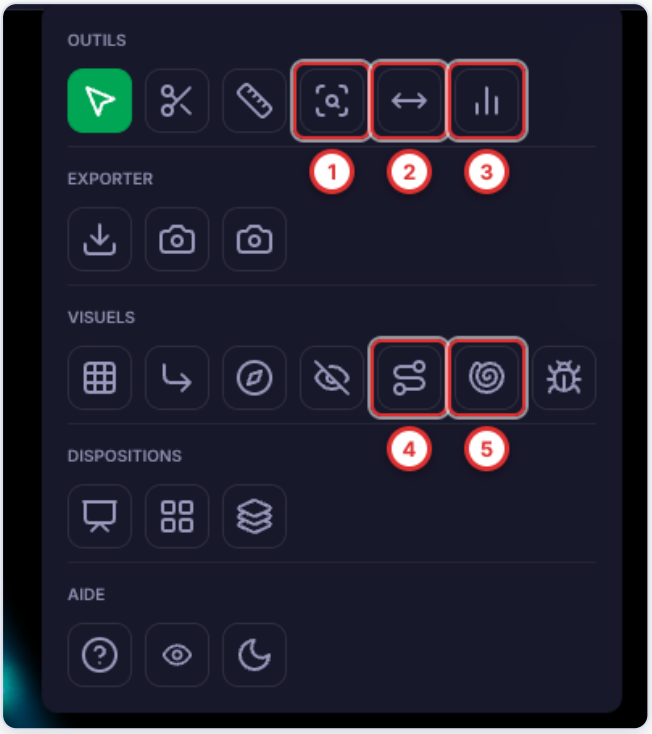
- Une **série temporelle** (type *Live*) ajoute une frise en bas de l'écran pour la lire.
- Si elle a un suivi cellulaire, **cinq outils** s'ajoutent : trajectoires, surface, inspecteur, distance entre cellules, graphiques.
- Exemple : le jeu de démonstration de 4 images avec suivi (50 cellules).



▲ La frise : lecture (1), vitesse (2), compteur (3), piste (4) ; et la couche **Points de suivi** (5).

N	CE QUE C'EST
1	Lecture / pause
2	Vitesse : un clic fait défiler 0,5 → 1 → 2 → 5 → 10 → 20 img/s
3	Compteur : image en cours / dernière image (000 / 003 = 4 images)
4	Piste : glisser pour aller à une image ; la bande indique les images déjà chargées
5	Couche de points de suivi : taille, opacité, mitoses, fusions, légende par région

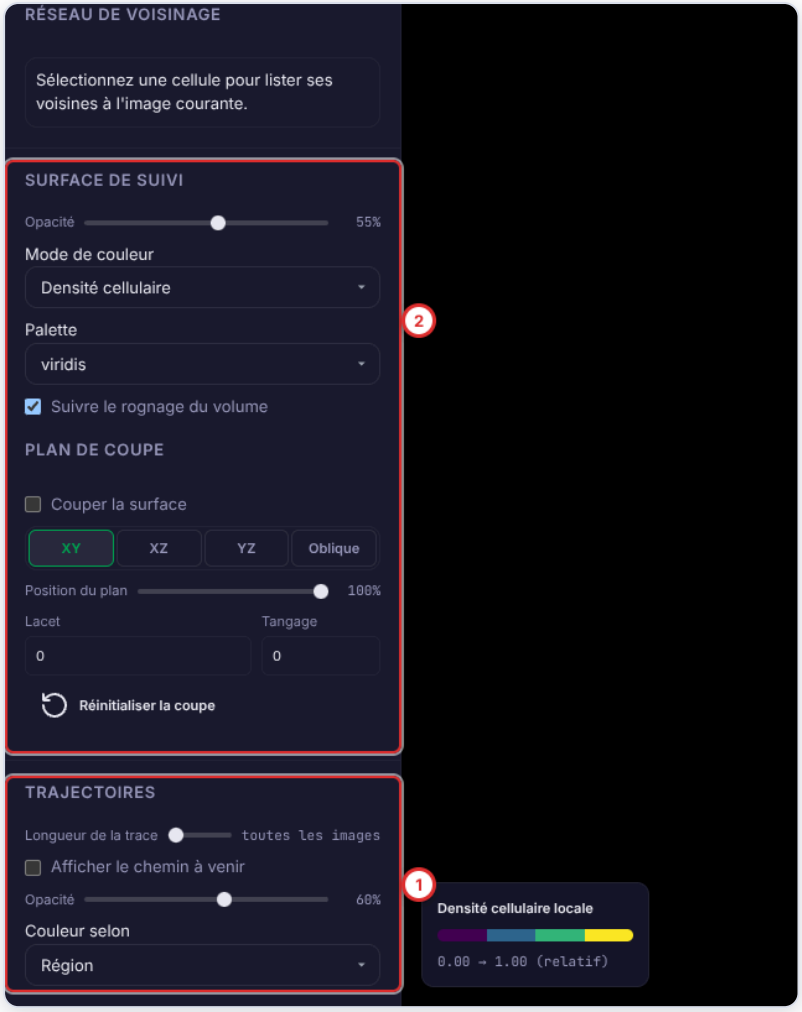
- La lecture saute des images plutôt que de les empiler ; l'horloge **attend** l'image si elle n'est pas arrivée (jusqu'à 15 s).
- Les images suivantes sont préchargées. **Tampon 4/4 · 512×512** indique ce qui tient en mémoire.
- Il n'y a pas d'horloge en minutes sur la frise : l'affichage est en numéro d'image.



▲ Sur cette série, la barre est repliée : le menu ☰ regroupe les outils de suivi.

N	CE QUE C'EST
1	**Inspecter une cellule** (I)
2	**Distance entre cellules** (D)
3	**Graphiques de suivi**
4	**Trajectoires**
5	**Surface de suivi**

Trajectoires et surface



▲ **Trajectoires** (1) et **Surface de suivi** (2) dans la colonne de gauche, dernière image.

- **Trajectoires** : longueur de la trace (0 = toutes les images), chemin à venir, opacité, couleur par **région** ou par **vitesse** (bleu lent → rouge rapide, échelle fixe sur toute la série).
- **Surface de suivi** : la surface du tissu (`model.glb`) dans le volume, colorée en uniforme, par **densité cellulaire** ou par région, avec une palette au choix et un **plan de coupe** (XY, XZ, YZ ou oblique).

> 🔧 **POUR LES CURIEUX**
>
> Densité locale d'un sommet : somme de gaussiennes $\exp(-|v-c|^2/2\sigma^2)$ sur les cellules de l'image courante, avec σ = 0,52 × rayon de voisinage, borné entre 8 et 54 µm, puis deux lissages sur le maillage. Vitesse d'un segment : |Δposition| en µm / Δt.

> ⚠️ **ATTENTION**
>
> Sur le jeu de démonstration, les cellules se déplacent de quelques micromètres seulement entre deux images : aucun trait n'était lisible dans nos captures en rendu logiciel. Les curseurs et le résumé s'appliquent bien ; essayez sur une série réelle.

Inspecter une cellule

▲ Cellule 15 sélectionnée (1). Colonne de gauche : identifiant (2), métriques (3), voisines (4).

1. Appuyez sur `I`, puis cliquez une sphère (ou tapez son numéro et **Trouver**).
2. Lisez les tuiles : piste, région, images, vitesse moyenne, longueur du trajet, déplacement net, rectitude, évènement.
3. Explorez le lignage (mère, filles) et les **voisines** à moins de 35, 55, 85 ou 120 µm.

> 🔧 **POUR LES CURIEUX**
>
> Longueur du trajet = Σ |p(f+1) − p(f)| ; déplacement net = |p(dernière) − p(première)| ; **rectitude = déplacement / longueur** (1 = ligne droite). Vitesse moyenne = longueur / durée, en µm/image, et en µm/min si l'intervalle est connu. Ici 4,35 µm de trajet pour 4,04 µm de déplacement : rectitude 0,93.

Boutons d'export : **CSV de la piste**, **CSV des voisines**, **JSON du lignage**.

Distance entre cellules et graphiques

▲ Deux cellules (1, 2) et le panneau (3) : 41,4 µm.

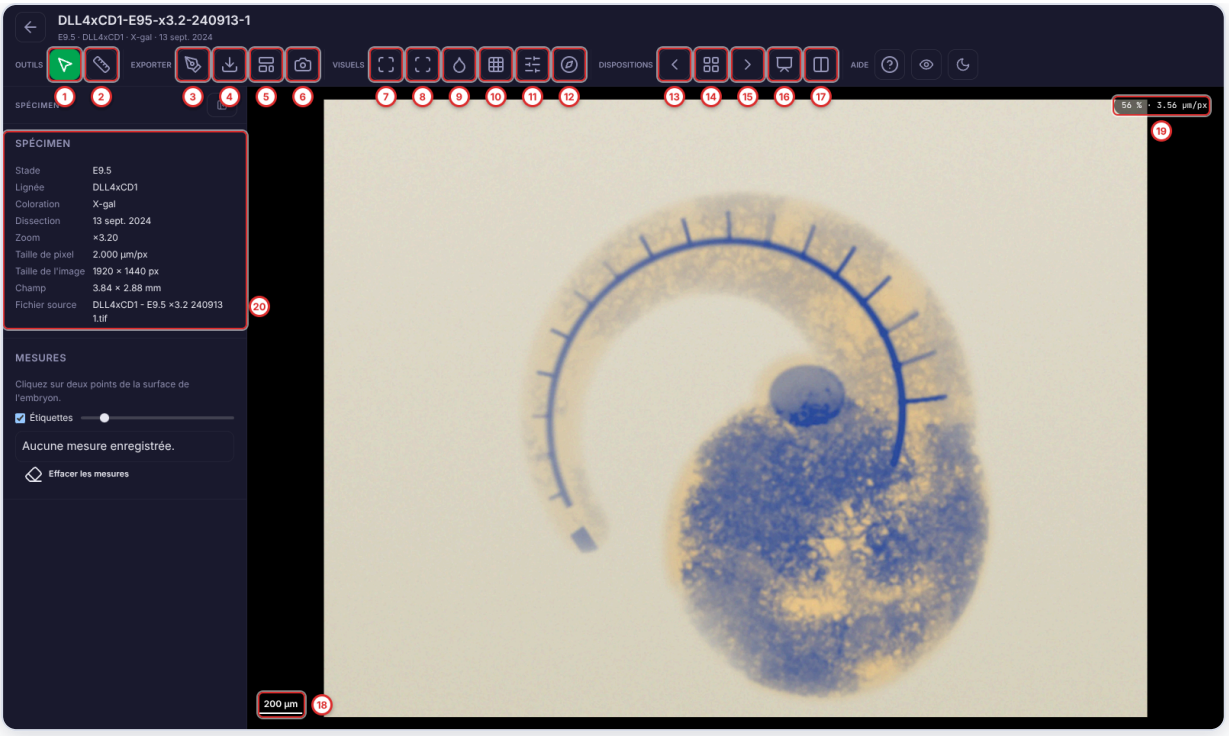
▲ Population par région (jeu de démonstration).

- **Distance entre cellules** (D) : cliquez deux cellules suivies. Mode **Instantané** (positions gardées) ou **Suivre les cellules** (relue à chaque image ; « Hors image » si l'une disparaît). Formule : $\sqrt{(\Delta x^2 + \Delta y^2 + \Delta z^2)}$ en µm.
- **Graphiques de suivi** : **Population**, **Vitesse**, **Voisines**, **Mitoses**, échelle **Linéaire** ou **Log**, une courbe par région.

12.10 La page 2D : les photographies

⏱ EN 30 SECONDES

- **À quoi ça sert** : afficher une photographie calibrée (stéréomicroscope, ici X-gal), la mesurer et la comparer.
- Le moteur est plus simple qu'en 3D : un canevas qu'on déplace et qu'on zoome.
- **Isoler le marquage** est une aide visuelle : rien de ce qui est mesuré ne la lit.

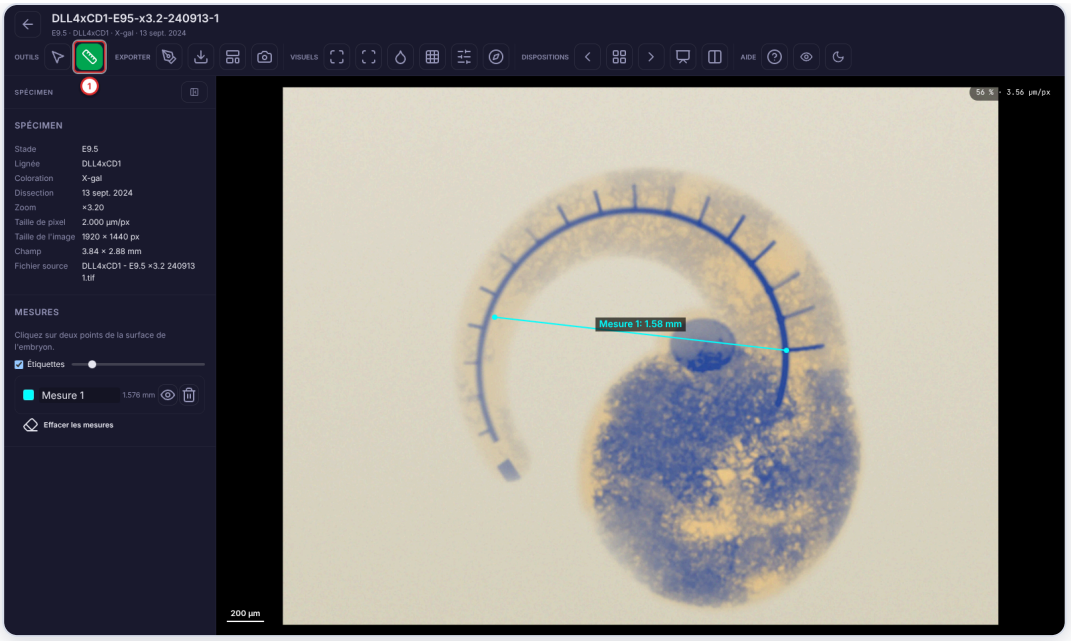


▲ La page 2D (1 à 17 : les boutons ; 18 : barre d'échelle ; 19 : zoom et µm/px ; 20 : fiche du spécimen).

N	CE QUE C'EST
1, 2	**Naviguer**, **Mesurer la distance**
3	**Ouvrir dans le Studio (annoter)**
4, 5, 6	Téléchargement, **Constructeur de planche**, capture
7, 8	**Ajuster à l'écran (F)**, **Résolution native 1:1**

N	CE QUE C'EST
9	**Isoler le marquage**
10, 11, 12	**Grille calibrée**, **Réglages d'affichage**, **Orientation**
13, 14, 15	Photographie précédente, parcourir la collection, suivante
16, 17	Mode présentation, **Vue divisée**

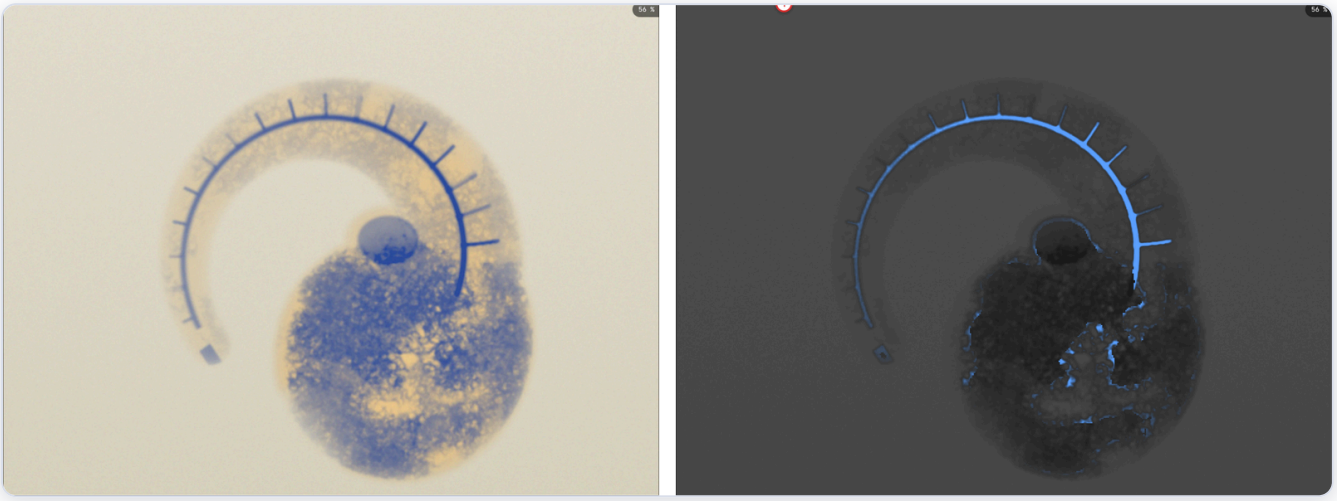
- **Navigation** : molette pour zoomer autour du curseur, glisser pour déplacer, double-clic pour ajuster. Le zoom va de 0,25 × l'ajustement à 32 pixels d'écran par pixel d'image.
- **Chargement en deux temps** : un aperçu (≈ 640 px) puis l'image native.
- **Barre d'échelle** en bas à gauche, valeur 1-2-5, calculée avec la taille de pixel (ici 2,000 µm/px). Sans calibration : **Non calibré**.
- **Parcourir la collection** (B) ouvre une planche-contact filtrable par stade et lignée.



▲ Une mesure : 1,58 mm (1,576 mm dans la liste).

La mesure est `√(Δx² + Δy²) × taille de pixel` : la photographie est plane, il n'y a pas de Z. Les étiquettes se déplacent à la souris.

Isoler le marquage (X-gal)



▲ Avant (à gauche) et après (à droite) : le marquage bleu ressort, le reste devient gris sombre.

Le X-gal rend un pixel plus **bleu** que le tissu voisin. Mais le fond mat porte aussi des grains bleutés. Pour ne pas les allumer, l'outil exige que le bleu soit **à l'intérieur d'un tissu jaune**.

« Isoler le marquage » : trois étapes pour chaque pixel

1 · Où est le tissu jaune ?

« jaunisme » d'un pixel =
(rouge + vert) / 2 − bleu

On le moyenne sur un carré d'environ 40 px de côté (image réduite 4 fois, flou de rayon 5).

fond mat : 2 à 6
tronc coloré large : 6 à 9

2 · Le pixel est-il bleu ?

rapport bleu / (rouge + 1)

Tissu non coloré : ≈ 0,6
Cœur très coloré : 2 à 4

force v : 0 si rapport ≤ 1,0
1 si rapport ≥ 1,8, linéaire entre

et seulement si le « jaunisme » local dépasse 5 (sinon v = 0)

3 · On repeint

gris = luminosité × 0,35 × (1 − v)

rouge = gris + 90 × v
vert = gris + 160 × v
bleu = gris + 255 × v

v = 0 : gris sombre
v = 1 : bleu vif (90, 160, 255)

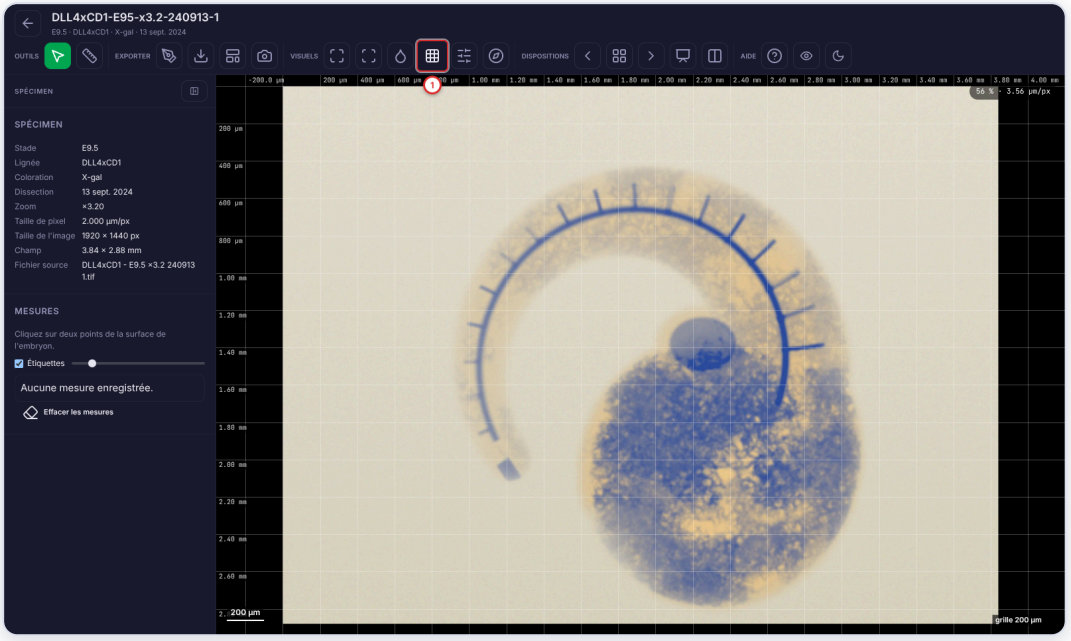
Micro-exemple
Pixel (R, V, B) = (60, 90, 170) : 170 / 61 = 2,8 → v = 1, dans un tissu à « jaunisme » 8 : il devient bleu vif.
Pixel (200, 190, 120) : 120 / 201 = 0,6 → v = 0 : il devient gris sombre (0,35 × sa luminosité).

▲ Le calcul, pixel par pixel.

🔧 **POUR LES CURIEUX**

Constantes du code : rapport bas 1,0, haut 1,8 ; contexte à 1/4 de résolution, flou de rayon 5 (≈ 40 px), seuil 5. Luminosité = 0,299 R + 0,587 V + 0,114 B. Le tout tourne dans un *worker* (la page reste fluide), seule la dernière requête est traitée, la carte de contexte est gardée tant que l'image ne change pas. L'outil est réglé pour du X-gal sur tissu jaune : d'autres colorations ou fonds peuvent être mal détectés.

Affichage, grille, orientation

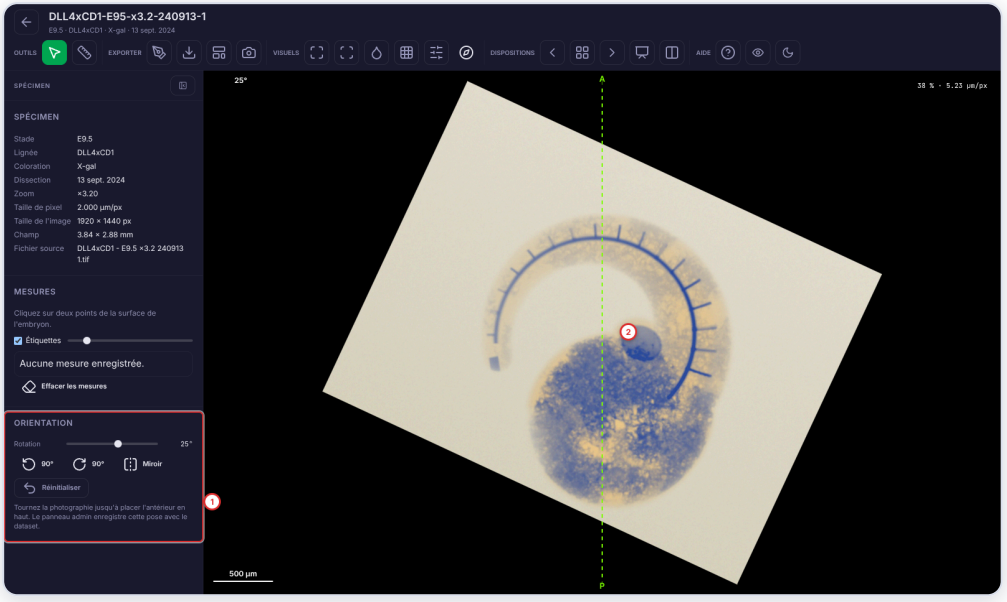


▲ **Grille calibrée** (200 µm) : une échelle de lecture sur la photographie.

▲ **Réglages d'affichage** (panneau **Affichage**).

- **Réglages d'affichage** : **Luminosité**, **Contraste**, **Gamma**, balance **Rouge** / **Bleu**, **Aplatir le fond (vignettage)**. *Affichage seulement* : les mesures lisent la photo intacte. Par canal : `x = (v/255 · balance − ½)(1 + contraste/100) + ½ + luminosité/100`, puis `x^(1/gamma)`.
- **Aplatir le fond** : une surface lissée (quadratique) est ajustée sur les 40 % de pixels les plus sombres (le fond) et le gain correspondant est appliqué, entre 0,5 et 3.

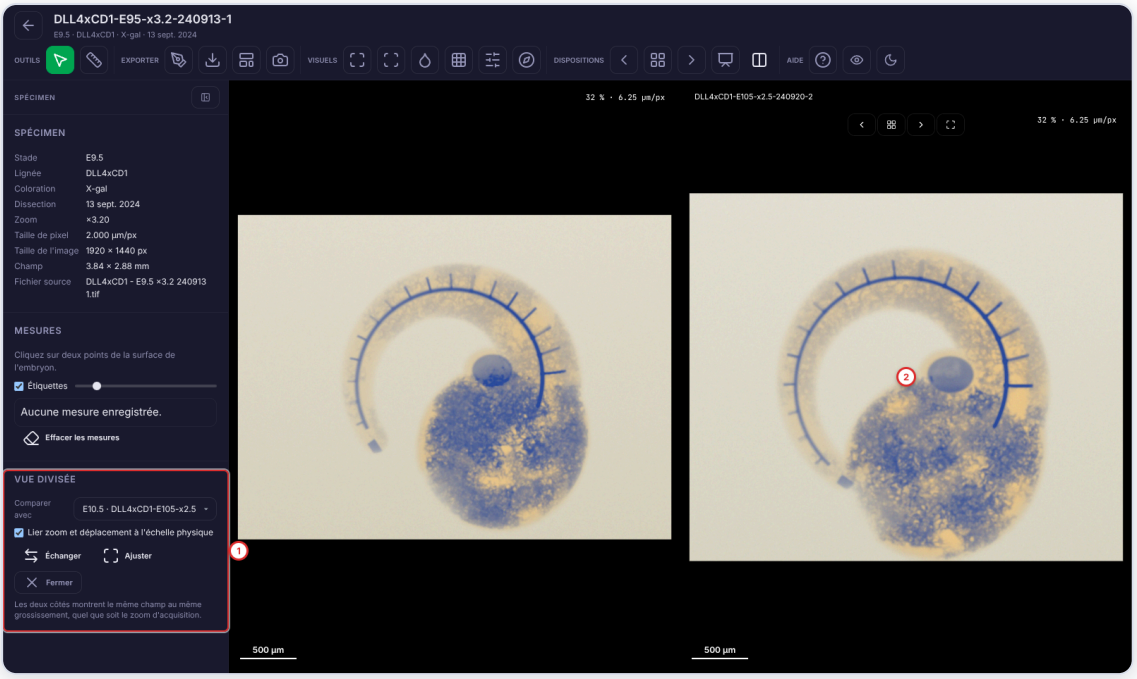
- **Grille calibrée** : un clic = aucune → large (≈ 120 px) → fine (≈ 60 px), pas en 1-2-5 µm.



▲ **Orientation** : rotation de 25°, boussole A (antérieur) / P (postérieur).

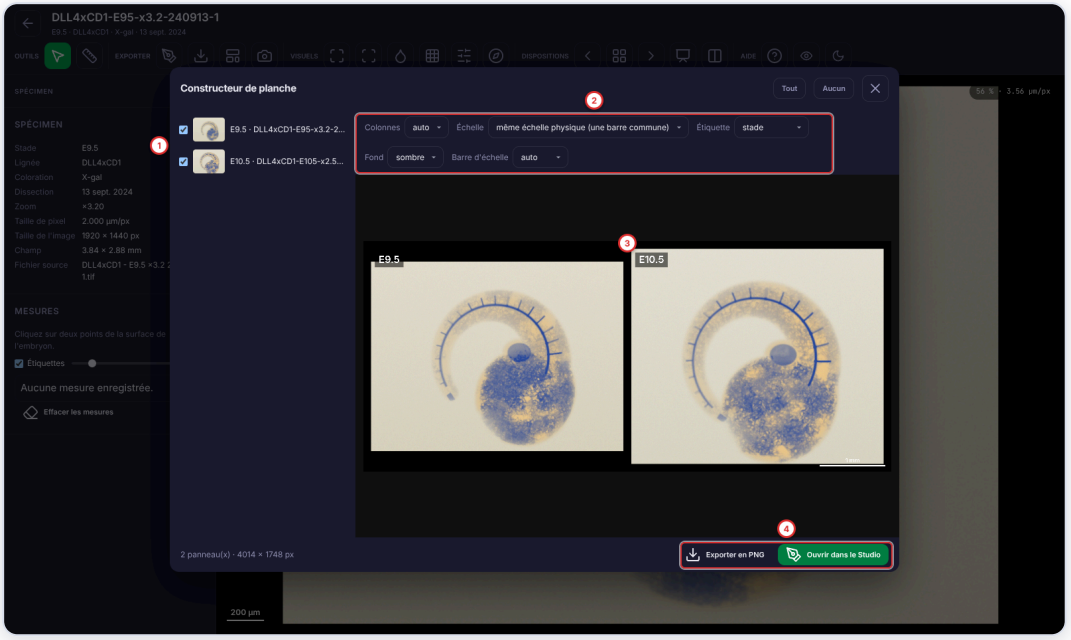
L'**orientation** tourne l'image sans changer l'échelle (rotation, ±90°, miroir). L'administrateur enregistre la pose avec le jeu de données ; elle est réappliquée à chaque ouverture.

Vue divisée et planche de figure



▲ **Vue divisée** : deux embryons au **même grossissement physique**.

La vue divisée échange une vue **physique** (µm par pixel d'écran + position) entre les deux côtés : deux photos prises à des zooms différents sont à la même échelle, et déplacer l'une déplace l'autre de la même distance réelle.



▲ Le **Constructeur de planche** : liste (1), options (2), aperçu (3), export (4).

La planche assemble plusieurs photographies en un PNG : **même échelle physique** (une barre commune, chaque image est ramenée au pixel le plus grossier, jamais agrandie) ou **même taille d'image** (une barre par panneau). Limites : 6000 px de large, 120 mégapixels. Elle s'ouvre aussi dans le Studio.

12.11 Raccourcis clavier et partage d'une vue

⏱ EN 30 SECONDES

- Les raccourcis sont ignorés quand vous tapez dans un champ.
- **Partager une vue** = copier l'adresse de la page : l'état est dans le lien.

Viewer 3D

TOUCHE	ACTION
`V` / `Échap`	Naviguer
`C`	Coupe
`M`	Mesurer
`I`	Inspecter une cellule
`D`	Distance entre cellules

Page 2D

TOUCHE	ACTION
← / →	Photo précédente / suivante
`B`	Planche-contact
`F`	Ajuster à l'écran

Studio

TOUCHE	ACTION
`V` `R` `E`	Sélection, rectangle, ellipse
`A` `L` `T`	Flèche, ligne, texte
`D` `G` `S`	Distance, angle, barre d'échelle
`Espace`	Déplacer (maintenu)
`Ctrl` + `K`	Palette de commandes
`Ctrl` + `Z` / `Ctrl` + `Maj` + `Z`	Annuler / rétablir
`Suppr`	Supprimer le calque
`[` / `]`	Tourner de −15° / +15°

Souris : clic gauche glissé = tourner ; `Maj` ou clic droit/milieu = déplacer ; molette = zoom.

Partager une vue

La page écrit toutes les secondes son état dans l'adresse (`#state=…` , compressé) : caméra, canaux, outil actif, plan de coupe, **mesures**, image de la série, qualité, réglages des outils. Copiez simplement l'adresse.

- Celui qui ouvre le lien voit : **Ce lien contient une vue enregistrée** avec **Ouvrir la vue enregistrée** ou **Ouvrir le jeu de données à neuf** .
- **Réinitialiser l'espace de travail** (bouton 21) remet vue, outils, canaux et mesures à l'état d'ouverture ; la **qualité** n'est pas modifiée.
- Le viewer n'a pas de bouton « Sauvegarder » : c'est l'adresse qui sert de sauvegarde. La page Comparer, elle, a **Sauvegarder** / **Restaurer** (§ 12.8).

📌 **À RETENIR**

- Une mesure est calculée en **µm** sur ce que **vous voyez** ; un clic dans le vide est refusé.
- Coupe oblique et Z-stack s'excluent l'un l'autre.
- Le Studio exporte au natif ; son JSON ne contient jamais de pixels.
- Comparer = les vraies pages dans des cadres, avec un budget mémoire partagé.
- « Isoler le marquage » est une aide visuelle, pas une mesure.

13. Les outils sous le capot

- Le chapitre 12 vous a appris **à vous servir** des outils. Celui-ci ouvre le capot : **comment ils fonctionnent**, avec les chiffres et les formules du code.
- Neuf sujets : le suivi cellulaire et la stabilisation, le Studio, Comparer, la page 2D et son importeur, la frise temporelle, l'état partagé (lien, mémoire du navigateur), les algorithmes de mesure, la galerie et les jeux liés, l'orientation.
- Rien ici n'est nécessaire pour utiliser la plateforme. Chaque section se lit seule ; les encadrés *Pour les curieux* peuvent être sautés.

ℹ️ **À SAVOIR**

Toutes les captures et tous les chiffres viennent du **jeu de démonstration** (embryons et photographies synthétiques générés pour cette documentation). Quand un exemple utilise des valeurs inventées pour la clarté (un budget mémoire, deux tailles de pixel), le texte le dit.

13.1 Le suivi cellulaire dans le viewer

⏱ **EN 30 SECONDES**

- Les cellules suivies sont dessinées comme des **sphères** (une par cellule) lues dans des **tableaux de nombres** préparés par un *worker*.
- La **stabilisation** des images n'abîme jamais un voxel : le shader change seulement **l'endroit où il lit** la texture, grâce à une matrice 4 × 4 par image.
- Cinq outils partagent **une seule copie** des données, par une façade nommée `ctx.tracking`.

Le chapitre 8 (§ 8.4) a expliqué comment le pipeline calcule le suivi et la rotation de Kabsch. Le chapitre 12 (§ 12.9) a montré les boutons. Voici **le milieu** : ce que le viewer fait de ces fichiers, image après image.

Du fichier aux sphères

▲ Le chemin des données de suivi, du fichier jusqu'à l'écran.

Le fichier `tracks.json` n'est pas lu par la page : un **worker** (un petit programme qui tourne en parallèle de la page, § 10.5) le télécharge. Il essaie d'abord la copie compressée `tracks.json.gz`, qu'il décompresse avec le navigateur. Si elle manque, ou si le serveur l'a déjà décompressée, il retombe sans bruit sur le fichier brut.

Le worker ne dessine rien. Il range tout dans des **tableaux typés** (des suites de nombres de taille fixe) qu'il *transfère* à la page sans les copier. Le viewer garde ces tableaux **une seule fois** : le dessin, les cinq outils et les exports lisent les mêmes.

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un annuaire et un plan de métro.** Le fichier JSON est un annuaire : pratique à lire, lent à feuilleter. Les tableaux sont le plan de métro : tout est à une adresse fixe. « Où est la cellule 15 à l'image 3 ? » devient **une lecture dans un tableau**, pas une recherche.

Les tableaux : deux familles

Par image (pour dessiner, 30 fois par seconde)

TABLEAU	CONTENU
`posStab`	positions stabilisées, image par image
`posRaw`	les mêmes, brutes
`cellIdx`	numéro de la cellule de chaque emplacement
`counts`	nombre de cellules par image

Par cellule (pour analyser)

TABLEAU	CONTENU
`parent`, `daughterStart/Idx`	lignée (mère ; liste des filles)
`flags`, `regionIdx`	mitose / fusion ; région
`firstFrame`, `lastFrame`	durée de vie
`cellFrameSlot`	emplacement de la cellule **à chaque image** (−1 = absente)

Le dernier tableau est le plus utile : `cellFrameSlot[cellule × images + image]` répond à « où est cette cellule à cette image ? » en **une seule lecture**. C'est lui qui rend la sélection, les trajectoires et les distances instantanées.

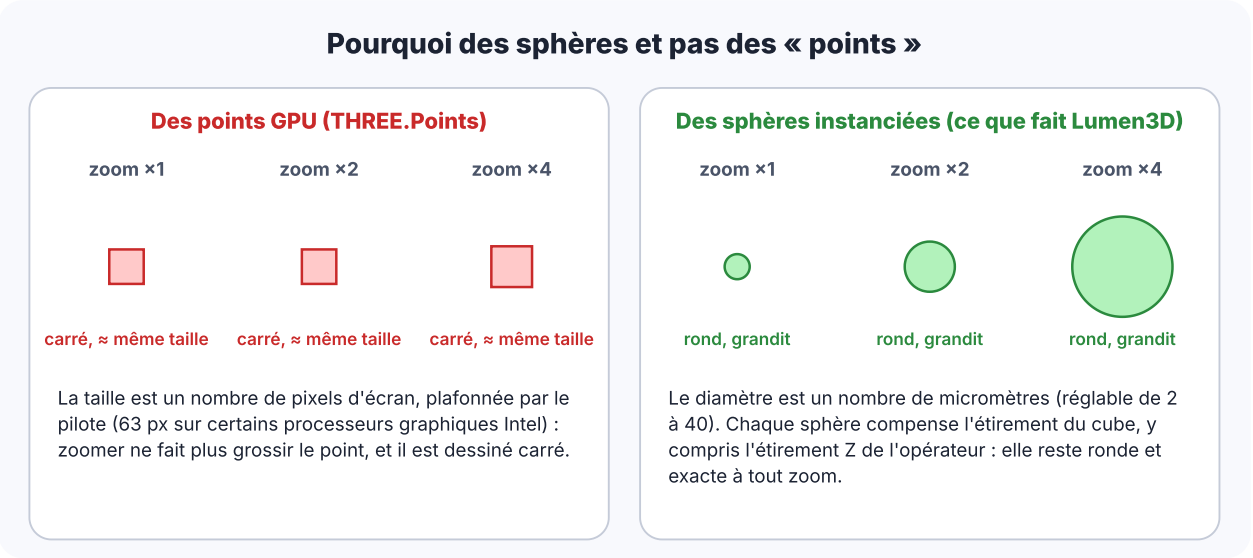
🔢 **MICRO-EXEMPLE**

Sur le jeu de démonstration (50 cellules, 4 images), le worker produit 8 048 octets de tableaux, à partir d'un fichier de 32 675 octets (10 991 compressé). Les images n'ont pas toutes le même nombre de cellules : **48, 48, 49, 49**. Les emplacements libres ne sont jamais dessinés, ils ne restent pas figés à leur dernière position.

🔧 **POUR LES CURIEUX**

- **32 bits partout** : compteurs et indices sont des `Uint32Array` / `Int32Array`. Un tableau de 16 bits déborderait silencieusement au-delà de 65 535 cellules ; ici rien ne déborde.
- Les cellules sont repérées dans le fichier par le texte de leur image (« 1 », « 2 »…, Imaris compte à partir de 1) alors que les briques comptent à partir de 0. Plutôt que d'ajouter 1 à la main, le worker **construit la correspondance** à partir de la liste triée `timepoints` du fichier.
- Une position incomplète ou non finie est **ignorée et comptée** (`malformed`) : une ligne défectueuse ne cache jamais toute la couche.
- La table `cellFrameSlot` est la seule à croître avec le produit cellules × images. Elle est refusée au-delà de 2^{28} entrées (1 Gio) avec un message qui dit pourquoi, au lieu d'une erreur opaque de mémoire.
- Une lignée peut désigner une cellule par sa clé, son `id` ou son `track_id` (l'exportateur a utilisé les trois) ; le worker essaie dans cet ordre, puis **rend le lien symétrique** (mère ↔ filles).

Pourquoi des sphères et pas des « points »



▲ Points GPU contre sphères instanciées.

Une sphère coûte un peu plus qu'un point, mais le diamètre se règle **en micromètres** (2 à 40, 12 par défaut) et reste juste à tout zoom. La couche est un *maillage instancié* : une seule sphère de référence, dessinée autant de fois qu'il y a de cellules, avec une matrice par instance.

🔧 **POUR LES CURIEUX**

- Les sphères sont **enfants du cube** du volume : l'orbite, le déplacement, les proportions physiques et l'étirement Z de l'opérateur s'appliquent sans code supplémentaire.
- Le diamètre monde est `diamètre_µm × (échelle_x du cube / taille_x de l'acquisition en µm)`. L'échelle de la sphère est ensuite **divisée par l'échelle propre de chaque axe** du cube : sans cette division, l'étirement Z en ferait un ellipsoïde.
- `depthTest = false` : le volume est un lancer de rayons additif **sans profondeur** ; un test de profondeur cacherait chaque centroïde derrière la face avant du cube.
- Le test « cette sphère est-elle dans la boîte rognée ? » est **le même** que celui du shader : la couche suit la tranche du Z-stack et les curseurs de rognage au lieu de flotter au-dessus d'un volume presque caché.
- Une cellule sélectionnée est dessinée 1,8 fois plus grosse. La couleur d'une cellule vient de sa région (une couleur par **cellule**, pas par cellule et par image : trente fois moins d'octets).

Deux repères, un interrupteur de dictionnaire

Le fichier porte **deux jeux de coordonnées** pour chaque cellule : stabilisées (`posStab`) et brutes (`posRaw`). La couche n'applique jamais la matrice de stabilisation à des points : elle **choisit le bon tableau**.

⚠️ **ATTENTION**

Appliquer la matrice à des points déjà stabilisés les enverrait dans un troisième repère. À l'image 1 la transformation est l'identité : on ne verrait rien. À l'image 30 d'une vraie série, l'erreur atteindrait environ 250 µm, et ressemblerait à un défaut général d'alignement. D'où la règle : **un dictionnaire, pas une multiplication**.

- Volume stabilisé à l'écran : la couche lit `posStab`.
- Volume brut : elle lit `posRaw`.
- Si le fichier n'a **pas** de coordonnées brutes pour toutes les cellules et que l'image affichée est brute, la couche se **masque** (« Coordonnées brutes manquantes : suivi masqué sur un volume non stabilisé »). Une couche plausible mais fausse serait pire qu'aucune couche. Votre propre réglage de visibilité n'est pas touché.

Cliquer une sphère

`pick()` lance un rayon à travers le point cliqué et le teste contre le maillage **tel qu'il a été dessiné** : une cellule filtrée (mitoses, fusions) ou rognée n'est jamais touchée, et une couche masquée n'est pas cliquable. Le résultat est le numéro de la cellule, ou −1.

La façade `ctx.tracking`

Les cinq outils (trajectoires, surface, inspecteur, graphiques, distance entre cellules) ne touchent jamais le maillage ni les tableaux directement. Ils passent par une **façade** que la page construit.

ELLE DONNE…	EXEMPLES
des données	`getData()`, `positionUm(cellule, image)`, `cellsAt(image)`, `whenLoaded()`
un état partagé	**une** sélection, **un** rayon de voisinage (5 à 500 µm, 55 par défaut)

ELLE DONNE…	EXEMPLES
des services	`pick(x, y)`, `umToObject()`, `isInsideClip()`, `getCamera()`
des événements	`loaded`, `frame`, `refresh`, `selection`, `options`, `style`

❓ **POURQUOI CE CHOIX ?**

Une sélection, pas cinq. Si l'inspecteur, les trajectoires et les graphiques avaient chacun la leur, cliquer une cellule éclairerait trois choses différentes. Avec la façade, une cellule choisie s'éclaire partout, et le rayon de voisinage règle d'un coup l'inspecteur, les graphiques et la densité de la surface.

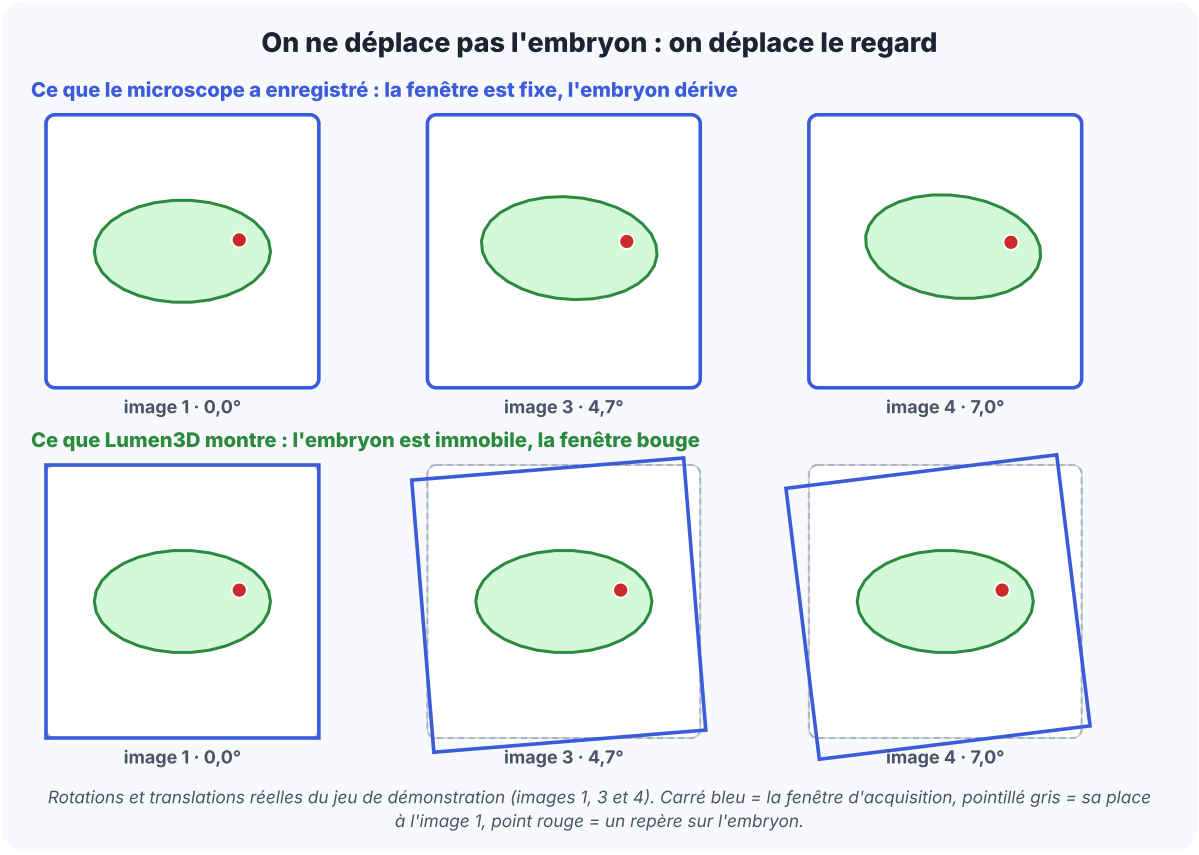
L'événement `frame` est émis **après** que la couche s'est déplacée : un outil qui dessine par-dessus les points ne devance jamais leur image d'une image.

13.2 La stabilisation : on déplace le regard

⏱ **EN 30 SECONDES**

- Un embryon dérive et tourne pendant des heures. La stabilisation **annule ce mouvement à l'écran**.
- Le viewer ne fabrique aucune nouvelle image : le shader lit chaque voxel **à une autre adresse** de la texture.
- Elle ne s'applique que si le pipeline a vérifié que le mouvement est **rigide**.

Le principe en trois images



Rotations et translations réelles du jeu de démonstration (images 1, 3 et 4). Carré bleu = la fenêtre d'acquisition, pointillé gris = sa place à l'image 1, point rouge = un repère sur l'embryon.

▲ Rangée du haut : ce qu'a enregistré le microscope. Rangée du bas : ce que montre le viewer.

Le microscope regarde toujours par la même fenêtre. Si l'embryon dérive, **c'est lui** qui bouge dans la fenêtre. Le viewer inverse le point de vue : l'embryon reste en place à l'écran, et c'est **la fenêtre** (le carré bleu) qui tourne et glisse autour de lui, comme une caméra qui suivrait la bête.

💡 **UNE IMAGE POUR COMPRENDRE**

Un caméraman, pas un décorateur. Pour garder un danseur au centre de l'image, on peut redessiner le décor autour de lui à chaque plan (rééchantillonner), ou simplement **tourner la caméra**. Lumen3D tourne la caméra : le décor, c'est-à-dire les voxels mesurés, n'est jamais retouché.

Deux façons de « stabiliser » des images

Rééchantillonner (non retenu)

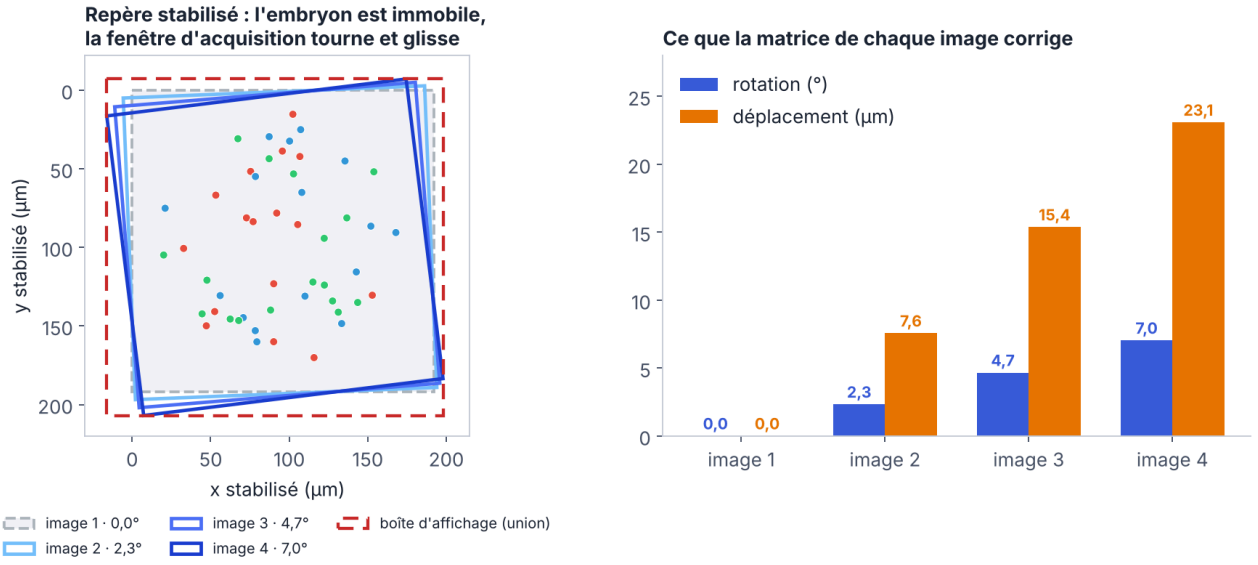
- Fabriquer, pour chaque image, une copie tournée et décalée du volume.
- Chaque voxel est recalculé par interpolation : il est un peu flou, donc ce n'est plus la mesure.
- Coût : tout le volume, à chaque image.
- Irréversible une fois les briques écrites.

Déplacer le regard (Lumen3D)

- La texture n'est jamais touchée : les voxels restent ceux de la mesure.
- Seul le point de lecture change : chaque rayon est envoyé à travers la matrice de l'image.
- Coût : une matrice 4 × 4 par image.
- Réversible en un clic (stabilisé ⇄ brut).

▲ Deux façons de stabiliser, et celle que retient Lumen3D.

Les repères, avec les vrais chiffres



Jeu de démonstration (synthétique) : rotations 0 ; 2,3 ; 4,7 ; 7,0° ; points = cellules de l'image 4 colorées par région.

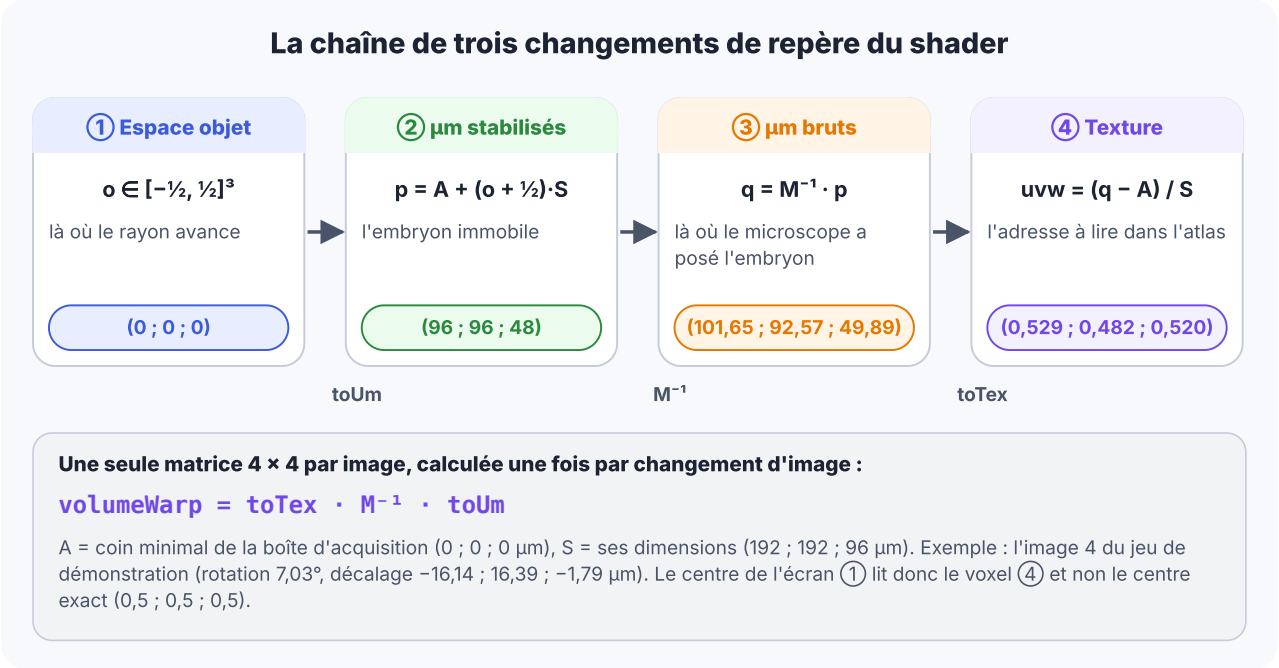
▲ Quatre fenêtres d'acquisition (une par image) dans le repère de l'embryon, et la boîte qui les contient toutes.

Trois repères sont en jeu :

- le **repère brut** : celui du microscope, la fenêtre de 192 × 192 × 96 µm est fixe ;
- le **repère stabilisé** : celui de l'embryon, immobile ; chaque image y a sa fenêtre, tournée de 0 ; 2,3 ; 4,7 ; 7,0° ;
- la **boîte d'affichage** : le plus petit pavé qui contient **toutes** les fenêtres, ici de (−16,3 ; −7,1 ; −2,4) à (197,9 ; 207,3 ; 96) µm.

La boîte d'affichage est ce que le cube géométrique doit couvrir : sans elle, la fenêtre de l'image 4 sortirait du cube. Le cube grandit donc, **sans toucher** à son échelle (la grille, la barre d'échelle, le plan de coupe et le cadrage de la caméra continuent de s'y fier).

Le shader : une chaîne de trois changements de repère



▲ Le point lu passe par trois repères, avec l'exemple de l'image 4.

Pour chaque point où le rayon prend un échantillon, le shader fait :

`volumeWarp = toTex · M⁻¹ · toUm`

1. `toUm` convertit la coordonnée de l'objet (de −½ à +½) en **micromètres stabilisés**.
2. `M⁻¹` revient aux **micromètres bruts** : c'est l'inverse de la matrice que le suivi a appliquée à l'image.
3. `toTex` convertit en **adresse de texture** (de 0 à 1).

🔢 **MICRO-EXEMPLE**

Image 4 du jeu de démonstration. Le centre exact de l'écran (objet 0 ; 0 ; 0) vaut (96 ; 96 ; 48) µm stabilisés. Après `M⁻¹` (rotation 7,03°, décalage −16,14 ; 16,39 ; −1,79 µm) il devient (101,65 ; 92,57 ; 49,89) µm bruts, donc l'adresse (0,529 ; 0,482 ; 0,520) dans la texture. On lit un voxel un peu à côté du centre : **c'est tout le « travail »** de la stabilisation.

La matrice est calculée **une fois par changement d'image** et passée au shader comme une constante. Au tout premier passage, le shader est recompilé avec l'option `VOLUME_WARP` ; ensuite, chaque image ne change qu'un nombre.

🔧 POUR LES CURIEUX

- **Entrée et sortie du rayon.** L'intersection rayon / boîte est calculée dans l'espace de la **texture source** (où la boîte d'acquisition est exactement le cube unité), par `hitBoxWarped`. Ainsi la densité d'échantillonnage ne dépend pas de la distance dont l'embryon a dérivé : l'image n'est pas plus claire ni plus grenue à l'image 30 qu'à l'image 1.
- **Rognage et coupe.** Les curseurs de rognage, la tranche du Z-stack et le plan de coupe agissent dans la **boîte d'affichage** : vous coupez ce que vous voyez, pas le repère de l'acquisition.
- **Garde-fous.** Une matrice non finie, ou de déterminant presque nul ($|\det| \le 10^{-9}$), est **ignorée** : l'image est affichée sans déformation et un avertissement apparaît en console. Une rotation rigide a un déterminant de ±1 : tout ce qui s'en écarte est suspect.
- **Orbite intacte.** Le warp ne touche ni la position ni la rotation du cube : ce sont les contrôles de la souris, ils sont enregistrés dans les espaces de travail. Mélanger une matrice par image à eux casserait les deux.
- **La coupe oblique et le Studio** échantillonnent par le même warp (`samplingSpace`, `planeSweep`) : une coupe d'une série stabilisée montre exactement la tranche de l'écran.

Quand est-elle appliquée ?

CONDITION	SI ELLE MANQUE
`registration.appliedToVolume` **est vrai (le test de rigidité a réussi : résidu ≤ 0,05 µm)**	l'image reste **brute**, seul le suivi est stabilisé
`acquisitionExtentUm` **est présent**	idem, avec un avertissement
la matrice de l'image est valide	cette image est affichée non déformée

Le test de rigidité du pipeline (chapitre 8) est donc **le seul feu vert**. Sur le jeu de démonstration le résidu vaut $7{,}6 \times 10^{-14}$ µm : la stabilisation est appliquée.

ℹ️ À SAVOIR

Il n'y a **pas de bouton** « stabilisé / brut » dans l'interface : la stabilisation est active dès qu'elle est applicable. L'interrupteur existe dans l'API de la page (`ViewerApp.setVolumeStabilized`) ; la couche de points et les outils suivent le changement à l'image suivante, car ils lisent le repère affiché dans l'événement d'image prête, jamais en le redemandant.

⚠️ ATTENTION

Sur le jeu de démonstration, l'image n'a pas réellement dérivé : seules les coordonnées brutes du suivi ont été générées avec une dérive (jusqu'à 23 µm, 7°). Appliquer la stabilisation décale donc légèrement le volume. C'est pourquoi les figures de cette section sont des **schémas calculés avec les vraies matrices**, et non des captures « avant / après » qui laisseraient croire à un gain visible.

La surface, la provenance, les mesures

- **La surface** `model.glb` contient un maillage **par image**, dans le repère brut et dans le repère stabilisé. Elle est enfant du cube comme les sphères : le même `o = (p − A)/S − ½` la place, donc orbite, étirement Z et stabilisation s'appliquent. Le lecteur de GLB est celui de Three.js, **chargé à la première utilisation** avec son empreinte d'intégrité (SRI) : la page du viewer ne le porte pas.

- **Trois colorations** : uniforme, densité locale, région. La densité d'un sommet est la somme de gaussiennes `exp(−|v − c|² / 2σ²)` sur les cellules de l'image, limitée à 2,8 σ et rangée dans une grille uniforme (un sommet ne visite que son voisinage), puis lissée deux fois sur le maillage. Le résultat est mémorisé tant que ni l'image, ni le rayon de voisinage, ni la palette ne changent.
- **La coupe de la surface** utilise des plans de découpe de Three.js, qui sont en **repère monde** alors que le cube tourne : les plans sont donc ré-exprimés juste avant chaque dessin. Un chapeau plein ferme la coupe.
- **La provenance.** `tracking.surfaceOrigin` dit si la surface a été **exportée** par l'analyse ou **reconstruite** à partir des cellules (c'est le cas du jeu de démonstration).
- **L'alignement.** À l'import, le pipeline vérifie que le suivi se trouve bien dans le volume : au moins 50 % des positions brutes dans la boîte d'acquisition élargie de 10 %, et un étalement d'au moins 1 % du plus grand côté. Sinon il propose l'unité probable (mm, nm, m), sans rien convertir. Le jeu de démonstration : 100 % dedans, étalement 0,83, `aligned: true`.
- **Les unités.** Les positions sont ramenées en µm à l'import ; `tracking.provenance.unit` garde l'unité déclarée, le facteur et son statut (*native*, *converted*, *assumed*, *unknown*).
- **Les mesures** de distance entre cellules sont rangées sous la portée `tracking` du magasin de mesures, et leur export note le repère (*stabilized* ou *raw*).

📌 **À RETENIR**

- Les cellules sont des **sphères instanciées** lues dans **des tableaux 32 bits**, en un seul exemplaire.
- La stabilisation est **un changement d'adresse de lecture** : `volumeWarp = toTex · M⁻¹ · toUm`.
- Rien n'est rééchantillonné : c'est réversible et exact.
- Le seul feu vert est le **test de rigidité** du pipeline.

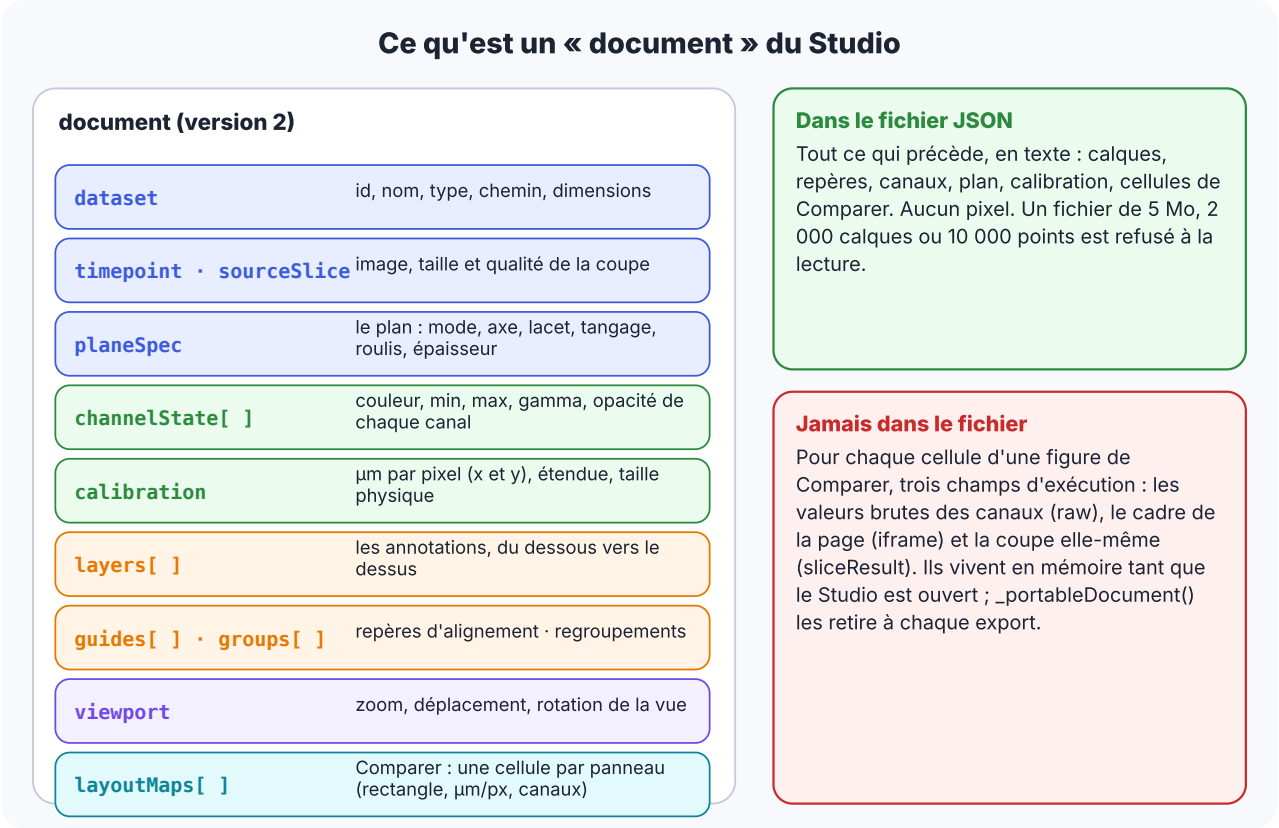
13.3 Le Studio en profondeur

⏱ **EN 30 SECONDES**

- Une figure du Studio est un **document** : des calques, des repères, des réglages de canaux, un plan, une calibration. Jamais de pixels dans le fichier JSON.
- Les canaux se **recolorent sans nouveau rendu**, avec la même arithmétique que le shader de coupe.
- La résolution **native** arrive après l'ouverture, par un chemin « plan » ou « atlas », sous la protection d'un **jeton** de document.

Le chapitre 12 (§ 12.7) a montré comment annoter. Voici ce qui se passe quand vous cliquez.

Le document

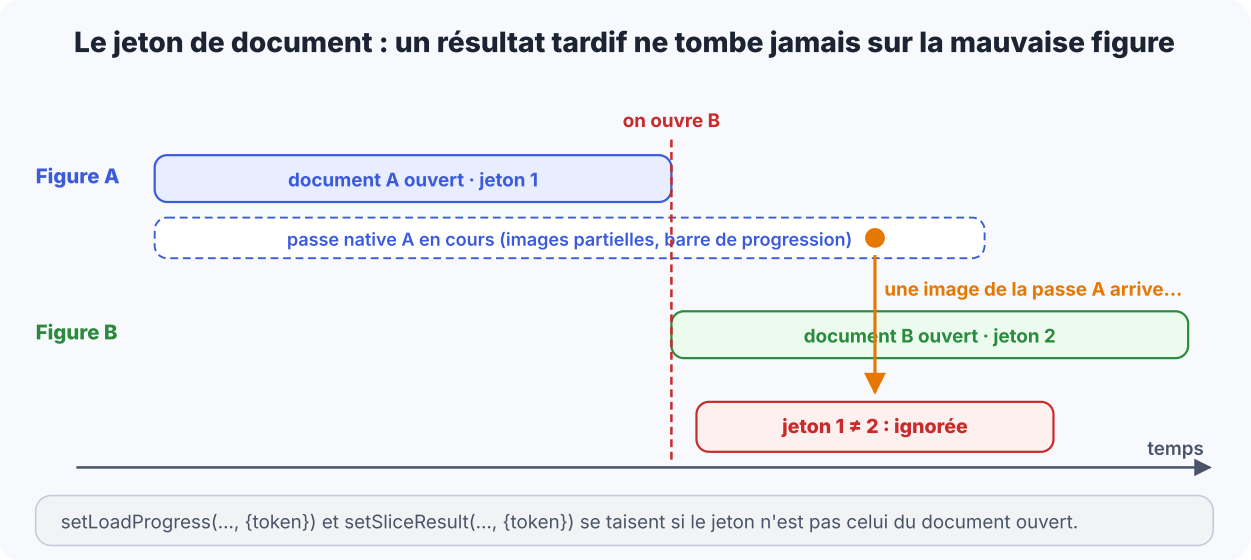


▲ L'anatomie d'un document du Studio, et ce que le fichier JSON ne contient jamais.

Le document tient en un objet. Le Studio le **copie entièrement** à chaque étape de l'historique, avec deux conséquences : l'annulation est instantanée et sans surprise, et le document ne doit pas contenir de gros objets. Dans une figure de Comparer, chaque cellule porte trois champs d'exécution (les valeurs brutes, le cadre de page, la coupe) : `_portableDocument()` les retire de chaque export, et la copie d'historique les garde **par référence**, pas par duplication.

- **L'historique** garde 80 étapes. Une modification de canal est une étape comme une autre : annuler défait la **dernière** modification, quelle qu'elle soit, et défaire une annotation ne remet jamais les canaux d'avant.
- Une rafale de modifications d'un même panneau (un curseur qu'on tire, un message par image) forme **une seule étape**, close par le relâchement, par une pause d'une seconde, ou par une autre action.
- **Types de calques** : rectangle, ellipse, texte, ligne, flèche, distance, angle, barre d'échelle. Chaque calque a un identifiant unique, un nom, deux drapeaux (visible, verrouillé), un groupe éventuel, une **rotation** en degrés (de −180 exclus à 180) et un style.
- Les coordonnées sont en **pixels de l'image**. Quand la passe native remplace l'aperçu par une image plus grande, chaque coordonnée est multipliée par le rapport des tailles : un calque reste sur la structure qu'il marque (les épaisseurs et corps de police, eux, restent en pixels d'écran).

Le jeton de document



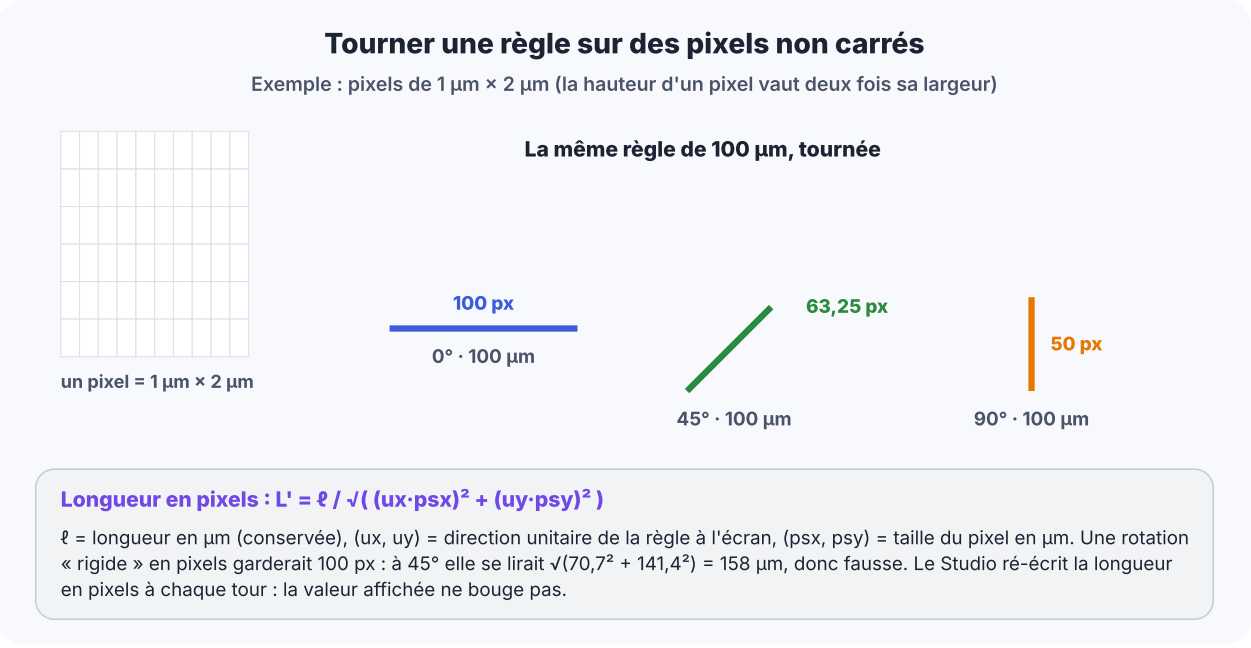
▲ Pourquoi une passe native tardive ne peut pas abîmer la figure suivante.

L'ouverture d'une figure renvoie un **jeton** (un numéro qui augmente à chaque document). La passe native, qui peut durer plusieurs minutes sur un gros volume, le présente à chacune de ses mises à jour. Si vous avez fermé la figure et en avez ouvert une autre, le jeton ne correspond plus : l'image tardive est **ignorée**, sa barre de progression aussi.

Fermer le Studio libère l'image native, l'historique et les cadres de Comparer, et annule le transfert en cours.

Tourner un calque sans fausser une mesure

Un rectangle, une ellipse ou un texte **tournent à l'affichage** autour de leur centre : seule la rotation est enregistrée. Une ligne, une flèche, une distance, un angle ou une barre d'échelle sont faits de **points** : la rotation est *cuite* dans les points, parce que leur étiquette se calcule à partir d'eux.



▲ Une règle de 100 µm tournée sur des pixels de 1 × 2 µm : sa longueur en pixels est recalculée.

- **Ligne, flèche** : rotation rigide des points autour de leur centre.
- **Angle** : tourné dans l'espace des **micromètres** (`p' = c + S⁻¹·R(θ)·S·(p − c)` , avec `S` la taille de pixel). Une rotation rigide en pixels changerait l'angle mesuré : sur des pixels de 1 × 2 µm, un angle droit tourné de 45° se lirait 53,1°.
- **Distance** : la direction tourne, la longueur en pixels est **recalculée** pour garder la même longueur en µm.
- **Barre d'échelle** : sa direction *est* sa rotation ; la longueur en pixels est celle qui mesure exactement sa valeur.

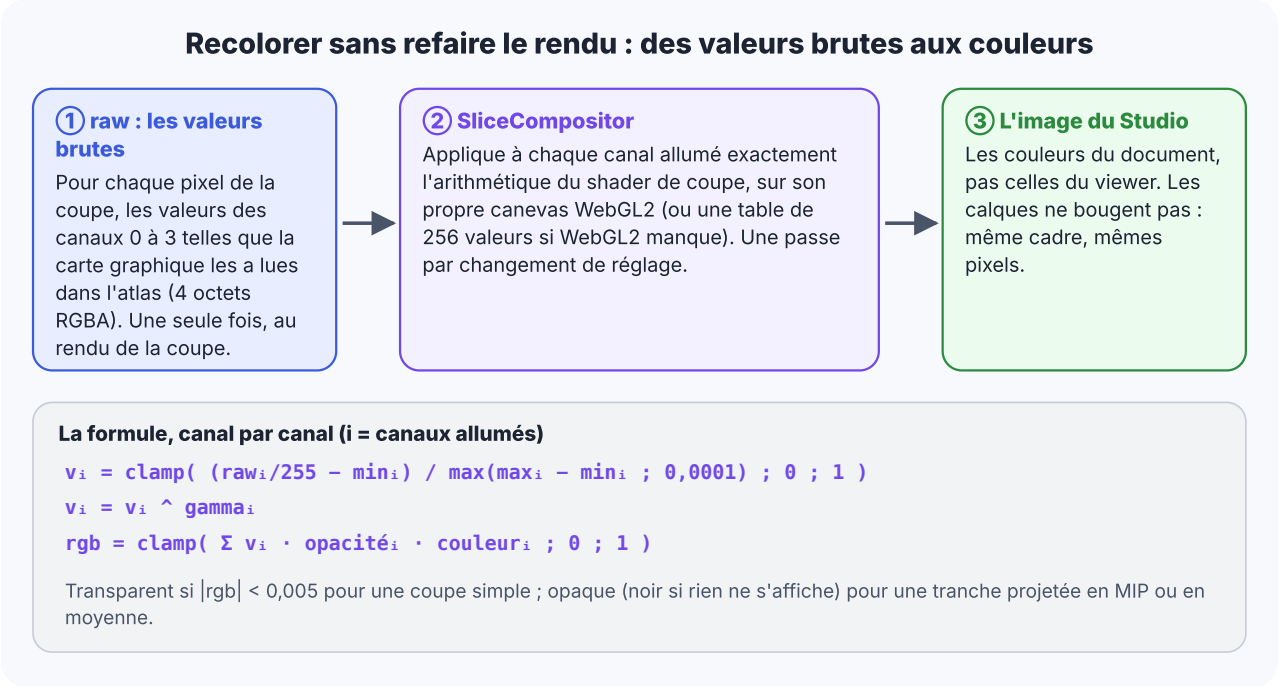
> 🔢 **MICRO-EXEMPLE**
>
> Distance de 100 µm sur des pixels de 1 × 2 µm. Horizontale : 100 px. À 45° : 100 / √(0,707² × 1² + 0,707² × 2²) = **63,25 px**. Verticale : **50 px**. La valeur affichée reste 100 µm.

Quelle taille de pixel pour un calque ?

Une coupe simple n'a qu'une calibration. Une **figure de Comparer** en a une **par cellule** (un panneau = une cellule avec son rectangle et son µm/pixel). Un calque de points prend la calibration **de la cellule qui contient le milieu de ses points** (la plus proche, si le milieu tombe dans une gouttière). C'est aussi le centre de ses rotations : tourner un calque ne le confie donc jamais à la cellule voisine.

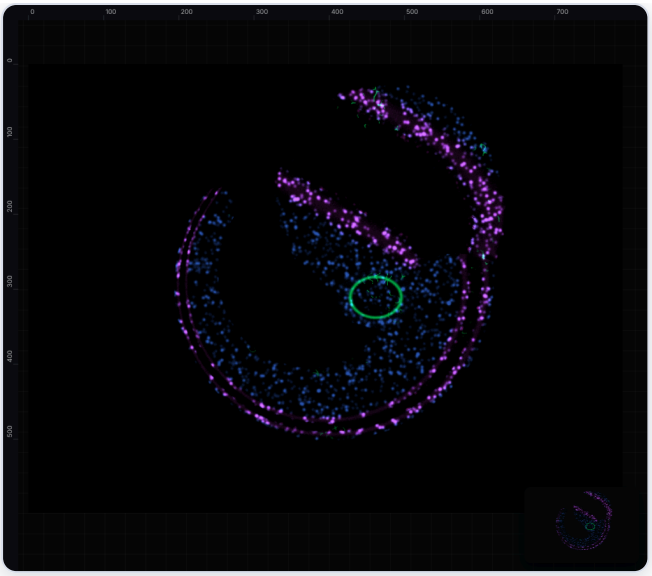
La barre d'échelle est plus subtile : son milieu dépend de la longueur qu'elle prend, qui dépend de la cellule. Le Studio cherche donc une cellule **cohérente** (dont la longueur propre place le milieu dans cette même cellule) ; à défaut, c'est la cellule du point de départ qui décide, car elle ne bouge pas quand l'autre bout est réécrit.

Recolorer sans refaire le rendu

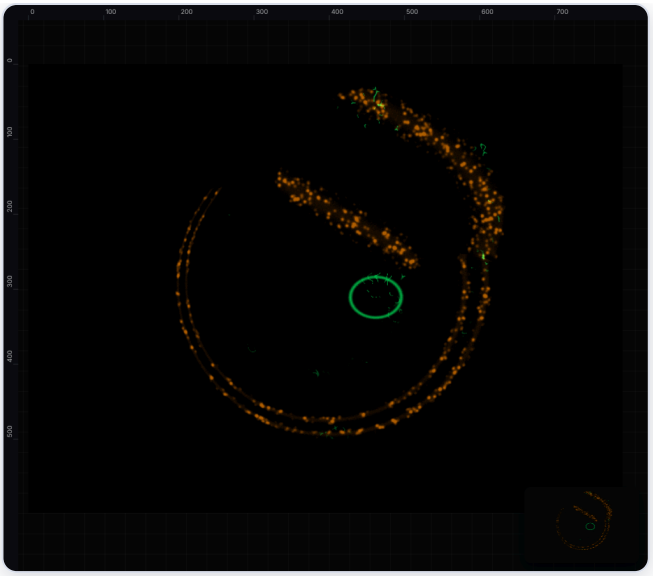


▲ Le chemin des valeurs brutes aux couleurs.

Quand le Studio ouvre une coupe, il reçoit deux choses : une image colorée, et les **valeurs brutes** des canaux (`raw` : quatre octets par pixel, un par canal 0 à 3). Le volume qui les a produites est un atlas jetable, disparu après la passe. La recoloration repart donc des valeurs brutes.



▲ Coupe XY du jeu de démonstration, couleurs du viewer.



▲ Même coupe : DAPI éteint, Sox2 en orange. Les calques n'ont pas bougé.

Le **compositeur** (`SliceCompositor`) applique à chaque canal allumé la formule du shader de coupe : fenêtre `(brut/255 − min) / max(max − min ; 10⁻⁴)` , puis gamma, puis `opacité × couleur` , et somme. Il tourne sur **son propre canevas WebGL2** (la page Comparer ne charge pas Three.js), avec une table de 256 valeurs par canal en repli.

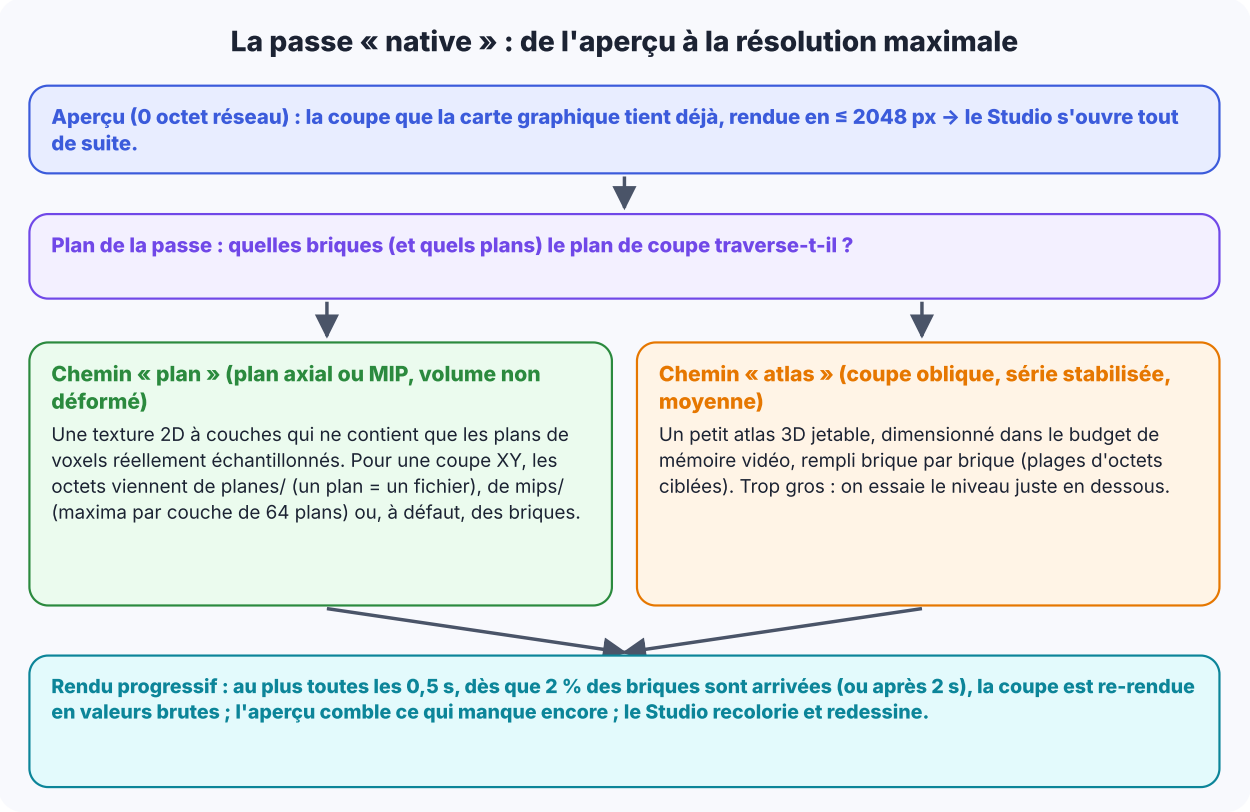
🔧 **POUR LES CURIEUX**

- La couleur de sortie est `clamp(Σ vᵢ·opacitéᵢ·couleurᵢ ; 0 ; 1) × 255` , **transparente** sous |rgb| < 0,005 pour une coupe simple (le shader de coupe écarte ces fragments), **opaque** (noire si rien ne s'affiche) pour une tranche projetée en MIP ou en moyenne.
- Une tranche de **quatre canaux** n'a pas d'octet libre pour dire où le volume existe : elle porte à part un **masque de couverture** d'un octet par pixel, lu sur l'alpha du rendu en couleur du même plan.
- Les textures des valeurs brutes sont gardées dans un budget de **256 Mio** (celles des masques dans un budget séparé de 64 Mio, pour qu'ils ne s'évincent pas). Un raw de même taille est **réécrit sur place** à chaque raffinement de la passe native (numéro `raw.version`).
- Le repli sur table calcule en double précision ; la carte graphique, en flottants 32 bits, peut différer d'**une unité** sur un octet à une frontière d'arrondi.

Les histogrammes du Studio n'ont pas la même origine que ceux du viewer (§ 11.5). Ils sont calculés **sur la coupe affichée** (jusqu'à 4 millions d'échantillons, un seul calcul par raw), pas sur tout le volume. C'est cohérent : vous réglez le contraste de ce que vous voyez dans la figure.

La passe native

Un volume de plusieurs gigaoctets n'a pas de coupe « à portée de main » en pleine résolution. Le Studio fait donc deux temps.



▲ De l'aperçu à la résolution native.

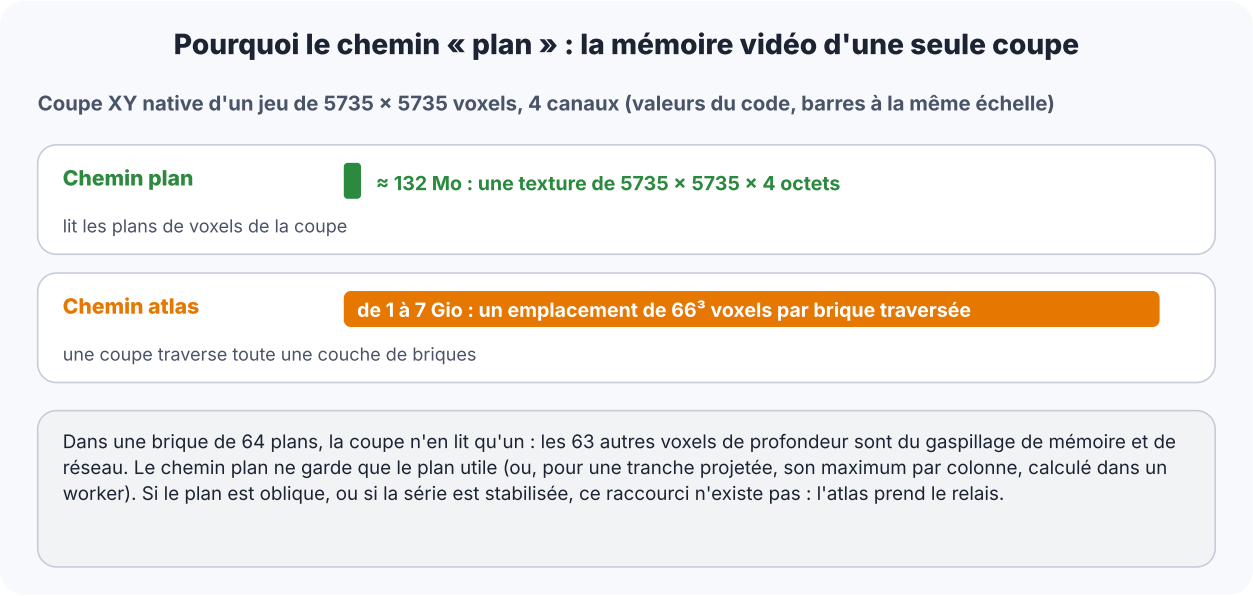
1. **L'aperçu** : la coupe que la carte graphique tient déjà, rendue en 2048 px au plus, sans aucun octet de réseau. Le Studio s'ouvre à l'instant.
2. **La passe native** : le viewer calcule quelles briques le plan traverse, les récupère à la résolution maximale (niveau 0), et re-rend la coupe en valeurs brutes **à intervalles réguliers**.

L'image de la passe est cadrée comme l'aperçu (même région du plan). Là où une brique manque encore, l'**aperçu comble le trou** : ce que vous voyez est toujours une image complète, de plus en plus nette. La taille de rendu est environ **un pixel par voxel de l'axe le plus long**, étendue au cadre de la coupe (× 1,5) : de 512 à 16 384 px.

🔢 **MICRO-EXEMPLE**

Jeu de démonstration, 768 voxels sur X : 768 × 1,5 = **1 152 px** de rendu. Chaque pixel couvre 1,5 × 921,6 µm (la plus grande extension physique) / 1 152 = **1,2 µm**, c'est-à-dire exactement la taille du voxel : le natif de ce jeu est aussi petit que l'aperçu.

Deux chemins pour le même plan



▲ Pourquoi un plan axial n'a pas besoin d'atlas.

- **Chemin « plan »** : pour un plan aligné sur un axe (XY, XZ, YZ) d'un volume non déformé. Le shader lit, pour chaque pixel, le voxel `floor(uvw × dim)` ; le long de la normale, la coordonnée ne dépend pas du pixel, donc les plans de voxels lus sont **connus d'avance**. Une texture 2D à couches ne contient que ceux-là. Pour une tranche MIP, chaque boîte de briques est réduite à son maximum par canal dans un *worker* (`studio-plane-worker.js`) puis les maxima des briques d'une colonne sont fusionnés.
- **Chemin « atlas »** : coupe oblique, série stabilisée, tranche en moyenne. Un atlas 3D jetable, dimensionné **avant** d'être alloué contre le budget de mémoire vidéo (chapitre 10) ; s'il ne rentre pas (`SVR_OVER_BUDGET`), le niveau juste en dessous est essayé et l'étiquette de progression le dit.

D'où viennent les octets d'un plan XY ?

SOURCE	QUAND	CE QU'ELLE ÉPARGNE
`planes/` **(format 2 et plus)**	coupe XY, niveau 0	les 63 autres plans de chaque brique
`mips/` **(format 3 et plus)**	tranche MIP sur toute la pile	un maximum par couche de 64 plans au lieu de 64 plans
briques par plages d'octets	tout le reste	les briques non traversées et les paquets entiers

Les formats sont détaillés au chapitre 7 (§ 7.8 et 7.9) ; ici, le Studio ne fait que **choisir le plus économe** qui existe, et retombe sur les briques si le dossier ou un fichier manque.

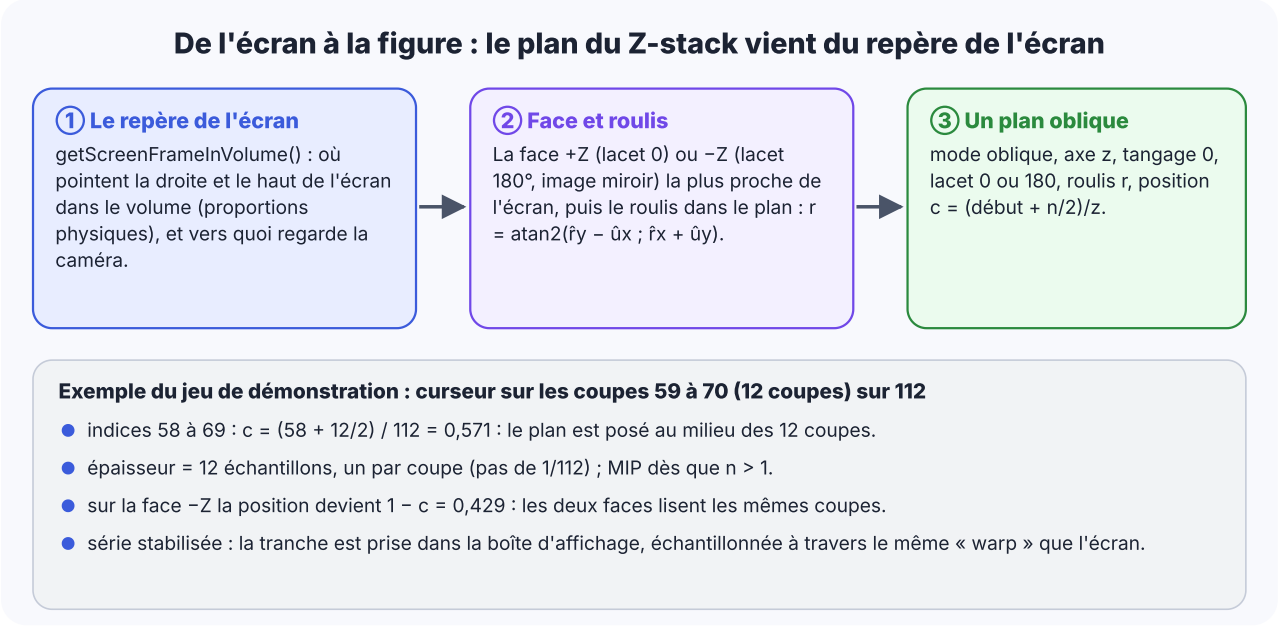
🔧 POUR LES CURIEUX

- **Brique vide ≠ brique manquante.** Une brique que le pipeline n'a pas stockée (tous ses voxels valent 0) est du **zéro**, jamais « en attente » : le chemin atlas pointe son entrée sur un seul emplacement de zéros ; le chemin plan marque ses texels présents d'emblée. Sinon l'aperçu remplacerait pour toujours chaque colonne qui traverse une brique vide.
- **Raffinement progressif** : au plus toutes les 0,5 s, dès que 2 % des briques de la passe sont arrivées (ou après 2 s quoi qu'il arrive). Un rendu n'est pas relancé plus souvent qu'un petit multiple du coût du précédent.
- **Échec** : un morceau qui échoue deux fois garde les pixels de l'aperçu et est compté (`missingChunks`). L'image finale est alors étiquetée « native partielle » et le Studio le dit.
- **Canaux éteints dans le viewer** : quand le transport est par canal, ils ne sont pas téléchargés ; allumés ensuite dans le Studio, ils s'affichent à la résolution de l'aperçu plutôt qu'en noir.
- **Indépendance** : le lot de briques de la passe a son propre groupe et son propre signal d'annulation : un chargement lancé par le viewer ne l'annule pas, fermer le Studio l'annule.

⚠️ ATTENTION

Plus de 256 Mio. Une figure de Z-stack épaisse peut réclamer chaque brique de toutes ses coupes. Au-delà de 256 Mio estimés, une boîte propose trois choix : charger le natif (avec le nombre de Mo), charger le niveau inférieur (plus léger), ou garder l'aperçu. L'estimation passe par `planes/` et `mips/` quand ils existent : elle peut être très inférieure à celle des briques, et la boîte ne propose jamais « plus léger » pour plus d'octets.

La figure d'un Z-stack



▲ Le plan d'une figure de Z-stack naît du repère de l'écran.

Quand vous ouvrez le Studio depuis l'explorateur Z-stack, le plan n'est pas celui de la coupe oblique : il est construit pour que la figure **soit** l'écran. Le viewer demande où pointent la droite et le haut de l'écran dans le volume (`getScreenFrameInVolume`), choisit la face **+Z** ou **−Z** (donc un lacet de 0 ou 180°) la plus proche, puis le **roulis** qui cale l'image sur l'écran. Les `n` coupes du curseur deviennent une tranche de `n` échantillons, un par coupe ; au-delà d'une coupe, c'est un MIP.

Exporter, importer, et les limites

	RÈGLE
PNG	taille de la figure, fond noir, **légende toujours incrustée** (jeu · plan · taille · pixel). Refusé au-delà de 16 384 px de côté ou 2^{28} pixels
JSON	calques, repères, canaux, plan, calibration, cellules de Comparer. Jamais de pixel
Lecture d'un JSON	refus au-delà de 5 Mo, 2 000 calques, 10 000 points ; texte limité à 2 000 caractères
Historique	80 étapes
Canaux	4 au plus (la texture est RGBA)

Importer un JSON ne remplace pas la figure ouverte : il l'**applique**. Le fichier est vérifié en entier avant tout changement (types de calques connus, géométrie numérique, couleurs, identifiants uniques).

- La figure ouverte garde son image, son cadre, sa calibration et ses cellules ; elle prend les **calques, repères et groupes** du fichier, avec les mesures relues contre la calibration de la figure ouverte (une barre d'échelle garde sa valeur, une distance ses points).
- C'est **la même figure** si le fichier a la même taille, nomme le même jeu (identifiant ou chemin), montre le même plan (à une tolérance près) et la même image de la série, et, pour Comparer, les mêmes cellules.
- Dans ce cas les **réglages de canaux** du fichier s'appliquent aussi, et seulement si la coupe peut être recolorée sans changer de cadre. Sinon (autre figure), seuls les **calques** passent, et un message l'indique.

📌 **À RETENIR**

- Un document = calques + repères + canaux + plan + calibration, **sans pixels**.
- La recoloration part des **valeurs brutes** : aucun nouveau rendu du volume.
- La passe native choisit **plan** ou **atlas**, lit `planes/` et `mips/` quand ils existent, et se protège par un **jeton**.
- Une rotation de calque **recalcule** les longueurs en pixels : la mesure ne bouge pas.

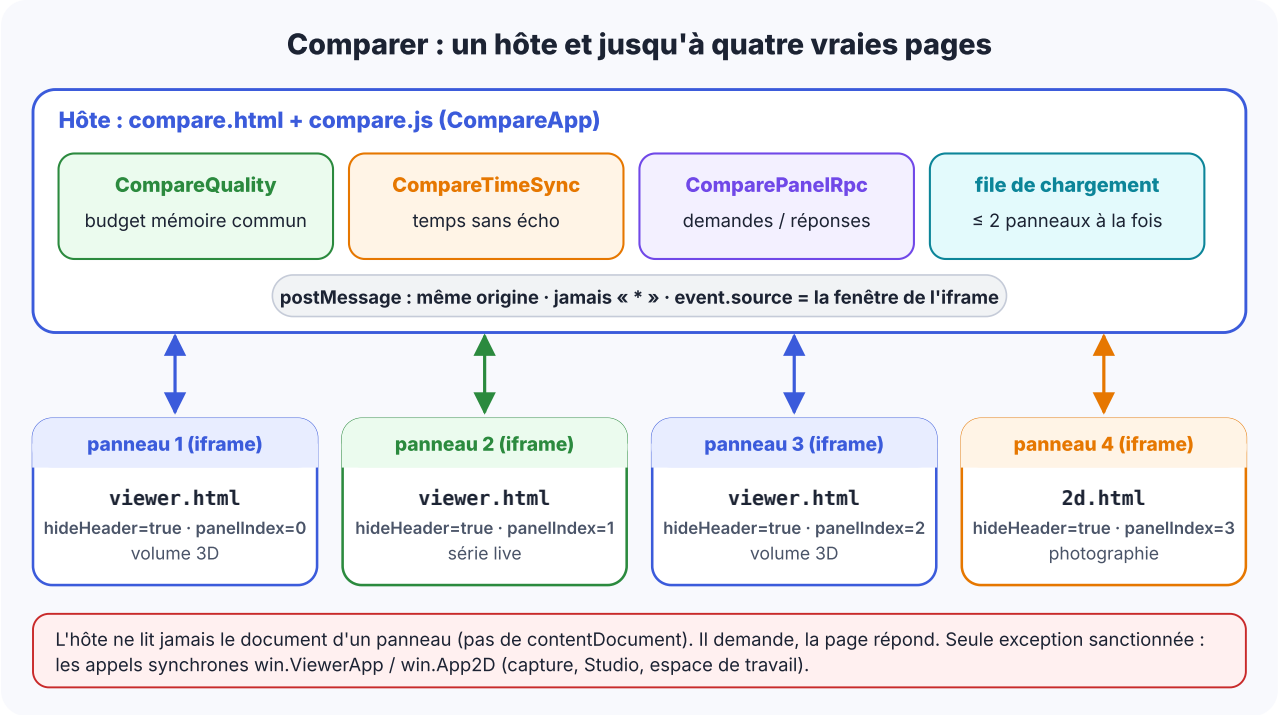
13.4 Comparer sous le capot

⏱ **EN 30 SECONDES**

- Comparer est un **hôte** qui pilote jusqu'à quatre **vraies pages** (`viewer.html` , `2d.html`) enfermées dans des cadres, par des messages.
- Chaque page **décrit** ce qu'elle sait faire ; l'hôte en dessine les boutons. Il ne fouille jamais dans leur document.
- Un **budget mémoire commun** de 1,5 Gio est réparti entre les panneaux par un petit algorithme, et le temps est aligné par **fraction**.

Le chapitre 12 (§ 12.8) a décrit la page. Voici le protocole qui la fait tenir.

L'architecture



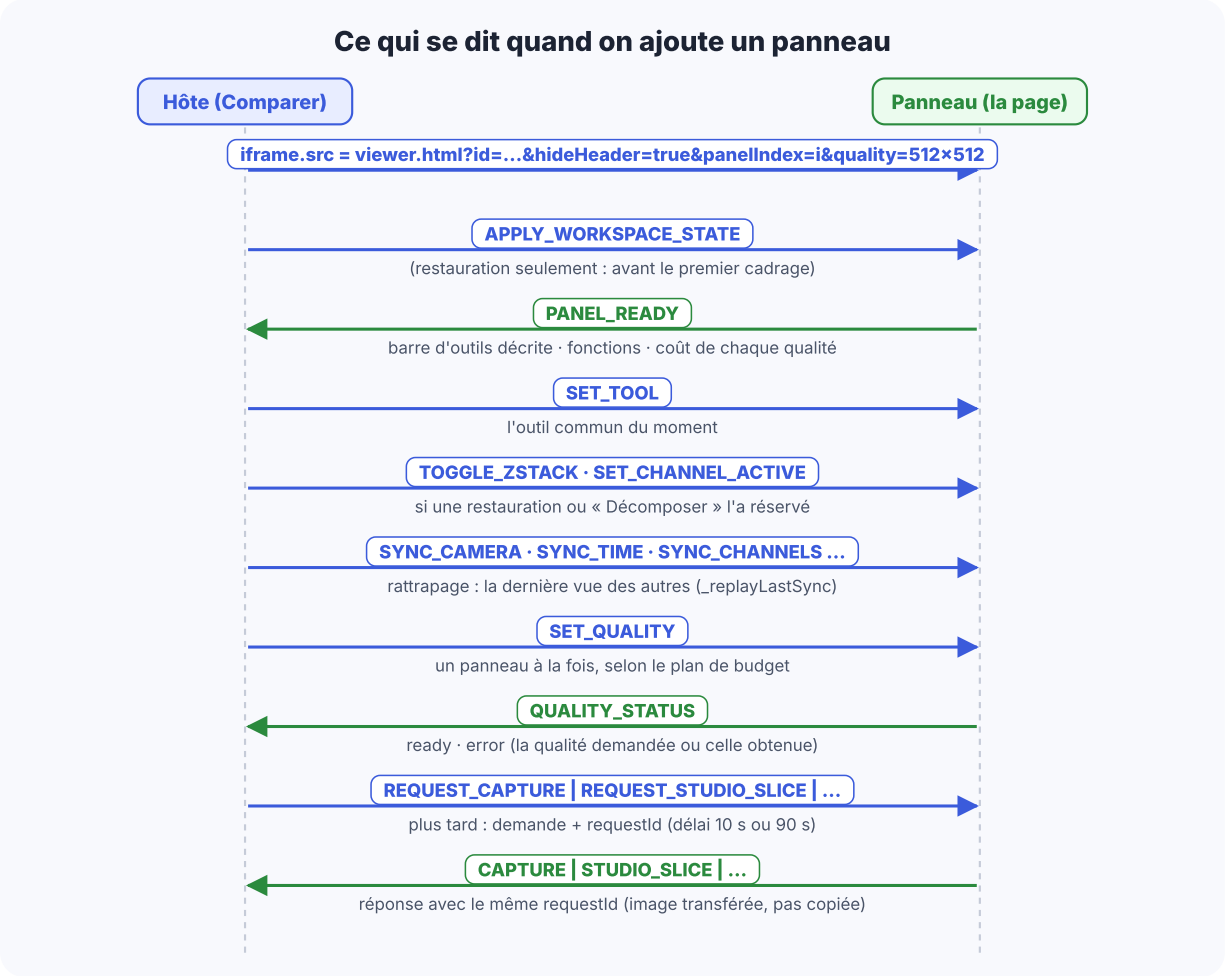
▲ Un hôte, quatre cadres, et un seul canal : les messages.

Chaque panneau est une **page complète** ouverte sans son en-tête (`hideHeader=true`) avec un numéro (`panelIndex`). Comme elle est chrome-less, sa barre d'outils est invisible : la page **décrit** donc sa barre à l'hôte, qui dessine les boutons dans l'en-tête du panneau.

❓ **POURQUOI CE CHOIX ?**

Pourquoi des pages entières et pas un moteur partagé ? Parce que chaque outil, chaque raccourci, chaque réglage existe déjà dans la page. Comparer n'a **rien à réimplémenter** : un nouvel outil devient disponible dans les panneaux dès qu'il déclare qu'il sait être piloté de l'extérieur (`contexts: ["page", "panel"]` , chapitre 15).

Une poignée de main en cinq temps



▲ Ce qui se dit quand on ajoute un panneau.

1. L'hôte donne au cadre son adresse (avec `quality=512x512` pour un volume : on démarre toujours petit).
2. Dès que la page est montée, elle envoie `PANEL_READY` : nom, type, **barre décrite** (outils et boutons à bascule, avec leur état), fonctions (volume, photographie, frise, nombre de canaux, suivi), qualité, et **le coût de chaque qualité** en octets de mémoire graphique.
3. L'hôte répond par l'état commun : l'outil du moment, puis le rattrapage des synchronisations (voir plus bas).
4. Il règle la qualité, **un panneau à la fois**.
5. Plus tard, il envoie des **demandes** (capture, coupe pour le Studio, état…) auxquelles la page répond.

Une page qui n'envoie pas `PANEL_READY` en **180 s** est déclarée en échec ; le panneau affiche un bouton **Réessayer** qui recharge le cadre à neuf. Une erreur après la mise en place n'ôte jamais l'état « prêt » : le volume déjà affiché est intact. Au plus **deux** panneaux chargent à la fois, les suivants font la queue.

Le catalogue des messages

SENS	MESSAGES
panneau → hôte	`PANEL_READY`, `PANEL_ERROR`, `PANEL_DATASET`, `PLUGIN_STATE`, `TOOL_CHANGED`, `QUALITY_STATUS`, `SIDEBAR_CLOSED`, `REQUEST_COMPARE_STUDIO`
synchronisations (panneau → hôte → autres)	`SYNC_CAMERA`, `SYNC_CHANNELS`, `SYNC_EXPOSURE`, `SYNC_Z`, `SYNC_TIME`, `SYNC_SLICER_SPEC`, `SYNC_ZSTACK_SLICE`, `WM_PHYSICAL_VIEW`
hôte → panneau	`SET_TOOL`, `PLUGIN_ACTIVATE`, `SET_QUALITY`, `TOGGLE_SIDEBAR`, `TOGGLE_ZSTACK`, `SET_CHANNEL_ACTIVE`, `APPLY_WORKSPACE_STATE`, `PANEL_HELLO`, `WM_SET_PHYSICAL_VIEW`
demandes (hôte → panneau)	`REQUEST_CAPTURE`, `REQUEST_STUDIO_SLICE`, `REQUEST_WORKSPACE_STATE`, `REQUEST_CHANNEL_STATE`
réponses (panneau → hôte)	`CAPTURE`, `STUDIO_SLICE`, `WORKSPACE_STATE`, `CHANNEL_STATE`

Qui a le droit de parler

- **Jamais d'origine générique.** Les messages partent vers l'origine de la page (`Utils.trustedTargetOrigin()`), jamais vers `*`.
- **À la réception**, l'hôte vérifie l'origine, puis que `event.source` est bien **la fenêtre du cadre qu'il a monté** à ce numéro. Une page nichée dans un panneau, qui porterait le même numéro, est ignorée.
- Une **réponse** n'est acceptée que du panneau auquel la demande est partie, avec le bon type, **une seule fois**.

Demandes et réponses

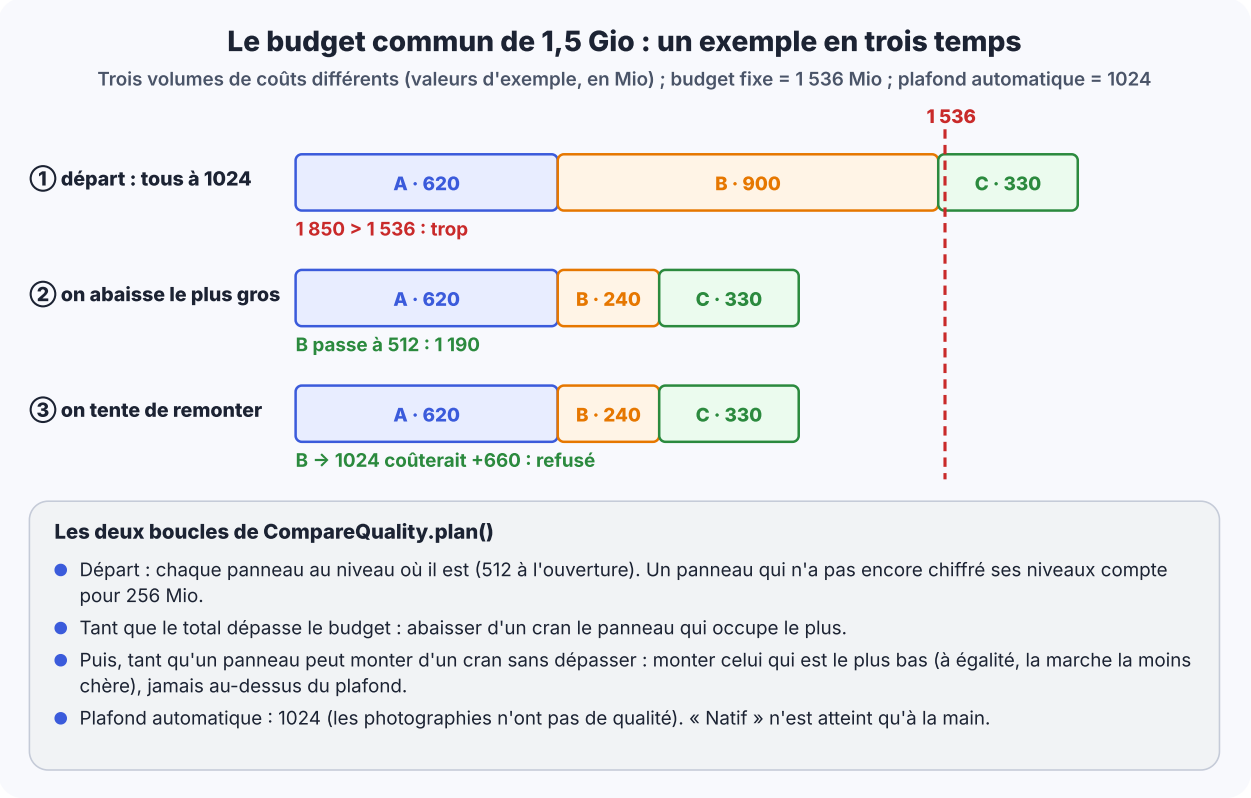
Chaque demande porte un `requestId` (préfixe aléatoire + compteur). La réponse le reprend. Si elle ne vient pas à temps, la demande échoue avec un code précis : `COMPARE_TIMEOUT`, `COMPARE_UNREACHABLE` (panneau fermé ou injoignable), `COMPARE_PANEL_ERROR` (la page a répondu « impossible »).

DEMANDE	DÉLAI
capture, état de l'espace de travail, état des canaux	10 s
coupe pour le Studio (peut rendre une coupe native)	90 s

Les images voyagent comme `ImageBitmap` **transférés** (pas copiés) : le panneau rend sa vue et la photographie **dans la même tâche** (son canevas WebGL ne conserve pas son image), puis la transfère. Si la réponse arrive trop tard, ses images sont libérées au lieu de fuir.

Le budget mémoire commun

Les panneaux partagent **un seul processeur graphique**, mais chacun croit disposer de toute la mémoire. « 1024 » peut valoir quelques centaines de Mo ou plus d'un Go selon le jeu. Compter les panneaux ne suffit donc pas : chaque volume **annonce le prix de chaque qualité** (`qualityBytes`, octets d'atlas), et l'hôte répartit un budget **fixe de 1,5 Gio** (une constante de `compare.js`, pas une mesure de votre carte).



▲ Un exemple en trois temps avec trois volumes (valeurs d'exemple).

L'algorithme (`CompareQuality.plan`) tient en deux boucles :

1. **Abaisser** : tant que le total dépasse le budget, le panneau qui occupe le plus descend d'un cran.
2. **Remonter** : tant qu'un panneau peut monter d'un cran sans dépasser, le plus bas monte (à égalité, la marche la moins chère), sans dépasser le plafond.

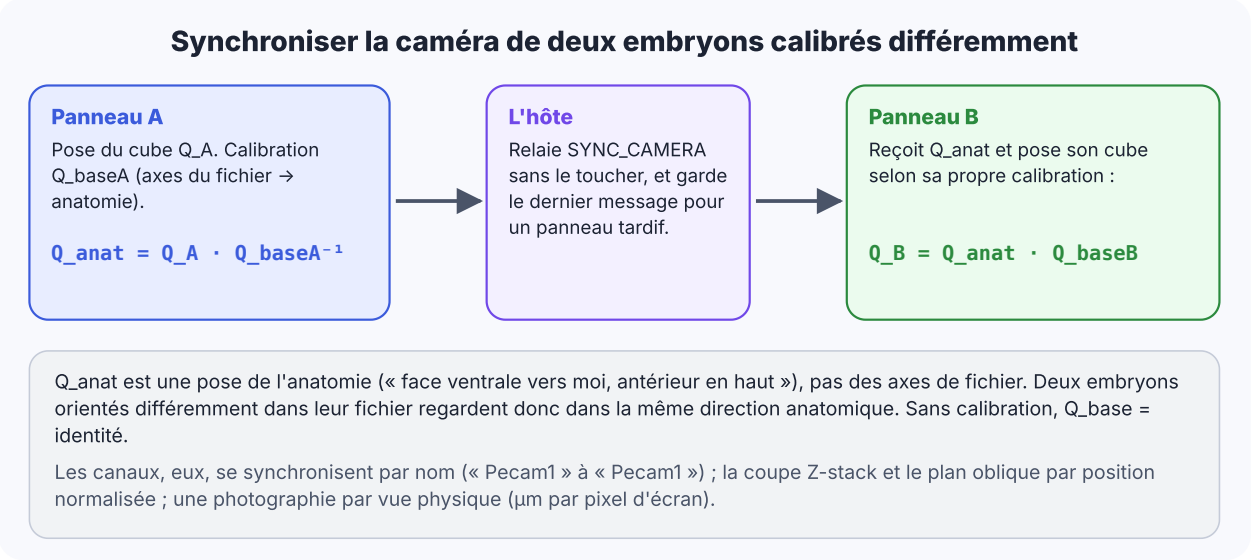
🔢 **MICRO-EXEMPLE**

Trois volumes à 1024 coûtent 620 + 900 + 330 = 1 850 Mio, pour un budget de 1 536. Le plus gros (900) passe à 512 (240) : total 1 190. Le remonter coûterait +660 : 1 850 > 1 536, refusé. Résultat : A et C à 1024, B à 512.

- Le **plafond automatique** est 1024 : « natif » ne s'obtient qu'en le choisissant à la main. Les **photographies n'ont pas de qualité** et ne comptent pas.
- Un panneau qui n'a pas encore chiffré ses niveaux **réserve 256 Mio**.
- Les montées se font **un panneau à la fois** : le suivant attend le `QUALITY_STATUS` du précédent. Un panneau qui ne se stabilise pas en **180 s** est abandonné (message en console), et le niveau qu'il n'a pas atteint n'est plus redemandé.
- Si la mémoire de la carte a fait descendre un panneau **plus bas** que demandé, ce niveau demandé compte aussi comme échoué : sinon l'hôte le redemanderait sans fin et rechargerait le volume à chaque passage.
- Une page ancienne, qui ne chiffre pas ses niveaux, suit l'ancienne règle par **comptage** : 1024 jusqu'à deux volumes, 512 au-delà, et elle ne fait que monter.
- Choisir 512, 1024 ou Natif à la main force ce niveau pour tous les volumes.

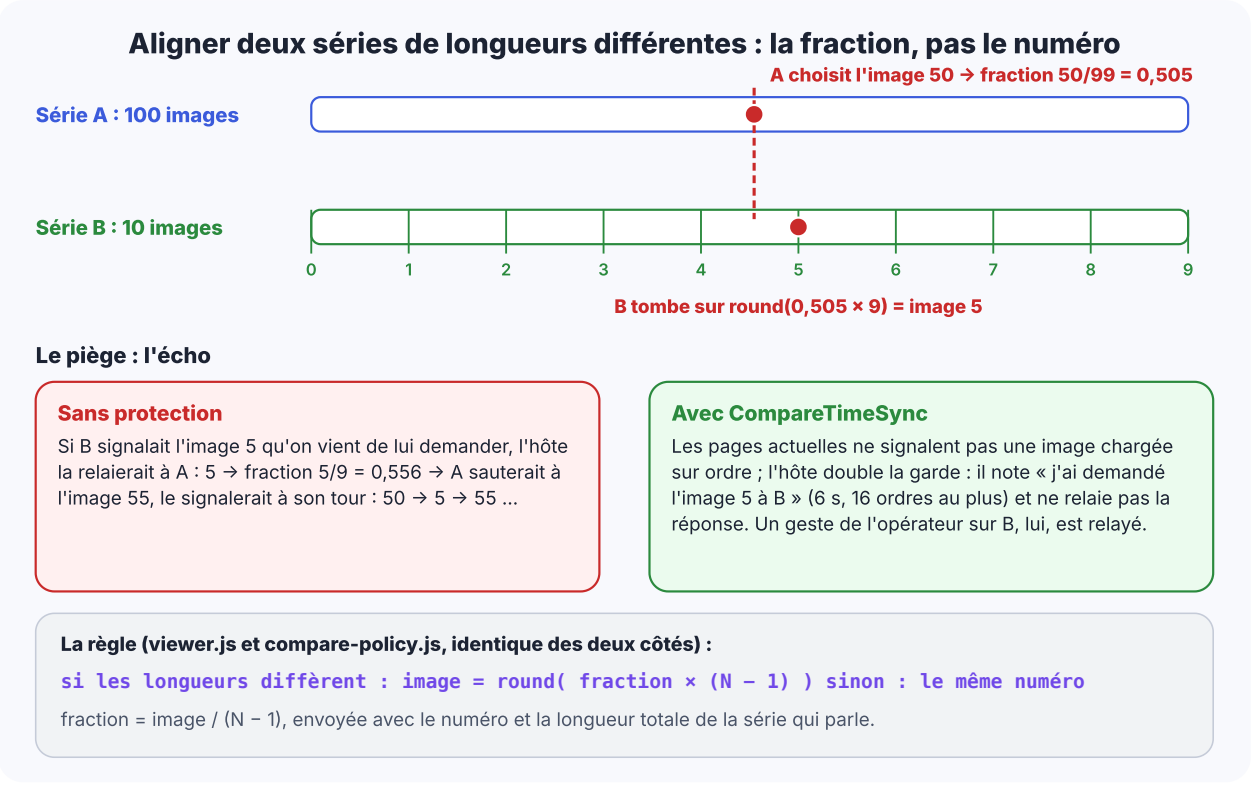
Garder les panneaux au même endroit

La caméra. Chaque jeu a sa propre calibration `Q_base` (§ 13.9). Deux embryons orientés différemment dans leur fichier n'ont donc pas la même pose de cube pour la même vue anatomique. Le panneau qui bouge envoie `Q_anat = Q_cube · Q_base⁻¹` ; le panneau qui reçoit pose son cube à `Q_anat · Q_base`. Pendant que le Z-stack verrouille la vue de dessus, l'orientation reçue est ignorée mais le zoom est gardé.



▲ La caméra se synchronise en pose anatomique, pas en pose de fichier.

Le temps. Deux séries de longueurs différentes s'alignent sur la **fraction** de leur durée.



▲ Pourquoi la fraction, et comment l'écho est évité.

La règle est la même des deux côtés : si les longueurs diffèrent, `image = arrondi(fraction × (N − 1))`, sinon le même numéro. Elle est doublée d'une garde : l'hôte note chaque image qu'il a demandée

à un panneau (fenêtre de 6 s, 16 ordres au plus). L'annonce de cette image par le panneau est la **réponse** à l'ordre, pas une action de l'opérateur : elle n'est pas relayée.

Les canaux se synchronisent **par nom** (« Pecam1 » avec « Pecam1 »), exposition comprise. **Le plan Z, le plan oblique et la tranche du Z-stack** voyagent en position normalisée. **Une photographie** échange une vue **physique** (§ 13.5) ; une photo non calibrée n'en envoie ni n'en reçoit.

Chaque famille est réglable par un interrupteur de l'en-tête (Plan Z, Temps, Caméra / vue, Canaux).

Un panneau qui arrive en retard

L'hôte **mémorise le dernier message de chaque famille** (caméra, vue physique, temps, chaque canal, exposition, plan Z, plan oblique, Z-stack) et le nom du panneau qui l'a produit. Quand un nouveau panneau devient prêt, `_replayLastSync` lui rejoue ces messages : il s'ouvre là où sont les autres, dans la pile, avec les mêmes canaux éteints. Si un panneau est fermé, ce qu'il avait produit est **oublié** : un panneau ajouté plus tard ne s'ouvrirait pas sur un jeu qui n'existe plus.

Une restauration d'espace de travail ne rejoue rien : l'état sauvegardé du panneau a la priorité.

La figure de la grille

L'export (§ 12.8) compose **la grille visible** sur un canevas de 2× la taille d'écran, plafonné à **4096 px** de côté. Pour chaque panneau l'hôte envoie `REQUEST_CAPTURE`, dessine l'image reçue dans sa cellule, ajoute le titre et une signature. Un panneau qui ne répond pas est remplacé par un cadre gris avec son nom, et le nombre de panneaux de secours est compté. Le format est PNG ou WebP (qualité 0,95) ; le fond suit le thème.

Le **Studio de Comparer** réutilise le Studio du § 13.3 : une cellule par panneau. En **échelle physique**, tous les panneaux calibrés sont dessinés à **un seul µm par pixel**, celui du plus grossier, et ne sont jamais agrandis : une barre d'échelle est vraie pour tous.

L'espace de travail

Le bouton **Sauvegarder** garde, dans le navigateur, l'état de **chaque panneau** plus celui de l'hôte :

`panels` · `panelTypes` · `layoutMode` · `layoutWeights` · `sync` · `tool` · `quality` · `panelZstackStates` · `panelSoloChannels` · `iframeStates`.

- L'hôte **attend que tous les panneaux soient prêts** ; un panneau en échec est gardé dans la liste (rouvrir l'espace de travail le réessaie) avec un état vide. Avant cela, l'état est « pas encore disponible » et l'adresse n'est pas mise à jour.
- Le **cache de briques** est retiré de l'état d'un panneau : il se remplit tout seul pendant le streaming et changerait l'adresse chaque seconde.
- À la **restauration**, l'état d'un panneau est envoyé dès que son document existe (événement `load` du cadre), pour que le viewer saute son premier cadrage de caméra. Une qualité inconnue est refusée : elle ne serait jamais acquittée et bloquerait la file des qualités.
- L'adresse (`#state=…` , § 13.7) n'est écrite qu'après un **changement de l'opérateur** : les panneaux ouverts par l'adresse elle-même (`?add=`) sont l'état intact de la page, et le point de comparaison est repris quand ils sont tous prêts.

📌 **À RETENIR**

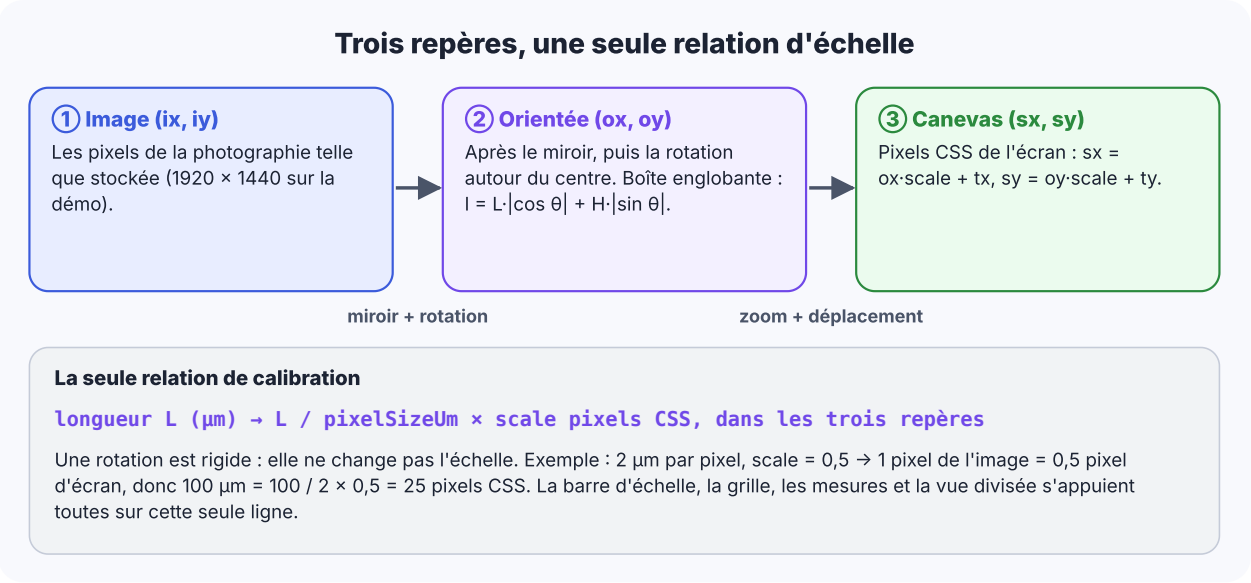
- Les panneaux sont les **vraies pages** ; l'hôte demande, la page répond.
- Origine explicite, `event.source` contrôlé, demandes avec `requestId` et délai.
- Qualité : **budget fixe de 1,5 Gio**, on abaisse le plus gros puis on remonte le plus bas.
- Temps : **fraction**, avec une garde contre l'écho. Caméra : pose **anatomique**.

13.5 La page 2D en profondeur

⏱ **EN 30 SECONDES**

- Le moteur est un **canevas** qu'on déplace et qu'on zoome : trois repères, une relation d'échelle.
- Réglages d'affichage et isolement du marquage tournent dans un **worker**, sur une copie ; la photographie mesurée n'est jamais modifiée.
- L'importeur lit la **taille de pixel** dans les balises du TIFF et le **nom du fichier** pour le stade.

Trois repères



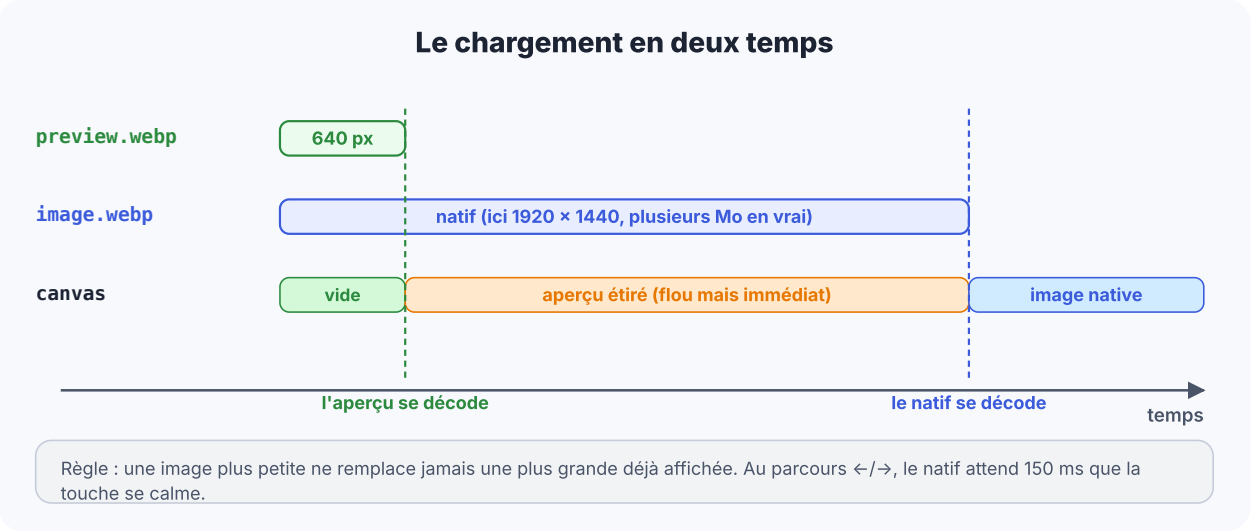
▲ Image, orientée, canevas : et la seule relation de calibration.

Le moteur `Viewer2D` n'utilise pas Three.js. Il dessine un canevas 2D avec la transformation `sx = ox·scale + tx`. Le miroir est appliqué **avant** la rotation, autour du centre de l'image ; la boîte englobante de l'image tournée sert à cadrer. L'angle est normalisé dans [−180 ; 180[, et un affichage « ajusté » le reste quand l'angle change.

🔧 **POUR LES CURIEUX**

- `scale` est le nombre de **pixels CSS par pixel d'image**. Il vaut au plus **32** et au moins **0,25 × l'ajustement**. L'ajustement laisse 2 % de marge de chaque côté (facteur 0,96).
- La molette zoome autour du curseur d'un facteur 1,1 par 100 unités ; un seul événement ne zoome jamais de plus de 3 crans, et Firefox, qui compte en « lignes », est ramené en pixels (1 ligne = 33 px).
- Un geste de moins de 4 pixels est un clic. Deux doigts font un zoom et un déplacement. Un double-clic ajuste. **Résolution native 1:1** règle `scale = 1 / devicePixelRatio` : un pixel d'image par pixel d'écran physique.
- La mesure garde le même contrat que celle du volume : `{ normalized, physicalUm, screen }`, avec `physicalUm = pixel × taille du pixel`.

Le chargement en deux temps

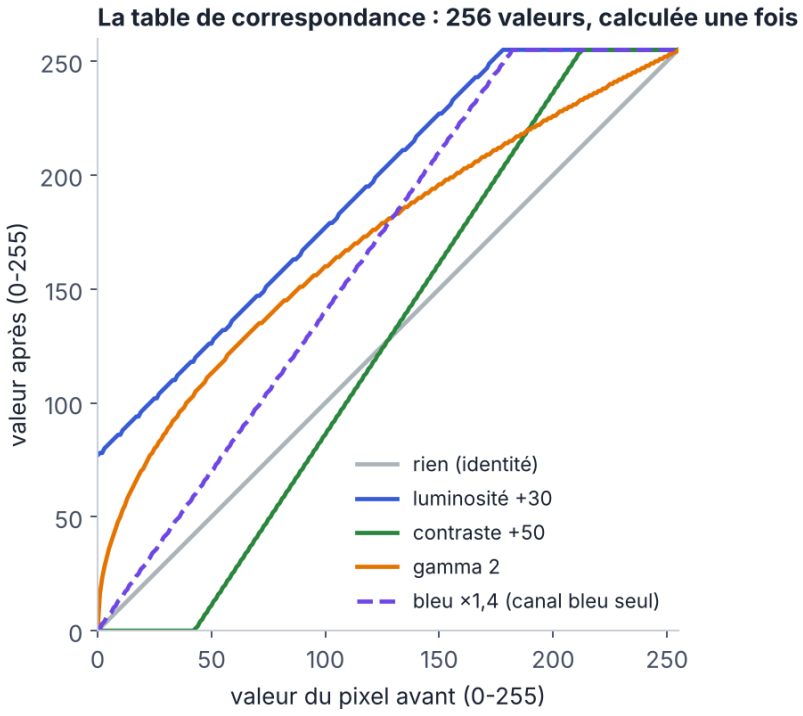


▲ L'aperçu d'abord, le natif ensuite.

Les deux fichiers sont demandés **ensemble** ; le premier décodé est dessiné. Une image plus petite ne remplace jamais une plus grande déjà à l'écran. Au parcours ←/→, la requête du natif attend 150 ms que la touche se calme : maintenir la flèche ne télécharge que la photographie sur laquelle on s'arrête. Une capture prise avant le natif porte l'aperçu agrandi : le Studio est prévenu (`quality: 'preview'`).

La pastille de chargement de la page distingue *aperçu*, *natif*, et *erreur du natif* (l'aperçu reste affiché).

Réglages d'affichage et isolement



▲ La table de correspondance d'un canal : 256 valeurs calculées une fois.

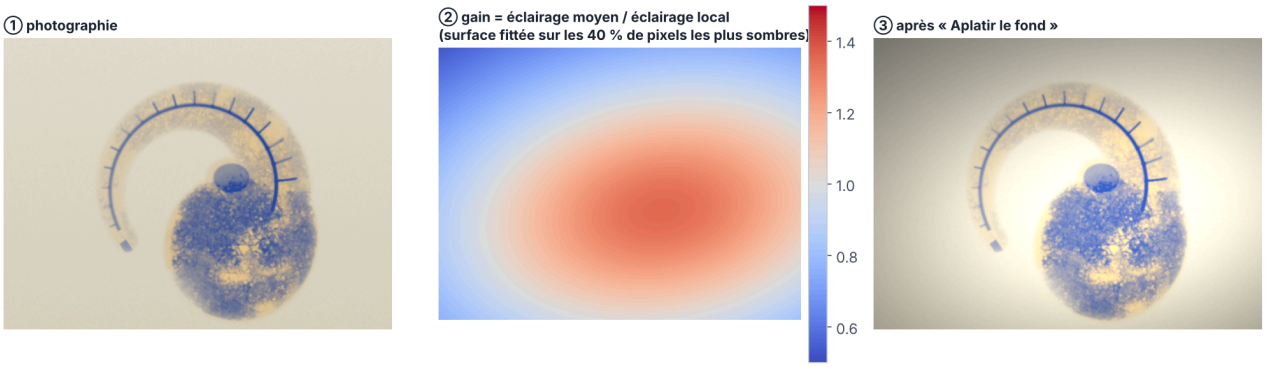
Pour chaque canal, le réglage est une **table de 256 valeurs** : `x = (v/255 · balance − ½)(1 + contraste/100) + ½ + luminosité/100`, bornée à [0 ; 1], puis `x^(1/gamma)`. La balance ne s'applique qu'au rouge et au bleu. Déplacer un curseur ne recalcule donc que la table, et la boucle sur les pixels est une simple lecture.

RÉGLAGE	PLAGE
Luminosité, Contraste	−100 à 100
Gamma	0,1 à 10
Balance rouge, bleu	0,1 à 4

Une valeur venue d'un curseur, d'un fichier d'espace de travail ou d'un lien est **assainie** : inconnue, ignorée ; hors plage, ramenée à la plage ; non numérique, refusée. Une valeur piégée ne peut donc pas rendre la photographie toute noire.

> ⚠️ **ATTENTION**
>
> **L'isolement du marquage a priorité sur les réglages** : tant qu'il est allumé, la vue montre le résultat de l'isolement seul. Les deux ne se cumulent pas.

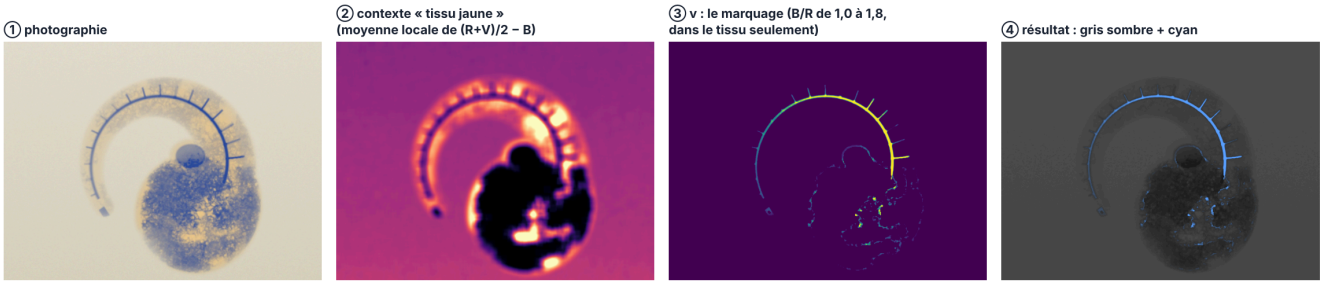
Aplatir le fond. Le gain d'un pixel est « éclairage moyen / éclairage local ». L'éclairage est modélisé par une **surface quadratique** $c_0 + c_1x + c_2y + c_3x^2 + c_4y^2 + c_5xy$ ajustée par moindres carrés sur la carte réduite au huitième, en ne gardant que les **40 % de cellules les plus sombres** (le fond mat) : une surface lisse ne peut pas suivre l'embryon, qui garde donc son propre contraste. Le gain est borné entre 0,5 et 3 ; un système singulier donne une surface plate.



Gain réel de cette photographie : de 0,53 à 1,35 (borné entre 0,5 et 3). Cette photo synthétique n'a pas de vrai vignettage : la figure montre le mécanisme, pas un bénéfice.

▲ La carte de gain d'une photographie et la correction qui en résulte.

Isoler le marquage. Le calcul tient en quatre cartes. Le contexte « tissu jaune » est une moyenne locale de `(R+V)/2 − B` sur un quart de résolution (boîte de 11 × 11 cellules, environ 40 pixels, calculée avec une table d'aires cumulées, donc en temps constant par pixel). Le marquage `v` vaut `(B/(R+1) − 1,0) / 0,8` borné à [0 ; 1], **seulement** si le contexte dépasse 5. Le pixel final est un gris à 35 % de sa luminosité, mêlé de cyan `(90 ; 160 ; 255)` à proportion de `v`.



Photographie de démonstration (synthétique). Le seuil du contexte est 5 : en dessous, v = 0 (le fond mat, même bleuté, ne s'allume pas).

▲ Les quatre cartes du calcul, sur la photographie de démonstration.

🔧 **POUR LES CURIEUX**

- **Pourquoi ce seuil de 5 ?** Un tronc marqué, large, tire la moyenne locale vers 6 à 9 ; le fond mat ne dépasse jamais 2 à 6. Le seuil est bas exprès : un marquage est du bleu **dans** un tissu jaune.
- **Le worker.** La page transfère un `ImageBitmap` (pas de copie), le worker dessine sur un `OffscreenCanvas`, applique les opérations de `pixel-ops-2d.js` et renvoie un bitmap. Les cartes d'éclairage et de contexte ne dépendent que de l'image : elles sont **gardées** tant que l'image ne change pas. Si des demandes s'empilent (un curseur qu'on tire), **seule la plus récente** est lancée.
- Sans Worker ni `OffscreenCanvas`, ou si le worker tombe, le même code tourne sur le fil principal, un travail à la fois. Une capture prend une version **exacte** pour les réglages du moment, jamais une version périmée.
- Ces deux opérations sont **de l'affichage** : les mesures et la grille lisent la photographie intacte. Le Studio, lui, reçoit ce qui est à l'écran (isolé ou ajusté, miroir et rotation compris), à la résolution native, calibré en µm/pixel.

L'orientation, la grille, la vue divisée

- **Orientation.** Le plugin pilote `setOrientation({ rotationDeg, flipH })` et dessine la boussole (A/P) ; la rotation étant rigide, l'échelle ne change pas. L'administrateur enregistre la pose avec le jeu de données (`orientation2d`), par les mêmes messages que l'orientation 3D (§ 13.9).

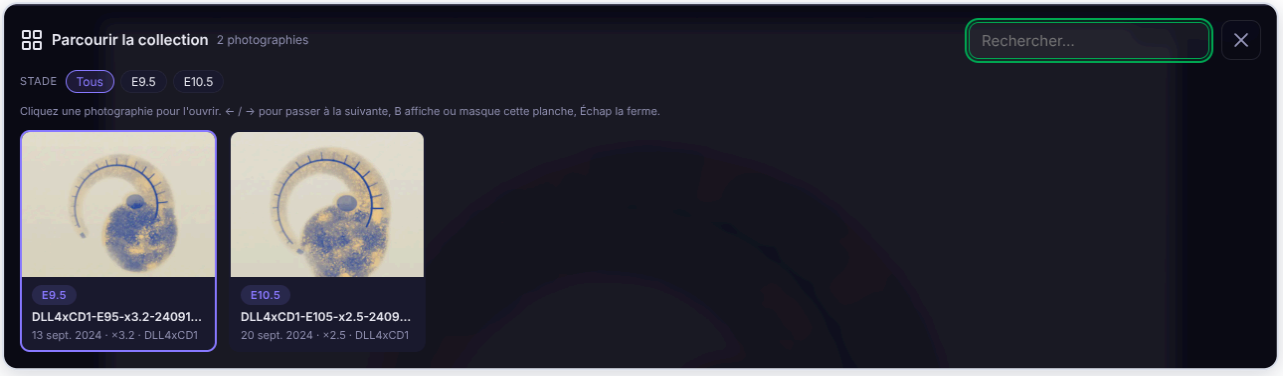
- **Grille calibrée.** Un pas **1-2-5** le plus proche de 120 px (grand) ou 60 px (fin), ancré sur le coin de l'image orientée ; les étiquettes (distance au coin) n'apparaissent que si le pas dépasse 44 px. Sans calibration, elle l'écrit au lieu de ne rien tracer.
- **Barre d'échelle.** `longueur = 1-2-5 le plus proche (120 px / px par µm)` ; elle n'existe que si la photographie est calibrée.

▲ La vue divisée échange une vue physique, pas des pixels.

La **vue physique** est le couple `umPerCss = taille de pixel / scale` et `centerUm` (le décalage du centre de l'écran par rapport au centre de la photo, en µm). Deux photos à 2,0 et 3,2 µm/px sont ainsi au même grossissement **réel**, et déplacer l'une déplace l'autre de la même distance réelle.

La collection

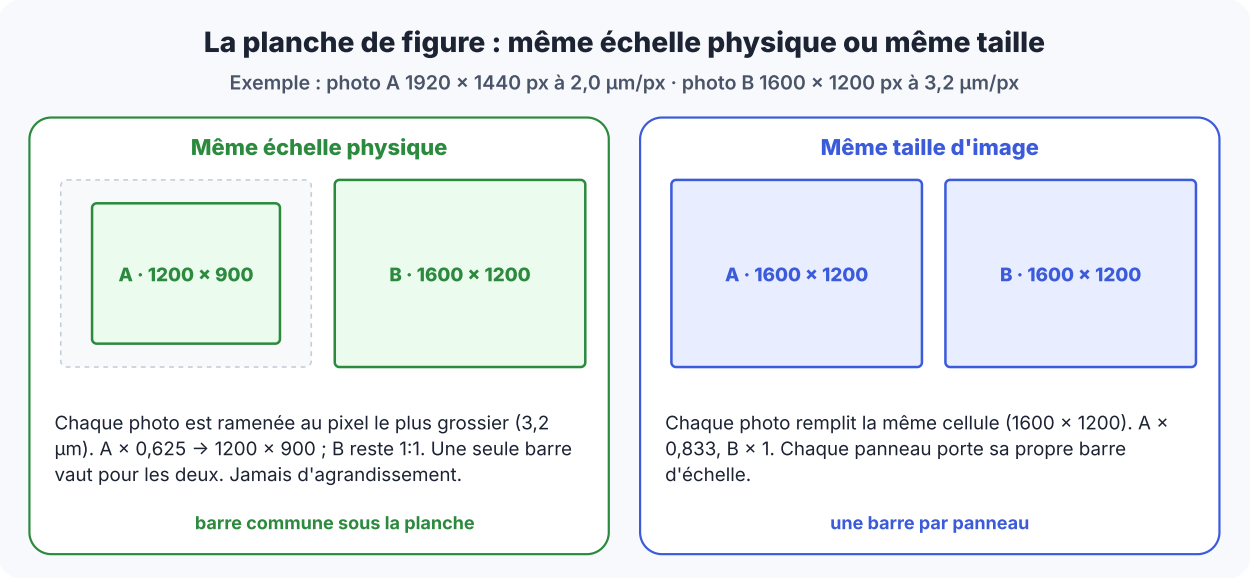
La planche-contact (touche B) est triée par stade, puis date de dissection, puis nom ; elle se filtre par stade, par lignée et par texte (nom, stade, lignée, coloration, description, date).



▲ La planche-contact de la collection (deux photographies de démonstration).

- Une fois la photographie à l'écran et la page au repos (400 ms), les **deux voisines** sont préchargées (aperçu et natif). Ni un panneau de Comparer, ni la vue divisée, ni l'aperçu de l'administration ne le font : cela multiplierait les téléchargements.
- Passer d'une photo à l'autre met à jour l'adresse avec l'historique du navigateur (`pushState`) ; le bouton Précédent ramène à la photographie d'avant sans recharger la page.
- Les **mesures** d'une photographie sont conservées en mémoire par identifiant : revenir à une photo les restaure.

La planche de figure

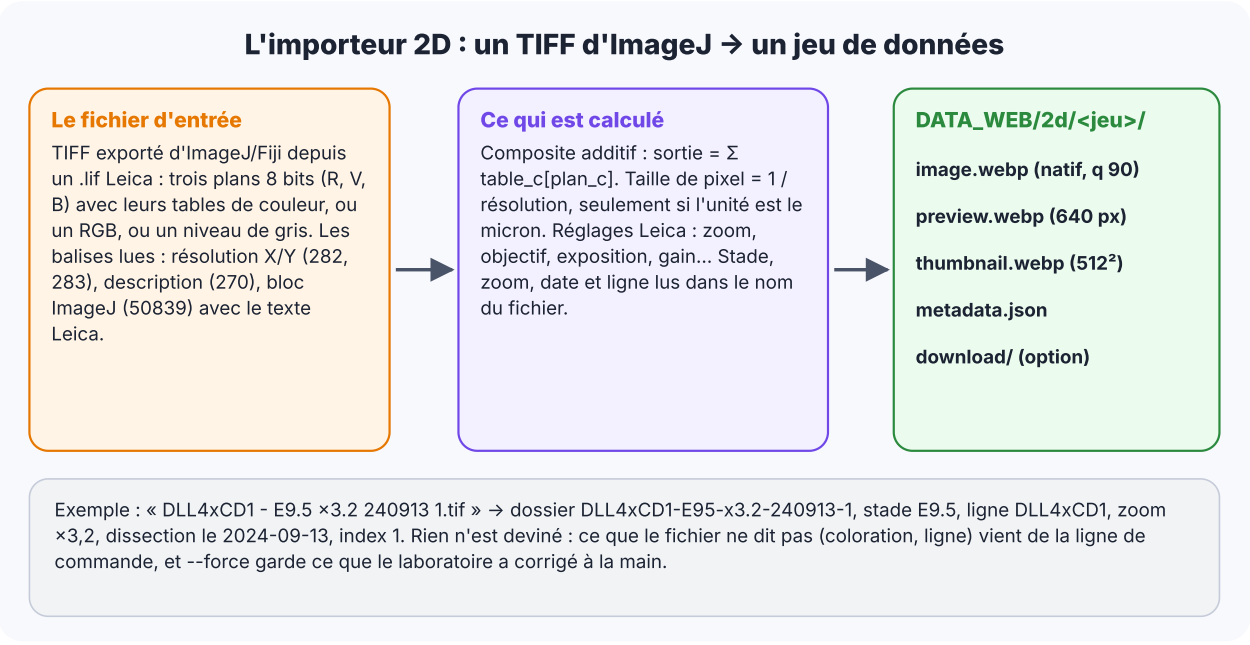


▲ Même échelle physique, ou même taille : deux façons de composer.

En mode « même échelle physique », chaque photographie est ramenée au **pixel le plus grossier** de la sélection (`facteur = px_i / px_max` , donc jamais d'agrandissement) et une barre commune vaut pour tout. En mode « même taille », chaque photographie remplit la même cellule, avec sa propre barre. Une photo **non calibrée** ne participe pas à l'échelle commune : elle est ajustée dans la cellule et marquée comme telle.

Le Studio reçoit **un rectangle de calibration par panneau** : une barre posée sur un panneau lit le µm/pixel de ce panneau, et le suit si on la déplace sur un autre. Limites : 6 000 px de large, 16 000 px de côté, 120 mégapixels (la figure est réduite pour y tenir), 48 mégapixels de photographies décodées gardées entre deux rendus, 3 décodages en parallèle.

L'importeur 2D



▲ De l'export ImageJ au dossier de données.

`preprocess/2d_importer.py` (version 0.18.0) transforme **un TIFF en un jeu de données**.

```
python 2d_importer.py --input <dossier|fichier.tif> --output DATA_WEB \
    [--line DLL4xCD1] [--staining X-gal] [--only "*E8.0*"] \
    [--with-downloads] [--force] [--lossless]
```

OPTION	EFFET
`--line`, `--staining`	ce que le fichier ne dit pas (ligne, coloration)
`--only`	un filtre de noms (« *E8.0* »)
`--with-downloads`	place le TIFF d'origine et un `README.txt` dans `download/`
`--force`	réimporte en **gardant** ce que le laboratoire a corrigé
`--lossless`	WebP sans perte (plus gros)

Lire les plans. Un hyperstack ImageJ range ses pages « canal en premier » : les `channels=` premières pages forment **un** composite ; les suivantes sont d'autres images, qu'on n'additionne pas. Le composite est additif, exactement ce qu'ImageJ affiche : `sortie = Σ table_c[plan_c]`, chaque plan 8 bits passant par sa **table de couleur** de 768 octets (3 rampes de 256). Sans table, trois plans sont lus comme R, V, B et un plan comme un gris. Un plan de 16 bits est ramené sur 8 par son maximum.

La taille de pixel. ImageJ écrit la résolution (balises 282 et 283) en **pixels par unité**. L'importeur ne la croit que si l'unité de la description est le **micron** ; la taille d'un pixel vaut alors `1 / résolution`, en x et en y (carrés si seule la résolution x existe). Sinon : `calibrationStatus: unknown`, pas de barre d'échelle, pas de mesure, et le texte le dit.

Le bloc Leica. ImageJ embarque le texte de LAS X. Pour la série lue, l'importeur retient le zoom, le grossissement, l'objectif, l'ouverture numérique, l'exposition (convertie en ms), le gain, le microscope et la caméra. Le texte répète chaque clé par bloc de travail ; la ligne du niveau supérieur (celle de l'exposition réellement prise) l'emporte, et les « 0 » de remplissage ne passent jamais devant une vraie valeur.

Le nom du fichier. `<lif> - <stade> x<zoom> <date aammjj> [<n> [<m>]]`. Le stade passe par **le même analyseur que les volumes** (`E8-5`, `E8.5`, `E85` valent E8.5), la date doit être un vrai jour, la ligne se lit dans le nom du `.lif` (`DLL4xCD1`). Rien n'est inventé : une partie absente reste vide.

> 🔢 **MICRO-EXEMPLE**
>
> « DLL4xCD1 - E9.5 ×3.2 240913 1.tif » donne le dossier `DLL4xCD1-E95-x3.2-240913-1`, stade E9.5, zoom ×3,2, dissection le 2024-09-13, index 1. Dossier : `<ligne>-<stade sans point>-x<zoom>-<aammjj>-<index>`.

Les fichiers écrits. `image.webp` (natif, qualité 90, méthode 6, ou sans perte), `preview.webp` (grand côté 640 px, LANCZOS, qualité 80), `thumbnail.webp` (carré de 512 px sur fond `#080a12`), `metadata.json` (écrit **en dernier**, atomiquement : un import à moitié fait n'est jamais monté). Côté maximal : 16 383 px (limite de WebP).

❓ POURQUOI CE CHOIX ?

Pourquoi `--lossless` existe. Le WebP avec perte sous-échantillonne la couleur (4:2:0), ce qui perturbe le rapport B/R par pixel que lit l'isolement du marquage, surtout aux contours nets. Pour une analyse du marquage, gardez la photo exacte.

Deux TIFF qui se décriraient de la même façon (même ligne, stade, zoom, date, index) produisent le même nom de dossier : le second est **refusé** avec un message, plutôt que d'écraser le premier ou d'être pris pour déjà importé. Un dossier existant n'est pas touché sans `--force`.

📌 À RETENIR

- Trois repères (image, orientée, canevas) et **une** relation : `L / pixelSizeUm × scale`.
- Réglages et isolement : **table de 256 valeurs** ou calcul en **worker**, sur une copie.
- L'isolement a priorité sur les réglages ; rien de mesuré ne les lit.
- L'importeur ne devine rien : taille de pixel des balises TIFF, stade du nom, le reste de la ligne de commande.

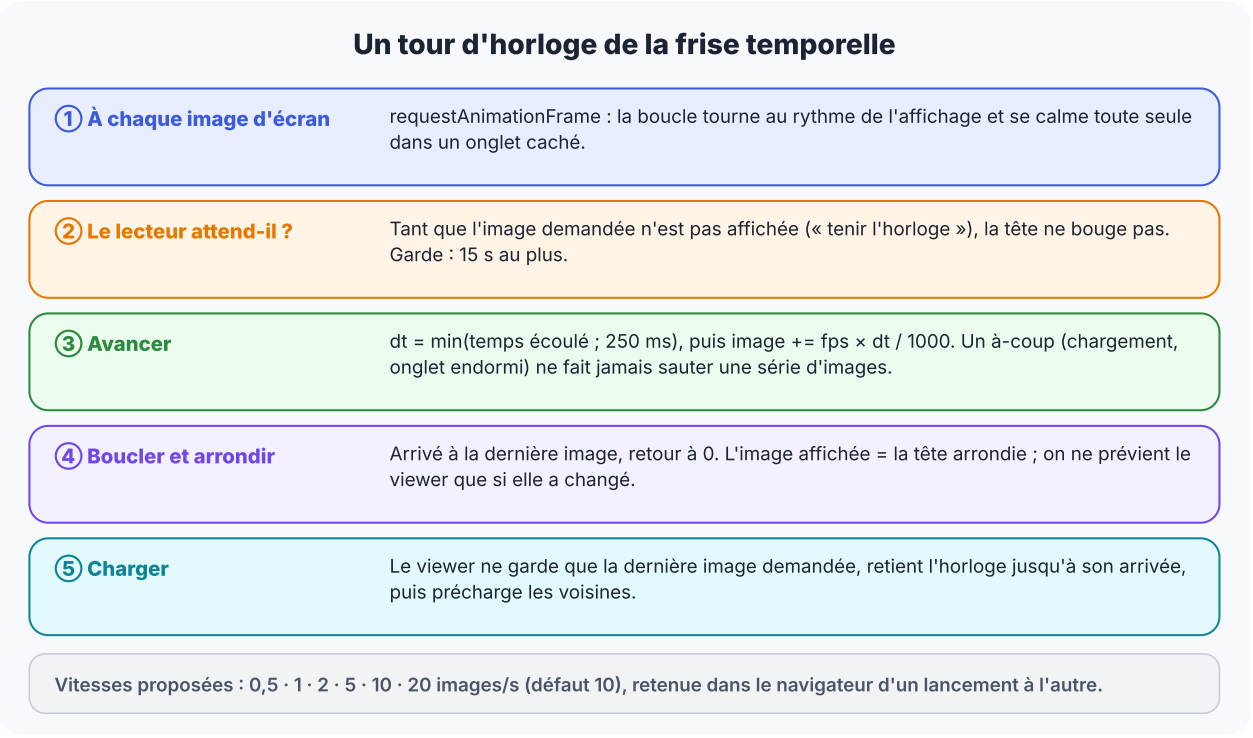
13.6 La frise temporelle et les séries live

⏱ EN 30 SECONDES

- La frise est un **composant autonome** (`Timeline`) : une horloge, une piste, un tampon. Le viewer lui dit quand attendre.
- L'horloge ne **saute** jamais une série d'images, et ne **court pas devant** l'image qui n'est pas arrivée.
- L'heure réelle n'est pas affichée, mais l'intervalle entre images sert aux vitesses en µm/min.

Le chapitre 10 (§ 10.7) a expliqué le chargement d'une série (tampon, préchargement, dernière image seulement). Voici l'horloge elle-même.

Un tour d’horloge



▲ Les cinq étapes d’un pas de lecture.

🔢 **MICRO-EXEMPLE**

À 10 images par seconde, sur un écran à 60 Hz, chaque pas dure environ 16,7 ms : la tête avance de `10 × 0,0167 ≈ 0,17` image. Il faut **six pas** pour changer d’image affichée. Un onglet endormi une seconde ne ferait pas avancer la tête de 10 images : le temps écoulé est plafonné à **250 ms**, soit 2,5 images au plus.

CONSTANTE	VALEUR
Vitesses proposées	0,5 · 1 · 2 · 5 · 10 · 20 images/s
Vitesse par défaut	10 images/s
Pas maximal	250 ms
Garde de l’attente d’une image	15 s
Clé du navigateur	`iribhm.viewer.playbackFps`

Un clic sur la vitesse passe à la **première vitesse strictement supérieure** à la vitesse actuelle, et boucle au sommet. Une vitesse mémorisée qui n’est pas dans la liste reste utilisable : elle passe simplement à la suivante.

Attendre sans bloquer

Une image native peut coûter 600 ms à streamer. Sans précaution, l’horloge avancerait de six images pendant ce temps, et le chargeur serait envoyé chasser une image que personne ne verra. Le viewer **retient donc l’horloge** (`setStalled`) pendant qu’il charge l’image qu’on lui a demandée : la tête ne bouge plus, mais la boucle continue de tourner.

❓ POURQUOI CE CHOIX ?

Pourquoi la garde de 15 s ? Un chargement peut ne jamais finir (un onglet en arrière-plan fige l'affichage, et le chargeur attend une image à dessiner). Sans plafond, la frise resterait gelée pour toujours. Le viewer ne compte pas non plus les chargements en cours : il relâche l'horloge à **chaque** fin de chargement, ce qui peut au pire la libérer un peu tôt, jamais la bloquer.

La piste est dessinée à `image / (N − 1)`. Quand vous la saisissez, la lecture **se met en pause** et l'image est celle sous le curseur ; relâcher arrondit à l'image entière. Le bouton Lecture réagit à l'appui (pas au relâchement), et reste utilisable au clavier.

Le texte du compteur est `courante / dernière`, sur trois chiffres : « 000 / 003 » veut dire image 0 sur une série de 4. Les images **déjà en mémoire** sont tracées en bandes sous la piste : une série se remplit autour de la tête et en perd en route, une simple largeur ne suffirait pas. L'image `i` occupe l'intervalle `[i − ½ ; i + ½]` pour que la tête reste dans la bande qui la représente.

L'heure réelle et la normalisation

- `timeline.intervalMinutes` et `timeline.timestamps` sont dans `metadata.json`. **Les horodatages ne sont pas affichés**, mais l'intervalle permet à l'inspecteur de donner la vitesse d'une cellule en **µm/min** en plus de µm/image.
- Le pipeline enregistre aussi `intensityNormalization` (les niveaux de signal de chaque canal et de chaque image) pour pouvoir **mesurer** le photoblanchiment. **Aucun réglage du viewer ne l'utilise aujourd'hui** : une série qui s'éteint s'affiche réellement plus sombre.
- Le composant sait aussi lisser (des positions fractionnaires) et afficher des curseurs de vitesse ; le viewer ne s'en sert pas.

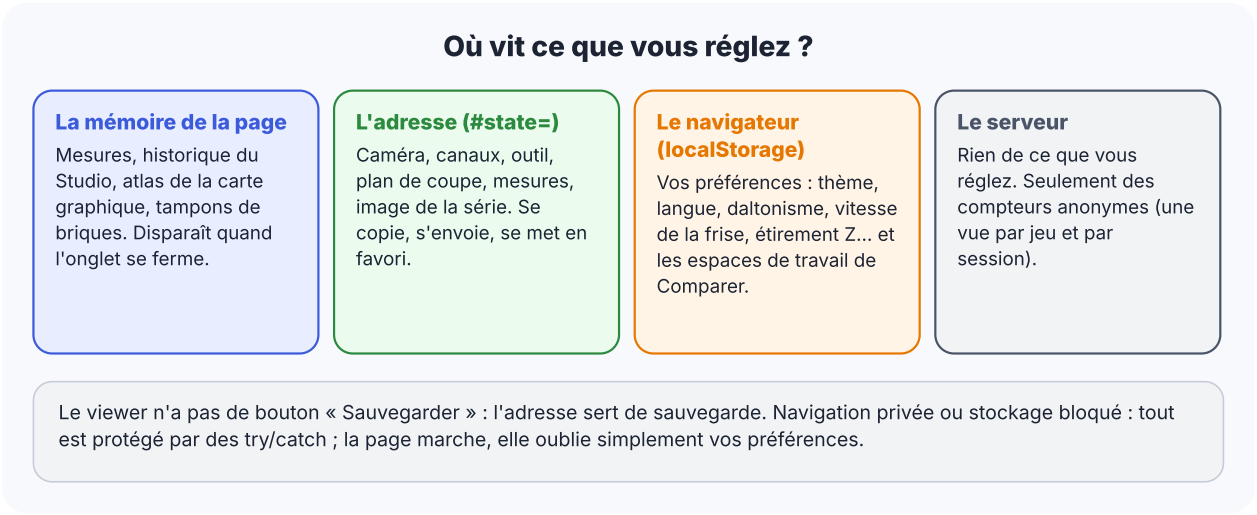
ℹ️ À SAVOIR

Deux séries de longueurs différentes dans Comparer s'alignent par **fraction** de durée (§ 13.4) : « la moitié de la série » est la moitié de chacune.

13.7 Ce que la plateforme retient : lien, espace de travail, navigateur

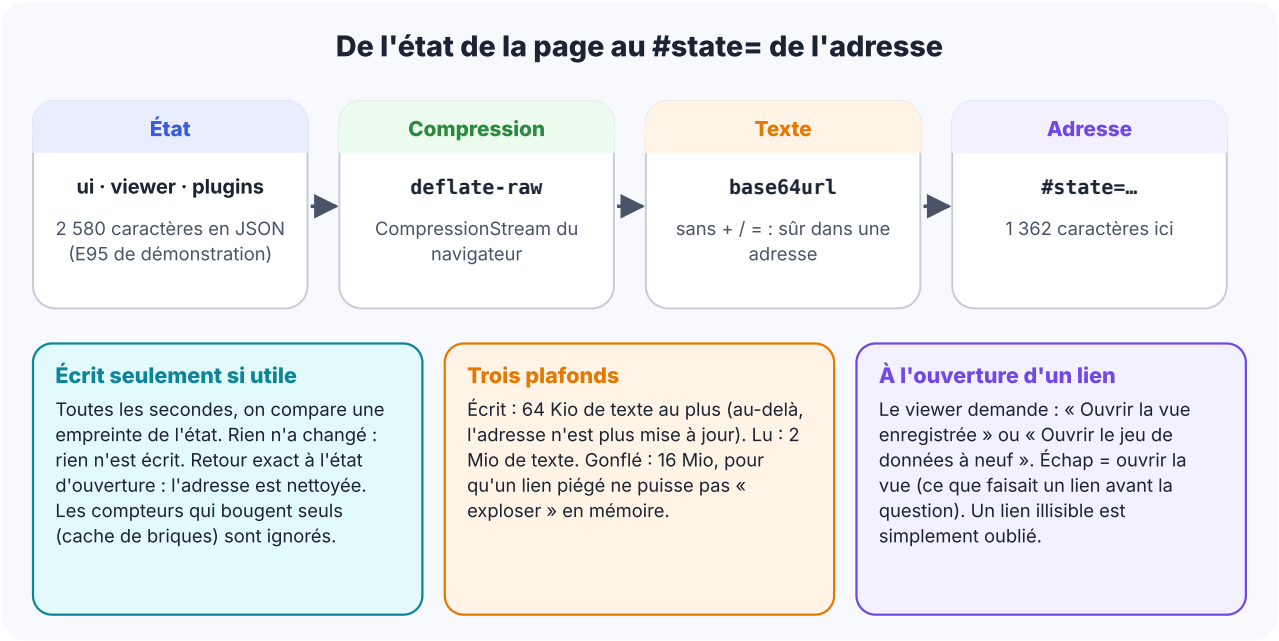
⏱ EN 30 SECONDES

- Quatre endroits : la **mémoire de la page**, **l'adresse**, le **navigateur**, le serveur (qui ne retient rien de vos réglages).
- L'adresse porte l'état entier d'une vue, compressé : elle **sert de sauvegarde**.
- Tout accès au navigateur est protégé : stockage bloqué ou navigation privée, la page marche et oublie.



▲ Où vit ce que vous réglez.

Le lien `#state=`



▲ De l'état de la page à l'adresse.

Toutes les secondes, la page demande son état à `_getWorkspaceState()`. Celui-ci rassemble trois blocs : `ui` (outil actif, barre latérale, fond), `viewer` (caméra, plan de coupe, mesures, canaux, exposition, grille, étirement Z, Z-stack, image de la série, qualité, calibration…) et `plugins` (l'état que chaque outil expose par `getState()`).

🔢 **MICRO-EXEMPLE**

L'état du viewer du jeu E95 de démonstration, au repos : **2 580 caractères** de JSON, qui deviennent **1 362 caractères** une fois compressés (deflate) et écrits en base64url. La caméra y tient en une ligne : `{"kind":"volume","cameraZ":1.13,"quaternion":[0,0,0,1],"position":[0,0,0],"zDisplayScale":1}`.

L'état n'est **encodé** que s'il a changé, d'après une **empreinte** (le JSON sans les compteurs qui bougent seuls, comme le cache de briques). Le jeu ouvert tel quel est la référence : tant que rien n'a changé,

l'adresse reste propre ; revenir exactement à l'état d'ouverture la nettoie. Le bouton **Réinitialiser l'espace de travail** ré-adopte l'état d'ouverture comme nouvelle référence, après avoir laissé la fin d'un rechargement se stabiliser (deux lectures identiques de suite, ou un délai).

🔧 POUR LES CURIEUX

- **Écriture** : au plus 64 Kio de texte, sinon l'adresse n'est plus mise à jour (un ancien `#state=` serait plus faux qu'aucun) et la console invite à utiliser « Save state ».
- **Lecture** : 2 Mio de texte au plus ; la décompression est lue par morceaux et **arrêtée à 16 Mio** (un lien piégé peut gonfler d'un facteur ~1 000).
- **Génération** : toute action qui invalide l'adresse (effacement, nouvelle référence) incrémente un compteur ; un calcul de compression qui finit après coup **jette** son résultat au lieu de ressusciter l'état effacé.
- Aucun appel réseau : le lien contient l'état, le serveur ne le voit jamais.

À l'ouverture d'un lien, une boîte demande : **Ouvrir la vue enregistrée** ou **Ouvrir le jeu de données à neuf**. Échap et le fond valent « ouvrir la vue » (ce que faisait un lien avant la question). Un contenu illisible est effacé sans question. Dans un panneau de Comparer, il n'y a pas de question : c'est l'hôte qui décide.

Les espaces de travail du navigateur

`WorkspaceState` range un espace de travail sous la clé `iribhm.workspace.<portée>.<identifiant du jeu>` (portée : `viewer`, `compare`…), version 2, avec la date. Il est normalisé en quatre blocs (`ui`, `viewer`, `tracking`, `compare`). Écrire peut **échouer** (quota, stockage bloqué) : l'erreur est rendue à l'appelant exprès, pour qu'un espace de travail qui n'a pas été enregistré ne soit jamais annoncé comme enregistré. Les clés d'avant le renommage des types (`fixed`, `wholemount`) sont migrées une fois, au passage.

Le viewer n'a **pas** de boutons Sauvegarder / Restaurer : c'est l'adresse. Ils existent dans la page **Comparer** (Centre de téléchargement) et dans le Studio (JSON).

Ce que garde votre navigateur

CLÉ	CONTENU
`iribhm-theme`, `iribhm-lang`, `iribhm-colorblind`	thème, langue, filtre daltonien (partagés entre pages et cadres par l'événement `storage`)
`iribhm.viewer.playbackFps`	vitesse de la frise
`iribhm.viewer.zScale.<jeu>`	étirement Z de ce jeu
`iribhm.workspace.<portée>.<jeu>`	un espace de travail enregistré
`iribhm-gallery-dock`	état du dock de la galerie (ouvert, large, puce)
`lumen3d.vramBudgetMB`	budget de mémoire vidéo imposé (opérateur)
`lumen3d.gpuContextLosses`	les 8 dernières pertes de contexte graphique (une semaine)
`sessionStorage` `lumen_view_<jeu>`	une seule vue comptée par session

ℹ️ **À SAVOIR**

Les **mesures** ne sont pas dans cette liste : elles vivent dans la mémoire de la page (`MeasurementStore` , une `Map` par portée et par jeu) et voyagent **dans le lien**. L'administration a ses propres clés (`adm-theme` , `adm-upload-dock-size` , `lumen-dupd-*`), décrites au chapitre 14.

Le budget graphique mérite d'être connu (§ 10.4) : il est **divisé par deux pour chaque perte de contexte graphique** de la semaine écoulée (plancher 256 Mio), sur ce profil de navigateur. Vider les données du site le remet à zéro.

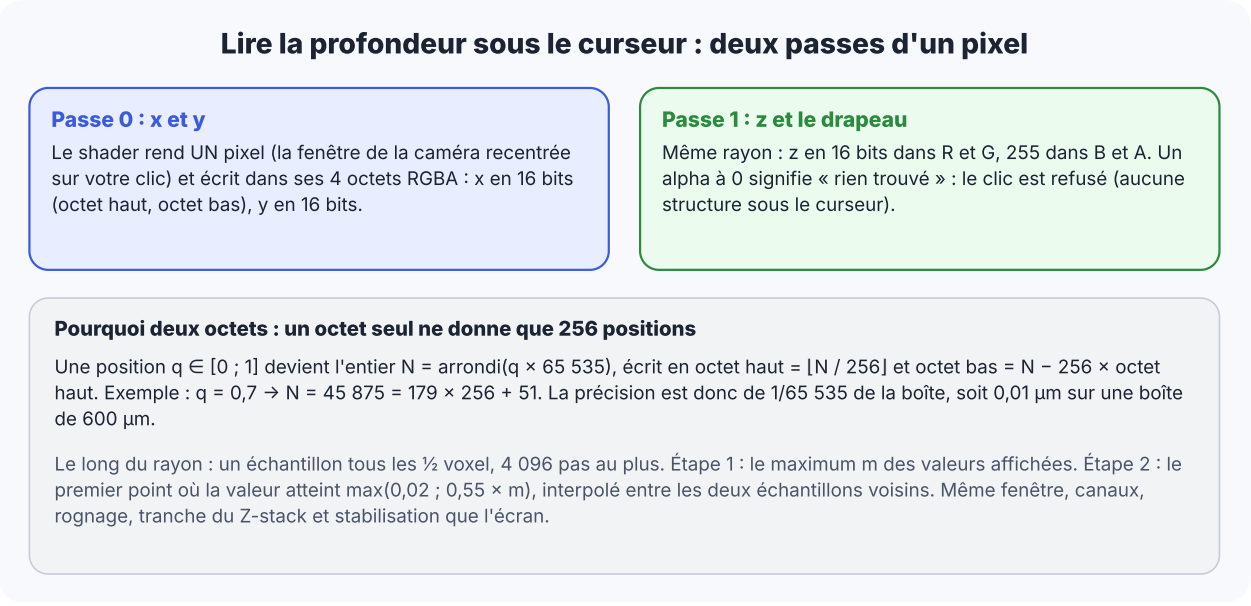
13.8 Algorithmes de mesure et de pose

⏱ **EN 30 SECONDES**

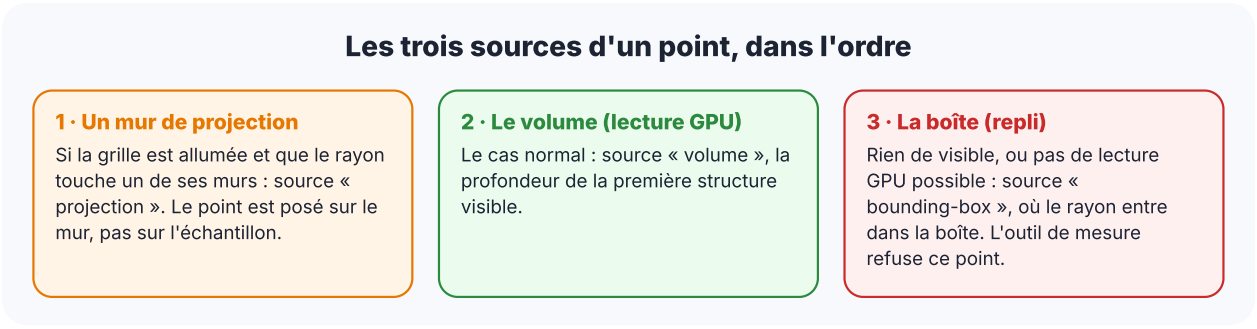
- Un clic de mesure lit la **profondeur sur le GPU** en deux passes d'un pixel, avec une précision de 16 bits.
- Mettre une pile à plat, c'est **deux rotations autour des axes de l'écran** : la pile ne culbute pas.
- L'export d'une grande image **recommence** si le volume ou les réglages changent pendant son rendu.

La profondeur sous le curseur

Le chapitre 12 (§ 12.2) a décrit la logique : le maximum du rayon, puis le premier franchissement de 55 %. Voici le mécanisme.



▲ Les deux passes d'un pixel et le codage sur 16 bits.



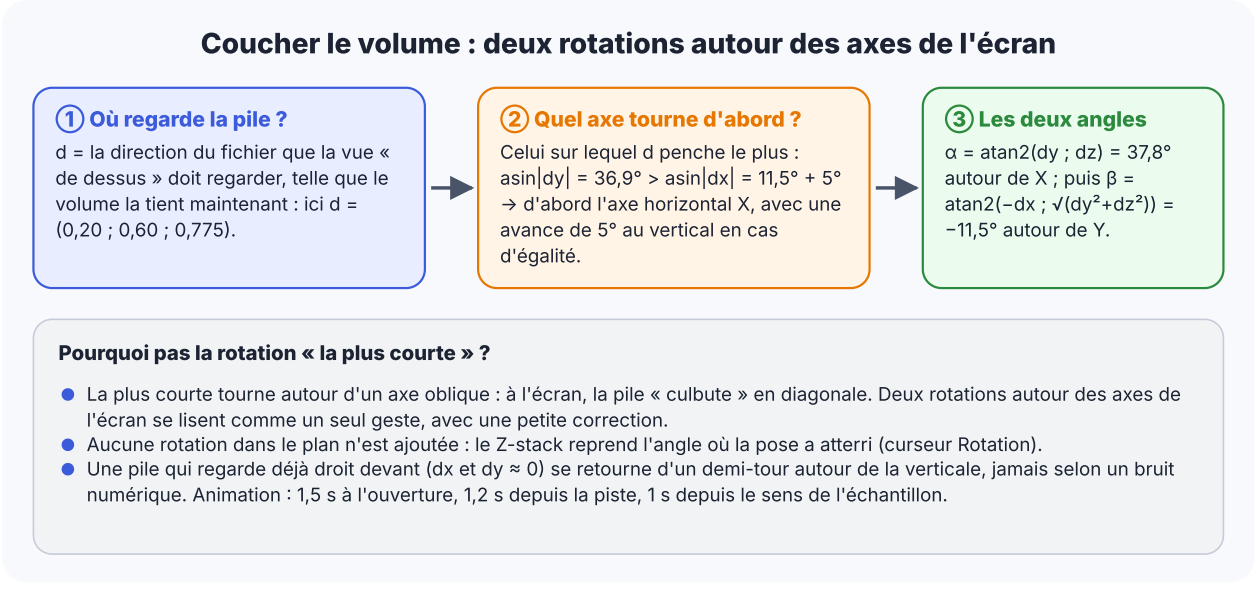
▲ Les trois sources d'un point, dans l'ordre où elles sont essayées.

🔧 **POUR LES CURIEUX**

- Le rendu d'un pixel utilise **les mêmes uniformes** que l'écran : mêmes fenêtre, canaux allumés, rognage, tranche du Z-stack, stabilisation, briques. Il est cadré par `camera.setViewOffset` sur le pixel cliqué.
- Les constantes sont `PICK_SURFACE = 0,55` , `PICK_MIN_VALUE = 0,02` , `PICK_MAX_STEPS = 4096` ; le pas est d'un demi-voxel (`1 / (2·nu)`), allongé si le rayon dépasserait 4 096 pas.
- Sous stabilisation, la position codée est celle de la **boîte d'affichage** (`clipBoxMin + c · clipBoxSize`) ; sans stabilisation, celle de la boîte unité. Le décodeur gère les deux.
- Une perte de contexte graphique, ou l'absence de volume actif, retire la voie GPU : le point retombe sur la boîte, et l'outil de mesure **refuse** ce point.

Coucher la pile : l'ouverture du Z-stack

Le chapitre 12 (§ 12.4) dit que la pile « se couche à plat en 1,5 s ». Voici la géométrie, qui sert aussi au sens de l'échantillon (§ 13.9).



▲ Deux rotations autour des axes de l'écran, calculées pour un exemple.

`setView(vue, options)` accepte plusieurs façons de choisir **l'angle dans le plan** de la vue :

SPIN	CHOIX
`'frame'` **(défaut)**	la rotation dans le plan la plus proche du repère calibré

SPIN	CHOIX
`'nearest'`	la plus proche de la pose **actuelle** (le plus petit mouvement, par un axe en général oblique)
`'tilt'`	**celle qu'obtiennent les deux rotations** ci-dessus ; utilisée par le Z-stack
un nombre	des degrés comptés depuis le repère

La **face** (`side`) est `front` , `back` (la pose de face retournée d'un demi-tour autour de la verticale), ou `top` / `bottom` : les faces haute et basse de l'échantillon, résolues selon `upsideDown` . L'animation est une interpolation d'un bout à l'autre, avec un adoucissement $3t^2 - 2t^3$; les deux rotations de `'tilt'` croissent ensemble pour se lire comme un seul geste. Une caméra n'est notifiée **qu'une fois**, à l'arrivée, pour que les panneaux de Comparer voient la pose finale ; saisir la souris annule le vol.

Exporter une très grande vue

L'export (§ 12.6) rend la vue **par tuiles** d'au plus 2048 px (moins si la carte graphique ne le permet pas, jusqu'à 256 px en dernier recours). Pour que la grande image soit cohérente, `renderViewImage` :

- prend la **pose de départ** une fois (celle où atterrit un vol en cours), avec le rapport d'aspect de l'image demandée ;
- attend que le volume ait **fini de streamer**, puis rend une tuile par tâche ;
- vérifie avant **chaque tuile** que le volume actif est le même et que l'empreinte d'affichage (canaux, exposition, mode de rendu, rognage, grille et axes, étirement Z) n'a pas changé ;
- sinon **efface et recommence**, trois fois au plus (erreurs `unstable` ou `display-changed` ensuite) ;
- continue le grain aléatoire d'une tuile à l'autre (`fragCoordOffset`) pour qu'aucune couture ne se voie, et écrit un fond transparent par une opacité par pixel (`exportAlpha`).

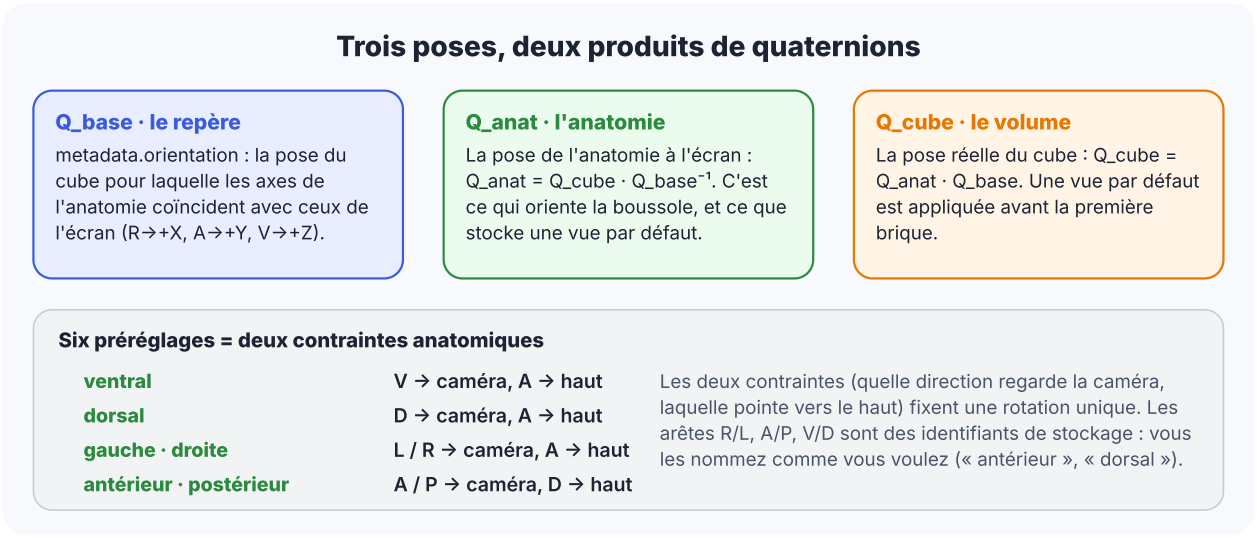
Les autres erreurs ont un code lisible : `busy` (un export tourne déjà), `size` , `canvas` (le navigateur refuse une image si grande), `gpu-memory` , `context-lost` .

13.9 L'orientation en pratique, et le sens de l'échantillon

⏱ EN 30 SECONDES

- Trois poses : **Q_base** (le repère), **Q_anat** (l'anatomie à l'écran), **Q_cube** (le volume). Deux produits les relient.
- L'administrateur ne calcule jamais un quaternion : il **demande** au plugin, qui répond par message.
- Le **sens de l'échantillon** corrige un fichier Imaris qui montre l'échantillon vu de dessous.

Le chapitre 12 (§ 12.5) a donné la théorie en une phrase. Voici le mécanisme complet, tel que l'administrateur le manipule (chapitre 14, et le guide de l'administrateur § 3.8).



▲ Les trois poses et les deux produits de quaternions.

Ce que stocke `metadata.json`

CHAMP	RÔLE
`orientation`	**Q_base** : la pose du cube pour laquelle l'anatomie coïncide avec les axes de l'écran
`orientationAxes.labels`	les noms que l'opérateur a donnés aux six bras (vide : R1/R2, G1/G2, B1/B2)
`orientationAxes.hidden`	les bras non dessinés (ils ne sont même **pas construits**)
`orientationAxes.defaultView`	la vue d'ouverture : `Q_anat` (un préréglage et/ou un quaternion)
`upsideDown`	le fichier montre l'échantillon vu de dessous

Les bras ont des **identifiants de stockage** (R/L pour ±X en rouge, A/P pour ±Y en vert, V/D pour ±Z en bleu) qui n'imposent aucune nomenclature : vous les nommez comme vous voulez.

La conversation entre l'administration et le viewer

L'aperçu de l'administration est le vrai viewer, dans un cadre. Le panneau ne calcule **aucune géométrie** : il envoie des messages au plugin *Orientation Axes* et enregistre ce qui revient. Une seule implémentation de la géométrie existe.

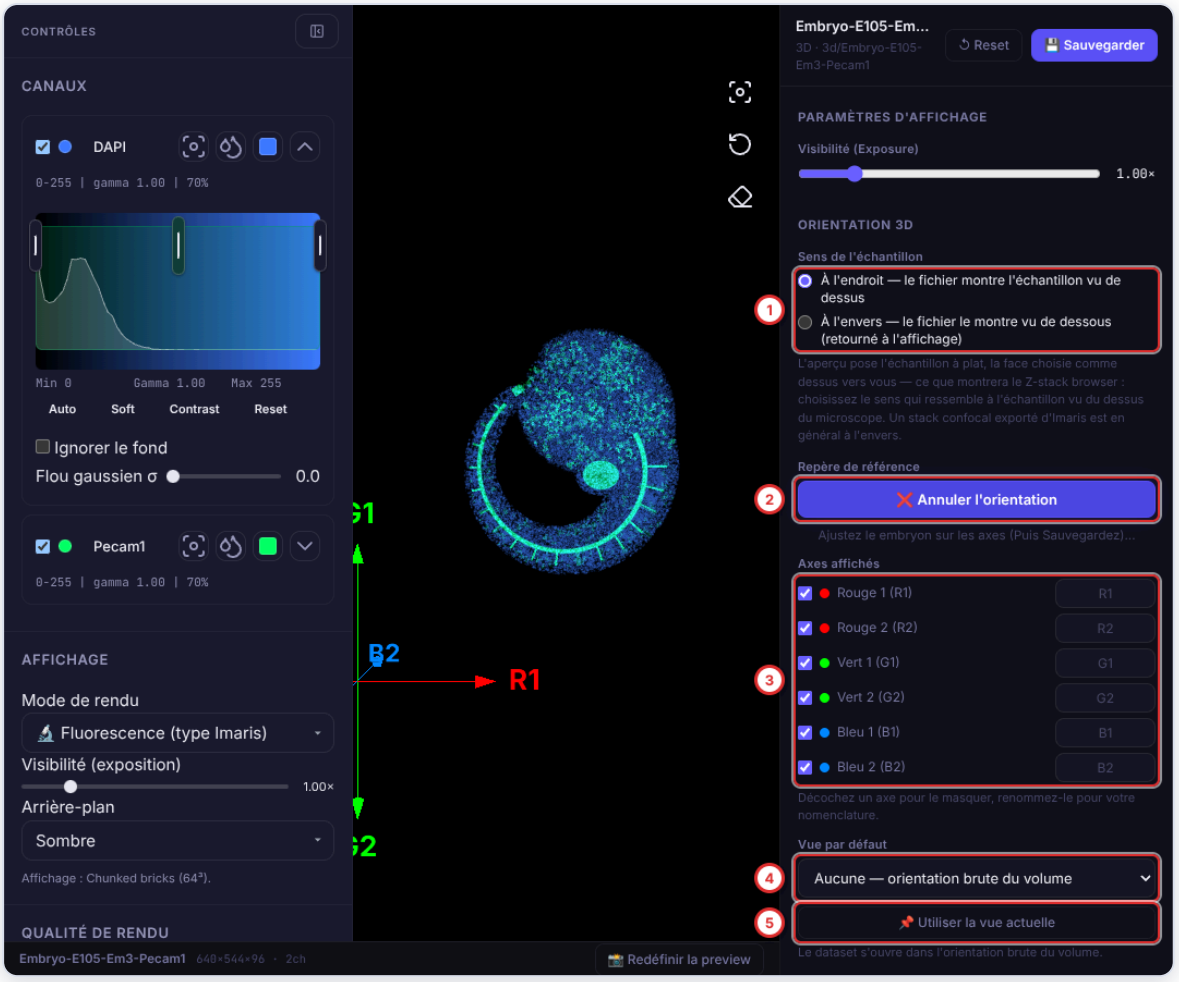
LE PANNEAU ENVOIE…	LE PLUGIN RÉPOND / FAIT
`CALIBRATE_ORIENTATION_START`	repart de l'alignement **enregistré** (on le raffine, on ne recommence pas), fige la boussole sur les axes de l'écran, l'affiche
`GET_ORIENTATION`	`ORIENTATION_RESULT` : la pose actuelle du cube, à enregistrer comme `Q_base`
`SET_ORIENTATION_AXES`	aperçu en direct des noms et de la visibilité des bras

LE PANNEAU ENVOIE…	LE PLUGIN RÉPOND / FAIT
`APPLY_ORIENTATION_VIEW`	pose le volume et renvoie `ORIENTATION_VIEW_RESULT` avec le `Q_anat` à stocker
`GET_ORIENTATION_VIEW`	capture la pose **actuelle** comme vue par défaut (en `Q_anat`)
`SET_SAMPLE_UPSIDE_DOWN`	prévisualise la face supérieure ; répond `SAMPLE_SIDE_RESULT`

🔧 **POUR LES CURIEUX**

- **Les six préréglages** sont chacun deux contraintes : quelle direction anatomique regarde la caméra (+Z de l'écran), laquelle pointe vers le haut (+Y). La rotation unique qui les satisfait s'obtient en construisant la base `(droite, haut, face)` et en prenant l'inverse. Ventral : face V, haut A ; dorsal : face D, haut A ; gauche / droite : face L / R, haut A ; antérieur / postérieur : face A / P, haut D.
- **La vue par défaut s'applique avant la première brique** (`prepare()` , appelé entre l'initialisation du viewer et le premier chargement) : tourner après coup montrerait le volume dans une pose puis le ferait claquer dans une autre pendant que les briques arrivent. Elle est enregistrée comme la **pose d'origine** du viewer : le bouton **Réinitialiser la vue** y revient, et non aux axes bruts du fichier.
- **Une vue par défaut est stockée en Q_anat**, une affirmation sur l'anatomie (« ventral vers moi »), donc affiner la calibration plus tard ne la périme pas.
- Quand un jeu n'a **pas** de calibration, les bras partent de la pose brute du fichier (retournée si `upsideDown`).
- La boussole est entraînée par $Q_{cube} \cdot Q_{base}^{-1}$; sa sphère centrale se déplace en glissant, et sa position est dans l'espace de travail.

Le sens de l'échantillon



▲ L'aperçu de l'administration : orientation 3D et gizmo d'axes.

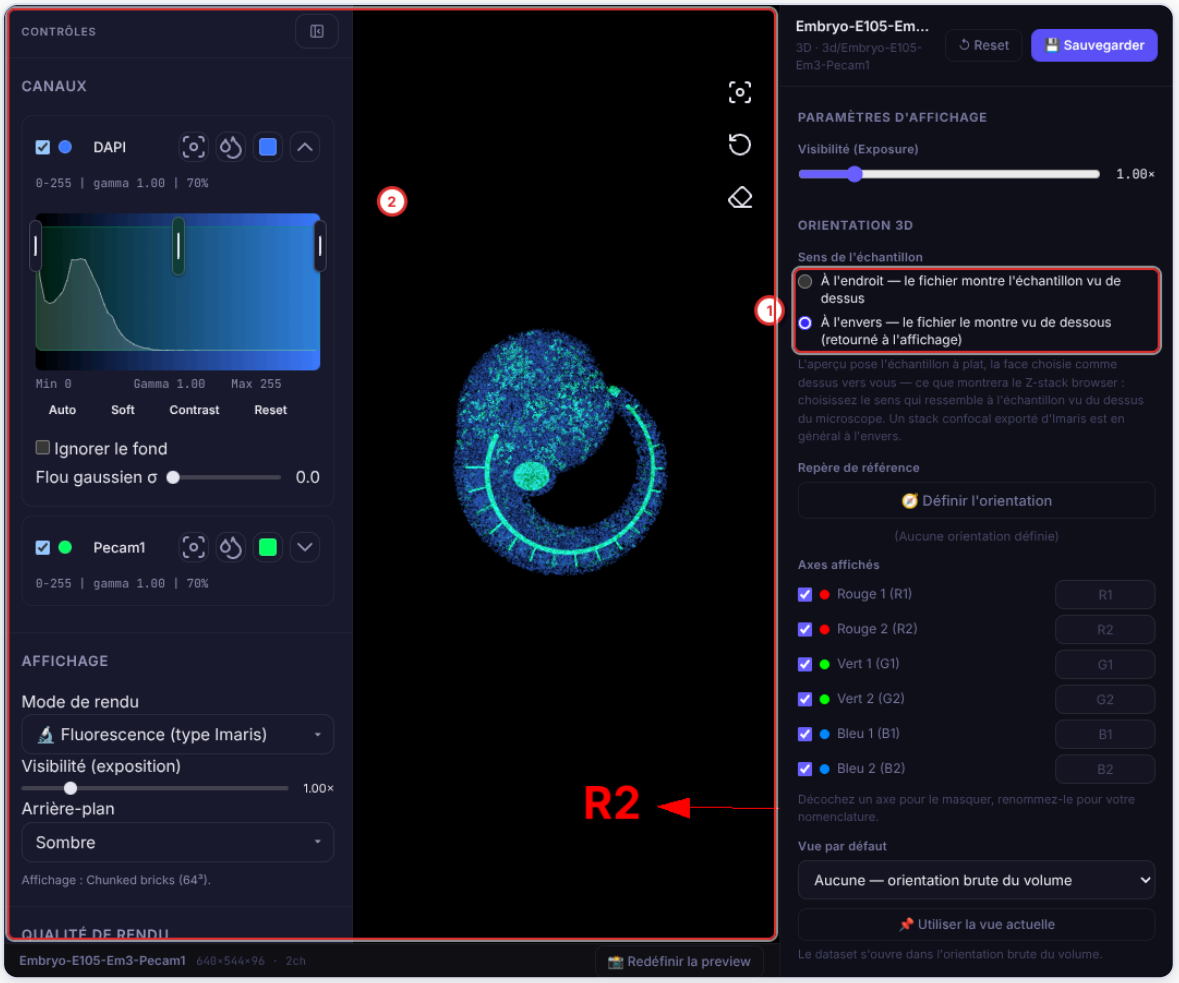
N	CE QUE C'EST
1	**Sens de l'échantillon** : deux boutons radio
2	🧭 **Définir l'orientation** : place le gizmo d'axes dans l'aperçu
3	**Axes affichés** : cocher, masquer, renommer
4	**Vue par défaut** : la pose d'ouverture du jeu
5	📌 **Utiliser la vue actuelle** : capture la pose de l'aperçu

Un stack confocal exporté d'Imaris est **en général vu de dessous** : l'objectif d'un microscope inversé est sous l'échantillon. Le viewer appelle « dessus » la face −Z dans ce cas (`upsideDown: true`), et la pose brute d'un jeu non calibré est retournée d'un **demi-tour autour de la verticale de l'écran** : la gauche et la droite s'échangent, le haut reste en haut.

❓ POURQUOI CE CHOIX ?

Pourquoi un demi-tour autour de la verticale et pas autour de l'axe X du fichier ? Dès qu'une calibration a incliné les axes du fichier à l'écran, un demi-tour autour de l'axe X du fichier ressemble à une culbute en diagonale. Autour de la verticale **de l'écran**, c'est toujours le même geste, quelle que soit la pose.

L'interrupteur de l'administration **prévisualise** la face haute : il couche le volume (`'tilt'`, 1 s) face à vous. C'est exactement ce que montrera ensuite l'explorateur Z-stack.



▲ Après avoir choisi « À l'envers » : l'aperçu se couche, la face choisie vers vous (mêmes repères numérotés).

- **Ni Q_base ni une vue par défaut ne dépendent du sens** : les deux sont des poses de l'anatomie ; l'éditeur n'enregistre que le drapeau.
- Le sens est pris en compte par la vue initiale d'un jeu non calibré, le bouton de réinitialisation, la vue « 3d », le Z-stack et les figures du Studio (lacet 180° sur la face −Z).

⚠️ ATTENTION

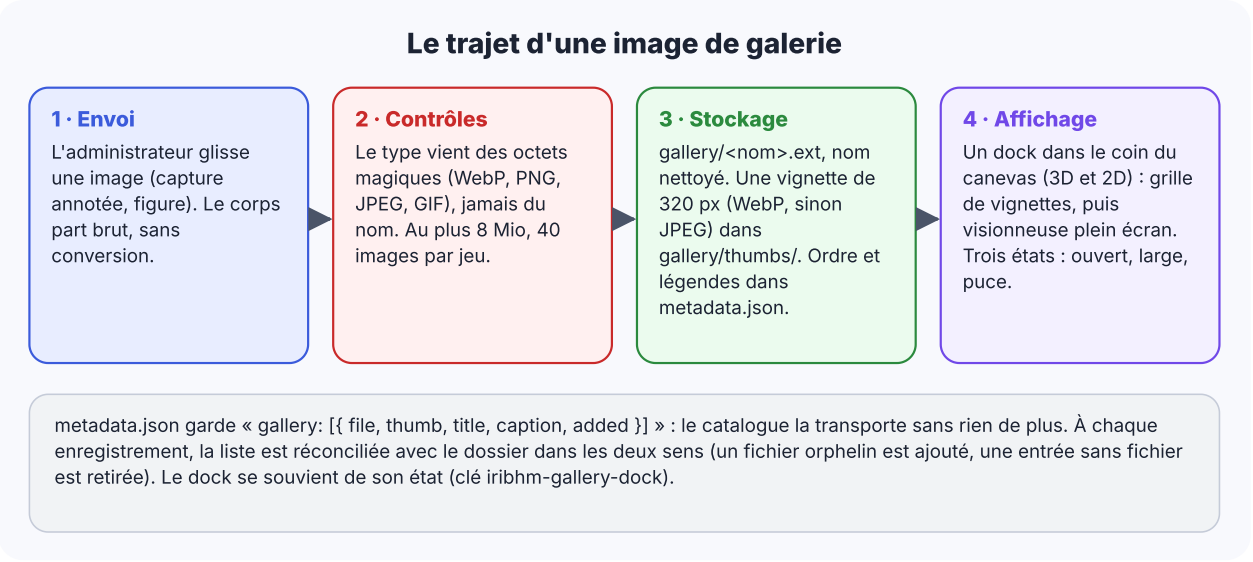
Un jeu dont le sens a été basculé sous la version 1.55.7 de la plateforme a un repère faussé (il avait été tourné autour de l'axe X du fichier). Il faut **définir de nouveau son orientation**.

13.10 Galerie, jeux liés, vignette

⏱ EN 30 SECONDES

- La **galerie** ajoute des images (captures annotées, schémas) à un jeu ; leur type vient des **octets magiques**, jamais du nom.
- Les **jeux liés** se déduisent de quatre règles, sans rien saisir.
- La **vignette** de l'explorateur est une capture de 512 × 512 que l'administrateur choisit.

La galerie d'images



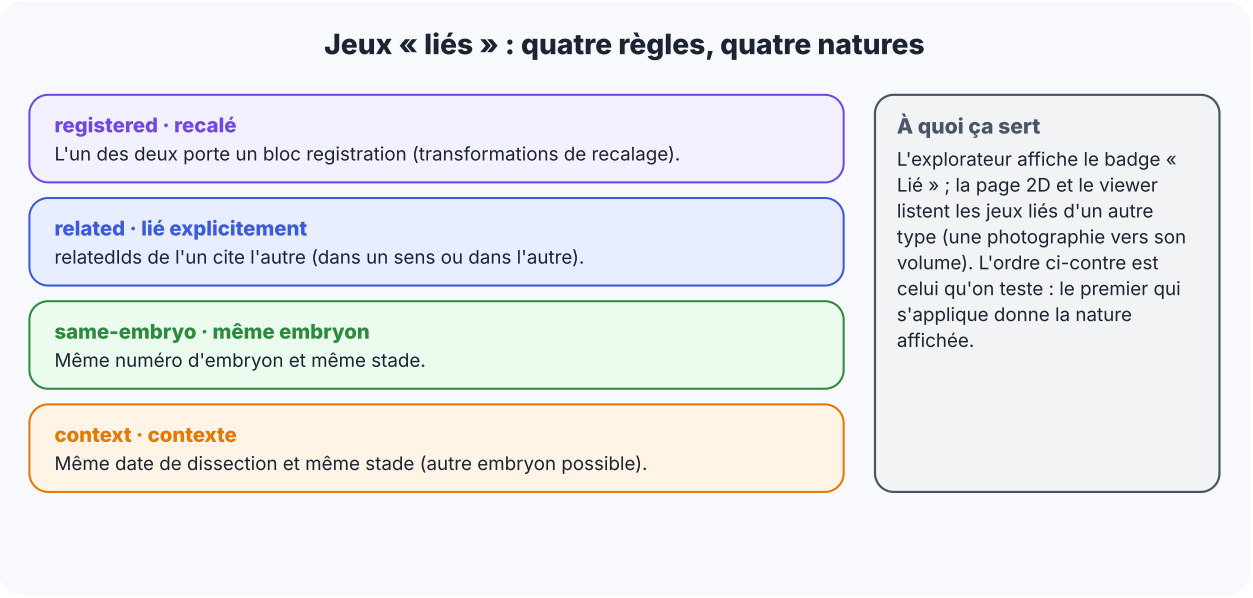
▲ Le trajet d'une image, de l'envoi à la visionneuse.

L'API accepte un corps d'image **brut** (jusqu'à la limite du serveur) ou, historiquement, une URL `data:`. Dans les deux cas, le type est reconnu à ses **octets magiques** (WebP, PNG, JPEG, GIF) ; le nom de fichier ne sert qu'à fabriquer un nom lisible (lettres, chiffres, tirets, 60 caractères au plus, numéroté en cas de doublon). Un fichier trop grand (8 Mio), un 41ᵉ envoi ou un contenu qui n'est pas une image sont refusés.

La **vignette** de 320 px sert la grille du dock : quarante originaux ne doivent pas se disputer les connexions dont les paquets de briques ont besoin. Sans Pillow ou sans WebP côté serveur, elle tombe sur du JPEG, ou est omise. Une image d'un jeu **en cours d'import** est refusée (409) : publiez d'abord.

Côté page, `DatasetGallery` ne construit son DOM qu'avec `createElement`, jamais en concaténant du texte : les légendes sont du texte d'opérateur et les noms de fichier viennent du disque.

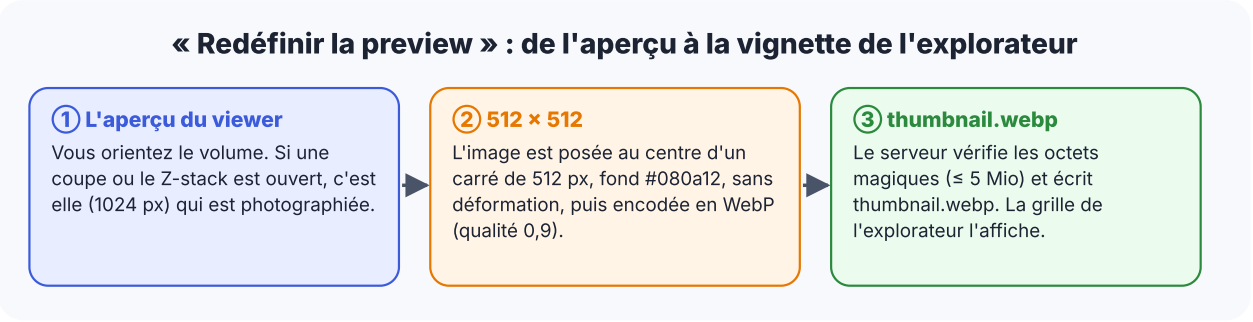
Les jeux liés



▲ Les quatre natures d'un lien entre deux jeux.

`Catalog.getRelated(id)` renvoie les jeux qui partagent **le même embryon et le même stade**, **la même date de dissection et le même stade**, ou qu'un `relatedIds` désigne (dans un sens ou dans l'autre). `getRelationMeta` donne ensuite la **nature** du lien, en testant dans cet ordre : *registered* (un bloc de recalage existe), *related* (lien explicite), *same-embryo*, *context* (le reste). L'explorateur affiche le badge « Lié » ; la page 2D et le viewer listent les jeux liés d'un autre type : une photographie renvoie vers le volume du même embryon.

La vignette de l'explorateur



▲ De l'aperçu de l'administration à thumbnail.webp.

Le bouton **Redéfinir la preview** demande une capture au viewer de l'aperçu. Si une coupe ou le Z-stack est ouvert, c'est **elle** qui est photographiée (rendue à 1024 px) ; sinon le canevas 3D. L'image est posée au centre d'un carré de 512 × 512 px de fond `#080a12` **sans déformation**, puis encodée en WebP (qualité 0,9). Le serveur vérifie les octets magiques et une taille d'au plus 5 Mio avant d'écrire `thumbnail.webp`.

📖 **POUR ALLER PLUS LOIN**

- Le format des dossiers `planes/` et `mips/`, et le journal des migrations : chapitres 7 et 17.
- Le système de plugins (`contexts`, `dataTypes`) et la confiance : chapitre 15.
- Les noms exacts des écrans d'administration : chapitre 14 et guide de l'administrateur.

📌 **À RETENIR**

Ce chapitre en une page :

- **Suivi** : sphères instanciées, tableaux 32 bits, une façade partagée.
- **Stabilisation** : `volumeWarp = toTex · M⁻¹ · toUm` ; aucun voxel retouché.
- **Studio** : document sans pixels, recoloration depuis les valeurs brutes, passe native plan / atlas, jeton.
- **Comparer** : vraies pages, messages vérifiés, budget 1,5 Gio, temps par fraction.
- **2D** : trois repères, worker de pixels, importeur qui ne devine rien.
- **Frise** : horloge plafonnée à 250 ms, retenue pendant le chargement.
- **État** : le lien est la sauvegarde ; le navigateur ne retient que des préférences.
- **Mesure** : profondeur lue sur le GPU en deux passes de 16 bits ; poses par deux rotations.
- **Orientation** : Q_base, Q_anat, Q_cube ; le sens de l'échantillon est un demi-tour de la verticale.

14. Le panneau d'administration en bref

- Le panneau d'administration est la **salle des machines** de Lumen3D : c'est là que l'on met des données en ligne, que l'on règle l'apparence du site et que l'on met la plateforme à jour.
- Il compte **15 onglets en 4 groupes**. Vous n'en utiliserez que quelques-uns au quotidien.
- Ce chapitre est un **survol**. Le mode d'emploi pas à pas est le « **Guide de l'administrateur** », publié dans l'onglet **Documentation** du panneau lui-même.

14.1 À quoi sert le panneau, et comment y accéder

Le site public est la vitrine : les visiteurs regardent des embryons. Le panneau d'administration est l'**arrière-boutique**. Il sert à :

- **mettre des datasets en ligne** et les décrire (nom, stade, couleurs, orientation) ;
- **choisir ce que le public voit** (visible ou masqué) ;
- **personnaliser le site** : nom, couleurs, pages, mentions légales ;
- **installer des plugins** et **mettre la plateforme à jour**.

ℹ️ **À SAVOIR**

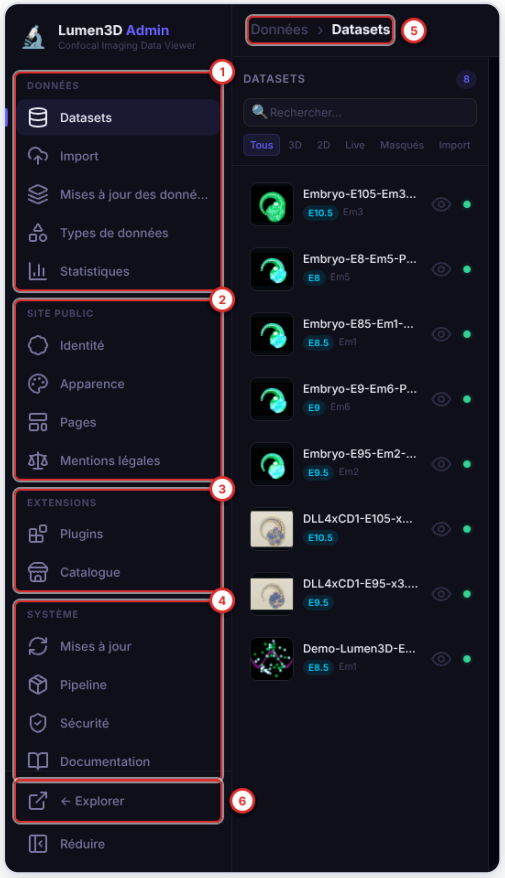
Le panneau n'a **aucun lien public**. On l'ouvre en tapant l'adresse `admpan.html` à la suite de celle du site (par exemple `https://votre-site/admpan.html`). Il demande un identifiant et un mot de passe. Le mot de passe est créé lors de la toute première visite, grâce à un assistant en 5 étapes (compte, identité, thème, textes, plugins).

14.2 Les 15 onglets, en 4 groupes

La barre latérale de gauche range les onglets en quatre groupes. Le fil d'Ariane en haut rappelle où vous êtes (par exemple **Données › Datasets**).

ONGLET	CE QU'IL FAIT	FRÉQUENCE
Données		
Datasets	Liste, aperçu, réglages	souvent
Import	Envoyer un dossier du pipeline	à chaque nouveau dataset
Mises à jour des données	Mettre au format 5	rarement
Types de données	Renommer 3D, 2D, Live	une fois
Statistiques	Visites, vues, téléchargements	à l'occasion
Site public		

ONGLET	CE QU'IL FAIT	FRÉQUENCE
Identité	Nom, vocabulaire, SEO, menu	à l'installation
Apparence	Couleurs, police, arrondis	à l'installation
Pages	Éditeur visuel de pages	parfois
Mentions légales	Textes légaux, par langue	une fois
Extensions		
Plugins	Activer, approuver	parfois
Catalogue	Installer des plugins signés	parfois
Système		
Mises à jour	Mettre la plateforme à jour	régulièrement
Pipeline	Télécharger le pack (ch. 4 à 8)	à chaque nouveau pack
Sécurité	Mot de passe, permissions	rarement
Documentation	Lire les guides publiés	à la demande



▲ Barre latérale du panneau (jeu de démonstration).

- 1 Groupe **Données** : les jeux de données
- 2 Groupe **Site public** : apparence et textes vus par les visiteurs
- 3 Groupe **Extensions** : les plugins
- 4 Groupe **Système** : mises à jour, sécurité, documentation
- 5 Fil d'Ariane : groupe puis onglet courant
- 6 Lien vers le site public (nouvel onglet)

En haut à droite : thème clair/sombre du panneau, langue (français, anglais, espagnol, néerlandais) et déconnexion.

📌 **À RETENIR**

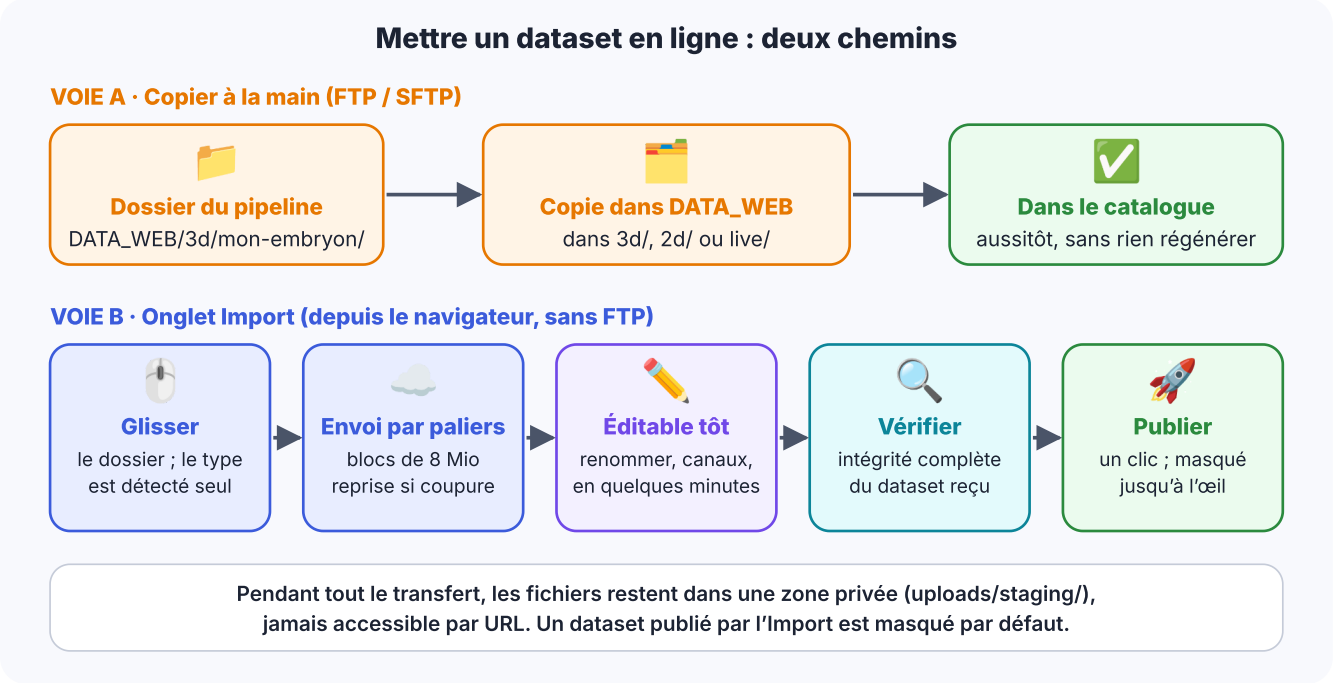
Deux onglets commencent par « Mises à jour » : **Mises à jour des données** (groupe Données : remettre **vos datasets** au format actuel) et **Mises à jour** (groupe Système : mettre à jour **la plateforme**). Ne les confondez pas.

👉 **ASTUCE**

Une pastille orange « Modifications non sauvegardées » s'allume en haut dès qu'un onglet contient des changements pas encore enregistrés. `Ctrl` + `S` enregistre l'onglet visible (Datasets, Types de données, Identité, Apparence, Mentions légales).

14.3 Mettre un dataset en ligne

Le pipeline (chapitres 4 à 8) produit **un dossier**. Il y a deux façons de le déposer sur le serveur.

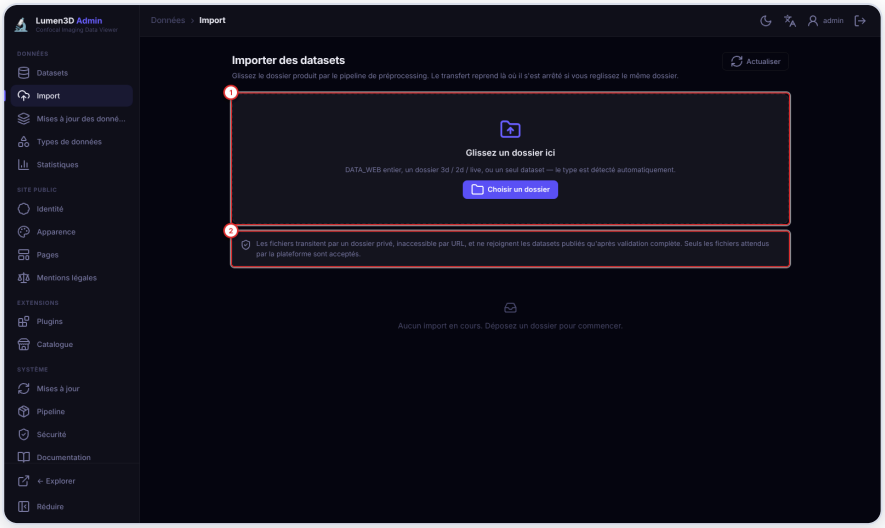


▲ Deux chemins pour mettre un dataset en ligne.

🔢 **MICRO-EXEMPLE**

Voie A : vous copiez le dossier dans `DATA_WEB/3d/` par FTP. Le dataset apparaît tout de suite : le catalogue est recalculé à chaque requête à partir des fichiers `metadata.json`, il n'y a rien à régénérer.

L’onglet Import, pas à pas



▲ L’onglet Import, avant le dépôt d’un dossier.

1 Zone de dépôt : glissez le dossier ou cliquez sur **Choisir un dossier** · 2 Rappel : transit par un dossier privé, seuls les fichiers attendus sont acceptés. Une fois le dossier déposé, l’onglet affiche la progression globale (pourcentage, volume, vitesse, temps restant) et une carte par dataset avec son état et ses boutons **Éditer**, **Vérifier**, **Publier**, **Supprimer**.

Les états d’un dataset en cours d’import :

ÉTAT AFFICHÉ	CE QUE CELA VEUT DIRE
Envoi — non éditable	les fichiers indispensables à l’ouverture ne sont pas tous là
Envoi — éditable	ouvrable en basse résolution : renommez, réglez les canaux pendant que le reste arrive
Envoyé — à publier	transfert complet et vérifié ; il reste à publier
Interrompu	reglissez le même dossier pour reprendre ; sans reprise, purge au bout de 7 jours

💡 **UNE IMAGE POUR COMPRENDRE**

Un déménagement par camion-navette. Le dossier part en petits cartons numérotés (blocs de 8 Mio) vers un entrepôt privé. Si le camion tombe en panne, on repart au dernier carton livré. Les cartons les plus utiles (fiche, plan, vue grossière) partent en premier : on peut commencer à décorer avant que tout soit arrivé.

⚠️ **ATTENTION**

Un dataset publié par l’Import est **masqué par défaut**. Allez dans **Datasets** puis activez l’œil (ou l’interrupteur **Visibilité**) pour le montrer au public.

14.4 Éditer un dataset

Cliquez un dataset dans la liste de l'onglet **Datasets** : l'aperçu s'ouvre au centre, les réglages à droite. Voici ce que vous pouvez y régler :

SECTION	CE QUE VOUS Y DÉCIDEZ
Visibilité	visible ou masqué dans l'explorateur (**appliqué immédiatement**)
Identification	nom d'affichage, stade, embryon, description
Galerie d'images	jusqu'à 40 images (8 Mo max) avec légendes, affichées dans le viewer
Calibration physique	taille du voxel X, Y, Z en µm
Paramètres d'affichage	exposition par défaut
Orientation	sens de l'échantillon, repère de référence, axes affichés, vue par défaut
Canaux (dans l'aperçu)	nom, couleur, min, max, gamma de chaque canal

> 📌 **À RETENIR**
>
> Ce que l'éditeur enregistre est un **réglage** : il est fusionné dans le `metadata.json` du dataset. Le panneau ne touche jamais aux pixels du niveau natif. La position de la caméra, le mode de rendu ou le plan de coupe ne sont pas conservés : ils appartiennent au visiteur.

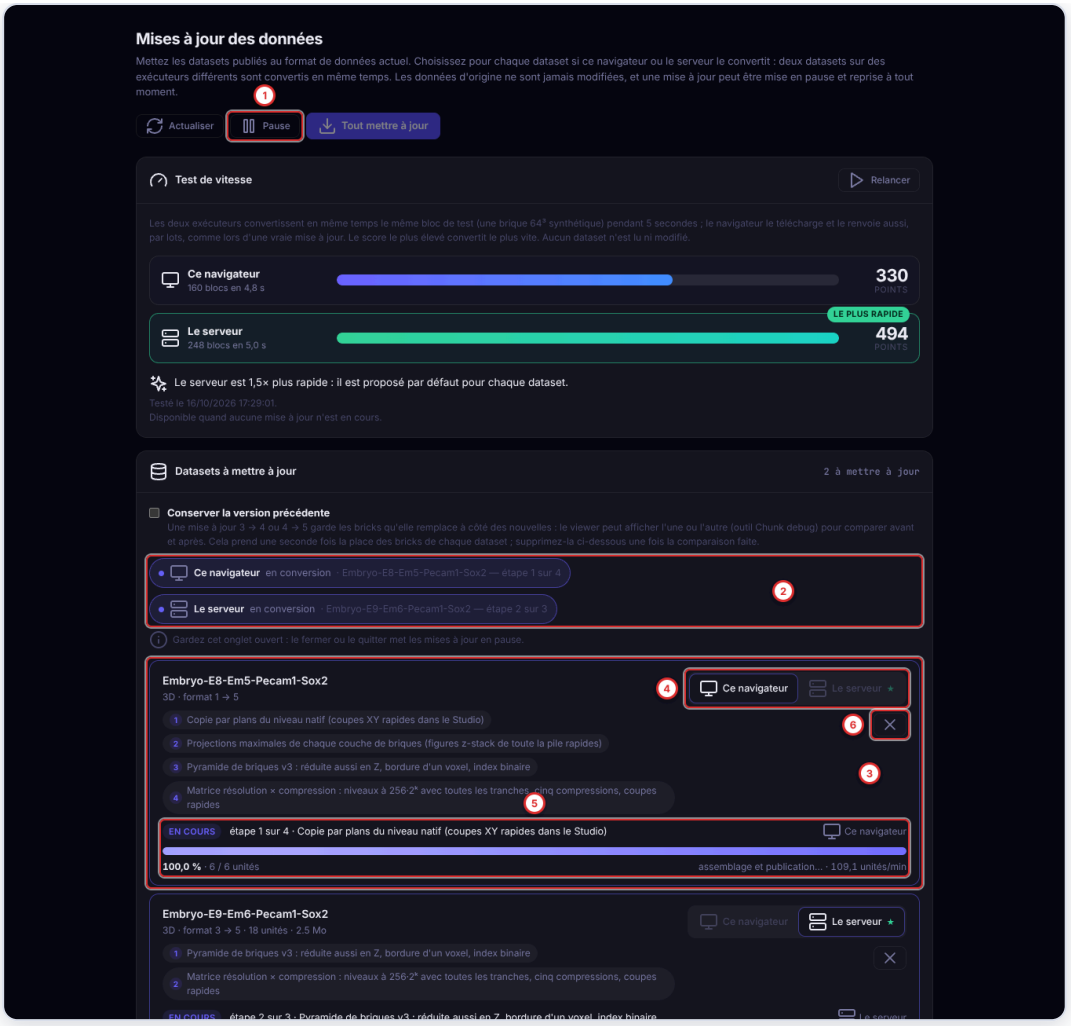
Le bouton **Redéfinir la preview** (sous l'aperçu) fait de la vue actuelle la vignette de l'explorateur. L'orientation 3D (axes, vue par défaut) est expliquée au chapitre 12.

14.5 Mises à jour des données

La façon dont les datasets sont stockés a évolué cinq fois (**formats 1 à 5**, voir chapitre 7). Un ancien dataset reste lisible, mais il peut être **mis au format actuel sans repasser par le pipeline**, comme on met à jour un logiciel.

Les quatre étapes ajoutent : `planes/` (un fichier par plan, coupes XY natives rapides dans le Studio), `mips/` (projection max de chaque couche de 64 plans, figures z-stack rapides), une pyramide de briques v3 (66^3, bordure d'un voxel, `index.bin`), puis la **matrice du format 5** (niveaux qui gardent toutes les tranches, cinq compressions, copies de coupe ; section 7.11).

> ❓ **POURQUOI CE CHOIX ?**
>
> Le niveau natif est conservé voxel pour voxel : rien n'est perdu. L'ancien arbre `bricks/` n'est supprimé qu'après le changement de version du dataset. Un dataset produit par le pipeline 0.22.0 arrive au format 4 (au format 5 avec `--format 5`) : l'onglet propose alors la seule étape 4 → 5. Sur un hébergement PHP, cette étape se fait **dans le navigateur**.



▲ L'onglet Mises à jour des données pendant une conversion vers le format 5 (jeu de démonstration).

1 **Pause** · 2 Les deux files : ce navigateur et le serveur, chacune avec son dataset en cours · 3 La carte d'un dataset : « format 1 → 5 », ses étapes · 4 Choix de l'exécutant, dataset par dataset (verrouillé pendant l'exécution) · 5 La progression · 6 Annuler. Le détail est dans le *Guide de l'administrateur*, chapitre 5.

> ⚠️ **ATTENTION**
>
> **Gardez l'onglet ouvert** pendant une mise à jour. Le quitter la met en pause ; rien n'est perdu, vous reprenez plus tard. N'ouvrez pas plusieurs onglets à la fois : un hébergeur a déjà bloqué l'adresse d'un opérateur qui lançait trop de requêtes. Le régulateur réseau de l'onglet limite le débit pour éviter cela.

Ces mises à jour ne sont **pas obligatoires** : le viewer lit les formats 1 à 5. Elles accélèrent certaines opérations du Studio et améliorent l'affichage.

14.6 Plugins et catalogue signé

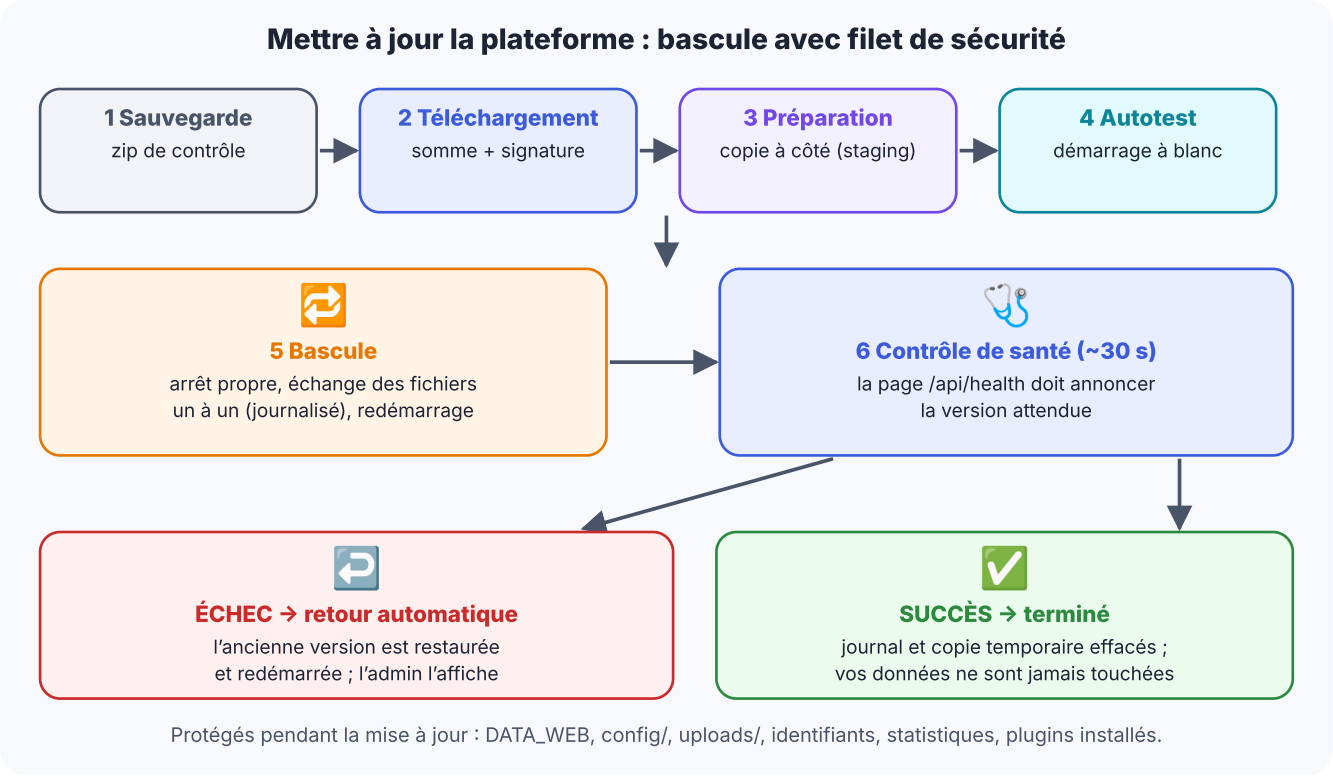
Les plugins sont les outils du viewer (chapitre 12). Ils **ne sont pas livrés avec la plateforme** : on les installe à la demande depuis le **Catalogue**, comme dans une boutique d'applications.

- Le catalogue est **signé** : un plugin non vérifié n'est jamais installé (chapitre 19).
- L'installation demande votre **mot de passe**, puis le plugin est installé et approuvé.

- L'onglet **Plugins** affiche trois cartes (Outils, Canaux, Modes de rendu) : un interrupteur par plugin et une étiquette de confiance (`intégré` , `approuvé` , `sandbox` , `non fiable` …).
- Certains plugins n'apparaissent pas dans l'aperçu de l'onglet Datasets, ni dans la page Comparer : ils ne se déclarent compatibles qu'avec la page complète. Ce n'est pas une panne.

14.7 Mettre à jour la plateforme

Depuis **Système › Mises à jour**, un bouton lance la mise à jour. Ce qui se passe derrière a été pensé pour qu'**un site ne reste jamais cassé**.



▲ La bascule de mise à jour et son filet de sécurité.

💡 UNE IMAGE POUR COMPRENDRE

Le déménagement bleu-vert. On monte le nouvel appartement à côté de l'ancien, on vérifie que l'eau et l'électricité marchent, et seulement alors on y emménage. Si quelque chose cloche, on retourne dans l'ancien, resté intact.

- L'onglet affiche les **notes de version** de chaque version qui sera installée, et vérifie vos plugins avant : ceux qui seraient incompatibles sont mis de côté, puis réactivés plus tard. ::: warning Ce filet de sécurité complet (bascule vérifiée par une sonde de santé, **retour automatique** à l'ancienne version) existe sur un serveur **Python**. Sur un hébergement **PHP**, la mise à jour se fait en une seule requête, avec la même vérification d'empreinte SHA-256 et de signature, mais **sans retour automatique** : les fichiers déjà copiés restent en place. → chapitre 18. :::
- Il montre aussi la version du **pack Pipeline** (onglet **Pipeline**) : c'est un autre logiciel, qui s'installe sur le poste de traitement, pas sur le serveur.

14.8 Personnaliser le site (marque blanche)

Lumen3D n'est pas lié à un laboratoire : tout ce que le public voit se règle sans écrire de code.

ONGLET	CE QUE VOUS Y PERSONNALISEZ
Identité	nom de l'instance, nom du « spécimen » (embryon, organe…), accroche, pied de page, liens du menu
Apparence	couleur de marque, police, arrondis, avec aperçu en direct
Pages	éditeur visuel : sections, colonnes, widgets ; sauvegarde automatique du brouillon
Mentions légales	sections de texte, une langue à la fois
Types de données	le **nom** donné à « 3d », « 2d » et « live » dans les badges et les filtres

ℹ️ **À SAVOIR**

Renommer un type de données ne change que le **texte affiché** : jamais les dossiers, les identifiants ni les adresses. Le nom du spécimen et des types s'adapte automatiquement dans tout le site.

📖 **POUR ALLER PLUS LOIN**

Pour le détail de chaque écran, des messages d'erreur et des cas particuliers, consultez le « **Guide de l'administrateur** » (onglet **Documentation**). Pour la sécurité du panneau, voir le chapitre 19.

15. Les plugins, la confiance et le catalogue signé

- Presque tout ce que vous faites dans le viewer (mesurer, capturer, régler un histogramme, choisir un mode de rendu) est fourni par un **plugin** : un petit dossier que la plateforme découvre, vérifie puis branche.
- Un plugin est du **code d'un tiers**. Lumen3D le traite comme une application de téléphone : il faut **vérifier son identité à l'installation**, lui accorder des **permissions précises**, et pouvoir l'**enfermer**.
- Ce chapitre explique les deux faces du système : **comment un plugin fonctionne** (§15.1 à §15.11) et **pourquoi on peut lui faire confiance** (§15.12 à §15.19).

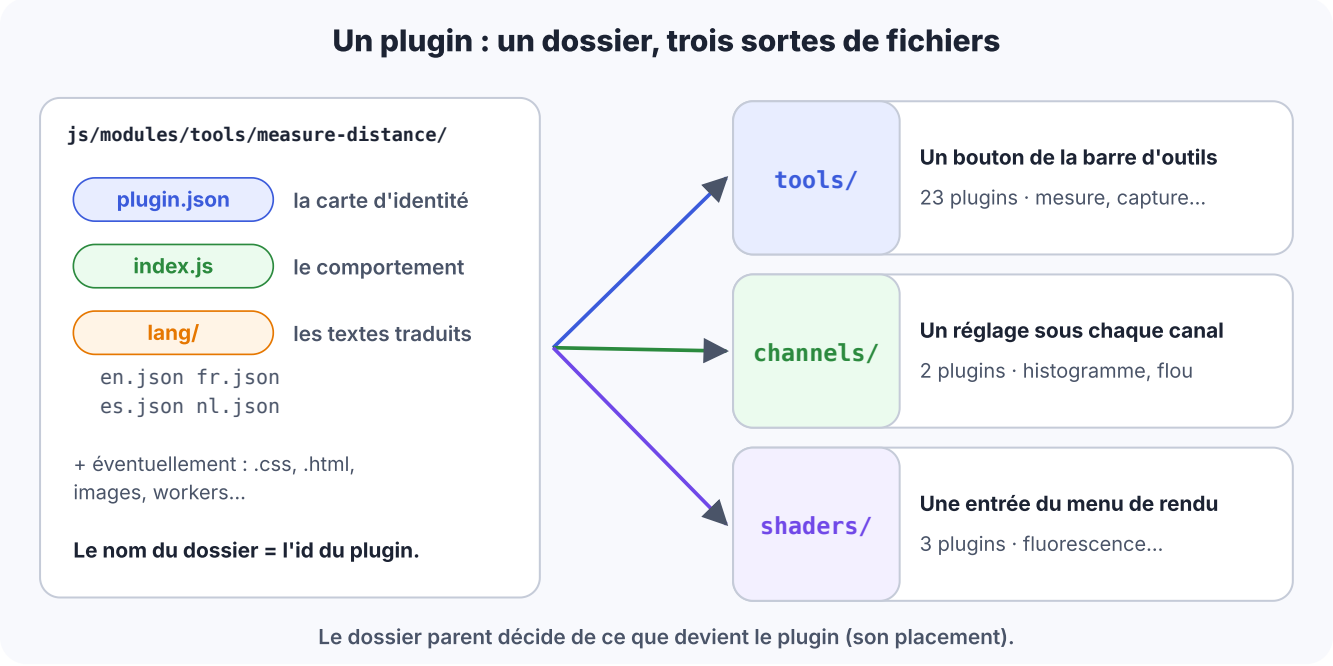
💡 **UNE IMAGE POUR COMPRENDRE**

Un smartphone et sa boutique d'applications. La plateforme est le téléphone. Un plugin est une application. Le catalogue signé est la boutique officielle : chaque application y porte un sceau. À l'installation, le téléphone contrôle le sceau (signature), l'empreinte du fichier (hash) et la compatibilité avec sa version (platformCompat). Il enferme ensuite l'application dans un bac à sable et ne lui donne que les permissions demandées (capacités).

Le chapitre 12 explique **comment utiliser** chaque outil, le chapitre 13 ce qu'il y a **sous le capot** de chacun, le chapitre 14 **les écrans** de l'administration. Ici, on regarde **le système qui les porte tous**.

15.1 Qu'est-ce qu'un plugin ?

Un plugin est un **dossier** rangé dans `js/modules/<placement>/<id>/`. Il contient trois sortes de fichiers.



▲ Un dossier, trois sortes de fichiers, trois destinations possibles.

FICHIER	RÔLE	ANALOGIE
`plugin.json`	Qui je suis, où je me place, ce que j'exige	La carte d'identité
`index.js`	Ce que je fais (appelle `PluginRegistry.implement`)	Le savoir-faire
`lang/<code>.json`	Mes textes en français, anglais, espagnol, néerlandais...	Le mode d'emploi traduit

Le **dossier parent** décide de ce que devient le plugin. C'est son **placement**.

PLACEMENT	CE QUE LE VISITEUR VOIT	NOMBRE AU CATALOGUE
`tools`	Un bouton de la barre d'outils (ou un outil exclusif)	23
`channels`	Un réglage sous chaque canal de fluorescence	2
`shaders`	Une entrée du menu **Mode de rendu**	3

Total : **28 plugins**, exactement les 28 entrées du catalogue signé (§15.17).

> ℹ️ **À SAVOIR**
>
> **Découverte automatique.** Pas de liste à modifier : on dépose un dossier, il apparaît ; on le retire, il disparaît. La liste est reconstruite à chaque chargement de page (§15.3).

15.2 La carte d'identité : `plugin.json` champ par champ

Voici le `plugin.json` réel de l'outil de mesure (ses champs sont expliqués dans le tableau suivant).

```
{
  "id": "measure-distance",   "name": "Measure Distance",   "version": "1.1.4",
  "platformCompat": ">=1.53.0",
  "dataTypes": ["3d", "live", "2d"],   "contexts": ["page", "panel"],
  "creator": "IRIBHM",   "placement": "tools",
  "group": "tools",   "subtype": "tool",   "icon": "ruler",   "order": 20,
  "tool": "measure",   "shortcut": "m",
  "description": "Pick two 3D surface points to measure calibrated physical distance in µm.",
  "i18nTitle": "title",   "i18nLanguages": ["en", "fr", "es", "nl"]
}
```

Le chargeur n'**exige** que l'`id` (et qu'il soit égal au nom du dossier). Tout le reste a une valeur par défaut raisonnable.

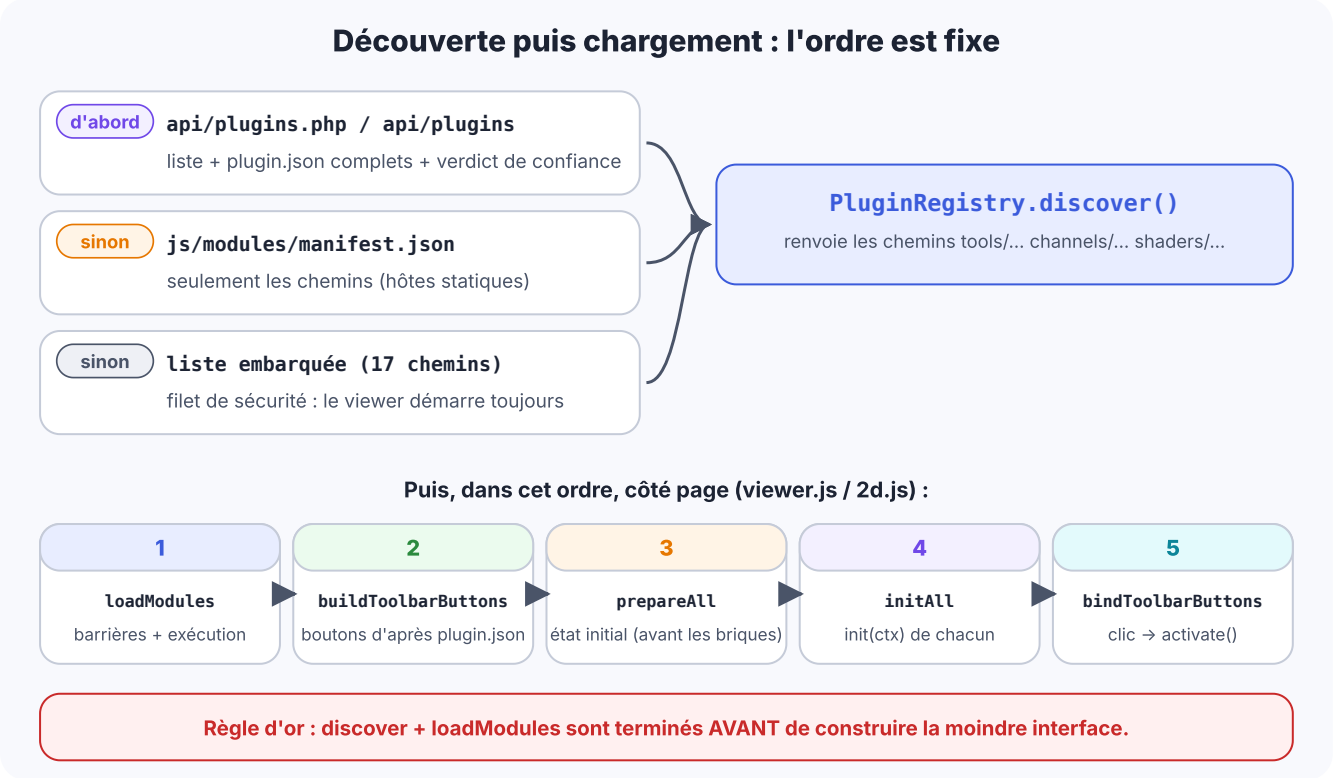
CHAMP	CE QU'IL DIT	VALEURS ET RÈGLES
`id`	Identité unique	**Égal au nom du dossier** (`[A-Za-z0-9_][A-Za-z0-9._-]*`), sinon quarantaine
`name` , `creator` , `description`	Texte d'affichage (carte du catalogue, onglet Plugins)	Libre
`version`	Version **du plugin** (SemVer)	Le catalogue la compare pour proposer une mise à jour
`placement`	Où il se place	`tools` , `channels` ou `shaders` ; doit **égaler le dossier parent**
`platformCompat`	Versions de la plateforme acceptées	Intervalle ou liste (§15.15) ; absent = compatible
`dataTypes`	Types de données couverts	`3d` , `2d` , `live` ; **absent** = ancien plugin du viewer de volumes
`contexts`	Où il sait vivre	`page` (page complète) et/ou `panel` (page embarquée) ; **absent** = `page` seulement
`requires`	Sources dont il a besoin	`bricks` , `webstack` , `tracking` ; le bouton reste **caché** tant qu'elles manquent
`group`	Grappe de la barre d'outils	`tools` , `export` , `visuals` , `layouts`
`subtype`	Sorte de bouton	`tool` , `toggle` , `action` (canaux : `per-channel`)
`icon`	Pictogramme	Nom d'une icône **Lucide**
`order`	Place dans la grappe	Croissant ; **999** si absent
`tool`	Nom de l'outil exclusif	Défaut = l'`id` ; seulement pour `subtype: "tool"`

CHAMP	CE QU'IL DIT	VALEURS ET RÈGLES
`shortcut`	Raccourci clavier	Une touche, pour un outil exclusif (`m` pour la mesure, `c` pour la coupe)
`buttonId`	Identifiant HTML du bouton	Simple jeton, accepté seulement si aucun autre élément ne le porte
`i18nTitle` , `i18nAria`	Clés du libellé et du texte d'accessibilité	Cherchées dans `lang/<code>.json` du plugin
`i18nLanguages`	Langues livrées	Le serveur la **recalcule** d'après le dossier `lang/`
`renderModeValue` , `default`	Modes de rendu seulement	Numéro de mode du lancer de rayons (0, 1, 2) ; `default: true` = mode choisi à l'ouverture
`sandbox` , `sandboxCapabilities`	Plugin écrit pour le bac à sable	§15.14

⚠️ **ATTENTION**

Un champ `trust` dans `plugin.json` est ignoré. Le plugin est écrit par un tiers : s'il pouvait déclarer lui-même son niveau de confiance, la vérification n'aurait plus de sens. Le registre **supprime** ce champ et ne lit le verdict que de la réponse du serveur.

15.3 Comment la plateforme trouve les plugins



▲ Trois sources par ordre de préférence, puis cinq étapes toujours dans le même ordre.

La page demande la liste à `PluginRegistry.discover()`. Trois sources, la première qui répond gagne.

1. **Le serveur** (`api/plugins.php`, puis `api/plugins` : l'adresse `.php` est essayée en premier car elle fonctionne sur les deux types de serveur). Il répond avec, pour chaque plugin, le `plugin.json` **complet**, les traductions déjà incluses et le **verdict de confiance** (§15.12).
2. **Le fichier** `js/modules/manifest.json` : des triplets `{chemin, placement, id}` seulement. Le serveur le **réécrit** à chaque découverte, pour qu'un hébergement purement statique hérite d'une liste à jour.
3. **La liste embarquée** dans le code : 17 chemins du noyau historique. C'est un filet : même si tout le reste échoue, le viewer démarre.

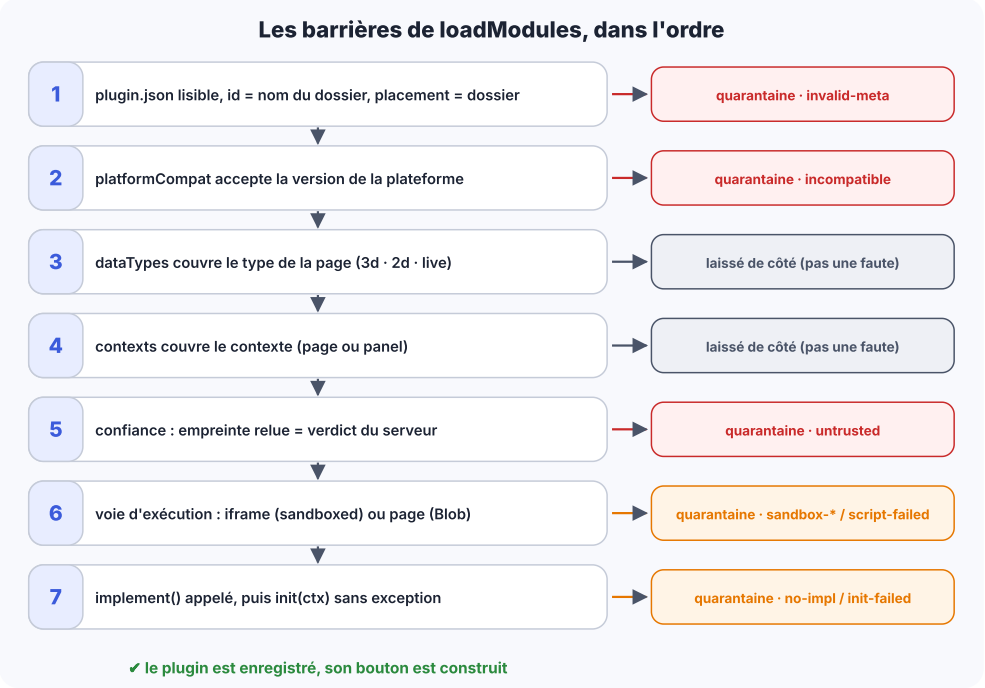
Le serveur **filtre** avant de répondre : il retire les plugins désactivés par l'administrateur, les plugins **incompatibles** et les plugins **non fiables**. Un plugin non fiable n'est donc même pas candidat au chargement.

> ❓ **POURQUOI CE CHOIX ?**
>
> **Pourquoi tout attendre avant de dessiner ?** Si la barre d'outils était dessinée avant la fin du chargement des plugins, elle apparaîtrait vide. Cette règle a été violée une fois (version 0.12.45) : le résultat était un viewer sans outils. Depuis, `discover` et `loadModules` sont **toujours terminés** avant la construction de la moindre interface.

> 🔧 **POUR LES CURIEUX**
>
> Un dossier n'est pris en compte que si son nom respecte `^[A-Za-z0-9_][A-Za-z0-9._-]*$` (aucun `..`, aucune barre oblique) et si son `plugin.json` est lisible et ne contredit pas le dossier parent. Sinon il est **ignoré silencieusement** par le serveur. Les tableaux de traductions ne sont joints que si l'anglais est présent. La réponse porte aussi `devTrust` (le mode développeur est-il actif ?) et `trustEpoch` (§15.14).

15.4 Le chargement : sept barrières

Une fois la liste en main, `loadModules` examine chaque plugin à travers une série de contrôles. Les quatre premiers sont des **choix** (ce plugin vise-t-il cette page ?), les suivants sont des **sécurités**.



▲ Chaque plugin franchit les barrières dans l'ordre ; à droite, ce qui arrive à ceux qui échouent.

Deux sortes d'échec, à ne pas confondre :

- **Laissé de côté** (barrières 3 et 4) : le plugin n'est pas fait pour cette page. Un outil de photographie 2D n'a rien à faire sur un volume 3D. Ce n'est **pas une faute** : seul un message d'information apparaît dans la console.
- **Mis en quarantaine** : quelque chose ne va pas. La raison est notée dans un registre (`PluginRegistry.getQuarantined()`) et un avertissement apparaît dans la console du navigateur.

🔧 POUR LES CURIEUX

Raisons de quarantaine (champ `reason`) : `meta-unreachable` (plugin.json introuvable), `invalid-meta` (JSON illisible, id absent ou différent du dossier, placement incohérent, plugin.json modifié pendant la vérification), `incompatible`, `untrusted`, `trust-unavailable` (le module de confiance n'est pas chargé : par prudence rien n'est exécuté), `sandbox-unsupported-placement`, `sandbox-unavailable`, `sandbox-boot-failed`, `script-failed` (index.js ne se charge pas), `no-impl` (index.js n'a jamais appelé `implement`), `init-failed` (init a levé une exception), `revoked` (approbation retirée à chaud), `load-error` (autre exception).

📌 À RETENIR

Un plugin en panne ne coûte jamais le viewer. Chaque plugin est chargé, initialisé et activé dans son propre `try/catch`, et une barrière globale entoure toute la phase des plugins : même si tout ce sous-système échouait, le canvas 3D démarrerait.

15.5 Qui se charge où ? `dataTypes` et `contexts`

Deux champs de `plugin.json` décident si le plugin est pris sur une page donnée.

`dataTypes` : le **type** de dataset (`3d`, `2d`, `live`).

- Si présent : le plugin tourne sur ces types et **sur aucun autre**.

`contexts` : la **situation** de la page.

- `page` : page complète, avec sa barre d'outils.

- Si absent : c'est un ancien plugin écrit pour le viewer de volumes ; seul le viewer 3D / live l'accepte (jamais la page 2D).

- `panel` : page **embarquée** sans barre (panneau de Comparer, volet de vue divisée, aperçu de l'admin).
- Si absent : `page` seulement (**refus par défaut** dans une page embarquée).

Qui se charge où ? (28 plugins × type × contexte)

		3d	3d	live	live	2d	2d
		page	panel	page	panel	page	panel
calibrated-grid	tools	·	·	·	·	●	●
chunk-debug	tools	●	·	●	·	·	·
decompose-channels	tools	●	·	●	·	·	·
display-adjust	tools	·	·	·	·	●	●
download-center	tools	●	·	●	·	●	·
figure-panel	tools	·	·	·	·	●	·
measure-distance	tools	●	●	●	●	●	●
orientation-2d	tools	·	·	·	·	●	●
orientation-axes	tools	●	●	●	●	·	·
presentation-mode	tools	●	·	●	·	●	·
screenshot	tools	●	·	●	·	●	·
screenshot-sandboxed	tools	●	·	●	·	·	·
slice-inspector	tools	●	●	●	●	·	·
split-view	tools	·	·	·	·	●	·
toggle-axes	tools	●	●	●	●	·	·
toggle-grid	tools	●	●	●	●	·	·
toggle-volume	tools	●	●	●	●	·	·
tracking-charts	tools	·	·	●	·	·	·
tracking-inspector	tools	·	·	●	●	·	·
tracking-measure	tools	·	·	●	·	·	·
tracking-surface	tools	·	·	●	●	·	·
tracking-trails	tools	·	·	●	●	·	·
zstack-browser	tools	●	●	●	●	·	·
gaussian-filter	channels	●	●	●	●	·	·
histogram	channels	●	●	●	●	·	·
fluorescence	shaders	●	●	●	●	·	·
natural-fluorescence	shaders	●	●	●	●	·	·
structure-dvr	shaders	●	●	●	●	·	·

● = chargé · = laissé de côté (calculé d'après les plugin.json réels)

▲ Résultat réel pour les 28 plugins, calculé d'après leurs plugin.json.

PAGE	PLUGINS CHARGÉS
Volume 3D, page complète	18
Volume 3D, dans un panneau	12
Timelapse (live), page complète	23
Timelapse (live), dans un panneau	15
Photographie 2D, page complète	9
Photographie 2D, dans un panneau	4

Dix plugins sont « page seulement » : Chunk Debug, Decompose by Channel, Download Center, Figure Panel Builder, Presentation Mode, Screenshot, Screenshot (sandboxed), Split View, Tracking Charts, Cell Distance. Un panneau de 300 pixels ne peut pas accueillir un plein écran, un menu modal

ou une vue divisée dans la vue divisée. **Ce n'est pas une panne** : l'administrateur le voit aussi dans l'aperçu de l'onglet Datasets.

> 🔢 **MICRO-EXEMPLE**
>
> Sur la page 2D, le viewer demande `dataType = 2d`. `measure-distance` déclare `["3d","live","2d"]` : pris. `zstack-browser` ne déclare rien : un ancien plugin du viewer de volumes, donc **refusé** par la page 2D (qui n'accepte pas les plugins sans `dataTypes`). `calibrated-grid` déclare `["2d"]` : pris sur la 2D, absent du volume 3D.

15.6 Le cycle de vie d'un plugin

▲ De l'enregistrement à la fin de la page, avec la quarantaine comme issue de secours.

Un plugin est un objet que `index.js` remet au registre : `PluginRegistry.implement('mon-id', { … })`. Les crochets, tous facultatifs :

CROCHET	APPELÉ QUAND	DÉTAIL
`prepare(ctx)`	Avant le premier chargement de briques	État **initial** de la scène (ex. Orientation Axes applique la vue par défaut). Synchrone
`init(ctx)`	Après le chargement du volume	Reçoit son contexte (§15.7) ; peut renvoyer son instance
`activate()`	Clic sur le bouton, ou changement de mode de rendu	Renvoie `{ active, icon }` pour une bascule
`deactivate()`	Désactivation	Retour à l'état « initialized »

CROCHET	APPELÉ QUAND	DÉTAIL
`getState()` / `setState(s)`	Sauvegarde / restauration de l'espace de travail	Les états de tous les plugins sont rassemblés dans un seul objet
`reset()`	**Réinitialiser l'espace de travail**	Seuls les plugins qui le déclarent sont touchés
`onLanguageChange()`	Changement de langue	Repeindre les textes dynamiques
`getExports()` / `getGraph()`	Ouverture du Centre de téléchargement	§15.9
`dispose()`	Fin de page	Libérer les ressources

Les trois **sortes de bouton** (`subtype`) se comportent différemment.

SUBTYPE	COMPORTEMENT	EXEMPLE
`action`	Un clic, une fonction, rien à retenir	Screenshot
`toggle`	Allumé / éteint, l'état est mémorisé et rendu au bouton	Toggle Grid
`tool`	**Outil exclusif** : un seul actif à la fois (géré par `ToolManager`)	Measure Distance

🔢 **MICRO-EXEMPLE**

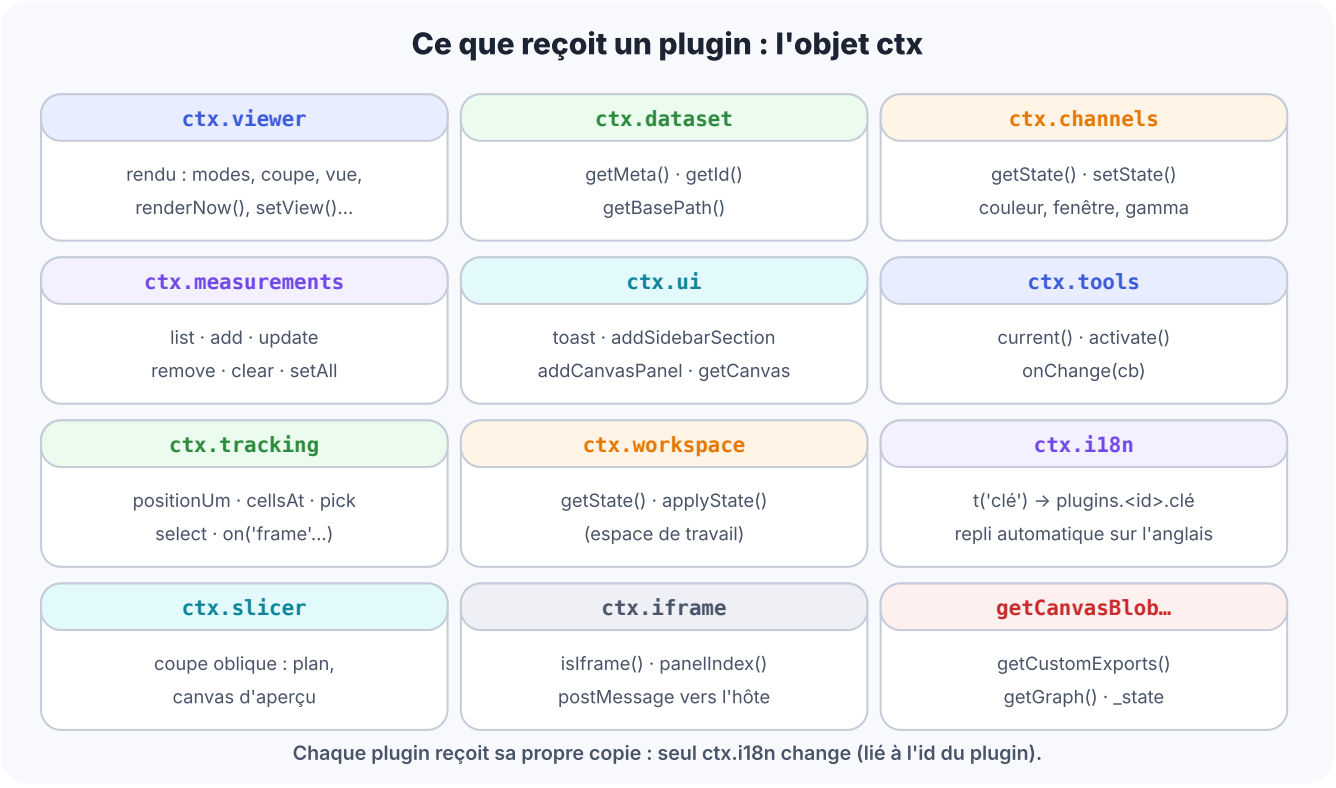
Voici le plugin Toggle Grid (version condensée). Chaque clic fait passer la grille d'« aucune » à « normale » puis « fine ».

```
PluginRegistry.implement('toggle-grid', {
  init(ctx) { this._ctx = ctx; this._mode = 0; return this; },
  activate() {
    this._mode = (this._mode + 1) % 3;
    VolumeViewer.setGridMode(this._mode);
    return { active: this._mode > 0 };       // le bouton s'allume
  },
  getState() { return { gridMode: this._mode }; },
  setState(s) {
    this._mode = s.gridMode;
    VolumeViewer.setGridMode(this._mode);
    PluginRegistry.syncToolbarButton('toggle-grid', { active: this._mode > 0 });
  },
  reset() { this.setState({ gridMode: 0 }); }
});
```

Les **plugins de canal** et de **mode de rendu** ont des contrats propres : un plugin de canal fournit `getChannelUI(canal)` (le HTML) puis `bindChannelUI(…)` (les écouteurs) ; un plugin de mode de rendu se contente de son `activate()` , qui règle le mode du lancer de rayons (chapitre 9).

15.7 Ce que reçoit un plugin : l'objet `ctx`

À l'`init`, la page remet à chaque plugin un **contexte** : une façade qui lui donne accès à ce dont il a besoin, rien de plus.



▲ Les douze rubriques du contexte d'un plugin.

RUBRIQUE	CE QU'ELLE OFFRE
`ctx.viewer`	Modes de rendu, coupes, vues, grille et axes, mesures, `renderNow()` (rendre l'image tout de suite avant de lire le canvas)
`ctx.dataset`	`getMeta()` (le `metadata.json`), `getId()`, `getBasePath()` ; sur la page 2D aussi la collection et `open(id)`
`ctx.channels`	L'état des canaux (couleur, fenêtre, gamma)
`ctx.measurements`	Les mesures mémorisées par dataset et par domaine
`ctx.ui`	`toast`, `addSidebarSection`, `addCanvasPanel`, `getCanvas`, `openStudio`…
`ctx.tools`	L'outil exclusif courant, `activate`, `onChange`
`ctx.tracking`	Le suivi cellulaire d'un timelapse (ci-dessous)
`ctx.data`	Les **voxels exacts**, sans perte, quelle que soit la qualité affichée (ci-dessous)
`ctx.workspace`	Lire / appliquer l'espace de travail

RUBRIQUE	CE QU'ELLE OFFRE
`ctx.i18n`	`t('clé')` : cherche `plugins.<id>.clé`, repli automatique sur l'anglais
`ctx.slicer`	Le plan de coupe oblique
`ctx.iframe`	Savoir si la page est embarquée ; `postMessage` vers l'hôte
Divers	`getCanvasBlob`, `getCustomExports`, `getGraph`

Chaque plugin reçoit **sa propre copie** du contexte : seule la rubrique `i18n` diffère, car elle est liée à l'identifiant du plugin.

> 🔧 **POUR LES CURIEUX**
>
> **Le contexte de la page 2D** a la même forme, avec les fonctions d'une photographie : `viewer.getView/setView`, `addOverlay`, `setOrientation`, `setAdjustments`, `getPhysicalView`, `getNativeCanvas`, et `ui.openStudioWith`. Un plugin qui s'adresse aux deux pages lit donc ses fonctions avec précaution (`ctx.viewer.renderNow?.()`).

Le suivi cellulaire : `ctx.tracking`

Les cinq plugins de suivi ne lisent jamais `tracks.json` eux-mêmes. La page en garde **une seule copie** (sous forme de tableaux compacts à 32 bits) et la leur prête.

FONCTION	RÔLE
`isAvailable()`, `whenLoaded()`	Le dataset a-t-il un suivi ? Attendre qu'il soit chargé
`positionUm(cellule, image, options, sortie)`	Position d'une cellule, en µm, à une image donnée
`cellsAt(image)`	Les cellules présentes à cette image
`pick(x, y)`	Cellule sous le curseur (−1 si rien)
`select(c)`, `getSelected()`	**Une seule sélection partagée** par tous les plugins
`getOptions()`, `setOptions({ neighborThresholdUm })`	Rayon de voisinage (5 à 500 µm) partagé
`on('loaded' · 'frame' · 'refresh' · 'selection' · 'options' · 'style', rappel)`	S'abonner ; renvoie la fonction de désabonnement

Résultat : cliquer une cellule dans l'Inspecteur la sélectionne aussi dans les trajectoires et les graphiques, sans qu'ils se parlent.

Les voxels exacts : `ctx.data`

Depuis la plateforme 1.64, un plugin qui **calcule** quelque chose (profil d'intensité, histogramme d'une région, mesure) passe par `ctx.data` : il lit la colonne **sans perte** des briques, jamais l'image de la carte graphique, qui peut être une colonne avec perte (section 7.11).

FONCTION	RÔLE
`screenToVoxel(x, y)`	Le point sous le curseur : `{ um, voxel, onBoundingBox }` (µm et voxel natif)
`toVoxel(um, { level })`, `toUm(voxel, { level })`	Passer des micromètres aux voxels d'un niveau, et retour
`locate(voxel, canal)`	Quel fichier, et quels octets, contiennent ce voxel sans perte
`readRegion(boîte, canaux, { level })`	Les voxels exacts d'une boîte (décodés dans un worker ; refusé au-delà de 512 Mio)
`valueAt(voxel)`	La valeur exacte de chaque canal en un voxel

🔢 **MICRO-EXEMPLE**

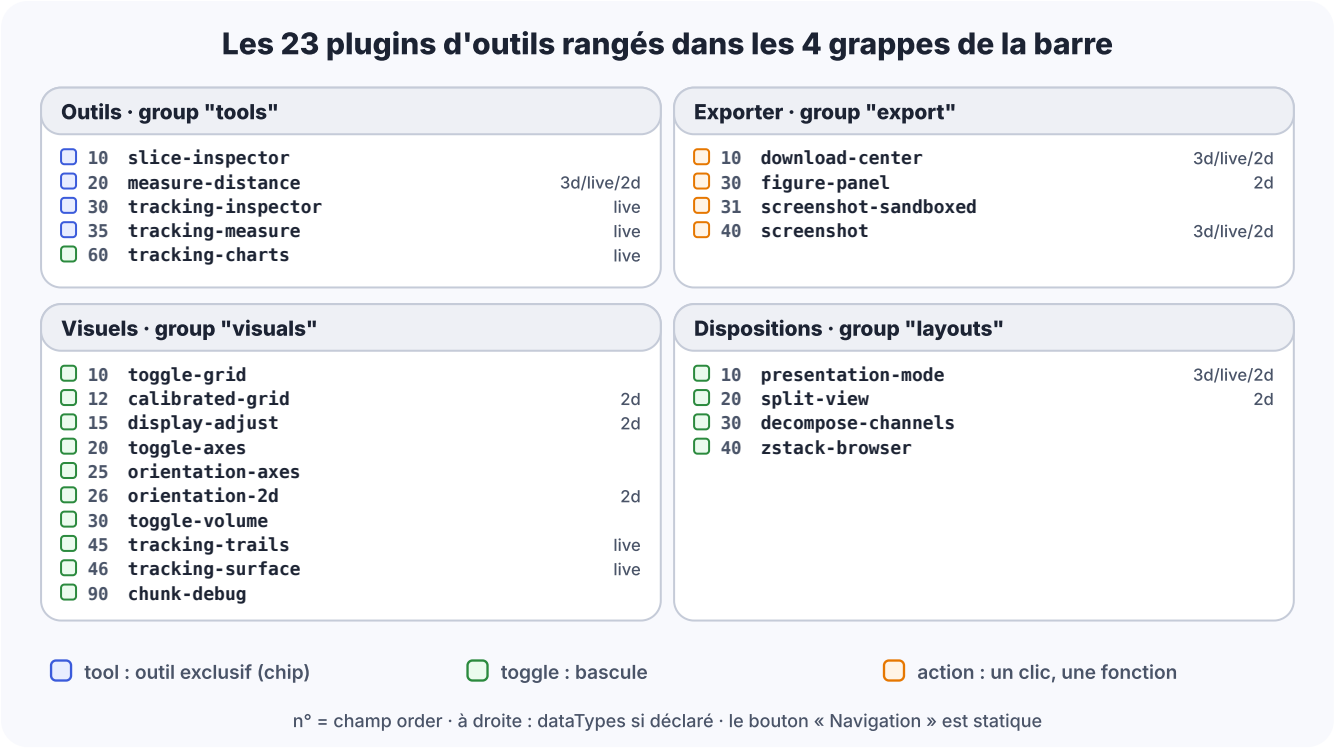
Un plugin qui affiche « intensité au point cliqué » appelle `ctx.data.screenToVoxel(x, y)`, puis `ctx.data.valueAt(voxel)`. Même si le visiteur regarde la case 256 · Très légère, la valeur rendue est celle du voxel natif, exacte. Fonctionne sur les formats 2 à 5.

Les textes d'un plugin

Un plugin apporte ses traductions dans `lang/<code>.json`. Elles sont greffées dans l'arbre de traduction de la plateforme sous `plugins.<id>`. Si une langue manque, c'est **l'anglais** qui s'affiche : un plugin livré en anglais seul reste utilisable partout.

15.8 La barre d'outils est fabriquée, pas écrite

Le fichier `viewer.html` ne contient **aucun bouton de plugin**. Il contient quatre **grappes** vides : **Outils**, **Exporter**, **Visuels**, **Dispositions**, plus un bouton statique « Navigation ». `buildToolbarButtons` crée ensuite un bouton par plugin d'outils, d'après son `plugin.json`.



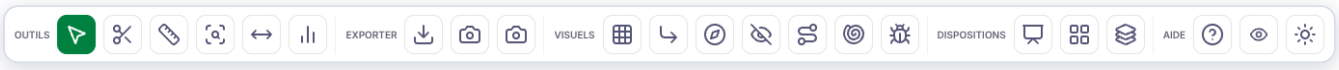
▲ Les 23 plugins d'outils, rangés d'après leur champ group, triés d'après order.

Pour chaque plugin, le constructeur :

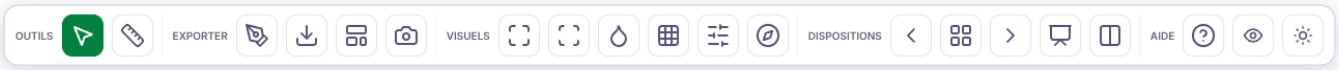
1. choisit la **grappe** d'après `group` (un groupe sans grappe sur la page : avertissement, bouton ignoré) ;
2. crée un bouton **outil exclusif** (`subtype: "tool"` , relié par `data-tool`) ou un bouton d'**action / bascule** (relié par `data-plugin-id`) ;
3. pose le pictogramme `icon` , l'infobulle `i18nTitle` (traduite et retraduite à chaque changement de langue) et le texte d'accessibilité ;
4. **cache** le bouton si `requires` n'est pas satisfait (`tracking` : le dataset a un bloc de suivi ; `bricks` / `webstack` : la source existe) ;
5. enregistre le raccourci `shortcut` auprès de `ToolManager` .

Résultat : **la barre change d'un dataset à l'autre**, sans une ligne de code de page.

▲ Barre d'outils d'un volume 3D (jeu de démonstration).



▲ Barre d'un timelapse suivi : trois outils de plus (inspecteur, distance, graphiques) et deux bascules de suivi (jeu de démonstration).



▲ Barre d'une photographie 2D : les cinq plugins 2D remplacent les outils de volume (jeu de démonstration).

> 👉 **ASTUCE**
>
> **Votre barre est différente de celle du manuel ?** C'est normal : l'administrateur a installé d'autres plugins (chapitre 14) ou en a désactivé. Un bouton absent n'est jamais une panne du viewer.

15.9 Plugins et espace de travail, exports, pages embarquées

Espace de travail

`getState()` de chaque plugin est rassemblé dans le **même objet** que la caméra et les canaux : c'est ce que gardent **Sauvegarder** et **Restaurer** du Centre de téléchargement de la page Comparer, et ce que contient l'adresse `#state=…`. Un plugin sans `getState` n'est pas affecté. Un plugin encore « registered » (pas encore initialisé) est sauté.

Exports

`getExports()` renvoie une liste d'entrées `{ action, handler, … }` que le Centre de téléchargement ajoute à ses propres exports. `getGraph()` renvoie un graphique à l'écran (Plotly) que le Centre sait exporter. Le registre **collecte** ces réponses en ignorant tout plugin qui lève une exception : **un export défectueux ne fait jamais perdre les exports de la plateforme**.

Pages embarquées (Comparer, vue divisée, aperçu)

Une page embarquée n'a pas de barre d'outils visible. Elle **décrit** donc ce qu'elle offre : `describeToolbar()` liste les outils exclusifs et les bascules (avec leur état courant) des seuls plugins qui acceptent le contexte `panel`. L'hôte (la page Comparer) **dessine ses propres boutons** à partir de cette description, puis envoie `PLUGIN_ACTIVATE` ou `SET_TOOL` ; l'état revient par `syncToolbarButton`. Le détail du protocole est au chapitre 13.

15.10 Les 28 plugins du catalogue

Modes de rendu (placement `shaders`, chargés partout).

PLUGIN	EN UNE LIGNE
Fluorescence	Rendu par défaut : chaque canal émet sa couleur (mode 1)
Natural Fluorescence	Émission-absorption : les structures denses masquent ce qui est derrière (mode 2)
Structure (DVR)	Rendu volumique avec profondeur et occlusion (mode 0)

Canaux (placement `channels`).

PLUGIN	EN UNE LIGNE
Histogram Controls	Histogramme d'intensité par canal et curseurs min / max / gamma
Gaussian Filter	Flou gaussien 2D dans le plan, par canal

Outils de volume (`3d` et `live`).

PLUGIN	CONTEXTE	EN UNE LIGNE
Slice through Volume	page + panneau	Coupe plane orientable ; la coupe prend le canvas
Z-Stack Browser	page + panneau	Parcourir les coupes (exclusif avec la coupe oblique)
Measure Distance	page + panneau	Deux points → distance en µm
Decompose by Channel	page	Un canal par panneau
Orientation Axes	page + panneau	Gizmo d'orientation et vue par défaut
Toggle Grid · Toggle Axes · Hide/Show 3D Volume	page + panneau	Grille, axes, volume
Chunk Debug	page	Frontières des briques (exige `bricks`)

Outils communs à tous les types.

PLUGIN	CONTEXTE	EN UNE LIGNE
Download Center	page	Fichiers du dossier `download/` du jeu, et au-dessus les exports d'analyse de la page et de ses plugins (mesures, tableaux du suivi cellulaire, graphique affiché)
Screenshot	page	Capture PNG de la vue
Screenshot (sandboxed)	page	La même, **en bac à sable** (plugin de référence)
Presentation Mode	page	Plein écran sans interface

Outils de photographie 2D (`2d`).

PLUGIN	CONTEXTE	EN UNE LIGNE
Calibrated Grid	page + panneau	Grille physique 1-2-5 en µm ou mm
Display Adjustments	page + panneau	Luminosité, contraste, gamma, balance, aplatissement
Orientation 2D	page + panneau	Rotation et miroir
Split View	page	Deuxième photographie côte à côte
Figure Panel Builder	page	Composer une planche de plusieurs photographies

Outils de suivi cellulaire (`live` , exigent `tracking`).

PLUGIN	CONTEXTE	EN UNE LIGNE
Tracking Trails	page + panneau	Trajectoires sur le volume
Tracking Surface	page + panneau	Surface de l'échantillon (`model.glb`)
Cell Inspector	page + panneau	Métriques, lignée, voisins, champ de vitesse
Tracking Charts	page	Graphiques de population, vitesse, mitoses
Cell Distance	page	Distance entre deux cellules

15.11 Écrire un plugin

1. Créez `js/modules/tools/mon-plugin/` (le nom du dossier est l'`id`).
2. Écrivez `plugin.json` (au moins `id` , `placement` , `group` , `subtype` , `icon` , `order`).
3. Écrivez `index.js` avec `PluginRegistry.implement('mon-plugin', { … })` .
4. Ajoutez `lang/en.json` (obligatoire comme repli), puis les autres langues.
5. Rechargez la page : le bouton apparaît. Sur un site de production, il porte d'abord l'étiquette **non fiable** : approuvez-le (§15.12).

Exemple complet : un bouton qui affiche le nom du dataset.

```
{ "id": "hello-dataset", "name": "Hello Dataset", "version": "1.0.0",
  "platformCompat": ">=1.53.0", "placement": "tools",
  "group": "export", "subtype": "action", "icon": "info", "order": 90,
  "dataTypes": ["3d", "live"], "contexts": ["page"], "creator": "Mon labo",
  "description": "Affiche le nom du dataset ouvert.",
  "i18nTitle": "title", "i18nLanguages": ["en", "fr"] }
```

```
PluginRegistry.implement('hello-dataset', {
  init(ctx) { this._ctx = ctx; return this; },
  activate() {
    const meta = this._ctx.dataset.getMeta();
    this._ctx.ui.toast(this._ctx.i18n.t('hello') + ' ' + meta.name);
    return { active: false };                // une action : pas d'état
  },
  dispose() { this._ctx = null; }
});
```

Avec `lang/en.json` : `{ "title": "Say hello", "hello": "Dataset:" }` .

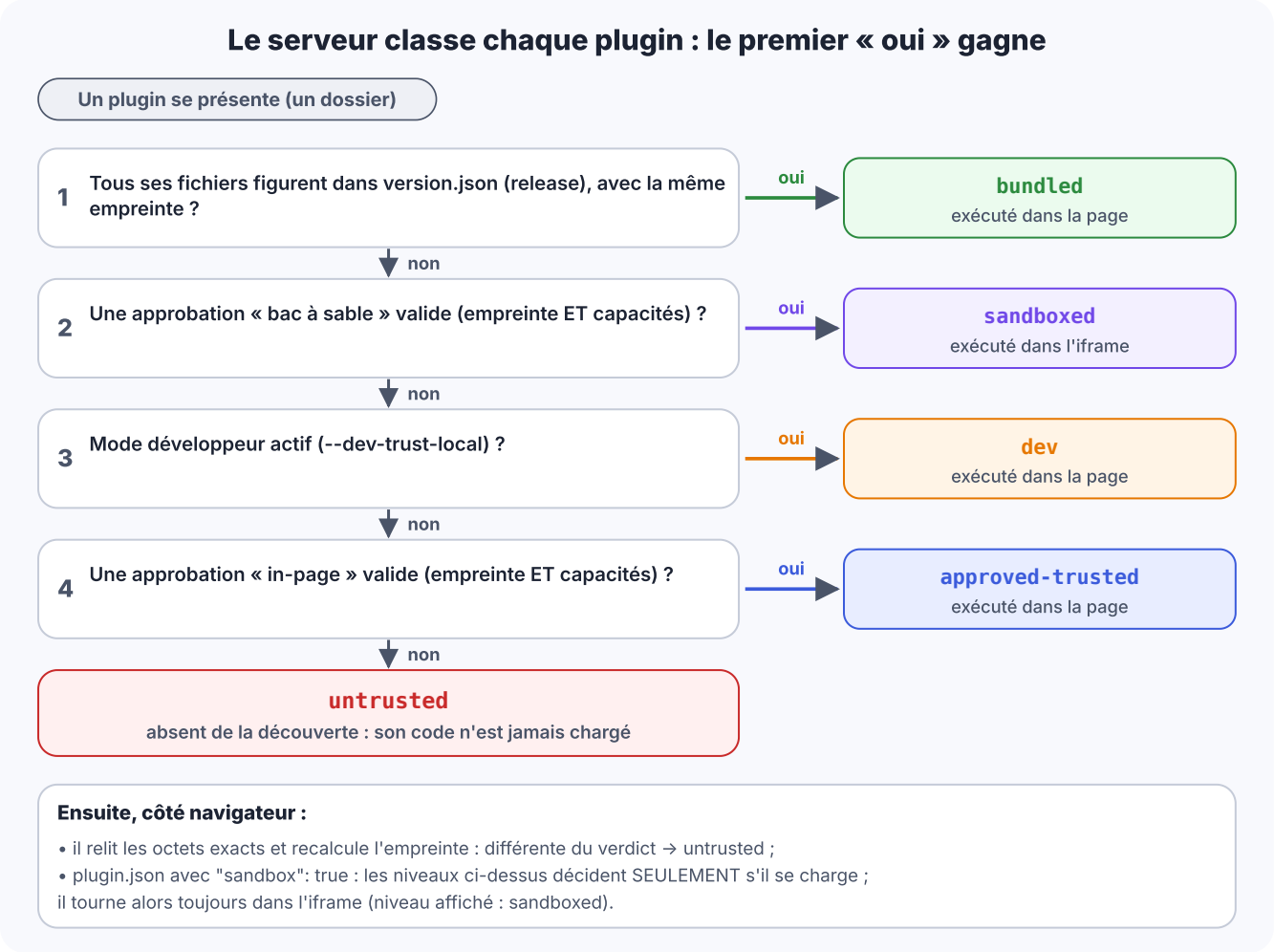
📖 **POUR ALLER PLUS LOIN**

Le guide complet pour les auteurs (contrats des canaux et des modes de rendu, exemples d'outils exclusifs, bonnes pratiques) est `DOCS/plugins/guide-creation-plugins.pdf` (source `.tex` dans le même dossier). Les tests des règles de ce chapitre : `tests/js/test_plugin_context_gate.mjs`, `test_plugin_datatype_gate.mjs`, `test_plugin_autonomy.mjs`, `test_plugin_lang.mjs`.

15.12 Pourquoi se méfier d'un plugin

Un plugin est un fichier JavaScript : **du vrai code qui s'exécute dans le navigateur de vos visiteurs et de l'administrateur**. Avant la version 1.6.0, un plugin avait les **pleins pouvoirs de la page** : accès à toute l'interface, aux cookies, et même aux appels réservés à l'administration. Un plugin malveillant aurait pu prendre le contrôle du site.

Depuis, la règle est **inversée** : *un plugin n'a pas le droit de s'exécuter tant qu'on ne l'a pas reconnu.*



▲ Le serveur applique ces quatre questions dans l'ordre ; la première réponse « oui » donne le niveau de confiance.

Les cinq niveaux, du plus au moins confiant :

NIVEAU	COMMENT ON L'OBTIENT	OÙ IL TOURNE
`bundled` (« intégré »)	Chaque fichier du dossier figure, avec **la même empreinte**, dans le `version.json` d'une release	Dans la page
`sandboxed` (« sandbox »)	L'opérateur l'a approuvé en mode **bac à sable** (ou le plugin déclare `sandbox: true`)	Dans une **iframe** isolée
`dev`	Le serveur a été lancé avec `--dev-trust-local`	Dans la page

NIVEAU	COMMENT ON L'OBTIENT	OÙ IL TOURNE
`approved-trusted` **(« approuvé »)**	L'opérateur l'a approuvé en mode **in-page**	Dans la page
`untrusted` **(« non fiable »)**	**Tout le reste**	**Nulle part**

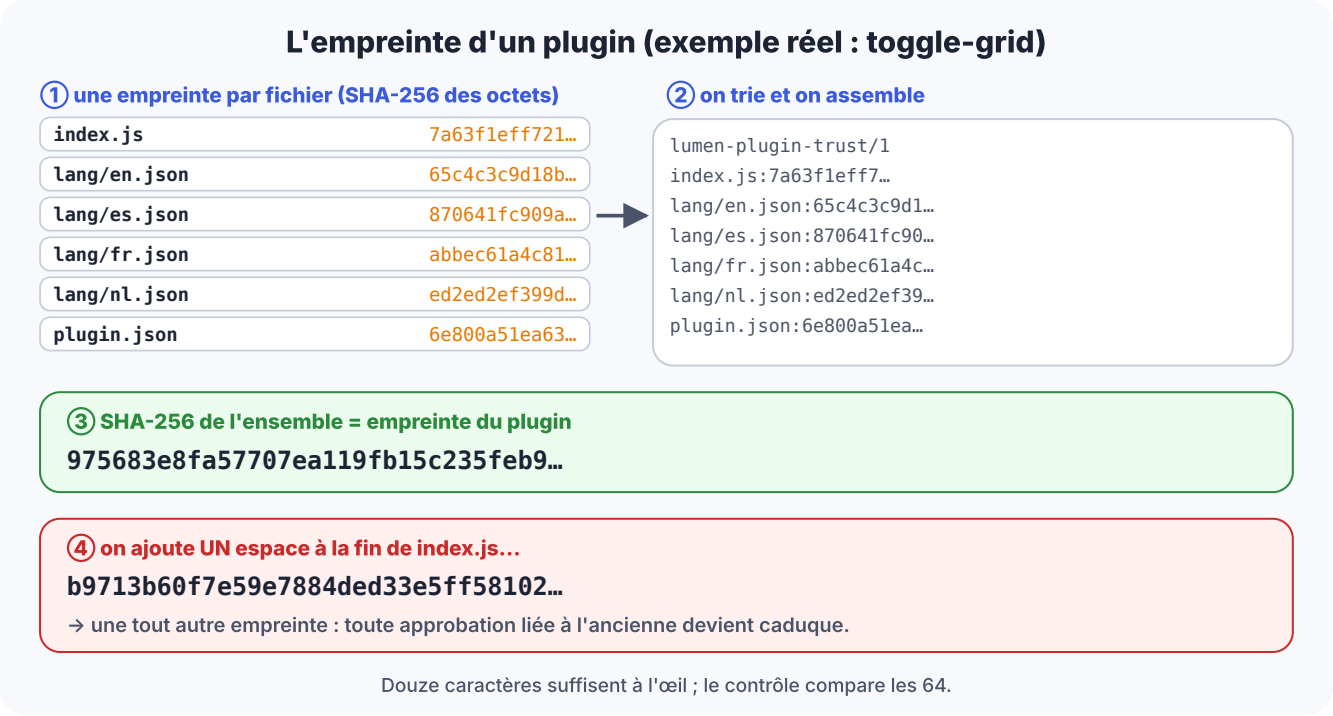
Trois précisions sur l'arbre :

- La confiance dépend du **contenu** des fichiers, jamais de leur chemin ni de l'absence d'un fichier. Un plugin qui porterait le même nom qu'un plugin officiel mais un contenu différent n'est **pas** `bundled`.
- Une approbation « bac à sable » **gagne** sur le mode développeur : si l'opérateur a choisi d'enfermer un plugin, une machine de développement ne le libère pas.
- Le niveau `dev` est un **signal positif** de l'opérateur (un drapeau). La présence d'un dossier `.git` n'en est pas un : une installation de production derrière un relais lui ressemble exactement. Sans le drapeau, un clone Git n'a **aucun** plugin local de confiance.

> 📌 **À RETENIR**
>
> **Le serveur est l'autorité.** Il classe, le navigateur **revérifie**. Le plugin, lui, n'a aucune voix dans l'affaire.

15.13 L'empreinte : une signature du contenu exact

Pour reconnaître un plugin, le système calcule son **empreinte** (hash) : un nombre de 64 chiffres hexadécimaux, propre à son contenu.



▲ Du contenu aux 64 caractères, et pourquoi un seul espace change tout (exemple réel).

💡 UNE IMAGE POUR COMPRENDRE

Un sceau de cire numérique. Changez une virgule de la lettre : le sceau ne correspond plus. Impossible de fabriquer deux plugins différents qui donnent la même empreinte.

Les règles exactes (identiques dans les trois implémentations : JavaScript, Python, PHP) :

1. Chaque fichier d'extension `.js`, `.json`, `.mjs`, `.css` ou `.html` du dossier reçoit son SHA-256, calculé sur les **octets bruts** tels que servis (aucune conversion de fin de ligne ni de BOM). Les fichiers cachés (point initial) sont ignorés ; les images ne comptent pas.
2. On forme les lignes `chemin:empreinte`, **triées** par chemin.
3. L'empreinte du plugin est le SHA-256 de `lumen-plugin-trust/1` + saut de ligne + ces lignes.

Les **traductions** (`lang/*.json`) comptent : modifier une phrase change l'empreinte. Un jeu de vecteurs de test (`tests/plugin-trust-vector.json`) vérifie que les trois implémentations donnent exactement le même résultat, y compris sur les cas piégeux (fins de ligne Windows, BOM).

Anti-TOCTOU : ce qui est vérifié est ce qui s'exécute

❓ POURQUOI CE CHOIX ?

TOCTOU signifie « time of check, time of use » : on vérifie un fichier, **puis** on l'exécute, et dans l'intervalle il a changé. C'est la ruse classique : présenter un fichier propre au contrôleur, un fichier piégé à l'exécution.

Lumen3D la ferme ainsi :

1. Le navigateur télécharge les octets de chaque fichier **une seule fois**, sans cache pour tout ce qui peut s'exécuter.
2. Il recalcule l'empreinte sur **ces** octets et la compare au verdict du serveur. Différente : `untrusted`, le plugin n'est pas exécuté.
3. Il exécute **ces mêmes octets**, via une adresse temporaire `Blob`, jamais en redemandant le fichier par son URL.
4. Le `plugin.json` qui a façonné la barre et les barrières doit être le document qui a été haché, sinon quarantaine.

🔧 POUR LES CURIEUX

Trois cas particuliers, par honnêteté. (1) Sur une adresse **non sécurisée** (HTTP sur une IP de réseau local), le navigateur ne fournit pas `crypto.subtle` : il ne peut pas recalculer l'empreinte et se fie alors au verdict du serveur, avec un avertissement en console. (2) Sur un hébergement **statique** sans autorité de confiance, seuls les plugins `bundled` sont reconnus, par comparaison avec `version.json`. (3) Sans module de confiance chargé, rien n'est exécuté (échec fermé).

15.14 L'approbation de l'opérateur, et le bac à sable

Approuver : lier un OUI à un contenu précis

Quand un plugin est déposé à la main (par FTP, par exemple), il est `untrusted`. L'administrateur l'approuve dans l'onglet **Plugins** (chapitre 14) : **Approuver (bac à sable)** ou **Approuver (in-page)**.

L'approbation est **épinglée** :

- le serveur **recalcule lui-même** l'empreinte sur le disque et exige qu'elle égale celle que l'opérateur a vue (sinon « le contenu du plugin a changé ») ;
- l'opérateur **retape son mot de passe** : approuver est la seule action qui laisse du code étranger s'exécuter ;
- elle enregistre aussi les **capacités** accordées (§ suivant) ;
- elle est écrite dans `api/plugin-trust.json` : `{ chemin, sha256, mode, caps, date, auteur }`.

Si le contenu **ou** les capacités demandées changent, l'approbation devient **caduque** (« approbation annulée : contenu ou capacités modifiés ») et le plugin redevient `untrusted`. Le message n'est pas une panne : c'est le système qui fait son travail.

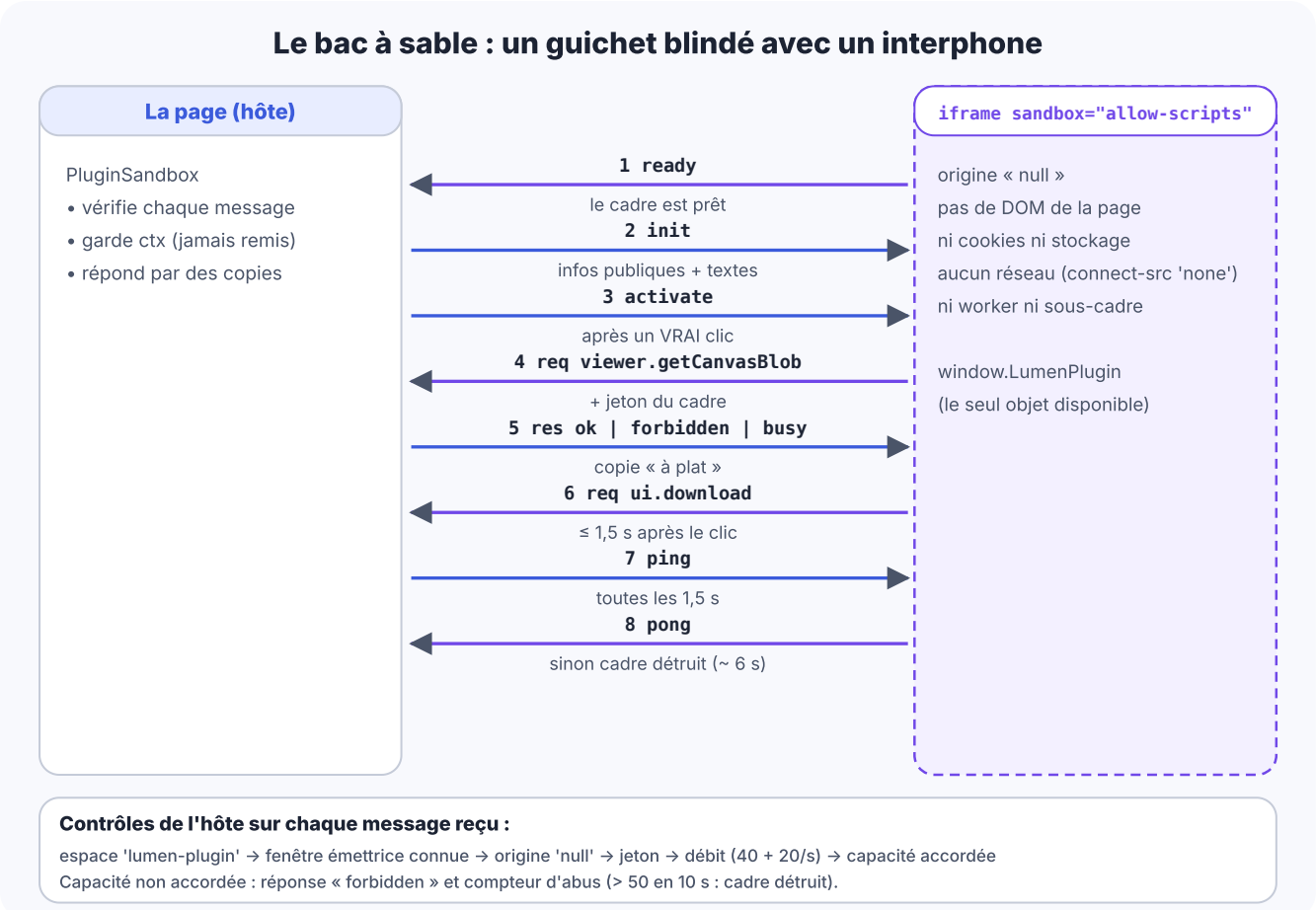
> 🔧 **POUR LES CURIEUX**
>
> Le fichier `api/plugin-trust.json` n'est **jamais servi** en HTTP, il est protégé contre les mises à jour, et une release qui en contiendrait un est **refusée** : une version piégée ne peut donc pas pré-approuver un plugin d'attaque. Chaque approbation, révocation, installation ou désinstallation incrémente un compteur, `trustEpoch`.

Révocation à chaud

Un viewer **déjà ouvert** interroge le serveur (`api/health`, toutes les 8 secondes et au retour sur l'onglet). Si `trustEpoch` a changé, il redemande la liste des plugins reconnus ; tout plugin en bac à sable qui n'y figure plus est **détruit sur-le-champ** et son bouton disparaît.

Pour un plugin « in-page », on ne peut pas défaire du code déjà exécuté : la révocation prend effet **au prochain chargement**. Et sur un hébergement PHP, qui n'émet pas `trustEpoch`, la révocation vaut aussi au prochain chargement seulement.

Le bac à sable : un guichet blindé avec un interphone



▲ Une iframe sans rien, un interphone surveillé, huit messages types.

💡 UNE IMAGE POUR COMPRENDRE

Un guichet blindé. Le plugin travaille dans une cabine fermée. Il ne voit pas la salle (le DOM), n'a pas de téléphone (aucun réseau). Il dispose d'un interphone : il pose une question, le guichetier (la page) vérifie qu'il en a le droit, puis lui répond avec **une copie** de l'information, jamais l'original.

La cabine : une `<iframe sandbox="allow-scripts">` **sans** `allow-same-origin`. Le navigateur lui donne une origine « null » : pas d'accès au DOM de la page, aux cookies, au stockage, ni aux appels d'administration. Une politique interne (`default-src 'none'`, `connect-src 'none'`) interdit tout réseau, tout worker, tout sous-cadre. Un test de l'intégration continue refuse toute modification qui ajouterait `allow-same-origin`.

L'interphone : `postMessage` dans un espace de noms `lumen-plugin`, jamais confondu avec les messages entre pages. Chaque message porte `{ ns, v, dir, id, plugin, token, type, payload }`.

Le seul objet du plugin : `window.LumenPlugin`.

APPEL DU PLUGIN	CAPACITÉ REQUISE	EFFET
`addButton({label, icon})`	`toolbar.addButton`	Déclare son bouton
`toast(texte)`	`ui.toast`	Message (200 caractères au plus)

APPEL DU PLUGIN	CAPACITÉ REQUISE	EFFET
`getInfo()`	`viewer.getInfo`	Nom, dimensions, voxel, nombre de canaux
`getCanvasBlob(opts)`	`viewer.getCanvasBlob`	Image PNG ou JPEG de la vue (une requête à la fois)
`download(nom, type, données)`	`ui.download`	Téléchargement, **sur vrai clic seulement**
`setRenderMode(mode)`	`viewer.setRenderMode`	Changer de mode de rendu (modes connus seulement)
`getChannels()`	`channels.getState`	Copie de l'état des canaux
`on('render' · 'channels-updated' · 'camera', f)`	`events.subscribe`	Recevoir des événements, sous forme de **copies à plat**
`saveState(s)`, `t('clé')`	(aucune)	Mémoire pour l'espace de travail (256 Kio au plus), traduction

Le plugin déclare dans `sandboxCapabilities` ce dont il a besoin ; l'opérateur en accorde un sous-ensemble ; les capacités **effectives** sont l'**intersection** de trois listes : ce que le disque déclare, ce que l'opérateur a approuvé, et la liste fermée des huit capacités du tableau.

Les contrôles à chaque message reçu, dans cet ordre :

1. l'espace de noms est `lumen-plugin` ;
2. la **fenêtre émettrice** est celle d'une cabine connue (c'est l'authentification principale : on se fie à l'identité de la fenêtre, pas à l'origine déclarée) ;
3. l'origine est bien « null » ;
4. la forme du message et le **jeton** aléatoire du cadre sont corrects ;
5. le **débit** est respecté : réserve de 40 messages, recharge de 20 par seconde ;
6. la capacité demandée a été **accordée** ; sinon réponse `forbidden`.

Les garde-fous de durée de vie :

GARDE-FOU	VALEUR
Démarrage de la cabine (`init`)	10 s au plus, sinon destruction
Battement de cœur (`ping`)	toutes les 1,5 s ; cabine détruite après environ 6 s sans réponse
Abus (messages invalides ou interdits)	plus de 50 en 10 s : destruction
Téléchargement	un par activation, dans les 1,5 s d'un vrai clic, 32 Mio au plus, types PNG / JPEG / JSON / CSV / texte, extension imposée par le type

⚠️ **ATTENTION**

Seuls les outils « action » et « bascule » peuvent être enfermés. Un mode de rendu compile du code GLSL sur la carte graphique à chaque image : il n'y a aucune frontière asynchrone derrière laquelle une cabine pourrait se placer. Un plugin de canal reçoit directement un morceau du DOM de la page, exactement le privilège que le bac à sable retire. Ces deux placements restent donc **en confiance totale** (réservés aux plugins officiels), et approuver l'un d'eux en bac à sable le met en quarantaine.

ℹ️ **À SAVOIR**

Un plugin déclaré `sandbox: true` **tourne toujours dans la cabine**, quel que soit son niveau de confiance. Inversement, un plugin écrit pour la page (qui appelle `PluginRegistry`) ne peut pas fonctionner dans la cabine, qui n'a que le SDK. Le plugin de référence est `screenshot-sandboxed` : il capture et télécharge sans jamais toucher au DOM ni au contexte.

15.15 La compatibilité : `platformCompat`

Un plugin dit avec quelles versions de la plateforme il fonctionne. Si la plateforme change de version, ce champ évite qu'un plugin trop ancien (ou trop récent) casse l'interface.

DÉCLARATION	SENS	PLATEFORME 1.59.2
absent	Pas de contrainte	compatible
`"*"` ou `"x"`	N'importe quelle version	compatible
`">=1.53.0"`	Au moins cette version	compatible
`">=1.60.0"`		**refusé**
`"^1.4.0"`	De 1.4.0 inclus à 2.0.0 exclu	compatible
`"~1.58.0"`	De 1.58.0 inclus à 1.59.0 exclu	**refusé**
`"1.x"` ou `"1.59"`	Préfixe	compatible
`["1.58", "1.59.x"]`	Liste : **l'une** des valeurs suffit	compatible
`">=1.51.0 <1.59.0"`	Plusieurs comparateurs : **tous** doivent tenir	**refusé**
`">=banane"`, `42`	Illisible	**refusé**

Règles : une chaîne séparée par des espaces est un **ET** ; un tableau est un **OU** (valeurs « nues » seulement, sans opérateur) ; trois chiffres = version exacte, moins de chiffres = préfixe ; un suffixe `-rc1` est ignoré.

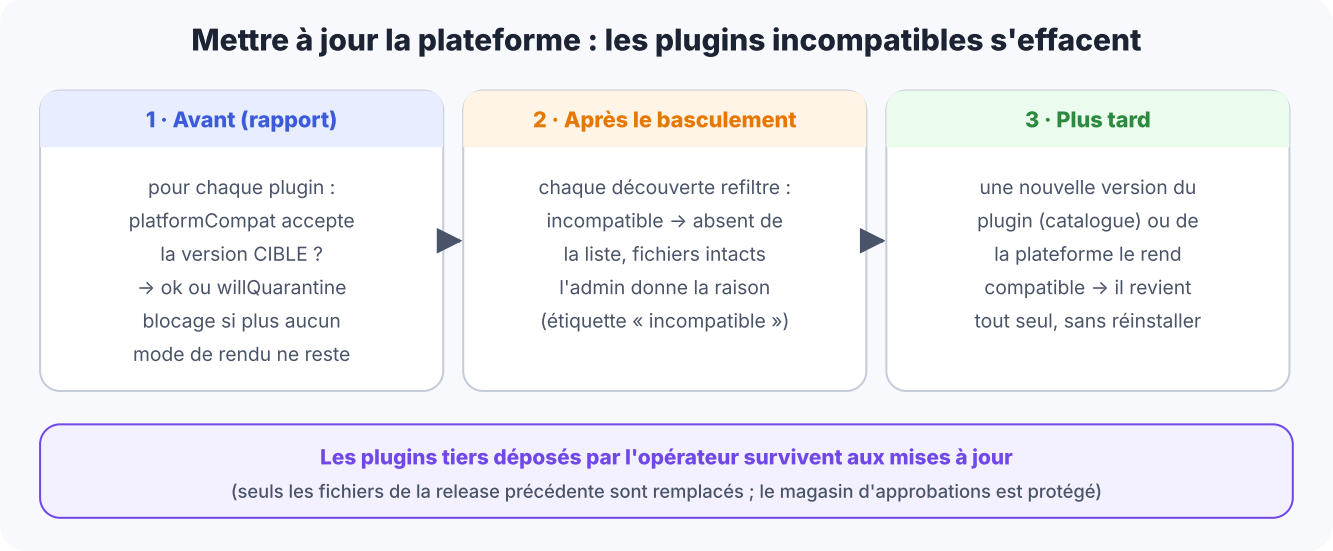
📌 **À RETENIR**

Illisible = incompatible (échec fermé). Une déclaration qu'on ne sait pas lire ne laisse jamais passer. La **seule** exception : si la version de la plateforme est inconnue (un clone sans `version.json` ni serveur de développement), la barrière est neutralisée et le dit dans sa raison.

🔧 **POUR LES CURIEUX**

Trois jumeaux, un seul comportement : `js/core/compat.js` (navigateur), `dev_server.py` (`_compat_satisfies`), `api/_admin_lib.php` (`admin_compat_satisfies`). Ils sont tous validés contre `tests/compat-vector.json` (44 cas) ; toute modification de sémantique doit passer les trois. La version de la plateforme est lue dans `version.json` (releases), sinon dans `api/health` (serveur de développement).

15.16 Mettre à jour la plateforme : l'effacement réversible



▲ Avant, après, plus tard : un plugin incompatible n'est jamais supprimé.

Quand l'administrateur prépare une mise à jour (chapitre 18), le **rapport de vérification** applique `platformCompat` à la version **cible**, plugin par plugin, **avant** toute installation :

- **compatibles** : rien à faire ;
- **mis en quarantaine** (`willQuarantine`) : ils ne fonctionneraient plus ;
- **blocage** : si plus **aucun mode de rendu** ne resterait compatible, le viewer serait inutilisable : la mise à jour ne peut pas être confirmée.

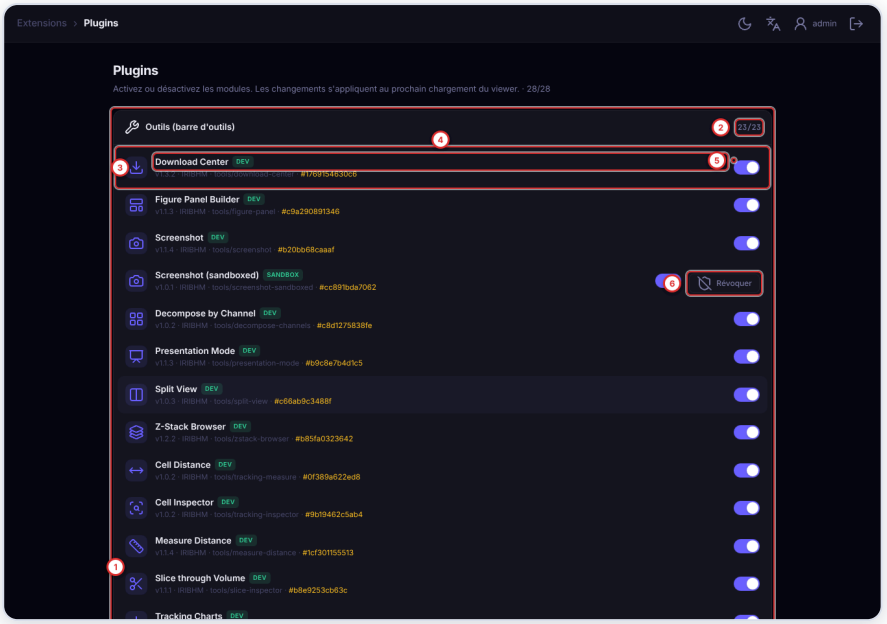
Après le basculement, rien n'est supprimé. La découverte **refiltre à chaque requête** : un plugin incompatible disparaît de la liste (ses fichiers restent), et **revient tout seul** dès qu'une version compatible du plugin ou de la plateforme est installée. Un plugin déposé à la main par l'opérateur survit à la mise à jour : seuls les fichiers de la **release précédente** sont remplacés.

ℹ️ **À SAVOIR**

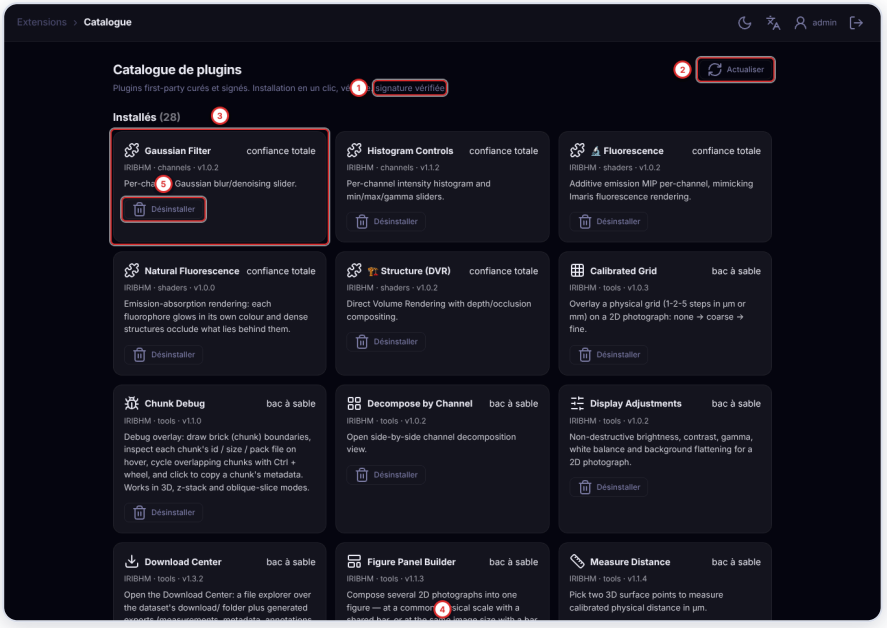
Les plugins officiels ne **voyagent pas** avec la plateforme : une installation neuve démarre sans plugin et les installe depuis le catalogue (§15.17). La compatibilité est ce qui fait que cette séparation reste sûre.

15.17 Le catalogue signé : la boutique officielle

Le **catalogue** est la liste des plugins que la plateforme propose d'installer, depuis l'onglet **Catalogue** ou depuis l'assistant de première installation (les plugins « recommandés » y sont pré-cochés). Il est **curé** : seul l'éditeur de la plateforme y publie.



▲ L'onglet Plugins de l'administration : trois cartes (outils, canaux, modes de rendu), un interrupteur et un niveau de confiance par plugin (« dev » sur cette machine de démonstration).



▲ L'onglet Catalogue : « signature vérifiée » en haut, une carte par plugin avec son mode (« confiance totale » ou « bac à sable »).

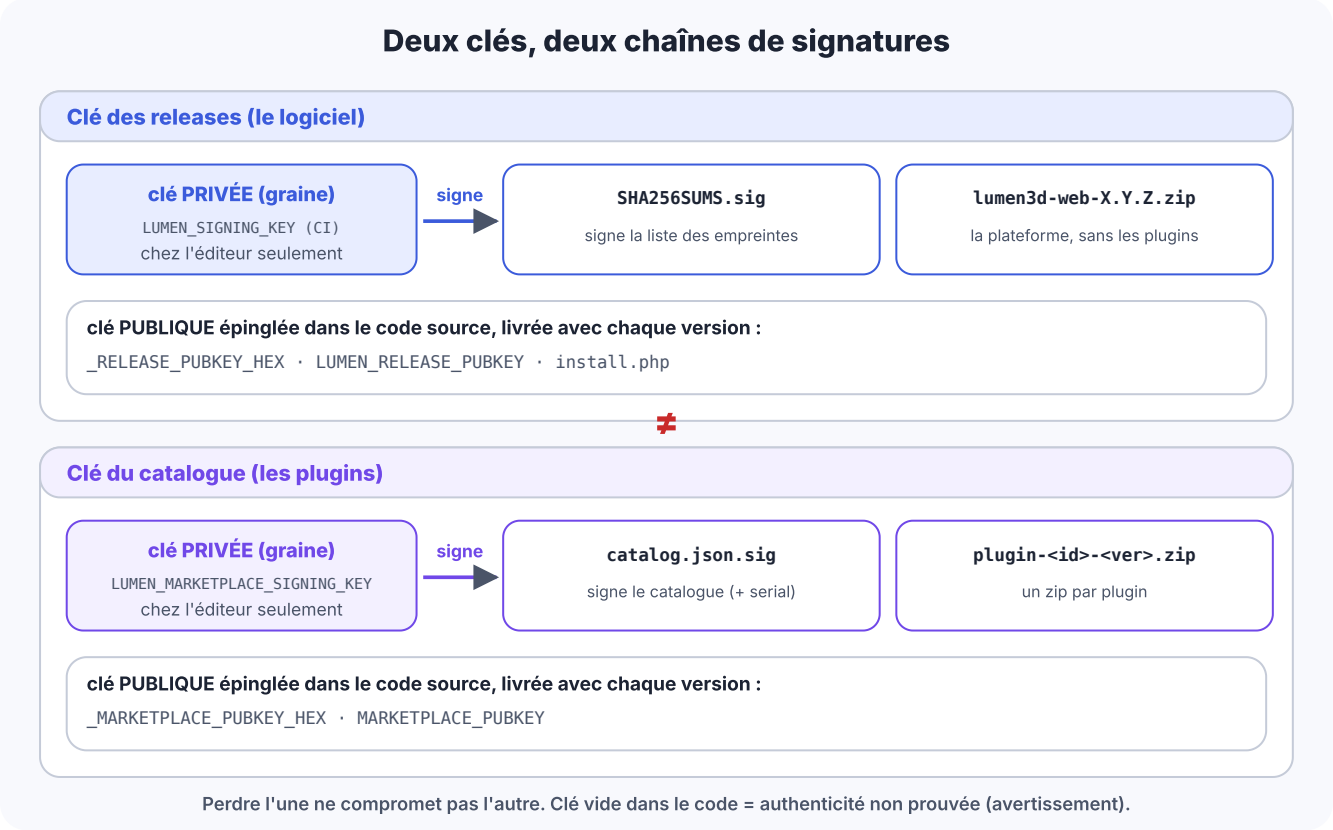
Ce que contient le catalogue

Un fichier `marketplace-catalog.json`, signé par `marketplace-catalog.json.sig`. Il porte `version`, un **numéro de série** (`serial`), la date d'émission (`issuedAt`) et une entrée par plugin :

CHAMP D'UNE ENTRÉE	RÔLE
`id`, `name`, `placement`, `subtype`, `description`, `creator`	La carte affichée
`latestVersion`, `platformCompat`	Dernière version, compatibilité
`sandboxCapabilities`, `recommended`	Capacités demandées ; pré-coché ou non à la première installation
`assetUrl`, `sumsUrl`, `sigUrl`	Où trouver le zip, ses empreintes et leur signature
`sha256`	L'empreinte **du zip** (authentifiée par la signature du catalogue)

Chaque plugin est publié sous forme d'un zip **déterministe** (entrées triées, date fixée à 1980, droits 0644 : le même contenu donne toujours les mêmes octets) accompagné de `SHA256SUMS`, `SHA256SUMS.sig` et `version.json`.

Deux clés, jamais la même



▲ Deux chaînes de signatures indépendantes : un compromis de l'une ne donne rien sur l'autre.

💡 **UNE IMAGE POUR COMPRENDRE**

Deux sceaux de cire différents. L'un authentifie les lettres officielles du logiciel, l'autre celles de la boutique. Dérober ou perdre l'un ne permet pas de contrefaire l'autre, et on peut en changer un sans toucher à l'autre.

- La signature est une signature **Ed25519** (norme RFC 8032), vérifiée par un module écrit pour l'occasion, sans dépendance, en Python ; en PHP par la bibliothèque libsodium.
- La **clé privée** (la graine) ne quitte jamais l'éditeur : variable d'environnement ou fichier hors du dépôt Git.
- La **clé publique** est **épinglée dans le code source** (`dev_server.py` , `api/_admin_lib.php` , `install.php` pour les releases) : elle voyage avec chaque version et survit aux mises à jour. Un serveur ne fait confiance qu'à ce qui se vérifie avec cette clé précise.
- Clé **vide** dans le code : la signature n'est pas prouvable, seule l'empreinte est contrôlée, avec un avertissement. **Clé renseignée** : une signature manquante ou fausse **refuse** le catalogue.

Anti-retour-arrière : le numéro de série

Un catalogue périmé mais encore **validement signé** pourrait servir à réinstaller une version d'un plugin dont on a corrigé une faille. Chaque publication **augmente** donc le `serial` . Chaque serveur retient le plus haut numéro accepté (`api/marketplace-state.json`) et **refuse** un catalogue de numéro inférieur. Seul un catalogue dont la signature est prouvée peut faire monter ce compteur : un faux numéro gigantesque ne peut pas figer un serveur.

L'administrateur voit alors « Catalogue refusé : il est plus ancien… » : il n'y a rien à faire de son côté.

🔢 **MICRO-EXEMPLE**

Au moment de la rédaction, le catalogue du dépôt est au `serial` 2. Un serveur qui a accepté le 2 refusera un catalogue de serial 1, même si sa signature est parfaitement valide.

15.18 Installer, mettre à jour, désinstaller

▲ Dix contrôles ; à la moindre erreur, tout est remis comme avant.

L'installation est lancée **par l'opérateur** (mot de passe re-saisi), jamais automatiquement. Les étapes, dans l'ordre du code :

1. mot de passe vérifié ;
2. le catalogue est téléchargé (1 Mio au plus) et sa **signature** vérifiée ;
3. son **serial** est comparé au plus haut déjà vu ;
4. l'entrée est trouvée, `platformCompat` est comparée à la version de la plateforme (« incompatible » sinon) ;
5. le zip est téléchargé, **8 Mio au plus** ;
6. son SHA-256 doit égaler celui du **catalogue signé** ; sans clé, il est comparé à `SHA256SUMS` (lui-même vérifié) ;
7. **extraction durcie** : pas plus de 500 entrées ni 24 Mio, aucun chemin absolu, avec `..`, avec antislash ou lecteur ; seules les extensions de fichiers de plugin sont admises (jamais `.php`, `.py`, ni fichier caché : le dossier est servi au web) ;
8. `plugin.json` : l'`id` et le `placement` doivent égaler ceux du catalogue ;
9. le dossier est installé ; en cas de **mise à jour**, l'ancienne copie est **mise de côté**, pas supprimée ;
10. le serveur **recalcule l'empreinte** du dossier installé et **approuve** le plugin : `bac à sable` s'il déclare `sandbox: true` (ou s'il est un outil avec capacités), sinon `in-page`.

Si **n'importe quelle** étape échoue, le dossier `js/modules/` est remis **exactement** dans l'état initial, y compris l'ancienne version en cas de mise à jour.

Désinstaller supprime le dossier **et** l'approbation. L'opération est idempotente, et refuse de retirer le **dernier mode de rendu** actif.

> ⚠️ **ATTENTION**
>
> Un plugin « de confiance totale » installé depuis le catalogue **s'exécute dans la page**. Il est approuvé parce que **l'éditeur l'a signé et que le serveur a vérifié la signature**, pas parce qu'il serait inoffensif. C'est aussi pourquoi le catalogue n'est jamais ouvert à des tiers.

15.19 Publier un plugin (côté éditeur)

Une seule commande prépare et publie :

```
python tools/publish_plugin.py chemin/vers/mon-plugin --push
```

Elle empaquette et signe le plugin, ajoute son entrée au catalogue, **augmente le** `serial`, **re-signe** le catalogue puis, avec `--push`, ne committe que le dossier `marketplace/`. Un plugin n'est **en ligne** que lorsque ces fichiers sont sur la branche `main` (le catalogue est servi depuis GitHub, branche `main`). L'outil refuse une graine dont la clé publique n'est pas celle épinglée dans le code, car tous les serveurs rejetteraient le catalogue.

Autres options : `--recommended false` (publié mais non pré-coché), `--remove <id>` (dépublier), `--resign` (réémettre le catalogue sous le serial suivant).

Pour changer un plugin déjà publié : **augmenter** `version` (et relever `platformCompat` si le plugin utilise sans repli une fonction récente du noyau). Republier un contenu inchangé ne produit rien.

15.20 Les modules du noyau qui font fonctionner tout cela

Aucun de ces fichiers n'est un plugin : ils font **tourner** les plugins.

MODULE	RÔLE
`js/core/plugin-registry.js` (`PluginRegistry`)	Découverte, barrières, cycle de vie, barre d'outils, quarantaine, révocation à chaud, état de l'espace de travail
`js/core/plugin-trust.js` (`PluginTrust`)	Empreinte, relecture des octets, verdict (jumeau client du serveur)
`js/core/plugin-sandbox.js` (`PluginSandbox`)	Créer les cabines, courtier de capacités, débit, battement de cœur
`js/core/compat.js` (`Compat`)	Résolveur de `platformCompat` ; version de la plateforme
`js/core/tool-manager.js` (`ToolManager`)	**Un seul outil exclusif** actif à la fois ; raccourcis clavier ; défauts `v` /Échap (navigation), `c` (coupe), `m` (mesure)
`js/core/ui-actions.js` (`UiActions`)	Boutons d'en-tête par attribut `data-action` plutôt que par code dans le HTML (la politique de sécurité interdit le code en ligne)
`js/core/i18n.js` (`I18n`)	Greffe les dictionnaires de plugin sous `plugins.<id>` , repli sur l'anglais
`js/core/export-manager.js` , `workspace-state.js`	Consommateurs de `getExports()` et de `getState()`
`js/components/channel-panel.js`	Hôte des plugins de canal (`getChannelUI` , `bindChannelUI`)
`js/pages/viewer.js` , `js/pages/2d.js`	Fabriquent le `ctx` et appellent le registre dans l'ordre du §15.3

Côté serveur : `dev_server.py` (`_list_plugins` , `_classify_plugin` , `_approve_plugin` , `_marketplace_*` , `_install_marketplace_plugin`) et leurs jumeaux PHP dans `api/_admin_lib.php` et `api/plugins.php` . La **liste** des modules utilitaires non liés aux plugins (`Utils` , `Catalog` ...) est dans l'annexe du chapitre 20.

15.21 Un plugin ne s'affiche pas : que vérifier ?

1. Ouvrez la **console** du navigateur : chaque quarantaine y écrit `[PluginRegistry] Quarantined "tools/…" (raison): détail` .
2. Raison `untrusted` : onglet **Plugins** , approuvez-le (ou il a changé depuis son approbation).
3. Raison `incompatible` : comparez son `platformCompat` à votre version (§15.15).
4. Pas de message de quarantaine, juste « left out » : il ne vise pas cette page (`dataTypes` , `contexts` , §15.5).

5 Bouton absent mais plugin chargé : un `requires` non satisfait (pas de suivi, pas de briques), ou un `group` sans grappe.

6 Absent de l'onglet Plugins : le dossier est mal nommé, ou son `plugin.json` est illisible ou contredit le dossier parent.

📌 **À RETENIR**

- Un plugin = un dossier (`plugin.json` + `index.js` + `lang/`) ; le **dossier parent** fixe son placement.
- La page **découvre, filtre, vérifie, puis initialise**, toujours dans cet ordre ; un plugin en panne ne coûte jamais le viewer.
- **Refus par défaut** : un plugin n'est exécuté que s'il est `bundled` , approuvé, ou le mode développeur est actif.
- L'approbation est **liée au contenu exact** (empreinte) et aux capacités ; ce qui est vérifié est ce qui est exécuté.
- Le **bac à sable** enferme les outils d'action et de bascule ; les canaux et modes de rendu restent en confiance totale.
- Le **catalogue** est signé par **sa propre clé**, protégé contre le retour en arrière (serial), et installé **tout ou rien**.

16. Personnaliser et traduire

- Lumen3D se **repeint à vos couleurs sans une ligne de code** : nom, vocabulaire, thème, pages, mentions légales. Tout tient dans quelques petits fichiers JSON du dossier `config/`.
- Le site parle **quatre langues** (anglais, français, espagnol, néerlandais) avec **1 943 textes** par langue ; ajouter une cinquième langue, c'est déposer un fichier.
- Le site respecte aussi les visiteurs : **thème clair ou sombre** sans clignotement, **simulation de daltonisme**, usage sur téléphone, et des **statistiques sans donnée personnelle**.

📖 **POUR ALLER PLUS LOIN**

Ce chapitre explique la **mécanique** : où vit chaque réglage et comment il arrive à l'écran. Le pas-à-pas de chaque onglet est dans le « Guide de l'administrateur » (chapitres 6 à 11) ; le survol est au chapitre 14.

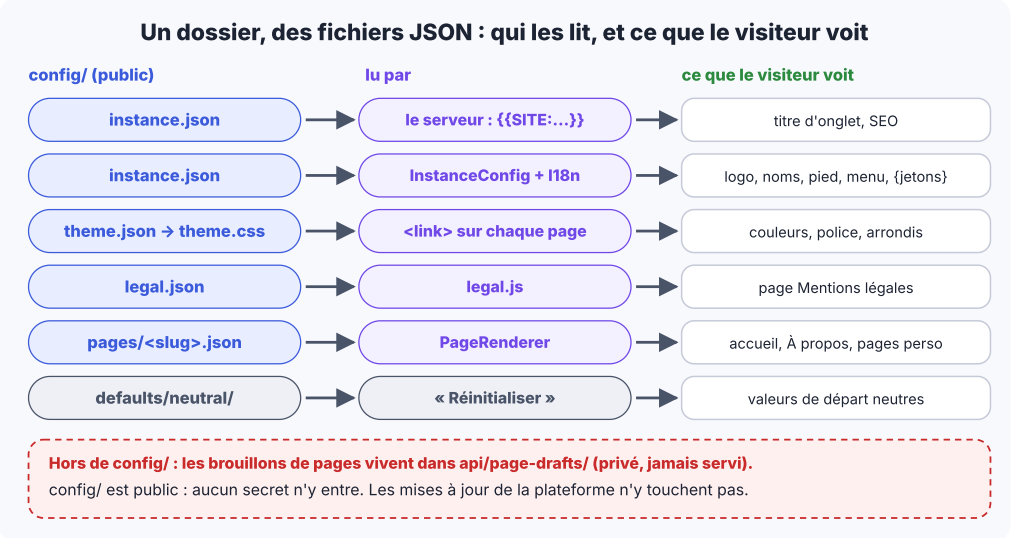
16.1 Une plateforme, plusieurs maisons

Lumen3D est né pour regarder des embryons de souris. Il sert aujourd'hui à d'autres objets : organes, échantillons, tissus. Le **moteur** ne change pas ; ce que les visiteurs lisent change.

💡 **UNE IMAGE POUR COMPRENDRE**

Un hôtel de chaîne. Les murs, l'électricité et l'ascenseur sont les mêmes partout (le moteur). La réception, l'enseigne, le menu du restaurant et la décoration portent le nom de l'établissement (la configuration).

Tout ce qui est propre à *votre* site se trouve dans le dossier `config/` :



▲ Le dossier config/ : chaque fichier, qui le lit, et ce que le visiteur en voit.

📌 **À RETENIR**

`config/` est **public** : n'importe quel visiteur peut lire ces fichiers. On n'y met donc **jamais de secret** (les mots de passe vivent dans `api/`, jamais servi).

Les fichiers, un par un

FICHIER	CE QU'IL CONTIENT	SE MODIFIE DANS L'ONGLET…
`instance.json`	nom, vocabulaire, SEO, pied de page, menu, noms des types, variables	**Identité**, **Types de données**, **Pages**
`theme.json`	les valeurs de couleur, police, arrondi que vous avez changées	**Apparence**
`theme.css`	la feuille de style **compilée** à partir de `theme.json`	(généré, jamais à la main)
`legal.json`	les sections des mentions légales, par langue	**Mentions légales**
`pages/<slug>.json`	une page par fichier : sa version publiée	**Pages**
`defaults/neutral/`	les valeurs de départ neutres, livrées avec la plateforme	bouton **Réinitialiser**
`uploads/`	les images envoyées depuis l'éditeur de pages	éditeur de pages

À côté, hors de `config/` : les **brouillons** de pages vivent dans `api/page-drafts/` (privé, voir 16.6).

🔧 **POUR LES CURIEUX**

Mise à jour et valeurs de départ. `instance.json`, `theme.json`, `theme.css`, `legal.json` et `pages/` figurent sur la liste de ce que la mise à jour de la plateforme ne touche jamais : vos réglages survivent. Les images envoyées ne font partie d'aucune version livrée, donc ne sont jamais remplacées. Les fichiers de `defaults/neutral/` voyagent, eux, avec chaque version.

Le plancher sans fichier. Si `instance.json` est introuvable ou illisible, le navigateur retombe sur un vocabulaire neutre intégré au code (« Lumen3D », « sample »). Le serveur, lui, laisse jouer les textes de repli écrits dans les pages. Le site ne casse pas.

Images. Elles sont limitées à 8 Mio, aux formats PNG, JPG, WebP, GIF et AVIF. Le **SVG est volontairement refusé** : servi depuis le même domaine, il pourrait contenir un script.

16.2 `instance.json`, champ par champ

C'est le fichier le plus riche. Voici ce que chaque bloc commande réellement.

Identité et organisation

CHAMP	OÙ IL APPARAÎT	SE RÈGLE DANS
`brand.name`	jeton `{brand}` ; titre de la page `page.html`	**Identité** · *Nom de l'instance*
`brand.shortName`	jeton `{brandShort}` ; titre de l'administration	*Nom court*

CHAMP	OÙ IL APPARAÎT	SE RÈGLE DANS
`brand.productName`	jeton `{product}`	*Nom du produit*
`brand.monogram`	les lettres du logo de la barre du haut (accueil, À propos…)	*Monogramme*
`brand.logoEmoji`	l'icône du logo de l'Explorateur et du panneau d'administration	*Emoji logo*
`brand.organization`	**le texte à côté du logo** de la barre du haut ; jeton `{orgShort}`	*Organisation*
`brand.tagline`	jeton `{tagline}` (par langue)	*Accroche*
`org.name`	jeton `{org}`	(assistant de première installation)

🔢 **MICRO-EXEMPLE**

Dans la démonstration, la barre du haut affiche « **IRIBHM — ULB** » : c'est `brand.organization`, pas `brand.name` (« IRIBHM Microscopy Platform »). Le monogramme « **IR** » est dans la pastille verte.

Vocabulaire, référencement, pied de page, menu

CHAMP	OÙ IL APPARAÎT	SE RÈGLE DANS
`specimen.singular`, `specimen.plural`	jetons `{specimen}`, `{specimenPlural}` et leurs versions à majuscule	*Terminologie* (par langue)
`datasetTypes.<type>.label`, `.title`	badges, filtres, cartes de l'accueil	**Types de données**
`seo.description`, `seo.keywords`	les balises `<meta>` de chaque page	*Accroche & SEO*
`pageTitles.<page>`	le titre d'onglet de chaque page (`home`, `explorer`, `viewer`, `compare`, `2d`, `about`, `admin`, `legal`)	(fichier)
`footer.copyright`	la ligne « © … » du pied de page	*Pied de page*
`nav.showExplorer`, `showCompare`, `showAbout`	affiche ou cache le lien dans la barre	*Navigation*
`nav.showLegal`	ajoute le lien **Mentions légales** **dans le pied de page**	*Navigation*
`nav.customPages`	les pages que vous avez créées (`slug`, libellé, visible)	ajouté à la première publication

ℹ️ **À SAVOIR**

Décocher masque le lien, pas la page. Si vous décochez **Afficher « Comparer »**, l'adresse `compare.html` reste valide : seul le lien disparaît de la barre. Les pages que vous créez sont ajoutées à la suite des liens, dans l'ordre de la liste, avec le libellé dans la langue du visiteur.

Le lien de l'organisation et les liens du pied de page

ℹ️ **À SAVOIR**

- **`footer.links`** (*Pied de page → Liens*) : le pied de page public (accueil, Explorateur, À propos, mentions légales, pages personnalisées) affiche vos liens **avant** **Mentions légales**. Seules les adresses `http(s)://`, `mailto:` et relatives sont acceptées ; un lien externe s'ouvre dans un nouvel onglet.
- **`org.url`** (*Lien de l'organisation*) : après le copyright, le pied de page ajoute « · » suivi du nom de l'organisation, lié à cette adresse (le nom d'hôte de l'adresse si aucun nom d'organisation n'est saisi ; rien si `org.url` est vide).
- Le copyright saisi **par langue** s'affiche dans la langue du visiteur.

Réglages sans écran d'administration

CHAMP	À QUOI IL SERT
`channelColorPresets`	la couleur de départ d'un canal d'après son nom (voir 16.8)
`variables`	vos variables personnalisées `{nom}` (gérées dans l'éditeur de pages, onglet Variables)

ℹ️ **À SAVOIR**

Chaque fichier porte son identifiant de version (`"$schema": "lumen3d-instance/1"`, `lumen3d-theme/1`, `lumen3d-legal/1`).

16.3 Comment une valeur arrive à l'écran

Le nom de votre institution apparaît à de nombreux endroits : dans l'onglet du navigateur, dans la barre du haut, dans la phrase d'accueil. Trois mécanismes se partagent le travail, **tous alimentés par le même fichier**.

▲ Trois chemins, un seul fichier : ils ne peuvent pas se contredire.

① Le serveur remplit l'en-tête

Dans chaque page HTML, le titre et les balises de description contiennent des **marqueurs** :

```
<title>{{SITE:pageTitles.home|Lumen3D — 3D Imaging Data Viewer}}</title>
```

Avant d'envoyer la page, le serveur remplace `{{SITE:chemin|repli}}` par la valeur trouvée dans `instance.json`. S'il n'y a rien (ou pas une chaîne de caractères), il met le **texte de repli** écrit après la barre `|`.

> ❓ **POURQUOI CE CHOIX ?**
>
> **Pourquoi côté serveur ?** Les moteurs de recherche et les aperçus de liens (messageries, réseaux) lisent l'en-tête **sans exécuter de JavaScript**. Remplir l'en-tête avant l'envoi donne le bon titre dès le premier octet, sans clignotement.

> 🔧 **POUR LES CURIEUX**
>
> - La valeur est **échappée** (`& < > "`) avant d'entrer dans la page : un nom contenant un chevron ne peut pas casser le HTML.
> - Seules les **chaînes** sont substituées : une valeur « par langue » (`{ "en": …, "fr": … }`) n'est pas résolue ici (le serveur ne connaît pas la langue du visiteur) ; c'est le navigateur qui la résout ensuite.
> - Le fichier est relu seulement quand il change (la date de modification sert de clé). Les deux serveurs (Python et PHP) appliquent la même règle.

② Le navigateur remplit la page

Le logo, le pied de page et le menu portent des attributs `data-instance` :

```
<span data-instance="brand.organization">Lumen3D</span>
<a data-instance-attr="title:brand.tagline">…</a>   <!-- variante pour un attribut -->
```

Au chargement, `InstanceConfig` lit `instance.json` et écrit les valeurs : `data-instance` remplit le **texte** d'un élément, `data-instance-attr` remplit des **attributs** (« attribut:chemin », séparés par des points-virgules ; les pages livrées n'en ont pas besoin aujourd'hui, mais le mécanisme est prêt). Il applique aussi les titres, les `<meta>` et les entrées du menu (`nav.*`).

> 🔧 **POUR LES CURIEUX**
>
> Le texte entre les balises est le **repli** : si le fichier est introuvable (hors ligne, installation neuve), la page affiche ce repli au lieu d'un trou. L'application peut être rappelée (après un changement de langue, ou quand l'aperçu de l'administration recharge la configuration).

③ Les jetons dans les phrases

Les fichiers de langue **ne contiennent jamais** le mot « embryon ». Ils écrivent un **jeton** entre accolades, que `I18n.t()` remplace au moment d'afficher :

JETON	VIENT DE	VALEUR DANS LA DÉMONSTRATION
`{brand}`	`brand.name`	IRIBHM Microscopy Platform
`{brandShort}`	`brand.shortName`	Lumen3D
`{product}`	`brand.productName`	Lumen3D
`{tagline}`	`brand.tagline`	Confocal Imaging Data Viewer
`{org}`	`org.name` , sinon `brand.organization`	IRIBHM — Université Libre de Bruxelles
`{orgShort}`	`brand.organization` , sinon `org.name`	IRIBHM — ULB
`{specimen}` , `{specimenPlural}`	`specimen.singular` , `.plural` **dans la langue du visiteur**	embryon, embryons
`{Specimen}` , `{SpecimenPlural}`	les mêmes, **première lettre en capitale**	Embryon, Embryons
`{type3d}` , `{type2d}` , `{typeLive}`	le nom affiché de chaque type	3D, 2D, Live

Treize jetons au total. L'éditeur de pages les liste (onglet **Variables**), avec un bouton pour copier chacun :

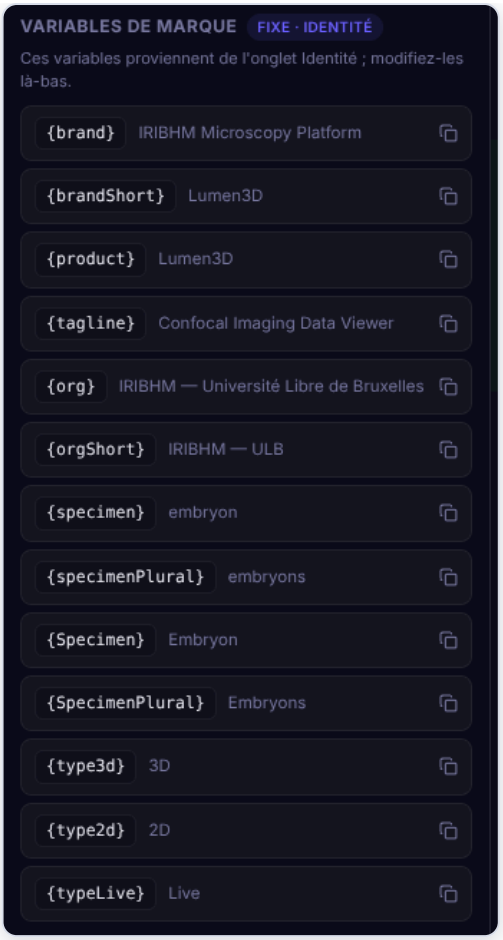
> 🔧 **POUR LES CURIEUX**
>
> **Ordre de résolution d'un jeton** : d'abord une valeur passée par le code appelant (`{count}` , `{name}` …), puis le jeton de l'instance, sinon l'accolade **reste telle quelle** à l'écran (un jeton inconnu se voit).
>
> **Un piège évité.** Les noms de types par défaut vivent eux-mêmes dans les fichiers de langue (`types.3d.label` …). Les lire par `I18n.t()` ferait appeler `t()` par la fonction qui sert `t()` : une boucle sans fin. Le code lit donc cette chaîne par **`I18n.raw()`** , la même recherche *sans* remplacement de jetons.

🔢 **MICRO-EXEMPLE**

La clé `landing.heroTitle` vaut `Explorez les {specimenPlural}` en français et `Explore {SpecimenPlural}` en anglais.

- avec `specimen.plural.fr = embryons` → « **Explorez les embryons** » ;
- si le laboratoire écrit « organoïdes » → « **Explorez les organoïdes** », sans toucher à un seul fichier de langue.

Les mêmes jetons marchent dans les **textes de page** que vous écrivez (16.6). L'éditeur y ajoute les **variables dynamiques** : année, date, heure, et les compteurs du catalogue.



▲ Les jetons de l'instance, vus de l'éditeur de pages (jeu de démonstration).

Une valeur « par langue »

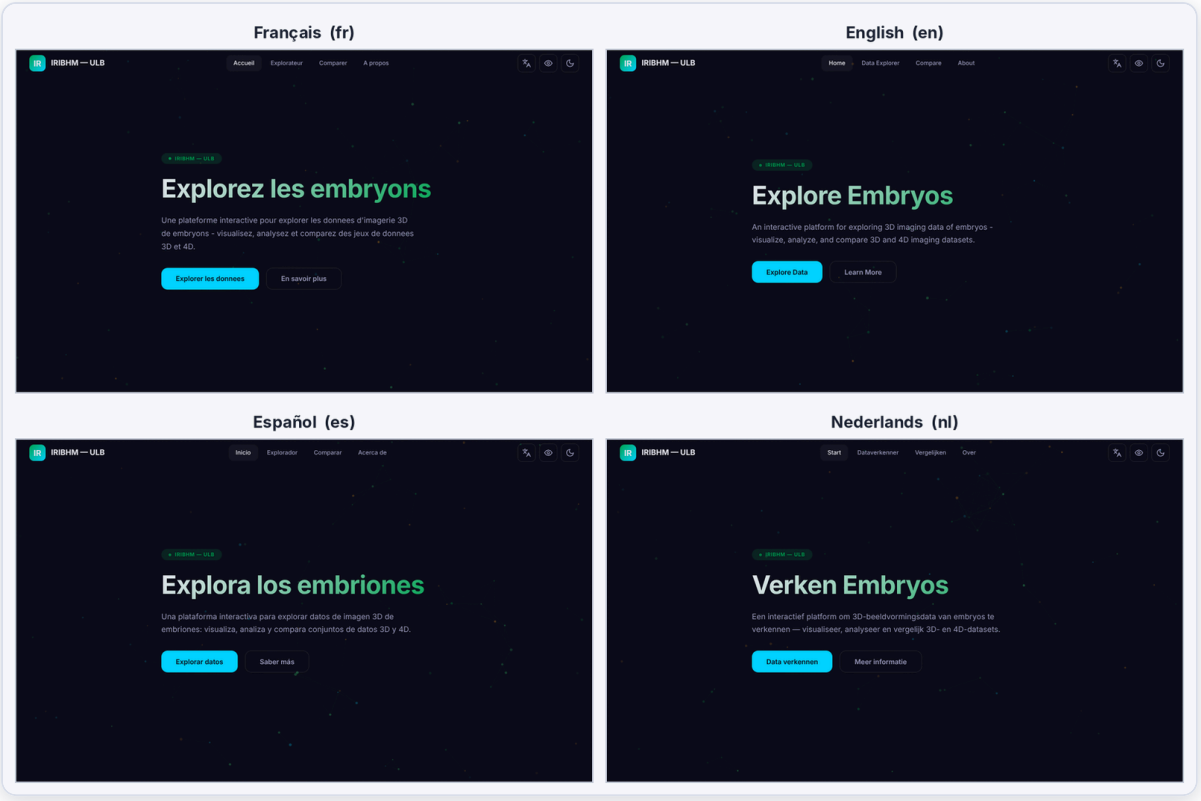
Le nom d'un spécimen, une accroche, une description SEO : ces textes dépendent de la langue. Ils s'écrivent soit comme **une simple chaîne** (identique partout), soit comme un **objet par langue**.

▲ Une valeur par langue : on répond avec la langue du visiteur, sinon l'anglais, sinon la première trouvée.

🔢 **MICRO-EXEMPLE**

Le néerlandais manquant. Dans la démonstration, `specimen.plural` existe en anglais, français et espagnol, mais pas en néerlandais. Un visiteur néerlandais voit donc : « Verken **Embryos** » : la phrase est néerlandaise, le nom est le repli anglais, et il est en capitale car le jeton est `{SpecimenPlural}`.

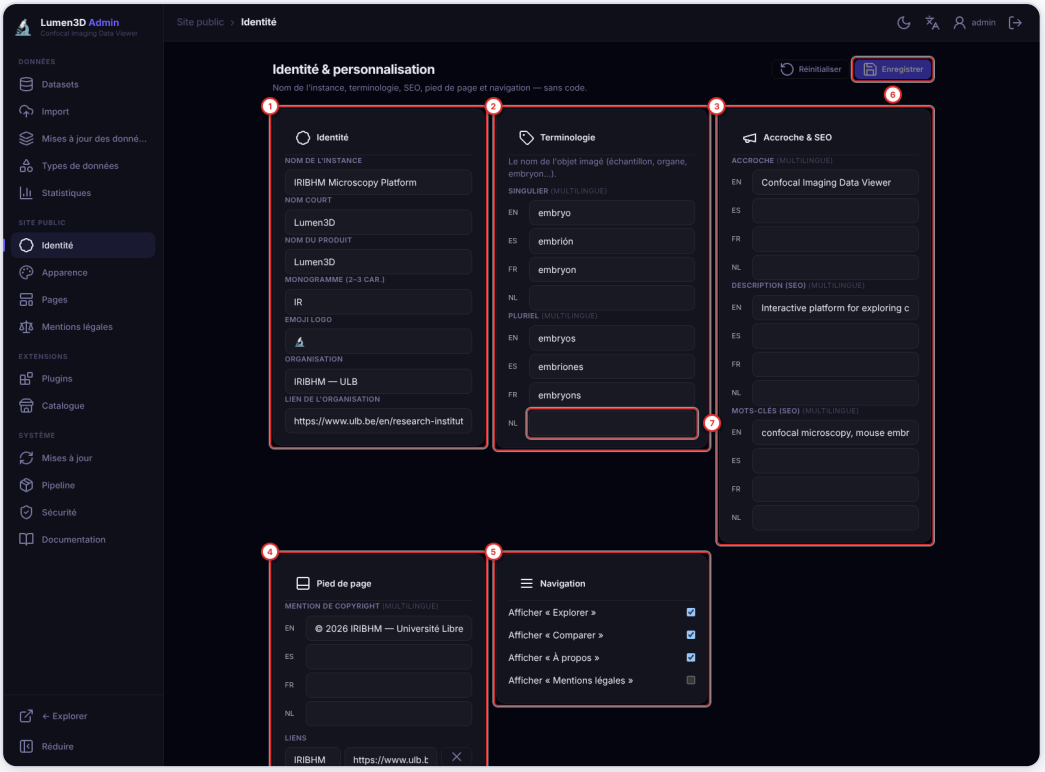
Ce n'est pas une panne, c'est le repli qui fonctionne : la solution est de remplir la ligne **NL** dans l'onglet **Identité**.



▲ La page d'accueil dans les quatre langues. Le nom de l'objet suit la langue ; en néerlandais, il retombe sur l'anglais faute de saisie.

16.4 Les écrans de personnalisation

L’onglet Identité

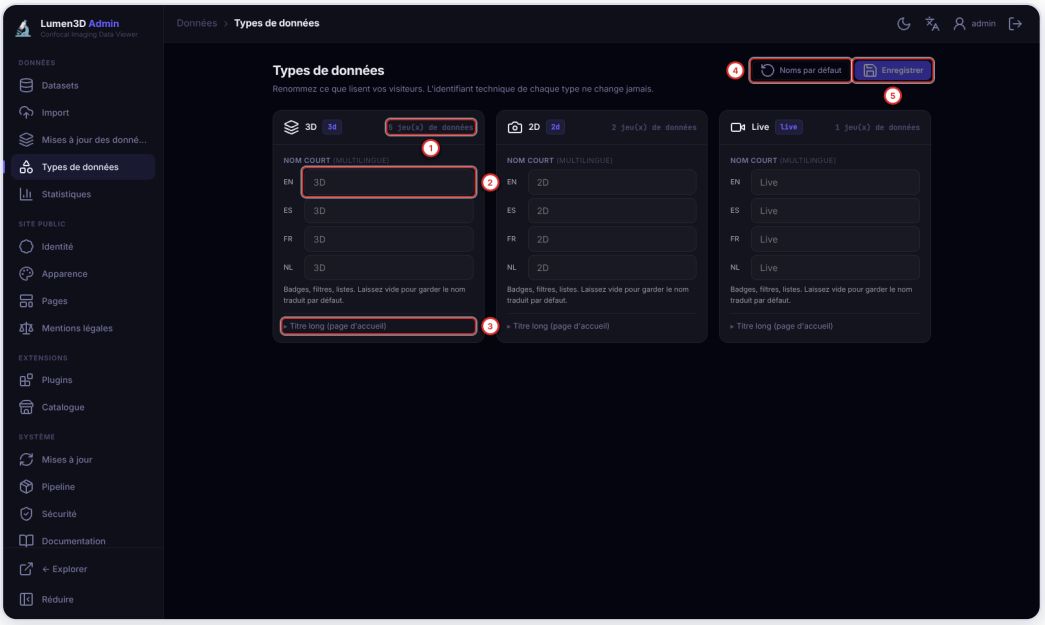


▲ L’onglet Identité & personnalisation (jeu de démonstration).

N	CE QUE C’EST
1	Carte **Identité** : noms, monogramme, emoji, organisation.
2	Carte **Terminologie** : le mot qui désigne vos objets d’étude, au singulier et au pluriel.
3	Carte **Accroche & SEO** : accroche, description et mots-clés, par langue.
4	Carte **Pied de page** : copyright (par langue) et liens, affichés dans le pied de page public (voir 16.2).
5	Carte **Navigation** : une case par entrée du menu public. **Mentions légales** est décochée par défaut.
6	**Enregistrer** : actif seulement quand un champ a changé.
7	Un champ **multilingue** : une ligne par langue disponible. Ici la ligne **NL** est vide.

Chaque champ multilingue montre **autant de lignes que la plateforme a de langues** : si vous ajoutez une langue (16.9), une ligne apparaît toute seule.

L’onglet Types de données



▲ L’onglet Types de données.

N	CE QUE C’EST
1	Le nombre de jeux de données de ce type (ici 5 jeux 3D).
2	**Nom court** : une ligne par langue. Vide = on garde la traduction par défaut.
3	**Titre long (page d’accueil)** : volet repliable, pour les grandes cartes.
4	**Noms par défaut** : **vide** les champs (il faut ensuite enregistrer).
5	**Enregistrer**.

Renommer « 3d » en « Volumes » change le **mot affiché**, jamais le dossier `DATA_WEB/3d/`, ni les adresses, ni les identifiants. La chaîne est : nom saisi → sinon traduction `types.<type>` → sinon l’identifiant lui-même.

> 🔧 **POUR LES CURIEUX**
>
> **Une bonne pratique du code.** Aucun écran n’écrit un nom de type en dur : tout passe par `Utils.datasetTypeLabel()` et `datasetTypeTitle()`. Un élément HTML qui porte `data-dataset-type="3d"` voit son texte réécrit par ce mécanisme, y compris à chaque changement de langue.

Pourquoi deux onglets ne s’écrasent pas

Trois onglets écrivent dans le même fichier `instance.json` : Identité, Types de données et l’éditeur de pages (variables, menu). Un onglet resté ouvert une heure ne doit pas effacer le travail des autres.

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un cahier partagé.** Chacun écrit **sur ses propres lignes** : le serveur n'accepte de modifier que les « chemins » annoncés (`?merge=brand,specimen,…`) sur le document **tel qu'il est maintenant**, sous un verrou. Il n'y a pas de photocopie périmée.

Pour les **pages**, une protection de plus : chaque lecture porte une **révision** (une empreinte du document). Un enregistrement qui arrive avec une révision périmée est **refusé** (409 « stale ») plutôt que d'écraser une version plus récente.

L'assistant de première installation

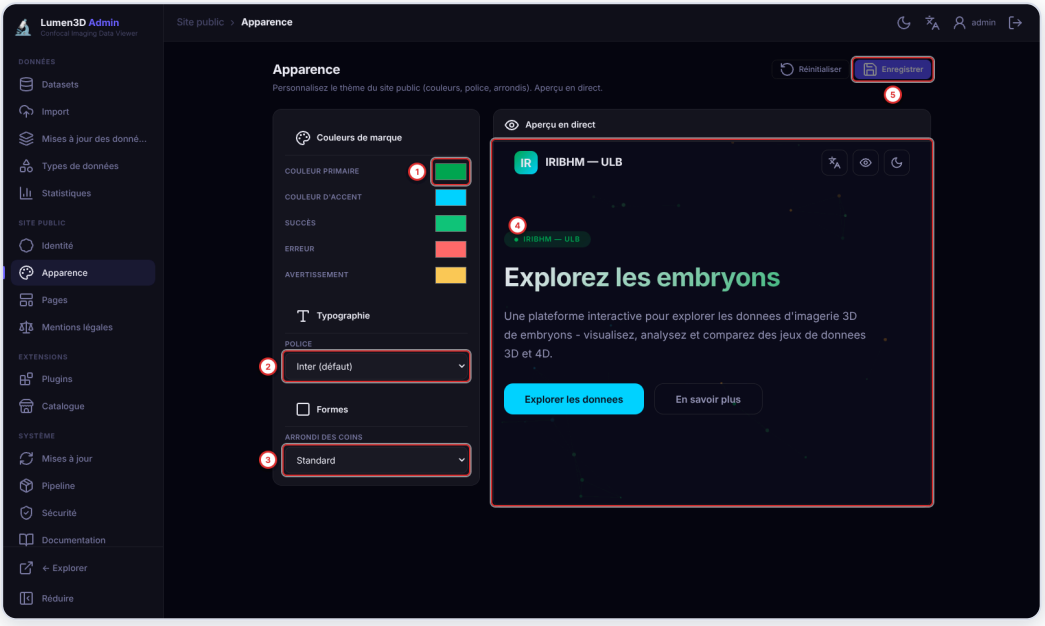
À la toute première visite (aucun mot de passe n'existe), le panneau ouvre un assistant en **cinq étapes**. Seule la première est obligatoire.

1. **Compte** : le mot de passe de l'administrateur.
2. **Identité** : nom de l'instance, organisation, nom de l'objet (singulier / pluriel).
3. **Thème** : une couleur de marque parmi six (vert, bleu, violet, turquoise, orange, carmin).
4. **Textes** : accroche et ligne de copyright.
5. **Plugins** : le catalogue signé (chapitre 15).

À la fin, l'assistant n'écrit que ce qu'il a rempli : `brand` (le monogramme prend les deux premiers caractères du nom, en majuscules), `specimen`, éventuellement `org.name` et `footer.copyright`, puis les jetons de couleur choisis dans `theme.json`.

16.5 Le thème : couleurs, police, arrondis

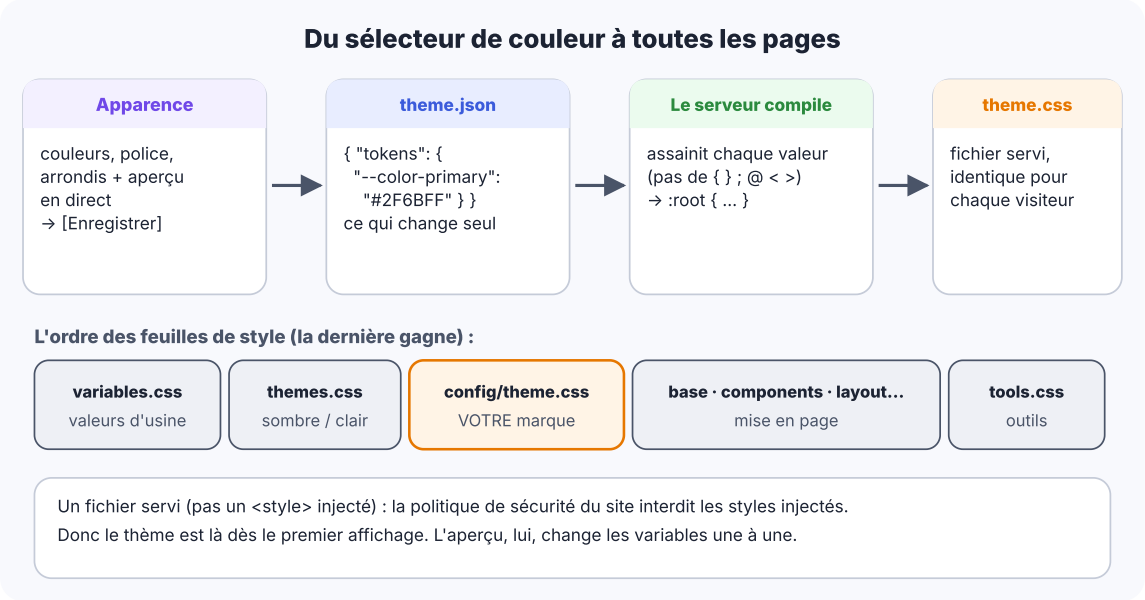
Le look du site est piloté par des **variables CSS** (des « jetons » de style) : `--color-primary`, `--font-sans`, `--radius-md` … L'onglet **Apparence** n'en modifie qu'une poignée, à dessein.



▲ L'onglet Apparence : cinq couleurs, une police, un arrondi, et un aperçu du vrai site.

N	CE QUE C'EST
1	Les **cinq couleurs de marque** : primaire, accent, succès, erreur, avertissement.
2	La **police** : Inter (défaut), Système, Grotesque, Serif, Arrondie.
3	L'**arrondi des coins** : Standard, Net, Doux, Rond.
4	L'**aperçu en direct** : le vrai site dans un cadre, qui change instantanément.
5	**Enregistrer** : rien n'est appliqué au site avant ce clic.

Du clic à toutes les pages



▲ Le chemin d'un réglage de thème, et l'ordre des feuilles de style.

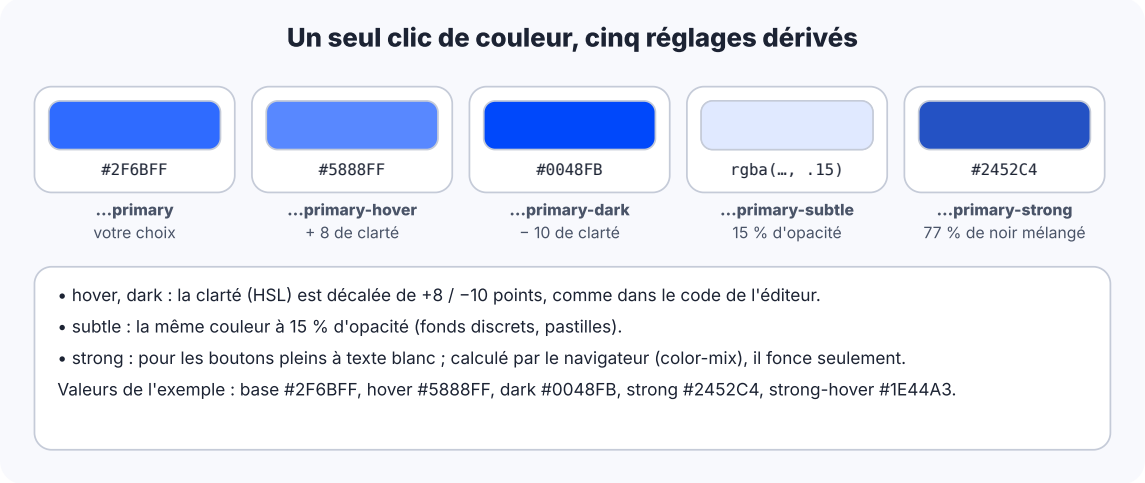
📌 **À RETENIR**

Le serveur ne laisse passer que des variables dont le nom commence par `--`, et retire de chaque valeur les caractères qui pourraient casser une feuille de style (`{ } ; < > \ @`) ; une valeur est coupée à 200 caractères. **Un thème mal saisi ne peut pas détruire la feuille entière.**

La feuille `theme.css` est chargée **après** `themes.css` (le thème sombre/clair d'usine) et avant les feuilles de mise en page : elle **gagne** donc sur les valeurs d'usine, mais seulement pour ce que vous avez changé. Le reste continue de venir de `variables.css`.

Une couleur, cinq réglages

Un seul sélecteur de couleur ne suffit pas : un bouton a besoin d'une teinte au survol, d'une teinte foncée, d'un fond discret. L'éditeur les **dérive** pour vous.



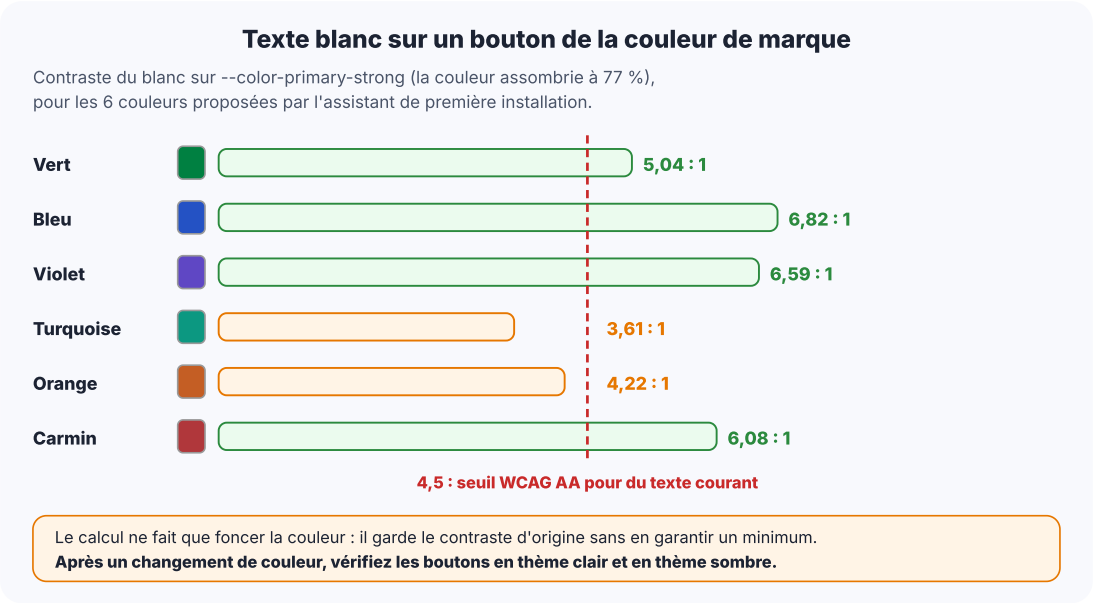
▲ Une couleur choisie (ici #2F6BFF) et ses dérivés.

RÉGLAGE	CALCUL
`--color-primary-hover`	la clarté (HSL) **+ 8** points
`--color-primary-dark`	la clarté **− 10** points
`--color-primary-subtle`	la même couleur à **15 %** d'opacité (accent : 12 %)
`--color-primary-strong`	**77 %** de la couleur sur du noir, puis assombrie d'un point à la fois jusqu'à **4,5 : 1** sous du blanc (survol : 64/77 de ce facteur)

Succès, erreur et avertissement dérivent de la même façon (survol, fond discret à 12 %).

Le contraste du texte blanc

Les boutons pleins portent du **texte blanc** sur `--color-primary-strong`. C'est la raison d'être du réglage « strong » : le vert d'usine (#00A654) ne donne que **3,19 : 1** sous du blanc, insuffisant pour du texte ; assombri, il passe à **5,05 : 1**. La règle WCAG AA demande **4,5 : 1**.



▲ Contraste du texte blanc sur le bouton, pour les six couleurs proposées par l'assistant.

ℹ️ **À SAVOIR**

La figure montre le simple mélange à 77 %. Dès qu'une couleur primaire est définie, le **compilateur du thème** (côté serveur) calcule lui-même la couleur des boutons pleins : il part de 77 % et assombrit d'un point à la fois jusqu'à ce que le texte blanc atteigne **4,5 : 1**. Le seuil AA est donc **garanti** pour toute couleur qu'il sait lire (`#rgb` , `#rrggbb` , `rgb()` / `rgba()`). Les teintes déjà sombres ne bougent pas ; **l'orange** #FF7A2F (4,22 à 77 %) donne #BD5A23 (**4,52 : 1**) et **la turquoise** #0FC5A8 (3,61 à 77 %) donne #0A8672 (**4,50 : 1**). L'aperçu de l'onglet Apparence montre les mêmes couleurs avant l'enregistrement, et un `theme.css` existant est recompilé une fois, automatiquement, après la mise à jour. Sans couleur définie, le vert d'usine vient de la feuille de style (#00803F, 5,05 : 1).

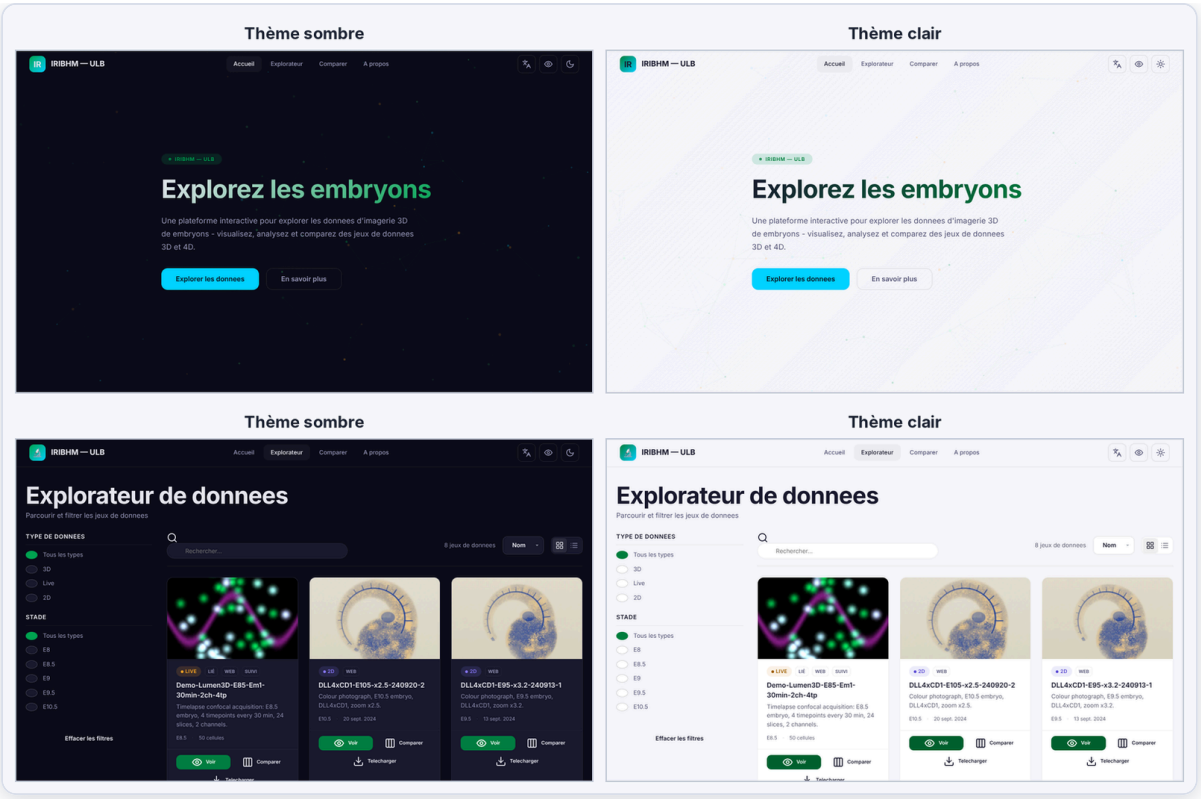
COULEUR DE L'ASSISTANT	BOUTON (SURVOL)	CONTRASTE
#00A654	#008041 (#006A36)	5,04 : 1
#2F6BFF	#2452C4 (#1E44A3)	6,82 : 1
#7C5CFF	#5F47C4 (#4F3BA3)	6,59 : 1
#0FC5A8	#0A8672 (#097060)	4,50 : 1
#FF7A2F	#BD5A23 (#9E4C1D)	4,52 : 1
#E5484D	#B0373B (#932E31)	6,08 : 1

🔧 **POUR LES CURIEUX**

Le contraste est le rapport de luminance relative WCAG, `(L1 + 0,05) / (L2 + 0,05)` , où `L` est calculée sur les composantes RVB **linéarisées** (loi sRVB). Les chiffres ci-dessus sont recalculés par le script de la figure à partir des couleurs de l'assistant (`DOCS/documentation/img/ch16/make_figures.py`).

Clair ou sombre

Le thème **sombre** est le défaut, par choix : une image de fluorescence se lit mieux sur fond noir. Le visiteur bascule d'un clic ; son choix est mémorisé **dans son navigateur** (`iribhm-theme`).



▲ Le même site en thème sombre (à gauche) et clair (à droite).

> **ℹ️ À SAVOIR**
>
> Le thème n'est **pas** calé sur le réglage du système au premier passage. Si le visiteur n'a **jamais** choisi, un changement du réglage clair/sombre de son système est suivi ; dès qu'il a choisi, c'est son choix qui compte.

Chaque thème règle aussi `color-scheme` : les menus déroulants natifs, les barres de défilement et les sélecteurs de date prennent le bon aspect, sans texte clair sur fond blanc.

Le fond du viewer 3D : un réglage à part

Le **fond du volume** ne suit pas le thème du site : c'est un réglage du viewer, dans la barre latérale (**Arrière-plan**), mémorisé avec l'état de travail.

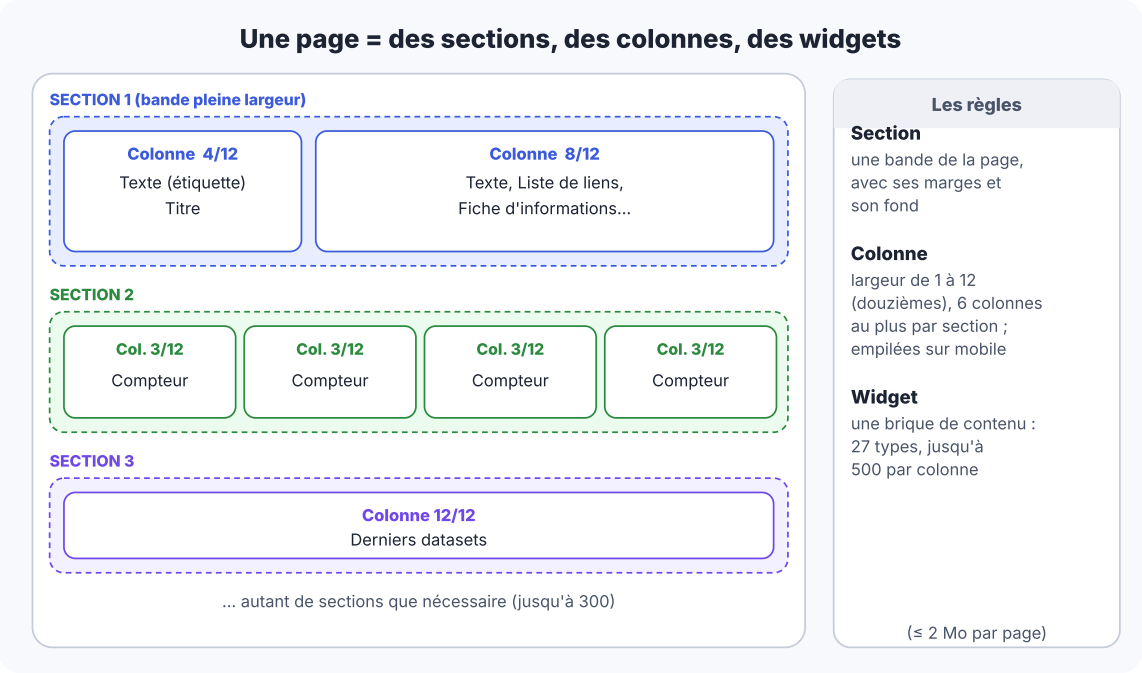
CHOIX	COULEUR
Sombre (défaut)	`#000000`
Clair	`#f4f6fb`
Papier	`#f8f5ec`
Transparent	aucun fond (les exports en PNG gardent la transparence)
Personnalisé	la couleur que vous choisissez (`#1a1d27` au départ)

👉 **ASTUCE**

Un fond **clair** ou **papier** convient aux figures imprimées. Un code de couleur invalide est refusé avec un avertissement en console : le viewer garde la couleur par défaut du choix plutôt que d'afficher une teinte au hasard.

16.6 Le constructeur de pages

Accueil, À propos, ou toute page que vous créez (protocoles, équipe, contact…) se construisent **à la souris**, comme dans un logiciel de mise en page.



▲ Une page = des sections (bandes), des colonnes (douzièmes), des widgets (briques).

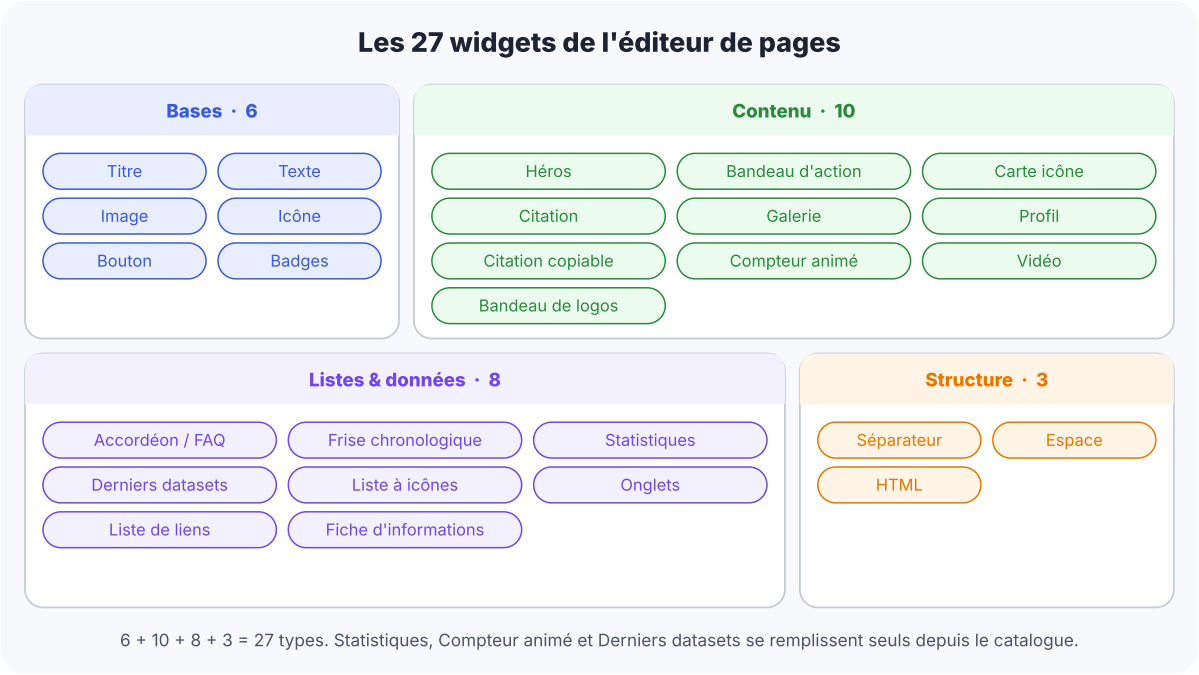
Le modèle : sections, colonnes, widgets

- Une **section** est une bande de la page : fond, marges, largeur maximale.
- Une **colonne** y occupe de **1 à 12 douzièmes** ; une section a **6 colonnes au plus** dans l'éditeur. Sur téléphone, les colonnes **s'empilent**.
- Un **widget** est une brique de contenu. Il y en a **27 types**.

💡 **UNE IMAGE POUR COMPRENDRE**

Un journal. La page est un numéro, les sections sont les bandeaux horizontaux, les colonnes sont les colonnes du journal, les widgets sont les articles, les photos et les encadrés.

Les 27 widgets



▲ Les 27 widgets, rangés comme dans la palette de l'éditeur.

Ce que fait chaque widget

WIDGET	CE QUE C'EST, ET SES RÉGLAGES
Bases	
Titre	un titre de section ; niveau de titre et alignement
Texte	un paragraphe ; mini-mise en forme `**gras**`, `*italique*`, `[lien](adresse)`
Image	une image avec légende et lien ; cadrage (remplir ou contenir), largeur, hauteur, texte alternatif
Icône	un pictogramme choisi dans la bibliothèque d'icônes (avec recherche) ; taille, couleur
Bouton	libellé et lien ; variante, icône à gauche ou à droite, taille, contour, pleine largeur
Badges	de petites étiquettes ; pastille colorée, police à chasse fixe, couleurs
Contenu	
Héros	le grand bandeau d'accueil : titre, sous-titre, deux boutons, badge, halo décoratif, fond
Bandeau d'action	un encart qui invite à cliquer : titre, sous-titre, deux boutons
Carte icône	icône, image ou monogramme + titre + texte + lien ; disposition horizontale ou verticale
Citation	texte, auteur, rôle, photo ; en barre ou en carte
Galerie	des images en grille ; colonnes, légendes, zoom au survol
Profil	une fiche de personne : nom, rôle, description, média
Citation copiable	une référence avec bouton « copier » et un bloc repliable (BibTeX…)

WIDGET	CE QUE C'EST, ET SES RÉGLAGES
Compteur animé	un chiffre qui défile ; valeur fixe **ou** source du catalogue ; préfixe, suffixe
Vidéo	un fichier `.mp4` / `.webm`, ou un lien YouTube/Vimeo qui s'ouvre **dans un nouvel onglet** ; image d'aperçu, boucle, lecture automatique muette
Bandeau de logos	une rangée de logos partenaires ; désaturés avec couleur au survol, plaque claire
Listes & données	
Accordéon / FAQ	des questions qui se déplient ; une seule ouverte à la fois, la première ouverte
Frise chronologique	une suite d'étapes datées
Statistiques	une rangée de chiffres clés ; valeur fixe **ou** compteur du catalogue (jeux de données, spécimens, cellules, régions)
Derniers datasets	les cartes des jeux les plus récents ; nombre, colonnes, type et date affichés ou non
Liste à icônes	une liste à puces illustrées ; verticale ou horizontale
Onglets	du contenu réparti en onglets
Liste de liens	des lignes séparées par des filets fins, avec une flèche : la façon sobre de présenter des liens
Fiche d'informations	un tableau « libellé : valeur »
Structure	
Séparateur	un trait ; épaisseur, largeur, style (plein, tirets, points), couleur
Espace	un vide réglable de 0 à 400 pixels (32 par défaut)
HTML	du HTML libre, **nettoyé par liste blanche** (voir ci-dessous)

🔧 POUR LES CURIEUX

Sécurité du contenu. Le texte est toujours écrit par `textContent` (jamais interprété comme du HTML). Le widget HTML passe par une **liste blanche** : les scripts, les gestionnaires d'événements, les SVG et les liens `javascript:` sont retirés ; chaque attribut `style` est filtré déclaration par déclaration (pas de `position: fixed`, pas de `z-index`, pas d'URL exotique). Les liens saisis refusent les schémas `javascript:`, `vbscript:`, `data:`. Les vidéos externes ne sont **pas** intégrées dans la page : la politique de sécurité du site interdit les cadres tiers, d'où le simple lien.

Le style d'un élément

Chaque **widget**, **colonne** et **section** a ses réglages de style, en sept familles :

FAMILLE	CE QU'ON Y RÈGLE
Texte	couleur (ou **dégradé** peint dans les lettres), taille, graisse, interligne, espacement des lettres, italique, majuscules, alignement
Fond & bordure	couleur ou image de fond, voile, arrondi, bordure (épaisseur, couleur, trait), ombre (légère, moyenne, grande, halo), opacité

FAMILLE	CE QU'ON Y RÈGLE
Espacement	marges et marges intérieures, côté par côté ou liés
Taille	largeur maximale, hauteur minimale
Effets	survol : lévitation, halo ou zoom
Visibilité	masquer sur mobile, ou sur ordinateur
CSS personnalisé	quelques déclarations CSS, **assainies** avant d'être appliquées

🔧 **POUR LES CURIEUX**

Le style est compilé en CSS **en ligne** (attribut `style`), sans feuille de style injectée : la politique de sécurité interdit les `<style>` ajoutés après coup. Seules les règles qui ont vraiment besoin d'une feuille (le survol `:hover`, les masquages par taille d'écran `@media`) vivent dans `css/pages.css`, un fichier du site. Chaque valeur passe par un filtre qui retire de quoi sortir d'une déclaration.

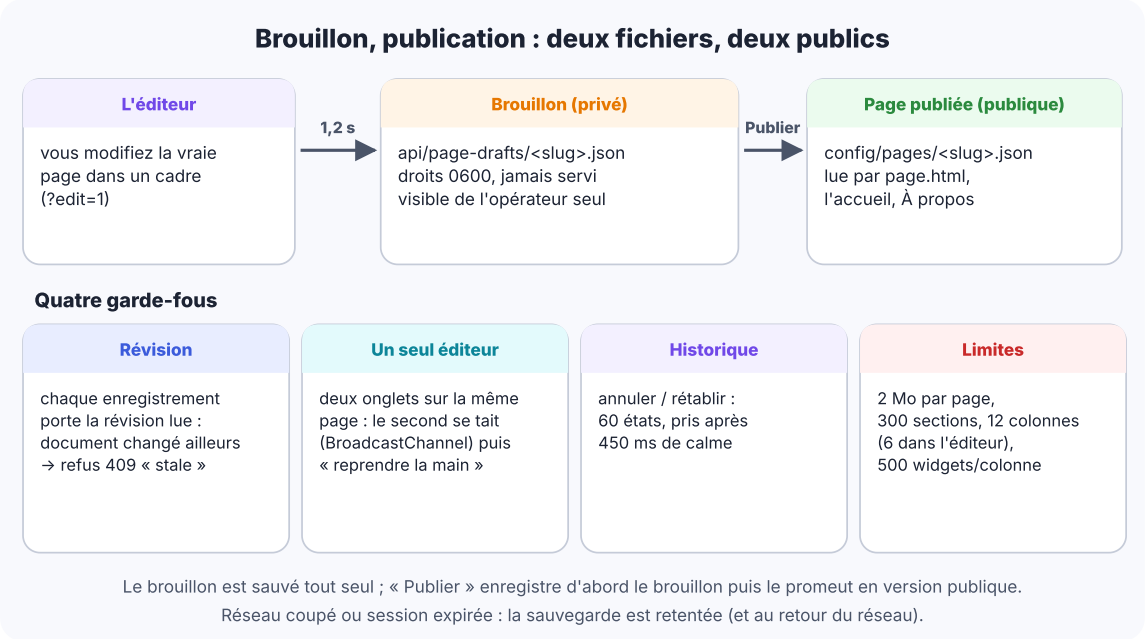
Variables, jetons et fonds

- **Variables dynamiques** : `{year}`, `{date}`, `{time}`, et les compteurs `{datasetCount}`, `{specimenCount}`, `{cellCount}`, `{regionCount}` (calculés à l'affichage, dans la langue du visiteur).
- **Jetons de l'instance** : les treize du paragraphe 16.3 fonctionnent aussi dans les textes de page.
- **Vos variables** : un nom (une lettre, puis lettres, chiffres ou `_`, 32 caractères au plus) et une valeur, par langue si besoin. Si leur nom est celui d'une variable existante, la vôtre **gagne**.
- **Fonds animés** : dix préréglages (particules flottantes, vagues, aurore, ciel étoilé, grille pulsante, constellation, orbes, ondes du curseur, champ de force, halo). Cinq vivent seuls, cinq réagissent à la souris. Avec « réduire les animations » activé dans le système, un seul dessin **fixe** remplace l'animation.

👉 **ASTUCE**

Les variables `{…}` marchent dans **tous** les textes de page, mais pas dans les fichiers de langue (qui n'ont que les jetons de l'instance).

Brouillon et publication



▲ Deux fichiers, deux publics : le brouillon est privé, la version publiée est publique.

> 📌 **À RETENIR**
>
> **Rien n'est public avant** **Publier** **.** Pendant que vous travaillez, les visiteurs voient l'ancienne version. Le brouillon est enregistré tout seul, mais il est rangé **hors de** `config/` : ce dossier se lit sans mot de passe, et l'éditeur enregistre environ une fois par seconde pendant que vous écrivez. Un brouillon laissé dans `config/` aurait permis à n'importe qui de regarder l'opérateur écrire.

Cela pose aussi des règles de bon sens à l'éditeur :

- **« Publier »** enregistre d'abord le brouillon, puis le *promeut* en version publique.
- **« Défaut »** (pages d'origine) revient au modèle livré ; sur une page à vous, il abandonne le brouillon et revient à la version publiée.
- Un **seul onglet d'édition par page** : le second se tait jusqu'à ce que vous « repreniez la main ».
- L'éditeur retient **60 étapes** d'annulation / rétablissement.
- Au premier **Publier** d'une page créée par vous, elle est ajoutée au **menu** automatiquement.

L'éditeur à l'écran

L'éditeur s'ouvre **dans son propre onglet** (`admpan.html?editor=<page>`) : la **vraie** page apparaît dans un cadre, avec son vrai menu et son vrai thème ; l'éditeur ajoute une couche de poignées par-dessus.

▲ L'éditeur de pages, sur la page « À propos » (jeu de démonstration).

N	CE QUE C'EST
1	La **page** en cours d'édition (la mention « intégrée » : elle utilise encore le modèle livré).
2	La **langue** que vous écrivez.
3	Les cinq volets : **Éléments**, **Réglages**, **Fond**, **Traduire**, **Variables**.
4	La **palette** des 27 widgets, en quatre familles. Un clic ou un glisser.
5	L'aperçu **ordinateur / tablette / mobile**.
6	**Brouillon** : enregistre sans publier.
7	**Publier** : met votre version en ligne.
8	**La vraie page** : cliquez un élément pour le sélectionner.

Les widgets de la page sont décrits par le **même code** pour l'éditeur et pour le site (`PageRenderer`) : ce que vous voyez dans l'éditeur est exactement ce qui sera publié.

Le modèle de la page À propos

Les pages **Accueil** et **À propos** existent d'origine. Tant que vous n'avez rien publié, la page affichée est un **modèle** écrit dans le même format que vos pages (`js/core/page-templates.js`). Il est à la fois :

- ce que **voient les visiteurs** (il n'y a plus de HTML de secours caché dans `about.html`) ;
- le **point de départ** de l'éditeur, qui l'ouvre avec la mention « Modèle de départ ».

Ses phrases sont déjà écrites en anglais, français et espagnol, avec des jetons (`{brandShort}`, `{SpecimenPlural}`, `{org}`, `{year}`…) : la page porte donc **votre** nom avant que vous n'ayez changé un mot.

Traduire une page

L'onglet **Traduire** liste **tous les textes** de la page, un champ par langue, et signale ceux qui manquent (« 24 textes · 7 traductions manquantes »). Il n'y a **aucune traduction automatique** : rien ne quitte votre serveur.

> 👉 **ASTUCE**
>
> Méthode conseillée : rédigez toute la page dans **une** langue, puis passez sur l'onglet Traduire. N'oubliez pas l'anglais : c'est le repli de toutes les autres langues.

Les limites

LIMITE	VALEUR
taille d'une page	2 Mo
sections par page	300
colonnes par section	12 côté serveur, 6 dans l'éditeur
widgets par colonne	500
largeur d'une colonne	1 à 12 douzièmes
adresse d'une page (`slug` **)**	minuscules, chiffres, `-` , `_` ; 64 caractères au plus

Un document hors limites est **refusé en entier**, jamais enregistré à moitié (seule la largeur d'une colonne est simplement ramenée entre 1 et 12). Une page supprimée ou inconnue ramène à l'accueil plutôt que d'afficher une page vide.

16.7 Les mentions légales

Un site public a besoin d'un **éditeur**, d'un **hébergeur**, d'une politique de **données**. L'onglet **Mentions légales** est un éditeur volontairement simple : une liste de **sections**, chacune avec un titre et un texte, **par langue**.

▲ La page legal.html, telle qu'un visiteur la voit (modèle neutre, en français).

N	CE QUE C'EST
1	Le **titre** d'une section (« Éditeur du site »).
2	Le **texte** : un paragraphe par ligne blanche. Les `[crochets]` sont à remplacer par vos informations.

- Tant que vous n'avez rien publié, la page montre le **modèle neutre** livré (éditeur, protection des données, cookies et stockage local, propriété intellectuelle, avertissement, contact). Il est écrit en **anglais, français, espagnol et néerlandais**.
- Le texte est affiché par `textContent` : aucun HTML, aucune mise en forme, une section par bloc.
- Le lien n'apparaît dans le pied de page que si la case **Afficher « Mentions légales »** est cochée (décochée par défaut).

> ⚠️ **ATTENTION**
>
> Ces textes sont des **points de départ**, pas des conseils juridiques. Faites-les relire pour votre pays. N'oubliez pas qu'un serveur web garde généralement un **journal d'accès** (adresses IP) selon sa configuration : c'est l'affaire de votre hébergeur, et elle se déclare ici (voir 16.11).

16.8 D'où vient la couleur de départ d'un canal ?

Quand vous ouvrez un jeu de données, chaque canal apparaît déjà en couleur. D'où vient cette couleur ? Le chapitre 11 montre comment on la **change** ; voici d'où elle **vient**.

▲ La première réponse non vide l'emporte.

Le panneau prend la **première** réponse disponible :

1. les **réglages d'affichage** du jeu (`display_defaults`) ;
2. une liste `colors` (ancien format) ;
3. la couleur du canal dans `metadata.json` (`channels[i].color`) : celle que le **pipeline** écrit au départ, ou que l'**éditeur de datasets** a enregistrée ;
4. un **préréglage par nom** de l'instance ;
5. un **cycle neutre** : vert, bleu clair, magenta, rouge.

Le préréglage par nom

Dans `instance.json`, une liste `channelColorPresets` associe un morceau de nom à une couleur :

```
{ "match": "dapi",  "color": "#00AAFF" },
{ "match": "pecam", "color": "#FF00FF" }
```

Le nom du canal est mis en minuscules ; **le premier** `match` contenu dans le nom l'emporte. La démonstration en contient huit : `gfp` (vert), `dapi` et `hoechst` (bleu clair), `pecam` et `picam` (magenta), `rfp`, `mcherry` et `alexa` (rouge).

> ⚠️ **ATTENTION**
>
> Comme le pipeline écrit **toujours** une couleur (étape 3), les préréglages ne servent en pratique qu'aux jeux dont les métadonnées n'en ont pas. Ils n'ont pas d'écran d'administration : on les modifie dans `instance.json`.

Quatre listes de couleurs, quatre usages

OÙ	COULEURS
le pipeline (`metadata.json`)	`#00FF00` , `#00AAFF` , `#FF00FF` , `#FF0000` , `#FFFF00` , `#00FFFF`

OÙ	COULEURS
l'éditeur de datasets (si rien)	`#00FF66` , `#FF3DFF` , `#2F6BFF` , `#FF3030`
le panneau de canaux (cycle neutre)	`#00FF00` , `#00AAFF` , `#FF00FF` , `#FF0000`
la vignette du jeu (image de l'Explorateur)	vert, magenta, bleu, rouge, jaune, violet, cyan

ℹ️ **À SAVOIR**

Ces listes ne sont pas synchronisées : elles servent à des moments différents. La couleur **qui compte** pour le visiteur est celle du panneau de canaux, qui part de `metadata.json`. Les 27 couleurs du sélecteur, elles, sont fixes (chapitre 11).

16.9 Traduire : comment le site parle quatre langues

Le site public, le viewer, l'administration et même les mots de chaque outil sont traduits. Tout repose sur un **fichier par langue** et une seule fonction, `t('clé')`, qui rend le texte dans la langue du visiteur.

Où vit chaque texte du site ?

`lang/<code>.json` l'interface (boutons, messages, panneau d'admin)	**Comment** 1 943 clés × 4 langues, parité testée
`plugins/…/lang/<code>.json` les mots de chaque outil	**Comment** 28 dictionnaires (240 clés) ; manque → anglais
`config/instance.json` nom, accroche, spécimen, pied de page, SEO, types	**Comment** valeurs « par langue » : { en, fr, es, nl }
`config/pages/<slug>.json` le texte des pages que vous construisez	**Comment** dans le document lui-même, par langue
`config/legal.json` les mentions légales	**Comment** sections { title, body } par langue

Les trois derniers appartiennent à VOTRE site : ni dans lang/, ni touchés par une mise à jour.

▲ Où vit chaque texte du site.

Les chiffres

langues livrées	**4** : anglais, français, espagnol, néerlandais
textes par langue	**1 943** (mesuré sur les quatre fichiers `lang/*.json`)
dictionnaires d'outils	**28**, soit 240 textes en anglais
langues de secours	l'**anglais**, toujours

Les 1 943 textes se répartissent ainsi :

FAMILLE DE CLÉS	TEXTES	CE QU'ELLE CONTIENT
`pages`	485	l'éditeur de pages
`admin`	365	le panneau d'administration
`viewer`	182	le viewer
`dupd`	119	l'onglet Mises à jour des données
`upl` + `upload`	104	l'import
`compare` , `about` , `studio` , `tips` …	688	le reste

📌 **À RETENIR**

Une clé d'une langue **existe dans les quatre**, avec les **mêmes jetons** `{…}` . Un test automatique le vérifie à chaque version (`tests/js/test_build_lang_parity.mjs`) : une clé manquante, vide ou dont les jetons changent fait échouer la livraison.

Quelle langue ? Quelle phrase ?

▲ À gauche : le choix de la langue au chargement. À droite : la recherche d'un texte.

- **Le choix.** La clé `iribhm-lang` du navigateur (écrite quand le visiteur clique dans le menu) ; sinon la langue du navigateur (« fr-BE » devient « fr ») ; sinon l'anglais.
- **Le chargement.** L'anglais est lu **d'abord** (toujours), puis la langue choisie. Si ce fichier échoue, l'interface reste en anglais.
- **La propagation.** Un clic dans le menu prévient les **autres pages ouvertes** de la même origine (événement `storage`) : dans Comparer, les panneaux embarqués changent de langue avec la page hôte.
- **Le document.** L'attribut `lang` de la page suit la langue ; `dir="rtl"` est posé pour une langue de droite à gauche (aucune n'est livrée).

▲ Le menu des langues de la barre du haut : un bouton par langue, avec son drapeau et son nom natif.

N	CE QUE C'EST
1	Le bouton **Langue**.
2	Le menu : généré **à partir des langues découvertes**, anglais en premier, puis ordre alphabétique.
3	Le filtre pour daltoniens (voir 16.10).
4	Le thème clair / sombre.

La découverte des langues

La liste des langues **n'est écrite nulle part en dur** : on la découvre, de la plus à la moins directe.

1. `GET api/languages.php` : le serveur liste les fichiers `lang/<code>.json` (le nom doit être de la forme `fr` ou `pt-BR`).
2. Sinon `lang/manifest.json`, l'index que le serveur Python **réécrit** quand la liste change, pour les hébergements statiques.
3. Sinon une liste de secours intégrée au code : anglais, français, espagnol.

> 🔧 **POUR LES CURIEUX**
>
> L'anglais est **toujours ajouté** à la liste, même s'il manquait : le menu ne peut pas être vide. Le néerlandais n'est pas dans la liste de secours : si les deux découvertes échouent, il disparaît du menu.

Les cinq façons d'attacher un texte à un élément

Dans le HTML, un attribut dit quel texte affiche l'élément :

ATTRIBUT	CE QU'IL REMPLIT
`data-i18n="clé"`	le texte de l'élément
`data-i18n-placeholder`	le texte grisé d'un champ de saisie
`data-i18n-title`	l'infobulle
`data-i18n-aria`	l'étiquette lue par un lecteur d'écran (`aria-label`)

ATTRIBUT	CE QU'IL REMPLIT
`data-i18n-html`	un texte qui contient du balisage (rarement)

Le remplacement n'a lieu que si la clé **existe** : sinon on garde le texte écrit dans la page (anglais), jamais une clé nue.

Les mots des outils

Chaque plugin porte ses propres textes : `js/modules/<famille>/<id>/lang/<code>.json`. Au chargement, ils sont rangés **sous** `plugins.<id>` dans le même arbre que les textes de la plateforme.

- La liste des langues d'un plugin est déclarée dans son `plugin.json` (`i18nLanguages`) : on ne tente pas de charger un fichier qui n'existe pas.
- Pour chaque plugin, le **repli est son propre anglais**. Le plugin de capture d'écran « sandboxé » n'a pas d'espagnol ; un visiteur espagnol le voit en anglais, sans erreur.
- Sur le serveur Python, les dictionnaires voyagent **dans la réponse de découverte des plugins** (une requête de moins par plugin et par langue) ; ailleurs, ils sont lus fichier par fichier, et seulement l'anglais et la langue choisie.
- Un plugin qui offre une langue que la plateforme n'a pas ne l'ajoute pas au menu : seule la plateforme décide.

Ajouter une langue

1. **Copiez** `lang/en.json` en `lang/de.json` (exemple : l'allemand) et traduisez les **valeurs**, jamais les clés ni les `{jetons}`.
2. **Rechargez** : la langue apparaît dans le menu (le code `de` est déjà connu : « Deutsch », drapeau). Un code inconnu fonctionne aussi, avec son sigle en majuscules et un drapeau neutre.
3. **Plugins** : ajoutez `lang/de.json` aux outils que vous voulez traduire et complétez `i18nLanguages` ; sans cela, l'anglais.
4. **Vos contenus** : une ligne `DE` apparaît toute seule dans les onglets Identité, Types de données, Mentions légales et dans l'éditeur de pages.
5. **Vérifiez** avec `node tests/js/test_build_lang_parity.mjs` (clés, vides, jetons).

> ℹ️ **À SAVOIR**
>
> Les langues qui s'écrivent de droite à gauche (l'arabe est connu du code) reçoivent `dir="rtl"`, mais aucune n'est livrée : la mise en page n'a pas été validée pour elles dans ce dépôt.

Que se passe-t-il si… ?

SITUATION	CE QUE VOIT LE VISITEUR
une clé manque en français	le texte **anglais**
une clé manque aussi en anglais	la **clé elle-même** (« landing.heroTitle »)

SITUATION	CE QUE VOIT LE VISITEUR
`specimen.plural` n'a pas de ligne `nl`	le nom **anglais** dans la phrase néerlandaise
la valeur d'une langue est vide	l'anglais (une ligne vide compte comme absente)
le fichier d'une langue est illisible	l'interface reste en **anglais**
un plugin n'a pas la langue	l'anglais **du plugin**
un jeton `{xyz}` n'existe pas	`{xyz}` s'affiche tel quel

16.10 Accessibilité, thème et mobile

L'accessibilité d'un outil d'imagerie a des limites (une image 3D reste une image). Voici ce que le code fait réellement.

Avant le premier affichage : le thème

▲ Comment le thème clair arrive avant le premier affichage.

🔧 POUR LES CURIEUX

Un petit script bloquant (`theme-boot.js` , quelques lignes) est placé dans l' `<head>` , après les feuilles de style. Il lit la clé du thème et, si elle vaut « light », change l'attribut `data-theme` **avant** que la page soit peinte. Le panneau d'administration le charge avec sa propre clé (`adm-theme`).

Au clavier et avec un lecteur d'écran

- Un lien d'évitement **« Aller au contenu »** est le premier élément focalisable de chaque page publique (9 pages sur 10 : seule la page de démonstration de widgets, non livrée, n'en a pas) ; il mène au `<main id="main">`.
- Les boutons d'icônes portent un nom (`aria-label` , ou une infobulle traduite) ; le texte vient des fichiers de langue.
- Les fenêtres modales (`Dialog`) sont déclarées `aria-modal` ; quand elles offrent une issue « annuler », Échap ou un clic sur le fond la choisit. Leur texte est écrit par `textContent`.

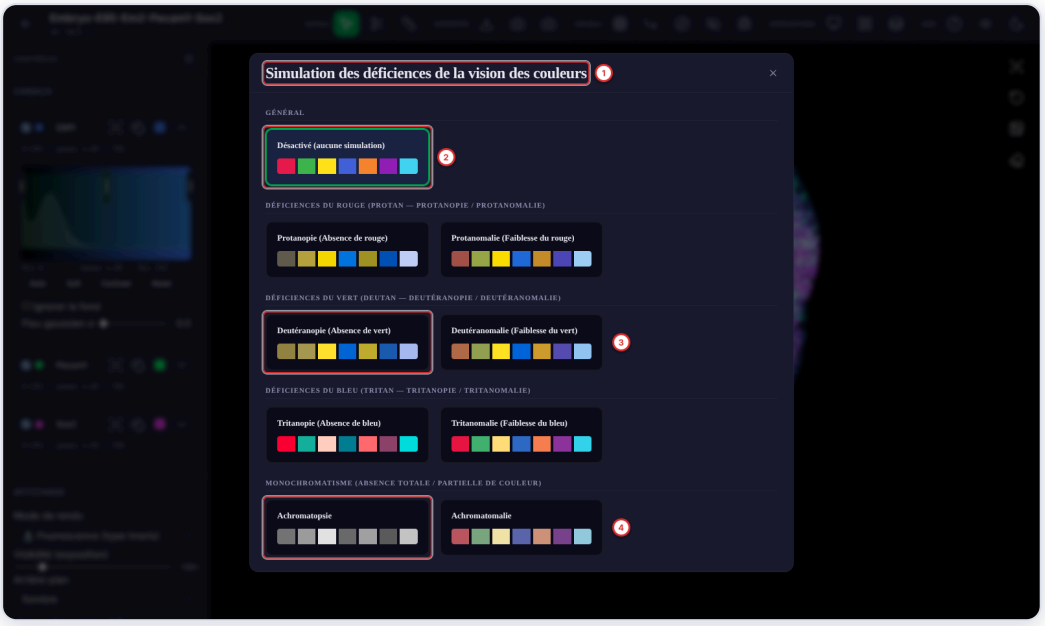
- Les outils du viewer ont des raccourcis (V, C, M, I, D…) : voir le chapitre 12.
- Les feuilles de style contiennent 25 occurrences de `:focus-visible` : l'élément qui a le focus clavier est entouré.

Mouvement réduit

Si le système demande « réduire les animations », **toutes** les animations et transitions CSS du site sont ramenées à presque zéro, et les fonds animés des pages deviennent un dessin fixe. Les mouvements de caméra du viewer (WebGL) ne sont pas du CSS : ils ne sont pas concernés.

Le filtre pour daltoniens

Le bouton en forme d'œil de la barre du haut ouvre le menu **« Simulation des déficiences de la vision des couleurs »** : 9 choix en 5 groupes.

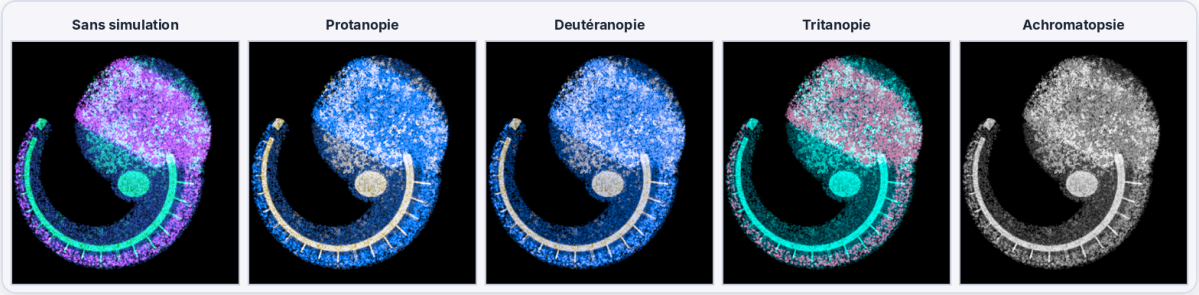


▲ Le menu de simulation : pour chaque choix, sept teintes de test déjà filtrées.

N	CE QUE C'EST
1	Le titre : c'est bien une **simulation**.
2	Le choix actif (ici « Désactivé »).
3	Deutéranopie : plus de sensibilité au vert.
4	Achromatopsie : plus aucune couleur.

GROUPE	DÉFICIENCE COMPLÈTE	DÉFICIENCE PARTIELLE
Rouge (protan)	protanopie	protanomalie
Vert (deutan)	deutéranopie	deutéranomalie
Bleu (tritan)	tritanopie	tritanomalie
Monochromatisme	achromatopsie	achromatomalie

Le filtre s'applique à **toute la page**, y compris le viewer 3D, pour voir ce que verrait une personne atteinte :



▲ Le même embryon (jeu de démonstration) avec les canaux DAPI, Pecam1, Sox2, sous quatre simulations.

🔢 **MICRO-EXEMPLE**

Dans cette démonstration, le vert et le magenta de la vue normale deviennent, en protanopie et en deutéranopie, des **bleus et des beiges** : l'opposition rouge-vert qui séparait les deux marquages disparaît. En tritanopie, le vert devient cyan et le magenta rose. C'est exactement ce qu'une figure de publication doit vérifier.

▲ Les mêmes sept teintes, calculées par le script d'après les matrices du code.

🔧 **POUR LES CURIEUX**

- **Le calcul.** Chaque simulation est une matrice 3×3 de **Machado, Oliveira et Fernandes (2009)**, appliquée à la lumière **linéaire** (`color-interpolation-filters="linearRGB"`). Les « …opies » sont la sévérité 1,0 (cône absent) ; les « …omalies », la sévérité 0,5. Chaque ligne fait 1 : un gris reste gris. L'achromatopsie remplace les trois canaux par la luminance (0,2126 R + 0,7152 V + 0,0722 B) ; l'achromatomalie en est le mélange à moitié.
- **Où il s'accroche.** Le filtre est posé sur l'élément racine `<html>` , pas sur `<body>` : un filtre sur un autre élément en ferait le référentiel de ses descendants `position: fixed` (barre du haut, fenêtres, notifications), qui défileraient avec la page.
- **Pendant le choix.** Le filtre de la page est levé tant que le menu est ouvert, pour que les aperçus (qui ont leur propre filtre) ne soient pas filtrés deux fois.
- **Mémoire.** La clé `iribhm-colorblind` du navigateur ; les panneaux de Comparer la suivent par l'événement `storage` .

⚠️ **ATTENTION**

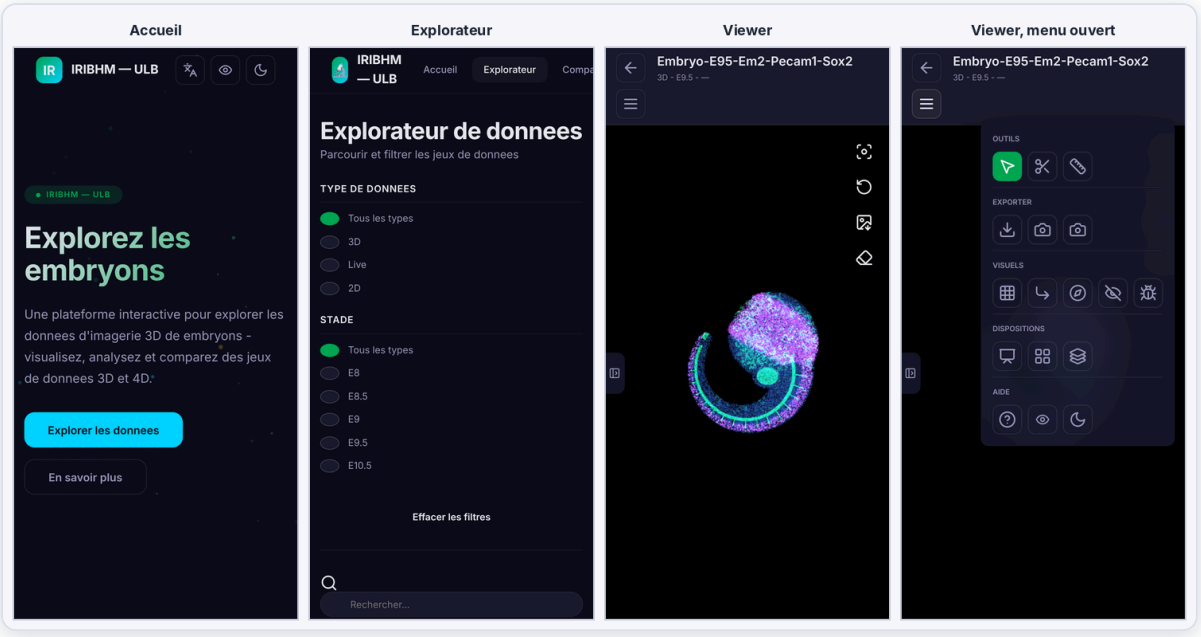
Ce n'est pas une correction. Le filtre ne recolore pas la page pour la rendre lisible à une personne daltonienne : il **montre** ce qu'elle voit, pour que vous jugiez *vos* figures. Il n'y a pas non plus de palette « adaptée aux daltoniens » livrée : les couleurs de canaux sont libres. Conseil du chapitre 11 : préférez vert + magenta, ou bleu + jaune, au rouge + vert.

ℹ️ **À SAVOIR**

La simulation est un effet d'**affichage** : les exports (capture, figure) sont faits à partir du rendu, pas de l'écran filtré. Pour juger une figure destinée à l'impression, exportez-la puis regardez l'image sous une simulation.

Sur téléphone et tablette

Le site est **utilisable sur téléphone**, avec des limites que le viewer ne peut pas lever.



▲ L'accueil, l'Explorateur, le viewer et son menu ☰, sur un écran de 390 pixels de large.

- **Tactile.** Un doigt fait tourner le volume ; **deux doigts** le déplacent et le rapprochent (pincer). Aucune touche de modification n'est nécessaire.
- **Barre d'outils repliée.** Quand les boutons ne tiennent plus sur une ligne, ils se rangent derrière un bouton ☰ (troisième et quatrième images).
- **Poignées plus larges** sur les écrans tactiles : les poignées de l'histogramme passent de 12 à 22 pixels.
- **Accueil.** Sous 900 pixels, les liens de la barre disparaissent (la barre ne garde que langue, filtre et thème) ; on passe par les boutons de la page.
- **Studio.** Sous 720 pixels, ses panneaux s'empilent sous l'image.

⚠️ **ATTENTION**

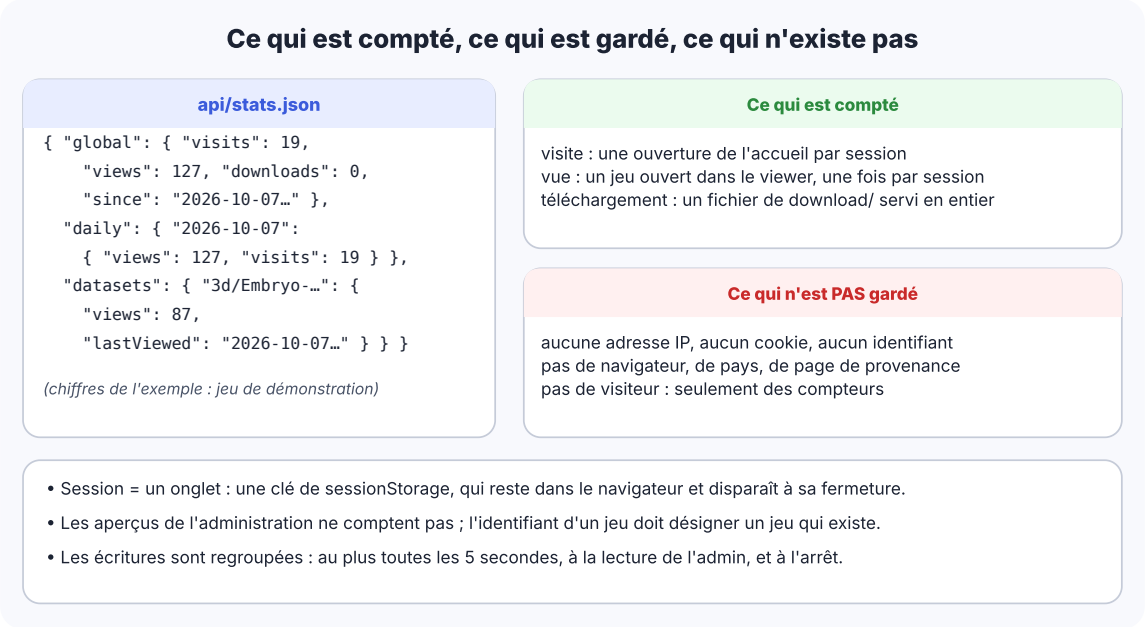
Les limites sont celles du matériel. Le viewer exige WebGL2 ; la mémoire graphique d'un téléphone est comptée au plus juste (chapitre 10). Un gros jeu à qualité maximale peut y être refusé avec un message : le site passe au niveau plus grossier plutôt que de planter.

Polices

La seule dépendance distante du site est **Google Fonts** (Inter, JetBrains Mono). Le lien est chargé **en différé** (`media="print"` , activé par `font-loader.js`) : la page ne l'attend pas pour s'afficher. Hors ligne ou si la police est bloquée, le navigateur utilise sa police sans empattement : le site reste lisible.

16.11 Les statistiques d'usage

Un laboratoire veut savoir si ses jeux de données sont regardés. Lumen3D le dit **sans rien savoir de vous**.



▲ Ce qui est compté, et ce qui n'est pas gardé.

Ce qui est compté

COMPTEUR	QUAND IL MONTE
Visites	une fois par **session de navigation** (un onglet), sur la **première page publique** ouverte, quelle qu'elle soit (accueil, explorateur, viewer, page 2D, comparaison, à propos, mentions légales, page personnalisée…)
Vues	une fois par session et par jeu de données, quand un jeu s'ouvre (volume dans le viewer ou photographie dans la page 2D)
Téléchargements	quand un fichier du dossier `download/` d'un jeu existant (celui que propose le Download Center) est servi **en entier** : un `GET` complet, ni un `HEAD` , ni une reprise partielle (`Range`)

🔧 POUR LES CURIEUX

- Les balises sont des requêtes `POST` envoyées par `navigator.sendBeacon`, sans attendre de réponse (elles survivent à la fermeture de l'onglet). Un `GET` est **refusé** (405) : une image tierce ne peut pas gonfler vos chiffres.
- « Une fois par session » veut dire : une clé de `sessionStorage` (`lumen_visit`, `lumen_view_<jeu>`) mémorise que c'est fait. Elle reste dans le navigateur et disparaît à la fermeture de l'onglet.
- Les **aperçus de l'administration** (`mode=admin`) ne comptent pas : modifier un jeu ne gonfle pas ses vues. Le panneau d'administration ne compte aucune visite, pas plus qu'une page intégrée dans une autre (panneau de la page Comparer, aperçu de l'administration, éditeur de pages).
- Un visiteur qui arrive **directement** sur un jeu (lien reçu par courrier) compte une **visite** et une **vue** : c'est un choix délibéré, la visite est comptée sur la page d'arrivée, quelle qu'elle soit.
- Les téléchargements sont comptés **par les deux serveurs**, avec la même règle, au moment où le fichier est servi (une reprise partielle n'est pas recomptée). Sur un hébergement Apache/PHP, le `.htaccess` racine fait passer la requête par `api/download.php`, qui la compte puis redirige (302) vers le même fichier marqué `?lumen_dl=1`, qu'Apache sert lui-même (reprise, plages et en-tête « pièce jointe » inchangés) ; sous `php -S`, c'est `router.php` qui compte.

Ce qui est gardé : `api/stats.json`

Rien d'autre que des **nombres** et une date :

- `global` : visites, vues, téléchargements, et la date de départ (`since`) ;
- `daily` : une ligne par **jour** (jamais purgée) ;
- `datasets` : par jeu (`3d/Embryo-…`), ses vues, ses téléchargements, sa dernière consultation.

Aucune adresse IP, aucun cookie, aucun identifiant de visiteur, aucun navigateur, aucun pays. Le fichier est protégé des mises à jour (votre historique survit) et n'est jamais servi (il vit sous `api/`).

🔧 POUR LES CURIEUX

- L'identifiant d'un jeu doit désigner un jeu qui **existe** : sinon la balise ne compte que globalement. Sans cette vérification, n'importe qui aurait pu ajouter des milliers de faux jeux au fichier.
- Les chiffres s'accumulent en mémoire et sont écrits **au plus toutes les 5 secondes**, à la lecture par l'administration et à l'arrêt du serveur ; l'écriture est atomique (un fichier temporaire remplace l'ancien).

Se protéger de la fraude : deux seaux à jetons

▲ Chaque balise coûte un jeton à deux seaux ; vide, elle reçoit 429.

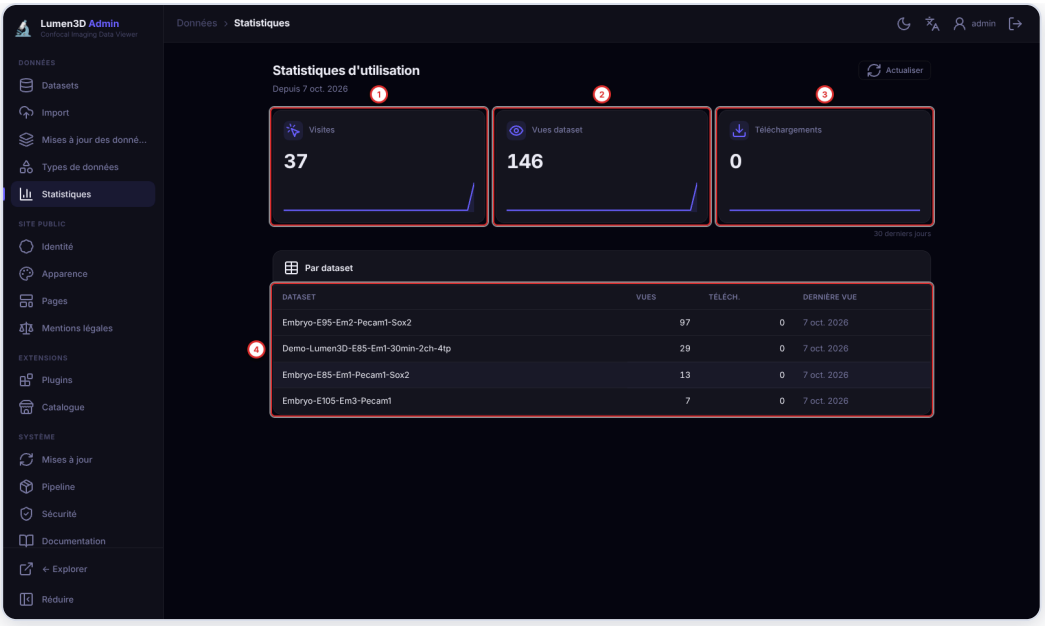
> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un guichet à tickets.** Chaque visiteur a un petit carnet de 60 tickets, qui se recharge à raison d'un par seconde. Le guichet entier a un carnet de 600, rechargé de 20 par seconde. Sans ticket dans l'un ou l'autre, on n'est pas servi.

L'adresse du visiteur sert **uniquement** à choisir un emplacement parmi 4 096 d'une table en mémoire (son empreinte SHA-256 est réduite à un numéro) ; elle n'est **ni écrite sur le disque, ni dans** `stats.json`. Même la table ne grandit jamais : deux adresses qui tombent sur le même emplacement repartent simplement d'un carnet neuf.

> ⚠️ **ATTENTION**
>
> Les compteurs ne gardent aucune donnée personnelle. Cela ne dit rien des **journaux d'accès** du serveur web lui-même (hébergeur, Apache…), qui peuvent contenir des adresses IP selon sa configuration : c'est à déclarer dans vos mentions légales (16.7).

Les lire

L'onglet **Statistiques** présente trois cartes (avec le tracé des **30 derniers jours** dessiné à la main en SVG, sans bibliothèque de graphiques) et un tableau **Par dataset** que l'on trie en cliquant un en-tête.



▲ L'onglet Statistiques (chiffres de test du jeu de démonstration).

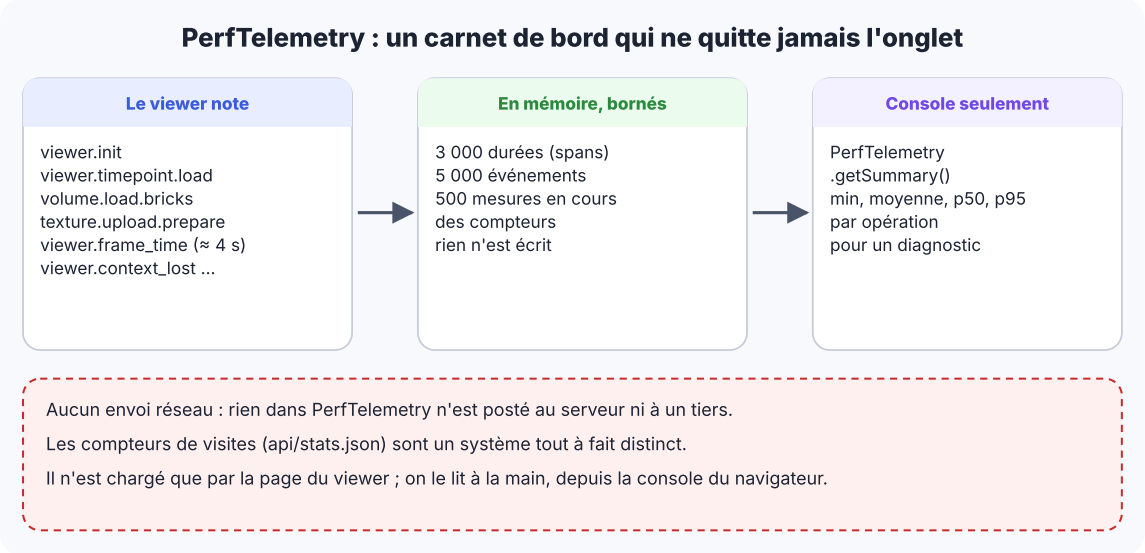
N	CE QUE C'EST
1	**Visites** (une par onglet, sur n'importe quelle page publique), avec la courbe des 30 derniers jours.
2	**Vues dataset** : l'indicateur le plus parlant.
3	**Téléchargements**.
4	Le détail par jeu : vues, téléchargements, dernière vue. Cliquez un en-tête pour trier.

Sous chaque chiffre, une ligne en rappelle la définition (« Une par onglet de navigateur, sur n'importe quelle page publique (hors panneau d'administration et pages intégrées). », « Une par dataset ouvert dans un onglet de navigateur. », « Un par téléchargement complet d'un fichier du dossier download/ d'un dataset. »).

Changer le **nom** d'un type de données ne change aucun de ces chiffres. Lors du passage à l'ancien vocabulaire des types, les compteurs des anciennes clés sont **additionnés** à ceux des nouvelles.

16.12 `PerfTelemetry` : le carnet de bord du viewer

À ne pas confondre avec les statistiques. `PerfTelemetry` est un chronomètre interne que le viewer remplit pour **diagnostiquer** ses performances.



▲ PerfTelemetry : un carnet de bord qui ne quitte jamais l'onglet.

📌 **À RETENIR**

Rien n'est envoyé nulle part. Le carnet vit dans la mémoire de l'onglet, borné (3 000 durées, 5 000 événements, 500 mesures ouvertes à la fois), et disparaît quand l'onglet se ferme.

Ce que le viewer note

MESURE	CE QU'ELLE CHRONOMÈTRE
`viewer.init`	l'ouverture de la page jusqu'au volume prêt
`viewer.timepoint.load`	le chargement d'un instant d'une série
`volume.load.bricks`	le chargement des briques d'une qualité
`texture.upload.prepare`	la préparation de la texture avant envoi à la carte graphique
`viewer.frame_time`	la fluidité : durée moyenne, médiane et 95e centile des images (toutes les 4 secondes, après 45 images)
`viewer.context_lost` / `restored`	une perte et une reprise du contexte graphique

Le lire

Dans la console du navigateur (F12) : `PerfTelemetry.getSummary()`. On obtient, **par opération**, le nombre d'occurrences, le minimum, la moyenne, le maximum, la médiane (p50) et le 95e centile (p95), plus les 80 derniers événements. C'est l'outil d'un développeur, pas d'un visiteur.

ℹ️ **À SAVOIR**

Seule la page du viewer charge `PerfTelemetry`. La ligne de progression que vous voyez en chargeant un volume (« 37 % — niveau 2 ») est une autre chose : un affichage, pas un journal.

16.13 Récapitulatif

Je veux… où dois-je aller ?

JE VEUX…	J'UTILISE
changer le nom, l'organisation, le logo	**Identité**
changer le mot « embryon » partout	**Identité** › *Terminologie*
renommer « 3D », « 2D », « Live »	**Types de données**
changer le vert du site, la police, les arrondis	**Apparence**
modifier l'accueil, la page À propos	**Pages**
créer une page « Protocoles »	**Pages** › **Nouvelle page**
rédiger les mentions légales	**Mentions légales**, puis cocher la case dans **Identité**
ajouter une langue	déposer `lang/<code>.json`
donner une couleur aux canaux d'après leur nom	`channelColorPresets` dans `instance.json`
savoir combien de personnes regardent mes données	**Statistiques**

Les erreurs fréquentes, expliquées

CONSTAT	EXPLICATION
« Mon changement d'Apparence n'apparaît pas »	il n'est appliqué qu'après **Enregistrer** ; videz le cache si le navigateur garde l'ancienne `theme.css`
« Le lien Mentions légales a disparu »	la case est décochée par défaut dans **Identité**
« Le néerlandais parle d'embryos »	la ligne **NL** de *Terminologie* est vide
« Ma page est sauvée mais invisible »	le brouillon est enregistré ; il faut **Publier**
« L'éditeur dit que la page est ouverte ailleurs »	un autre onglet l'édite : reprenez la main ou fermez l'autre

Pour aller plus loin

📖 **POUR ALLER PLUS LOIN**

- **Pas à pas** : Guide de l'administrateur, chapitres 6 (Types de données), 7 (Statistiques), 8 (Identité), 9 (Apparence), 10 (Pages), 11 (Mentions légales).
- **Les tests** qui verrouillent ce chapitre : `tests/js/test_build_lang_parity.mjs` (parité des langues), `tests/js/test_plugin_lang.mjs` (langues des plugins), `tests/test_dev_server_site.py` (magasin de configuration, marqueurs `{{SITE:…}}`, compilation du thème), `tests/js/test_admin_pages_save.mjs` (brouillon, révision, verrou), `tests/js/test_admin_branding_merge.mjs` (enregistrement par chemins), `tests/test_v3_minor_telemetry.py` (seaux à jetons).
- **La sécurité** de ces mécanismes (listes blanches, politique de sécurité du contenu, droits des fichiers) : chapitre 19. Les **plugins** et leurs langues : chapitre 15. Les **mises à jour** qui préservent `config/` : chapitre 18.

17. Les données en profondeur : fichiers, formats, migrations, import

- Un jeu de données est **un dossier** : chaque fichier a un auteur (le pipeline, l'opérateur ou la plateforme), une raison d'être et, le plus souvent, une empreinte qui permet de le contrôler.
- Il existe **cinq formats de données**. Une mise à jour de données passe de l'un à l'autre **sur place**, par petites unités notées dans un journal, avec un échange final atomique.
- L'import dans le navigateur envoie un dossier **bloc par bloc**, vérifié et reprenable, dans une zone privée ; les fichiers qui rendent le jeu ouvrable partent en premier.

Les chapitres 7 et 8 ont montré **à quoi ressemblent** les briques, `planes/`, `mips/` et `metadata.json`. Le chapitre 14 a montré **où cliquer** dans l'administration. Ce chapitre ouvre le capot : il dit **exactement** ce que contient chaque fichier, qui l'écrit, comment il change de format et comment il arrive sur le serveur.

Ce chapitre, en une carte

Trois questions, trois parties : de quoi est fait un jeu de données, comment il change de format, comment il arrive.

A — Les fichiers

De quoi est fait un jeu de données ?

17.1 les trois arbres annotés
17.2 qui écrit quoi
17.3 metadata.json, champ par champ
17.4 manifestes, index, tracks.json
17.5 catalogue et vocabulaire
17.6 download/ et gallery/

B — Les formats et les migrations

Comment un jeu change-t-il de format ?

17.7 les formats 1 à 4
17.8 trois migrations
17.9 unités et journal
17.10 deux exécutants
17.11 finalize
17.12 régulateur, hébergeurs lents

C — L'import dans le navigateur

Comment un jeu arrive-t-il sans SFTP ?

17.13 le parcours
17.14 la liste blanche
17.15 les paliers
17.16 blocs et journal
17.17 états et purge
17.18 valider, publier, remplacer

Fil rouge : un jeu de données est un dossier, et chaque fichier a un auteur, une raison d'être et un contrôle d'intégrité.

▲ La carte du chapitre : trois parties, dix-huit sections.

ℹ️ **À SAVOIR**

Tous les exemples chiffrés viennent des **jeux de démonstration** (synthétiques, produits par le vrai pipeline) ou d'**essais réels** faits sur des jeux de test dans un dossier temporaire. Aucune donnée du laboratoire n'est montrée.

17.1 Un jeu de données, un dossier : les trois arbres

Un jeu de données vit dans `DATA_WEB/<type>/<dossier>/`. Le **type** (`3d`, `2d` ou `live`) est le nom du dossier parent ; le dossier lui-même est l'identifiant du jeu : `3d/Embryo-E95-Em2-Pecam1-Sox2`.

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Une valise de voyage.** On l'ouvre et chaque objet a sa place : la fiche d'identité (`metadata.json`), la photo d'identité (la vignette), le contenu (les briques) et des souvenirs à emporter (`download/`). Les pastilles de couleur des figures disent **qui a rangé** quoi dans la valise.

L'arbre d'un jeu de données 3D (jeu de démonstration)

Les pastilles disent qui crée chaque élément ; les tailles sont celles du disque.

3d/Embryo-E95-Em2-Pecam1-Sox2/	549 Mo		un dossier = un jeu de données
├ metadata.json	1,8 ko	🔵🟠	la carte d'identité (voir 17.3)
├ thumbnail.webp	33,6 ko	🔵🟠	vignette ; l'opérateur peut la redéfinir
├ **bricks/**	9,3 Mo	🔵🟢	pyramide de briques v3 (format 4)
│ ├ manifest.json	3,7 ko	🔵🟢	niveaux, histogrammes, empreinte de l'index
│ ├ index.bin	7,8 ko	🔵🟢	paquet + début + longueur de chaque brique
│ ├ l0/c0/p00000.bin p00001.bin …	7,9 Mo	🔵🟢	niveau 0 (natif), un dossier par canal
│ └ l1/ l2/ l3/	1,4 Mo	🔵🟢	niveaux de plus en plus grossiers
├ **planes/**	8,6 Mo	🔵🟢	coupes XY rapides (format 2)
│ ├ manifest.json	360 o	🔵🟢	lumen-planes-v1
│ └ z00000.bin … z00111.bin	112 fichiers	🔵🟢	un fichier par plan Z
├ **mips/**	473 ko	🔵🟢	MIP par couche de 64 plans (format 3)
│ └ manifest.json l00000/1.bin		🔵🟢	lumen-mips-v1
├ **download/**	530 Mo	🔵	toujours servi en pièce jointe
│ ├ ….ims ….tif _web.zip README		🔵	original (lien physique), composite, archive
│ └ …_C1_DAPI_MIP.png (un par canal)		🔵	aperçus de projection
├ **gallery/**	(absent ici)	🟠	images jointes : ≤ 40 images de ≤ 8 Mio
│ └ a.png thumbs/a.png.webp …		🟠	extension d'après les octets magiques

🔵 écrit par le pipeline 🟠 saisi / ajouté par l'opérateur 🟢 posé par la plateforme (migration, import)

▲ L'arbre d'un jeu 3D : de la fiche d'identité aux briques, planes, mips et téléchargements.

Trois choses à remarquer dans cet arbre :

- les dossiers `bricks/`, `planes/` et `mips/` portent **une pastille bleue et une verte** : le pipeline les écrit directement (format 4) ou la plateforme les a construits par une migration (section 17.8) ;
- `gallery/` n'existe que si l'opérateur a joint des images ;
- tout `DATA_WEB/` est servi tel quel, sauf `download/` qui est **toujours** servi en pièce jointe (section 17.6).

L'arbre d'une série temporelle « live » (jeu de démonstration)

Même idée qu'un jeu 3D, mais chaque image (tNNN) a son propre arbre ; le suivi vit à la racine.

live/Demo-Lumen3D-E85-Em1-30min-2ch-4tp/	5,8 Mo		série temporelle : 4 images, 2 canaux
├ metadata.json	7,7 ko	●●	+ timeline, tracking, registration…
├ thumbnail.webp	10,3 ko	●●	
├ tracks.json (+ .gz)	32,7 ko / 11,0 ko	●	le suivi : 50 cellules (JSON + copie gzip)
├ model.glb	412 ko	●	surface 3D du suivi (glTF binaire)
├ **bricks/**	233 ko	●●	un arbre de briques PAR image
│ ├ manifest.json	10,7 ko	●●	timepoints[] : un index.bin par image
│ ├ t000/ index.bin l0/c0/ c1/	52,1 ko	●●	l'image 0 (index de 108 octets)
│ └ t001/ t002/ t003/	170 ko	●●	les autres images, même forme
├ **planes/**	251 ko	●●	un sous-dossier par image
│ └ t000/ z00000.bin … z00023.bin	60,5 ko	●●	24 plans par image
├ **mips/**	31,6 ko	●●	
│ └ t000/ manifest.json l00000.bin	7,7 ko	●●	une couche (24 plans < 64)
└ download/	4,8 Mo	●	l'.ims contient les 4 images

● écrit par le pipeline ● saisi / ajouté par l'opérateur ● posé par la plateforme (migration, import)

▲ L'arbre d'une série temporelle : un arbre de briques par image, le suivi à la racine.

Dans une série `live`, **chaque image** a son propre arbre : `bricks/t000/`, `bricks/t001/`… avec son `index.bin`. Les dossiers `planes/` et `mips/` suivent le même découpage (`planes/t000/`, `mips/t000/`). Le suivi cellulaire (`tracks.json`, `model.glb`) est à la racine du jeu : il décrit toute la série.

L'arbre d'une photographie 2D (jeu de démonstration)

Pas de briques, pas de canaux : trois copies d'une image et une carte d'identité.

2d/DLL4xCD1-E95-x3.2-240913-1/	8,5 Mo		une photographie de stéréomicroscope
├ metadata.json	1,3 ko	●●	bloc image{}, pixelSizeUm, acquisition{}…
├ image.webp	191 ko	●	la photo affichée (WebP avec perte, q90)
├ preview.webp	10,6 ko	●	640 px, peinte en premier
├ thumbnail.webp	12,8 ko	●	carrée, pour l'explorateur
├ **download/**	8,3 Mo	●	le TIFF d'origine (intact) et une notice
│ └ ….tif (original) README.txt		●	
└ gallery/	(absent ici)	●	possible aussi pour une photo

● écrit par le pipeline ● saisi / ajouté par l'opérateur ● posé par la plateforme (migration, import)

▲ L'arbre d'une photographie 2D : trois copies d'une image et une carte d'identité.

Une photographie 2D n'a ni briques ni canaux. Le fichier `image.webp` est la copie d'affichage ; le TIFF d'origine reste intact dans `download/` (c'est un lien physique : il n'occupe pas de place en double).

Les dossiers de chantier

Pendant une opération, la plateforme construit **à côté** du jeu, puis échange. Ces dossiers temporaires portent un nom reconnaissable ; un dossier dont le nom commence par un point n'est **jamais** listé dans le catalogue.

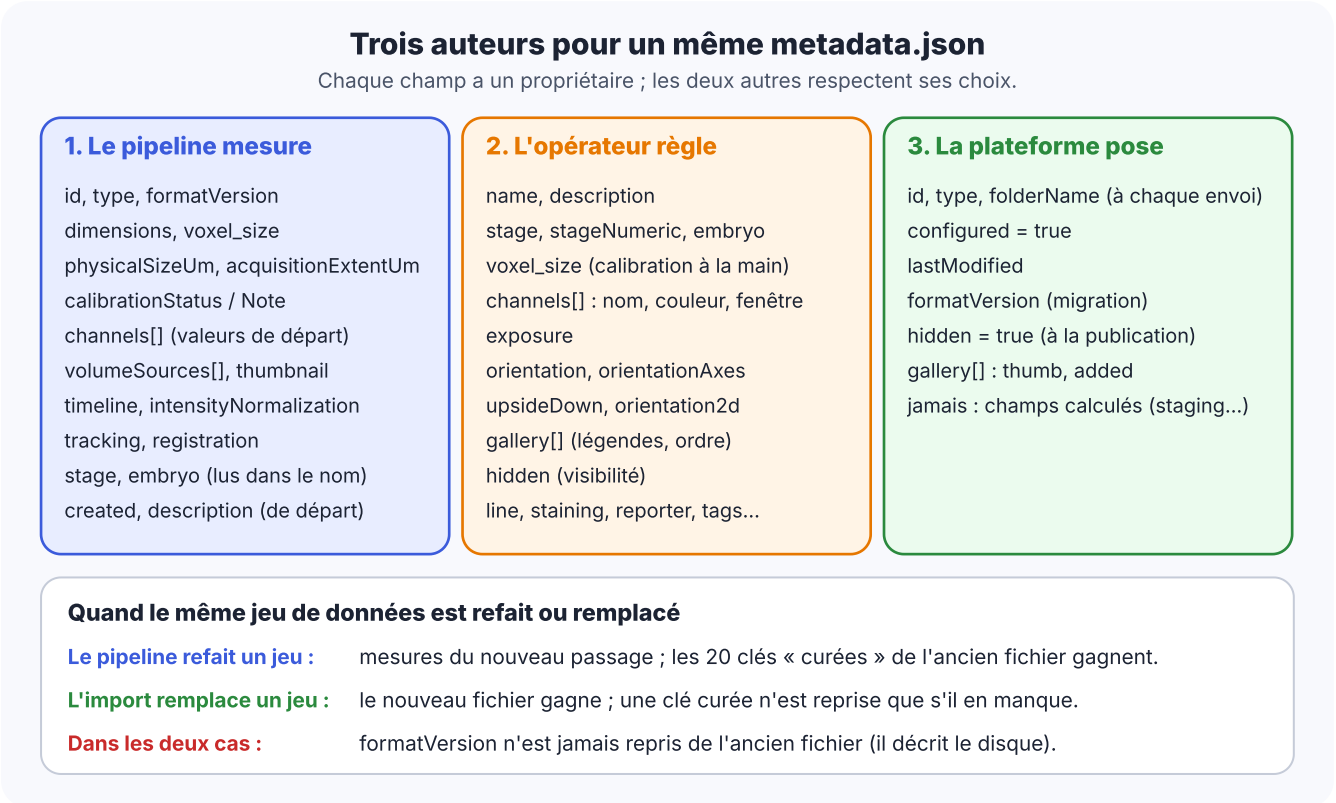
NOM	CRÉÉ PAR	DISPARAÎT QUAND
`.planes-incoming/`, `.mips-incoming/`, `.bricks-incoming/`	une migration (assemblage)	l'échange est fait
`.planes-old/`, `.mips-old/`, `bricks.v2-old/`	une migration (ancien état mis de côté)	la version est montée
`.incoming-<jeu>-<date>/`	l'import (copie entre deux disques)	le renommage final, ou au démarrage suivant
`.replaced-<jeu>-<date>/`	l'import (ancien jeu remplacé)	la publication est terminée
`.swap-in-progress`	le pipeline (installation d'un nouveau passage)	l'installation est terminée
`.lumen-tmp-*`	toute écriture « tout ou rien » d'un fichier	le renommage final

📌 **À RETENIR**

Un fichier n'est jamais écrit « sur place » : on écrit un fichier temporaire voisin, puis on le **renomme**. Un renommage est atomique : le visiteur voit l'ancien fichier ou le nouveau, jamais la moitié de chacun.

17.2 Qui écrit quoi : trois auteurs

Un même `metadata.json` est touché par trois auteurs. Pour que chacun respecte le travail des autres, **chaque champ a un propriétaire**.



▲ Le pipeline mesure, l'opérateur règle, la plateforme pose ; deux règles de fusion.

Le pipeline mesure

Le pipeline écrit tout ce qui se **mesure** dans l'acquisition : dimensions, tailles de voxel, canaux de départ, sources de volume, et pour une série : `timeline`, `intensityNormalization`, `tracking`, `registration`. Il écrit aussi la valeur de départ de champs que l'opérateur pourra corriger (nom, stade, description).

L'opérateur règle

L'éditeur de l'administration (chapitre 14, section 14.4) envoie les champs qu'il modifie. Le serveur les **fusionne** dans le fichier existant, sous un verrou, puis impose lui-même `id`, `type`, `folderName`, `configured` et `lastModified`. La galerie est **réconciliée** avec le dossier `gallery/` : l'envoi décide de l'ordre et des légendes, le dossier décide des fichiers qui existent.

Les vingt clés « curées »

Quand un jeu est **refait** ou **remplacé**, une liste fixe de vingt clés est protégée, dans le pipeline comme dans l'import (`CURATED_KEYS`) :

FAMILLE	CLÉS
Identité	`name`, `description`, `stage`, `stageNumeric`, `embryo`, `line`, `staining`, `reporter`, `tags`, `notes`, `created`
Affichage	`hidden`, `orientation`, `orientationAxes`, `upsideDown`, `defaultView`, `exposure`
Liens et images	`gallery`, `linkedTrackingId`, `relatedIds`

Deux directions de fusion

La protection ne marche pas pareil dans les deux cas, et c'est un détail qui compte.

SITUATION	QUI GAGNE	CONSÉQUENCE
Le pipeline refait un jeu publié	l'**ancien** fichier, pour les 20 clés curées	votre nom corrigé, votre orientation et votre galerie survivent
L'import remplace un jeu publié	le **nouveau** fichier ; une clé curée n'est reprise **que si le nouveau ne la contient pas**	`orientation` et `gallery` survivent ; un `name` que le nouveau fichier énonce **gagne**

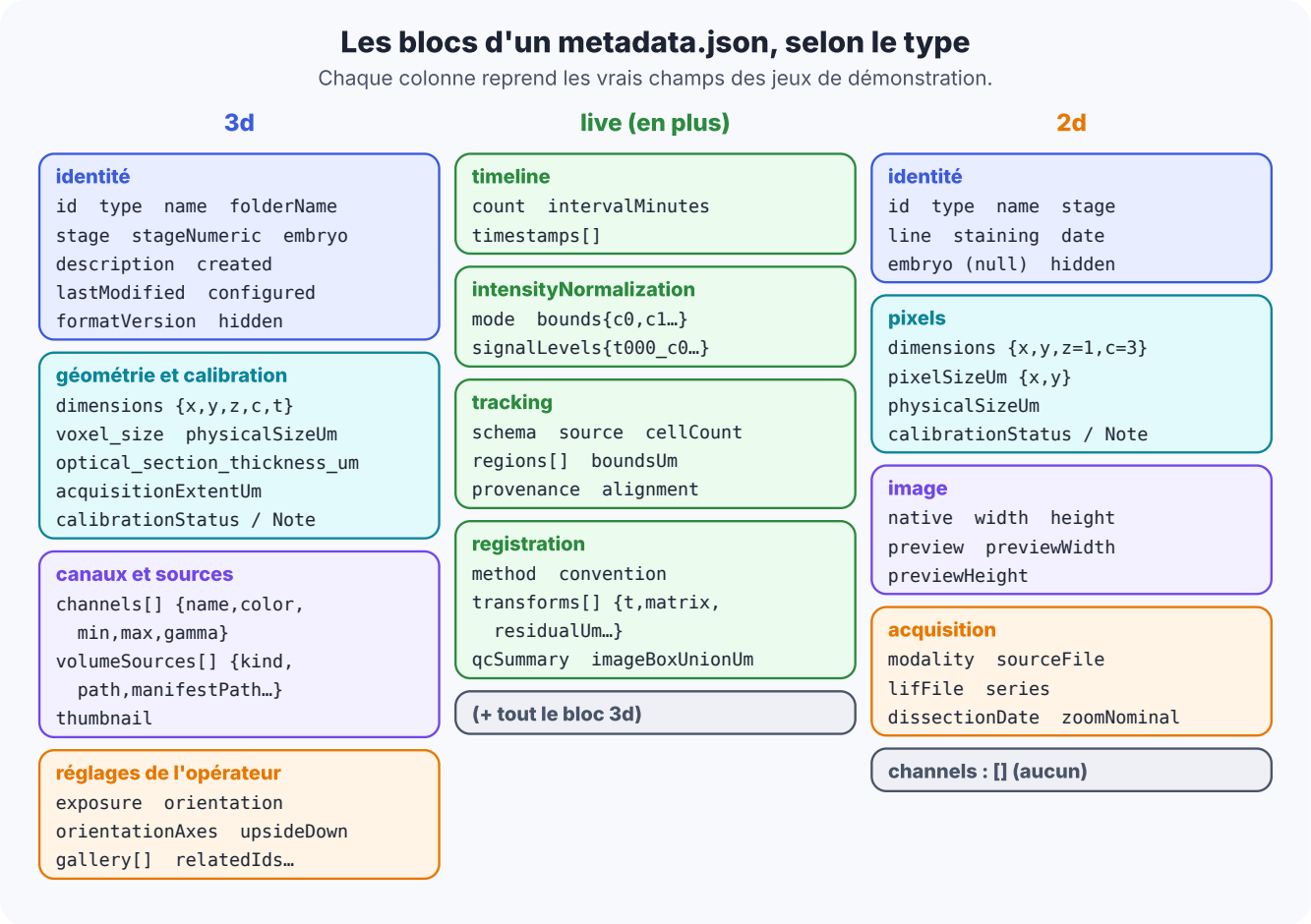
🔢 **MICRO-EXEMPLE**

Un essai réel (moteur d'import, dossier temporaire). Un jeu publié a été édité : `orientation`, une galerie, un nom corrigé. On remplace par le dossier du pipeline. La réponse du serveur : `carriedKeys = [gallery, orientation]` et `carriedGallery = true`. Le nom, lui, est **revenu à celui du pipeline** : le nouveau `metadata.json` le contient, donc il l'emporte. Et `formatVersion` vaut **4** (celui du nouveau fichier), jamais l'ancien chiffre.

⚠️ **ATTENTION**

`formatVersion` n'est **jamais** repris de l'ancien fichier, dans aucun des deux cas : il décrit ce qui est réellement sur le disque.

17.3 `metadata.json`, champ par champ



▲ Les blocs d'un metadata.json : le même squelette, trois formes.

Les tableaux qui suivent donnent, pour chaque champ, **ce qu'il veut dire** (avec la valeur réelle du jeu de démonstration) et **qui l'écrit**.

Identité et état

CHAMP	CE QUE C'EST	ÉCRIT PAR
`id`	`"3d/Embryo-E95-Em2-Pecam1-Sox2"` : type et dossier	pipeline ; **imposé** à chaque enregistrement
`type`	`3d`, `2d` ou `live` ; toujours égal au dossier parent	pipeline ; imposé
`name`	nom affiché	pipeline (nom du dossier) puis opérateur
`folderName`	nom du dossier	pipeline ; imposé
`stage`, `stageNumeric`	`"E9.5"` et `9.5`, lus dans le nom (chapitre 8)	pipeline ; l'éditeur recalcule `stageNumeric` quand vous changez `stage`
`embryo`	`"Em2"`	pipeline ; opérateur

CHAMP	CE QUE C'EST	ÉCRIT PAR
`description`	phrase de départ, par exemple « Confocal imaging stack: E9.5 fixed embryo, 112 slices, 3 channels. »	pipeline ; opérateur
`created` , `lastModified`	dates ISO ; `created` est protégée ; `lastModified` est remise à jour à chaque enregistrement	pipeline ; plateforme
`configured`	`true` ; sert à décider si le jeu entre au catalogue (section 17.5)	pipeline ; imposé
`formatVersion`	`4` ou `5` ; absent = 1	pipeline, ou fin d'une migration
`hidden`	`true` = invisible du public, visible de l'administration	opérateur ; **l'import publie toujours masqué**

Géométrie et calibration

CHAMP	CE QUE C'EST	ÉCRIT PAR
`dimensions`	`{x:768, y:576, z:112, c:3, t:1}` : voxels en X, Y, Z, nombre de canaux, nombre d'images	pipeline
`voxel_size`	`{x:1.2, y:1.2, z:3.0}` en µm	pipeline ; l'opérateur peut la corriger à la main
`physicalSizeUm`	`{x:921.6, y:691.2, z:336.0, sliceThickness:3.0, voxelX, voxelY, voxelZ}` : étendue réelle en µm	pipeline
`optical_section_thickness_um`	`3.0` : épaisseur de coupe, déclarée **égale au pas en Z**	pipeline
`acquisitionExtentUm`	`{unit:"um", min:[0,0,0], max:[921.6,691.2,336]}` : la boîte d'acquisition, dans le repère d'Imaris	pipeline
`calibrationStatus` , `calibrationNote`	`exact` ou `metadata-missing` , avec une phrase d'explication	pipeline

🔧 **POUR LES CURIEUX**

Pourquoi déclarer `sliceThickness` égal au pas en Z ? Le viewer modélise la profondeur comme (D − 1) pas plus une épaisseur de coupe. Sans valeur explicite, il devine `min(pas Z, voxel X)` , ce qui sous-estime la profondeur d'un empilement anisotrope (329,50 µm au lieu des 333,87 µm qu'Imaris déclare). Or tout ce qui est enregistré dans les coordonnées d'Imaris (les cellules suivies) doit retomber sur les bons voxels.

Le **statut de calibration** a deux vocabulaires : le pipeline n'écrit que `exact` ou `metadata-missing` ; le viewer recalcule son propre statut à partir de ce qu'il reçoit (`exact` seulement si une épaisseur de coupe est présente, sinon `estimated` ou `metadata-missing` , chapitre 9).

Canaux et sources

CHAMP	CE QUE C'EST	ÉCRIT PAR
`channels[]`	`{name:"DAPI", color:"#3D7BFF", min:0.0, max:1.0, gamma:1.0}` ; l'éditeur y ajoute `active`	pipeline (valeurs de départ), puis opérateur
`volumeSources[]`	`{kind:"bricks", label, priority:-1, available:true, multiscale:true, path, manifestPath}` : où le viewer trouve les briques ; les genres connus sont `bricks` et `webstack`	pipeline
`thumbnail`	chemin de la vignette, ou `null` ; le catalogue le **recalcule** d'après l'existence du fichier	pipeline

Les réglages de canal sont conservés quand un jeu est refait **tant que le nombre de canaux n'a pas changé** : avec un autre jeu de canaux, les anciens réglages appartiendraient à d'autres données.

Réglages de l'opérateur

CHAMP	CE QUE C'EST	ÉCRIT PAR
`exposure`	multiplicateur d'exposition à l'ouverture (le curseur va de 20 % à 500 %, soit 0,2 à 5)	opérateur
`orientation`	quaternion `Q_base` : il fait passer des axes du fichier à l'anatomie (chapitre 12)	opérateur, via l'outil d'orientation
`orientationAxes`	`{labels:{A:"Rostral"}, hidden:["D"], defaultView:{preset, quaternion}}` : noms des bras, bras masqués, vue par défaut	opérateur
`upsideDown`	`true` si le fichier brut montre l'échantillon par en dessous	opérateur (**Face de l'échantillon**)
`gallery[]`	`[{file, added, thumb, title, caption}]`	opérateur ; la plateforme crée la vignette
`relatedIds`, `linkedTrackingId`	liens entre jeux, saisis à la main ; la conversion du vocabulaire (section 17.5) les réécrit	opérateur
`line`, `staining`, `reporter`, `tags`, `notes`	informations libres ; `line` et `staining` sont posés par l'importeur de photographies	importeur 2D, opérateur

Ce qu'une série `live` ajoute

BLOC	CONTENU RÉEL DU JEU DE DÉMONSTRATION
`timeline`	`{count:4, intervalMinutes:30.0, timestamps:["2026-05-12T09:00:00", …]}`
`intensityNormalization`	`{mode:"global", bounds:{c0:{bgFloor, sigMax}, c1:{…}}, signalLevels:{"t000_c0":33107.0, …}}` : la fenêtre commune de toute la série et un niveau de signal par image et par canal
`tracking`	`{schema:"iribhm-tracks-v1", source, tracksPath:"tracks.json", surfacePath:"model.glb", surfaceOrigin:"reconstructed", cellCount:50, timepointCount:4, mitosisCount:1, fusionCount:0, regions:[{name, cells, color}], boundsUm:{stabilized, raw}, provenance:{…}, alignment:{…}}`
`registration`	`{method:"tracking-procrustes", coordinateSpace:"acquisition-um", timepointOffset:-1, appliedToVolume:true, transforms:[…], qcSummary:{…}, imageBoxUnionUm:{…}}`

ℹ️ **À SAVOIR**

À propos de `intensityNormalization` . Le pipeline enregistre ces niveaux « pour qu'un réglage facultatif puisse un jour compenser le photoblanchiment ». **Aucun réglage du viewer ne les lit encore** : une série dont le signal s'éteint s'affiche réellement plus sombre.

Le bloc `tracking` a trois sous-blocs utiles :

- `provenance` : d'où viennent les pistes (`kind: "scene8"` = les objets Imaris à l'intérieur du `.ims` , ou `excel` = un classeur exporté), le nombre de points (`spots: 194`) et de pistes (`tracks: 48`), l'unité déclarée et sa conversion en µm (`toUm: 1.0` , `status: "native"`), le rayon des sphères, l'outil d'extraction ;
- `alignment` : le contrôle fait à l'import — `insideAcquisitionBox: 1.0` (100 % des positions tombent dans la boîte du volume), `spanRatio` , `aligned: true` ;
- `surfaceOrigin` : `exported` si le fichier `.glb` vient du logiciel de suivi, `reconstructed` si la plateforme l'a reconstruit à partir des cellules.

Le bloc `registration` dit **si la stabilisation peut être appliquée à l'image** : `appliedToVolume: true` seulement si l'ajustement des pistes est **rigide** (résidu ≤ 0,05 µm ; ici 7,6 × 10^{-14} µm). Chaque `transforms[]` donne la matrice 4 × 4 (colonne par colonne, pour `THREE.Matrix4.fromArray`), l'angle (`rotationDeg: 7.0317` à la dernière image) et le déplacement en µm. Le chapitre 8 explique le calcul ; le chapitre 12 montre ce que le viewer en fait.

Ce qu'une photographie `2d` change

CHAMP	CE QUE C'EST
`dimensions`	`{x:1920, y:1440, z:1, c:3, t:1}` : trois canaux couleur, jamais de profondeur
`pixelSizeUm`, `physicalSizeUm`	`{x:2.0, y:2.0}` µm par pixel et `{x:3840, y:2880}` µm : lus dans les balises de résolution du TIFF
`image`	`{native:"image.webp", width, height, preview:"preview.webp", previewWidth:640, previewHeight:480}` : noms de fichiers **fixes**, l'import les exige tels quels
`acquisition`	`{modality:"brightfield-stereo", sourceFile, series, dissectionDate, zoomNominal:3.2…}` : lu dans le nom du fichier et le bloc Leica
`line`, `staining`, `date`	`"DLL4xCD1"`, `"X-gal"`, `"2024-09-13"`
`orientation2d`	`{rotationDeg, flipH}` : posé par l'opérateur avec l'outil d'orientation 2D
`channels`	`[]` : une photographie n'a pas de canaux

17.4 Les manifestes et les fichiers de description

`bricks/manifest.json` (format 4)

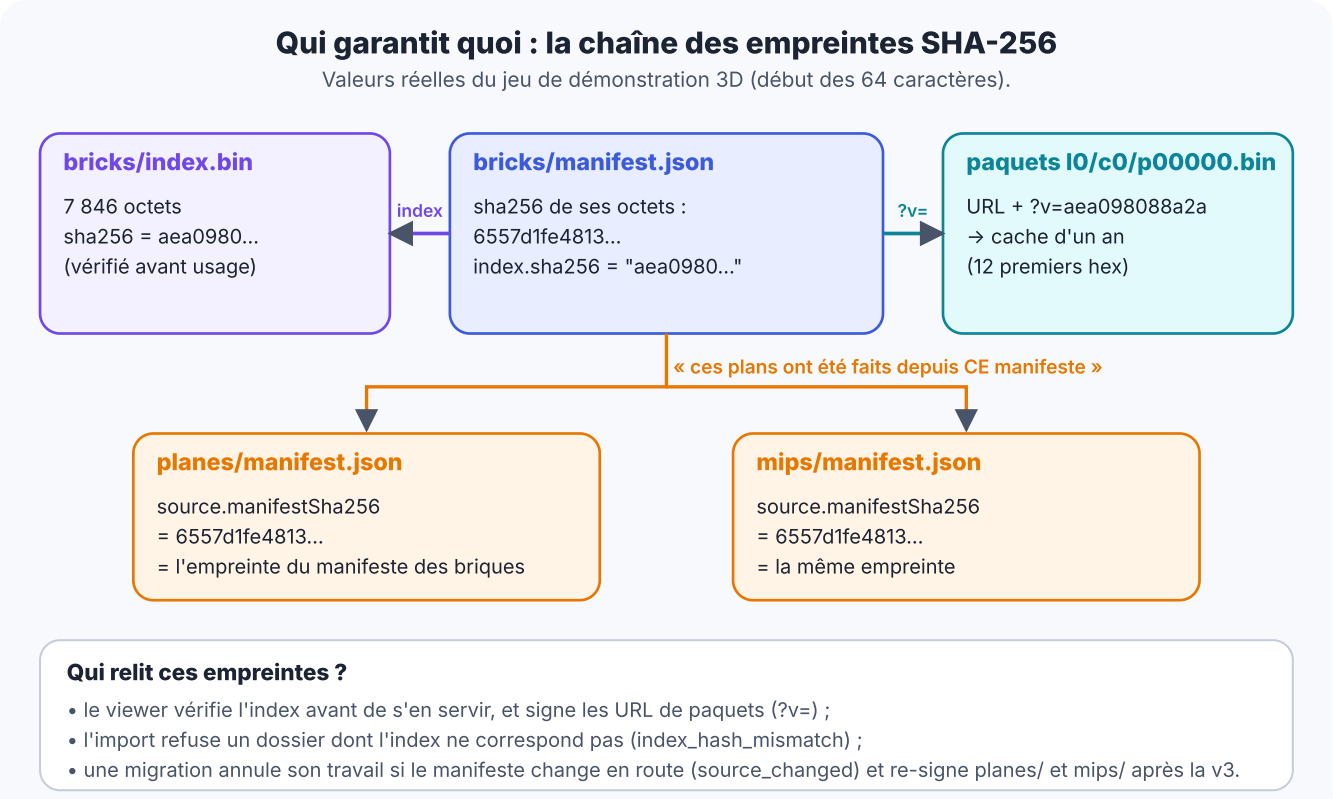
C'est la table des matières des briques. Voici **tous** ses champs, avec les valeurs du jeu de démonstration 3D.

CHAMP	CE QUE C'EST
`schema`, `version`, `formatVersion`	`"iribhm-bricks-v3"`, `3`, `4` : le lecteur choisit sa méthode d'après `schema`
`dataset`	nom du jeu
`channels`	`3`
`brickSize`, `apron`	`64` et `1` : cœur de 64 voxels, bordure d'un voxel (brique stockée : 66^3)
`brickPacking`	`{mode:"grid", cols:9, rows:8, slice:66}` : la mosaïque de 66 coupes de 66^2
`encoding`	`"webp-lossless"`
`levels[]`	`{level, dimensions, voxelSize, gridSize, brickCount}` par niveau ; ex. niveau 0 : `768×576×112`, voxel `1.2×1.2×3.0`, grille `12×9×2`, `269` briques présentes
`timepoints`	`null` pour un jeu 3D ; pour une série, une liste `[{path:"t000", index:{url, bytes, sha256}}, …]`

CHAMP	CE QUE C'EST
`index`	`{url:"index.bin", bytes:7846, sha256:"aea0980…"}` ; **absent** pour une série (chaque image porte le sien)
`histograms[]`	par canal : `counts` (64 cases), `edges` (65 bornes), `total` , `max` , `mean` , `std` , `backgroundFloor`
`timepointHistograms`	pour une série : les mêmes histogrammes, image par image
`producer` , `createdAt`	`"pipeline"` (ou un exécutant de migration) et la date

🔧 POUR LES CURIEUX

Les histogrammes sont calculés sur le niveau le plus grossier (ici `total: 96768` = 96 × 72 × 14 voxels). `backgroundFloor` vaut toujours `0` dans le fichier : c'est le viewer qui estime lui-même le plancher de bruit (chapitre 11).



▲ Qui garantit quoi : l'empreinte du manifeste signe les plans et les MIP, celle de l'index signe les paquets.

`bricks/manifest.json` (format 5)

Le manifeste du format 5 (`"schema": "iribhm-bricks-v5"` , `"version": 5`) décrit toute la matrice dans un seul fichier. Ce qu'il ajoute au format 4 :

CHAMP	CE QUE C'EST (JEU DE DÉMONSTRATION CONVERTI)
`levels[]`	une ligne par niveau, avec `target` : `"native"` , puis `512` , `256` ; dimensions `768×576×112` , `512×384×112` , `256×192×112`

CHAMP	CE QUE C'EST (JEU DE DÉMONSTRATION CONVERTI)
`columns[]`	les cinq compressions : `ll`, `b2`, `b4` (avec leur table `lut` de 256 valeurs), `q80`, `q50`
`stores`	les copies de coupe : `xy` (tuiles 512², PNG), `xz` / `yz` (paquets de 64 lignes), `cuts` (briques 32 × 32 × `brickZ`, ici 16 ou 32 selon le niveau)
`index`	un index par colonne (`index/ll.bin` …) et un par niveau de coupes (`cuts/l0/index.bin` …), chacun avec sa taille et son empreinte SHA-256
`bytes`	les octets de chaque case, par niveau, colonne et canal : ce que la matrice du viewer affiche **sans rien télécharger d'autre**

Les adresses des paquets portent `?v=` suivi des 12 premiers caractères de l'empreinte de **leur** index : changer un index change l'adresse, donc le cache du navigateur ne sert jamais un vieux paquet.

`index.bin`

L'index est expliqué octet par octet au chapitre 7 (section 7.6). Trois compléments :

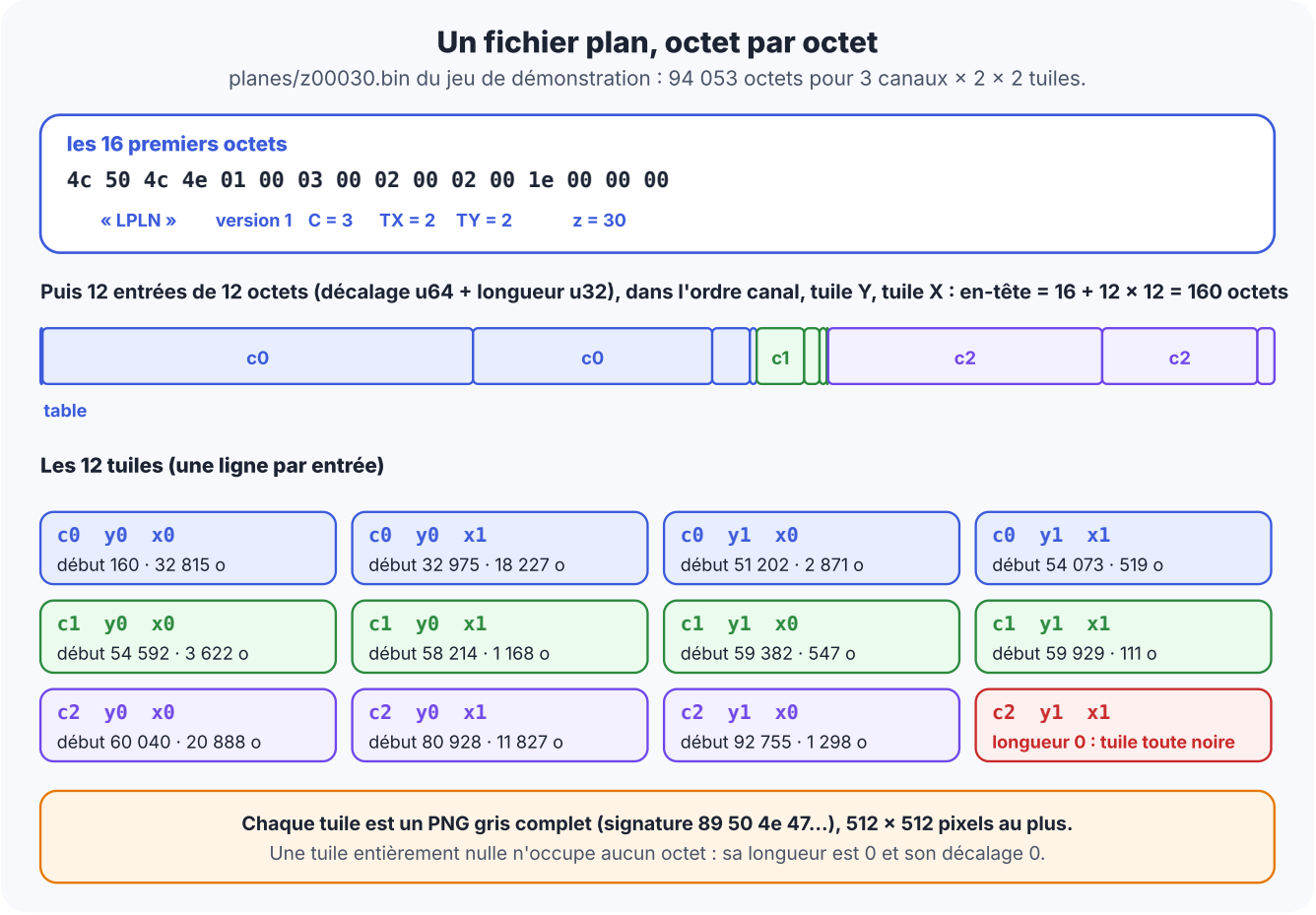
- sa **taille est prévisible** : `12 + 16 × (nombre de niveaux) + 10 × (canaux) × (somme des cases de grille)`. Pour le jeu 3D : 12 + 16 × 4 + 10 × 3 × 259 = **7 846 octets**, exactement ce que dit le manifeste ;
- les paquets sont nommés **sans chaîne dans l'index** : le paquet *p* du niveau *k* et du canal *c* s'appelle `l{k}/c{c}/p{p:05d}.bin` ;
- l'import contrôle la longueur **exacte**, l'empreinte et la cohérence avec les niveaux du manifeste (section 17.18).

`planes/manifest.json` et `mips/manifest.json`

Les deux ont la même forme ; `mips/` ajoute deux champs.

CHAMP	VALEUR RÉELLE (`PLANES/`)	SENS
`schema`	`"lumen-planes-v1"` (`"lumen-mips-v1"`)	
`formatVersion`	`2` (`3`)	le format qui a introduit la structure
`level`	`0`	toujours le niveau natif
`dimensions`, `channels`	`{768, 576, 112}`, `3`	égaux à ceux des briques
`tileSize`, `tiles`	`512`, `{x:2, y:2}`	tuiles de 512² : 768 = 512 + 256, 576 = 512 + 64
`codec`	`"png-gray8"`	PNG gris 8 bits, filtre « None »
`packPattern`, `headerBytes`	`"z{z}.bin"` (`"l{l}.bin"`), `160`	`16 + 12 × canaux × tuiles Y × tuiles X`

CHAMP	VALEUR RÉELLE (`planes/`)	SENS
`layers` , `layerDepth`	(mips) `2` , `64`	une projection par couche de 64 plans
`source.manifestSha256`	`"6557d1fe…"`	empreinte du manifeste des briques d'où ils viennent
`producer` , `createdAt`	`"pipeline"`	`"pipeline"` , `"migration-browser"` , `"migration-server"` ou `"mixed"`



▲ Un fichier plan réel, octet par octet : l'en-tête, la table de 12 entrées, puis des PNG.

Un fichier `MIP` (signature `LMIP`) a **exactement la même disposition** qu'un fichier plan : le champ « z » de l'en-tête y est l'indice de la couche.

Les briques « v2 » en cinq lignes

Avant le format 4, `bricks/manifest.json` listait **chaque brique** dans `brickTransport.brickToPack` : une clé `lod0/c0/x000_y000_z000.webp` pour une entrée `{url:"lod0/c0/pack_00.bin", offset, length}`. Les briques faisaient 64^3 sans bordure, rangées en mosaïque 8 × 8 (image 512^2) ; les paquets s'appelaient `lodN/cC/pack_NN.bin` ; une série avait un dictionnaire `timepoints` plutôt qu'une liste. Les migrations de la partie B transforment ces fichiers en leur équivalent moderne.

`tracks.json`

Le suivi d'une série est dans un fichier à part (`tracks.json` , et sa copie compressée `tracks.json.gz`). Structure réelle du jeu de démonstration :

CLÉ	CONTENU
`schema`, `source`, `sourceId`, `generated`	`"iribhm-tracks-v1"`, nom de la source, identifiant unique, date
`timepoints`	`[1.0, 2.0, 3.0, 4.0]` : les numéros d'image (Imaris compte à partir de 1)
`cells`	un objet par cellule, indexé par son numéro (50 cellules)
`layout`	`{x_range, y_range, z_range}` : l'étendue des positions stabilisées

Une cellule (la 23, qui se divise) :

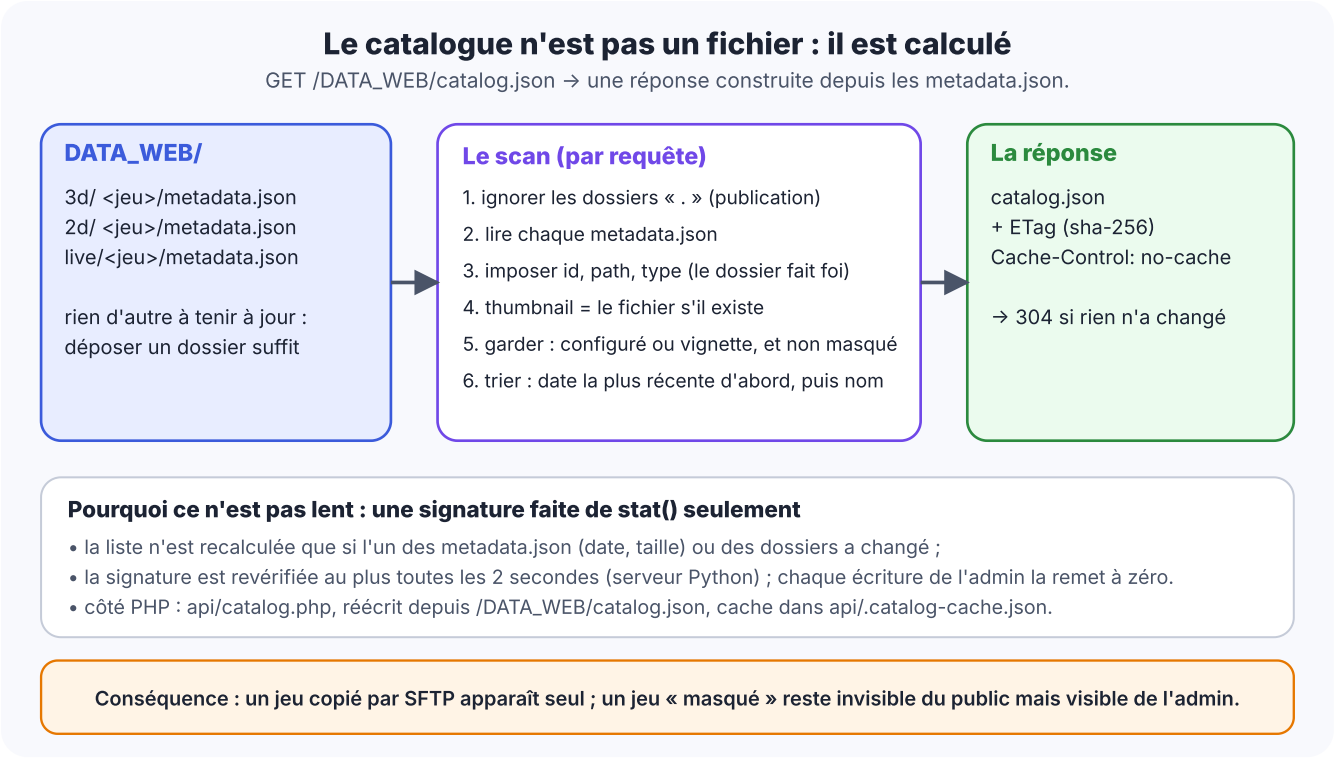
```
{ "id": "23", "track_id": 1000000023, "region": "Posterior", "color": "#2ecc71",
  "positions": { "1": [119.6, 121.3, 69.5], "2": [119.4, 121.9, 68.6] },
  "raw_positions": { "1": [119.6, 121.3, 69.5], "2": [120.3, 121.6, 69.4] },
  "parent": "", "daughters": ["24", "25"],
  "is_mitosis": true, "is_fusion": false, "markers": { "2": "red" } }
```

`positions` sont les coordonnées **stabilisées** en µm, `raw_positions` les coordonnées d'acquisition ; `parent` et `daughters` forment la **généalogie** (une mitose : une mère, deux filles qui partagent le même `track_id`). Le viewer charge ce fichier dans un Web Worker qui en fait des tableaux 32 bits compacts.

17.5 Le catalogue et le vocabulaire des types

Un catalogue qui n'est pas un fichier

La liste des jeux que voit le public (`/DATA_WEB/catalog.json`) n'est **stockée nulle part** : elle est calculée à chaque requête à partir des `metadata.json`.



▲ Le catalogue est calculé à la demande, avec une signature faite de stat() pour rester rapide.

Les règles exactes de la construction :

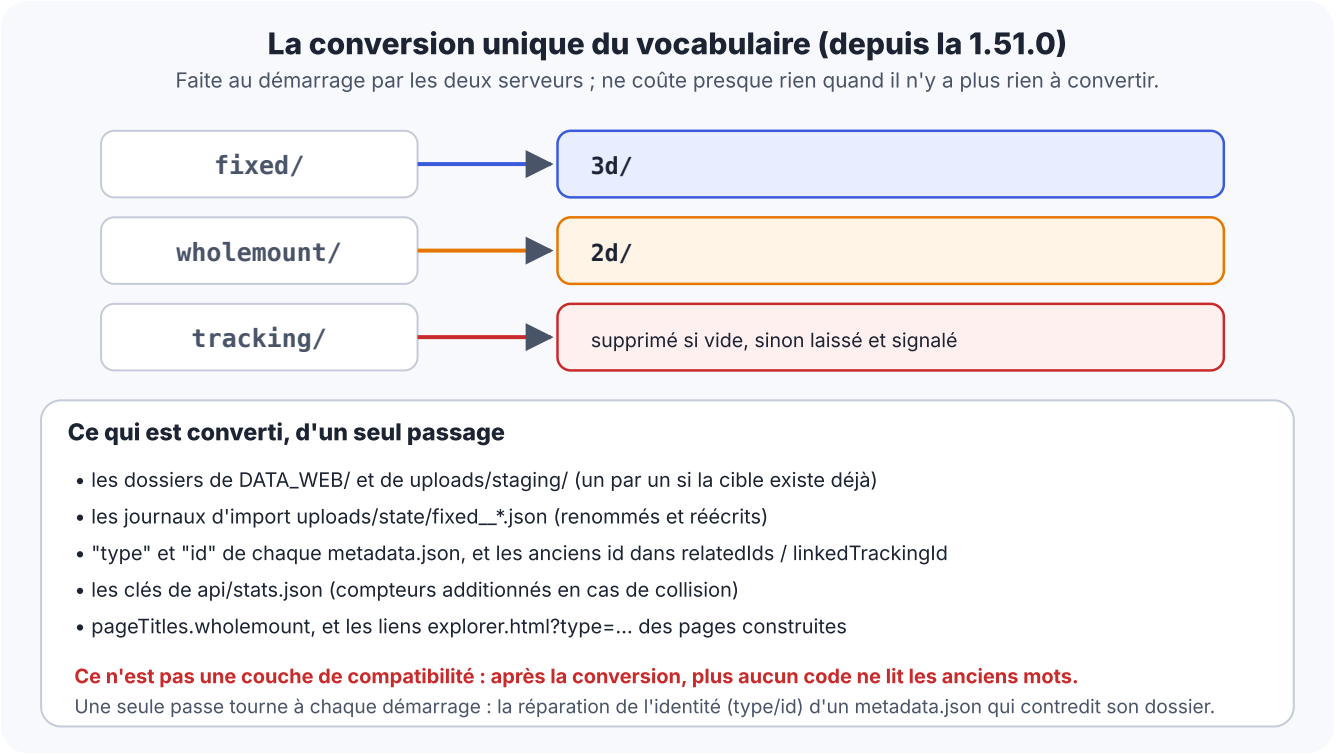
- un dossier dont le nom commence par un point n'est **jamais** un jeu (c'est un chantier) ;
- un dossier **sans** `metadata.json` (traitement en cours) donne une ligne « non configurée », sans vignette — donc absente du catalogue public ;
- `id`, `path`, `type`, `folderName` sont **imposés** d'après l'emplacement ; `thumbnail` est le fichier s'il existe, `null` sinon ;
- on garde les jeux **configurés ou vignettés**, et **non masqués** ;
- l'ordre met la date la plus récente d'abord ; les jeux sans date (les volumes en ont rarement) viennent ensuite, par nom décroissant.

❓ **POURQUOI CE CHOIX ?**

Pourquoi pas un fichier ? Autrefois, le catalogue était un fichier que seul le panneau d'administration réécrivait : un jeu copié par SFTP restait invisible jusqu'à un clic sur un bouton de régénération. Maintenant le catalogue **n'a aucune information propre** : rien à tenir à jour, donc rien à oublier.

Un vocabulaire, trois mots

Depuis la version 1.51.0, un type de jeu s'écrit `3d`, `2d` ou `live`, et ce même mot est à la fois le dossier sous `DATA_WEB/`, le début de l'identifiant, le champ `type`, la valeur d'un filtre d'URL… Les anciens mots (`fixed`, `wholemount`, `tracking`) n'existent plus nulle part dans le code.



▲ La conversion du vocabulaire : faite une fois au démarrage, par les deux serveurs.

Un déploiement ancien est converti **une seule fois**, par une passe **idempotente** (la rejouer ne change rien) lancée au démarrage de chaque serveur. Elle ne laisse **aucun marqueur** : quelques tests d'existence de dossier suffisent à savoir qu'il n'y a plus rien à faire.

> 🔧 **POUR LES CURIEUX**
>
> **Une passe tourne à chaque démarrage.** La réparation de l'identité d'un `metadata.json` dont `type` ou `id` contredit son dossier. Elle existe parce qu'un hébergeur PHP avait écrit, avant la 1.54.1, la charge de l'éditeur (qui ne contient pas `type`) en remplacement du fichier. Elle ne réécrit que les fichiers incohérents.

Ce que **voit** l'opérateur n'est jamais le mot technique : les noms affichés (« 3D », « Photo », « Live »...) viennent de la configuration du site (chapitre 16).

17.6 `download/` et `gallery/`

`download/` : les souvenirs à emporter

Ce dossier est rempli par le pipeline (option `--with-downloads`, chapitre 8). Trois règles de la plateforme le concernent :

- **tout** ce qu'il contient est servi **en pièce jointe** : un rapport HTML ou XML déposé là ne s'affiche jamais comme une page de votre site ;
- il est **exclu** des mises à jour de la plateforme et des migrations de données : on n'y touche jamais ;
- l'import n'y accepte que des extensions **de données** : `ims tif tiff png jpg jpeg webp gif zip txt md csv json pdf gz h5 hdf5`.

`gallery/` : les images jointes

L'opérateur peut joindre des images à un jeu (captures annotées, figures). Elles vivent **dans** le dossier du jeu, pour voyager avec lui.

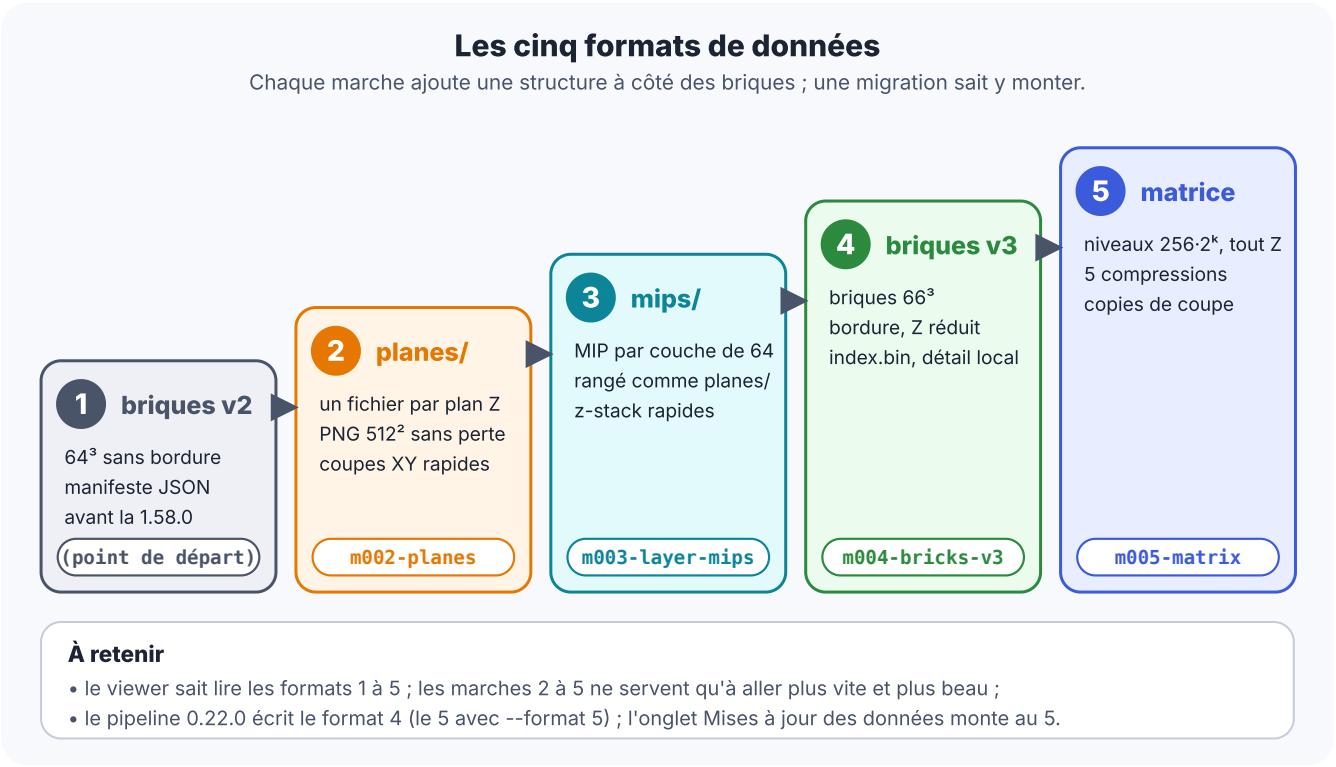
RÈGLE	VALEUR
Nombre maximal	**40** images par jeu
Taille maximale	**8 Mio** par image
Formats	WebP, PNG, JPEG, GIF ; l'extension est déduite des **octets magiques**, jamais du nom envoyé
Vignettes	`gallery/thumbs/<fichier>.webp` (320 px au plus, jamais agrandie) pour la grille ; la visionneuse charge l'original
Ordre et légendes	dans `metadata.json`, clé `gallery` : `{file, added, thumb, title, caption}`
Légende	400 caractères au plus ; titre 120

Un fichier nommé `a.php.png` dont les octets ne sont pas ceux d'un PNG est **refusé** : seuls les octets comptent.

> ⚠️ **ATTENTION**
>
> La galerie **ne voyage pas** avec l'import (aucune règle de la liste blanche ne l'accepte). Quand un jeu est remplacé, c'est la plateforme qui **déplace** la galerie de l'ancien dossier vers le nouveau (section 17.18).

17.7 Les formats 1 à 5

Un jeu publié porte un numéro de **format** (`formatVersion`). Le visiteur n'a rien à faire : le viewer lit les cinq formats. Mais les formats récents ouvrent des possibilités que les anciens n'ont pas.



▲ Cinq marches : chacune ajoute une structure ; une migration sait y monter.

Ce que débloque chaque marche :

FORMAT	CE QU'IL PERMET	OÙ C'EST EXPLIQUÉ
2 (`planes/`)	une coupe XY **native** du Studio lit un plan au lieu d'une couche de 64 ; les figures z-stack s'ouvrent vite	chapitres 7 et 12
3 (`mips/`)	une projection maximale de toute la pile lit une image par couche	chapitre 7
4 (briques v3)	filtrage **sans couture** entre briques, **détail local** (« Zoom detail »), atlas plus légers pour 1 ou 2 canaux, niveaux de qualité exacts	chapitres 9 et 10
5 (matrice)	des niveaux qui gardent **toutes les tranches**, **cinq compressions** au choix, des coupes XZ / YZ / obliques natives rapides, des exports et des mesures toujours **sans perte**	chapitres 7, 10 et 12

Ce qu'un numéro de format promet

Un jeu **ne déclare jamais une structure qu'il n'a pas**. Le pipeline écrit le plus haut format dont **toutes** les structures sont complètes sur le disque (chaque arbre d'une série compris) ; la plateforme fait de même. Si un jeu dit « format 3 » mais que `mips/` est abîmé, l'onglet **Mises à jour des données** le propose en **réparation**.

Une brique v2 et une brique v3

Une brique v2 et une brique v3

Ce que la migration m004 change, point par point.

	v2	v3
taille	64 × 64 × 64	66 × 66 × 66
bordure	aucune	1 voxel (clamp-to-edge)
mosaïque	8 × 8 coupes de 64² (512²)	9 × 8 coupes de 66² (594 × 528)
lecture GPU	plus proche voisin (NEAREST)	LINÉAIRE, sans couture
niveaux	seuil d'occupation 0,05 %	exacts : un voxel ≥ 1 suffit
où est la brique ?	JSON : une entrée par brique	index.bin : 10 octets par case
paquets	`lod0/c0/pack_00.bin`	`l0/c0/p00000.bin`
gros embryon	manifeste ≈ 7,6 Mo	index ≈ 233 ko

Le niveau natif est recopié voxel pour voxel : la brique v2 absente du manifeste vaut zéro, la brique v3 absente de l'index vaut zéro, bordure comprise.

▲ Ce que change la migration m004, point par point.

Quand un niveau réduit-il aussi Z ?

Dans l'ancien format, on réduisait surtout en XY. Le format 4 applique une règle précise : le niveau natif est copié **tel quel**, puis chaque niveau divise X et Y par 2 ; il divise **aussi Z** seulement si le voxel n'est pas déjà bien plus épais en Z qu'en XY (`vz ≤ 1,5 × vxy` du niveau suivant).

Quand un niveau réduit-il aussi Z ?

Règle : on divise Z par 2 seulement si vz ≤ 1,5 × (la taille XY du niveau suivant).

Jeu de démonstration 3D (768 × 576 × 112 ; 1,2 × 1,2 × 3 µm)

niveau 0	niveau 1	niveau 2	niveau 3
768×576×112	384×288×56	192×144×28	96×72×14
vxy 1.20	vxy 2.40	vxy 4.80	vxy 9.60
vz 3.00	vz 6.00	vz 12.00	vz 24.00
natif (verbatim)	Z réduit	Z réduit	Z réduit

Gros embryon anisotrope (3789 × 3789 × 257 ; 0,43 × 0,43 × 2,06 µm)

niveau 0	niveau 1	niveau 2	niveau 3	niveau 4	niveau 5
3789×3789×257	1895×1895×257	948×948×129	474×474×65	237×237×33	119×119×17
vxy 0.43	vxy 0.86	vxy 1.72	vxy 3.44	vxy 6.89	vxy 13.77
vz 2.06	vz 2.06	vz 4.11	vz 8.23	vz 16.46	vz 32.91
natif (verbatim)	Z gardé	Z réduit	Z réduit	Z réduit	Z réduit

Pourquoi ? Pour qu'un niveau grossier ait des voxels presque cubiques :
tant que le voxel est bien plus épais en Z qu'en XY, on ne réduit que XY ; dès qu'il devient comparable, on réduit les trois axes.
On s'arrête quand max(X, Y) ≤ 128 voxels. La moyenne est entière, arrondie au plus proche (demi vers le haut).

▲ La règle de réduction en Z, sur le jeu de démonstration et sur un gros embryon anisotrope.

🔢 **MICRO-EXEMPLE**

Le gros embryon (3789 × 3789 × 257, voxel 0,43 × 0,43 × 2,06 µm). Niveau 1 : 0,86 µm en XY ; 2,06 > 1,5 × 0,86 = 1,29, donc Z est **gardé**. Niveau 2 : 1,72 µm en XY ; 2,06 ≤ 1,5 × 1,72 = 2,58, donc Z est **réduit** (129 plans). Six niveaux au total, jusqu'à 119 × 119 × 17.

La réduction est une **moyenne entière**, arrondie au plus proche (la moitié vers le haut) : `(somme + n/2) // n`. La même arithmétique est utilisée par le pipeline, Python, PHP et le navigateur : c'est ce qui rend leurs résultats **identiques voxel pour voxel**.

17.8 Quatre migrations, un registre

Une **migration** transforme un jeu de la version *v* à la version *v + 1*. Les quatre migrations forment un **registre ordonné**, identique dans trois langages.

IDENTIFIANT	DE → VERS	CE QU'ELLE CRÉE	S'APPLIQUE À
`m002-planes`	1 → 2	`planes/` : le niveau natif recoupé en plans XY	`3d` , `live`
`m003-layer-mips`	2 → 3	`mips/` : le maximum de chaque couche de 64 plans	`3d` , `live`
`m004-bricks-v3`	3 → 4	un nouvel arbre `bricks/` (66^3, index binaire)	`3d` , `live`
`m005-matrix`	4 → 5	un nouvel arbre `bricks/` : la matrice et les copies de coupe	`3d` , `live`

Chaque entrée porte un identifiant, les versions, les types concernés et un **titre et une description en quatre langues** (c'est ce que l'onglet affiche).

🔧 **POUR LES CURIEUX**

Les trois jumeaux. Le registre existe en Python (`dataset_migrations.py`), en PHP (`api/_migrations_lib.php` , `_migrations_formats.php` , route `api/migrations.php`) et dans le navigateur (un module par migration : `js/migrations/m002-planes.js` …). Le **contrat** commun, y compris chaque octet des formats, est la spécification `DOCS/dataset-migrations/SPEC.md` . Ajouter une migration = ajouter une entrée dans les trois jumeaux, son découpage en unités, ses deux exécutants et un module navigateur ; rien d'autre ne change.

Quel jeu a besoin de quoi ?

Un jeu à la version *v* reçoit **dans l'ordre** toutes les migrations dont le « de » est ≥ *v* ; une migration ne s'applique que si le jeu est **exactement** à sa version de départ. Un jeu au format 1 reçoit donc m002, puis m003, puis m004, puis m005.

L'onglet décide ainsi, pour chaque jeu **publié** de type `3d` ou `live` (les photographies 2D sont toujours « à jour » ; un jeu encore en import n'est pas concerné) :

CONSTAT	CE QUE L'ONGLET PROPOSE
`formatVersion` plus bas que le courant	les migrations manquantes, dans l'ordre
la version annonce une structure absente ou invalide (`planes/` manquant, `mips/` abîmé…)	une **réparation** : on refait la migration qui produit cette structure
version 4 mais briques v3 abîmées	un **problème** (`bricks_v3_invalid`), pas une réparation : l'ancien arbre n'existe plus
version 5 mais arbre v5 abîmé	un **problème** (`bricks_v5_invalid`), pour la même raison
manifeste illisible ou absent (`manifest_invalid` , `no_manifest`)	un problème signalé sur la ligne du jeu
série dont les images n'ont pas toutes les mêmes dimensions	refus (`trees_differ`) : m004 ne sait pas la convertir

Avant de commencer, le serveur **refuse un travail que le disque ne peut pas contenir** (erreur 507, comme l'import) : il estime la place du résultat — environ 1,3 fois le niveau natif pour `planes/` , 5 % de la taille de `planes/` pour `mips/` , environ 1,3 fois l'ancienne pyramide pour les briques v3 — et exige qu'elle tienne à la fois dans le magasin de travail et à côté du jeu (deux fois sur un même disque), avec une réserve de 512 Mio.

Un essai réel de bout en bout

Pour vérifier chaque affirmation de ce chapitre, un jeu **synthétique de format 1** (520 × 520 × 130 voxels, 2 canaux, du bruit aléatoire) a été converti par le vrai moteur Python, dans un dossier temporaire.

ÉTAPE	UNITÉS	DURÉE	TAILLE AVANT → APRÈS
m002 → format 2	24 (dont 1 vide)	2,4 s	`planes/` : 42,0 Mo (les briques : 40,1 Mo)
m003 → format 3	24 (dont 1 vide)	0,6 s	`mips/` : 0,70 Mo
m004 → format 4	30	16,3 s	`bricks/` : 40,1 Mo → 48,9 Mo

> ⚠️ **ATTENTION**
>
> Ces tailles ne sont **pas représentatives** : le bruit aléatoire est incompressible, et le format 4 ajoute 9,7 % de bordure. Sur de vraies images (surtout du noir, du signal lisse), la pyramide v3 est en général plus légère. Retenez les **unités** et la **mécanique**, pas les mégaoctets.

m005 : quatre sortes d'unités

La migration vers le format 5 lit le **niveau natif sans perte** (le dossier `planes/` et les briques du format 4), jamais le fichier `.ims` . Son travail est découpé en quatre étapes, A à D :

ÉTAPE	UNE UNITÉ =	CE QU'ELLE PRODUIT
A	un plan natif d'un canal	ce plan, réduit à chaque niveau (moyenne exacte de surface)
B	un super-bloc de 4 × 4 × 4 briques d'un niveau et d'un canal	ces briques dans les cinq colonnes (au niveau 0, toutes sauf « sans perte », reprise du format 4)
C	64 lignes XZ (ou 64 colonnes YZ) d'un niveau et d'un canal	un paquet `xz/` ou `yz/`
D	un super-bloc de 8 × 8 × 8 petites briques de coupe	un enregistrement de `cuts/`

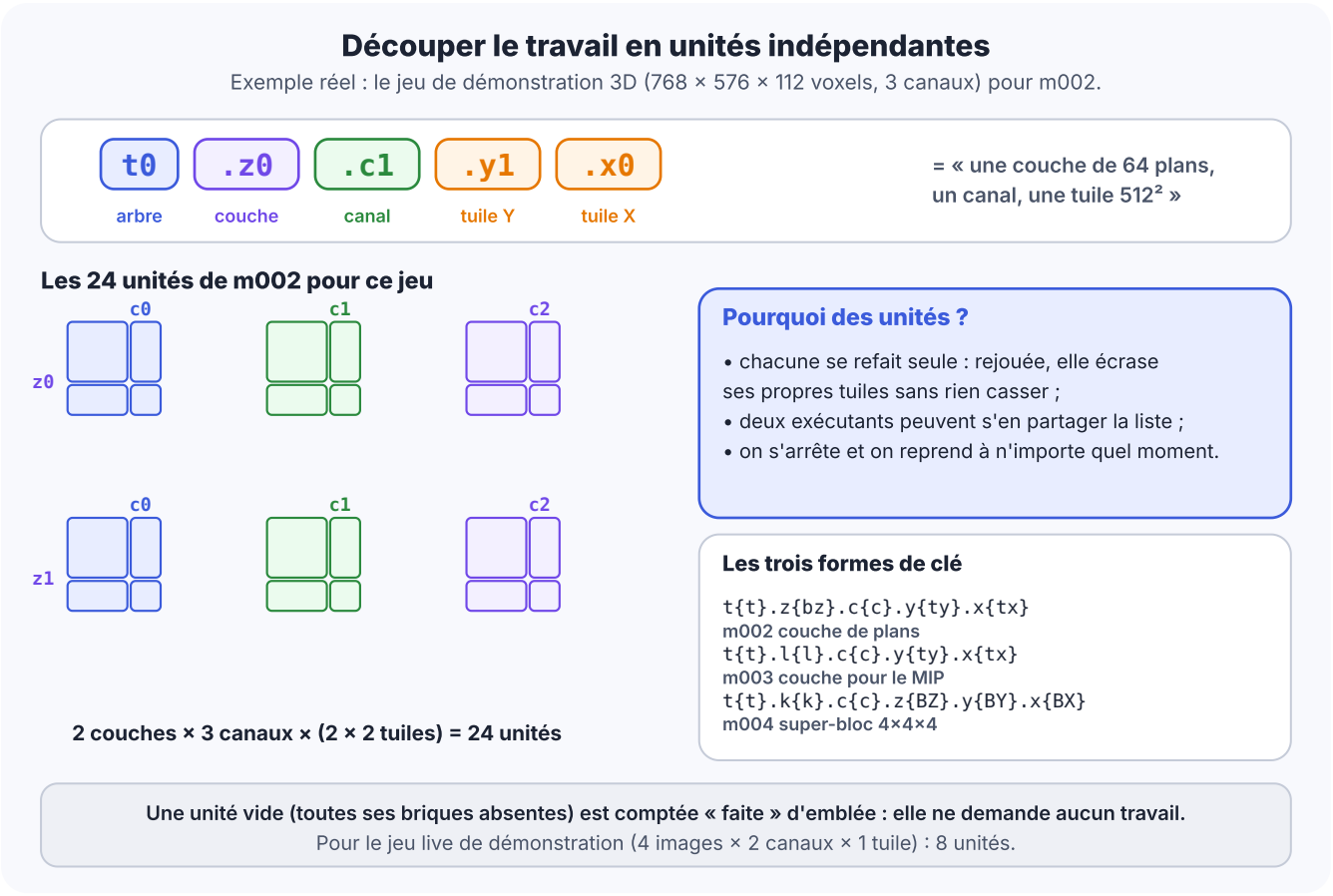
- Les étapes B, C et D d'un niveau réduit **attendent** l'étape A de leur canal (elles lisent ses plans réduits) ; celles du niveau natif partent tout de suite.
- Une unité dont la zone ne contient **aucune brique** du format 4 est « vide » : marquée faite dès le plan, jamais calculée.
- À la fin, `finalize` assemble le nouvel arbre dans `.bricks5-incoming/`, le **vérifie en entier**, puis l'échange avec `bricks/` ; l'ancien est supprimé (ou gardé en `bricks.previous/` si l'option **Conserver la version précédente** est cochée).

🔢 **MICRO-EXEMPLE**

Mesuré dans ce conteneur (exécutant Python, 22 cœurs, outil `tools/convert_dataset.py`) : le jeu de démonstration E9.5 (768 × 576 × 112, 3 canaux) passe du format 4 au format 5 en **546 unités et 38 secondes** ; E10.5 en 28 s ; E8.5 en 11 s. Ses briques passent de 9 Mo à environ 80 Mo sur le disque (section 7.11).

17.9 Les unités de travail et le journal

Convertir un jeu de 10 Go d'un seul tenant échouerait au premier incident. La plateforme le découpe donc en **unités** indépendantes.



▲ Une unité = une couche de 64 plans, un canal, une tuile de 512² : ici, les 24 unités du jeu de démonstration.

Pour m002, une unité traite **une couche de briques** (au plus 64 plans en Z), **un canal** et **une tuile** de 512 × 512 pixels (= 8 × 8 briques) : elle décode les briques correspondantes puis émet jusqu'à 64 tuiles PNG, une par plan.

🔢 **MICRO-EXEMPLE**

Les 24 unités du jeu 3D de démonstration. 112 plans = 2 couches (64 + 48) ; 768 × 576 pixels = 2 × 2 tuiles ; 3 canaux. 2 × 3 × 4 = **24 unités**. Pour le jeu `live` : 4 images × 1 couche × 2 canaux × 1 tuile = **8 unités**.

Les trois migrations n'ont pas la même forme d'unité :

MIGRATION	CLÉ D'UNITÉ	CE QUE FAIT L'UNITÉ
m002	`t{t}.z{bz}.c{c}.y{ty}.x{tx}`	décode les briques d'une couche, produit des tuiles PNG
m003	`t{t}.l{l}.c{c}.y{ty}.x{tx}`	lit les ≤ 64 tuiles de plan d'une couche, en garde le **maximum** pixel par pixel
m004	`t{t}.k{k}.c{c}.z{BZ}.y{BY}.x{BX}`	fabrique les briques v3 d'un **super-bloc 4 × 4 × 4** du niveau *k*

Une unité dont **toutes** les briques sont absentes est **vide** : elle est comptée « faite » d'emblée et ne demande aucun travail.

Le niveau suivant attend le précédent

Pour m004, le niveau *k + 1* se calcule à partir du niveau *k* : il ne peut donc démarrer **qu'une fois le niveau précédent complet**. Le serveur renvoie les unités **exécutables maintenant** (`runnable`) et le total restant (`pending`).

NIVEAU (ESSAI RÉEL)	DIMENSIONS	UNITÉS
0	520 × 520 × 130	18
1	260 × 260 × 65 (Z réduit)	8
2	130 × 130 × 33	2
3	65 × 65 × 17	2

Au début, 18 unités sont exécutables sur 30 ; les autres attendent. Le navigateur **re-planifie par vagues** jusqu'à ce qu'il n'y en ait plus.

La feuille de pointage

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un déménagement.** Chaque unité est une **caisse**. Le déménageur coche chaque caisse livrée sur une feuille. Si le camion tombe en panne, ou si on change de camion, on repart de la dernière caisse cochée. Si une caisse est livrée deux fois, personne ne s'en aperçoit : la deuxième écrase la première.

Cette feuille est le **journal**, un fichier JSON par migration et par jeu, dans `uploads/migrations/` (une zone **jamais servie**). À côté, un **magasin de tuiles** garde les résultats des unités terminées.

La feuille de pointage du déménagement

Journal et magasin de tuiles d'une vraie conversion m002 (jeu synthétique de test).

uploads/migrations/3d__<jeu>__m002-planes.json

```
{ "migration": "m002-planes",
  "dataset": "3d/Synthetique-E95",
  "sourceManifests": { "0": "46475b1a…" },
```

empreinte du manifeste de départ

```
  "units": { "total": 24, "empty": 1 },
```

24 caisses, dont 1 vide d'office

```
  "done": [ "t0.z0.c0.y0.x0", …(23) ],
```

les caisses déjà cochées

```
  "executors": { "browser": 0, "server": 23 },
```

qui a porté combien de caisses

```
  "state": "running",
```

running → assembling → swapped

```
  "updatedAt": "2026-10-07T18:49:55Z" }
```

Le magasin de tuiles

```
uploads/migrations/
└ 3d__…__m002-planes/
   └ t0/
      ├ z0/  c0.y0.x0.png
      │      c0.y0.x1.png
      │      c0.y1.x0.png …
      ├ z1/  …
      └ z129/ …
```

un PNG gris par plan et par tuile :
une unité = jusqu'à 64 petits fichiers

ici : 42,0 Mo de tuiles avant l'assemblage

jamais servi : c'est un atelier, pas une vitrine

Pourquoi une feuille de pointage ?

Comme un déménageur coche chaque caisse livrée : si le camion tombe en panne, ou qu'on change de camion (navigateur → serveur), on repart de la dernière caisse cochée. Une caisse livrée deux fois ne gêne personne.

▲ Le journal d'une vraie conversion m002 et son magasin de tuiles.

CHAMP DU JOURNAL	SENS
`migration`, `dataset`	quelle migration, sur quel jeu
`sourceManifests`	`{arbre: sha256}` du manifeste de départ de chaque arbre
`inputManifests`	(m003) empreinte du manifeste de `planes/` qui sert d'entrée
`units`	`{total, empty}` : nombre d'unités et nombre d'unités vides
`done`	la liste des clés d'unités terminées
`executors`	`{browser: n, server: n}` : qui a porté combien d'unités
`state`	`running`, `assembling`, `swapped` ou `failed`
`producer`, `assembledAt`, `assembly`	à partir de l'assemblage : qui a produit, quand, et l'avancement
`error`, `createdAt`, `updatedAt`	cause d'un échec ; dates

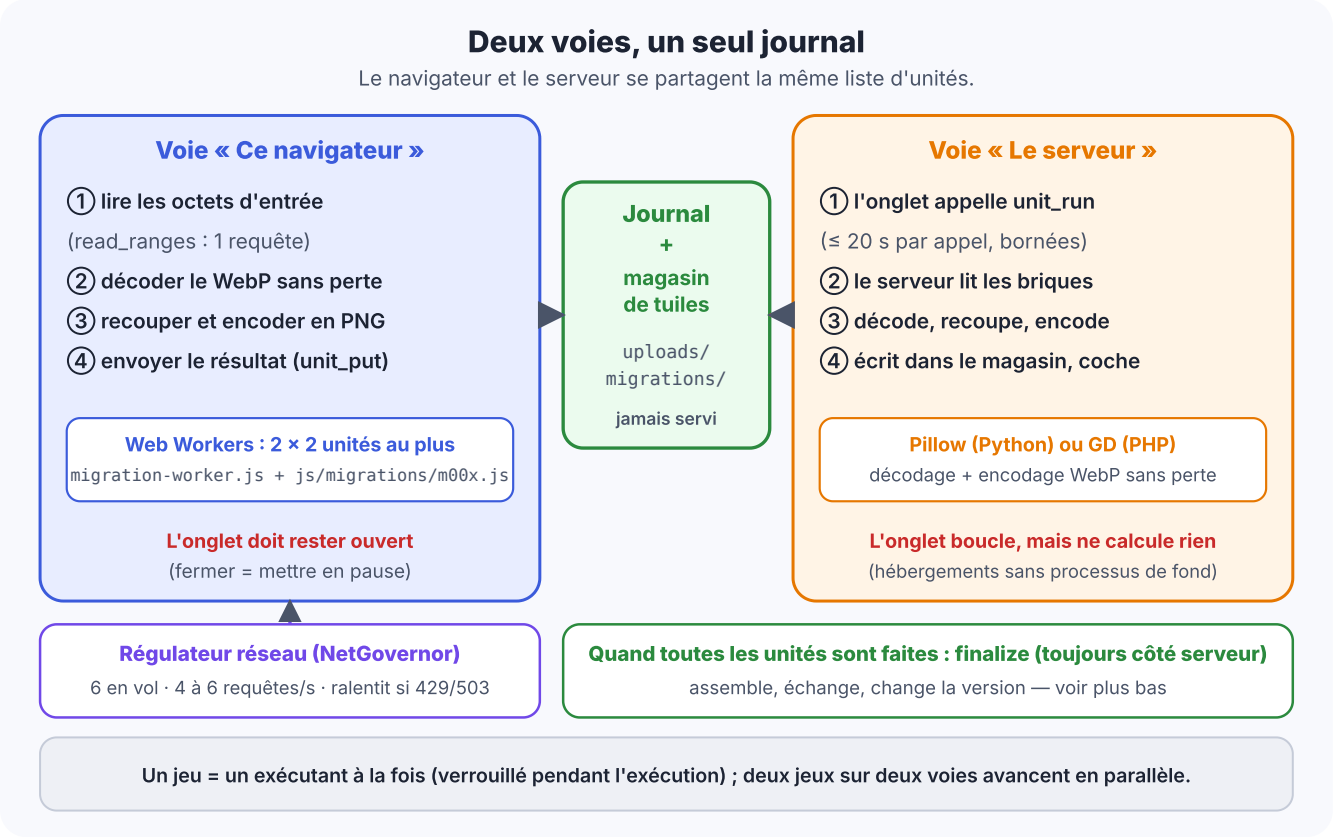
Les propriétés qui font la solidité de ce mécanisme :

- le journal est écrit **« tout ou rien »**, sous un verrou : deux écritures simultanées ne se mélangent pas ;

- une unité **rejouée** est inoffensive : elle remplace ses propres tuiles ;
- si le **manifeste des briques change** pendant le travail (le jeu a été retraité entre-temps), le travail est invalidé (`source_changed`) : on ne mélange pas deux versions d'un même jeu ;
- un jeu supprimé en cours de route n'est **jamais recréé** par un travail qui finit.

17.10 Deux exécutants

Le travail de chaque unité peut être fait par **deux exécutants** qui écrivent dans **le même journal**.



▲ Deux voies, un seul journal : le navigateur et le serveur se partagent la liste d'unités.

Le navigateur

Un **Web Worker** (`js/workers/migration-worker.js`) fait tout le calcul, avec un petit module spécifique à la migration (`js/migrations/<id>.js`). Pour chaque unité :

1. **lire** les octets d'entrée des paquets, par plages d'octets, dans **une seule requête** (`read_ranges`) ;
2. **décoder** les briques WebP sans perte, exactement comme le viewer ;
3. **recouper** et **encoder** : des tuiles PNG (m002, m003) ou des briques WebP v3 (m004) ;
4. **envoyer** le résultat en une requête (`unit_put` , 32 Mio au plus).

L'onglet lance **au plus 2 workers × 2 unités** (le nombre de workers vaut la moitié des cœurs, de 1 à 2) : une unité retient jusqu'à environ 40 Mio en mémoire. Le serveur **revérifie** tout ce qu'il reçoit : chaque PNG doit commencer par la bonne signature et avoir les bonnes dimensions ; chaque brique v3 doit être du WebP sans perte de 594 × 528 pixels, les 6 cases inutilisées à zéro.

Le serveur

L'onglet appelle `unit_run` **en boucle** ; chaque appel dure au plus 20 secondes (moins sur un hébergement qui limite le temps) et traite autant d'unités que possible. Le serveur lit lui-même les briques et écrit dans le magasin. Un « bail » de 120 secondes empêche deux appels de prendre la même unité.

> ❓ **POURQUOI CE CHOIX ?**
>
> **Pourquoi une boucle pilotée par le navigateur ?** Les hébergements mutualisés n'ont pas de tâche de fond : tout calcul doit tenir **dans une requête**. C'est l'onglet ouvert qui relance, d'où la consigne de ne pas le fermer.

Le cas du format 5

- **Le navigateur** ne planifie pas lui-même une unité de m005 : il demande au serveur **quoi lire** (`unit_inputs`), lit ces octets en requêtes groupées, calcule l'unité avec la **même arithmétique** que le moteur Python, et l'envoie. Ses images sans perte se décodent en **exactement** les mêmes voxels que celles du serveur ; ses images avec perte restent dans les mêmes bornes.
- **Le serveur Python** convertit plusieurs unités à la fois (en fils d'exécution).
- **Un hébergement PHP** n'a **pas d'exécutant m005** : l'onglet affiche « sur ce serveur, cette mise à jour s'exécute dans le navigateur », et c'est **Ce navigateur** qui fait le travail. Le serveur PHP prépare les entrées de chaque unité, vérifie ce qu'il reçoit et assemble l'arbre final, octet pour octet comme le ferait Python.
- La sonde du navigateur vérifie en plus qu'il sait encoder du **WebP avec perte** (sinon : « ce navigateur ne sait pas encoder d'images WebP avec perte »).

Convertir sur son propre ordinateur

Pour un très gros jeu, ou un hébergement lent, la conversion peut se faire **en local** : `python tools/convert_dataset.py DATA_WEB/3d/<dossier>` applique toutes les migrations en attente avec l'exécutant Python (`--list` les affiche sans rien faire ; `--keep-previous` garde l'ancien arbre). Le journal est le même que celui de l'onglet : une conversion interrompue **reprend**. Le dossier converti se glisse ensuite dans l'onglet **Import** de l'autre site.

Qui sait faire quoi : les sondes

Un exécutant n'est proposé que s'il **sait** faire l'étape. La capacité est mesurée **par migration** et **par exécutant**, par un petit essai réel.

RAISON AFFICHÉE À L'ÉCRAN	CODE	CE QUI MANQUE
le serveur ne sait pas décoder les images WebP sans perte	`no_webp_decode`	Pillow sans WebP, ou `imagecreatefromwebp` absent / fautif
le serveur ne sait pas encoder d'images WebP sans perte	`no_webp_lossless_encode`	PHP < 8.1 avec WebP, ou Pillow sans l'option sans perte
le serveur n'a pas NumPy	`no_numpy`	(Python, m004)
le serveur n'a pas la compression zlib	`no_zlib`	

RAISON AFFICHÉE À L'ÉCRAN	CODE	CE QUI MANQUE
le serveur autorise moins de 128 Mio de mémoire par requête	`low_memory`	
le serveur interrompt les requêtes après moins de 10 secondes	`exec_time_too_short`	
ce navigateur n'a pas de flux de compression	`no_compression_stream`	`DecompressionStream`
ce navigateur ne relit pas les tuiles PNG à l'identique	`no_png_codec`	
ce navigateur ne sait pas encoder d'images WebP sans perte	`no_webp_lossless_encode`	

La sonde du serveur décode un minuscule WebP **intégré** et compare chaque valeur de pixel ; celle de m004 encode une brique de test puis la décode et compare **chaque voxel**. La sonde du navigateur, pour m004, vérifie qu'un `OffscreenCanvas` produit bien du WebP **sans perte** (en le relisant).

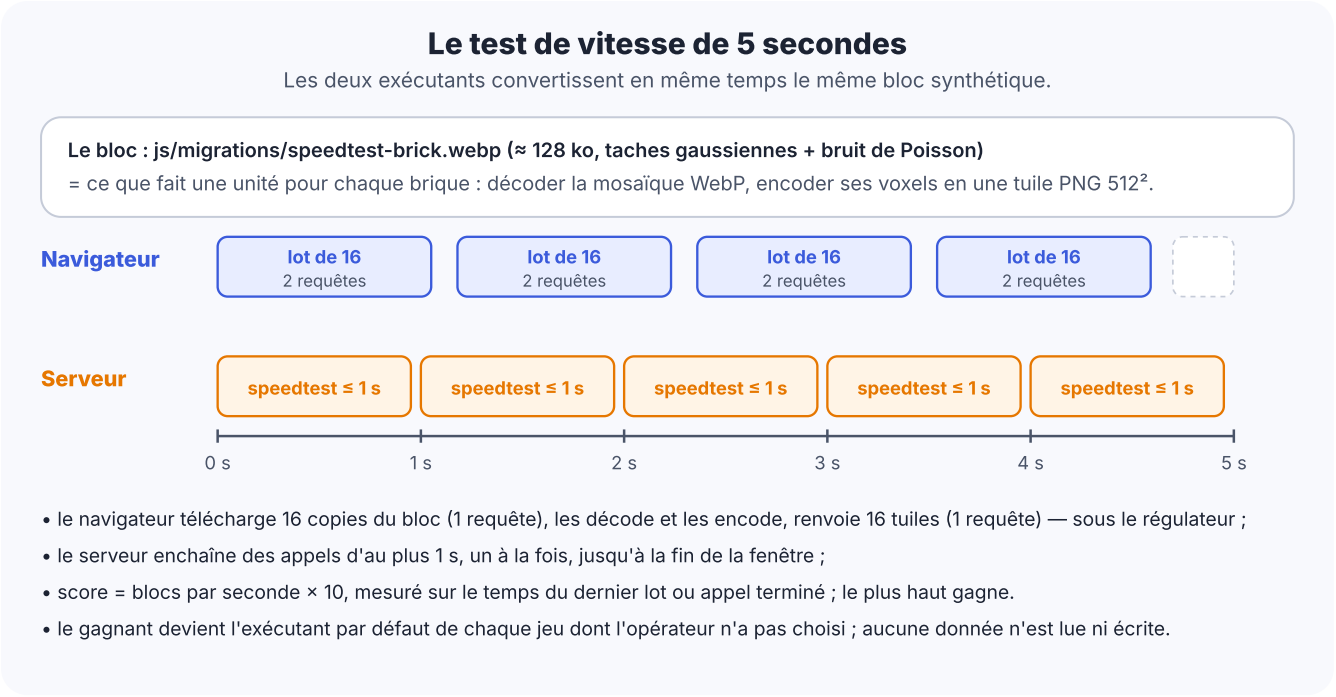
Qui est choisi quand personne ne choisit ?

Le choix se fait **par jeu**, avant le départ, et il est **verrouillé** pendant l'exécution. L'ordre de décision :

1. le choix manuel de l'opérateur (mémorisé dans ce navigateur) ;
2. sinon le **gagnant du test de vitesse** ;
3. sinon le navigateur ;
4. sinon le serveur.

Chaque exécutant a **sa propre file** : un jeu confié au navigateur et un autre confié au serveur avancent **en parallèle** ; deux jeux confiés au même exécutant avancent l'un après l'autre. Une étape que l'exécutant choisi ne sait pas faire est confiée à l'autre, **dans la même file** : les étapes d'un jeu restent dans l'ordre.

Le test de vitesse



▲ Cinq secondes, un même bloc synthétique, deux exécutants.

Le test ne lit et n'écrit **aucun jeu** : il utilise un bloc fixe livré avec la plateforme. Le **score** est le nombre de blocs convertis par seconde, multiplié par 10, mesuré sur la durée du dernier lot ou appel terminé : un lot coupé par l'échéance ne fausse pas la mesure. Si le navigateur ne termine aucun lot, il reçoit une fenêtre de plus. Le résultat est mémorisé dans ce navigateur.

Les actions de l'API

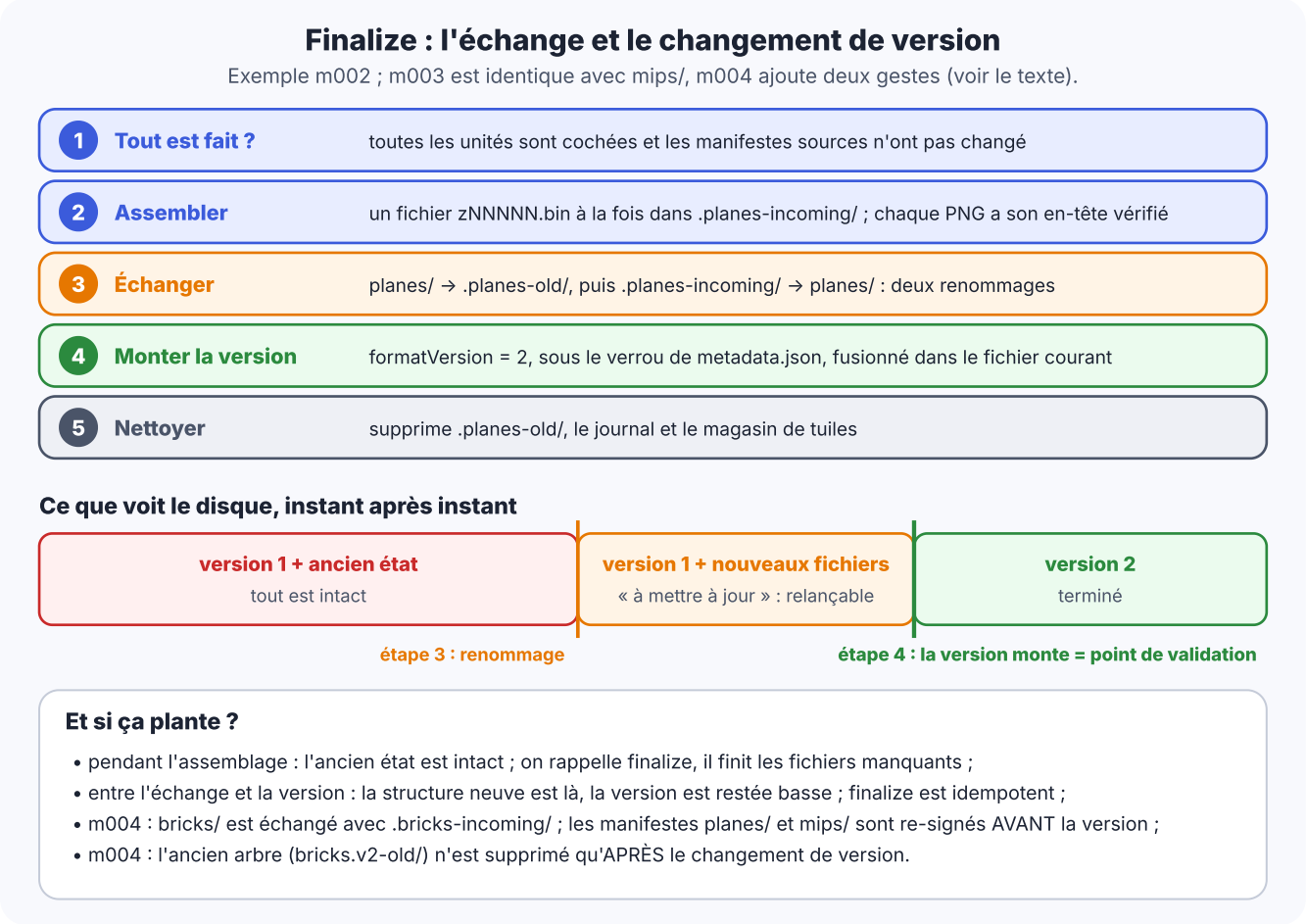
Toutes exigent la session d'administration ; celles qui modifient exigent aussi l'en-tête anti-CSRF et la méthode POST (chapitre 19).

ACTION	RÔLE
`status`	tout l'état : registre, jeux, capacités du serveur
`plan`	crée ou reprend un travail ; renvoie les unités exécutables
`unit_put`	reçoit le résultat d'une unité faite par le navigateur
`unit_run`	fait des unités côté serveur, dans un temps borné
`finalize`	assemble, échange, monte la version (rappelable)
`cancel`	supprime journal et magasin
`bench`	mesure le serveur sur quelques unités (outil)
`speedtest` , `speedtest_put` , `speedtest_sample`	les trois morceaux du test de vitesse
`read_ranges`	renvoie jusqu'à 1 024 plages d'octets (32 Mio) de paquets, en une réponse

ACTION	RÔLE
`store_get_many`	renvoie jusqu'à 128 briques v3 du magasin, en une réponse
`unit_inputs`, `store_get`	liste les entrées d'une unité ; lit une brique stockée

17.11 `finalize` : l'échange et le changement de version

Quand toutes les unités sont faites, il reste à **assembler** les résultats en une structure complète, à l'**installer** et à **changer le numéro de version**. C'est le rôle de `finalize`, **toujours exécuté par le serveur**, quel qu'ait été l'exécutant des unités.



▲ Les cinq gestes de finalize, et ce que voit le disque à chaque instant.

Deux idées font sa robustesse.

Il est reprenable. L'assemblage écrit les fichiers un par un dans le dossier voisin ; s'il est interrompu (limite de temps, coupure), il répond `complete: false` avec son avancement et on le **rappelle**. Il reprend où il s'était arrêté.

🔢 **MICRO-EXEMPLE**

Un essai réel. Avec un budget de temps minuscule, `finalize` a répondu `{complete: false, assembly: {planes: 130, written: 0}}` : 130 plans à écrire, aucun écrit. L'appel suivant a répondu `{complete: true, formatVersion: 2}`. Le journal passait par l'état `assembling`, avec `producer: "migration-server"`.

Le changement de version est le point de validation. Pendant tout l'assemblage et même après l'échange, l'ancien numéro reste en place. Si le serveur s'arrête entre l'échange et le changement de version, le jeu a ses **nouveaux fichiers** et l'**ancien numéro** : l'onglet le propose de nouveau, et `finalize` — idempotent — termine le travail.

Les gestes propres à chaque migration

- **m002 et m003** : `planes/` (ou `mips/`) est renommé en `.planes-old/`, le dossier voisin le remplace, puis l'ancien est supprimé.
- **m004** : `bricks/` est échangé avec `.bricks-incoming/` ; les manifestes de `planes/` et de `mips/` — qui avaient noté l'empreinte de l'ancien manifeste — sont **re-signés** avec celle du nouveau (leurs voxels, ceux du niveau natif, n'ont pas changé) ; la version monte ; **ensuite seulement** `bricks.v2-old/` est supprimé.
- avant d'échanger, chaque structure assemblée est **relue et validée** (`assembly_invalid` sinon) ;
- la version est montée **sous le verrou de** `metadata.json`, en **fusionnant** dans le fichier courant : un enregistrement fait dans l'éditeur pendant ce temps n'est pas perdu, aucun autre champ n'est touché ;
- sans journal (perdu par un plantage), `finalize` regarde si la structure est déjà là et valide : il se contente alors de re-signer et de monter la version.

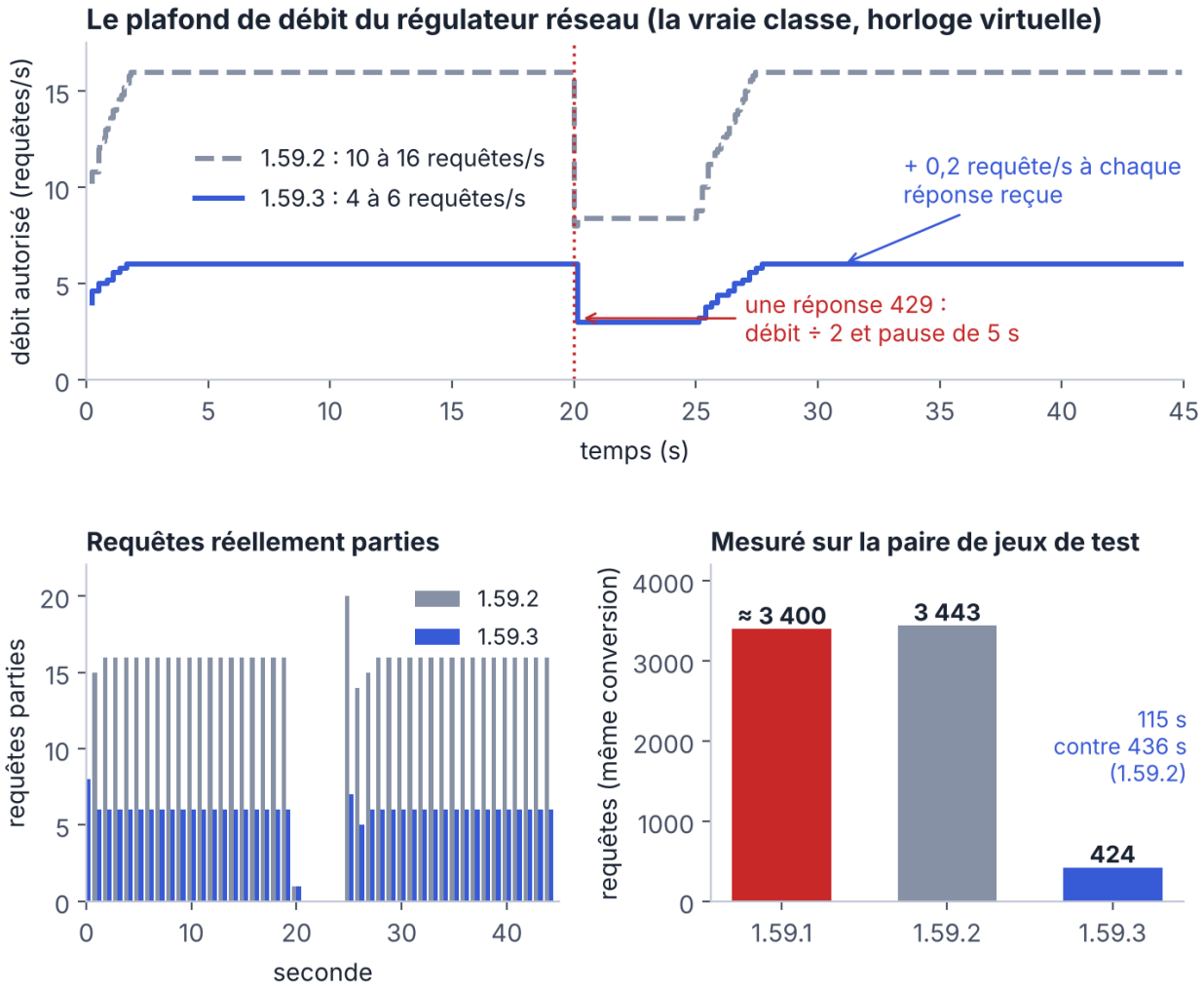
🔧 **POUR LES CURIEUX**

Pourquoi `bricks.v2-old/` reste jusqu'à la fin. Le viewer peut encore être en train de lire l'ancien arbre quand l'échange a lieu. Les noms de fichiers des paquets v3 (`l0/c0/p00000.bin`) ne ressemblent pas à ceux de la v2 (`lod0/c0/pack_00.bin`) et le manifeste porte un `schema` différent : un lecteur ne confond jamais les deux.

17.12 Le régulateur réseau et les hébergeurs lents

Un incident à l'origine du régulateur

En version 1.59.1, un afflux de requêtes (deux groupes de workers, une requête par brique stockée, un test de vitesse sans borne) a conduit un hébergeur à **bannir l'adresse** de l'opérateur. Depuis, **toutes** les requêtes de l'onglet — ses appels et ceux de ses workers — passent par **un seul régulateur** (`NetGovernor`).



▲ Le régulateur (vraie classe, horloge virtuelle) et ce que la 1.59.3 a gagné sur la paire de jeux de test.

Ses règles, mesurées dans le code :

- **6 requêtes en vol** au plus ;
- un **seau à jetons** : on démarre à **4 requêtes par seconde** et on ne dépasse jamais **6** (la 1.59.2 allait de 10 à 16, avant que les entrées d'une unité voyagent ensemble) ;
- une requête **sans réponse**, un **429** ou un **503** : le débit est **divisé par 2** (plancher 0,5 requête par seconde) et on **retient toute nouvelle requête** pendant 5 secondes, puis 10, 20… jusqu'à 60 tant que les échecs continuent ;
- chaque réponse reçue **relève** le débit de 0,2 requête par seconde.

L'onglet le dit : « L'hébergeur répond lentement : les requêtes sont ralenties pour qu'il ne bloque pas cette adresse. »

Environ trois requêtes par unité

Depuis la 1.59.3, une unité lit **toutes** ses plages d'entrée dans **une** réponse (`read_ranges`), ses briques de niveau *k* par paquets de 128 (`store_get_many`) et envoie son résultat en **une** requête. Mesuré sur la même paire de jeux de test (deux volumes de 2 canaux, formats 1 → 4) :

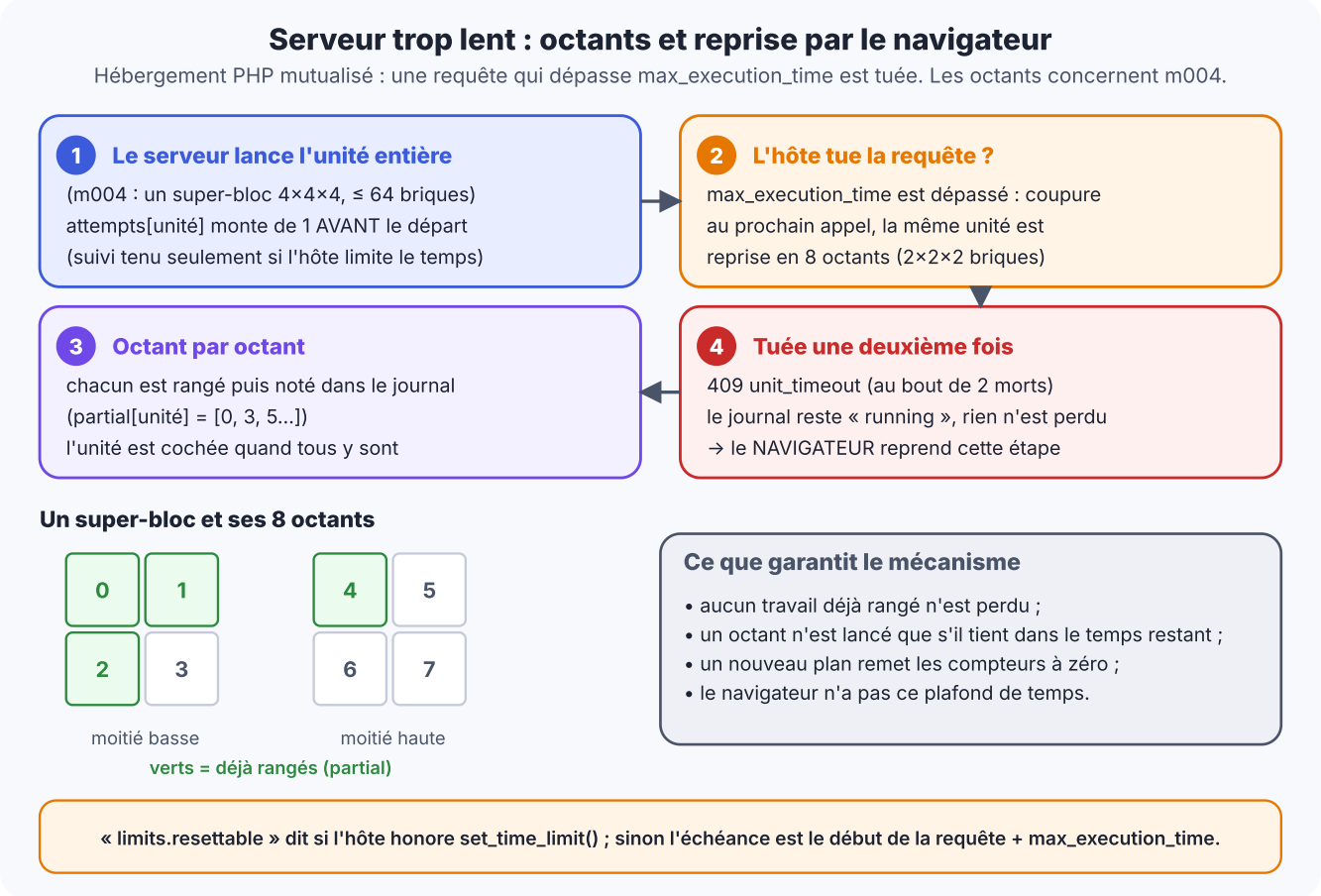
VERSION	REQUÊTES	DÉBIT MOYEN	DURÉE
1.59.1	environ 3 400 (plus de 40 par seconde)	—	—

VERSION	REQUÊTES	DÉBIT MOYEN	DURÉE
1.59.2	3 443	8 par seconde	436 s
1.59.3	**424**	3,7 par seconde (11 au plus dans la pire seconde)	**115 s**

Un serveur qui ne connaît pas `read_ranges` répond « action inconnue » (400) : le worker lit alors plage par plage, à l'ancienne.

Quand le serveur est trop lent : les octants

Sur un hébergement PHP mutualisé, une requête qui dépasse `max_execution_time` est **tuée** par l'hébergeur. Le serveur ne peut pas le savoir à l'avance ; il se protège donc.



▲ Une unité trop lente est découpée en 8 octants ; deux morts de requête passent la main au navigateur.

Avant chaque étape, le serveur appelle `set_time_limit()` ; `status.server.limits.resettable` dit si l'hôte l'honore. Puis :

1. **avant** de lancer une unité, il note `attempts[unité] + 1` dans le journal ;
2. pour m004, une unité peut être faite en **8 octants** (2 × 2 × 2 briques du super-bloc), chacun rangé et noté (`partial[unité]`) avant le suivant : l'unité est cochée quand tous y sont. Il le fait d'office tant qu'aucune unité entière du niveau n'a été chronométrée, quand la plus lente (+ 25 %) ne tiendrait pas dans le temps restant, ou après une mort de requête ;
3. une unité dont les étapes ont tué la requête **deux fois** reçoit la réponse **409** `unit_timeout`. Le journal reste `running` , **rien n'est perdu**, et l'onglet confie cette migration au navigateur, qui n'a pas

ce plafond.

Ce que dit l'onglet quand quelque chose ne va pas

CODE	MESSAGE À L'ÉCRAN
`source_changed`	le jeu de données a été retraité depuis le début de la mise à jour
`brick_undecodable`	une brique n'a pas pu être décodée
`incomplete`	certaines unités ne sont pas encore traitées
`no_progress`	le serveur n'a pas progressé
`request_failed`	une requête échoue à répétition
`prepare_failed`	le jeu de données n'a pas pu être lu
`unit_blocked`	un niveau inférieur de la pyramide n'est pas encore terminé
`bad_webp`	le serveur a refusé une brique convertie
`insufficient_disk`	espace disque insuffisant sur le serveur (requis / libre)
`no_executor`	ni ce navigateur ni le serveur ne peuvent exécuter cette mise à jour
`unit_timeout`	le serveur ne peut pas convertir une unité dans son délai

📌 **À RETENIR**

- Une migration = des **unités** cochées dans un **journal** ; deux exécutants, un navigateur ou un serveur, y écrivent.
- Le **dernier geste** est le changement de version, **sous le verrou** de `metadata.json` ; tant qu'il n'a pas eu lieu, rien n'est perdu.
- Le **régulateur** protège l'adresse de l'opérateur ; **trois requêtes par unité** suffisent depuis la 1.59.3.

17.13 Le parcours d'un import

Le chapitre 14 (section 14.3) montre l'onglet **Import**. Voici ce qui se passe **sous** chaque geste.

▲ Du dossier déposé au jeu publié : six étapes, trois qui transportent des octets.

1. Déposer : lire le dossier

Le navigateur lit le dossier glissé avec l'interface `webkitGetAsEntry`, seule à fonctionner pour un **dossier** dans tous les navigateurs actuels. Elle ne rend qu'une centaine d'entrées par appel : on **vide** chaque dossier en boucle, sinon un jeu serait tronqué à ses cent premiers paquets. Huit fichiers sont résolus à la fois (les ouvrir un par un demanderait des minutes pour 100 000 paquets) et une borne de 400 000 entrées protège la mémoire ; ce qui n'a pas pu être lu est **compté et signalé**.

Un jeu est **reconnu** par son `metadata.json`. Son **type** vient du champ `type` de ce fichier ; à défaut, du nom du dossier parent s'il s'appelle `3d`, `2d` ou `live`. Le **nom du dossier déposé est l'identifiant du jeu** : d'où le message « Déposez le DOSSIER du dataset, pas son contenu ».

> ⚠️ **ATTENTION**
>
> L'import exige une **connexion sécurisée** (HTTPS, ou `localhost`) : les empreintes SHA-256 reposent sur `crypto.subtle`, que le navigateur n'offre pas en HTTP simple. Le panneau le dit une seule fois, d'emblée, plutôt que d'échouer fichier par fichier.

2. Planifier : une requête par jeu

Pour chaque jeu, le navigateur envoie **la liste de ses fichiers avec leurs tailles** (jamais les octets). Le serveur **redérive** tout : la forme du type et du dossier, chaque chemin contre la liste blanche, et l'état de ce qui est déjà stocké ou publié. Il répond, pour chaque fichier : accepté ou refusé, palier, numéro de fichier (`fileId`), et — pour un fichier partiellement reçu — les **blocs manquants**.

Quelques garde-fous de la planification :

LIMITE	VALEUR
fichiers par jeu	200 000
jeux par dépôt	200

LIMITE	VALEUR
taille d'un fichier	1 Tio
place à laisser libre sur le disque	512 Mio

Si le disque ne peut pas contenir ce qui reste à envoyer, la réponse est `insufficient_disk` **avant le moindre octet** (avec `neededBytes` et `freeBytes`) : un disque plein en cours de route bloquerait tous les autres écrivains de l'hôte. Une liste trop grosse pour la taille de requête de l'hébergeur (`post_max_size`) est signalée aussi.

3 et 4. Envoyer et refermer

La suite est détaillée aux sections 17.16 (blocs et journal) et 17.18 (vérifier, publier). Tout le transfert se fait dans un **Web Worker** : lire une tranche de 8 Mio et la hacher coûte des dizaines de millisecondes de calcul pur ; sur le fil principal, ce serait une image perdue à chaque bloc.

Les actions de l'API d'import

ACTION	RÔLE
`limits`	taille de bloc, plafond, parallélisme, durée de grâce, type de serveur
`ping`	un aller-retour authentifié : distingue « réseau coupé » de « session expirée »
`list`, `state`	les jeux en cours d'import ; le détail d'un jeu ; **la liste lance aussi le nettoyage**
`plan`	planifie un jeu (ci-dessus)
`chunk`	reçoit un bloc
`file_done`	referme un fichier
`validate`	contrôle d'ensemble du jeu
`blob`	lit un fichier en cours d'import (aperçu) — sous session admin
`metadata`, `save_metadata`, `save_thumbnail`	l'édition pendant l'envoi
`publish`, `discard`, `gc`	publier, supprimer, nettoyer

17.14 La liste blanche

Seul ce que **produit le pipeline** est accepté. Tout le reste est refusé **avant le premier octet**, par deux barrières successives.

La liste blanche : tout ce qui n'est pas prévu est refusé

Deux barrières, dans l'ordre ; aucun octet n'est accepté avant d'avoir passé les deux.

Barrière 1 — la forme du chemin (_safe_rel)

refuse : octet nul, chemin absolu, « \ », « . » et « .. », fichier caché (« .xxx »), plus de 12 niveaux, segment > 200 caractères.
puis, sur le disque : resolve() doit retomber dans le dossier du jeu.

Barrière 2 — une des règles suivantes (classify_path), selon le type

	palier	types
✓ `metadata.json` `thumbnail.webp`	0	3d live 2d
✓ `bricks/manifest.json` `bricks/index.bin`	0	3d live
✓ `bricks/t001/index.bin` (une image)	0	live
✓ `preview.webp` `image.webp`	1 · 2	2d
✓ `tracks.json` `tracks.json.gz`	1	live
✓ `bricks/l3/c0/p00000.bin` `bricks/lod2/c1/pack_00.bin`	1…3	3d live
✓ `model.glb`	3	3d live
✓ `planes/z00007.bin` `planes/t001/…` `mips/l00000.bin`	4	3d live
✓ `download/<nom>.ims .tif .png .zip .txt …`	4	3d live 2d

Refusés (résultats réels du moteur)

✗ `evil.php`	pas de règle	✗ `.htaccess`	fichier caché
✗ `../x.png`	chemin remontant	✗ `download/report.html`	HTML/SVG/XML : rendu actif
✗ `gallery/a.png`	la galerie ne voyage pas	✗ `bricks/l0/c0/p0.bin`	nom de paquet hors motif

Un « .php » caché dans « a.php.png » passe (extension finale png) : download/ n'est jamais exécuté et toujours servi en pièce jointe.

▲ Deux barrières : la forme du chemin, puis une règle explicite par type de jeu.

❓ POURQUOI CE CHOIX ?

Pourquoi une liste fermée plutôt qu'une liste d'interdits ? Une liste d'interdits oublie toujours quelque chose (une extension rare, un nom de périphérique Windows). Une liste de ce qui est **permis** ne peut pas oublier : ce qu'on n'a pas prévu est refusé. La zone d'arrivée est de plus **jamais servie** (chapitre 19), et la publication n'a lieu qu'après validation.

Les détails des règles, tels que le moteur les applique :

- les paquets : `lod<N>/c<M>/pack_<n>.bin` (format 1 à 3), `l<k>/c<c>/p<NNNNN>.bin` (format 4) ou `l<k>/<colonne>/c<c>/p<NNNNN>.bin` (format 5), sous `bricks/` ou sous `bricks/tNNN/` pour une série (type `live` seulement) ;
- format 5 : les index `index/<colonne>.bin` (palier 0), les copies de coupe `xy/`, `xz/`, `yz/` et `cuts/` (dernier palier, comme `planes/`) ;
- `planes/` et `mips/` : `manifest.json` et les paquets nommés (`zNNNNN.bin`, `lNNNNN.bin`), avec un sous-dossier `tNNN/` pour une série ;
- `download/` : un nom simple (pas de sous-dossier), jusqu'à 181 caractères, extension de **données** (liste de la section 17.6), **jugée sur sa dernière extension** ; les noms réservés de Windows (`CON`, `NUL`…) et ceux qui finissent par un point ou une espace sont refusés ;
- jamais acceptés : les fichiers `.php`, `.js`, `.htaccess`, tout fichier caché, tout chemin remontant (`..`), et les formats qu'un navigateur **exécute** à l'ouverture (HTML, SVG, XML).

ℹ️ **À SAVOIR**

Un fichier nommé `a.php.png` **passe** (sa dernière extension est `png`) : ce n'est pas une faille, car `download/` n'est jamais exécuté et toujours servi en pièce jointe. Ce qui compte, c'est que le fichier arrive dans un dossier où **rien ne s'exécute**.

17.15 Les paliers : pourquoi un jeu est éditable avant la fin

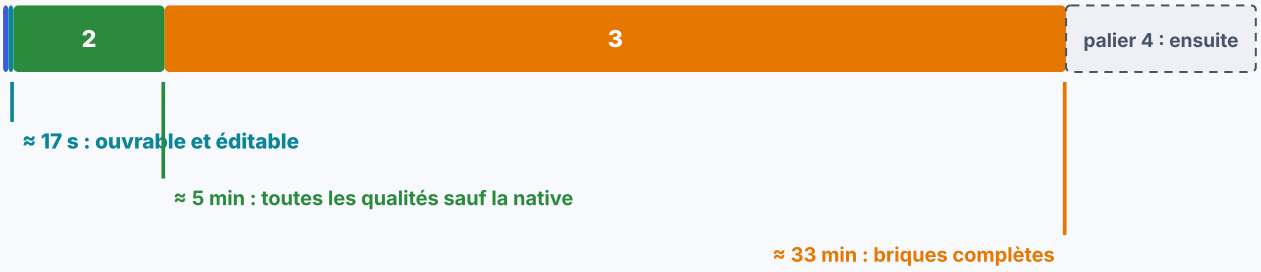
Un jeu complet pèse des dizaines de gigaoctets, et l'envoi peut durer des heures. Les fichiers sont donc envoyés **par priorité** : les plus utiles d'abord.

Les paliers : ouvrable en secondes, complet en heures

Haut : les vrais octets du jeu de démonstration. Bas : le même partage appliqué à 10 Go de briques, à 5 Mo/s.

palier	ce qu'il contient	fichiers	octets
0 · cœur	metadata.json, manifeste, index.bin, vignette	4	47,0 ko
1 · aperçu	le niveau le plus grossier, tous canaux	3	31,9 ko
2 · niveaux intermédiaires	les niveaux entre les deux, du plus grossier au plus fin	6	1,3 Mo
3 · niveau natif	niveau 0 ; images 1, 2, 3… d'une série	6	7,9 Mo
4 · planes, mips, originaux	planes/, mips/, download/ (hors les originaux : 530 Mo)	117	9,1 Mo

Scénario : 10 Go de briques (mêmes proportions) à 5 Mo/s



Dès le palier 1, le jeu est ouvrable en basse résolution ; une fois édité, metadata.json est verrouillé : la suite du transfert ne remplace pas vos réglages.

▲ Les cinq paliers, avec les vrais octets du jeu de démonstration et un scénario de 10 Go à 5 Mo/s.

PALIER	CONTENU	POURQUOI DANS CET ORDRE
0 · cœur	`metadata.json`, manifeste, `index.bin`, vignette	de quoi **monter** le jeu
1 · aperçu	le niveau le plus **grossier**, tous canaux (et `tracks.json` pour une série)	assez pour afficher quelque chose
2 · intermédiaires	les niveaux entre le plus grossier et le natif, du plus grossier au plus fin	la qualité monte progressivement
3 · natif	le niveau 0, et les images 1, 2, 3… d'une série	du travail de pleine qualité
4 · le reste	`planes/`, `mips/`, `download/`	jamais nécessaires pour **ouvrir**

Le palier d'un paquet dépend de l'**ensemble** du jeu (quel niveau est le plus grossier ?) : le serveur le recalcule donc une fois la liste complète connue. Une série ouvre sur sa **première image** ; les suivantes sont du palier 3.

> 🔢 **MICRO-EXEMPLE**
>
> **Le scénario de la figure.** Le jeu de démonstration a 47 ko de palier 0 et 32 ko de palier 1 : à l'échelle de 10 Go de briques (mêmes proportions) et à 5 Mo/s, le jeu est **ouvrable et éditable en environ 17 secondes**, toutes les qualités sauf la native sont là en 5 minutes, les briques complètes en 33 minutes. Les paliers 4, eux, peuvent durer plus longtemps que tout le reste.

Quand le jeu devient « éditable »

Le jeu passe à l'état **éditable** quand **tous** les fichiers de palier 0 et 1 sont terminés et que `metadata.json` et un fichier de montage (manifeste ou aperçu 2D) sont arrivés. Vous pouvez alors l'ouvrir en basse résolution et renommer, régler les canaux, orienter.

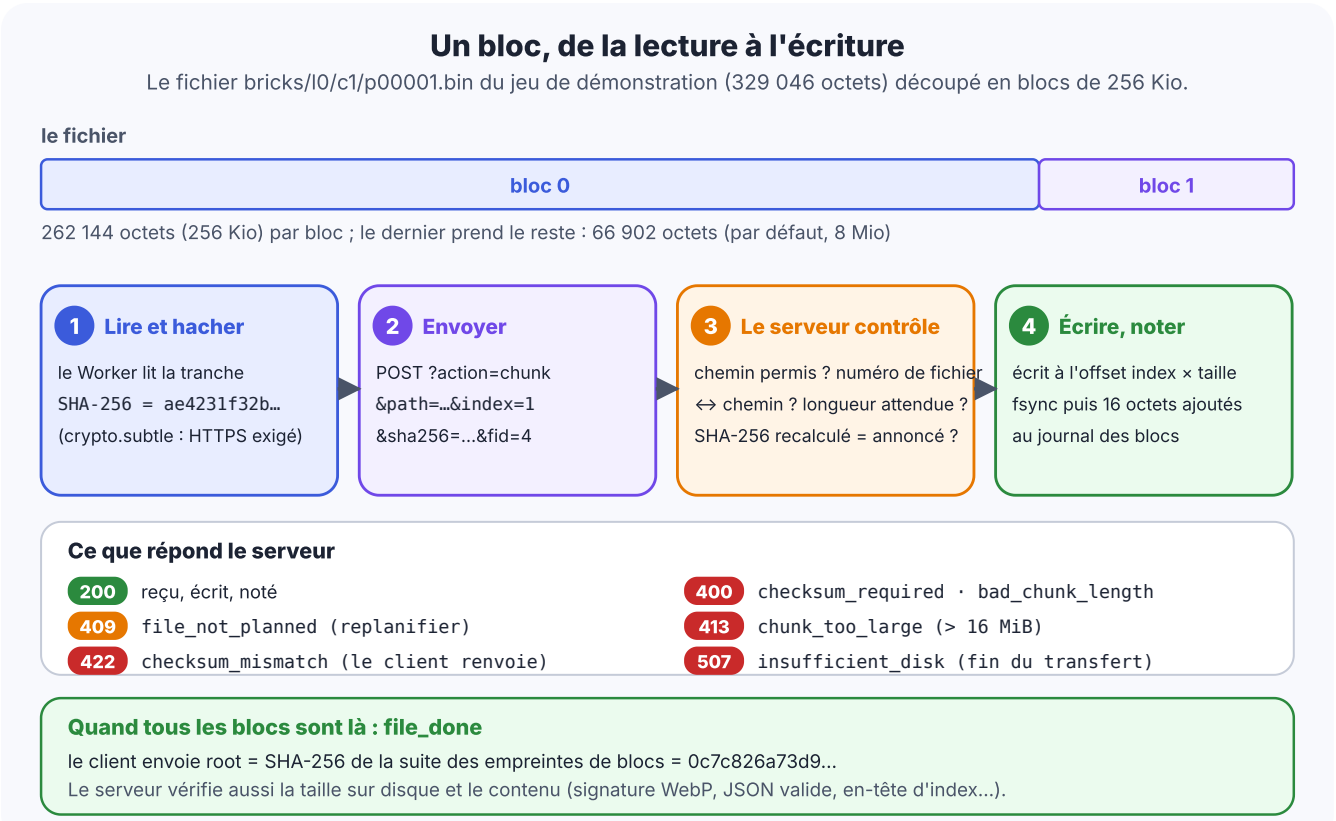
Le verrou de `metadata.json`

Dès que vous **enregistrez** une édition sur un jeu en cours d'envoi, le serveur écrit votre `metadata.json` et pose un verrou (`metaLocked`). Le transfert **ne le réécrit plus** : la réception d'un nouveau bloc de `metadata.json`, ou une nouvelle planification, saute le fichier. Sans cela, la fin du transfert aurait silencieusement **effacé votre travail** en renvoyant le fichier d'origine.

17.16 Les blocs et le journal d'import

Un bloc

Chaque fichier est découpé en **blocs de 8 Mio** par défaut. Le serveur peut imposer plus petit (jamais plus grand) : un hébergeur PHP limite la taille d'une requête.



▲ Un bloc, de la lecture à l'écriture : hachage, contrôle, écriture à son décalage exact, note au journal.

LIMITE DE BLOCS	VALEUR
taille par défaut	8 Mio
plafond (Python)	16 Mio
plancher	256 Kio
hôte PHP	80 % de `post_max_size`, et au plus le quart de `memory_limit`
blocs en parallèle	4

Si un hébergeur refuse la taille choisie (`body_truncated`), l'interface **réduit** la taille, replanifie et dit « Taille de bloc réduite à … — relancez le dossier pour reprendre ». Les blocs déjà reçus restent valables : ils sont comptés d'après la taille **enregistrée** pour chaque fichier.

🔢 **MICRO-EXEMPLE**

Le chiffre de l'exemple. Un original de 22 Gio représente 2 816 blocs de 8 Mio ; la liste de ce qui est reçu tient en 2 816 bits, soit **352 octets**. Les blocs peuvent arriver **dans le désordre et en parallèle** : chacun est écrit à son décalage exact (`indice × taille de bloc`).

Trois niveaux de contrôle

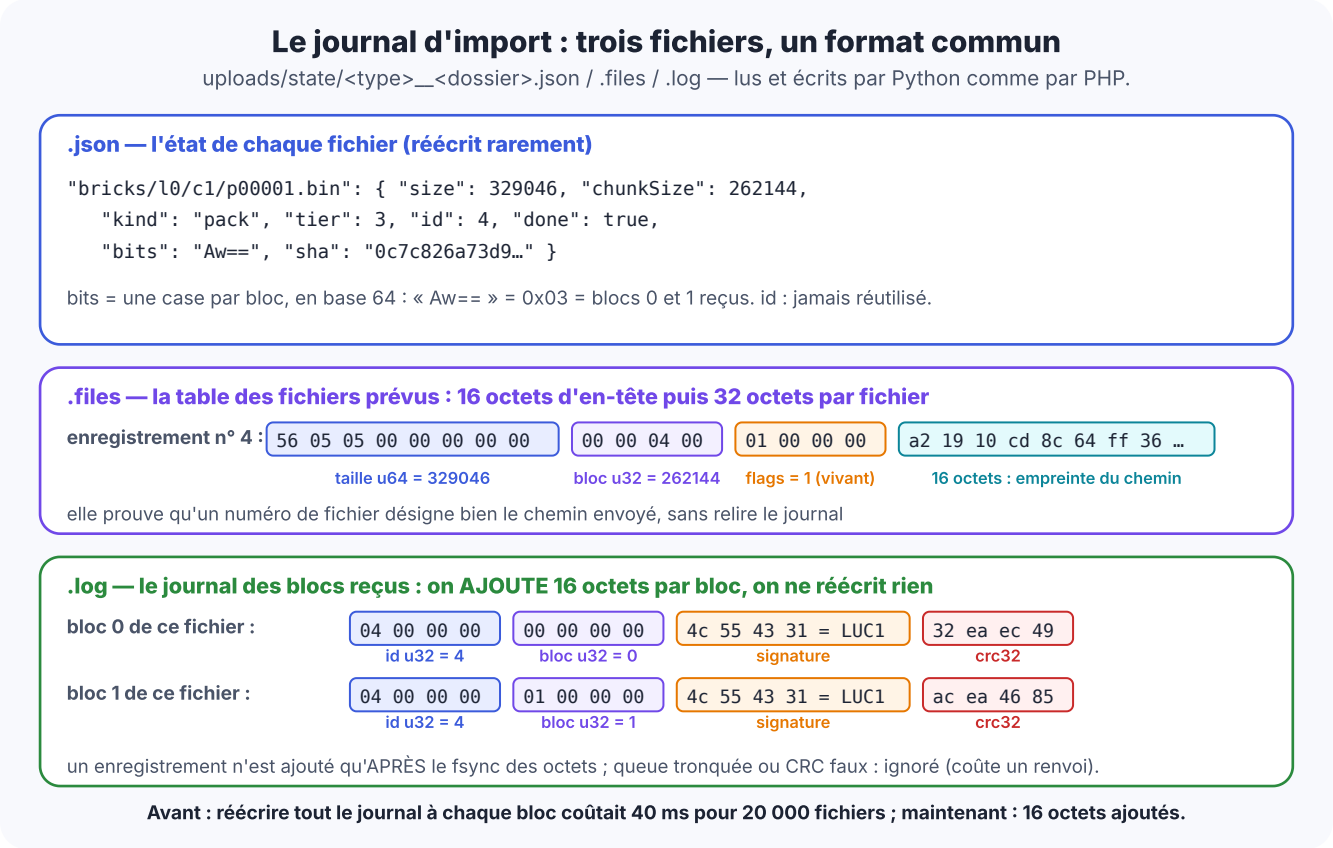
QUAND	QUOI
à chaque bloc	SHA-256 recalculé **avant** d'écrire ; longueur attendue ; le numéro de fichier correspond-il au chemin ?
à la fermeture d'un fichier	tous les blocs présents, taille sur disque exacte, **empreinte globale** = SHA-256 de la suite des empreintes de blocs, contenu plausible (signature WebP, JSON valide, en-tête d'index…)
à la validation du jeu	cohérence d'ensemble (section 17.18)

❓ **POURQUOI CE CHOIX ?**

Pourquoi une « empreinte des empreintes » ? Un navigateur ne sait pas hacher un fichier de 22 Gio en continu, et le relire une seconde fois doublerait le travail. L'empreinte de la **suite des empreintes de blocs** prouve la même chose — chaque bloc vérifié, dans le bon ordre, aucun manquant — **sans** relire quoi que ce soit.

Quand l'envoi échoue, deux familles de pannes sont traitées différemment : une panne **du serveur** (erreur 5xx, saturation, empreinte fausse) consomme une des **6 tentatives**, avec une attente croissante de 1 à 30 secondes (± 25 %) ; une panne **du réseau** (aucune réponse) ne consomme **rien** : tout le transfert se met en attente jusqu'au retour de la connexion. Une panne de plusieurs minutes coûte du temps, jamais des fichiers.

Le journal : trois fichiers, un format commun



▲ Le journal d'import : l'état de chaque fichier, la table des fichiers prévus, le journal des blocs reçus.

Ils vivent dans `uploads/state/` et portent le nom `<type>__<dossier>`. Python et PHP les lisent **et** les écrivent : **un import commencé sous un serveur reprend sous l'autre**.

FICHIER	RÔLE	RÉÉCRIT
`.json`	l'état de chaque fichier : taille, taille de bloc, palier, nature, numéro, case par bloc (base 64), `done`	à la planification et à chaque fichier terminé
`.files`	la table des fichiers prévus : 16 octets d'en-tête (`LUFT`) puis 32 octets par fichier	quand un numéro change
`.log`	un enregistrement de **16 octets par bloc reçu** : numéro de fichier, indice de bloc, `LUC1` , CRC32	**jamais** : on ajoute seulement

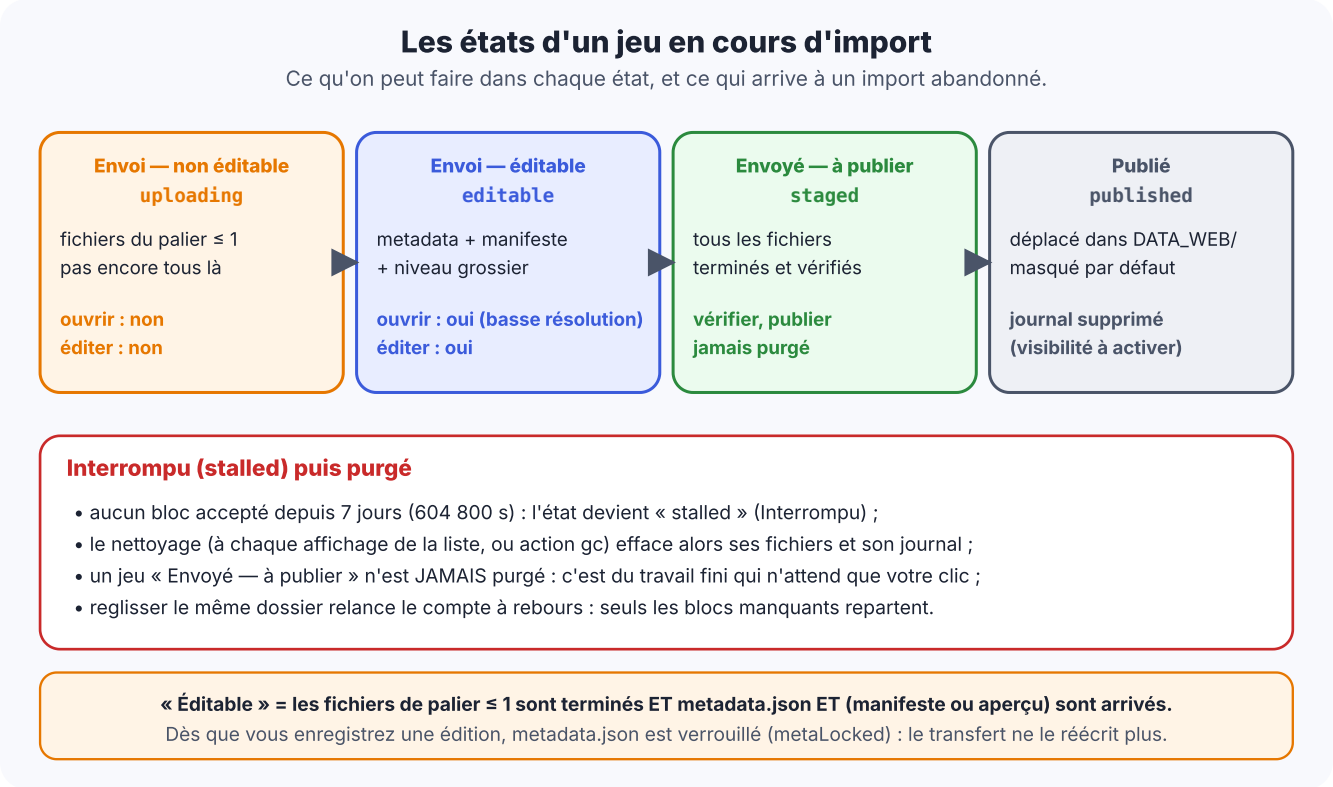
🔧 POUR LES CURIEUX

Pourquoi un journal « à ajout seulement » ? Réécrire tout le fichier JSON à chaque bloc de 8 Mio coûtait 40 ms pour 20 000 fichiers et une demi-seconde pour 200 000, **sous le verrou du jeu** : les quatre flux parallèles se faisaient la queue. Ajouter 16 octets ne coûte presque rien. Le journal est **compacté** (replié dans le `.json` , puis le `.log` est vidé) à la planification, à la fin d'un fichier et à chaque écriture du journal.

Ce qui rend le journal sûr même après un plantage :

- un enregistrement n'est ajouté qu'**après** l'`fsync` des octets : un enregistrement qui survit décrit toujours des octets réellement sur le disque ;
- une queue tronquée, ou un enregistrement dont la signature ou le CRC ne correspond pas, est **ignoré** : cela coûte au pire un renvoi ;
- le numéro de fichier n'est **jamais réutilisé** : quand un fichier est replanifié (taille changée) ou remis à zéro, il reçoit un nouveau numéro, et un enregistrement écrit pour l'ancien ne peut plus tomber sur le nouveau ;
- le chemin du fichier est vérifié par une **empreinte de 16 octets** stockée dans la table : on prouve qu'un numéro désigne bien le chemin envoyé sans relire le journal.

17.17 Les états d'un import et la purge



▲ Quatre états ; un jeu abandonné est purgé au bout de sept jours, un jeu terminé jamais.

ÉTAT (CODE)	LIBELLÉ À L'ÉCRAN	CONDITION
`uploading`	Envoi — non éditable	les fichiers des paliers 0 et 1 ne sont pas tous là
`editable`	Envoi — éditable	paliers 0 et 1 terminés, `metadata.json` et un fichier de montage arrivés
`staged`	Envoyé — à publier	**tous** les fichiers planifiés sont terminés
`stalled`	Interrompu	aucun bloc accepté depuis 7 jours

La purge à sept jours

Un jeu que personne ne termine occupe du vrai disque dans la zone d'arrivée. L'opérateur a **sept jours** (604 800 secondes) pour glisser de nouveau le dossier et reprendre ; passé ce délai, le nettoyage libère ses fichiers et son journal, et l'envoi repart de zéro.

- le nettoyage tourne **à chaque affichage de la liste**, ou à la demande (action `gc`) ;
- le compte à rebours se mesure depuis la **dernière activité** (un fichier qui s'envoie pendant des heures compte comme de l'activité) ;
- un jeu Envoyé — à publier n'est **jamais** purgé : il est complet et validé, il n'attend que votre clic. Le supprimer au bout d'une semaine détruirait des dizaines de gigaoctets de travail fini pour épargner un renvoi que personne n'a demandé.

Qu'est-ce qui reste lisible pendant l'envoi ?

L'aperçu d'un jeu non publié passe par l'action `blob` : un fichier de la zone d'arrivée est servi, **sous session d'administration**, comme des octets opaques (avec `nosniff`), jamais comme un document. La zone d'arrivée n'est atteignable par **aucune adresse** ; elle est bloquée dans le serveur Python, dans le `.htaccess` racine, dans `router.php` et par un fichier d'interdiction qui lui est propre.

17.18 Valider, publier, remplacer

Valider

L'action de validation relit le jeu **en entier** et vérifie qu'il forme un tout. Elle renvoie une liste de codes ; il y en a quelques dizaines, dont voici les familles.

CODE	CE QUE CELA VEUT DIRE
`missing_metadata`, `metadata_not_json`	pas de `metadata.json`, ou illisible
`metadata_type_mismatch`	le type déclaré n'est pas celui du dossier de dépôt
`metadata_no_dimensions`, `metadata_bad_dimensions`, `metadata_no_channels`	dimensions ou canaux invalides
`metadata_no_image`, `metadata_bad_image`	(2D) bloc `image` absent ou noms de fichiers non conformes
`missing_manifest`, `manifest_not_json`	pas de `bricks/manifest.json`, ou illisible
`manifest_no_levels`, `manifest_level_N_…`, `manifest_bad_brick_size`, `manifest_bad_encoding` …	manifeste incohérent (niveaux, 64 voxels, codage)
`missing_index:…`, `index_hash_mismatch:…`	`index.bin` absent, ou différent de l'empreinte du manifeste
`index_bad_magic`, `index_bad_length`, `index_shape_mismatch:…`, `index_grid_mismatch:…`	l'index n'a pas la bonne forme ou ne décrit pas les niveaux du manifeste
`missing_pack:<chemin>`	un paquet que l'index cite n'est pas arrivé
`manifest_v5_…`, `missing_store:<chemin>`	(format 5) manifeste v5 incohérent, ou une copie de coupe (`xy/`, `xz/`, `yz/`) manque
`truncated_pack:<chemin>`	un paquet est plus court que ce que l'index réclame

CODE	CE QUE CELA VEUT DIRE
`manifest_pack_escapes`	un paquet pointe hors du dossier du jeu
`incomplete_files`	des fichiers planifiés ne sont pas terminés
`stray_files`	un fichier présent dans la zone d'arrivée n'a jamais été accepté par un plan
`missing_preview`, `missing_image`	(2D) une des deux copies manque ou est vide

🔢 **MICRO-EXEMPLE**

Essais réels sur le jeu de démonstration, dans un dossier temporaire. Tout est arrivé : `{ok: true}`. On **tronque** de 1 000 octets le paquet `l0/c1/p00001.bin` : `truncated_pack:l0/c1/p00001.bin`. On le **supprime** : `missing_pack:l0/c1/p00001.bin`. On **altère** `index.bin` : `index_hash_mismatch:bricks`. On dépose un fichier `stray.txt` : `stray_files`. Un bloc dont l'empreinte est fausse reçoit `422 checksum_mismatch` ; un bloc sans empreinte, `400 checksum_required`.

Un paquet tronqué passe ses propres empreintes de blocs si le client n'a simplement jamais envoyé la fin : seule la validation d'ensemble l'attrape, **avant** qu'un visiteur ne voie un écran noir.

Publier

▲ Valider, reprendre le travail de l'opérateur, échanger les dossiers, conclure.

La publication déplace le jeu validé de la zone d'arrivée vers `DATA_WEB/<type>/<dossier>/`.

- elle est **masquée par défaut** : l'opérateur décide quand le public voit le jeu ;

- c'est un **renommage** (quasi atomique) quand la zone d'arrivée et `DATA_WEB/` sont sur le même disque ; sinon, une copie dans un dossier voisin `.incoming-…` puis un renommage, pour qu'un jeu à moitié copié ne soit jamais visible sous son vrai nom ;
- avant de déplacer, les fichiers temporaires d'écriture sont balayés : ils ne voyagent jamais jusqu'au site.

Remplacer un jeu déjà publié

Si le dossier de destination existe, la publication répond `409 already_exists`, et l'interface demande de confirmer le remplacement. Avec `overwrite`, dans l'ordre :

1. les clés curées de l'ancien fichier que le nouveau **ne contient pas** sont reprises (section 17.2) ;
2. l'ancien dossier est renommé `.replaced-<jeu>-<date>` ;
3. le nouveau est renommé à sa place ;
4. la **galerie** de l'ancien dossier (images et vignettes) est déplacée dans le nouveau, puis la liste `gallery` ne garde que les fichiers réellement présents ;
5. l'ancien dossier est supprimé, ainsi que le journal d'import.

Si le serveur s'arrête **entre les deux renommages**, au démarrage suivant `recover_publish_leftovers()` remet l'ancien jeu en place ; un `.replaced-…` dont le jeu existe est supprimé, un `.incoming-…` (copie partielle) l'est toujours.

> ℹ️ **À SAVOIR**
>
> L'interface ne **renvoie pas** un dossier dont le nom existe déjà dans `DATA_WEB/` : elle le considère comme « déjà publié » plutôt que de renvoyer des gigaoctets. Le remplacement se propose au moment de **Publier**, quand le serveur signale que le nom existe.

Ce qu'on retient

> 📌 **À RETENIR**
>
> - Une **liste blanche fermée** : ce qui n'est pas prévu est refusé avant le premier octet ; la zone d'arrivée n'est **jamais servie**.
> - Des **paliers** : les fichiers qui rendent le jeu ouvrable partent d'abord, et un jeu édité garde son `metadata.json`.
> - Des **blocs hachés** et un **journal à ajout seulement** : une coupure coûte au pire un bloc, et l'import reprend sous l'un ou l'autre serveur.
> - Une **publication « tout ou rien »**, masquée par défaut, qui reprend le travail de l'opérateur et la galerie.

> 📖 **POUR ALLER PLUS LOIN**
>
> Les écrans, les boutons et le dock flottant de l'import sont dans le chapitre 14 ; la sécurité de la zone d'arrivée dans le chapitre 19 ; le contrat exact des migrations dans `DOCS/dataset-migrations/SPEC.md`.

18. Héberger, mettre à jour, publier une version

- Lumen3D se sert depuis **trois sortes de serveurs** (Python, PHP, ou simples fichiers). Ce chapitre dit ce que chacun sait faire, comment il répond au navigateur et pourquoi les pages restent rapides.
- Une **mise à jour** est un colis **signé** : la plateforme refuse tout ce qu'elle ne peut pas prouver authentique, puis bascule en gardant un chemin de retour (sur Python).
- Chaque version est **testée en entier** (201 fichiers de tests), signée, puis publiée d'un coup. La plateforme marche **sans Internet**.

Ce chapitre s'adresse à la personne qui **installe et entretient** le site : l'administrateur, ou la personne du service informatique qui l'aide. Les chapitres 9 à 12 expliquent ce que voit le visiteur ; celui-ci explique **la machine derrière**.

Rien ici n'oblige à écrire du code. Mais comprendre ces mécanismes permet de **diagnostiquer** un site lent, une mise à jour refusée ou un hébergeur capricieux.

ℹ️ **À SAVOIR**

Les numéros de version cités sont ceux de ce document : plateforme web **1.65.0**, pipeline **0.22.0**, format de données **5**. Le panneau d'administration en bref est au chapitre 14 ; la sécurité en général au chapitre 19.

18.1 Trois façons de servir Lumen3D

Le navigateur parle toujours le même langage (HTTP). Ce qui change, c'est **le programme qui répond** de l'autre côté. Il en existe trois familles.

Qui sait faire quoi : les façons de servir Lumen3D

	dev_server.py Python	fast_server.py Python, lecture	api/*.php PHP + Apache	Hôte statique fichiers seuls
Distribue les fichiers	oui	oui	oui	oui
Nonce + CSP sur les pages HTML	oui	oui	oui	non
Catalogue des jeux (à la demande)	oui	oui	oui	non
API d'administration	oui	non	oui	non
Import par le navigateur	oui	non	oui	non
Mise à jour de la plateforme	avec retour arrière	non	sans retour arrière	non
Sonde /api/health	oui	oui	non	non

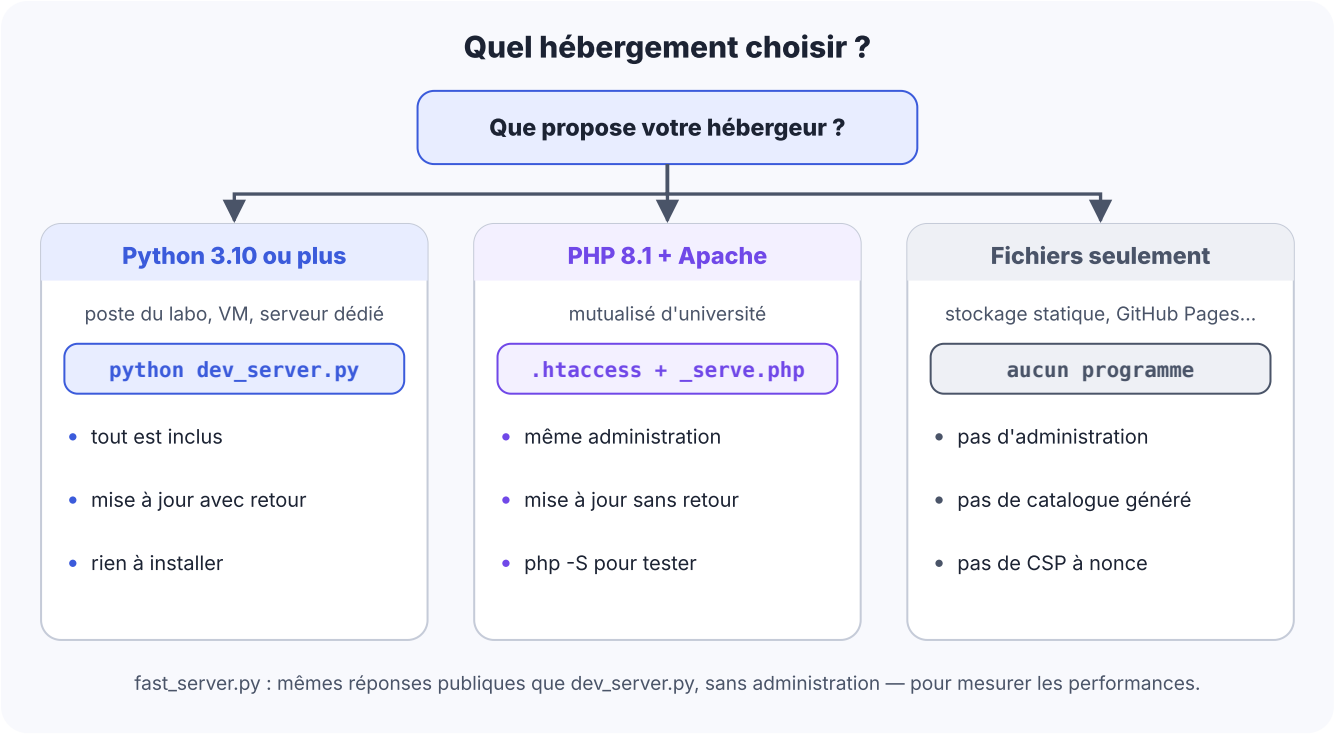
▲ Ce que sait faire chaque façon de servir : de la plus complète à la plus simple.

💡 **UNE IMAGE POUR COMPRENDRE**

Un guichet de bibliothèque. Le guichet complet (Python ou PHP) vérifie votre carte, cherche le livre, tient le registre des prêts. Un simple rayonnage en libre-service (hôte statique) vous laisse prendre un livre, mais ne tient aucun registre et ne vérifie personne.

- `dev_server.py` : le serveur Python. Il fait **tout** (pages, catalogue, administration, import, mise à jour). C'est le serveur recommandé.
- `fast_server.py` : le même serveur, en **lecture seule**. Il répond comme `dev_server.py` à un visiteur, mais **toute route d'administration est refusée** (404). Il sert à mesurer les performances.
- **PHP** (dossier `api/`, `router.php`, `_serve.php`, `.htaccess`) : pour les hébergements d'université qui offrent PHP mais pas Python. Mêmes fonctions, mêmes fichiers, même mot de passe.
- **Hôte « fichiers seulement »** : possible pour regarder, mais sans administration, sans catalogue généré et sans la protection à « nonce » du chapitre 19.

Lequel choisir ?



▲ L'arbre de décision d'un hébergement.

🔢 MICRO-EXEMPLE

Un laboratoire avec un poste dédié. Il lance `python dev_server.py --dev-trust-local` : une seule commande, rien à installer, mise à jour avec retour arrière. **Une université qui ne propose que PHP** : elle dépose `install.php` (§18.14) ; Apache lit le fichier `.htaccess` fourni, qui aiguille les pages vers `_serve.php`.

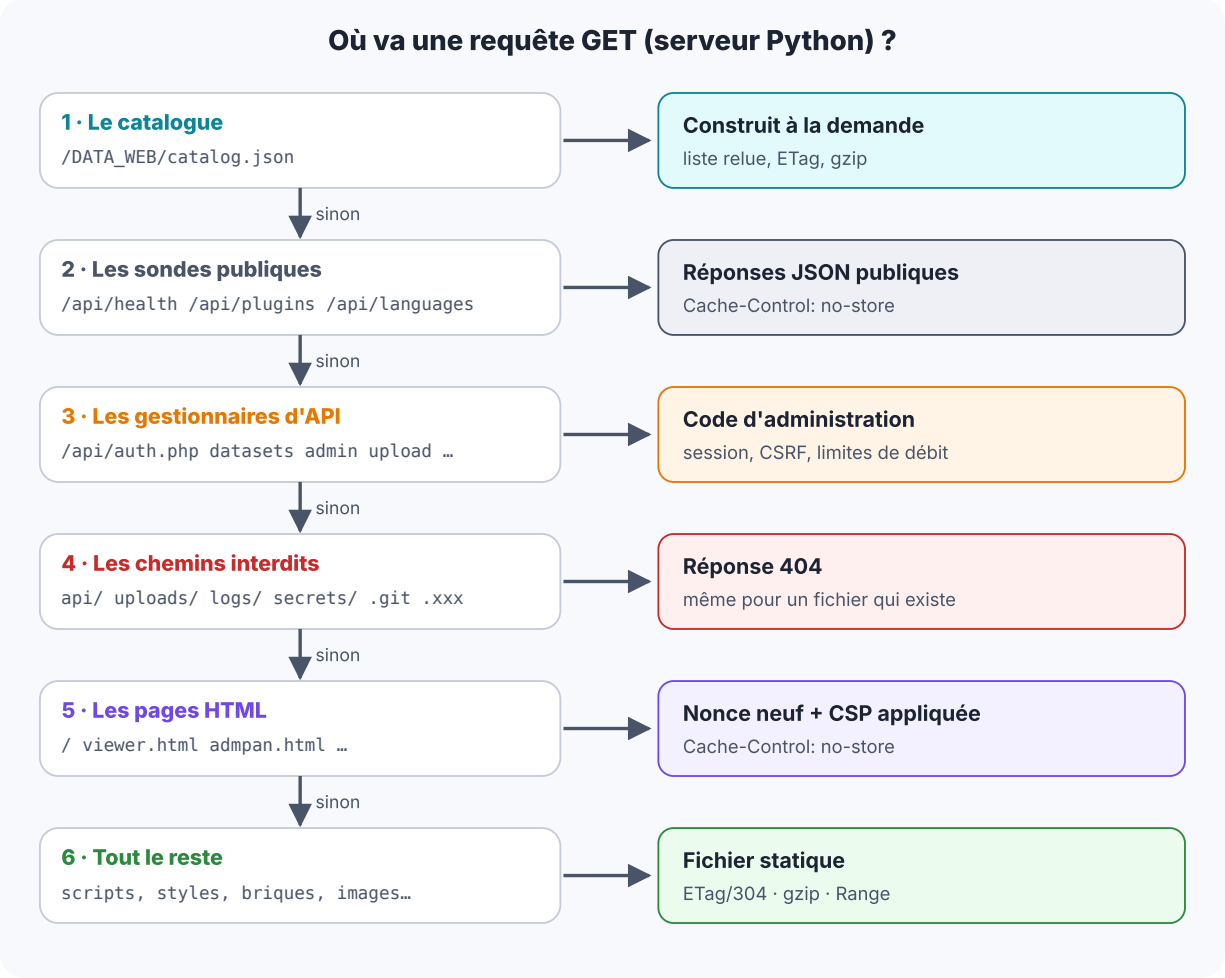
🔧 POUR LES CURIEUX

Options de `dev_server.py` : `--host` (défaut `localhost`), `--port` (défaut `8080`), `--set-password` (changer le mot de passe administrateur en ligne de commande), `--check` (valider une installation sans ouvrir de port, §18.10), `--root` (installation à contrôler), `--dev-trust-local` (faire confiance aux plugins locaux ; **réservé au poste de développement**), `--trusted-proxy` (§18.6), `--migrate-types`, `--verbose` (journaliser chaque requête). Par défaut aucune requête n'est écrite dans la console : une écriture synchrone par requête ralentirait les réponses.

Versions minimales : Python 3.10 (le README l'indique) ; PHP 8.1, version testée par l'intégration continue avec les extensions `zip`, `mbstring`, `sodium` et `gd`. L'installeur `install.php` n'exige, lui, que PHP 7.4 pour démarrer.

18.2 Le trajet d'une requête

Un visiteur demande une adresse. Le serveur Python range cette demande dans **une des six catégories** ci-dessous, dans cet ordre. Dès que l'une convient, il s'arrête.



▲ Les six destinations d'une requête GET sur le serveur Python.

- Le **catalogue** et les **sondes** sont des réponses fabriquées, jamais des fichiers.
- Les chemins **interdits** reçoivent un 404, **même si le fichier existe** : on ne confirme pas qu'il est là.
- Les pages `.html` passent par un traitement à part : un code secret neuf (le « nonce ») est inséré dans chaque page à chaque chargement (chapitre 19).
- Tout le reste est un **fichier statique** : briques, scripts, images.

Le catalogue n'est pas un fichier

L'adresse `DATA_WEB/catalog.json` **n'existe pas sur le disque**. Le serveur la fabrique à la demande en lisant les fiches `metadata.json` de chaque dossier de `DATA_WEB/3d`, `2d` et `live`.

- Sont listés les jeux **configurés ou dotés d'une vignette**, et **non masqués**.
- Tri par date décroissante ; un jeu sans date passe **en dernier**.
- Résultat mémorisé, avec une empreinte (ETag) ; la liste n'est recalculée que si une fiche a changé (le disque n'est consulté qu'une fois toutes les 2 secondes au plus).

🔢 **MICRO-EXEMPLE**

Ajouter un jeu à la main. Vous copiez un dossier terminé dans `DATA_WEB/3d/` : à la prochaine visite, il apparaît dans l'Explorateur. Rien à régénérer. Sur un hôte PHP, c'est `api/catalog.php` qui fait le même travail, appelé par une règle de réécriture du `.htaccess`.

🔧 POUR LES CURIEUX

Sur Apache, la réécriture est indispensable : sans le module `mod_rewrite`, l'adresse du catalogue répond 404, et les pages HTML sortent sans la protection à nonce. Le fichier `.htaccess` fourni protège chaque directive par `<IfModule …>`, car une directive d'un module non chargé ferait tomber **tout le site** en erreur 500. Il n'emploie jamais `Options` (qui exige un droit que beaucoup de mutualisés n'accordent pas) : les listes de dossiers sont refusées par une règle de réécriture.

La sonde de santé

L'adresse `/api/health` répond une ligne de JSON minimale : `{ok, web, server, trustEpoch}` (et `lastUpdate` juste après une mise à jour). Elle est **publique** et ne dit que ce qui l'est déjà (la version sur GitHub).

Elle sert à deux choses : le superviseur de mise à jour (§18.11) l'interroge pour savoir si la version neuve vit, et vous pouvez la confier à un outil de supervision. **Les hôtes PHP n'ont pas cette sonde.**

18.3 Le cache HTTP : ne rien retélécharger pour rien

Un jeu de données pèse des gigaoctets ; la plateforme, une trentaine de scripts. Sans cache, chaque visite repartirait de zéro. Le serveur dit donc au navigateur **combien de temps garder** chaque fichier.

💡 UNE IMAGE POUR COMPRENDRE

Le frigo et l'étiquette. Un yaourt porte une date : jusqu'à cette date, inutile de le redemander au magasin. Un journal du matin, lui, doit être **revérifié** : « est-ce toujours celui d'aujourd'hui ? ». Le cache HTTP, c'est ce jeu d'étiquettes.

Le tableau de décision du cache HTTP

Quel fichier ?	URL avec ?v=…	URL sans ?v=
Page HTML (nonce neuf à chaque fois)	no-store	no-store
Paquet de briques `.bin .rgba .gz`	immuable, 1 an	no-cache + ETag
Script, style `.js .mjs .css`	immuable, 7 jours	no-cache + ETag
JSON, manifeste, catalogue le ?v= ne change rien	no-cache + ETag	no-cache + ETag
Réponses de l'API `/api/…`	no-store	no-store

no-cache = le navigateur garde le fichier mais le redemande : le serveur répond « 304, inchangé » sans renvoyer les octets.

Images, polices : no-cache + ETag sur le serveur Python ; 7 jours sur Apache (.htaccess).

▲ Le tableau de décision : une règle par type de fichier, selon la présence d'un ?v=.

Trois règles à retenir

1. **Une adresse « versionnée » ne change jamais de contenu.** Si elle porte `?v=…`, le navigateur la garde sans redemander : **1 an** pour un paquet de briques, **7 jours** pour un script ou un style.
2. **Tout le reste est « no-cache + ETag ».** Le navigateur garde le fichier, mais le **redemande** avec l'empreinte qu'il possède ; le serveur répond « 304, inchangé » **sans renvoyer les octets**.
3. **Une page HTML n'est jamais gardée** (`no-store`) : son code secret change à chaque chargement.

D'où vient le `?v=` ?

- **Les paquets de briques** : le chargeur (chapitre 10) ajoute `?v=` suivi d'une empreinte de la **liste des paquets** du manifeste. Retraiter un jeu sous le même nom change les paquets, donc **les adresses** : on ne peut jamais relire d'anciens voxels par mégarde.
- **Les scripts et styles** : au moment de la publication d'une version, l'outil de fabrication ajoute `?v=<numéro de version>` à chaque lien local, et regroupe les scripts d'une page en **un seul fichier** (`js/bundle/<page>.js?v=<empreinte>`).
- **Le thème** (`config/theme.css`) : son `?v=` est la date de dernière modification du fichier, pour qu'un changement de couleurs apparaisse **tout de suite** malgré le cache.

❓ POURQUOI CE CHOIX ?

Pourquoi un tel soin pour un script ? Certains hébergeurs limitent le nombre de requêtes par adresse : une trentaine de scripts redemandés à chaque page peuvent déclencher une erreur 429 (« trop de requêtes ») et empêcher le viewer de démarrer. Les adresses versionnées évitent ces redemandes.

Ce que voit un navigateur, d'une visite à l'autre

▲ Première visite, puis visites suivantes : ce qui est redemandé.

Voici des réponses **réellement mesurées** sur le serveur Python de démonstration (jeu synthétique) :

REQUÊTE	CACHE-CONTROL	REMARQUE
`viewer.html`	`no-store`	page + nonce neuf
`js/core/utils.js`	`no-cache`	ETag `…-gz` (version compressée)
`js/core/utils.js?v=1.59.3`	`public, max-age=604800, immutable`	7 jours
`…/bricks/manifest.json`	`no-cache`	gzip : 1 151 octets
`…/p00000.bin`	`no-cache`	1 383 748 octets
`…/p00000.bin?v=abc`	`public, max-age=31536000, immutable`	1 an
`/api/health`	`no-store, no-cache, must-revalidate`	66 octets

⚠️ **ATTENTION**

Sans `?v=`, un paquet est revalidé, pas gardé un an. Une longue durée sur une adresse que rien ne change aurait servi les **anciens voxels** après un retraitement : une erreur scientifique silencieuse. C'est pourquoi la règle « 1 an » exige la preuve (`?v=`) que l'adresse est neuve.

🔧 **POUR LES CURIEUX**

Les deux backends appliquent la même table, dans `dev_server.py` (`_static_cache_policy`) et dans le `.htaccess` racine (variables d'environnement posées par `mod_rewrite` quand la requête contient `v=`). Un test compare les deux (`tests/test_backend2_twins_parity.php`).

Une différence à connaître pour les images et polices : le serveur Python les répond `no-cache` + ETag ; Apache, via le `.htaccess` , leur donne 7 jours. L'ETag Python se compose de la date de modification et de la taille (`"18dc4a43bc302352-4a17-gz"` : le suffixe `-gz` distingue la version compressée).

18.4 Compression et plages d'octets

gzip : des fichiers texte plus légers

Un manifeste de briques peut peser des dizaines de Mo de JSON : compressé, il en pèse sept fois moins. Le serveur compresse donc à la volée les fichiers **texte** : `.json` , `.js` , `.mjs` , `.css` , `.svg` , `.txt` , `.csv` , `.md` .

- Seuil : au moins **1 Kio** (et au plus 256 Mio) ; niveau 6.
- Le résultat est **mémorisé** (budget 96 Mio) et ne se refait que si le fichier change.
- Pas de compression quand le navigateur demande une **plage** d'octets, ni pour les fichiers de `download/` (qui sortent toujours en pièce jointe).
- Les **paquets de briques ne sont jamais compressés** : ce sont déjà des images WebP.

Sur Apache, la compression est confiée au module `mod_deflate` (JSON, HTML, CSS, JS, SVG), toujours pas aux paquets.

Les plages d'octets : n'en demander qu'un morceau

Pour afficher une **coupe** du volume, le viewer n'a pas besoin de tout un paquet de 16 Mio : il en demande seulement quelques tronçons (chapitre 10). Le navigateur écrit alors un en-tête `Range: bytes=…` .

Demander seulement un morceau d'un fichier : les plages d'octets

▲ Une plage, plusieurs plages, une plage absurde : les réponses du serveur.

💡 **UNE IMAGE POUR COMPRENDRE**

Photocopier quelques pages d'un livre. Plutôt que d'emporter tout le volume, on demande « les pages 1 à 3 et 200 à 210 ». Le serveur répond avec exactement ces pages, chacune étiquetée de sa position dans le livre.

Le serveur Python gère les **trois cas** de la norme HTTP (RFC 9110) :

- **Une plage** : réponse `206` avec `Content-Range: bytes 0-99/1383748`. Une plage « les N derniers octets » (`bytes=-N`) est comprise.
- **Plusieurs plages** (`bytes=0-99, 500-599`) : une seule réponse `206` de type `multipart/byteranges`, **un aller-retour** au lieu de deux. Les plages qui se touchent sont fusionnées ; au-delà de **64 plages** distinctes (ou si l'en-tête est mal formé), le serveur renvoie le **fichier entier** (`200`), ce que la norme permet.
- **Une plage hors du fichier** : `416` avec `Content-Range: bytes */<taille>`.

🔢 **MICRO-EXEMPLE**

Vérifié sur le jeu de démonstration. Sur un paquet de 1 383 748 octets : `bytes=0-99` répond `206` de 100 octets ; `bytes=0-99, 500-599` répond `206 multipart/byteranges` de 473 octets au total (deux fois 100 octets, plus les en-têtes de partie) ; `bytes=99999999-` répond `416` .

🔧 POUR LES CURIEUX

`If-Range` est respecté (une plage n'est servie que si l'ETag ou la date correspondent encore). Les requêtes à plage ne comptent **pas** comme un téléchargement dans les statistiques. Côté Apache, le serveur HTTP gère les plages lui-même, avec ses propres limites (`MaxRanges`). Le chargeur de briques a, de son côté, une **solution de repli** par hébergeur quand les plages multiples ne sont pas comprises (chapitre 10).

18.5 Les dossiers fermés et les fichiers-garde

Un site web sert des fichiers ; mais tous ne doivent pas sortir. Les mots de passe, les graines de signature, les imports non validés **ne doivent jamais** être téléchargeables.

Les dossiers du site : lesquels sont servis, lesquels jamais

`api/`	*.json (mot de passe, stats…) et _*.php	fermé
`uploads/`	imports non validés : 4 verrous	fermé
`secrets/ logs/ backups/`	graines de signature, journaux, copies	fermé
`.git .htaccess .xxx`	tout nom commençant par un point	fermé
`*.lumen-old|new|backup`	fichiers parqués par une mise à jour	fermé
`DATA_WEB/`	exécution interdite ; download/ en pièce jointe	sans exécution
`js/`	exécution interdite (les plugins y atterrissent)	sans exécution
`config/`	public ; protégé des mises à jour	servi

▲ Quels dossiers sont servis, lesquels sont toujours refusés.

💡 UNE IMAGE POUR COMPRENDRE

Les portes d'un bâtiment public. Le hall est ouvert à tous ; les bureaux sont fermés à clé ; la salle des archives a **plusieurs serrures différentes**, pour que l'oubli d'une seule ne laisse pas entrer.

Quatre verrous pour `uploads/`

Les octets importés (chapitre 14) ne sont pas encore validés. Quatre barrières **indépendantes** les protègent : la liste des racines interdites du serveur Python, le `.htaccess` racine, le routeur `router.php`, et un `.htaccess` « tout refuser » écrit dans le dossier lui-même. La prévisualisation d'un jeu en cours d'import passe par une adresse d'API qui exige la session administrateur.

Les fichiers-garde « lumen-guard v2 »

Deux dossiers servis (`DATA_WEB/` et `js/`) reçoivent un `.htaccess` qui **interdit l'exécution de scripts** : même si quelqu'un y dépose un `.php` malveillant, il ne s'exécutera pas. `uploads/` et la médiathèque ont leur propre garde.

Chaque garde porte la marque `lumen-guard v2`. Le serveur ne **récrit** un garde que si cette marque manque : les deux serveurs (Python et PHP) ne se disputent donc jamais le même fichier pour un simple espace en trop.

❓ **POURQUOI CE CHOIX ?**

Pourquoi `DATA_WEB/` a-t-il besoin d'un garde ? Parce que ce dossier est à la fois **servi** au public et **ouvert en écriture** à l'opérateur (par FTP ou par l'import). Un fichier piégé y serait la porte d'entrée idéale. Le garde est récrit à l'exécution : une mise à jour ne le livre jamais (`DATA_WEB` lui échappe, §18.13).

🔧 **POUR LES CURIEUX**

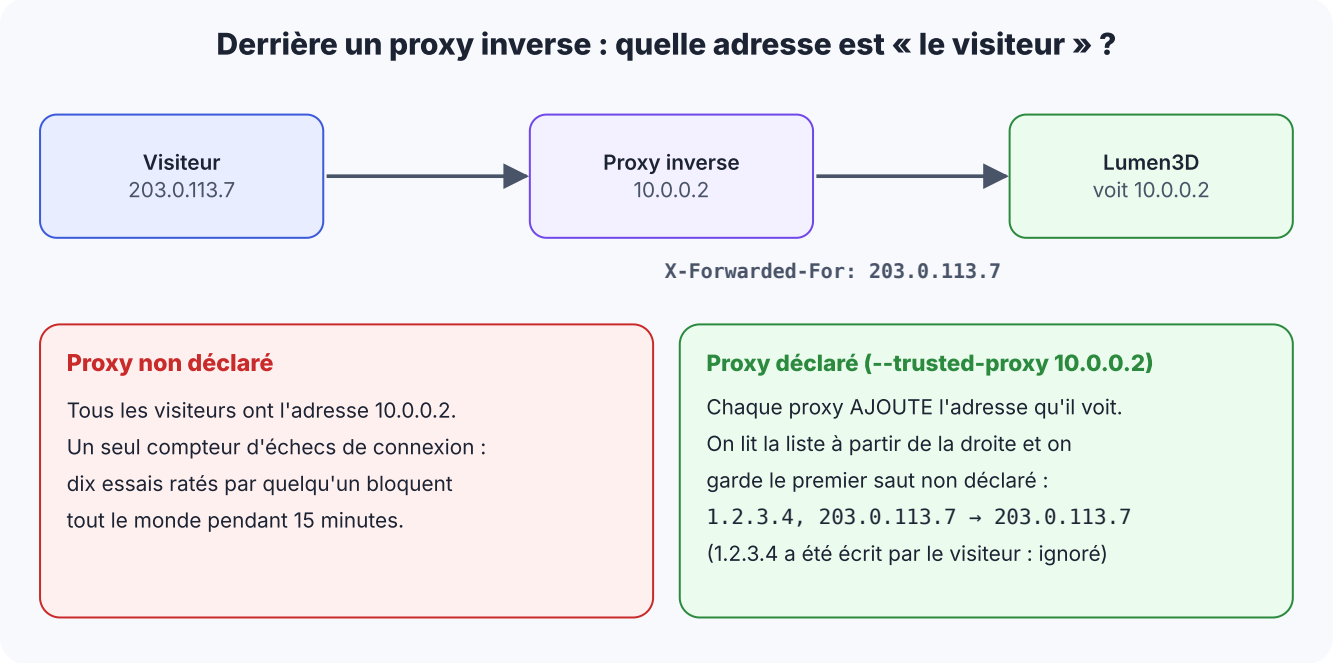
Détails du routeur PHP. `router.php` (pour `php -S`) refuse tous les `api/*.json` et les `api/_*.php` , les dossiers `secrets/ logs/ backups/ uploads/ .git` , tout nom commençant par un point (sauf `.well-known`) et les `*.lumen-old|new|backup` . Il calcule le chemin **sans** `parse_url()` : une adresse commençant par deux barres obliques (`//api/admin_credential.json`) fait lire à `parse_url()` « api » comme un nom de site, et le fichier de mot de passe sortait avec un `200` . Le chemin est donc découpé à la main, décodé, normalisé, puis comparé aux règles. Les fichiers parqués par une mise à jour (`*.lumen-old`) seraient servis comme du texte et publieraient l'ancien code : d'où leur interdiction.

Permissions des fichiers. Les fichiers créés par le serveur héritent des **droits du dossier racine** : `0770/0660` sur un mutualisé où le serveur web et votre compte FTP partagent un groupe, `0755/0644` ailleurs. Si le serveur tourne sous un autre utilisateur que le propriétaire, des droits d'écriture de groupe sont ajoutés pour que le propriétaire garde la main (FTP, suppression). Surcharge : `LUMEN_DIR_MODE` et `LUMEN_FILE_MODE` (valeurs octales). Les secrets de `api/` gardent des droits restrictifs.

18.6 Derrière un proxy, limites de débit et verrou de session

Le proxy inverse et l'adresse du visiteur

Beaucoup d'instituts placent un **proxy inverse** (Nginx, Apache…) devant le serveur : il gère le HTTPS et transmet les requêtes. Problème : Lumen3D ne voit plus que **l'adresse du proxy**.



▲ Sans déclaration, tous les visiteurs partagent l'adresse du proxy et un seul compteur de connexion.

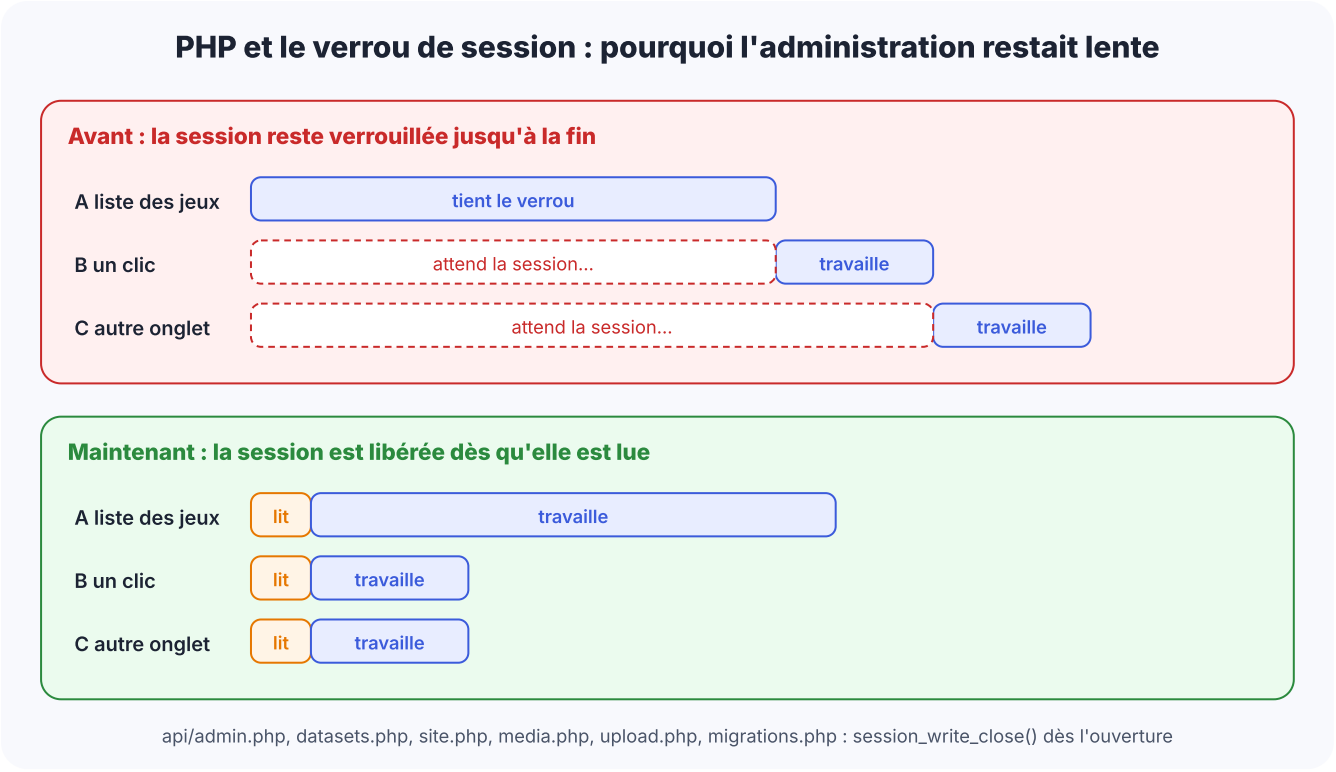
- **Déclarer le proxy** : `--trusted-proxy 10.0.0.2` (option répétable, adresses ou réseaux), la variable `LUMEN_TRUSTED_PROXIES="ip,réseau"`, ou le fichier `api/trusted-proxies.json` (`{"proxies":[…]}`), que **les deux serveurs** lisent.
- **Pourquoi lire la liste par la droite ?** Chaque proxy **ajoute** l'adresse qu'il voit. Tout ce qui est à gauche a pu être écrit par le visiteur : on ne s'y fie pas.
- **Cookie sécurisé** : la session reçoit l'attribut `Secure` si un proxy déclaré annonce du HTTPS (`X-Forwarded-Proto`), ou si `LUMEN_COOKIE_SECURE=1` le force.

Les limites de débit

OÙ	LIMITE	POUR QUOI FAIRE
Connexion admin	10 échecs / 15 min / adresse, plafond global de 200	contrer les essais en masse
Compteurs publics (`telemetry`)	seau à jetons : 60 d'avance puis 1/s par adresse ; 600 puis 20/s pour tous ; GET refusé	empêcher de gonfler les statistiques
Import, mises à jour de données	régulateur du navigateur (chapitre 17)	ne pas faire bannir l'adresse de l'opérateur
GitHub (admin)	réponse mémorisée 5 minutes	60 demandes/heure par adresse sans compte

Le verrou de session PHP

Sur PHP, **une session est un fichier verrouillé** tant qu'une requête s'en sert. Deux requêtes d'un même administrateur se **mettent en file**. Or l'onglet *Mises à jour* lance trois vérifications vers GitHub : l'onglet *Datasets* attendait derrière elles.



▲ Avant : les requêtes font la queue. Maintenant : la session est lue puis libérée.

📌 **À RETENIR**

Règle du code : tout point d'entrée qui ne fait que **lire** la session la libère aussitôt (`session_write_close()`). Seule la connexion la garde, parce qu'elle l'écrit. Un nouveau point d'entrée PHP doit suivre la même règle.

18.7 Exploiter un serveur au quotidien

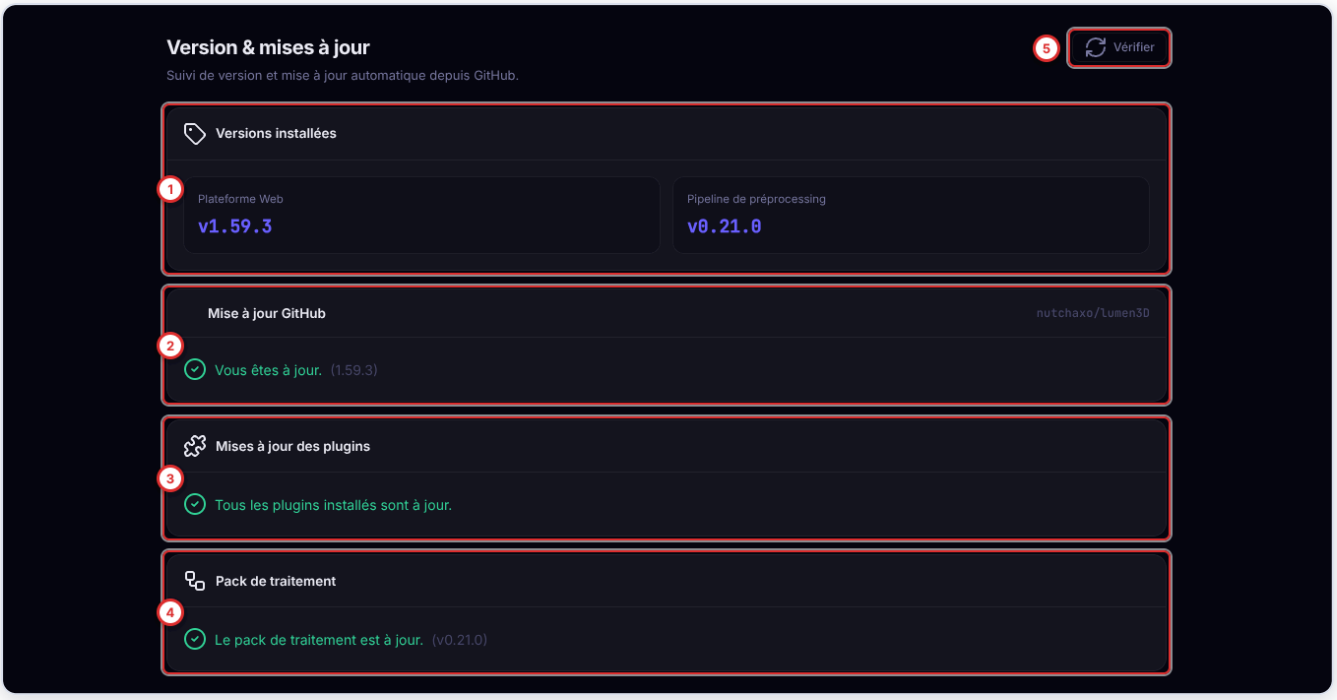
BESOIN	OÙ REGARDER
Le site répond-il ?	`/api/health` (Python)
Journaux du serveur	dossier `logs/` (le serveur relancé par une mise à jour écrit `dev-server-<horodatage>.log`)
Une mise à jour a échoué	`logs/update-pivot-*.log` , `backups/pivot-journal.json` , `backups/last-update.json`
Sauvegardes avant mise à jour	`backups/backup-<version>-<date>.zip` (les 3 plus récentes)
Changer le mot de passe	onglet *Sécurité*, ou `python dev_server.py --set-password`
Valider une installation	`python dev_server.py --check`

👉 **ASTUCE**

Avant de bricoler, lancez `--check`. Cette commande vérifie, sans ouvrir de port, que les fichiers essentiels sont là, que `dev_server.py` se compile, que les fichiers de langue sont lisibles et que la version est cohérente. Code de sortie 0 = tout va bien ; 1 = un problème (un rapport JSON dit lequel).

18.8 Mettre à jour la plateforme : le principe

Une mise à jour se déclenche depuis **Système › Mises à jour**. L'administrateur voit l'état, clique, et la plateforme fait le reste.



▲ L'onglet Mises à jour de l'administration (jeu de démonstration, à jour).

N	CE QUE C'EST
1	**Versions installées** : plateforme web et pipeline de préprocessing
2	**Mise à jour GitHub** : « Vous êtes à jour », ou la mise à jour disponible
3	**Mises à jour des plugins**
4	**Pack de traitement** : le logiciel de préparation des données
5	**Vérifier** : redemande l'état à GitHub

Quatre numéros, quatre rôles

▲ Les numéros de version que l'on croise.

- La **plateforme** n'a **pas** de constante de version : c'est le **nom du fichier de changelog le plus récent** (`changelog_1.65.0.md`). Publier = créer ce fichier.
- Le **serveur Python** a sa propre version (`0.16.0`), qui **dérive volontairement** : elle suit l'outil serveur, pas la plateforme.
- Le **pipeline** (0.22.0) est un autre logiciel, qui s'installe sur le poste de traitement.

Comment la plateforme apprend qu'une version existe

1. Elle interroge GitHub : `releases/latest` du dépôt de la plateforme (écrit en dur dans le code, jamais saisi par l'utilisateur). La réponse est **gardée 5 minutes**.
2. Elle compare avec sa version. Si la plus récente est strictement plus grande, une mise à jour est **disponible**.
3. Elle cherche **exactement** les éléments nécessaires dans la version GitHub.

⏱ EN 30 SECONDES

Éléments d'une version : `lumen3d-web-<version>.zip` (la plateforme), `SHA256SUMS` (la liste d'empreintes), `SHA256SUMS.sig` (la signature de cette liste), `lumen3d-release-notes.json` (toutes les notes de version), et les **packs de traitement** nommés d'après la version **du pipeline**.

🔧 POUR LES CURIEUX

Le nom du pack porte la version du pipeline, ce qui permet à l'onglet *Pipeline* de comparer le pack installé et le pack publié **sans rien télécharger**, donc de proposer une mise à jour du pipeline sans mise à jour de la plateforme.

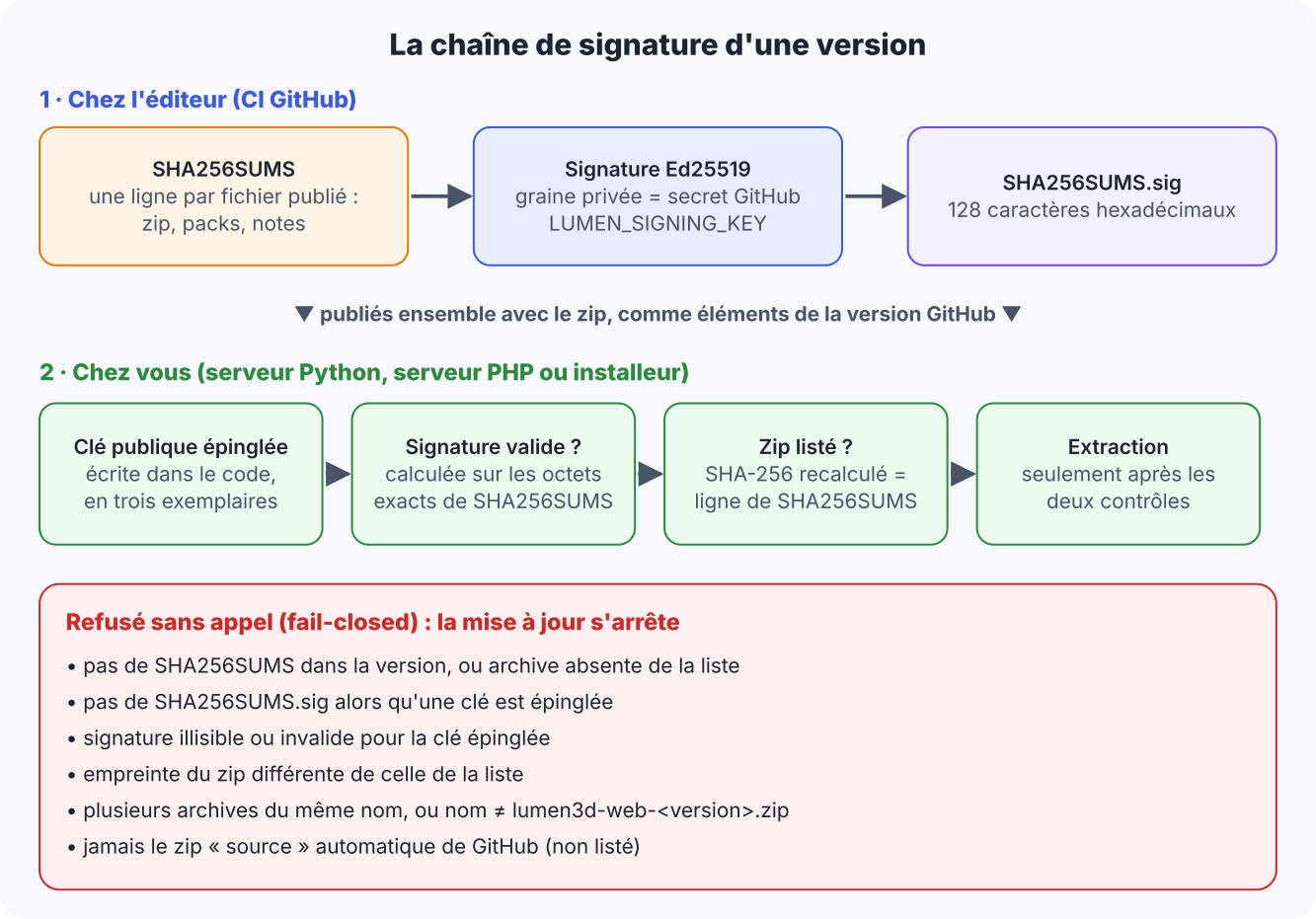
Les notes de version : chaque version embarque les changelogs de **toutes** les versions (niveau plat de `changelog/` , pas l'archive). Un site en retard de plusieurs versions lit ainsi les notes de chacune de celles qu'il va absorber, sans appeler GitHub une fois par version. Rendu : un arbre repliable par version (titre en gras de chaque entrée), aussi disponible en page seule (`admpan.html?changelog=1`).

Limite de GitHub : sans compte, 60 demandes par heure et par adresse. Derrière un NAT d'université, elle s'épuise vite : le panneau affiche alors « trop de demandes » avec le délai, au lieu d'un « 403 » trompeur.

18.9 La signature : prouver qu'un colis est authentique

Un colis qui arrive par Internet pourrait avoir été **remplacé en chemin**. Lumen3D n'installe donc rien qu'il ne puisse prouver : l'archive doit correspondre à une liste d'empreintes, et cette liste doit porter la signature de l'éditeur.

> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Un sceau de cire sur une lettre.** Seul l'éditeur possède le sceau (la clé privée). Tout le monde peut **vérifier** qu'une lettre le porte (la clé publique, connue de votre serveur), mais personne ne peut en fabriquer un faux.



▲ La chaîne : liste d'empreintes, signature Ed25519, clé publique épinglée, puis extraction.

Deux contrôles, dans cet ordre

1. **La signature** (algorithme Ed25519) est vérifiée sur les **octets exacts** de `SHA256SUMS`, avec la clé publique que votre serveur connaît déjà.
2. **L'empreinte SHA-256 de l'archive**, recalculée, doit égaler **sa ligne** dans `SHA256SUMS`. Une archive absente de la liste est refusée.

Le mot « **fail-closed** » (échec fermé) résume la règle : au moindre doute, la porte reste fermée. La mise à jour **s'arrête** et affiche la raison. Sans signature valide, rien n'est installé.

Où vit la clé publique ?

Dans **trois constantes**, qui doivent être identiques : le serveur Python (`_RELEASE_PUBKEY_HEX`), l'installeur (`install.php`) et le code PHP d'administration (`LUMEN_RELEASE_PUBKEY`). La clé **privée** n'existe que dans le secret `LUMEN_SIGNING_KEY` de GitHub.

⚠️ **ATTENTION**

Pourquoi la clé est « dans le code » et non « dans le colis ». Une clé livrée avec le colis serait celle du faussaire. La clé épinglée est donc posée dans le **code source du dépôt**, et voyage dans chaque version. Un serveur n'accepte que la clé qu'il connaît.

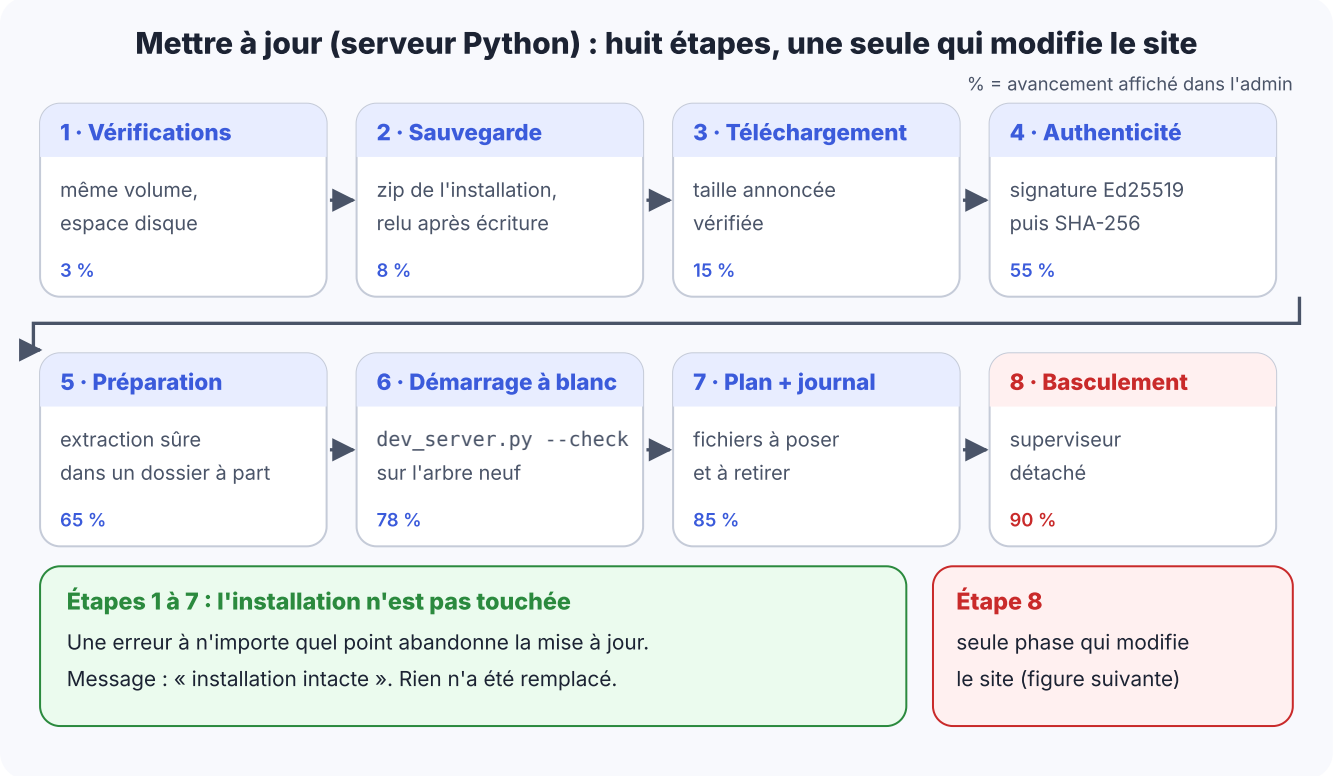
🔧 **POUR LES CURIEUX**

Changer de clé (rotation). La version qui **introduit** la nouvelle clé doit encore être signée avec l'**ancienne** : le serveur la vérifie avec la clé qu'il connaît, puis installe le code qui épingle la nouvelle. Seules les versions suivantes sont signées avec la nouvelle. L'outil `tools/gen_signing_key.py` affiche une paire de clés (il n'écrit rien sur disque). **Le catalogue de plugins a sa propre paire de clés** (chapitre 15) : jamais la même graine pour les deux.

Sur PHP, la signature est vérifiée par la bibliothèque **libsodium** (`sodium_crypto_sign_verify_detached`) ; sur Python, par un vérificateur Ed25519 écrit en Python pur (`ed25519_pure.py` , RFC 8032), qui n'exige rien de plus que la bibliothèque standard. Une clé laissée vide dégraderait la vérification en « SHA-256 seul » avec un avertissement ; la clé est aujourd'hui renseignée.

18.10 La mise à jour sur un serveur Python : huit étapes

Tant que le serveur n'a pas basculé, **l'installation en cours n'est pas touchée**. La mise à jour se déroule dans un fil d'exécution du serveur **vivant**, qui continue de répondre aux visiteurs.



▲ Huit étapes ; seule la dernière modifie le site.

ÉTAPE	CE QUI EST FAIT	CE QUI L'ARRÊTE
1 Vérifications	même volume, espace libre ≥ max(3 × l'archive, 300 Mo), `backups/` inscriptible	un basculement précédent non réconcilié
2 Sauvegarde	zip de tout ce que la version gère (les chemins protégés, §18.13, sont exclus) ; relu après écriture	un fichier illisible : la sauvegarde **abandonne**
3 Téléchargement	taille comparée à celle annoncée	archive tronquée
4 Authenticité	signature puis SHA-256 (§18.9) ; archive intacte (`testzip`)	tout doute
5 Préparation	extraction sûre dans un dossier à part	entrée suspecte (voir ci-dessous)
6 Démarrage à blanc	`dev_server.py --check` **sur l'arbre neuf**	un fichier essentiel manque
7 Plan + journal	liste des fichiers à poser et à retirer, écrite sur disque	—
8 Basculement	un superviseur détaché prend la main (§18.11)	—

⚠️ **ATTENTION**

Une version qui contient `api/plugin-trust.json` est refusée. Ce fichier liste les plugins que l'opérateur a approuvés : une version malveillante pourrait sinon **pré-approuver** un plugin piégé. De même, la version stagée doit porter **exactement** la version annoncée, et chaque fichier doit correspondre à l'empreinte inscrite dans son `version.json` .

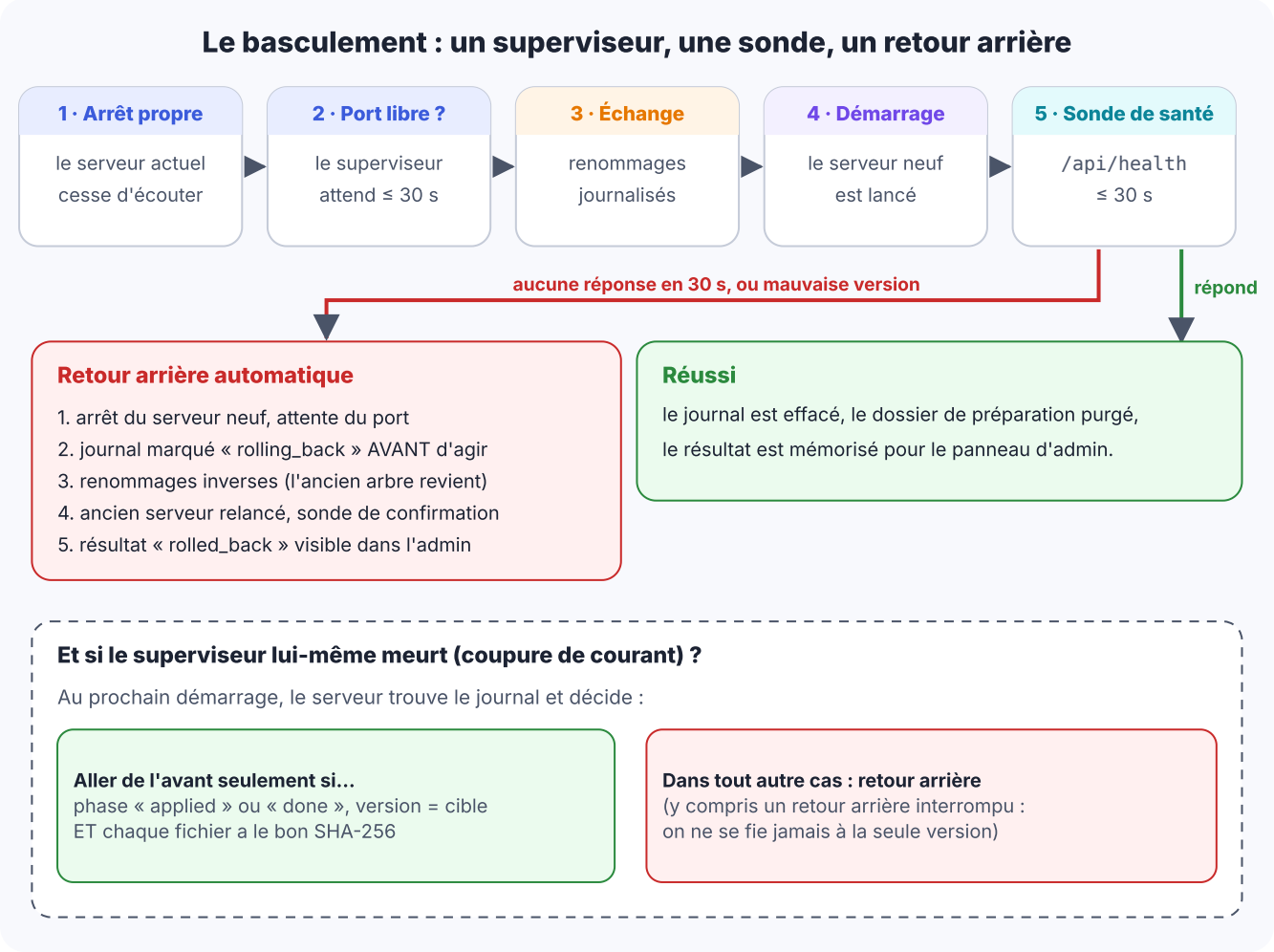
🔧 **POUR LES CURIEUX**

Extraction sûre. Chaque entrée de l'archive est contrôlée avant extraction : chemin absolu, lettre de lecteur, antislash ou `..` sont refusés (une archive reçue d'Internet est une entrée non fiable). Un nettoyage garde les **3** dernières sauvegardes et supprime les dossiers temporaires périmés.

Quatre approches avaient été comparées à la conception (journal en place, checkout git, superviseur permanent, échange par préparation) : la dernière a été retenue parce que **le site n'est touché que par des renommages**, jamais par une copie en place. Une interruption à mi-parcours laissait sinon un arbre mi-ancien mi-neuf, impossible à démarrer.

18.11 Le basculement et le retour arrière

Le serveur qui a préparé la mise à jour ne peut pas se remplacer lui-même : ses fichiers sont **en cours d'utilisation**. Il lance donc un **superviseur** : une **copie** de `dev_server.py` , placée dans un dossier temporaire privé, qui ne tient aucun fichier de l'installation.



▲ Le basculement, étape par étape, avec sa branche de retour arrière.

💡 **UNE IMAGE POUR COMPRENDRE**

Le déménagement bleu-vert. On monte le nouvel appartement à côté de l'ancien, on vérifie que l'eau et l'électricité marchent, et seulement alors on y emménage. Si quelque chose cloche, on retourne dans l'ancien, resté intact.

Les cinq temps

1. Le serveur actuel **cesse d'écouter** (une seconde après avoir laissé partir les réponses en cours).
2. Le superviseur attend que **le port se libère** (30 s au plus).
3. Il **échange** les fichiers par **renommages**, chacun noté dans le journal. Aucun octet n'est copié : tout est quasi instantané.
4. Il **lance le serveur neuf**.
5. Il **interroge** `/api/health` toutes les demi-secondes pendant 30 s. Succès = la sonde répond avec **la version cible**.

Si la sonde échoue

Le superviseur arrête le serveur neuf, marque le journal « rolling_back » **avant** d'agir, applique les renommages **à l'envers**, relance l'ancien serveur et le sonde à son tour. Le panneau affiche alors : « La nouvelle version n'a pas démarré — restauration automatique effectuée. L'ancienne version fonctionne. »

🔢 MICRO-EXEMPLE

Un renommage à deux temps. Pour remplacer `viewer.html` : l'ancien passe dans un dossier miroir (`old/`), puis le neuf prend sa place. À chaque pas, le superviseur **regarde le disque** (et non sa mémoire) : s'il est interrompu et relancé, il reprend **exactement** là où il s'était arrêté. Le retour arrière est sûr à **n'importe quel** état intermédiaire.

Et si le superviseur meurt aussi ?

Au prochain démarrage, le serveur trouve le **journal**. Il n'avance que si **tout** est prouvé : phase « applied » ou « done », version égale à la cible **et** chaque fichier avec la bonne empreinte. Dans **tout** autre cas, il revient en arrière. La version seule ne suffit pas à conclure : ce n'est qu'un fichier parmi des milliers.

🔧 POUR LES CURIEUX

Détails Windows qui ont façonné le design : un processus serveur verrouille son dossier de travail (le superviseur tourne depuis le dossier temporaire) ; les antivirus posent des verrous passagers sur les fichiers frais (le renommage est retenté jusqu'à 10 fois) ; le serveur est relancé en processus **détaché**. Une adresse d'écoute `0.0.0.0` n'est pas joignable : la sonde utilise `127.0.0.1` . Sinon chaque mise à jour sur `--host 0.0.0.0` aurait été annulée à tort.

Fichiers de suivi : `backups/pivot-journal.json` (phases `planned` → `applying` → `applied` → `done` ; ou `rolling_back`), `logs/update-pivot-<horodatage>.log` , et `backups/last-update.json` , le « dernier résultat » que le panneau lit après le redémarrage (`done` ou `rolled_back`). Les tests `tests/test_update_pivot.py` et `tests/test_update_reconcile.py` couvrent l'application complète, la restauration, le rejeu après un crash partiel et le double retour arrière.

18.12 La mise à jour sur un hébergement PHP

Un hôte PHP **se met aussi à jour**, mais autrement : PHP n'a pas de serveur qui tourne en continu, donc rien à redémarrer. Tout se passe dans **une seule requête**, du clic à la réponse.

Deux façons de se mettre à jour, selon le serveur

	Python — dev_server.py	PHP — api/_admin_lib.php
Déroulement	tâche de fond + superviseur	une seule requête, synchrone
Sauvegarde zip	oui (3 conservées)	non
Démarrage à blanc	--check	non
Échange des fichiers	renommages journalisés	ancien fichier parqué, puis nouveau posé
Redémarrage + sonde	oui, /api/health	sans objet : PHP repart à chaque requête
Retour arrière	automatique	aucun : on relance (idempotent)
Fichiers supprimés	différence des version.json	liste fixe (api/config.php)
Signature + SHA-256	identiques, fail-closed	identiques, fail-closed

▲ Les deux façons de se mettre à jour, côte à côte.

⚠️ **ATTENTION**

Pas de retour arrière automatique sur PHP. Une mise à jour PHP qui échoue à mi-parcours ne se « défait » pas toute seule. Elle est conçue pour qu'**on puisse simplement la relancer** : la copie est idempotente, et les notes de version (`changelog/`) sont copiées **en dernier**, de sorte qu'un échec laisse l'**ancienne** version déclarée et qu'une nouvelle tentative soit autorisée.

Le déroulement

1. **Verrou** : une seule mise à jour à la fois (un second clic reçoit « mise à jour en cours »).
2. **Version cible** : même recherche que sur Python ; seule l'archive nommée `lumen3d-web-<version>.zip` est acceptée.
3. **Authenticité** : même double contrôle (signature libsodium, SHA-256).
4. **Extraction** dans un dossier de travail **sous la racine du site** (même disque : un renommage y est instantané ; hors du site, il échouait sur certains mutualisés).
5. **Contrôles de l'archive** : au plus 5 000 entrées, 128 Mio décompressés, aucun chemin suspect (`..` , `.` , vide, antislash, lecteur).
6. **Pose des fichiers** : chaque ancien fichier est **parqué** (`.lumen-old`) puis remplacé ; les chemins protégés sont sautés (§18.13).
7. **Ménage** : anciens fichiers retirés, cache de code PHP (opcache) invalidé pour que la **nouvelle** version réponde dès la requête suivante.

Les cas particuliers

- **Fichier occupé** (Windows) : le fichier neuf est parqué en `.lumen-new` et terminé à une requête ultérieure.

- `.htaccess` **racine** : le bloc `LUMEN3D` est **réécrit**, mais **vos lignes** (`AddHandler` , `RewriteBase` , `php_value` ...) sont gardées, et une copie `.lumen-backup` est conservée. Sans cela, une mise à jour pouvait faire servir le PHP **comme du texte**.
- **Fichiers retirés** : sur PHP, seule une courte liste est supprimée (`api/config.php` , un ancien fichier qui contenait une empreinte de mot de passe). Il n'y a pas de comparaison des `version.json` .

> 🔧 **POUR LES CURIEUX**
>
> Le temps d'exécution est étendu à 600 s. Les fichiers propres à Python (`dev_server.py` , `fast_server.py` , `ed25519_pure.py` , `start.bat`) sont **sautés** sur un hôte PHP. Le dossier `js/modules/` n'est jamais écrasé (un plugin installé n'est pas dans la version).

18.13 Ce qu'une mise à jour ne touche jamais

Une version **remplace le code**, jamais **vos données ni votre travail**.

Ce qu'une mise à jour touche, et ce qu'elle ne touche jamais

Jamais touché

`DATA_WEB/`
vos jeux de données

`uploads/`
imports en cours

`logs/ backups/ secrets/`
journaux, copies, graines

`api/admin_credential.json`
votre mot de passe (empreinte)

`api/stats.json`
les compteurs d'usage

`api/plugin-trust.json …`
approbations, plugins désactivés

`api/marketplace-state.json`
numéro de série anti-retour

`config/instance.json theme legal pages/`
votre personnalisation

`api/page-drafts/`
brouillons non publiés

`install.php`
l'installeur reste verrouillé

Remplacé par la nouvelle version

Pages HTML
index, viewer, compare, 2d, admpan ...

`js/ css/ lang/`
code, styles, traductions

`api/*.php`
le code PHP du serveur

`dev_server.py …`
serveur Python et moteurs

`changelog/`
notes de version (la version = le plus récent)

`.htaccess`
bloc LUMEN3D réécrit, vos lignes gardées

`config/defaults/`
modèles neutres

Le code d'un plugin installé (js/modules/) n'est pas dans la version : il n'est donc jamais écrasé.

▲ À gauche ce qui est intouchable, à droite ce que la mise à jour remplace.

- La liste des chemins protégés est dans `_UPDATE_PROTECT` (Python) et `admin_update_protected()` (PHP). Les deux sont des **jumeaux**.
- Dans `config/` , seuls les fichiers que l'opérateur modifie sont protégés ; `config/defaults/` fait partie de la version et **reste** mis à jour.
- Les **suppressions** sont calculées sur Python par différence entre les deux `version.json` (l'ancien et le neuf) : un fichier livré avant mais plus maintenant est supprimé ; **un fichier inconnu des deux** (que vous avez ajouté) n'est jamais touché.

Les plugins pendant une mise à jour

Avant de basculer, l'onglet *Mises à jour* fait un **pré-vol** : il demande quels plugins installés seraient **incompatibles** avec la version cible (champ `platformCompat` de chaque plugin, chapitre 15).

- Un plugin incompatible est **mis de côté** (« quarantaine ») après la bascule, pas supprimé : son dossier reste en place.
- Dès qu'une mise à jour du plugin, ou de la plateforme, le rend compatible, il est **réactivé automatiquement** à la découverte suivante.
- Un seul cas **bloque** la mise à jour : si **plus aucun mode de rendu** ne resterait disponible (le viewer ne pourrait plus rien afficher).

> 📌 **À RETENIR**
>
> **Un plugin qui plante ne fait jamais tomber le viewer.** Le registre isole chaque plugin (essai/capture par plugin, plus une barrière globale) : le canevas 3D démarre toujours, même si tout le sous-système de plugins échoue.

18.14 Première installation : `install.php`

Pour un hébergement vide, **un seul fichier** suffit : on dépose `install.php` dans un dossier web vide et on l'ouvre dans un navigateur.

▲ Les six étapes de l'installeur.

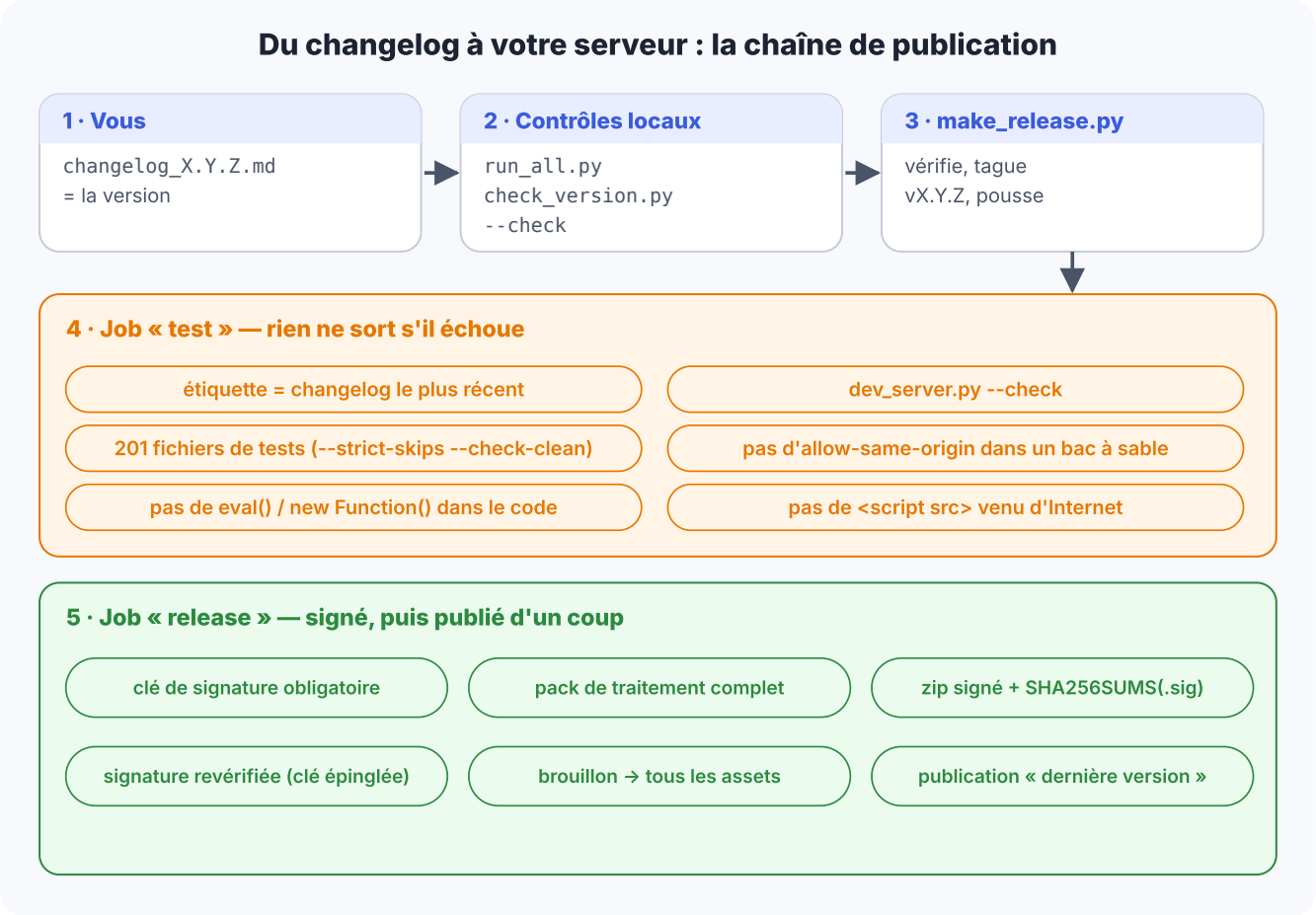
- Il télécharge **la dernière version** du dépôt de la plateforme, **vérifie** la signature et l'empreinte (§18.9), extrait, puis demande de créer le **compte administrateur** (8 caractères minimum).
- Le téléchargement se fait **par tranches** (8 Mio ou 10 s) avec reprise, pour survivre aux limites de temps des hébergements mutualisés. L'extraction se fait par lots de 200 entrées.
- À la fin, il écrit `.install-lock` et **propose de se supprimer**. S'il trouve un compte administrateur ou ce verrou, il **refuse de tourner** : on ne peut pas le rejouer pour écraser un site vivant.

🔧 **POUR LES CURIEUX**

Garde-fous supplémentaires : dépôt GitHub **codé en dur** (aucune adresse n'est saisie) ; chaque entrée de l'archive contrôlée (pas de traversée, de lien symbolique, de chemin absolu) ; **bombes zip** bornées (20 000 entrées, 500 Mo décompressés, archive ≤ 512 Mo) ; `install.php` ne s'écrase jamais lui-même ; jeton anti-falsification sur chaque requête ; état dans `.install-state.json` (sans secret, supprimé à la fin). Le fichier du compte est créé en **création exclusive** et au même format que celui des deux serveurs.

18.15 Publier une version (pour l'éditeur)

Cette section décrit la chaîne côté **éditeur** : la personne qui fait évoluer la plateforme. Elle montre pourquoi la version qui arrive chez vous a de bonnes chances d'être correcte.



▲ De l'idée à votre serveur : tout le chemin d'une version.

1. **Écrire le changelog.** Créer `changelog/changelog_X.Y.Z.md` : **c'est le numéro de la version**. Sections `[ADDED]`, `[OPTIMIZED]`, `[FIXED]`, `[CHANGED]` ; une entrée par puce, `- **Titre court**: ce qui change et pourquoi`. Le panneau d'administration n'affiche que le titre en gras en mode replié.

2. **Contrôler en local** : `python tests/run_all.py -j 4`, `python tools/check_version.py --tag vX.Y.Z`, `python dev_server.py --check`.

3. **Taguer** : `python tools/make_release.py` (ou `--dry-run` pour simuler). Il vérifie le changelog, refuse un arbre sale ou une autre branche que `main`, pose l'étiquette `vX.Y.Z` et **pousse l'étiquette**.

4 **La CI prend le relais** : elle ne publie que si **tout** passe (§18.16).

Ce que fait le job « release »

- **Refus sans clé** : sans le secret `LUMEN_SIGNING_KEY`, il s'arrête : jamais de version non signée.
- **Packs de traitement** : le pack complet est fabriqué **avant** le zip, pour que son empreinte figure dans la liste signée. Le pack léger est copié depuis l'arbre.
- **Zip « curé »** : fabriqué depuis une **liste blanche** de fichiers (tout ce qui n'y est pas est exclu, donc la documentation, les outils ou les données ne fuient jamais par accident). Horodatage fixe : deux fabrications donnent les mêmes octets.
- **Signature** de `SHA256SUMS`, puis **re-vérification** contre la clé épinglée du code : on refuse une version que les serveurs refuseraient.
- **Publication en trois temps** : la version est créée en **brouillon**, reçoit **tous** ses fichiers, **puis** est publiée. Un site qui interroge « la dernière version » ne voit donc jamais une version sans son zip.

> ❓ **POURQUOI CE CHOIX ?**
>
> **Pourquoi la CI crée la version, et pas `make_release.py` ?** Une version créée à la main apparaîtrait d'abord **sans fichiers** : le programme de mise à jour se rabattrait alors sur le zip « source » automatique de GitHub, **invérifiable**. On laisse donc la création à la CI.

Deux autres formes de publication

	PLATEFORME	PLUGIN DE LA PLATEFORME	PLUGIN TIERS
Version	nouveau `changelog_X.Y.Z.md`	`plugin.json` → `version`	idem (indicatif)
Publier	`make_release.py` → CI	`publish_plugin.py <dossier> --push`	déposer le dossier
Appliquer	admin → Mises à jour	admin → Catalogue	approuver (empreinte)
Signature	Ed25519 de la version	Ed25519 du catalogue (autre clé)	hors périmètre

Les plugins de la plateforme ne **partent pas** avec la version : ils sont publiés dans le **catalogue signé** et installés à la demande (chapitre 15). Leur publication est effective au passage de `dev` à `main`, car le catalogue est lu sur `main`.

18.16 La suite de tests : ce qui garde la porte

Une version n'est signée **que si** la suite de tests est entièrement verte. Cette suite se lance par **une seule commande**.

▲ 201 fichiers de tests, trois langages.

201	124	55	22
fichiers de tests	Node (viewer, workers, protocoles)	Python (serveur, pipeline, import)	PHP (serveur jumeau)

- `python tests/run_all.py` découvre tous les fichiers `tests/test_*.py|js|php` et `tests/js/test_*.mjs`, et lance **chacun dans son propre processus**.
- `--strict-skips` : un test qui ne peut pas s'exécuter (une extension PHP manquante) doit le dire, et ce mode le compte comme un **échec**. Un test ne peut donc pas s'arrêter de tourner sans que personne ne le voie.
- `--check-clean` : un test qui **salit le dépôt** (fichier créé ou modifié) échoue. Tout doit se passer dans des dossiers temporaires.

❓ POURQUOI CE CHOIX ?

Pourquoi tester les « jumeaux » avec les mêmes vecteurs ? Plusieurs règles existent en **deux ou trois langages** (compatibilité de version, empreinte de confiance, import, notes de version). Un même fichier de cas est joué par Python, PHP et JavaScript : s'ils divergent, un test échoue. Par exemple, `tests/compat-vector.json` contient 42 cas.

L'intégration continue, à chaque modification

Deux flux GitHub : `ci.yml` (à chaque envoi de code sur `main` ou `dev`, et pour toute demande de fusion) et `release.yml` (à chaque étiquette `v*`). Ils enchaînent :

CONTRÔLE	CE QU'IL GARANTIT
`check_version.py`	l'étiquette est égale au changelog le plus récent ; pas de doublon ni de fichier mal nommé
`dev_server.py --check`	une installation neuve démarre à blanc
suite complète	aucun comportement ne s'est dégradé
pas d'`allow-same-origin`	un bac à sable de plugin ne perd jamais son isolement
pas de `eval()` / `new Function()`	le code reste compatible avec la politique de sécurité stricte
pas de `<script src="http…">`	aucune bibliothèque ne vient d'Internet

18.17 Travailler sans Internet

Lumen3D a été conçu pour un **réseau fermé**. Tout ce dont le **visiteur** a besoin est dans le dossier du site.

Ce qui marche sans Internet, et ce qui en demande

Sans Internet (réseau fermé d'un institut)

- ✔ toutes les pages et leurs scripts
- ✔ Three.js, Lucide, Plotly : fichiers du site
- ✔ vos jeux de données et le catalogue
- ✔ le panneau d'administration
- ✔ l'import et les mises à jour de données
- ✔ l'affichage 3D (carte graphique locale)

Demande Internet — l'administrateur seulement

`api.github.com`
chercher une nouvelle version

`github.com`
télécharger la version et les packs

`raw.githubusercontent.com`
lire le catalogue de plugins signé

Un visiteur ne contacte jamais GitHub.

La seule exception côté visiteur

`fonts.googleapis.com fonts.gstatic.com`

Les polices ne gênent pas l'affichage : sans réseau, la page s'affiche avec les polices du système.

▲ Ce qui marche hors ligne, et ce qui demande Internet.

BIBLIOTHÈQUE	VERSION	FICHIER
Three.js	0.147.0	`js/vendor/three.min.js` (+ chargeurs GLTF et OrbitControls)
Lucide	0.344.0	`js/vendor/lucide.min.js`

BIBLIOTHÈQUE	VERSION	FICHIER
Plotly	2.27.0	`js/vendor/plotly.min.js` (chargé seulement à l'ouverture d'un graphique)

- Chaque bibliothèque est vérifiée par une **empreinte d'intégrité (SRI)** : si un octet change, le navigateur la refuse. Le dossier `js/vendor/` est protégé des conversions de fin de ligne pour que l'empreinte reste valable.
- La politique de sécurité n'autorise **aucun script étranger** : ajouter une balise `<script src="https://…">` casserait la page, et l'intégration continue le refuse.
- **Seule exception côté visiteur** : les polices (Google Fonts). Elles sont déclarées pour se charger **en arrière-plan** : sur un réseau qui les filtre, la page s'affiche aussitôt avec les polices du système.
- **Côté administrateur** : GitHub n'est contacté que pour chercher une version, la télécharger et lire le catalogue de plugins. **Un visiteur ne contacte jamais GitHub.**

🔢 **MICRO-EXEMPLE**

Un réseau d'institut sans sortie. Le viewer, l'Explorateur, l'import et les mises à jour de **données** marchent. Seul l'onglet *Mises à jour* de l'administration affichera « injoignable » ; pour mettre à jour, déposez la version à la main (ou faites-le depuis un poste connecté).

18.18 Que faire si… ?

SYMPTÔME	CAUSE PROBABLE	QUE FAIRE
Mise à jour refusée : « Cette version ne peut pas être vérifiée »	archive absente de `SHA256SUMS`, ou signature invalide	ne rien forcer ; prévenir l'éditeur
Mise à jour « restauration automatique effectuée »	la version neuve n'a pas répondu à `/api/health` en 30 s	lire `logs/update-pivot-*.log`
« Le serveur ne répond plus » après une mise à jour	basculement long ou crash	vérifier `logs/`, puis recharger ; le serveur réconcilie seul au démarrage
Après une mise à jour PHP, une page est cassée	copie interrompue	**relancer** la mise à jour (idempotente)
Un plugin a disparu après une mise à jour	son `platformCompat` ne couvre plus la version	onglet *Plugins* : mise à jour du plugin
Erreur 429 sur le viewer	l'hébergeur limite les requêtes par adresse	vérifier que les adresses `?v=` sont servies (cache)
Tout le monde est bloqué à la connexion	proxy non déclaré (une seule adresse vue)	`--trusted-proxy` (§18.6)
L'administration PHP est lente	verrou de session	vérifier que le code est à jour (§18.6)

SYMPTÔME	CAUSE PROBABLE	QUE FAIRE
Sans HTTPS, l'import refuse de démarrer	le navigateur exige un contexte sûr pour le calcul d'empreintes	HTTPS, ou `localhost`
Un nouveau jeu n'apparaît pas	fiche masquée, ou ni configurée ni vignette	éditer la fiche (chapitre 14)

📌 **À RETENIR**

- Trois serveurs : **Python** (tout), **PHP** (tout, sans retour arrière auto), **fichiers** (lecture seule).
- Un `?v=` dit au navigateur « cette adresse ne change jamais » : paquets 1 an, scripts 7 jours ; tout le reste se **revalide** (304).
- Une version se **prouve** (signature Ed25519 épinglée + SHA-256), se **teste** (201 fichiers) et se **bascule** avec filet de sécurité.
- Vos **données, mots de passe et personnalisations** ne sont jamais touchés par une mise à jour.

📖 **POUR ALLER PLUS LOIN**

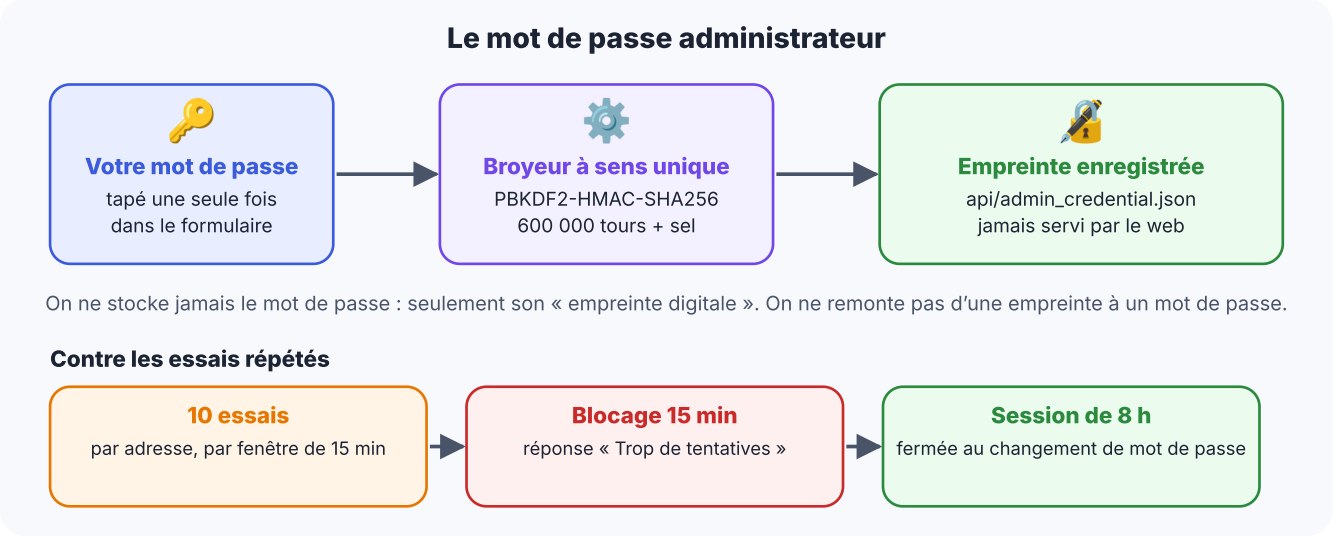
- Chapitre 10 : le chargeur de briques, les plages d'octets en pratique et le `?v=` des paquets.
- Chapitre 14 : l'onglet *Import*, l'édition d'un jeu et les écrans de l'administration.
- Chapitre 15 : plugins, confiance, bac à sable et catalogue signé.
- Chapitre 19 : mot de passe, politique de sécurité (nonce) et fiabilité.
- Détails de conception : `DOCS/update-system/` (technique, publication, cheminement).

19. Sécurité et fiabilité

- Lumen3D protège le panneau d'administration, le code exécuté dans les pages et les fichiers déposés. Chaque protection a une **raison simple**.
- Aucune donnée d'étude n'est envoyée à un tiers et la plateforme fonctionne **hors ligne**.
- La fiabilité repose sur des publications **tout ou rien** et sur une suite d'environ **200 fichiers de tests** qui bloque toute version défectueuse.
- Un tableau unique, « **Que se passe-t-il si… ?** » (sections 19.8 à 19.12), liste chaque panne prévue et le repli qui l'attend.

19.1 Le mot de passe administrateur

Il n'existe **aucun mot de passe par défaut** : le premier visiteur du panneau crée le compte (8 caractères minimum), et cette création ne peut jamais écraser un compte existant.



▲ Du mot de passe à l'empreinte, et les protections contre les essais répétés.

💡 UNE IMAGE POUR COMPRENDRE

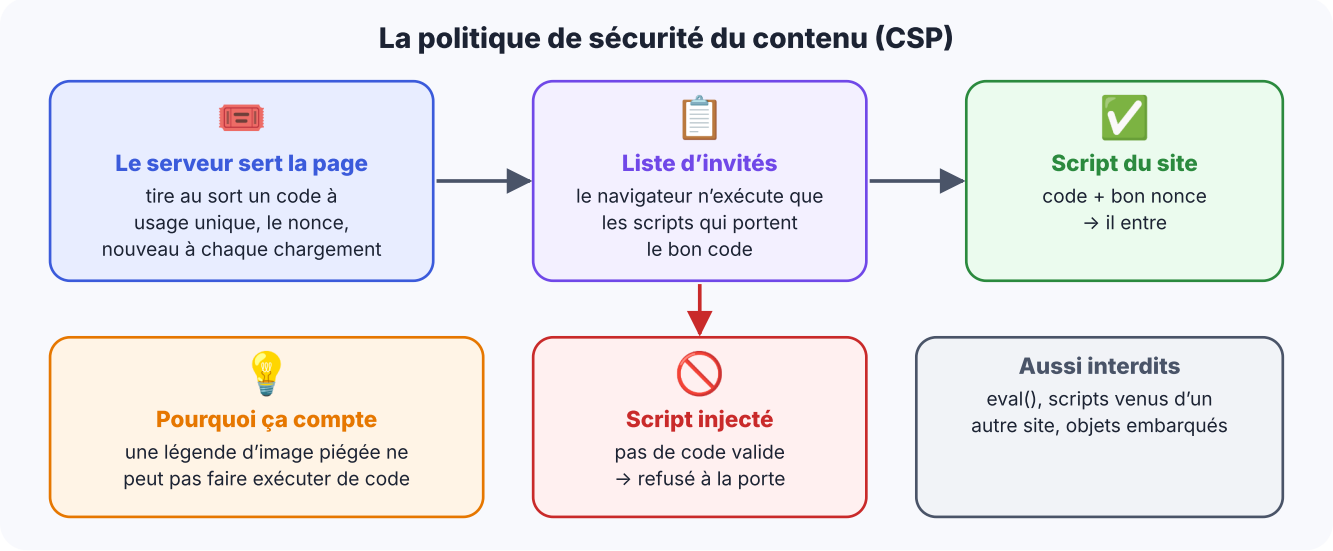
Une empreinte digitale. Le serveur garde l'empreinte de votre mot de passe, pas le mot de passe. Avec une empreinte on peut reconnaître la bonne personne, mais on ne peut pas refabriquer le doigt.

- L'empreinte est calculée par **PBKDF2-HMAC-SHA256**, avec un sel propre au compte et 600 000 tours : volontairement lent pour décourager les essais en masse.
- Après **10 échecs** en 15 minutes depuis une même adresse, l'accès est bloqué 15 minutes. Les échecs sont comptés **avant** la vérification, et un mauvais identifiant coûte le même temps qu'un mauvais mot de passe.
- Une session dure **8 heures**. Changer le mot de passe ferme toutes les autres sessions.

- Toute modification passe aussi par un jeton anti-falsification (CSRF).

19.2 La politique de sécurité du contenu (CSP)

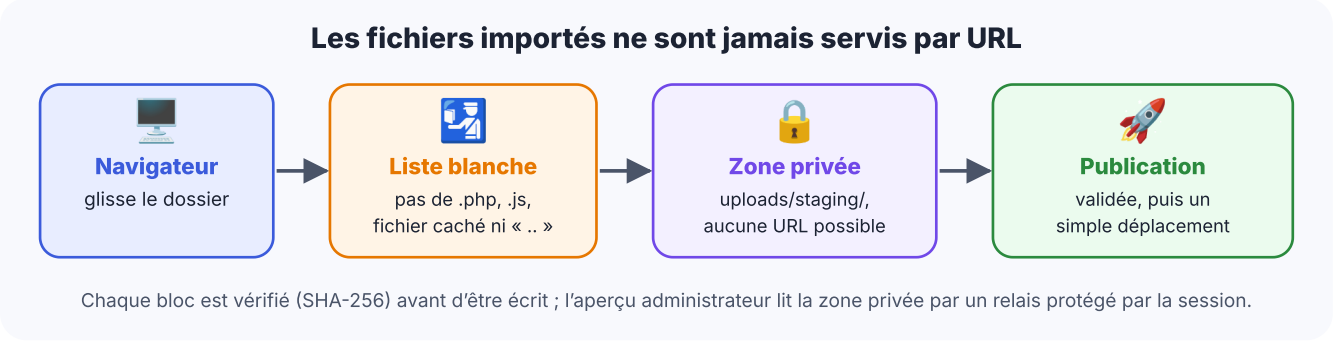
> 💡 **UNE IMAGE POUR COMPRENDRE**
>
> **Une liste d'invités à l'entrée.** À chaque chargement de page, le serveur tire un code secret (le « nonce ») et le remet aux scripts du site. Le navigateur ne laisse entrer que ceux qui le présentent.



▲ Le nonce, une liste d'invités renouvelée à chaque chargement.

La politique est **appliquée** (et non simplement observée). Les bibliothèques (Three.js, Lucide, Plotly) sont hébergées par le site lui-même : aucun script ne vient d'ailleurs.

19.3 Les fichiers importés ne sont jamais servis



▲ Le trajet d'un fichier importé jusqu'à sa publication.

- Les fichiers arrivent dans `uploads/`, un dossier **inaccessible par URL** (quatre verrous indépendants).
- Une **liste blanche** refuse avant écriture tout ce que le pipeline ne produit pas : pas de `.php`, de `.js`, de fichier caché, de chemin remontant (`..`).
- Seule une **publication explicite**, après validation, déplace le dataset vers `DATA_WEB/`. Ce dossier interdit l'exécution de scripts.

19.4 Les plugins : confiance et cage

▲ Niveaux de confiance, approbation liée à une empreinte, bac à sable.

❓ POURQUOI CE CHOIX ?

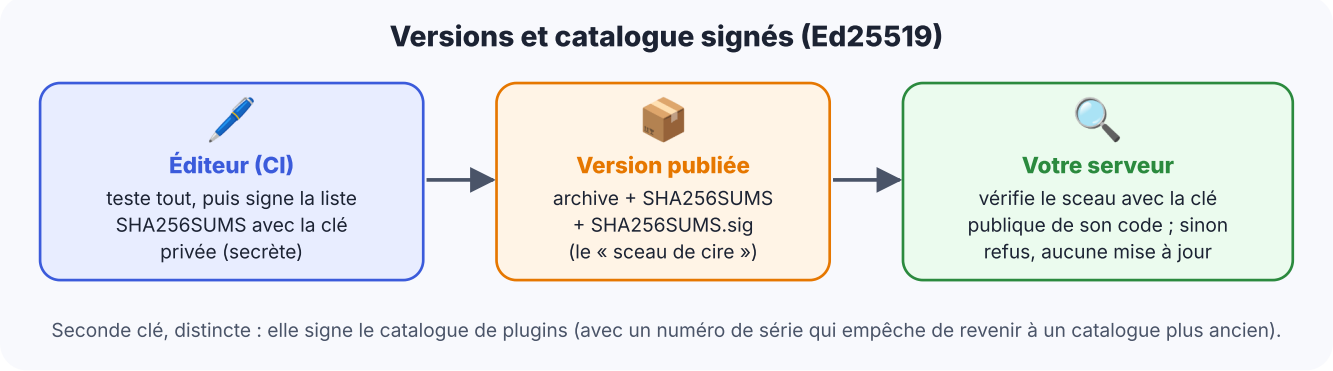
Un plugin est du code. Le laisser entrer sans contrôle serait comme donner un double des clés à un inconnu. Par défaut, un plugin non reconnu **n'est pas chargé**.

- Votre approbation est liée au **contenu exact** du plugin (empreinte) : modifié, il perd sa confiance.
- Un plugin déclare aussi avec quelles versions de la plateforme il est compatible : en cas de doute, il est écarté.

19.5 Versions et catalogue signés

💡 UNE IMAGE POUR COMPRENDRE

Un sceau de cire. Seul l'éditeur possède le sceau (la clé privée). Tout le monde peut vérifier qu'une lettre le porte (clé publique), mais personne ne peut en fabriquer un faux.



▲ La chaîne de confiance d'une mise à jour : signature, vérification, refus.

- La signature est une **Ed25519**. La clé publique est **intégrée au code** du serveur : une clé fournie avec le téléchargement ne serait pas fiable.

- Une version sans signature valable est **refusée** : le programme de mise à jour n'applique que l'archive nommée d'après la version et listée dans la liste signée.
- Le catalogue de plugins a **sa propre clé** et un numéro de série croissant, pour qu'on ne puisse pas réinstaller une ancienne version corrigée depuis.

19.6 Vos données restent chez vous

- **Aucune donnée d'étude** n'est envoyée à un tiers.
- **Hors ligne** : tout le code est hébergé sur le serveur. Seules les polices Google Fonts sont chargées à distance. GitHub n'est contacté que par l'administrateur, pour vérifier les mises à jour.
- Les statistiques sont de simples compteurs (visites, vues, téléchargements). Ce qui est **conservé** : un total global, un total par jour et un total par jeu de données (avec la date de dernière vue), dans `api/stats.json`. Rien n'y est lié à une personne.
- L'adresse du visiteur ne sert qu'**en mémoire**, pour limiter le débit : elle est réduite à l'un de 4 096 emplacements d'une table de « jetons » (60 comptages en rafale par adresse, puis 1 par seconde ; 600 en rafale pour tout le site, puis 20 par seconde). Elle n'est écrite nulle part.

19.7 Fiabilité

RISQUE	PROTECTION
Une mise à jour casse le site (serveur Python)	bascule vérifiée par une sonde de santé, **retour automatique**, sauvegarde préalable
Une mise à jour échoue (hébergement PHP)	vérifications SHA-256 et signature **avant** de toucher aux fichiers ; pas de retour automatique, mais la copie est **rejouable** sans dégât (chapitre 18)
Une publication s'interrompt	**tout ou rien** : le pipeline construit tout à l'écart et publie par un renommage ; une interruption est rattrapée au lancement suivant
Une coupure réseau pendant un import	reprise au bloc près
Une brique corrompue	signalée et ignorée, jamais envoyée à la carte graphique
Une régression dans le code	environ **200 fichiers de tests** doivent passer avant chaque version

📌 **À RETENIR**

Chaque version est **testée en entier** avant d'être signée et publiée. Une version qui échoue aux tests ne peut tout simplement pas sortir.

19.8 Que se passe-t-il si… ?

- Une plateforme fiable n'est pas une plateforme où rien ne tombe en panne : c'est une plateforme où **chaque panne a un plan**.
- Cette partie les passe toutes en revue : ce que fait Lumen3D, ce que vous voyez, ce que vous avez à faire.
- Quatre familles : **l'image** (briques, carte graphique), **les données** (import, migration, métadonnées), **le serveur** (mise à jour, accès) et **le navigateur**.

Trois principes guident tous les replis. Ils expliquent pourquoi la réponse est presque toujours « le reste continue ».

Dégrader, pas planter
Moins de détail vaut mieux qu'un onglet figé. On passe au niveau plus grossier, on écarte la brique douteuse, on affiche un message.

Tout ou rien
Une publication, une conversion ou une mise à jour s'applique **en entier** ou pas du tout. Jamais de moitié de dataset en ligne.

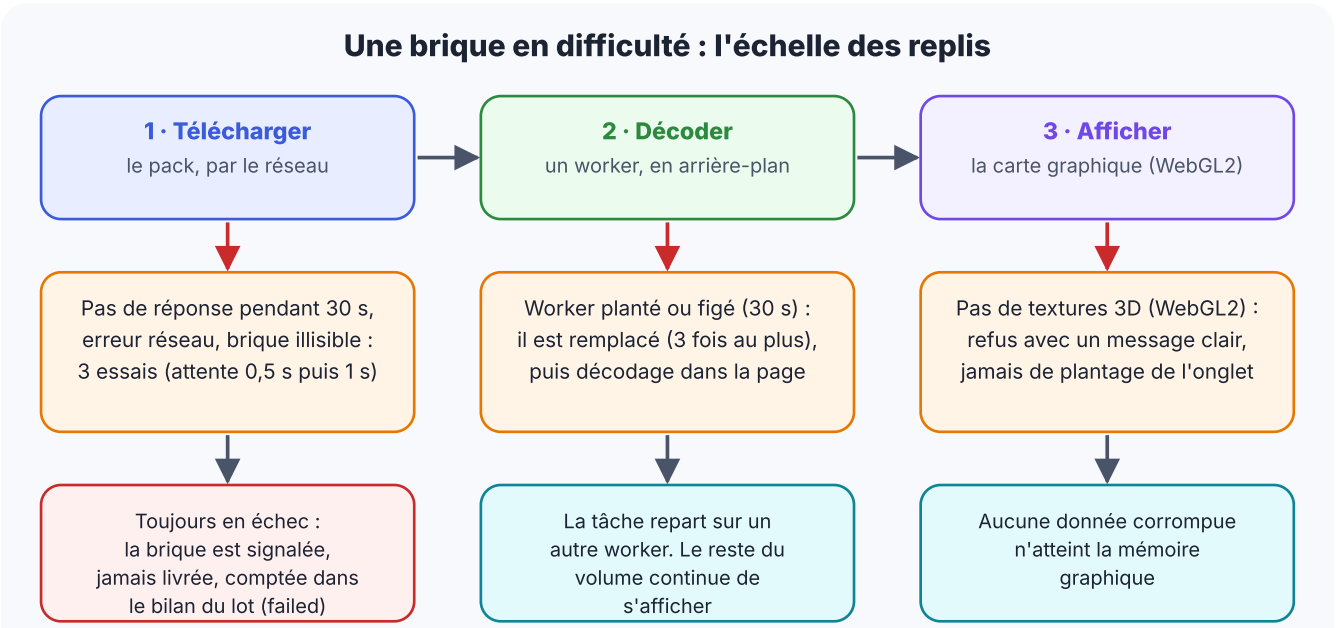
Reprendre, pas recommencer
Les opérations longues tiennent un **journal** : après une coupure, elles reprennent là où elles en étaient.

💡 UNE IMAGE POUR COMPRENDRE

Une ceinture, des bretelles et un gilet. Chaque risque a son filet de sécurité, et souvent deux : la brique corrompue est retentée *puis* écartée ; le serveur est testé avant *puis* après la bascule.

19.9 À l'écran : briques, workers et métadonnées

Une brique traverse trois étapes (réseau, décodage, carte graphique). Chacune a son repli.



▲ Les trois étapes d'une brique et le repli de chacune.

SI...	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
Une brique est corrompue (décodage impossible, taille inattendue)	Elle est **signalée** (`onBrickError`) et **jamais livrée** à la carte graphique ; 3 essais, avec 0,5 s puis 1 s d'attente	Le reste du volume s'affiche. Rien à faire
Un pack ne répond plus (rien ne passe pendant 30 s)	Le téléchargement est abandonné, puis retenté (3 essais)	Une barre de progression qui repart. Rien à faire
Une brique est absente du fichier d'index	Ce n'est pas une panne : une brique entièrement vide n'est pas stockée, c'est un volume de zéros (chapitre 7)	Rien : c'est du vide
Un worker de décodage plante ou se fige plus de 30 s	Il est remplacé (3 fois au plus) et sa tâche repart ; sans worker, le décodage se fait dans la page	Un léger ralentissement, jamais un onglet figé
Un autre jeu est ouvert pendant un chargement	Le lot de téléchargement en cours est annulé (raison `dataset-switch`), sans erreur	Rien : l'ancien jeu cesse simplement d'être chargé
Le jeu change de dossier de briques en cours de route	Les tâches planifiées sur l'ancien dossier sont refusées (`BRICKS_MOUNT_CHANGED`) plutôt que mélangées	Rien : le chargement est replanifié
La carte graphique n'offre pas les textures 3D (WebGL2 absent)	Refus **avec un message** ; l'onglet ne plante pas	Utilisez un navigateur ou un ordinateur récent

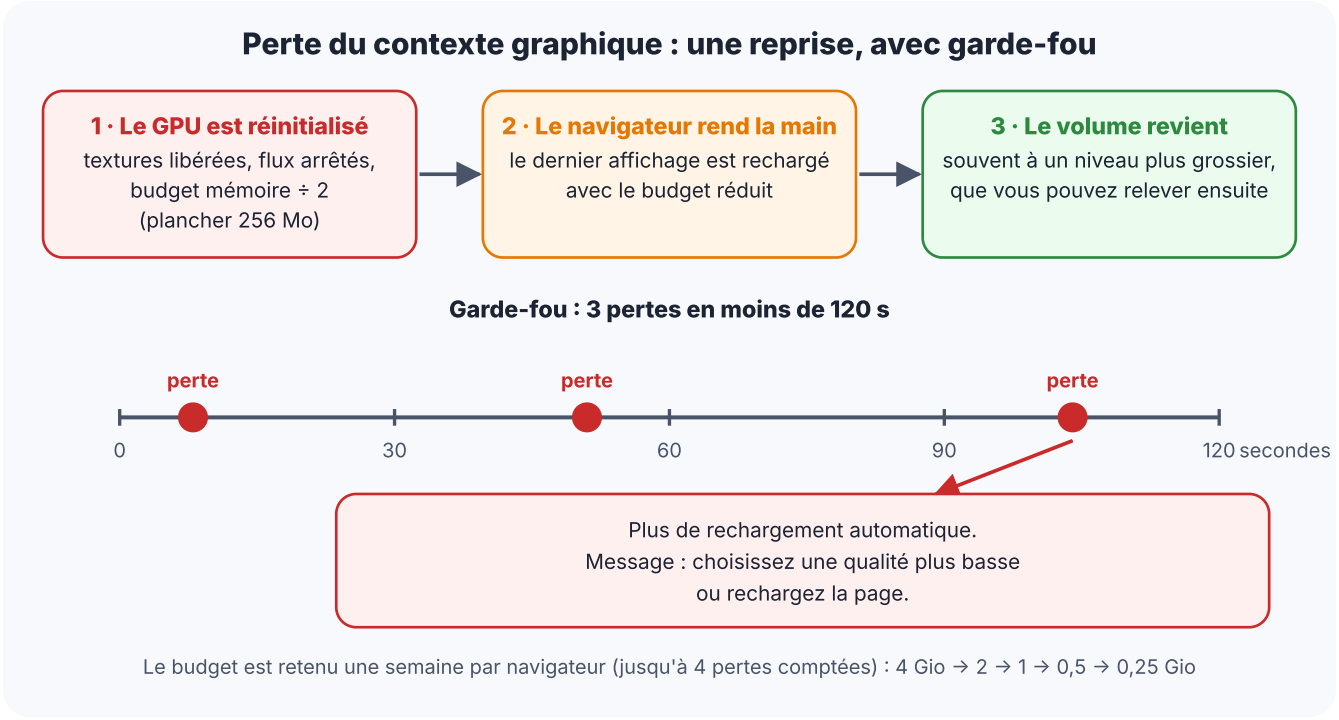
SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
Le fichier de métadonnées est mal formé	Il est **rejeté en entier**, jamais monté en partie : racine qui n'est pas un objet, dimensions x/y/z invalides, nombre de canaux incohérent, tailles de voxel invalides…	Un message donne la raison ; l'administrateur corrige le jeu
Un bloc facultatif est mal formé (stabilisation, suivi cellulaire, galerie…)	Ce bloc seul est **écarté avec un avertissement** ; le jeu se monte sans cette fonction	Le volume s'affiche, sans la couche concernée
Le jeu a plus de 4 canaux	Le pipeline les a tous écrits ; le viewer en dessine **quatre** (texture RGBA) et le dit au montage	Les quatre premiers canaux sont visibles
La calibration est absente	Pas de barre d'échelle 3D ; une mesure de distance 3D est **refusée** ; dans le Studio, les mesures se font en pixels. Aucune échelle n'est inventée (chapitres 9 et 12)	L'échelle n'apparaît pas
Vous cliquez à côté du spécimen pour mesurer	Le point tombe sur la boîte englobante, rien de visible dessous : il est **refusé** avec une notice	Cliquez sur la structure elle-même
Un panneau de Comparer n'arrive pas à s'ouvrir	Son erreur (`PANEL_ERROR`) est affichée sur lui seul, avec un bouton **Réessayer** ; l'enregistrement et le lien n'attendent pas ce panneau	Les autres panneaux continuent
Un plugin est non reconnu, modifié ou incompatible	Il n'est **pas chargé** (mis en quarantaine) ; les autres plugins ne sont pas touchés	Le bouton n'apparaît pas (chapitres 15 et 19.4)
Un lien `#state=` est trop gros ou altéré	Il n'est pas appliqué (au-delà de 2 Mio de texte, ou 16 Mio une fois décompressé, il est refusé) ; la page s'ouvre sur sa vue normale	Le lien ne restaure rien ; la page fonctionne
Le stockage du navigateur est bloqué (navigation privée)	Toute lecture ou écriture est protégée : la page fonctionne, simplement sans mémoire	Thème, langue, réglages ne sont pas retenus

🔧 **POUR LES CURIEUX**

Le plafond de **2 Mio** de texte et de **16 Mio** décompressés, comme celui de **64 Kio** écrits dans l'adresse, protègent contre une « bombe de décompression » : un petit lien qui s'étendrait en gigaoctets. Voir l'annexe B pour toutes les mémoires du navigateur.

19.10 La carte graphique : mémoire pleine et contexte perdu

La mémoire de la carte graphique est le seul vrai plafond dur. Lumen3D la **planifie avant** de s'en servir (chapitre 10), mais une carte peut quand même refuser, ou se réinitialiser en pleine image.



▲ Perte du contexte graphique : un seul rechargement automatique, puis un garde-fou.

SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
Le niveau demandé dépasse le budget de mémoire	Il n'est pas alloué : refus `SVR_OVER_BUDGET`, passage au niveau **plus grossier** suivant	Un message d'état dit que le niveau n'a pas pu être alloué et que le suivant est essayé
La carte refuse une allocation malgré le budget	Refus `SVR_ALLOC_FAILED` ; le budget est plafonné **sous la taille ayant échoué** pour toute la session, puis niveau plus grossier	Même message ; une qualité plus basse s'affiche
Aucun niveau ne tient	Le chargement est déclaré indisponible (mémoire insuffisante à tous les niveaux)	Fermez d'autres onglets, ou choisissez une qualité plus basse
Le contexte WebGL est perdu (pilote, veille, trop de mémoire)	Textures libérées, flux arrêtés ; le budget est **divisé par deux** (plancher 256 Mio), mémorisé une semaine	Un message d'état : le contexte graphique est perdu, en attente de son retour
Le contexte est rendu	Le dernier affichage est rechargé avec le budget réduit	Un message d'état : contexte restauré, rechargement avec un budget plus bas
3 pertes en moins de 120 s	Le rechargement automatique **s'arrête** (sinon la boucle bloquerait le GPU)	Choisissez une qualité plus basse ou rechargez la page

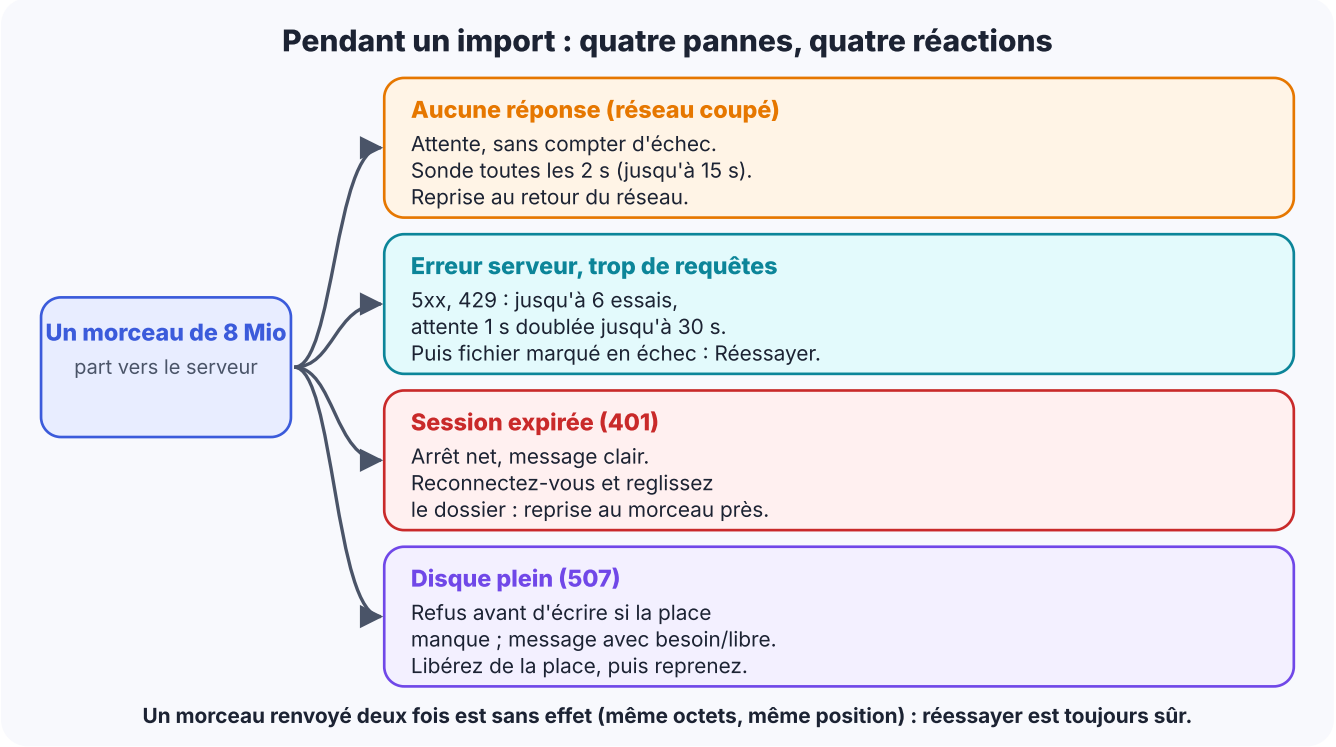
SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
Une image au repos est trop longue (plus de 200 ms)	Le plafond d'échantillons par rayon est réduit (chapitre 9) ; une perte de contexte le divise par deux	L'image est un peu moins fine, la vue reste fluide
Vous voulez forcer un budget	Réglage de l'opérateur : la clé `lumen3d.vramBudgetMB` du navigateur remplace l'estimation	Annexe B

❓ **POURQUOI CE CHOIX ?**

Pourquoi diviser le budget ? Une perte de contexte vient souvent d'un pilote qui coupe une image trop longue ou d'une mémoire saturée. Recharger avec la même demande referait la même erreur : on demande donc **moins** la fois suivante.

19.11 Imports, conversions et métadonnées

Un import de plusieurs gigaoctets dure des heures : la coupure est la règle, pas l'exception. Tout repose sur un **journal** (chapitre 17).



▲ Pendant un import : quatre pannes, quatre réactions.

SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
Le réseau est coupé pendant un import	Le transfert passe en attente (sans compter d'échec), sonde le réseau (2 s, puis jusqu'à 15 s) et reprend seul	Le message « Connexion perdue : le transfert reprendra tout seul dès que le réseau revient », puis la reprise

SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
Le serveur répond 5xx, 429 ou une somme de contrôle fausse	Jusqu'à **6 essais**, attente de 1 s doublée jusqu'à 30 s	Si tout échoue : fichier en échec, bouton **Réessayer**
La session expire pendant l'import	Arrêt net, avec le message « Session expirée — reconnectez-vous puis reglissez le dossier »	Reconnectez-vous et reglissez : reprise **au morceau près**
Le disque est plein	Avant d'écrire, le besoin est comparé à la place libre ; en cours de route, erreur 507 avec « besoin / libre »	Libérez de la place puis reprenez
Vous fermez l'onglet en plein import	Le journal du serveur garde les morceaux reçus (carte binaire des morceaux)	Reglissez le dossier : seuls les morceaux manquants repartent
Un morceau est envoyé deux fois, ou dans le désordre	Écrit à sa position exacte ; renvoyé, il est sans effet	Rien
Un fichier est interdit (`.php` **,** `.js` **, fichier caché, chemin avec** `..` **)**	Refusé **avant** toute écriture, par la liste blanche	Le fichier est listé comme refusé
Un dossier de type inconnu est déposé, ou le type annoncé ne correspond pas au dossier	Refusé (`metadata_type_mismatch` , `metadata_bad_type`) ; l'éditeur dit « type invalide »	Seuls `3d` , `2d` et `live` existent (chapitre 17)
Un pack est tronqué ou manquant	La publication est **refusée** : l'index des briques doit pointer dans des packs assez longs (`incomplete_files`)	Terminez le transfert, puis publiez
Un fichier inattendu traîne dans l'espace d'attente	Publication refusée (`stray_files`) plutôt que de l'emporter dans le site	Supprimez-le ou refaites l'import
`metadata.json` **est absente, illisible ou invalide**	Publication refusée avec la liste des erreurs (`missing_metadata` , `metadata_no_dimensions` , `metadata_bad_dimensions` …)	Corrigez dans l'éditeur du jeu
Un manifeste annonce une brique ≠ 64 ou des niveaux dans le désordre	Refusé à l'import, par les deux serveurs	Refaites passer le pipeline
Le dataset source change pendant une conversion	Le travail est invalidé (`source_changed`) plutôt que de mélanger deux versions	Relancez la conversion
Une conversion est interrompue (onglet fermé, serveur redémarré)	Le journal et le stock de tuiles sont communs au navigateur et au serveur : reprise au même point, même avec l'autre exécutant	Rouvrez l'onglet **Mises à jour des données**

SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
La finalisation d'une conversion est interrompue	Elle est reprenable (`complete:false` tant qu'elle n'est pas finie), puis remplace l'ancien contenu d'un seul geste	Relancez : elle se termine
Une unité tue deux fois la requête (hôte à temps limité)	Réponse `unit_timeout` : cette conversion bascule sur le navigateur ; les autres unités gardent leur progrès	Rien : le travail continue
L'hôte ne répond plus ou refuse (429, 503)	Le régulateur de requêtes divise son rythme et fait une pause (5 s, doublée jusqu'à 60 s)	Une conversion plus lente, jamais d'inondation
Le serveur ne sait pas décoder le WebP ou le navigateur ne sait pas l'encoder sans perte	Une sonde de capacité le détecte ; l'autre exécutant est proposé	Un message dans l'onglet
Le pipeline est interrompu en pleine publication	Le marqueur `.swap-in-progress` permet au lancement suivant de **terminer ou d'annuler** l'échange	Relancez le pipeline
Un jeu est republié par-dessus un jeu existant	Les réglages du laboratoire (masqué, orientation, galerie, stade corrigé…) sont **conservés**	Rien à refaire

🔢 **MICRO-EXEMPLE**

Une coupure de 20 minutes. Vous importez un embryon de 8 Go. À 60 %, le Wi-Fi tombe. Le transfert passe en « hors ligne », sonde toutes les 2 à 15 s, puis reprend seul au retour du réseau : **aucun fichier n'est perdu** et aucun essai n'est compté. Si vous aviez fermé l'ordinateur, il suffirait de reglisser le dossier.

19.12 Serveur : mises à jour, accès et limites d'hébergement

SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
La nouvelle version ne démarre pas (serveur Python)	Contrôle à blanc avant la bascule ; après la bascule, sonde `/api/health` (≈ 30 s) ; échec : **retour automatique** à l'ancienne version	Un état « annulée » dans l'onglet Mises à jour
Le serveur s'arrête pendant la bascule	Un journal reste ; au démarrage suivant, il est soit terminé (version cible atteinte), soit **entièrement défait**	Relancez le serveur

SI…	CE QUE FAIT LUMEN3D	CE QUE VOUS VOYEZ OU DEVEZ FAIRE
La mise à jour échoue sur un hébergement PHP	**Pas de retour automatique** : la copie est rejouable sans dégât, un fichier occupé est mis de côté (`.lumen-new`) puis terminé à la requête suivante	Relancez la mise à jour
Deux administrateurs lancent la mise à jour ensemble	La seconde demande est refusée (`update_in_progress`)	Attendez la fin de la première
Le téléchargement est tronqué	La taille annoncée est vérifiée, la somme SHA-256 aussi : erreur et abandon	Relancez
La place manque pour mettre à jour	Le contrôle préalable (même volume, espace disque, dossier de sauvegardes) refuse avant de toucher à rien	Libérez de la place
La version n'est pas signée, ou la signature est fausse	Elle est **refusée** ; seule l'archive nommée d'après la version et listée dans la liste signée est appliquée	Un refus (`unverifiable_release`) dont la raison est affichée dans l'onglet Mises à jour
Le catalogue de plugins est plus ancien que le dernier vu	Refusé (numéro de série décroissant : `catalog_rollback`)	Aucun plugin n'est installé depuis ce catalogue
Un plugin devient incompatible après une mise à jour	Il est mis de côté et réactivé quand une version compatible existe	Voir l'onglet Plugins
Vous tapez dix mauvais mots de passe	Blocage de 15 minutes pour cette adresse ; les échecs sont comptés avant la vérification	Attendez, ou utilisez le bon mot de passe
Votre session d'administration expire (8 h)	Les modifications non enregistrées du formulaire sont **conservées** à l'écran	Reconnectez-vous, puis enregistrez
Un compteur de visite est inondé de requêtes	Deux seaux de jetons (par adresse et global) ignorent l'excédent ; un GET ne compte pas	Rien
L'hébergement limite le temps ou la mémoire de PHP	Chaque étape de conversion règle son délai ; les grosses unités sont coupées en 8 octants ; les corps de requête sont plafonnés (32 Mio par unité)	Une conversion plus lente, jamais corrompue
Pas d'Internet	Tout fonctionne : les bibliothèques sont locales ; seules les polices (repli sur celles du système) et les contrôles GitHub de l'administrateur en ont besoin	Mises à jour et catalogue indisponibles

⚠️ **ATTENTION**

Les replis **protègent**, ils ne **réparent** pas tout. Un pack effacé du disque, une base de métadonnées modifiée à la main, ou un hébergement sans espace libre demandent une action humaine : la plateforme refuse proprement et vous dit pourquoi, c'est tout.

19.13 Ce qu'il faut retenir

📌 **À RETENIR**

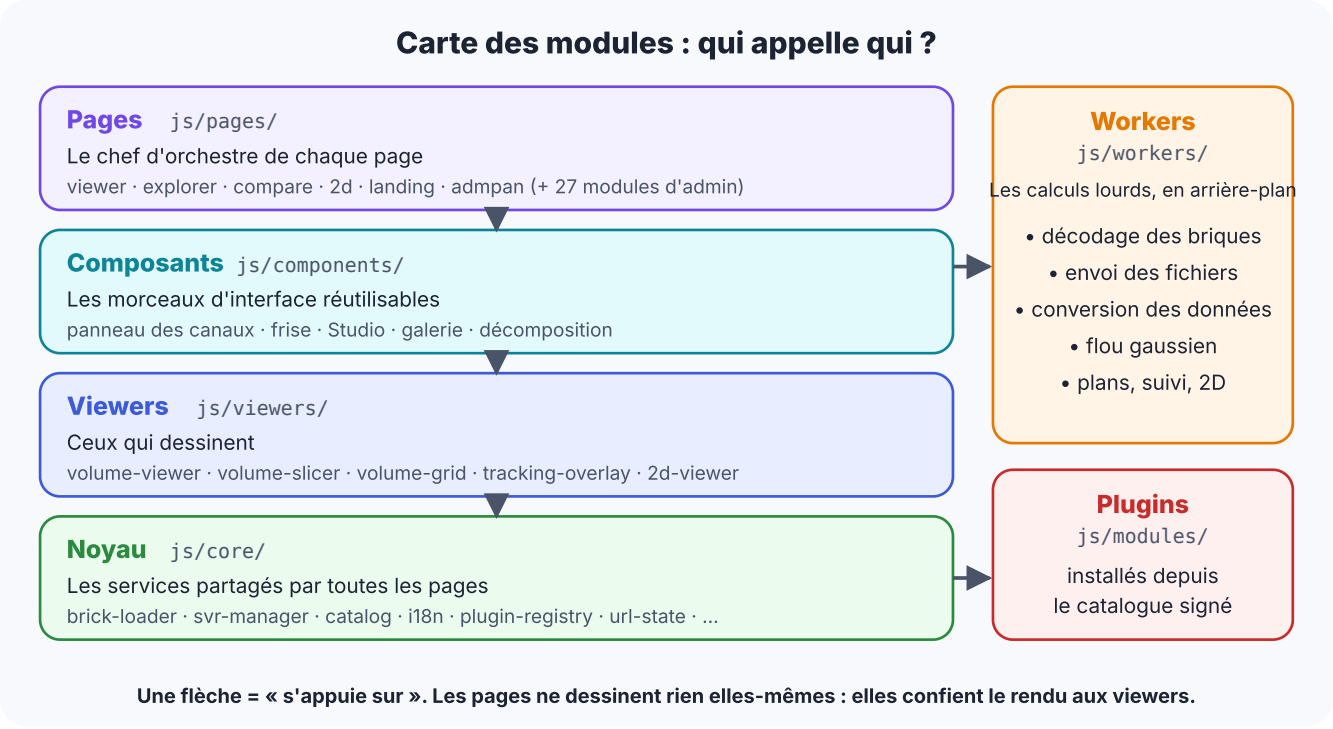
- **Image** : une brique douteuse est écartée, jamais affichée de travers ; la carte graphique est ménagée avant d'être sollicitée.
- **Données** : on publie en entier ou pas du tout ; les opérations longues se **reprennent**.
- **Serveur** : Python revient seul à l'ancienne version ; PHP se relance sans risque, mais à la main.
- **Aucune coupure** (réseau, onglet, session) ne fait perdre un import commencé.

20. Annexes

- Cinq annexes de **référence** : à consulter, pas à lire d'une traite.
- **A** : où est quoi dans le code. **B** : ce que votre navigateur retient et les paramètres d'adresse. **C** : ce que Lumen3D ne fait pas.
- **D** : tous les chiffres importants au même endroit, avec le chapitre qui les explique. **E** : l'histoire des versions.

20.A Carte des modules

Lumen3D est écrit en JavaScript « simple » : pas de framework, pas de compilation. Chaque fichier a **un rôle** et porte un nom qui le dit. Cette annexe en est l'index, un fichier par ligne.



▲ Cinq familles de fichiers. Les pages orchestrent, les viewers dessinent, le noyau rend service, les workers calculent.

💡 UNE IMAGE POUR COMPRENDRE

Un restaurant. Les *pages* sont les serveurs qui prennent votre commande. Les *viewers* sont les cuisiniers qui dessinent le plat. Le *noyau* est la réserve, les couteaux et le plan de travail. Les *workers* sont la plonge, en coulisses : on ne la voit pas, mais rien ne sortirait sans elle.

Le noyau : `js/core/` (33 fichiers)

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`brick-loader.js`	va chercher les briques dans les packs, par lots, sans télécharger deux fois la même chose	10
`brick-decode-worker.js`	worker : décode les images WebP et redécoupe les mosaïques en briques	10
`svr-manager.js`	gère l'atlas 3D en mémoire graphique et son budget de mémoire	10
`cut-store.js`	lit les copies de coupe du format 5 (`xz/` , `yz/` , `cuts/`) pour le Studio	7, 12
`data-access.js`	`ctx.data` : les voxels exacts, sans perte, pour les outils qui mesurent	15
`catalog.js`	charge la liste des jeux de données, la filtre, la cherche, trouve les jeux liés	3
`utils.js`	boîte à outils commune : vocabulaire des types (3d, 2d, live), dates, barres d'échelle 1-2-5	1
`tool-manager.js`	sait quel outil est actif (naviguer, couper, mesurer) et gère les raccourcis clavier	12
`volume-source-manager.js`	normalise la description des « sources » d'un jeu (briques, série temporelle)	10
`url-state.js`	écrit et relit l'état de la vue dans l'adresse (`#state=`)	12
`workspace-state.js`	enregistre et restaure l'espace de travail dans le navigateur	12
`measurement-store.js`	garde en mémoire les distances mesurées pendant la session	12
`export-manager.js`	moteur du Centre de téléchargement : espace de travail, mesures, métadonnées, citation	12
`display-presets.js`	fonds de l'affichage 3D : sombre, clair, papier, transparent, personnalisé	9
`slice-compositor.js`	recolore une coupe à partir de ses valeurs brutes, pour le Studio	12

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`plane-loader.js` / `plane-codec.js`	lisent les plans XY du format 2 et définissent leur format PNG	7, 17
`plugin-registry.js`	découvre, charge et branche les plugins ; construit la barre d'outils	15
`plugin-trust.js`	vérifie l'empreinte d'un plugin et son niveau de confiance	15
`plugin-sandbox.js`	la cage (iframe isolée) des plugins tiers approuvés	15
`compat.js`	dit si un plugin est compatible avec la version de la plateforme	15
`i18n.js`	traductions : anglais, français, espagnol, néerlandais et langues ajoutées	16
`instance-config.js`	lit la configuration du site (nom, spécimen, textes) : la « marque blanche »	16
`theme.js` / `theme-boot.js`	bascule clair/sombre ; `theme-boot` l'applique avant le premier affichage	16
`colorblind.js`	huit simulations de daltonisme, appliquées à toute la page	16
`page-renderer.js`	dessine une page construite dans l'éditeur (sections, colonnes, widgets)	16
`page-templates.js`	contenu par défaut de la page À propos	16
`page-edit-frame.js`	mode édition en direct de l'éditeur de pages	16
`page-vars.js`	variables dans les textes de page (année, date, nombre de jeux…)	16
`page-background.js`	fonds animés des pages construites	16
`dialog.js`	petites fenêtres de choix (Confirmer / Annuler) ; Échap = annuler	16
`ui-actions.js`	gère les clics des boutons d'en-tête (la politique de sécurité interdit les gestionnaires dans le HTML)	19
`font-loader.js`	active les polices Google sans retarder le premier affichage	19

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`perf-telemetry.js`	chronomètres de diagnostic, gardés sur place, jamais envoyés	10

Les viewers : `js/viewers/` (5 fichiers)

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`volume-viewer.js`	le cœur : la scène 3D, le lancer de rayons, la caméra, les captures	9
`volume-slicer.js`	calcule une coupe plane (droite ou oblique) dans le volume	12
`volume-grid.js`	grille de référence, repère d'axes et barre d'échelle 3D	12
`tracking-overlay.js`	dessine les cellules suivies d'un timelapse par-dessus le volume	12
`2d-viewer.js`	dessine une photographie 2D (zoom, déplacement, échelle)	12

Les composants : `js/components/` (5 fichiers)

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`channel-panel.js`	le panneau des canaux : couleur, fenêtre, gamma, opacité	11
`decomposition-panel.js`	l'interface « Décomposer par canal »	12
`studio-editor.js`	le Studio : annoter et exporter une figure	12
`timeline.js`	la frise temporelle et le lecteur de séries	12
`dataset-gallery.js`	la galerie d'images rattachées à un jeu, avec son agrandissement	12

Les workers : `js/workers/` (9 fichiers)

Un worker est un employé d'arrière-plan : il travaille sans bloquer l'écran (chapitre 2).

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`gaussian-blur-worker.js`	flou gaussien d'un canal, plan par plan	11
`plane-decode-worker.js`	décode les tuiles PNG des plans XY	7

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`studio-plane-worker.js` / `studio-plane-ops.js`	maximum par canal sur une épaisseur ; les calculs purs sont dans `ops`	12
`pixel-2d-worker.js` / `pixel-ops-2d.js`	réglages d'affichage et isolement du marquage d'une photographie ; calculs purs dans `ops`	12
`tracks-load-worker.js`	prépare les tables de suivi cellulaire (entiers 32 bits)	12
`upload-worker.js`	envoie les fichiers d'un import : lecture, empreinte, envoi, reprises	17
`migration-worker.js`	exécute les unités d'une conversion de format côté navigateur	17

Les pages : `js/pages/` (10 fichiers)

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`viewer.js`	la page principale : assemble tout pour un volume 3D ou une série	3
`2d.js`	la page des photographies 2D	12
`explorer.js`	la grille de jeux de données, ses filtres et la recherche	3
`compare.js`	la page Comparer : jusqu'à quatre panneaux synchronisés	12
`compare-policy.js`	choisit la qualité de chaque panneau pour tenir dans un budget commun	12
`landing.js`	la page d'accueil	3
`about.js` / `legal.js` / `page-view.js`	À propos, Mentions légales, pages personnalisées	3, 16
`admpan.js`	la page d'administration (démarre le module `shell`)	14

L'administration : `js/pages/admin/` (27 fichiers)

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`shell.js` / `shared.js` / `bus.js`	le cadre (menu, thème, connexion), les outils communs (requêtes, messages) et la messagerie entre onglets	14

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`tab-datasets.js`	l'onglet **Datasets** : liste, éditeur, orientation, galerie	14
`tab-upload.js`, `upload-manager.js`, `upload-dock.js`	l'onglet **Import**, son orchestrateur et la pastille flottante	17
`tab-dataset-updates.js`, `migration-runner.js`	**Mises à jour des données** et son moteur (dont le régulateur de requêtes)	17
`tab-dataset-types.js`	**Types de données** : renommer 3d, 2d, live	16
`tab-branding.js`, `tab-appearance.js`, `tab-legal.js`	**Identité**, **Apparence**, **Mentions légales**	16
`tab-pages.js` + `pages-controls.js`, `pages-translate.js`, `pages-variables.js`	l'éditeur de pages et ses panneaux	16
`tab-plugins.js`, `tab-marketplace.js`, `plugin-update.js`	**Plugins**, **Catalogue** et la règle commune de mise à jour des plugins	15
`tab-updates.js`, `tab-changelog.js`, `markdown.js`	**Mises à jour**, la page des notes de version, le lecteur de Markdown	18
`tab-pipeline.js`, `tab-docs.js`, `tab-security.js`, `tab-stats.js`	**Pipeline**, **Documentation**, **Sécurité**, **Statistiques**	14

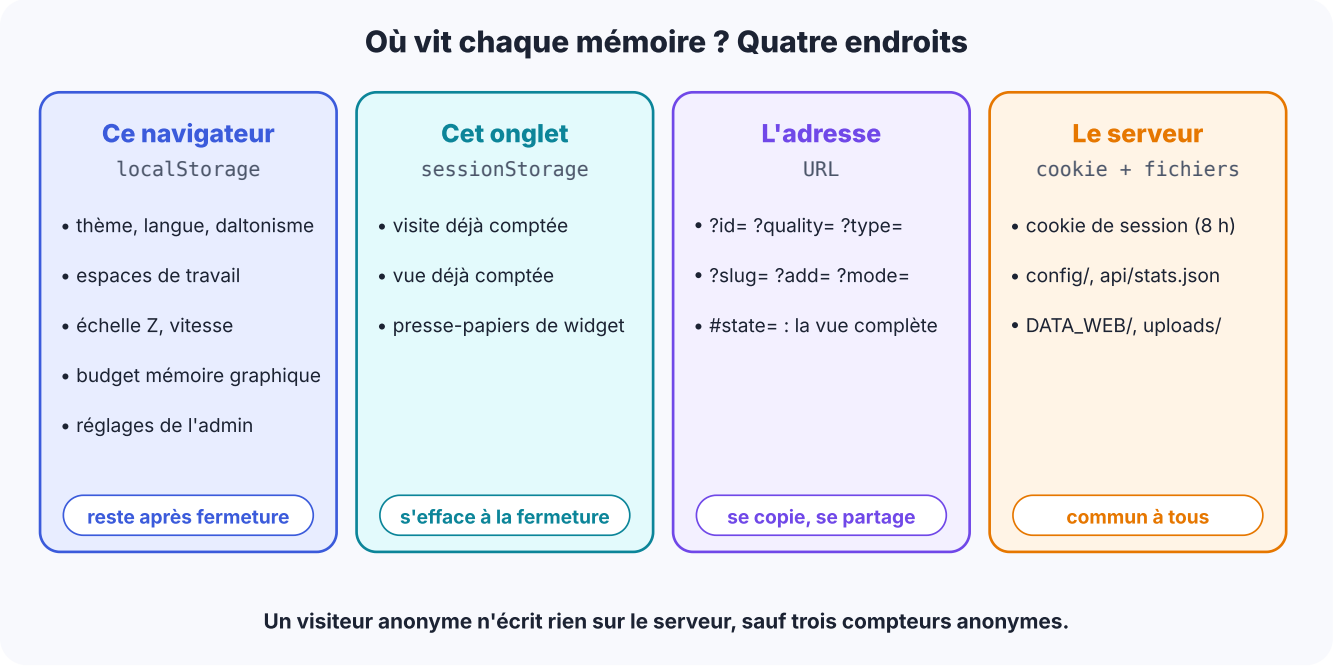
Hors de `js/`

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`dev_server.py`	le serveur Python : sert les pages et fait aussi le travail de l'administration	2, 18
`api/*.php` et `router.php`	le serveur jumeau en PHP, pour les hébergements sans Python	2, 18
`fast_server.py`	un serveur statique rapide, sans administration (tests de performance)	2
`upload_staging.py`	le moteur d'import (liste blanche, journal, validation, publication)	17
`dataset_migrations.py`	les conversions de format de données, côté Python	17

FICHIER	CE QU'IL FAIT, EN CLAIR	CHAP.
`format5.py`	le moteur partagé du format 5 (niveaux, compressions, mosaïques, paquets, copies de coupe), utilisé par la migration et par le pipeline	7, 17
`tools/convert_dataset.py`	convertir un jeu au dernier format sur son propre ordinateur	17
`ed25519_pure.py`	vérificateur de signatures Ed25519, en Python pur	19
`js/migrations/`	les trois conversions (`m002` , `m003` , `m004`) côté navigateur, et un bloc de test de vitesse	17
`js/modules/`	les plugins : `tools/` (barre d'outils), `channels/` (panneau des canaux), `shaders/` (modes de rendu)	12, 15
`js/vendor/`	les bibliothèques tierces, hébergées sur place (Three.js, Lucide, Plotly)	2
`preprocess/`	le pipeline de préparation, en Python	4 à 8
`tests/` , `tools/`	la suite de tests (201 fichiers) et les outils de publication	18

20.B Mémoires du navigateur et paramètres d'adresse

Un navigateur offre plusieurs « tiroirs » où un site peut ranger de petites choses. Lumen3D en utilise **quatre** et n'y met jamais de donnée d'étude.



▲ Quatre endroits où vit une information.

> ℹ️ **À SAVOIR**
>
> Tout accès à ces tiroirs est protégé : en navigation privée, ou si le stockage est bloqué, la page fonctionne, simplement sans mémoire (chapitre 19). Vider les données du site efface tout ce qui suit sans rien casser.

B.1 Ce que ce navigateur retient (`localStorage`)

CLÉ	CE QU'ELLE GARDE
`iribhm-theme`	le thème choisi, clair ou sombre
`iribhm-lang`	la langue de l'interface
`iribhm-colorblind`	le filtre de simulation de daltonisme
`iribhm.workspace.<portée>.<id>`	l'espace de travail enregistré d'un jeu (caméra, canaux, outils), en JSON ; `<id>` est du type `3d/Nom-du-jeu`
`iribhm.viewer.zScale.<id>`	l'étirement d'affichage en Z, jeu par jeu
`iribhm.viewer.playbackFps`	la vitesse de lecture préférée des séries temporelles (une seule pour le viewer)
`iribhm-gallery-dock`	l'état de la galerie : ouverte, large ou réduite en pastille

CLÉ	CE QU'ELLE GARDE
`lumen3d.gpuContextLosses`	les dates des dernières pertes de contexte graphique (8 au plus) : chaque perte de la semaine divise le budget par deux
`lumen3d.vramBudgetMB`	**réglage d'opérateur** : impose un budget de mémoire graphique, en Mo. Jamais écrit par le site (chapitre 10)
`adm-theme` , `adm-sidebar-collapsed`	thème et menu replié du panneau d'administration
`adm-upload-dock-size`	taille de la pastille d'import
`lumen-dupd-ds-executor` , `lumen-dupd-speedtest` , `lumen-dupd-log`	conversions de données : exécutant choisi par jeu, dernier test de vitesse, journal des 20 dernières opérations

> 👉 **ASTUCE**
>
> Pour qu'un budget graphique trop prudent (après plusieurs pertes de contexte) revienne à la normale, supprimez `lumen3d.gpuContextLosses` dans les outils du navigateur, ou attendez une semaine.

B.2 Ce que cet onglet retient (`sessionStorage`) et le cookie

NOM	RÔLE
`lumen_visit`	« la visite a déjà été comptée » : un compteur de visites par session, pas par page
`lumen_view_<id>`	« la vue de ce jeu a déjà été comptée » (les aperçus de l'administration ne comptent pas)
`lumenWidgetClip`	presse-papiers d'un widget copié dans l'éditeur de pages
cookie `admpan_token`	la session d'administration : 8 heures, invisible aux scripts (`HttpOnly`). Les visiteurs n'en reçoivent pas

B.3 Les paramètres d'adresse

Ils se placent après `?` (par exemple `viewer.html?id=3d/Embryo-E95-Em2-Pecam1-Sox2&quality=high`).

PAGE	PARAMÈTRE	SENS
`viewer.html` , `2d.html`	`id=<type>/<dossier>`	le jeu à ouvrir
`viewer.html`	`quality=`	`preview` (256), `balanced` (512), `high` (1024), `2048x2048` , `4096x4096` ou `native` ; les formes `512x512` , `1024x1024` , `low` , `medium` sont acceptées

PAGE	PARAMÈTRE	SENS
`viewer.html`, `2d.html`	`hideHeader=true`	page « sans bandeau » : c'est ainsi qu'un panneau de Comparer l'intègre
idem	`panelIndex=<n>`	numéro du panneau dans Comparer
idem	`mode=admin`	aperçu de l'administration : la vue n'est **pas** comptée dans les statistiques
idem	`path=<type>/<dossier>` ou `staging:<type>/<dossier>`	avec `mode=admin` : ouvrir un jeu pas encore publié
`explorer.html`	`type=3d`, `2d` ou `live`	pré-coche le filtre de type
`compare.html`	`add=<id>` (répétable)	jeux à mettre dans les panneaux (quatre au plus)
`page.html`	`slug=<nom>`	la page personnalisée à afficher
`page.html`, `about.html`, `index.html`	`preview=draft`	affiche le brouillon plutôt que la version publiée
`page.html`	`edit=1`	mode édition (utilisé dans l'éditeur de pages)
`admpan.html`	`editor=<slug>`	ouvre l'éditeur de pages seul sur cette page
`admpan.html`	`changelog=1`	la page des notes de version, sans le panneau
toutes les pages de vue	`#state=…`	**la vue complète**, compressée ; écrite toute seule quand vous changez quelque chose, au plus une fois par seconde

🔧 **POUR LES CURIEUX**

`#state=` est un JSON compressé (deflate) puis écrit en base64. Il peut contenir caméra, canaux, outils, mesures et état des plugins. Plafonds : 64 Kio écrits dans l'adresse, 2 Mio de texte lus, 16 Mio une fois décompressé (protection contre les « bombes »). Un lien trop gros ou altéré est ignoré. Les paramètres d'une page embarquée dans Comparer sont posés par Comparer lui-même.

20.C Ce que Lumen3D ne fait pas

Un bon outil dit ce qu'il ne sait pas faire. Voici les limites **assumées**, vérifiées dans le code.

▲ Huit choses que Lumen3D ne fait pas.

LIMITE	POURQUOI, ET QUE FAIRE
Pas de quantification calibrée de l'intensité	Le pipeline ramène 12 ou 16 bits sur 0–255 entre un fond et un signal estimés ; les gris affichés sont **relatifs** (chapitres 5 et 11). Pour comparer des intensités entre échantillons, revenez au fichier Imaris d'origine
Pas d'objets plus fins qu'environ 3 voxels	Le nettoyage retire les points isolés puis lisse le fond par une médiane : un point, un fil ou une feuille d'un ou deux voxels d'épaisseur est effacé, même brillant (chapitre 5, § 5.10). Vérifiez dans le fichier Imaris d'origine
4 canaux affichés au maximum	Les volumes sont chargés dans une texture à quatre composantes (rouge, vert, bleu, alpha). Le pipeline écrit tous les canaux ; le viewer dessine les quatre premiers (chapitre 11)
Pas de déconvolution, pas de correction de champ plat	Le pipeline soustrait un fond et règle une fenêtre ; il ne corrige ni le flou de l'objectif (fonction d'étalement) ni l'éclairage inégal
Pas de segmentation, pas de détection de cellules	Le suivi cellulaire est **lu** dans l'analyse faite sous Imaris, jamais recalculé (chapitre 8)
Pas d'édition de vos voxels	Le viewer lit les données et ne les modifie pas ; flou, fenêtre ou gamma sont des réglages d'**affichage**. Le seul changement de valeur est la réduction 16 → 8 bits du pipeline (chapitre 6)

LIMITE	POURQUOI, ET QUE FAIRE
Pas de compensation du photoblanchiment	Le pipeline en **mesure** le niveau par image, mais aucun réglage du viewer ne l'utilise : une série qui s'éteint s'affiche plus sombre (chapitre 5)
Mesures non enregistrées sur le serveur	Elles vivent en mémoire pendant la session et dans le lien ou l'espace de travail que vous enregistrez (chapitre 12)
Pas de statistiques de région	Distances, coupes, figures : oui. Volume d'une structure ou intensité moyenne d'une zone : non
Un seul compte d'administration	Le public n'a pas de compte ; l'administrateur est unique, protégé par un mot de passe (chapitre 19)
WebGL2 obligatoire	Sans textures 3D, les briques ne peuvent pas être affichées : un message l'explique (chapitre 19). Sur un mobile, la mémoire graphique limite la qualité
Entrées : fichiers Imaris `.ims` et photographies TIFF	D'autres formats de microscope doivent d'abord être convertis ; les photographies attendent le TIFF exporté par ImageJ/Fiji depuis un fichier Leica (chapitre 4)
Pas de pyramide d'images 2D	Une photographie est un seul fichier WebP natif accompagné d'un aperçu ; l'ancien visualiseur à tuiles a été supprimé

⚠️ **ATTENTION**

« Ne le fait pas » ne veut pas dire « le fait mal ». Si votre question scientifique demande l'une de ces fonctions, faites-la dans l'outil d'analyse adapté, puis utilisez Lumen3D pour **montrer** le résultat.

20.D Chiffres clés de la plateforme

Toutes les constantes importantes, regroupées. La dernière colonne dit où l'idée est expliquée.

D.1 Les données

CONSTANTE	VALEUR	CHAP.
Brique	64^3 voxels utiles ; 66^3 stockés avec 1 voxel de bordure (formats 4 et 5)	7
Mosaïque d'une brique	9 × 8 coupes de 66^2 : une image WebP sans perte de 594 × 528 px	7
Pack	au plus 64 briques ou 16 Mio, par super-blocs de 4 × 4 × 4 briques	7
Niveaux de détail	divisés par deux en X et Y ; en Z seulement tant que vz ≤ 1,5 × vxy	6, 7
Format de données courant	4 (les formats 1 à 3 sont convertis en place)	7, 17
Plan XY (format 2)	un fichier par plan ; tuiles PNG de 512 × 512	7, 17
Projection par couche (format 3)	une projection d'intensité maximale par couche de 64 plans	7
Taille maximale acceptée	2^{20} voxels par axe ; 64 canaux au plus acceptés, **4 affichés**	19
Réduction de profondeur	16 ou 12 bits → 8 bits (une seule perte du pipeline)	5, 6

D.2 Le nettoyage du fond (pipeline)

CONSTANTE	VALEUR	CHAP.
Plancher de fond `bg_floor`	99e percentile des 8 cubes de coin	5
Saturation `sig_max`	99,9e percentile (d'un voxel sur 4)	5
Seuil du masque	plus de 1,1 × `bg_floor`	5
Nettoyage du masque	ouverture, puis dilatation × 3	5
Série temporelle	`sig_max` calculé sur les seuls voxels au-dessus du fond, s'ils sont au moins 1 000	5

D.3 Le streaming et la mémoire graphique

CONSTANTE	VALEUR	CHAP.
Chargements simultanés de briques	24	10

CONSTANTE	VALEUR	CHAP.
Corps de pack simultanés	4	10
Mémoire de packs compressés gardés	192 Mio (packs) · 64 Mio (plages) · 32 Mio (préchargement)	10
Anticipation	64 Mio d'avance sur la brique décodée	10
Délais d'attente	30 s sans réponse (téléchargement) · 30 s (décodage)	10, 19
Essais d'une brique	3, avec 0,5 s puis 1 s d'attente	19
Remplacements d'un worker	3 au plus, puis décodage dans la page	19
Qualité 512	niveau le plus fin dont le plus grand côté XY est ≤ 768 voxels	10
Qualité 1024	idem ≤ 1 536 voxels	10
Atlas	jusqu'à 8 pages	10
Budget graphique : rendu logiciel	256 Mio	10
Budget graphique : carte intégrée	0,5 · 1 · 2 Gio selon la mémoire de l'appareil	10
Budget graphique : carte dédiée	3 ou 4 Gio	10
Budget graphique : inconnue	1 ou 2 Gio	10
Perte de contexte graphique	budget ÷ 2 par perte de la semaine (plancher 256 Mio)	19
Garde-fou	3 pertes en moins de 120 s : plus de rechargement automatique	19
Comparer	4 panneaux au plus ; budget commun 1,5 Gio ; 180 s pour qu'un panneau se stabilise	12

D.4 Le rendu

CONSTANTE	VALEUR	CHAP.
Échantillons par rayon (au repos)	`clamp(6·10⁹ / pixels, 256, 4096)`	9
Image visée	16,7 ms (60 images/s) ; on dégrade au-delà de 1,3×, on remonte sous 1,1×	9
Image au repos trop lente	au-delà de 200 ms, le plafond d'échantillons est réduit	9

CONSTANTE	VALEUR	CHAP.
Garde-fou Windows (TDR)	un calcul graphique de plus de 2 s environ réinitialise la carte	9

D.5 Sécurité et accès

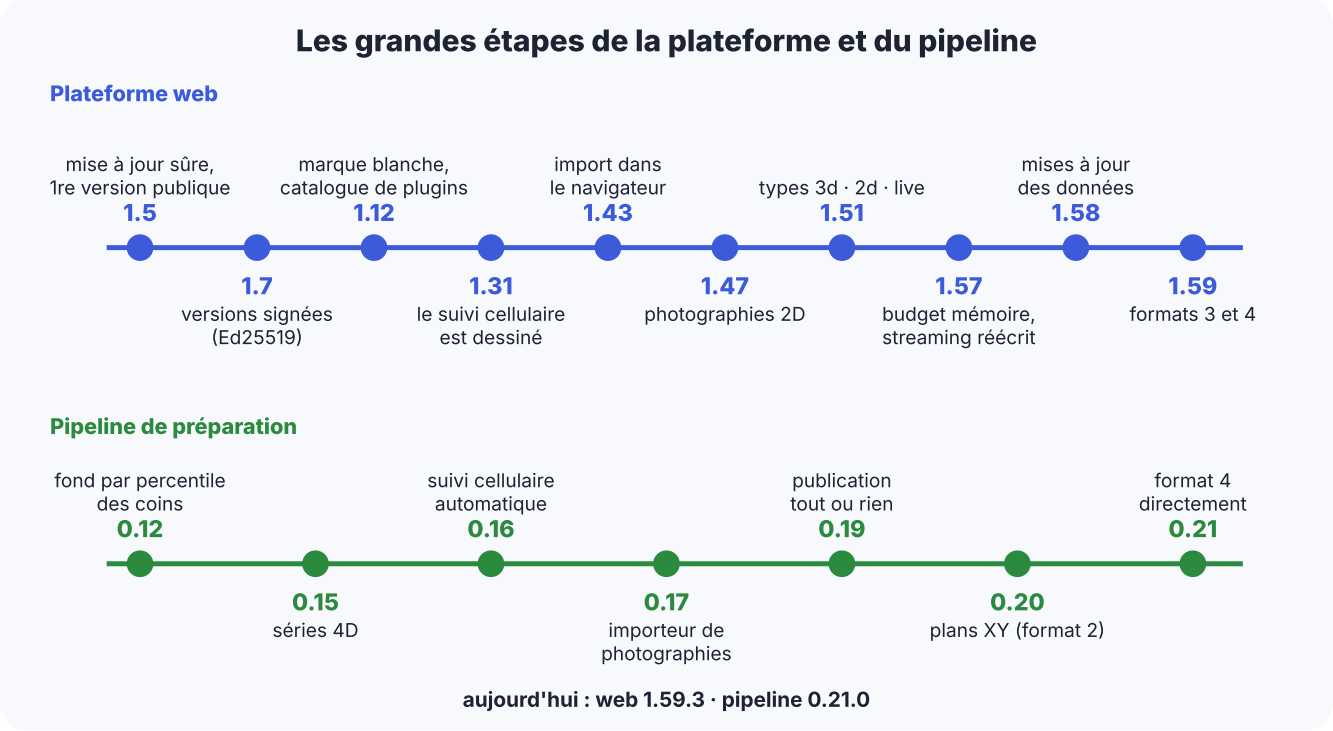
CONSTANTE	VALEUR	CHAP.
Mot de passe	8 caractères au moins ; PBKDF2-HMAC-SHA256, 600 000 tours	19
Échecs de connexion	10 en 15 minutes depuis une adresse → blocage de 15 minutes ; plafond global de 200 par fenêtre	19
Session d'administration	8 heures ; fermée à chaque changement de mot de passe	19
Lien `#state=`	64 Kio écrits · 2 Mio lus · 16 Mio décompressés	12, 20.B
Galerie d'un jeu	40 images, 8 Mio chacune	14
Compteurs de visites	par adresse : 60 en rafale puis 1 par seconde ; global : 600 puis 20 par seconde ; 4 096 emplacements	19

D.6 Import, conversions et mises à jour

CONSTANTE	VALEUR	CHAP.
Morceau d'import	8 Mio (entre 256 Kio et 16 Mio) ; 4 envois en parallèle	17
Essais d'un morceau	6, attente 1 s doublée jusqu'à 30 s ; sonde réseau 2 s à 15 s	19
Délai d'une requête d'import	au moins 60 s (jusqu'à 50 Ko/s au plancher)	17
Régulateur de requêtes (conversions)	6 en vol, 0,5 à 6 par seconde (départ à 4), pause de 5 s à 60 s	17
Briques lues par requête	128	17
Corps d'une unité de conversion	32 Mio au plus (hôtes PHP)	17
Unité de conversion	une couche de 64 plans × un canal × une tuile de 512² (8 × 8 briques)	17
Sonde de santé après mise à jour (Python)	environ 30 s	18
Plugins au catalogue signé	28	15
Fichiers de tests	201 (un processus par fichier)	18

20.E Versions et historique

La plateforme web et le pipeline de préparation sont deux logiciels, avec **deux numéros** indépendants. Chaque version a une note, `changelog/changelog_X.Y.Z.md` pour la plateforme et `preprocess/changelog/` pour le pipeline.



▲ Les grandes étapes de la plateforme et du pipeline.

> ℹ️ **À SAVOIR**
>
> La version de la plateforme n'existe que dans le **nom du dernier fichier de notes** : pour passer à une nouvelle version, on ajoute une note. La version du serveur Python (`dev_server.py`) est un troisième numéro, distinct et plus ancien : elle suit l'outil serveur, pas la plateforme.

E.1 Plateforme web

VERSION	CE QU'ELLE APPORTE
1.3 – 1.4	Centre de téléchargement en explorateur de fichiers ; refonte complète du panneau d'administration
1.5	**Première version publique sur GitHub** : mise à jour sûre par échange de dossiers avec retour arrière, sonde de santé, installeur en un fichier, compatibilité plugin ↔ plateforme
1.6	Isolation des plugins tiers : confiance, bac à sable, politique de sécurité stricte, bibliothèques hébergées sur place
1.7	Versions signées (Ed25519), politique de sécurité aussi sur les hébergements PHP, durcissement du bac à sable

VERSION	CE QU'ELLE APPORTE
1.8 – 1.13	Marque blanche : configuration du site, assistant de premier lancement, catalogue de plugins signé (1.12), éditeur de pages sections → colonnes → widgets (1.13)
1.14	Mise à jour des hébergements PHP (1.14.2)
1.25	La page À propos devient une page modifiable
1.31	Le suivi cellulaire est enfin dessiné dans le viewer
1.43 – 1.44	Import d'un jeu depuis le navigateur, par paliers et reprenable ; pack Pipeline téléchargeable comme une version
1.45 – 1.49	Galerie d'images par jeu ; viewer de photographies 2D (1.47)
1.51	Un seul vocabulaire de types : `3d` , `2d` , `live`
1.53 – 1.54	Le suivi cellulaire devient une couche d'une série (1.53) ; Comparer pilote les vraies pages (1.54)
1.55 – 1.56	Poses de caméra et explorateur Z-stack ; export de la vue 3D, rotation des annotations, recoloration des canaux dans le Studio
1.57	« Vague d'audit » : streaming réécrit en lots, budget de mémoire graphique, version signée et suite de tests obligatoire à chaque publication
1.58	Mises à jour des données : conversion en place d'un jeu vers le format 2
1.59	Formats 3 et 4 (briques v3 avec bordure, `index.bin` , détail au zoom) ; exécutant choisi par jeu et test de vitesse (1.59.1) ; régulateur de requêtes (1.59.2) ; huit fois moins de requêtes (1.59.3)
1.60 – 1.62	Compression graphique des timelapses (1.60) ; corbeille de 60 jours pour les jeux supprimés (1.62) ; import / export des pages (1.61)
1.63 – 1.65	**Format 5** : la matrice résolution × compression, les copies de coupe, l'accès sans perte pour les outils (`ctx.data`), un choix « sans perte » à chaque export (1.64) ; conversion dans le navigateur sur tout hébergement, import d'un arbre v5, outil de conversion locale (1.65)

E.2 Pipeline de préparation

VERSION	CE QU'ELLE APPORTE
0.12 – 0.13	Seuil d'Otsu abandonné (0.12.0) ; fond estimé par percentile des coins et débruitage par masque (0.12.13 à 0.13.0)
0.14	Lanceur autonome en un seul fichier
0.15 – 0.16	Vraies séries 4D ; suivi cellulaire rattaché tout seul
0.17 – 0.18	Importeur de photographies ; nouveau vocabulaire de types
0.19	Publication « tout ou rien » en zone d'attente, avec reprise après interruption
0.20	Plans XY écrits directement (format 2)
0.21	Format 4 écrit directement, identique octet pour octet à la conversion ; saut exact de l'espace vide
0.22	Format 5 sur demande (`--format 5`), identique octet pour octet à la conversion m005

📌 **À RETENIR**

Version actuelle de ce document : plateforme **1.65.0**, pipeline **0.22.0**, format de données **5**.

21. Glossaire

- Les termes sont classés **par ordre alphabétique**. Chaque définition tient en une ou deux phrases.
- « → chapitre N » vous envoie là où le mot est expliqué en détail.
- Les noms entre crochets, comme **Studio**, sont ceux que vous voyez à l'écran.

Atlas

Grande texture 3D en mémoire graphique où l'on range les briques à afficher, comme les cases d'un casier. → chapitre 10

Atomique

Se dit d'une opération qui réussit en entier ou pas du tout, sans état intermédiaire visible. Une publication de dataset est atomique. → chapitres 8 et 17

Bac à sable (sandbox)

Cage dans laquelle un plugin tiers s'exécute : il ne voit ni la page, ni vos cookies, et ne peut demander que ce qu'on lui a explicitement permis. → chapitre 15

Bleu-vert (déploiement)

Méthode de mise à jour : la nouvelle version est préparée à côté de l'ancienne, testée, puis substituée ; l'ancienne reste disponible pour un retour arrière. → chapitre 18

Boîte englobante (bounding box)

Parallélépipède qui entoure tout le volume. Un clic qui n'atteint que cette boîte, sans structure visible dessous, n'est pas une mesure valable. → chapitre 12

Bordure (apron)

Voxel supplémentaire autour d'une brique, copié depuis ses voisines, pour que le filtrage ne laisse pas de couture entre deux briques. Les briques des formats 4 et 5 font 66^3 pour 64^3 utiles. → chapitre 7

Brique

Petit cube de 64×64×64 voxels : l'unité de découpage et de chargement d'un volume. → chapitre 7

Bruit de fond

Signal parasite présent même là où il n'y a rien à voir (lumière diffuse, électronique du capteur). → chapitre 5

Bruit de Poisson

Variation aléatoire d'un signal faible, due au nombre limité de photons reçus : plus le signal est faible, plus il paraît granuleux. S'y ajoute le bruit de la caméra. → chapitre 5

Cache

Mémoire où l'on garde ce qui a déjà été téléchargé pour ne pas le redemander. → chapitre 10

Calibration (physique)

Taille réelle d'un voxel, en micromètres, dans chaque direction. Sans elle, la barre d'échelle et les mesures n'existent pas. → chapitres 4 et 9

Canal

Une image du même objet pour une couleur de fluorescence donnée (par exemple DAPI, Pecam1, Sox2). → chapitre 4

Catalogue

Deux sens à distinguer : la liste des jeux de données du site, générée à la demande à partir des fichiers, et le catalogue de plugins signé, d'où l'on installe les outils. → chapitres 3 et 15

Compression avec perte, sans perte

Avec perte, le fichier est plus petit mais les valeurs sont légèrement modifiées ; sans perte, on retrouve exactement les valeurs d'origine. Les briques sont stockées sans perte ; le format 5 ajoute, pour l'affichage seulement, des colonnes ±2, ±4 et avec perte. → chapitres 6 et 7

Colonne (de compression)

En format 5, l'une des cinq façons de stocker un même niveau : sans perte, ±2, ±4, légère (WebP 80), très légère (WebP 50). Choisie dans la matrice de qualité. → chapitres 7 et 10

Copies de coupe

En format 5, trois copies du volume recoupées (plans XY, lignes XZ / YZ, petites briques obliques) qui rendent les coupes natives du Studio rapides. → chapitre 7

Confocal (microscope)

Microscope qui n'enregistre que la lumière venue d'un plan très mince du spécimen, plan après plan ; l'empilement des plans forme le volume. → chapitre 4

Contexte (page ou panneau)

Situation dans laquelle une page tourne : seule, ou intégrée comme panneau (Comparer, vue divisée). Un plugin ne se charge dans un panneau que s'il l'a déclaré. → chapitre 15

Contexte WebGL (perte de)

Réinitialisation de la carte graphique par le navigateur ou le pilote : les textures disparaissent. Le viewer recharge la vue avec un budget mémoire réduit. → chapitre 19

Coupe optique, plan focal

Image d'une tranche mince du spécimen, nette à une profondeur donnée. L'épaisseur de coupe est la distance entre deux plans (3,0 µm dans le jeu de démonstration). → chapitre 4

CSP (politique de sécurité du contenu)

Règle donnée au navigateur : n'exécuter que les scripts portant un code à usage unique (nonce) et provenant du site lui-même. → chapitre 19

CSRF (jeton anti-falsification)

Attaque où un site tiers fait envoyer une requête à votre insu, depuis votre session. Un jeton secret, exigé à chaque modification, l'empêche. → chapitre 19

DAPI, Pecam1, Sox2

Les trois canaux du jeu de démonstration : DAPI colore l'ADN (les noyaux), Pecam1 marque les vaisseaux, Sox2 le tissu neural. → chapitre 4

Déconvolution

Calcul qui tente de défaire le flou de l'optique pour affiner l'image. Lumen3D ne le fait pas. → chapitre 20

Dilatation

Opération qui épaissit les zones d'un masque d'un ou plusieurs pixels. → chapitre 5

Échantillon (rendu)

Valeur lue en un point du volume pendant le lancer de rayons. Un rayon en prend des centaines. → chapitre 9

Ed25519 (signature)

Méthode de signature numérique : un « sceau de cire » que seul l'éditeur peut apposer et que tout le monde peut vérifier. → chapitre 19

Empreinte (hachage, SHA-256)

Court code calculé à partir du contenu d'un fichier : si un seul octet change, l'empreinte change. Elle sert à vérifier un téléchargement, un plugin, un morceau d'import. → chapitres 15 et 19

ESS (Empty Space Skipping, saut de l'espace vide)

> Les briques dont tous les voxels valent 0 ne sont ni stockées ni parcourues, ce qui économise octets et calcul. Aux formats 4 et 5 la règle est exacte (un seul voxel non nul suffit à garder la brique) ; avant, un seuil d'occupation de 0,05 % écartait aussi les briques presque vides. → chapitres 7 et 9

Exécutant

> Celui qui fait le travail d'une conversion de données : le navigateur ou le serveur. Il peut changer en cours de route, le journal est commun. → chapitre 17

Exposition

> Réglage de luminosité globale appliqué à l'affichage ; il n'altère pas les données. → chapitre 11

Fenêtrage

> Choix des valeurs min et max affichées : en dessous, noir ; au-dessus, pleine intensité. → chapitre 11

Filtre médian

> Remplace chaque valeur par la médiane de ses voisines : il supprime les points isolés sans brouiller les contours. → chapitre 5

Finalisation

> Dernière étape d'une conversion de données : les nouveaux fichiers remplacent les anciens d'un seul geste, puis le numéro de format est mis à jour. → chapitre 17

Fluorophore

> Molécule qui absorbe de la lumière et en réémet d'une autre couleur ; c'est lui qui marque une structure dans l'échantillon. Un canal correspond à un fluorophore. → chapitre 4

Fonction de transfert

> Règle qui convertit une valeur du volume en couleur et en opacité. Dans Lumen3D, c'est la chaîne fenêtre, gamma, opacité et couleur du panneau des canaux ; le chapitre 11 la décrit sans employer ce terme. → chapitre 11

Format de données

> Façon dont un dataset est rangé sur le disque ; numéroté de 1 à 5. Le format courant est le 5. → chapitres 7 et 17

Gamma

> Réglage qui éclaircit ou assombrit les tons moyens sans toucher aux extrêmes. → chapitre 11

GPU

> Processeur de la carte graphique, spécialisé dans le calcul parallèle ; il exécute le lancer de rayons. → chapitre 9

HDF5

> Format de fichier scientifique qui range de gros tableaux de nombres, comme un classeur à tiroirs. Les fichiers Imaris en sont un. → chapitre 4

Histogramme

> Graphique qui compte combien de voxels ont chaque valeur d'intensité ; il aide à régler min et max. → chapitre 11

i18n (internationalisation)

> Mécanisme qui permet d'afficher l'interface en plusieurs langues (anglais, français, espagnol, néerlandais…) et d'en ajouter. → chapitre 16

Idempotent

> Se dit d'une opération qu'on peut refaire sans dégât : le résultat est le même qu'une seule fois. C'est ce qui rend les reprises sûres. → chapitre 17

.ims

> Fichier Imaris : un conteneur HDF5 qui contient les canaux, les niveaux de résolution et les métadonnées d'une acquisition. → chapitre 4

index.bin

> Fichier binaire des formats 4 et 5 (un par colonne en format 5) qui dit dans quel pack se trouve chaque brique ; il remplace le gros manifeste JSON. → chapitre 7

Intensité

Valeur d'un voxel : combien de lumière a été mesurée à cet endroit. Dans le viewer elle est relative (0 à 255), pas calibrée. → chapitres 4 et 11

Interpolation trilinéaire

Estimation d'une valeur entre voxels en mélangeant les huit voisins, ce qui donne une image lisse. → chapitre 9

Jeton

Petit code qui prouve un droit : jeton anti-falsification (CSRF) d'une modification, ou « seau à jetons » qui limite le nombre de requêtes admises par seconde. → chapitre 19

Jour embryonnaire (E8.5)

Âge d'un embryon de souris : E8.5 signifie huit jours et demi après la fécondation. La plateforme lit le stade dans le nom du fichier (E8-5 vaut E8.5). → chapitre 8

Journal

Fichier tenu à jour pendant une opération longue (import, conversion) : il note ce qui est fait, ce qui permet de reprendre après une coupure. → chapitre 17

Kabsch (algorithme)

Méthode qui trouve la rotation et le décalage alignant au mieux deux nuages de points ; utilisée pour stabiliser un timelapse. → chapitre 8

Lancer de rayons (ray marching)

Pour chaque pixel de l'écran, on suit un rayon à travers le volume en cumulant couleur et opacité. → chapitre 9

Lignée, mitose, fusion

Dans un suivi cellulaire : la lignée est la suite d'une cellule et de ses descendantes ; la mitose est une division en deux cellules ; la fusion réunit deux cellules en une. → chapitre 12

LOD (niveau de détail) / niveau

Version du volume à une résolution donnée ; le niveau 0 est le plus fin. → chapitre 6

LOD0

Le niveau de détail le plus fin : la résolution native de l'acquisition. → chapitre 6

Lot (de briques)

Ensemble de briques demandées ensemble au chargeur, qui gère son annulation et rend un bilan (livrées, échouées, écartées). → chapitre 10

Manifeste

Fichier qui décrit le contenu d'un dataset (niveaux, briques, packs). → chapitre 7

Marque blanche

Capacité de la plateforme à prendre le nom, les couleurs, les textes et les types de données de l'institution qui l'héberge, sans écrire de code. → chapitre 16

Masque

Image en noir et blanc qui dit quels pixels comptent (blanc) et lesquels sont écartés (noir). → chapitre 5

Migration

Mise à jour d'un dataset déjà publié vers un format plus récent, sans repasser par le pipeline. → chapitre 17

MIP (projection d'intensité maximale)

Image où chaque pixel prend la valeur la plus forte rencontrée le long d'un axe. → chapitres 7 et 12

Matrice (de qualité)

Le tableau « résolution × compression » du panneau de qualité d'un jeu au format 5 : une ligne par niveau, une colonne par compression ; chaque case affiche son poids et sa mémoire graphique. → chapitre 10

Mosaïque

Plusieurs petites images collées en une grande ; une brique est stockée comme une mosaïque de ses coupes. → chapitre 7

Nonce

Code aléatoire à usage unique, tiré à chaque chargement de page, qui autorise les scripts du site. → chapitre 19

Opacité

> Facteur appliqué à l'intensité d'un canal : 0 le rend invisible, 1 le laisse à pleine intensité. Ce n'est pas l'absorption physique du tissu, qui n'existe (constante) que dans le mode Fluorescence naturelle. → chapitre 11

Ouverture morphologique

> Érosion suivie d'une dilatation : elle efface les petits points isolés d'un masque et garde les grandes formes. → chapitre 5

Pack

> Fichier `.bin` qui regroupe de nombreuses briques pour limiter le nombre de téléchargements. → chapitre 7

Palier (tier)

> Groupe de priorité d'un import : les fichiers qui rendent un jeu ouvrable (métadonnées, niveau le plus grossier) partent d'abord. → chapitre 17

PBKDF2

> Méthode qui transforme un mot de passe en empreinte volontairement lente à calculer (600 000 tours), pour décourager les essais en masse. → chapitre 19

Percentile

> Valeur sous laquelle se trouve un pourcentage donné des mesures : le 99e percentile est dépassé par 1 % des valeurs. → chapitre 5

Photoblanchiment

> Perte progressive de fluorescence d'un échantillon éclairé longtemps. Dans une série temporelle, le signal baisse alors sans que la structure change. → chapitre 5

Pile Z (Z-stack)

> L'ensemble des plans successifs d'un volume, empilés selon l'axe Z. → chapitre 12

Pivot

> Moment de la mise à jour où un superviseur remplace l'ancien dossier du site par le nouveau, relance le serveur et vérifie qu'il répond ; sinon il revient en arrière. → chapitre 18

Pixel

> Plus petit élément d'une image 2D. → chapitre 4

Plans

> Dossier `planes/` du format 2 : un fichier par plan z du niveau natif, pour lire une coupe XY sans charger 64 plans. → chapitre 7 (et 17)

Plugin

> Outil optionnel qui s'ajoute au viewer (mesure, capture, filtre…) ; il est installé depuis le catalogue signé. → chapitres 12 et 15

PNG

> Format d'image sans perte ; les plans du format 2 sont des tuiles PNG de 512×512. → chapitre 7 (et 17)

Profondeur de bits

> Nombre de bits par valeur : 8 bits donnent 256 niveaux, 16 bits en donnent 65 536. → chapitre 4

PSF (fonction d'étalement du point)

> Forme floue que l'optique donne d'un point lumineux. La déconvolution cherche à la défaire ; Lumen3D ne la corrige pas. → chapitre 20

Pyramide

> Ensemble de versions du volume à des résolutions de plus en plus grossières, empilées comme les étages d'une pyramide. → chapitre 6

Quaternion

> Quatre nombres qui décrivent une rotation 3D sans blocage ; ils servent à mémoriser l'orientation d'un embryon. → chapitre 12

Recalage (registration)

> Alignement des images successives d'un timelapse pour compenser le déplacement de l'embryon (algorithme de Kabsch). → chapitre 8

Retour arrière (rollback)

> Retour automatique à l'ancienne version quand la nouvelle ne démarre pas. Le serveur Python le fait ; l'hébergement PHP, non. → chapitres 18 et 19

Saturation

> Valeur qui dépasse le maximum représentable et est écrêtée. Le pipeline sature volontairement les 0,1 % de voxels les plus brillants (`sig_max`). → chapitre 5

Shader

> Petit programme exécuté par la carte graphique ; celui du viewer fait le lancer de rayons. → chapitre 9

Signature

> Voir Ed25519. → chapitre 19

Sonde de santé

> Petite requête (`/api/health`) qui vérifie que le serveur répond avec la bonne version après une mise à jour. → chapitre 18

Spécimen

> Le nom que l'institution donne à ce qu'elle observe (embryon, organe…) ; il change dans toutes les interfaces. → chapitre 16

SRI (intégrité des sous-ressources)

> Empreinte attendue d'un script ou d'une feuille de style, inscrite dans la page : si le fichier chargé diffère, le navigateur le refuse. → chapitre 19

Staging

> Zone privée où arrivent les fichiers importés avant publication ; elle n'est jamais accessible par URL. → chapitres 17 et 19

Stéréomicroscope

> Microscope à faible grossissement qui donne une image en couleur de l'embryon entier ; il produit les photographies du type 2D. → chapitre 12

Streaming

> Chargement progressif : on affiche ce qui est arrivé et on complète en continu, sans attendre tout le fichier. → chapitre 10

Studio

> Outil de figures : recadrage, annotations, barres d'échelle et export d'images pour publication. → chapitre 12

Suivi cellulaire

> Suivi de chaque cellule d'image en image dans un timelapse (trajectoires, lignées, divisions). → chapitre 8

Super-bloc

> Groupe de 4×4×4 briques voisines qu'un pack range en bloc pour qu'une région se lise d'un seul tenant. → chapitre 7

Table de pages

> Tableau qui dit où chaque brique se trouve dans l'atlas, comme un plan de casiers. → chapitre 10

Three.js

> Bibliothèque JavaScript qui simplifie l'affichage 3D dans le navigateur. → chapitre 2

Timelapse

> Série d'acquisitions du même spécimen au fil du temps, chaque image étant un volume 3D. → chapitre 4

Unité (de conversion)

> Morceau de travail d'une conversion de données : une couche de 64 plans, d'un canal, pour une tuile de 512×512. → chapitre 17

Voxel

> Pixel en 3D : un petit cube de valeur d'intensité. Sa taille réelle (en µm) est la calibration. → chapitre 4

VRAM

> Mémoire de la carte graphique. Son budget limite la taille d'atlas que le viewer peut utiliser. → chapitre 10

WebGL2

> Technologie du navigateur qui donne accès à la carte graphique, notamment aux textures 3D. → chapitre 9

WebP sans perte

> Format d'image compressé qui restitue exactement les valeurs d'origine ; il stocke les mosaïques de briques. → chapitre 6

Widget

> Bloc de contenu d'une page construite dans l'éditeur (texte, image, bouton…), rangé dans une colonne d'une section. → chapitre 16

Worker (Web Worker)

> Processus d'arrière-plan du navigateur : il décode et calcule sans geler l'écran. → chapitre 10

X-gal

> Coloration qui rend bleues les cellules exprimant un gène rapporteur ; le viewer 2D sait isoler cette coloration. → chapitre 12

Zone morte

> En format 5, seuil (de 0 à 12) sous lequel une brique compressée avec perte remet les petites valeurs à 0, pour que le fond reste noir ; écrit dans un octet à la fin de la brique. → chapitre 7